CODE: TERRAFORM

PLAYER DOCUMENTATION

Build 58191a1

Contents

GUIDE

Start Here	Getting Started	15
Start Here	First Steps	16
Start Here	Command Results	17
Start Here	Long-Running Scripts	19
Start Here	Frequently Asked Questions	22
Tutorials	How This Game Works	27
Tutorials	Choosing Your First Credit Path	28
Tutorials	Terraform Index and planetary completion	29
Tutorials	Power & Terraforming Machines	31
Tutorials	First Biology Loop	33
Tutorials	First Harvesting Route	35
Tutorials	Contracts	36
Tutorials	Reading Errors and Console Output	37
Tutorials	First Pioneer	38
Tutorials	Sonar: scanning and surveying	39
Tutorials	Mining: from discovery to delivery	42
Tutorials	Using Plan Mode	44
Tutorials	First Outpost	45
Tutorials	Storage and remote outpost logistics	46
Tutorials	Industrial upgrades and deep oil	48
Tutorials	Nuclear power: uranium to electricity	51
Tutorials	Expanding Production	53
Programming	Variables	54
Programming	Operators	55
Programming	Numbers	57
Programming	Print	59
Programming	Self & Components	60
Programming	If / Elif / Else	62
Programming	Loops & Scripts	65
Programming	Match / Case	66
Programming	Conversion Functions	68
Programming	Strings & F-strings	69

Programming	**Regular Expressions**	**71**
Programming	**Lists & Tuples**	**73**
Programming	**Dictionaries**	**75**
Programming	**Built-in Functions Overview**	**77**
Programming	**Exceptions**	**81**
Programming	**Utility Helpers**	**84**
Programming	**Imports & Libraries**	**87**
Programming	**Docstrings**	**90**
Programming	**Writing Classes**	**92**
Editor & Tools	**Code Editor**	**98**
Editor & Tools	**Vim Mode**	**101**
Editor & Tools	**Script Commands**	**102**
Editor & Tools	**Script Variants**	**104**
Editor & Tools	**Debug Mode**	**106**
Editor & Tools	**Control Room**	**109**
Editor & Tools	**External Editor**	**113**
Editor & Tools	**Keyboard Shortcuts**	**120**
Automation Systems	**Contracts**	**121**
Automation Systems	**Vehicle Proximity & Service Areas**	**122**
Automation Systems	**Earth Orders**	**124**
Automation Systems	**Signal Bus**	**126**
Automation Systems	**Automations**	**135**
Automation Systems	**Data Archive**	**137**
Automation Systems	**Map Markers**	**140**
Automation Systems	**Drones**	**142**
Production & Logistics	**Batteries & Charging**	**144**
Production & Logistics	**Refinement & Storage**	**148**
Production & Logistics	**Input & Output**	**149**
Production & Logistics	**Battery Holder**	**151**
Production & Logistics	**Cargo Rack**	**152**
Production & Logistics	**Portable Battery**	**153**
Production & Logistics	**Portable Storage Bin**	**154**
World & Infrastructure	**Biosphere, Biomass Tier**	**155**
World & Infrastructure	**Biosphere, Plants**	**157**
World & Infrastructure	**Plant Terraformer**	**160**
World & Infrastructure	**Wildlife, Overview**	**162**

World & Infrastructure **Wildlife, Supply (Gases & Liquids)** 163
World & Infrastructure **Wildlife, Husbandry** 165
World & Infrastructure **Habitat Development** 168
World & Infrastructure **Wildlife, Progression** 169
World & Infrastructure **Wildlife, Creature Profiles** 171
World & Infrastructure **Weather System** 172
World & Infrastructure **Flow Networks, Fluids** 174
World & Infrastructure **Infrastructure & Pipes** 180
World & Infrastructure **Power Networks** 183
World & Infrastructure **Thermal Vents** 185
World & Infrastructure **Tier 3 Progression** 186
World & Infrastructure **Water & Oil Wells** 189
Reference **Components** 190

COMMANDS

System 193
Infrastructure 198

BUILT-IN FUNCTIONS

Built-in Functions 199
Argument Types 273

BUILT-IN MODULES

random 275
functools 280
re 284
dataclasses 292
enum 297
json 307
heapq 310
traceback 313
math 314
itertools 332
operator 342
string 357
collections 360

LANGUAGE

Basics	**Variables**	**363**
Basics	**Print**	**364**
Basics	**Type Annotations**	**365**
Basics	**Ellipsis**	**366**
Basics	**Tracebacks**	**367**
Basics	**Regular Expressions**	**368**
Basics	**Raw Strings**	**369**
Basics	**Bitwise & Set Operators**	**370**
Control Flow	**If / Else**	**371**
Control Flow	**While Loop**	**372**
Control Flow	**For Loop**	**373**
Control Flow	**Match / Case**	**374**
Data Structures	**Lists & Tuples**	**375**
Data Structures	**Dictionaries**	**376**
Data Structures	**TypedDict (record shapes)**	**377**
Data Structures	**JSON**	**378**
Data Structures	**Priority Queues**	**379**
Functions	**Functions**	**380**
Functions	**Generators (yield)**	**381**
Functions	**Caching Results**	**382**
Classes	**Classes**	**383**
Classes	**@dataclass**	**384**
Classes	**Enumerations (enum)**	**385**
Classes	**Inheritance & super()**	**386**
Classes	**Custom exceptions**	**387**
Classes	**Operators & dunder methods**	**388**
Classes	**Decorators**	**389**
Classes	**@property**	**390**
Classes	**Generic types**	**391**

COMPONENTS

Core Systems	**Clock**	**392**
Core Systems	**Commander**	**396**
Core Systems	**Console**	**398**

Core Systems	**GPS**	**405**
Core Systems	**Nocturna**	**409**
Core Systems	**Research**	**415**
Core Systems	**Shop**	**417**
Core Systems	**Transmitter**	**422**
Sensors	**Biomass Sensor**	**427**
Sensors	**Oxygen Sensor**	**429**
Sensors	**Plants Sensor**	**433**
Sensors	**Pressure Sensor**	**435**
Sensors	**Thermometer**	**439**
Sensors	**Wildlife Sensor**	**442**
Exploration	**Journal**	**444**
Exploration	**Map Markers**	**453**
Exploration	**Scanner**	**458**
Power	**Battery**	**462**
Power	**Battery Charger**	**464**
Power	**Lightning Rod**	**471**
Power	**Oil Generator**	**477**
Power	**Reactor**	**482**
Power	**Solar Generator**	**488**
Power	**Steam Turbine**	**492**
Power	**Vehicle Charging Station**	**497**
Terraforming	**Atmosphere**	**508**
Terraforming	**Bio Caster**	**511**
Terraforming	**Bio Collector**	**528**
Terraforming	**Bio Conditioner**	**534**
Terraforming	**Bio Exchange**	**544**
Terraforming	**Bio Lab**	**553**
Terraforming	**Bio Luminizer**	**563**
Terraforming	**DNA Sequencer**	**571**
Terraforming	**Feed Maker**	**579**
Terraforming	**Habitat**	**589**
Terraforming	**Heat Generator**	**620**
Terraforming	**Oxygen Generator**	**628**
Terraforming	**Pressure Generator**	**637**
Terraforming	**Refiner**	**645**

Terraforming	**Terraforming**	**656**
Production & Storage	**Fabricator**	**659**
Production & Storage	**Fuel Assembler**	**674**
Production & Storage	**Harvester**	**682**
Production & Storage	**Inventory**	**704**
Production & Storage	**Large Warehouse**	**712**
Production & Storage	**Lead Cask**	**720**
Production & Storage	**Mining Drill**	**725**
Production & Storage	**Smelter**	**729**
Production & Storage	**Storage Bin**	**737**
Production & Storage	**Warehouse**	**747**
Production & Storage	**Waste Processor**	**755**
Vehicles & Modules	**Constructor Module**	**764**
Vehicles & Modules	**Drill Module**	**767**
Vehicles & Modules	**Drone**	**770**
Vehicles & Modules	**Fleet**	**788**
Vehicles & Modules	**Nav Module**	**790**
Vehicles & Modules	**Pioneer**	**795**
Vehicles & Modules	**Rover**	**809**
Vehicles & Modules	**Sonar Module**	**821**
Logistics & Orders	**Data Archive**	**826**
Logistics & Orders	**Drone Depot**	**832**
Logistics & Orders	**Drone Service Station**	**838**
Logistics & Orders	**Earth Orders**	**851**
Logistics & Orders	**Signal Bus**	**855**
Logistics & Orders	**Supply Dock**	**868**
Infrastructure & Fluids	**Construction Blueprint**	**877**
Infrastructure & Fluids	**Exotic Gas Cap**	**888**
Infrastructure & Fluids	**Exotic Spring Tap**	**895**
Infrastructure & Fluids	**Gas Tank**	**902**
Infrastructure & Fluids	**Large Liquid Tank**	**907**
Infrastructure & Fluids	**Liquid Tank**	**913**
Infrastructure & Fluids	**Oil Pump**	**919**
Infrastructure & Fluids	**Outpost**	**926**
Infrastructure & Fluids	**Outpost Network**	**931**
Infrastructure & Fluids	**Power Control**	**933**

Infrastructure & Fluids	**Run Control**	**938**
Infrastructure & Fluids	**Ship Computer**	**947**
Infrastructure & Fluids	**Steam Condenser**	**953**
Infrastructure & Fluids	**Thermal Cap**	**959**
Infrastructure & Fluids	**Water Pump**	**969**
Biosphere	**Biomass Mixer**	**976**
Biosphere	**Crop Automator**	**984**
Biosphere	**Dispenser**	**997**
Biosphere	**Essence Liquifier**	**1003**
Biosphere	**Grow Lamp**	**1010**
Biosphere	**Plant Terraformer**	**1016**
Biosphere	**Seed Maker**	**1026**
Biosphere	**Sprinkler**	**1034**
Storage & Inventory	**Item Catalog**	**1041**
Weather & Sky	**Weather Station**	**1043**

DATABASE

Production	**Production Tiers**	**1049**
Recipes	**Fabricator Recipes**	**1050**
Recipes	**Smelter Recipes**	**1076**
Recipes	**Feed Maker Recipes**	**1079**
Recipes	**Refiner Recipes**	**1085**
Recipes	**Fuel Assembler Recipes**	**1087**
Equipment	**Power Equipment**	**1088**
Equipment	**Atmosphere Equipment**	**1091**
Equipment	**Biology Equipment**	**1095**
Equipment	**Biosphere Equipment**	**1097**
Equipment	**Mining Equipment**	**1102**
Equipment	**Logistics Equipment**	**1108**
Equipment	**Production Equipment**	**1109**
Equipment	**Modules & Vehicle Gear**	**1112**
Equipment	**Fluid Infrastructure**	**1120**
Items	**Minerals**	**1123**
Items	**Refined Materials**	**1126**
Items	**Fabricated Components**	**1129**
Items	**Agriculture & Feeds**	**1142**

Items	**Life Forms**	**1150**
Items	**Field Resources**	**1158**
Items	**Biology Samples**	**1160**
Items	**Lab Reagents**	**1187**
Fluids	**Fluids**	**1189**
Research	**Research**	**1196**

TYPES

Core Data	**Bounds**	**1228**
Core Data	**Component**	**1230**
Core Data	**PointOfInterest**	**1231**
Core Data	**Position**	**1233**
Core Data	**ScriptCommand**	**1235**
Core Data	**ScriptStatusReport**	**1237**
World & Sites	**Outpost**	**1239**
World & Sites	**OutpostNetwork**	**1242**
World & Sites	**OutpostRef**	**1244**
World & Sites	**BuildingRef**	**1248**
World & Sites	**HarvestingMachineRef**	**1250**
World & Sites	**Planet**	**1252**
World & Sites	**ScanResult**	**1253**
World & Sites	**Site**	**1255**
World & Sites	**ExoticDeposit**	**1258**
World & Sites	**GeologicalAnomaly**	**1264**
World & Sites	**MiningSite**	**1267**
World & Sites	**OilWell**	**1271**
World & Sites	**ThermalVent**	**1275**
World & Sites	**WaterWell**	**1280**
Storage & Inventory	**Battery**	**1284**
Storage & Inventory	**Holder**	**1286**
Storage & Inventory	**PortableBattery**	**1288**
Storage & Inventory	**Cargo**	**1290**
Storage & Inventory	**Rack**	**1293**
Storage & Inventory	**Bin**	**1294**
Storage & Inventory	**DiscardResult**	**1296**
Storage & Inventory	**InputSlot**	**1298**

Storage & Inventory	**ItemInfo**	**1307**
Storage & Inventory	**ItemResult**	**1310**
Storage & Inventory	**ItemStack**	**1311**
Storage & Inventory	**OutputSlot**	**1313**
Storage & Inventory	**PickupOutputSlot**	**1318**
Storage & Inventory	**Recipe**	**1319**
Storage & Inventory	**SaleResult**	**1323**
Storage & Inventory	**ShopItem**	**1325**
Storage & Inventory	**Slot**	**1326**
Storage & Inventory	**TransferResult**	**1329**
Storage & Inventory	**VehicleInputSlot**	**1332**
Storage & Inventory	**WarehouseSlot**	**1337**
Fleet & Vehicles	**BlockedContact**	**1339**
Fleet & Vehicles	**CollectResult**	**1341**
Fleet & Vehicles	**ConstructorModule**	**1343**
Fleet & Vehicles	**DrillModule**	**1346**
Fleet & Vehicles	**DroneBattery**	**1348**
Fleet & Vehicles	**DroneCargo**	**1350**
Fleet & Vehicles	**DroneOilTank**	**1356**
Fleet & Vehicles	**Fleet**	**1358**
Fleet & Vehicles	**DroneRef**	**1360**
Fleet & Vehicles	**MobileUnitRef**	**1365**
Fleet & Vehicles	**VehicleRef**	**1369**
Fleet & Vehicles	**MountSlot**	**1374**
Fleet & Vehicles	**NavModule**	**1376**
Fleet & Vehicles	**SonarModule**	**1380**
Fleet & Vehicles	**SonarScanResult**	**1385**
Fleet & Vehicles	**SurveyResult**	**1387**
Infrastructure & Fluids	**ComputerDeployResult**	**1388**
Infrastructure & Fluids	**Construction**	**1390**
Infrastructure & Fluids	**FluidPort**	**1393**
Infrastructure & Fluids	**FluidConnection**	**1396**
Infrastructure & Fluids	**Pipe**	**1399**
Infrastructure & Fluids	**PowerGrid**	**1403**
Infrastructure & Fluids	**PowerGridMember**	**1407**
Infrastructure & Fluids	**PowerSummary**	**1410**

Infrastructure & Fluids	**RunControlStatus**	**1412**
Infrastructure & Fluids	**ScriptVariantRef**	**1414**
Orders & Comms	**BroadcastInfo**	**1415**
Orders & Comms	**CommsMessage**	**1416**
Orders & Comms	**DockSlot**	**1418**
Orders & Comms	**Order**	**1419**
Orders & Comms	**ReceiveResult**	**1423**
Orders & Comms	**SendResult**	**1424**
Orders & Comms	**TransmitterInfo**	**1425**
Orders & Comms	**WaitAnyResult**	**1426**
Orders & Comms	**WaitBroadcastResult**	**1428**
Panels	**Panel**	**1429**
Panels	**PanelBounds**	**1470**
Panels	**PanelClick**	**1472**
Panels	**PanelKey**	**1473**
Panels	**PanelMouse**	**1475**
Panels	**PanelSize**	**1477**
Contracts	**Contract**	**1479**
Contracts	**BeatTheSystemContract**	**1482**
Contracts	**Arbiter**	**1484**
Contracts	**BuriedFiveContract**	**1487**
Contracts	**Analyzer**	**1490**
Contracts	**ColdBootContract**	**1491**
Contracts	**CoreSampleContract**	**1493**
Contracts	**CoreDevice**	**1496**
Contracts	**CorruptedArchiveContract**	**1498**
Contracts	**Archive**	**1500**
Contracts	**CrosstalkContract**	**1502**
Contracts	**DataTabletContract**	**1505**
Contracts	**DataTablet**	**1507**
Contracts	**ProbeResult**	**1509**
Contracts	**DriftingSignalContract**	**1510**
Contracts	**SlabDevice**	**1512**
Contracts	**LatticeContract**	**1513**
Contracts	**LatticeGrid**	**1515**
Contracts	**RelayHackContract**	**1518**

Contracts	**RelayLock**	**1520**
Contracts	**SealedVaultContract**	**1522**
Contracts	**Vault**	**1524**
Contracts	**VaultPosition**	**1526**
Contracts	**TerminalBreachContract**	**1528**
Contracts	**AlienTerminal**	**1530**
Contracts	**GuessResult**	**1532**
Contracts	**TheLoomContract**	**1533**
Contracts	**Loom**	**1536**
Contracts	**ThreeEchoesContract**	**1537**
Contracts	**ThreeEchoesBroadcast**	**1539**
Contracts	**XenogeneticsContract**	**1540**
Contracts	**ContractScript**	**1542**
Contracts	**LatticeProbeResult**	**1543**
Contracts	**VaultEscapeResult**	**1544**
Built-in Types	**Counter**	**1545**
Built-in Types	**defaultdict**	**1547**
Built-in Types	**deque**	**1548**
Built-in Types	**dict**	**1553**
Built-in Types	**float**	**1558**
Built-in Types	**generator**	**1560**
Built-in Types	**int**	**1562**
Built-in Types	**list**	**1565**
Built-in Types	**OrderedDict**	**1570**
Built-in Types	**set**	**1572**
Built-in Types	**slice**	**1579**
Built-in Types	**str**	**1580**
Built-in Types	**tuple**	**1597**
Biosphere	**AnalyzeResult**	**1599**
Biosphere	**BioExtractionResult**	**1600**
Biosphere	**BioScanResult**	**1601**
Biosphere	**Cell**	**1602**
Biosphere	**LifeFormScanResult**	**1607**
Biosphere	**LifeFormSample**	**1608**
Biosphere	**PlantRequirement**	**1611**
Biosphere	**PortableBioExtractor**	**1613**

Biosphere	**PortableBioScanner**	**1614**
Biosphere	**SeedRecipe**	**1615**
Biosphere	**SeedResult**	**1619**
Built-in Modules	**CacheInfo**	**1621**
Built-in Modules	**Field**	**1623**
Built-in Modules	**Match**	**1625**
Built-in Modules	**Pattern**	**1630**
Exploration	**CatalogedCreature**	**1636**
Exploration	**CatalogedFragment**	**1641**
Exploration	**Marker**	**1646**
Infrastructure	**BlueprintPlanResult**	**1648**
Power	**BatteryChargerSlot**	**1650**
System	**ActionResult**	**1653**
System	**CommandResult**	**1660**
System	**CountResult**	**1661**
System	**CropJob**	**1662**
System	**CropJobResult**	**1664**
System	**JobReceipt**	**1667**
Terraforming	**AnalyzeInfo**	**1669**
Terraforming	**BioCasterRecipe**	**1674**
Terraforming	**BioOrder**	**1676**
Terraforming	**ChamberFragment**	**1679**
Terraforming	**ChamberSample**	**1681**
Terraforming	**FragmentLocation**	**1683**
Terraforming	**HabitatBonusNode**	**1687**
Terraforming	**HabitatBonusTree**	**1691**
Terraforming	**HabitatInsight**	**1693**
Terraforming	**Specimen**	**1695**
Terraforming	**TerraformPillar**	**1700**
Terraforming	**WasteDumpResult**	**1703**
Vehicles & Modules	**BatterySwapResult**	**1704**
Weather & Sky	**SignalReceiver**	**1706**
Weather & Sky	**SignalTransmission**	**1707**
Weather & Sky	**Storm**	**1710**
Weather & Sky	**WeatherEventForecast**	**1713**
Weather & Sky	**WeatherReport**	**1715**

Weather & Sky	**WeatherSignalBoard**	**1717**
Weather & Sky	**WeatherSignalBoardStatus**	**1721**
Weather & Sky	**WeatherStrike**	**1723**
Weather & Sky	**Zone**	**1725**

Getting Started

Welcome to Code: Terraform.

You are connected to a remote terminal on an uninhabited planet. Your goal is to terraform it, make it livable, by writing code.

Everything on this planet is controlled through code. Systems, sensors, machines, you bring them online by programming them.

Start by typing code in the editor and clicking ▶ to run it.

```
print("Hello, planet!")
```

The console shows your output and any errors. Sensor readings update as you activate systems.

LOOKING THINGS UP WHILE YOU CODE

- **Hover** over any method or function name in your script to see its signature, description, and return type inline.
- **Cmd+click (macOS) / Ctrl+click (Windows / Linux)** on any method, function, or component id to open DOCS and jump straight to that entry. The block you clicked is briefly highlighted so you can see where you landed.

See the **Code Editor** DOCS page for the full set of editor shortcuts (rename, autocomplete, parameter hints, find/replace, debug stepping).

First Steps

After **First Contact**, the boot/power/sensors/uplink tutorial chain, you have **2,500 credits** and the **Shop** and **Biology Lab** are unlocked. This page is the handoff guide for what to do next. It remains available in DOCS; search for **First Steps** anytime you want a refresher.

NEXT STEPS

1. **Pressure Sensor and Oxygen Sensor are already deployed.** Repair **Pressure** to open Harvesting and Contracts; repair **Oxygen** to open Atmosphere. Research is already available.
2. **Open the Shop** in the left sidebar and buy a **Solar Generator**, a **Battery**, and an **Oxygen Generator**.
3. **Open Inventory** and **Deploy** all three.
4. **Script the Solar Generator and Oxygen Generator.** At **1 ppt oxygen**, Auto Feeders unlock and the Biology Lab can be automated.

> Repair the sensors in either order. This starter setup reserves enough credits and power for manual Biology now and automated Biology after Auto Feeders unlock.

EARNING CREDITS

Four early approaches teach different skills. Harvesting and Contracts sit side by side: which feels easier depends on whether you prefer visible machines or contained logic puzzles.

- **MANUAL BIOLOGY** *(easiest, no code)*, use the Biology workbench buttons to collect, analyze, extract, and deliver samples. This is the simplest first payout, but repeated orders become a click-heavy process.
- **HARVESTING** *(first automation, after Pressure repair)*, write a simple grid sweep that moves, collects, stores, and sells. Heat and the held-item slot need attention, but pathfinding and route optimization are optional.
- **CONTRACTS** *(contained puzzles, after Pressure repair)*, solve isolated programming problems with minimal machine state and safe retries. These favor loops, collections, and abstract reasoning.
- **AUTOMATED BIOLOGY** *(advanced automation, after Auto Feeders)*, coordinate the Bio Collector, Bio Lab, and Bio Exchange through separate scripts while managing orders, reagents, item ports, completed output, and partial state.

Every credit feeds back into the Shop. More machines means more terraforming progress.

Command Results

THE BASIC PATTERN

Every command that changes the world returns a fixed result object. You may ignore the result when your script does not need to react:

```
self.set_power(5)
```

Store it when the outcome matters:

```
result = self.extract()
if result.status == "ok":
  print("sample ready")
else:
  print(result.message)
```

Every result has:

- `.status`, a stable machine-readable code used for program logic
- `.message`, a localized player-readable explanation used for display and debugging

Never branch on `.message`; its wording can change or be translated. A command with no payload returns `ActionResult`. A payload-bearing command returns a specialized result with the same two fields plus named payload fields. `AnalyzeResult` adds `.info`, `SonarScanResult` adds `.sites`, and `TransferResult` adds `.requested` and `.moved`.

RESULTS ARE NOT BOOLEANS

A result object is a structured value, not a success boolean. The object itself remains truthy even when its status describes a rejection. Do not write:

```
if self.analyze():
  print("success")
```

Store the result and compare its documented status instead. `"ok"` is common, but it is not the universal spelling of success. Depending on the command, successful or accepted outcomes can include `"started"`, `"queued"`, `"charging"`, `"complete"`, or another domain-specific code. The command's DOCS outcome table is authoritative.

PAYLOAD FIELDS

Read a payload only after a status that documents it:

```
analysis = self.analyze()
if analysis.status == "ok":
  recipe = analysis.info.required_recipe
else:
  print(analysis.message)
```

Rejected results keep the same result type. They do not turn into `False`, `None`, an empty `dict`, or a bare status string.

QUERIES RETURN NATURAL VALUES

Read-only queries do not perform gameplay commands, so they return their natural value directly. `self.is_running()` returns a boolean, `self.cargo.count()` returns a number, and `self.sonar.tier()` returns a string. An optional query can return `None` only for the one absence meaning stated in its DOCS entry.

WAITING AND BACKGROUND WORK

Some commands yield: the script pauses while that attempted operation finishes, then receives its result. `self.analyze()` is one example. Other commands start or queue persistent work and return immediately. A charging station's `charge(...)` result can report `"charging"` or `"queued"` while the station continues working in the background.

Do not assume that every timed system waits in the same way. The command description states whether it yields, starts work, or queues work.

BUSY AND EXCEPTIONS

`"busy"` is an ordinary transient outcome for commands that define it. Store that result, allow game time or the blocking state to change, then call the command again when your script wants a fresh result. Do not retry every non-success status as though all outcomes meant `"busy"`.

Wrong argument types and documented malformed values can raise precise exceptions such as `TypeError` or `ValueError`. Ordinary mutable world conditions use the result statuses listed for that command. Open the command's DOCS entry for its exact statuses, payload fields, timing, and exceptions. See **Long-Running Scripts** for using these results in automation that remains safe after a game load.

Long-Running Scripts

WHAT HAPPENS WHEN A GAME LOADS

A script that was running begins again from its first line when the game loads. The game saves the script's source and running status, but it does not restore the current line, local variables, call stack, or pending return value. If a script was paused when saved, it loads paused, but only the paused status is preserved. Its execution position is reset. Pressing Resume begins a new run from the start of the source; it does not continue from the previous line.

Saved world state can outlive one script run. A machine may still contain cargo, a specimen, staged materials, queued jobs, completed output, or timed work when the restarted script reaches it. That state is not an error. It is the starting point for the next decision.

MAKE THE MACHINE THE SOURCE OF TRUTH

Build a long-running script as a reconciliation loop. Each pass should:

1. Read the machine's current state.
2. Decide which step is still missing.
3. Attempt that step once.
4. Check the returned `.status`.
5. After a transient blocker such as `"busy"`, let game time pass before retrying the step.

Do not depend on a local variable from an earlier pass to remember what the machine contains. Read the chamber, cargo, ports, queue, or current job again.

BIO LAB EXAMPLE

This loop can begin with an empty Lab, a collected specimen, an analysis already underway, or an analyzed specimen. It always re-reads the chamber before deciding what to do:

```
collector = get_component("bio_collector_1")

while True:
  specimen = self.specimen

  if specimen is None:
    transfer = self.take_from(collector)
    if transfer.status == "busy":
      sleep(0.2)
      continue
    if transfer.status != "ok":
      print(transfer.message)
      sleep(0.2)
      continue
    continue

  if specimen.stage == "collected":
    analysis = self.analyze()
    if analysis.status == "busy":
      sleep(0.2)
      continue
    if analysis.status != "ok":
      print(analysis.message)
      sleep(0.2)
      continue
    print(analysis.info.required_recipe)
    continue

  print(specimen.recipe)
  break
```

The payload rule matters here: `analysis.info` is available after `analysis.status == "ok"`. A `"busy"` result has no analysis payload, so the script waits and starts another pass instead of reading `.info`.

MAKE WRITES SAFE TO REPEAT

Before transferring, buying, loading, or queuing something, inspect what is already present. Calculate the missing quantity instead of submitting the full planned quantity again. Drain completed output before starting more work, and distinguish a transient `"busy"` from blockers such as `"output_full"`, `"insufficient_input"`, or an invalid argument. The command's DOCS entry lists which outcomes can change when the world changes.

PERSIST KNOWLEDGE DELIBERATELY

Machine contents are world state, but ordinary script variables belong to one run. Use the **Data Archive** when automation must remember learned data, checkpoints, or decisions across script stops and game loads. Use **Command Results** for the general result contract and **Loops & Scripts** for loop and pacing basics.

Frequently Asked Questions

Short answers to the questions commanders ask most. Each one points at the DOCS page that owns the full detail.

SAVES, FILES, AND TOOLS

Where are my saves and scripts on disk?
Each save stores its scripts as ordinary `.py` files in a `<save_id>_scripts/` folder under your platform's app-data directory, with that save's `.json` file beside it. Windows uses `%APPDATA%\io.codeterraform.game\`, macOS uses `~/Library/Application Support/io.codeterraform.game/`, and Linux uses `~/.local/share/io.codeterraform.game/`. To back a save up, copy both the `.json` file and its matching `_scripts` folder. **External Editor** lists every file in that folder and what it is for.

Can I use VS Code or another editor instead of the in-game one?
Yes, and edits sync live in both directions while the game runs. Settings > Editor > External Editor has **Set Up VS Code**, which installs the bundled extension and opens the scripts folder; completion, diagnostics, hover, and breakpoints then use the same analysis as the in-game editor. Other editors read the generated stubs sitting in the same folder. See **External Editor**.

Can I read the DOCS without the clock running?
Use **Export** in the DOCS toolbar. You choose which parts to include, and the manual saves as a PDF, a web page, or Markdown.

I found a problem, or I have an idea. Where does it go?
Use the **Feedback** button on the main menu or the pause menu. It sends the report with your build id attached, so it can be matched to the exact build it came from. The same screens link to Discord, which is where questions, follow-up, and opt-in test builds are discussed.

ITEMS AND STORAGE

Do I have to carry items to an outpost, or can it use my Inventory?
Inventory is a physical storeroom at Nocturna Base, not a planet-wide store. Machines and vehicles can use it as a freight endpoint only at home. Anywhere else, reagents, ore, and parts must already be in that outpost's own storage or aboard a vehicle parked there, so a Pioneer or a drone is how they arrive.

How do I get rid of items I do not want?
It depends where the item sits. `shop.sell(item_id, quantity)` turns Base Inventory stock into credits. `inventory.drop(slot)` deletes one unit from a Base Inventory slot by its zero-based index, and a vehicle uses `self.cargo.discard(rack_index)` against the racks that `self.cargo.racks()`

reports. For a continuous overflow stream, research **Waste Processing** and fabricate a **Waste Processor**: it permanently destroys one selected stream, items through its feeder input, liquid through `liquid_in`, or gas through `gas_in`.

A machine input is holding items I want back. Can I recover them?
Yes. `input.eject(destination, item_id, count)` moves buffered items to another endpoint without changing the port's source connection, and it preserves item properties and respects destination capacity. `input.flush()` is the other tool, and it destroys the entire input buffer, so reach for it only when that is exactly what you want.

How do I move items between a Warehouse, a Storage Bin, and Inventory?
Stores never talk to each other on their own. A machine port connects to one endpoint at a time with `connect(name)`, then `take(item_id, count)` and `send(item_id, count)` move the goods, and a transfer takes time proportional to the units moved. At Nocturna Base `"inventory"` is a valid endpoint; elsewhere both ends must sit at the same outpost. See **Input & Output**.

Why does `take()` or `send()` return `research_required`?
Item flow through machine ports needs **Auto Feeders** research. Until then the ports exist but move nothing, which is why the Biology chain runs by hand first. **Fast Feeders**, much later, halves the duration of newly started transfers.

BUILDING AND OUTPOSTS

How do I found an outpost?
Buying the Outpost Kit is only the material step. You also need **Outpost Construction** and **Constructor Module** research, plus a Pioneer with a Nav Module and a Constructor Module mounted, a Battery Holder with a charged Portable Battery, and a Cargo Rack holding the kit. Place an Outpost blueprint in Plan Mode or with `construction_blueprint.plan_structure("outpost", x, y)`, drive there, brake, and build it. **First Outpost** walks the whole sequence.

How do I deploy a building at an outpost once it exists?
Deploying is not freight, so there is usually nothing to drive out. Buy the shop kit and deploy it from **Inventory** straight into any founded outpost; upgrade packs are consumed into a finished machine the same way. Only constructor-built Planet Map machines, such as Thermal Caps, pumps, and field drills, need a Pioneer to carry the kit and build them in place.

What is the building cap, and what happens when I pass it?
It is a soft cap and it never blocks a deploy. Nocturna Base holds **25** buildings and a founded outpost holds **20**; **Outpost Expansion** research adds **5** to every current and future outpost. Past the cap each extra building costs **10%** efficiency at that outpost, down to a floor of **20%**. Each outpost counts on its own, so spreading out removes the penalty. `outpost_network.outposts()` reports `.buildings_used`, `.buildings_capacity`, and `.is_full`.

How do I remove or move something I built?
The route follows how it was placed. A shop kit deployed from Inventory is undeployed in **Ship Computer > System**. A fixed Harvesting-field machine is recovered with the local Harvester. A constructor-built Planet Map machine is deconstructed by a Pioneer, which reclaims the kit into its cargo. A founded outpost has its own Ship Computer > System decommission, which needs the outpost empty and returns one Outpost Kit. Undeploy is blocked only while real goods are still stored inside; stored charge and fluid buffers never block it.

EXPLORATION AND MINING

What does "exceeds this sonar's detection limit" mean, and why does my vehicle keep rescanning the same spot?
The sweep worked, but a contact nearby is harder than your sonar can identify. Minerals run from hardness **1** to **4**, and `sonar.hardness_limit()` and `drill.hardness_limit()` report what your current modules handle. Such a contact stays unscanned, so a loop filtering on `not point.scanned` reselects it forever. The sweep lists those contacts in `scan.blocked` with their coordinates and a reason: record them and skip them until you upgrade the sonar and scan again.

What is the difference between a point of interest and a site?
A point of interest is a permanent `?` on the Planet Map with whole-number coordinates and a `kind` that stays `"unknown"` until a scanner reaches it. `scan()` classifies contacts in range into sites, and `survey()` then reveals a productive site's details. `journal.surveyed_sites(planet_id)` lists everything fully resolved.

Where is titanium?
Titanium is hardness **2**, so a starter sonar cannot identify it and a starter drill cannot cut it even when it is right beside you. Contracts can ask for it well before you can reach it, which is normal. Upgrade the sonar first, rescan the contacts your Journal recorded as too hard, then upgrade the drill.

What do I do when a vehicle runs out of power in the field?
A **Vehicle Charging Station** can send a field-service drone. Call `self.dispatch_rescue(name)` from the station's own script, optionally with a target battery fraction. There is no distance limit, and the station only needs power at launch; the mission survives a later outage. Only one rescue runs at a time, so guard it with `if not self.is_rescuing():`, and the vehicle itself reports `self.is_being_rescued()` and `self.rescue_status()`.

Rover or Pioneer, what is the difference?
The Rover is the introductory vehicle. Its three fixed slots take a Nav Module plus the basic sonar and drill only; a Pioneer has eight universal slots, so it does the same field work, mining included, with the higher-tier modules the Rover cannot fit, and it is the only chassis that carries a Constructor Module and builds. The trade is outfitting: the Rover arrives with an integrated **100 Wh** battery and a **10-unit** hold, while a bare Pioneer has nothing built in, not even a battery. See **First Pioneer**.

What is the Harvester for?
It is the slow field vehicle for the surface grid: collecting what is scattered there, planting and tending crops, harvesting Forage, and deploying field automation. The items lying on the field at the start are a finite early credit source, so selling them is fine. The field's long-term job is the Plants pillar. See **First Harvesting Route**.

Is the harvesting field part of the Planet Map?
No. They are separate places and they do not interact. The harvesting field is a fixed local grid around Nocturna Base, 8 rows (A to H) by 24 columns (1 to 24), addressed `A1` through `H24`; it is generated once per save and there is only one, so founding an outpost does not add another. The Planet Map is the planet itself, where sonar contacts, sites, outposts, and constructor-built machines live. A Harvester works only its own grid, a Rover or Pioneer only the Planet Map, and neither reaches the other.

SCRIPTING

How do I get every machine of one type?
`get_component("outpost_network").outposts()` gives every outpost, and each one has `.buildings(type_id)` returning `BuildingRef` snapshots with `.id`, `.name`, `.type_id`, `.powered`, and `.position`. Loop the outposts to cover the whole planet, and pass `ref.id` to `get_component()` for live type-specific reads. Two families are deliberately not in that list: mobile units come from `get_component("fleet").vehicles()` and `.drones()`, and fixed Harvesting-field machines such as Crop Automators, Grow Lamps, Sprinklers, and Dispensers come from `outpost.harvesting_machines(type_id)` on the home outpost, because they occupy field cells rather than outpost building slots.

Do I copy my script into every new machine of the same type?
No. Save the code as a named variant in the editor's **Variants** tab, then use **Apply to all**, which copies it into every matching machine and restarts the running ones. It is a one-time copy rather than a live link, so later edits to the variant do not reach machines already holding a copy. See **Script Variants**.

How do I turn a machine on or off, or start and stop its script, from code?
Those controls live on shared components, not on the machine. `get_component("power_control").set_powered(machine_id, on)` is the breaker: switching it off pauses the script, which resumes when power returns. `get_component("run_control")` is the Run and Stop buttons: `start(id)`, `stop(id)` (stays off until started again), `is_running(id)`, and `apply_variant(id, variant_id)` to switch a stopped machine to a saved variant. See **Power Control** and **Run Control**.

Why does only one machine get my Signal Bus message?
`receive()` consumes a message, so the first reader takes it and everyone else sees `"empty"`. Send once per consumer, or publish a broadcast value and read it with `latest()`, which does not consume. See **Signal Bus** for queues, broadcasts, and `wait()`.

Can a script wait on two things at once? Are there threads?
There are no threads, and none are needed. Every machine's script runs independently, so the usual answer is to split the work across the machines that own it and coordinate over the Signal Bus. Inside one script, `comms.wait(channel)` idles only that script until a message arrives, while the planet and every other script keep running.

Does the language support classes?
Yes, including `__init__`, instance and class attributes, inheritance, `super()`, `isinstance`, and `issubclass`. Inside a method `self` is the object, which never collides with a machine script's top-level `self`; pass the machine in if a method needs to drive it. `import json`, `re`, `random`, `heapq`, `functools`, and `dataclasses` are available too. See **Writing Classes**.

Why is `self` undefined inside my Library?
Library scripts run in their own shared scope, so `self` and `panel` are never defined there. Pass the component, or its id, into the helper as a parameter and call it from the machine script that owns the hardware. See **Imports & Libraries**.

How This Game Works

Code: Terraform is a systems game where progress comes from understanding machines, then writing small scripts that make those machines useful.

THE BASIC LOOP

1. Pick one machine or problem.
2. Read that machine's **Info** tab.
3. Check DOCS for the methods and return values it exposes.
4. Write a small script that does one clear thing.
5. Run it, inspect returned results, watch the console, then improve it.

The game is not asking you to memorize everything. It is asking you to learn where information lives. Info tabs explain intent. DOCS explain APIs. A command result explains the exact programmatic outcome. The console shows output your script chooses to print, uncaught errors, and dedicated test transcripts, while the dashboard shows whether the world changed.

COMMANDS AND GAME TIME

Gameplay commands consistently return a result object: branch on `result.status`, read `result.message`, and then use any command-specific payload fields. Commands do not print their ordinary outcomes automatically. Some commands pause the script while their attempted work finishes; others start or queue background work and return its current state immediately. The planet keeps running in either case. See **Command Results** for the complete model.

GOOD HABITS

Start with one machine. Store every important command result. Check `.status`; print `.message` while learning. Once a tiny script behaves, make it loop. Once one loop behaves, connect it to another system.

Choosing Your First Credit Path

After First Contact, credits become the fuel for expansion. You have four early approaches, and each teaches a different skill. Manual Biology is open immediately. Harvesting and Contracts appear after you repair the Pressure Sensor. Automated Biology becomes practical after your starter Oxygen Generator reaches **1 ppt oxygen** and unlocks **Auto Feeders**.

MANUAL BIOLOGY - EASIEST FIRST PAYOUT

Your **2,500 starting credits** cover a Solar Generator, Battery, and Oxygen Generator, with credits left for starter reagents. Use the Biology workbench buttons to collect a specimen, analyze it, load its revealed reagents, extract the sample, and deliver it. No code is required. This makes the workflow easy to learn, but the repeated actions become slow after the first few orders.

HARVESTING - FIRST MACHINE AUTOMATION

A basic Harvester script can sweep the grid, collect what is present, store it, and sell from Inventory. It must account for heat and the one held-item slot, but it does not need pathfinding or an optimized route to work. This is the most direct first automation project for players who like visible machine behavior.

CONTRACTS - CONTAINED PROGRAMMING PUZZLES

Contracts are not production machines. They read data, compute an answer, and transmit it to Earth. They involve little machine API state and failed answers are safe to retry, but their loops, collections, and abstract reasoning may feel easier or harder than Harvesting depending on the player.

AUTOMATED BIOLOGY - ADVANCED EARLY PROJECT

After Auto Feeders unlock, Biology changes from one manual workbench flow into three cooperating machine scripts. The Bio Collector, Bio Lab, and Bio Exchange each own their actions. Reliable automation must coordinate order progress, reagent staging, item ports, completed output, and whatever specimen or reagent state remains after a script stops.

CHOOSING WELL

Start with manual Biology if you want the simplest first payout. Choose Harvesting for visible automation or Contracts for a focused logic problem. Automate the full Biology chain when you want a more advanced multi-machine project. All four approaches remain valid ways to earn early credits.

Terraform Index and planetary completion

The Terraform Index is your overall progress toward a livable planet. It has a fixed goal of **1,000,000 Terraform Points (TP)**. Those points come from six pillars, so steady progress eventually means looking beyond the first machines you built.

THE SIX PILLARS

Group	Pillars	Maximum share of the Index
Atmosphere	Temperature, oxygen and pressure	700,000 TP together
Biosphere	Biomass, plants and wildlife	100,000 TP each

The three atmospheric pillars are available first. **Biosphere** research opens at **210,000 TP**, making the other three pillars visible. Their production systems open along the later research path. Unlocking Biosphere does not change the million-point goal or take away points you already earned.

Each pillar advances through named phases. You earn progress between phase boundaries as well as when crossing them. The meters use different units and different phase thresholds, so one extra unit of heat, oxygen or biomass does not mean one TP. Compare progress and the next milestone rather than adding raw readings yourself.

A FINISHED PILLAR CANNOT REPLACE ANOTHER ONE

Each pillar's Index contribution stops growing when it reaches its final target. Extra output in that pillar cannot make up for unfinished plants or wildlife. Even a completely finished atmosphere contributes at most **700,000 TP**.

If the Index slows while one set of machines is still producing, check whether their pillar is already complete. Put the next expansion into an unfinished pillar or its supply chain. Do not shut down a useful supply blindly: some machines also support other systems, such as the atmospheric carbon cycle.

USE RESEARCH TO CHOOSE THE NEXT PROJECT

Research unlocks automatically when its own requirement is reached. Some entries follow the combined Terraform Index; others follow temperature, oxygen, pressure or a Biosphere pillar. Read the requirement on the research card. More pressure will not satisfy a target that asks for plants.

Oxygen- and pressure-based research also need their matching sensors repaired and online. If an expected unlock is missing, check the sensor as well as the production reading.

Research and recipe rewards are separate. An unlock can open a machine, a shop item or a particular recipe. Earth Orders supply other recipes and blueprints. If you own a machine but cannot select the recipe you want, check its recipe list and the order rewards instead of simply producing more TP. See `orders_guide` for that progression.

CHECK WHAT IS ACTUALLY HOLDING YOU BACK

Start with an unfinished pillar, then follow its inputs. Is its machinery powered? Does it have the required material or fluid? Is an output full? Does it need an upgrade for the current stage? Expanding a stopped machine's fleet usually repeats the same shortage.

Scripts can read current pillar snapshots:

```
terraforming = get_component("terraforming")
print(terraforming.total_tp(), terraforming.index_progress())
for pillar in terraforming.pillars():
  print(pillar.id, pillar.progress, pillar.complete)
```

The pillar list contains the currently unlocked pillars. Each also exposes `.value`, `.target` and `.next_target` for planning. Query again when you need fresh readings.

For the first atmospheric setup, see `power_and_terraforming_machines`. The later production paths are in `biosphere_tier`, `plants_overview` and `wildlife_overview`. All these guides are available before the corresponding equipment unlocks, so you can plan ahead.

Power & Terraforming Machines

Terraforming machines are the core engines of the base. Heat Generators, Oxygen Generators, and Pressure Generators turn power into planetary progress.

POWER COMES FIRST

A machine that has no power cannot help the planet. Early solar power changes with day and night, so a base that works at noon may stall after sunset. Batteries smooth that gap by storing daytime surplus for nighttime use.

A reliable early setup is: add a generator, add a battery, run one terraforming machine, then expand power before adding too many more consumers.

TERRAFORMING STARTS WITH THREE TRACKS

Temperature, oxygen, and pressure are the first three tracks, each with their own machines and upgrades. They all matter. You can focus one track for a while, but the planet only becomes livable when all three keep advancing. Later progress opens more terraforming tracks; for now, make these first systems dependable.

The first machines are slow. Add power, add machines, then expand to more outposts and higher tiers to raise the pace.

OXYGEN RUNS ON CO2, THE CARBON CYCLE

Oxygen Generators convert atmospheric CO2 into oxygen, one-for-one. The planet starts with a large CO2 reserve, and that reserve is enough for a long time, but large-scale oxygen production outpaces it. CO2 is returned to the atmosphere by burning oil (Oil Generator), incinerating items (Waste Processor), and, at real scale, by wildlife: established colonies exhale CO2. A small volcanic trickle keeps CO2 from ever bottoming out completely, so a starved oxygen fleet always recovers. Watch `atmosphere.get_co2()` and remember the efficiency sweet spot moves with it.

SCRIPTS SHOULD REACT

Good machine scripts read state before acting. A generator can react to sun angle. A battery or power monitor can warn when storage is low. A terraforming machine can throttle down when power is tight and resume when the base recovers.

If generators stop at night or terraforming machines pause from low power, that is usually not a bug. Power production, storage, and consumption have to fit together.

See `terraforming_progress` for how the six pillars contribute to the Index and open research. For the late-game uranium supply chain, see `nuclear_power`.

First Biology Loop

Manual Biology is the gentlest first credit path because the workbench guides each narrow machine step. Automating the same pipeline is an advanced early project because its separate machine scripts must coordinate persistent state.

MANUAL AND AUTOMATED LOGISTICS

The manual Biology workbench has exactly **one stockroom per outpost**. At Nocturna Base that stockroom is Base Inventory. At every other outpost, choose one deployed local Warehouse in the Manual Stockroom bar; all manual Biology cards there draw from and return to that Warehouse. The selection does not move existing items. Shop purchases still arrive in Base Inventory, so carry supplies to remote Warehouses with vehicles or drones.

Manual buttons do not create hidden I/O connections or move items through a machine's `self.input` and `self.output` buffers. They load supplies directly from the manual stockroom into the workbench and return finished or recovered items directly to that stockroom. A full machine input or output buffer therefore does not block the matching manual action.

WHAT CHANGES WHEN YOU START SCRIPTING

Scripts do not use the manual stockroom selector. After **Auto Feeders** research, your code must manage each machine's scripted logistics explicitly: connect item ports where present, move supplies into `self.input`, load them into the machine when required, and drain completed items from `self.output`. Every stationary transfer remains local to that outpost.

The Bio Collector, Bio Lab, and Bio Exchange each own their hardware actions. Stopping or restarting a script does not empty a specimen chamber, unload reagents, drain an output, or reset Bio Order progress. Reliable code must inspect that partial state before deciding what to do next.

```
input_link = self.input.connect("Reagent Warehouse")
output_link = self.output.connect("Sample Warehouse")
if input_link.status != "ok":
  print(input_link.message)
```

THE PIPELINE

- The Bio Collector retrieves a specimen.
- The Bio Lab analyzes it, stages reagents, and extracts a sample.

- Biome machines transform the sample.
- The Bio Exchange delivers it toward an active Bio Order.

Every gameplay command returns a result object. Branch on `.status`, use `.message` for the exact reason, and read any method-specific payload. For example, `analyze()` returns `AnalyzeResult.info`, while item transfers return `TransferResult.moved`. Statuses such as `"busy"`, `"no_input"`, and `"output_full"` never arrive on their own: a call always returns them inside a result object.

Properties belong to each item stack and survive supported transfers. Bio Orders come from a Bio Exchange, not the Earth Supply Dock `orders` component. Subtract both `.delivered` and `.in_transit` when planning production. See **Long-Running Scripts** for the restart-safe loop pattern used by unattended automation.

First Harvesting Route

Harvesting is the most direct early path into visible machine automation. It can earn more than manual Biology, while keeping the first working version much smaller than a reliable automated Biology chain.

WHAT YOU ARE BUILDING

A useful harvesting route usually has three parts: choose where to go, collect material, then store and sell when Inventory says it is time. The main decision is not the individual call; it is deciding when to move on.

WHAT TO INSPECT

Read the Harvester Info tab first. Then check the Scanner and Harvester entries in DOCS. The Scanner teaches what is known about the surface. The Harvester teaches movement, collection, heat, and the held-item slot.

KEEP THE FIRST ROUTE SIMPLE

You do not need pathfinding. A fixed sweep across adjacent grid cells can collect and store everything it finds. Check heat before moving, handle the held item before collecting again, and optimize the route only after the basic loop works.

A GOOD FIRST TARGET

Start with one short trip that reaches a known area, collects something, stores it, and leaves it ready to sell from Inventory. Then extend the sweep or add smarter choices.

Contracts

Contracts are Earth-side programming jobs. They are closer to puzzle problems than factory automation.

WHAT MAKES THEM DIFFERENT

A machine script usually changes the planet. A contract script reads the contract data, computes an answer, and sends it back. The reward comes from correctness, not from keeping a production loop running. Contract state is contained, and a wrong answer is safe to inspect and retry.

HOW TO APPROACH ONE

Read the briefing first. Then inspect the available contract data from the script. Work out the answer in small steps and print intermediate values while you are learning the shape of the data. When the answer is ready, use the transmitter workflow described in the contract and DOCS.

DIFFICULTY

Contracts and Harvesting test different skills, so neither is universally harder. Contracts avoid routes, heat, cargo, and multi-machine recovery, but ask for more abstract reasoning with loops and collections. They are a good choice when you want a focused coding puzzle and a bigger one-time payout. If a contract feels too abstract, leave it and build the base for a while; it will still be there later.

Reading Errors and Console Output

The console is part of the interface. It shows output your script explicitly prints, uncaught exceptions, and dedicated transcripts such as sensor calibration tests.

OPENING IT

Click **Console** in the bottom window bar to open or focus it. In dock mode, the console sits in the lower-right pane beside the editor. Use the console filter to switch between THIS SCRIPT, ALL output, WARNINGS, and ERRORS.

RESULTS ARE THE PROGRAM INTERFACE

Gameplay commands return fixed result objects without printing ordinary success or rejection messages automatically. `result.status` is the stable code to branch on, and `result.message` explains the exact outcome. Payload-bearing results add named fields such as `.moved`, `.sites`, or `.info`.

```
result = self.extract()
if result.status != "ok":
  print(result.message)
```

Your script decides which command outcomes belong in the console. Never compare `.message` in program logic because it is localized display text. Malformed calls raise typed exceptions such as `TypeError` or `ValueError`; ordinary game states such as `"busy"` or `"no_cargo_space"` remain result statuses so the script can decide what to do.

BUSY IS NOT FAILURE

A `"busy"` status means the machine is finishing another timed action, not that something broke. For the retry pattern, see **Command Results**.

DEBUGGING RHYTHM

Store the result of an important command, inspect `.status` and `.message`, then read any payload fields only for statuses that provide them. Use **Trace** when you want to see where the script is parked, and **Debug** when you need to inspect variables directly.

First Pioneer

A Pioneer is the first vehicle that turns remote plans into built infrastructure. The usual flow is: buy a Pioneer, mount the modules it needs, send it to the site, and let its local script do the work.

WHAT MAKES IT DIFFERENT

The Rover is the introductory vehicle. A Pioneer does the same field work, mining included, and does it better and faster, and it is the only chassis that can build. The chassis is only the carrier; mounted modules decide what it can actually do.

For early expansion, look for four module families:

- **Nav Module** - moves the Pioneer across the full planet coordinate plane.
- **Battery Holder** - carries Portable Batteries. At least one charged Portable Battery must be installed or the Pioneer cannot move.
- **Cargo Rack** - hosts Portable Storage Bins that carry kits, pipe segments, power-line segments, and other build materials.
- **Constructor Module** - executes Planet Map blueprints once the right materials are on board.

GOOD FIRST GOAL

Before trying to found an outpost, practice with a small job: put a simple blueprint near home, install a charged Portable Battery, load the required kit or segments into Pioneer cargo, move it to the blueprint, brake, and execute it. The blueprint can come from Plan Mode or `get_component("construction_blueprint")`; either path creates the same persistent job. If the Pioneer cannot leave, check its Battery Holders and installed batteries. If construction is rejected, check the Constructor Module and cargo.

WHERE TO LOOK

The Info tab explains the module APIs available through `self`. Plan Mode shows material requirements before placement. Query `get_component("construction_blueprint").pending_constructions()` for the queued `Construction` snapshots; every one exposes `required_item` and `required_count` to scripts.

Sonar: scanning and surveying

Sonar helps a Rover or Pioneer find resources on the Planet Map. **Scan** to identify nearby contacts, then **survey** a site to learn its extraction details. Driving past a contact does not scan it automatically.

CHOOSE A SONAR

Sonar	Range	Mineral hardness limit	Energy per scan or paid survey
Basic (Sonar Module)	50 m	1	0.5 Wh
Extended Sonar	180 m	3	1 Wh
Deep Sonar	280 m	4	2 Wh
Seismic Sonar	280 m	4	3 Wh

The hardness limit includes every softer mineral. Basic detects Iron and Silicon. Extended adds Titanium, Cobalt, Lead and Rare Earth. Deep and Seismic also detect Neutronium. **Detecting a mineral does not mean your drill can mine it**: the drill has its own hardness limit.

Basic fits the Rover's Sonar slot or a Pioneer universal slot. Extended, Deep and Seismic require a **Pioneer universal slot**. Replace the previous sonar when upgrading. Research **Sonar Module**, **Extended Sonar** or **Deep Sonar** to buy the corresponding module; **Advanced Oil Extraction** unlocks Seismic Sonar.

WHAT EACH SURVEY REVEALS

For **thermal vents and exotic deposits**, Basic reveals the active or dormant phase; an exotic survey also identifies the fluid, medium and rarity. Extended adds the base and current flow rates. Deep adds active and dormant durations and the time until the next phase. Seismic provides the same details as Deep, plus deep oil prospecting beneath inert formations.

A mineral survey reveals the mineral, hardness and purity. A water or oil well survey reveals its yield and flow rate. These sites do not use the gradual thermal/exotic detail levels.

Research and sonar capability both matter. A stronger sonar does not bypass research:

Contact	Required research	Sonar needed
Thermal vents	Geological Survey	Any
Water wells	Hydrology Survey	Any
Ordinary oil wells	Petroleum Survey	Deep or Seismic
Common and uncommon exotic deposits	Exotic Fluids	Any

| Rare exotic deposits | Rare Exotics | Any |
| Deep oil beneath inert formations | Petroleum Survey and Advanced Oil Extraction | Seismic |

SCAN, THEN SURVEY

Use the Planet Map's question-mark contacts to choose where to explore. Scripts can read them with `get_component("nocturna").points_of_interest()`. Travel within the mounted sonar's range, stop, and call `self.sonar.scan()`. The sweep checks around the vehicle's current position and records classified sites in the **Journal**.

Read the scan's `.sites` for usable contacts and `.blocked` for contacts it could not classify. **A sweep can find usable sites and blocked contacts together**, so inspect both even when some contacts need better equipment or research. Repeating the same scan cannot overcome those requirements. Biological contacts need a Portable Bio Scanner instead of vehicle sonar.

Call `self.sonar.survey(site)` on a discovered site while it remains in range. A scan takes **0.15 h**; an ordinary paid survey takes **0.25 h**. For ordinary sites, repeating a survey at the same or a lower sonar tier is free and instant. After upgrading sonar, survey known thermal vents and exotic deposits again to unlock the deeper details.

Run this from a stopped vehicle with sonar and enough battery charge. It surveys productive contacts; inert formations use the deep-oil workflow below:

```
sweep = self.sonar.scan()
print(sweep.message)
for site in sweep.sites:
  if site.kind() != "inert":
    result = self.sonar.survey(site)
    if result.status == "ok":
      print(result.site.id, result.site.kind())
    else:
      print(result.message)
```

Query `get_component("journal").discovered_sites("nocturna")` for previously classified sites. The Journal remembers discoveries, so you can plan return trips without scanning the whole map again.

REVISIT INERT FORMATIONS FOR DEEP OIL

Inert formations are the targets for Seismic Sonar. Any sonar can identify them as `GeologicalAnomaly` sites with `site.kind() == "inert"`. Ordinary scanning marks them surveyed, but that does not rule out oil deeper underground. **Revisit them even if the Journal already marks them surveyed.**

After Petroleum Survey and Advanced Oil Extraction, mount Seismic Sonar and scan within **280 m**. Read `site.seismic_status`: `"potential"` means a possible deep reservoir; `"dry"` means no deep oil.

Survey a potential contact with `self.sonar.survey(site)` to confirm an Oil Well at the same coordinates. A new deep-oil survey takes **0.5 h** and costs **3 Wh**; re-surveying a known dry formation is free. Three inert formations on each map conceal these additional reservoirs.

See `industrial_upgrades` for the full deep-oil example and Oil Pump upgrades. For individual commands and returned fields, see `SonarModule`.

For turning surveyed mineral sites into delivered ore, continue with `mining_guide`.

Mining: from discovery to delivery

Finding a deposit is the first step. To turn it into useful supplies, you need a drill that can cut it, somewhere for the ore to go, and a way to bring that ore to your machines.

START WITH A SURVEYED SITE

Use sonar to find a mineral contact, then survey it. A `MiningSite` tells you the mineral, its hardness and its purity. See `sonar_guide` for choosing sonar and working through blocked contacts.

Sonar and drills have separate hardness limits. Better sonar can show you Titanium while your starter drill still cannot mine it. Check both before sending a vehicle on a long trip.

Hardness limit	Vehicle drill	Stationary Mining Drill
1	Basic Drill Module	Mk I
3	Industrial Drill	Mk II
4	Heavy Drill	Mk III

Each tier handles every hardness up to and including its limit. Hardness 1 covers Iron and Silicon; hardness 3 also covers Titanium, Cobalt, Lead and Rare Earth; hardness 4 adds Neutronium. The Rover takes the basic drill. The advanced vehicle drills need a Pioneer's universal slot.

TAKE A VEHICLE, OR LEAVE A DRILL THERE?

A Rover or Pioneer mines into its own cargo. Mount the right Drill Module, drive to the surveyed deposit, brake, and call `self.drill.mine()`. Each successful dig adds one unit. You need enough charge to finish the dig and cargo space for that mineral. This is a useful way to bring home your first batches without building field infrastructure.

A stationary Mining Drill keeps extracting while powered. Fabricate its kit after earning the recipe, place a blueprint on the surveyed site, and have a Constructor-equipped Pioneer bring the kit and build it. Connect it to a working power network. Ore collects in the drill's own stockpile, so the vehicle that built it can leave.

The stationary drill needs no repeated dig command. Its script can inspect output, but collection is the carrier's job. See `plan_mode` for construction and `power_networks` for supplying field machines.

PURITY CHANGES THE TRIP

A rich site mines **2×** as fast as a standard site; a pure site mines **3×** as fast. Higher vehicle drill tiers also finish sooner, but draw more power while digging. A faster drill can therefore use more energy per unit. The worked time and energy comparison is in `batteries`.

For a vehicle, budget for **the outward trip, the ore you want to dig, and the return trip**. More cargo space does not help if the battery runs out before you get home. On a Pioneer, bins latch to one material each, so check space for the particular ore you are collecting, not just total empty capacity.

BRING THE ORE HOME

A field drill's output is **pickup-only**. It does not send ore through a pipe or directly to Inventory. When the stockpile fills, extraction pauses until a carrier makes room.

- A Rover or Pioneer parks at the drill, connects its own input to the drill and takes ore. Automated item transfers need Auto Feeders research.
- A drone completes `go_to_drill(...)`, loads through `cargo.load(...)`, and carries the ore back to a Drone Depot.
- At the destination, unload into home Inventory or compatible local storage. Remote factories need the ore delivered to their own outpost.

Start with one deposit, one carrier and one destination. Make that route work before adding more drills. See `remote_logistics` for the delivery rules and `drone` for aerial carriers.

WHEN MINING STOPS

Check the full route: surveyed deposit, compatible drill, power or vehicle charge, free cargo or stockpile space, then destination capacity. A stationary drill that reports zero extraction may simply be full. A vehicle beside a site may still be moving; brake before mining or loading. If ore reaches storage but your Smelter sits idle, check its recipe and input transfers next. See `refinement` for the first processing loop.

Using Plan Mode

Plan Mode is the visual construction-planning layer on the Planet Map. It does not build instantly. It creates persistent blueprints that a Constructor-equipped Pioneer can execute later.

VISUAL PLANNING FLOW

1. Open the Planet Map and turn on Plan Mode.
2. Choose a category, then choose a tool.
3. Place a blueprint where the map allows it.
4. Read the placement card and blueprint hover card for the required material.
5. Load that material into a Pioneer with a Constructor Module.
6. Send the Pioneer to the blueprint and execute construction from the Pioneer script.

SCRIPT PLANNING

Scripts write to the same queue through `get_component("construction_blueprint")`. Use `plan_structure(kind, x, y)` for Outposts, Thermal Caps, Exotic Gas Caps, Water Pumps, Oil Pumps, Exotic Spring Taps, and Mining Drills; use `plan_pipe`, `plan_power_line`, and `plan_bridge` for utilities. A successful planning call places the ghost immediately without sending a vehicle there. The Pioneer is still required for the physical build.

```
plans = get_component("construction_blueprint")
result = plans.plan_structure("thermal_cap", vent.x, vent.y)
if result.status == "ok":
    print(result.blueprint_ids[0])
```

LOCKED TOOLS

Visual tools and script calls enforce the same research. Outposts have two gates: **Outpost Construction** unlocks the kit, and **Constructor Module** unlocks the Pioneer module that can build the blueprint.

PLANNING HABITS

Keep plans small until the first one works. One blueprint plus the exact material it asks for is easier to debug than a whole remote base. Once you trust the loop, scale up to outposts, pipes, power lines, pumps, and field extractors. Internal outpost machines, including drone facilities, deploy directly from Inventory and never appear as map blueprints.

First Outpost

An Outpost expands your build area to a new part of the planet. Buying an **Outpost Kit** is only the material step; founding it also needs the Pioneer construction workflow.

REQUIREMENTS

- **Outpost Construction** research to buy the Outpost Kit.
- **Constructor Module** research to build Planet Map blueprints.
- A **Pioneer** with a **Nav Module** and a **Constructor Module** mounted.
- A **Battery Holder** with at least one charged **Portable Battery** installed. Without stored energy, the Pioneer cannot drive.
- A **Cargo Rack** with enough Portable Storage Bin space for one **Outpost Kit**.
- One Outpost Kit loaded into Pioneer cargo, not left in Base Inventory.
- An Outpost blueprint placed in Plan Mode or with `construction_blueprint.plan_structure("outpost", x, y)`.

WHAT TO DO

Place the outpost footprint first. Plan Mode gives visual placement feedback; the script API returns an exact planning status. Either path creates the same job marker. Then load the kit, move the Pioneer to the blueprint, brake, and build it from the Pioneer. If the Pioneer cannot leave, inspect its battery capacity and charge. If the kit is in inventory but nothing builds, check that the Constructor Module is mounted and the kit is physically in Pioneer cargo.

WHY THIS MATTERS

A single home base becomes slow later. The outpost step is when terraforming becomes geographic: more build areas, more machines, more local power, and eventually networks that connect distant production.

Once the outpost is built, use `remote_logistics` to plan its first supply route.

Storage and remote outpost logistics

A new outpost gives you somewhere to build. It still needs supplies. Think through a delivery the same way you would a vehicle trip: where are the goods now, who carries them, and where will they be unloaded?

PICK THE RIGHT STORE

Store	What it is for
Base Inventory	Purchases and general stock at Nocturna Base
Storage Bin	A local supply of one item type; empty it to change the material
Warehouse	Local storage for several item types and their property variants
Vehicle cargo	Carrying goods between sites and outposts
Lead Cask	Raw Uranium or Fuel Rods, one radioactive material per cask

You can open Inventory from anywhere, but **its goods are physically at Nocturna Base**. A machine at a remote outpost cannot pull them across the map. Give that outpost a local store and bring supplies by Pioneer or drone.

A connection does not move goods on its own. After **Auto Feeders** research, an input uses `take(...)` and a routed output uses `send(...)`. These transfers take time and can move fewer items than requested if supply or capacity runs short. Read the result's `.moved` value before deciding that a delivery is complete.

YOUR FIRST REMOTE DELIVERY

Suppose you want a remote Smelter to process Iron Ore.

1. Deploy the Smelter and a Warehouse at the founded outpost, and provide power.
2. Park a cargo-equipped Pioneer at home. Connect its input to `"inventory"` and load Iron Ore with `take(...)`.
3. Drive to the destination outpost and brake inside its service area.
4. Connect the Pioneer's output to that outpost's Warehouse and unload with `send(...)`.
5. From the Smelter's own script, connect its input to the Warehouse, select the iron recipe and take ore into the input. Move finished ingots out of its output as they become ready.

Use your actual machine ids or display names in connections. A connection remembers its target; it does not follow the vehicle or switch to a new store when you arrive. Reconnect deliberately for the next leg.

A drone route uses its cargo commands and Drone Depots instead. Either way, the useful pattern is **load, travel, unload, process**. Check the receiving store has room before sending the next load. See

`refinement` for processing and `io_slots` for transfer examples.

CLOSE ENOUGH, AND STOPPED

Stationary machines and ordinary stores exchange goods within the same outpost. A ground vehicle must be parked in the target's service area to load or unload. Vehicle-to-vehicle handoffs need both vehicles stopped within about **2 m**. A connection to a faraway target does not make it reachable.

Route to the destination's service position, wait for arrival, then brake before starting the handoff. A ground vehicle's movement also stops if its controlling script ends or fails. See `vehicle_service` for the complete parking and docking rules.

BUYING A MACHINE IS DIFFERENT FROM FEEDING IT

Deploying a shop kit needs no delivery. Inventory can deploy an ordinary outpost building straight into a founded outpost. You do not need a cargo trip just to commission that building. Its working supplies still follow the local delivery rules above.

Planet Map construction needs a Pioneer and materials. Field Mining Drills, pumps, caps and utilities use blueprints. Their kits or segments must be in the Constructor-equipped Pioneer's cargo. Creating the blueprint does not move the materials or build the machine. See `first_outpost` and `plan_mode`.

Manual Biology has a separate stockroom choice. It uses Inventory at home and one selected local Warehouse at a remote outpost. Changing that selection does not transport anything, and scripted machine ports still need their own connections.

WHEN A TRANSFER WILL NOT GO THROUGH

Check location, parking, research, available items, free space and whether either endpoint is already busy. A bin may have room but be latched to another material; a Warehouse may need a free physical slot for a different property variant. Raw Uranium and Fuel Rods need the shielded route in `nuclear_power`.

If the wrong goods are already inside a machine input, `input.eject(...)` can recover them to a compatible destination. **`input.flush()` destroys the contents**; it is not an unload command. Read the returned message when a transfer is refused, fix that condition, then try again.

Industrial upgrades and deep oil

Research unlocks the means to expand production. Manufacture one pack for each machine you want to upgrade.

FASTER FACTORIES

Industrial Machinery Mk II unlocks at **375,000 Terraform Index**. Smelters and Fabricators upgraded to Mk II process at **2×** original speed with **1.5×** operating power. Industrial Machinery Mk III unlocks at **625,000 Terraform Index**; a Mk III machine processes at **4×** original speed with **3×** operating power. Install Mk II before Mk III. Recipes consume the same ingredients and produce the same batch quantities.

Make packs in a **Fabricator** using their named recipes. The recipe browser shows the exact ingredients, Water intake and operating power. Mk II packs use Machine Frames, Control Units and either Circuit Panels (the Fabricator pack) or Pressure Valves (the Smelter and Oil Pump packs). The pump pack also needs Cobalt Ingots. Mk III packs add Rare Earth Cores. Completing the corresponding Earth Orders unlocks those intermediate recipes.

Packs are manufactured goods and cannot be purchased. Apply factory packs through **Inventory -> Upgrade** or `computer.upgrade(item_id, machine)` after Ship Computer research. Check power headroom first. Upgrading preserves the machine, its recipe, connections, scripts, stored goods and batch progress. Undeploying returns the base kit and installed packs.

BETTER OIL PUMPS

Advanced Oil Extraction unlocks at **475,000 Terraform Index**. A Mk II Oil Pump extracts **1.5×** its well's active rate and uses **100 W** at full throttle. Dormant phases still stop production. Stockpile active-phase oil in Liquid Tanks to keep factories supplied between phases.

Fabricate an **Oil Pump Mk II Upgrade Pack**, transfer it into a **Pioneer cargo bin**, and mount a **Constructor Module**. Select **Plan Upgrade** on the pump's Extraction card, or call `construction_blueprint.plan_upgrade("oil_pump_upgrade_pack_mk2", pump)` from a script. Park the Pioneer within the pump's service area and run `self.constructor.execute(blueprint_id)`. Installation takes **0.5 h**. The pump keeps working at Mk I until completion. Paused jobs retain their paid pack and progress; resume the same blueprint. Cancelling a paid job requires a Pioneer with cargo room at the pump to reclaim the pack. Dismantling requires room for both the original pump kit and installed pack.

FIND THREE MORE RESERVOIRS

Advanced Oil Extraction also unlocks **Seismic Sonar** in the Shop for **7,500 credits**. Mount it in a Pioneer universal slot, replacing its previous sonar. It has Deep Sonar's **280 m** range and hardness-4 mineral capability, with a **3 Wh** charge per scan or survey. Ordinary Petroleum Survey research remains necessary for oil discovery.

Look for **inert formations**, returned by sonar as `GeologicalAnomaly` sites with `site.kind() == "inert"`. Three of these formations on each map conceal additional deep oil reservoirs. Their positions stay fixed across save/load. Any sonar can classify a formation as inert, but seeing it on the map or scanning it with ordinary sonar does not reveal its deep oil potential. **Revisit inert formations even if they are already marked as surveyed.** That earlier survey does not rule out deep oil.

Mount **Seismic Sonar**, move within **280 m** of an inert formation and call `self.sonar.scan()`. The scan distinguishes **potential deep oil** (`site.seismic_status == "potential"`) from **dry** formations (`"dry"`). Call `self.sonar.survey(site)` on a potential contact to confirm the reservoir. A deep geological survey takes **0.5 h** and turns a confirmed reservoir into a rich **16 t/h** Oil Well at the same location; a Mk II pump extracts **24 t/h** while it is active. A dry survey consumes its first charge and time; revisiting a known dry formation is free. Journal queries thereafter return the confirmed contact as an Oil Well. Survey it, plan an Oil Pump and connect its output to your network.

SCRIPT EXAMPLES

Choose this recipe from a Fabricator script after Mk II research:

```
result = self.set_recipe("craft_fabricator_pack_mk2")
print(result.status, result.message)
```

Apply a fabricated factory pack from Base Inventory. Replace the example machine id with yours:

```
computer = get_component("computer")
result = computer.upgrade("fabricator_upgrade_pack_mk2", "fabricator_1")
print(result.status, result.message)
```

From a Pioneer fitted with Seismic Sonar, scan within range of inert formations, including previously surveyed ones, and survey the potential contacts:

```
sweep = self.sonar.scan()
print(sweep.status)
for site in sweep.sites:
  if site.kind() == "inert" and site.seismic_status == "potential":
    result = self.sonar.survey(site)
    print(result.status, result.message)
```

Park a Constructor-equipped Pioneer at your pump with its pack in cargo, then plan and execute its installation:

```
planner = get_component("construction_blueprint")
plan = planner.plan_upgrade("oil_pump_upgrade_pack_mk2", "oil_pump_1")
if plan.status == "ok":
  result = self.constructor.execute(plan.blueprint_ids[0])
  print(result.status, result.message)
else:
  print(plan.message)
```

An upgraded pump uses the same control script. Connect a completed liquid route to the named tank, then stop drawing power during dormancy or when the tank is nearly full:

```
tank = get_component("liquid_tank_1")
result = self.oil_out.connect("liquid_tank_1")
print(result.status, result.message)
while True:
  target = 1.0 if self.well_active() and tank.fill_pct() < 0.9 else 0.0
  result = self.set_throttle(target)
  if result.status != "ok":
    print(result.message)
  sleep(1)
```

Upgraded factories need enough ingredients, liquid flow, cargo transport and output space to sustain their faster work. Spread exploration and upgrades across the supply chain; a blocked output or empty input still stops production.

Nuclear power: uranium to electricity

A Reactor can supply up to **5,000 W**, but it needs a working fuel chain and steady cooling. Build that chain before relying on it to keep the rest of the base running.

WHAT YOU ARE BUILDING

The route is: **dust-storm aftermath, drone collection, Raw Uranium Lead Cask, Fuel Assembler, Fuel Rod Lead Cask, Reactor**. Ordinary storage supplies Lead Plates to the assembler, and a water connection cools the Reactor.

Research opens the machinery; Earth Orders provide important recipes. Nuclear Program unlocks the Reactor, while Fuel Assembler research opens the assembler. Check the order rewards for Lead Casks, Shield Plating and Fuel Rods. The Fuel Rod recipe comes from the Vestibule order that requests **12 Raw Uranium**. Owning the assembler does not grant every recipe it can make.

COLLECT URANIUM AND BRING IT BACK

Dust storms can leave Raw Uranium after they end. Decode the weather message to recover the coordinate, then send a drone there before the aftermath expires. Sonar does not find this supply. See `weather_overview` for receiving weather signals and recovering the location.

One successful `collect()` takes at most **5 units**. Without Shield Plating, each uranium collection adds **40 exposure**; at **100**, the drone scrambles. The collected cargo stays aboard. Shield Plating prevents that collection exposure, but halves Cargo Pod capacity and increases fuel burn **1.5×**. Plan room and range for the shielded trip, not the empty drone's usual trip.

Drone Service Stations handle decontamination and rescue. Do not wait for the third unshielded collection to discover that the drone needs help. The `drone` guide covers service and recovery.

KEEP RADIOACTIVE CARGO IN LEAD CASKS

Raw Uranium and Fuel Rods are radioactive cargo. Ordinary Inventory, Storage Bins, Warehouses and Depot stockpiles are not their storage route. Use a **Lead Cask** and the compatible nuclear-machine ports. Drones move radioactive cargo directly through the outpost's Lead Cask, rather than stocking it in a Depot.

Use **two casks** near the assembler: one for incoming uranium and one for finished rods. A cask holds **100 units** and latches to one radioactive material at a time, so a cask still holding uranium cannot also receive rods. Lead Plates belong in ordinary local storage.

MAKE THE FIRST RODS

One Fuel Rod needs **4 Raw Uranium + 2 Lead Plates**, takes **6 h**, and draws **1,800 W** while crafting. Have solar, steam, oil or stored energy ready to support that first batch. A Reactor waiting for its first rod cannot power the assembler making it.

The assembler's input has one source connection at a time. Take uranium from its cask, reconnect to the local Lead Plate supply and take the plates. Select `"craft_fuel_rod"`, and keep its output clear by sending finished rods to the second cask. Transfers need Auto Feeders research. A full output or missing ingredient can stall the chain even when power is available.

The assembler can also make Nuclear Batteries after their recipe unlocks. Those are ordinary power-storage products, not Reactor fuel, and can go to compatible ordinary storage.

START THE REACTOR WITH BOTH SUPPLIES READY

Connect the Reactor's input to the Fuel Rod cask and use `take(...)` to keep rods in its input buffer. It automatically starts the next buffered rod when the active one runs out. That does not pull replacements from the cask for you.

Connect `water_in` to a reachable water provider or Liquid Tank. The cooling loop consumes **0.5-1 t/h** while heating. Missing water pauses heating and fuel use until the supply returns. See `flow_networks` for local connections, remote pipes and explicit tank buffering.

`self.set_heat(...)` accepts **0-1**, but temperature responds gradually:

Core temperature	What happens
Below 300 °C	No power output
300-600 °C	Output rises to 50%
600-900 °C	Output rises to 100%
900-950 °C	Output falls as the core gets hotter
950 °C	Shutdown and automatic cooling; recovery at 600 °C

More heat is not always more power. Read `self.temperature()` and `self.power_output()` as you adjust. The core's internal condition changes every **12 h**, so a heat setting that worked earlier may need correction. Let the temperature respond between adjustments. The Reactor's Info tab explains how to build a controller that adapts to these changes and to the world's pacing.

If output disappears, read `self.status()` first: an empty fuel buffer, lost cooling and overheating need different fixes. Keep some backup power and a reserve of rods and water while you get the loop working.

Expanding Production

After the first outpost, progression depends on both expansion and upgrades. Mk II machines improve output, but they do not replace geography. New outposts let you multiply production by building more machines in more places.

EXPANSION PATTERN

- Add an outpost when the home footprint is crowded or travel is wasting time.
- Bring power first, then the terraforming machines for that site.
- Keep each site understandable: a few machines, clear storage, clear script responsibility.
- Upgrade machines when the research lands, but expect more outposts to carry the next phase in a reasonable time.

WHAT TO WATCH

With one outpost, later phases usually run slowly. Add outposts when travel time, footprint space, or local power becomes the bottleneck. If progress jumps too far from one machine-tier swap alone, that is economy tuning feedback worth reporting.

Variables

Variables store values that you can use later.

```
x = 10
name = "oxygen"
active = True
```

You can use variables in expressions:

```
a = 5
b = 3
result = a + b
print(result)
```

Output: 8

Operators

Operators combine values, compare values, and build conditions.

ASSIGNMENT VS EQUALITY

Use `=` to store a value. Use `==` to compare two values. Scripts do not use `===`.

```
target = 10
if value == target:
  print("match")
```

`=` changes a variable. `==` asks a question and returns `True` or `False`.

COMPARISONS

```
a == b  # equal
a != b  # not equal
a < b  # less than
a <= b  # less than or equal
a > b  # greater than
a >= b  # greater than or equal
```

BOOLEAN LOGIC

Use `and`, `or`, and `not` to combine conditions:

```
if powered and battery > 100:
  print("ready")
if not blocked:
  print("clear")
```

MATH AND DIVISION

```
a + b  # add
a - b  # subtract
a * b  # multiply
a / b  # divide
a // b  # floor division
a % b  # remainder, also called modulo
a ** b  # power
```

Use `/` when you want normal division. Use `//` when you want the whole-number quotient. Use `%` when you want the remainder after division.

Modulo is useful for repeating patterns and even/odd checks:

```
value % 2 == 0 # even
value % 2 == 1 # odd
```

MEMBERSHIP AND `NONE`

Use `in` to check whether a value is inside a `list`, `tuple`, `set` or string, or among a `dict`'s keys. Use `is None` for the special empty value.

```
if target in scanned:
  print("known")
if result is None:
  print("nothing found")
```

Numbers

OVERVIEW

Numbers are one family, there is no separate stored `int` and `float` type. `int`, `float`, and `number` are three **views of the same value**, decided by the value itself.

`INT`, `FLOAT`, `NUMBER`

- `int`, a **whole** numeric value (no decimal part)
- `float`, a value **with a decimal part**
- `number`, **either** of the above

The label follows the current value, not how you wrote it:

```
type(5)    # int
type(0.6)   # float
type(4.0)   # int  (whole value)
type(10 / 2) # int  (10 / 2 is 5.0)
type(10 / 3) # float
```

> This is **not** Python's model. In Python `type(4.0)` is `float` because the literal was written as a float. Here it is `int` because the value is whole, the number itself is identical, only the label differs.

CHECKING A NUMBER

```
isinstance(x, int)     # True if x is whole
isinstance(x, float)    # True if x has a decimal part
isinstance(x, int | float) # True for either numeric category
isinstance(x, "number")   # True for any number
x % 1 == 0         # True if x is whole
```

Use `isinstance(x, "number")` when you only care that a value is numeric at all, and `int(x)` to chop a value to a whole number (`int(3.7)` is `3`).

NUMBERS IN PARAMETER TYPES

DOCS, hover and signature help write each parameter's type the way Python does, and there `int` and `float` say what a parameter accepts rather than which view a value has:

- `count: int` takes a whole value however you wrote it (`2.0` is fine) and refuses one with a decimal part (`2.5` raises)
- `x: float` takes any number, whole or not

This is Python's own convention: wherever a `float` is expected, an `int` is accepted too.

PRECISION

Decimal numbers use standard floating-point arithmetic, so tiny rounding can show up:

```
0.1 + 0.2  # 0.30000000000000004, not exactly 0.3
```

This is normal for calculated decimal values. Use `isclose()` when you mean close enough rather than bit-for-bit equality:

```
if isclose(0.1 + 0.2, 0.3):
  print("equal for this calculation")
```

The default `rel_tol` handles ordinary rounding relative to the size of the values. Use `abs_tol` when your accepted difference has a real gameplay meaning. For example, this treats a Rover within half a meter on both axes as being at the target:

```
target_x = 100
target_y = 40
pos = self.nav.get_position()

if isclose(pos.x, target_x, abs_tol=0.5) and isclose(pos.y, target_y, abs_tol=0.5):
  print("at target")
```

Choose the tolerance from what the value represents, such as meters, Wh, or a **0-1** level. Knowing only that a value is floating-point does not determine the right gameplay tolerance.

Print

Use `print()` to output normal text to the console.

```
print("Hello")
print(42)
```

You can print multiple values separated by spaces:

```
temp = -63
print("Temperature:", temp, "°C")
```

Use `warn()` for monitor messages you want to stand out in the persistent console without showing a toast:

```
if self.efficiency() < 100:
  warn("oxygen generator below 100% efficiency")
```

Use `debug()` for noisy telemetry you only want while tuning a script. Debug lines are hidden from ALL unless you enable debug output from the console options menu:

```
debug("target", target_sector, "heat", self.get_heat())
```

The console can show THIS SCRIPT, ALL output, WARNINGS, or ERRORS.

Self & Components

Most scripts in Code: Terraform run **inside a machine**. A Scanner script runs inside the Scanner; a Bio Lab script runs inside the Lab. Inside that script, `self` means **this machine**.

```
# Scanner script
scan = self.scan("E14")
```

```
# Bio Lab script
analysis = self.analyze()
if analysis.status == "ok":
  info = analysis.info
```

HARDWARE ACTIONS ARE LOCAL

Actions that change physical state or spend game time are usually SELF ONLY. Store their result, branch on `.status`, and read `.message`; specialized results add payload fields.

```
move = self.move("E14")
if move.status == "ok":
  pickup = self.collect()
```

READING ANOTHER COMPONENT

Use `get_component(id)` for read-only coordination:

```
scanner = get_component("scanner_1")
scanned = scanner.get_scanned()
```

Sometimes a local action accepts another component reference:

```
collector = get_component("bio_collector_1")
taken = self.take_from(collector)
if taken.status != "ok":
  print(taken.message)
```

The action still belongs to the Bio Lab's `self`; it is not remote control of the Collector. Use shared state or Signal Bus messages to coordinate separate machine scripts.

Stable ids such as `"bio_collector_1"` are safest for long-running scripts. Display names are convenient but can change.

If / Elif / Else

Branching is how a script reacts to what it reads. A chain starts with `if`, adds any number of `elif` branches, and may end with `else`.

```
temp = get_component("thermometer").get_value()

if temp > 20:
  print("Warm")
elif temp > 0:
  print("Above freezing")
elif temp > -40:
  print("Cold")
else:
  print("Dangerously cold")
```

The first branch whose condition is true runs, and every branch after it is skipped. That is why the order matters: write the most specific condition first. Putting `temp > -40` first would print Cold for every temperature above -40, including temperatures that should print Warm or Above freezing.

`elif` is not the same as a second `if`. A chain runs at most one branch. With `else`, exactly one branch runs; separate `if` statements each get tested, so more than one can run.

```
# At most one of these runs.
if level > 80:
  self.set_throttle(0)
elif level < 20:
  self.set_throttle(1)

# Both of these can run.
if level > 80:
  print("high")
if level > 50:
  print("over half")
```

`else` is optional. Leave it off when there is nothing to do in the remaining case.

WHAT COUNTS AS TRUE

A condition is any expression. Comparisons and `and` / `or` / `not` are covered on the **Operators** page. A bare value works too, and these are the ones that count as false:

- `False` and `None`

- the number `0`
- an empty string, `list`, `tuple`, `dict`, or `set`

Everything else is true, so `if result.sites:` reads as "if the scan found any sites".

```
result = self.sonar.scan()
if result.sites:
  print("found", len(result.sites))
else:
  print("nothing in range")
```

Use `is None` rather than `== None`, and be careful with a value that can legitimately be `0`: `if count:` treats a real count of zero as false, while `if count is not None:` does not.

BRANCHING ON A RESULT

Commands return a result object with a `.status` field. Do not test the result itself, because the object is true even when its status reports a rejection. Compare the status instead, and let the chain name each outcome:

```
result = self.input.take("iron_ore", 10)

if result.status == "ok":
  print("took", result.moved)
elif result.status == "partial":
  print("only got", result.moved)
else:
  print("failed:", result.message)
```

Every status a command can return is listed in its DOCS entry, so the chain can be written before the script is ever run.

CHOOSING A VALUE INLINE

When both branches only pick a value, the inline form is shorter than four lines:

```
mode = "day" if sun > 0 else "night"
```

WHEN TO USE MATCH / CASE INSTEAD

A long chain that tests the same value over and over is what `match` is for. Reach for it when branching on the shape of a value or on many fixed alternatives, and see the **Match / Case** page. Keep `if` / `elif` /

`else` for ranges, combined conditions, and anything testing more than one value.

Loops & Scripts

Scripts run once by default. To keep a script running continuously, use a `while` loop:

```
while True:
  # this runs every game tick
  print("running")
```

The interpreter automatically gives time back to the game while loops run. World systems and other scripts keep moving, so you do not need `sleep()` to make a loop safe.

Use `sleep()` only when you intentionally want game time to pass before the script continues:

```
while True:
  print("every 5 seconds")
  sleep(5)
```

`for` loops iterate over a list:

```
for i in range(5):
  print(i)
```

See **Long-Running Scripts** for designing loops that re-read machine state and remain safe when a running script starts again after a game load.

Match / Case

`match` lets a script branch on the shape of a value instead of writing a long chain of `if` checks.

```
packet = ["ore", 12]

match packet:
  case ["ore", amount] if amount > 0:
    print("ore", amount)
  case ["ice", amount]:
    print("ice", amount)
  case _:
    print("unknown packet")
```

PATTERNS

- Literal patterns: `case "ok":`, `case 0:`, `case True:`, `case None:`
- Wildcard: `case _:`
- Captures: `case amount:`
- Alternatives: `case "busy" | "cooling":`
- Guards: `case [kind, amount] if amount > 0:`
- List/tuple patterns: `case [x, y]:`, `case [head, *rest]:`
- Dict patterns: `case {"status": "ok", "value": value}:`

Dict-style patterns also match result objects by field name:

```
result = self.input.take("iron_ore", 10)
match result:
  case {"status": "ok", "moved": moved}:
    print("moved", moved)
  case {"status": reason, "message": message}:
    warn(reason, message)
```

Class patterns check a value's type and read its attributes:

```
match shape:
  case Circle(radius=r):
    print("circle", r)
  case Point(0, y):
    print("on the y axis at", y)
  case int(n) | float(n):
    print("a number", n)
```

Keyword sub-patterns name attributes. Positional ones follow the class's `__match_args__`, which `@dataclass` sets to its fields in order. For built-in types such as `int` and `str`, one positional sub-pattern captures the whole value.

Conversion Functions

Convert between types using built-in functions.

```
str(42)    # "42"
int("7")    # 7
float("3.14") # 3.14
```

Character conversion, useful for building sector IDs (`"A1"`, `"B2"`, etc.):

```
chr(65)  # "A"
chr(66)  # "B"
ord("A")  # 65
```

Example, generate sector names:

```
row = 0
while row < 8:
  letter = chr(65 + row)
  sector = letter + str(1)
  print(sector) # A1, B1, C1...
  row = row + 1
```

Strings & F-strings

F-strings let you embed expressions directly in strings:

```
heat = 45
print(f"Heat: {heat}/100")
name = "Crystal Shard"
print(f"Found {name} worth {300} credits")
```

Slicing extracts part of a string or list:

```
pos = "E13"
letter = pos[0]    # "E"
column = pos[1:]   # "13"
first3 = pos[:3]   # "E13"
```

Use `in` to check membership:

```
if "A1" in scanned_list:
  print("already scanned")

if sector not in visited:
  print("new sector")
```

`in` works with lists, dictionaries (checks keys), and strings (checks substring).

STRING METHODS

- `.upper()` / `.lower()`, case conversion
- `.strip()` / `.lstrip()` / `.rstrip()`, remove whitespace
- `.split(sep)`, split into list
- `.join(list)`, join list into string: `", ".join(["a", "b"])` → `"a, b"`
- `.find(sub)`, index of substring, `-1` if not found
- `.index(sub)`, like `.find()` but raises error if not found
- `.replace(old, new)`, replace all occurrences
- `.startswith(s)` / `.endswith(s)`, check prefix/suffix
- `.count(sub)`, count occurrences
- `.title()` / `.capitalize()`, title case / capitalize first

- `.isdigit()` / `.isalpha()` / `.isalnum()` / `.isspace()`, character checks
- `.zfill(width)`, pad with zeros
- `.center(w)` / `.ljust(w)` / `.rjust(w)`, alignment
- `.format(args)`, `"Hello {}".format("world")`
- `%` formatting, `"%d items at %.1f%%" % (count, share)`

Regular Expressions

Use the built-in `re` module when plain string methods are not enough: matching a pattern, extracting groups, replacing variable-shaped text, or splitting on several separators.

`re` is a built-in module. It works without Shared Library research, but you import it so your script clearly signals that it is using regular expressions:

```
import re

text = "ore_42 at E13"
match = re.search("ore_(\d+)", text)
if match:
  print(match.group(1))  # 42
```

MATCH HELPERS

- `re.search(pattern, string, flags=0)`, find the first match anywhere
- `re.match(pattern, string, flags=0)`, match only at the start
- `re.fullmatch(pattern, string, flags=0)`, match the whole string

These return a `Match` object, or `None` when there is no match.

```
import re

m = re.fullmatch("([A-Z])(\d+)", "E13")
if m is not None:
  print(m.group(0))  # E13
  print(m.group(1))  # E
  print(m.group(2))  # 13
  print(m.span())   # (0, 3)
```

`Match.group(0)` is the whole match. Capturing groups start at `1`. `Match.groups()` returns all captured groups as a `tuple`.

LISTS AND REPLACEMENTS

```
import re

print(re.findall("\d+", "A12 B7"))
print(re.split("[,;]\s*", "iron, ice; quartz"))
print(re.sub("ore_(\d+)", "ore-\\1", "ore_42"))
```

`re.findall()` returns strings when the pattern has no groups, one captured value when it has one group, or `tuple` values when it has multiple groups. `re.sub()` replacement text supports numeric backreferences such as `\\1` and `\\g<1>`.

FLAGS

Flags are module constants. Combine them with `|`:

```
import re

flags = re.IGNORECASE | re.MULTILINE
print(re.findall("^ore", "Ore\nice", flags))
```

Available flags:

- `re.IGNORECASE` / `re.I`
- `re.MULTILINE` / `re.M`
- `re.DOTALL` / `re.S`

SYNTAX NOTE

Code: Terraform exposes a Python-shaped `re` API backed by a linear-time pattern engine. Common patterns like `\d+`, `[A-Z]+`, `.*`, `^`, `$`, groups `(...)`, and alternation `a|b` are supported. Backtracking-only features such as lookaround and pattern backreferences are not supported. Replacement backreferences in `re.sub()` remain supported.

Patterns are capped at **512** characters. Quantified groups such as `(a+)+` are safe to use because matching does not backtrack.

Lists & Tuples

Lists are ordered collections. Create with brackets. Tuples are fixed ordered collections; create them with parentheses when you want a stable pair, coordinate, or small record:

```
items = [1, 2, 3]
empty = []
mixed = ["hello", 42, True]
point = (12, 8)
single = (42,)
```

Access by index (0-based, negative from end):

```
first = items[0]  # 1
last = items[-1]  # 3
x = point[0]    # 12
```

Slicing with step:

```
items[1:3]   # [2, 3]
items[::2]   # [1, 3], every other
items[::-1]  # [3, 2, 1], reversed
point[:]    # (12, 8)
```

List comprehension, build lists concisely:

```
squares = [x * x for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

Tuple unpacking and multiple assignment:

```
a, b, c = [1, 2, 3]
first, second = ("hello", "world")


def choose_route():
  return "E14", ["E13", "E14"]

target, route = choose_route()

routes = [(0, "nav_module"), (1, "cargo_rack")]
for slot, item in routes:
  print(slot, item)
```

LIST METHODS

- `.append(value)`, add to end
- `.pop()`, remove and return last item
- `.insert(index, value)`, insert at position
- `.remove(value)`, remove first occurrence
- `.index(value)`, find position of value
- `.count(value)`, count occurrences
- `.sort()`, sort in place
- `.reverse()`, reverse in place
- `.copy()`, shallow copy
- `.extend(list_or_tuple)`, add all items from another sequence
- `.clear()`, remove all items
- `.length`, number of items

Tuples are immutable. They support indexing, slicing, `.index(value)`, `.count(value)`, `.length`, `len()`, and `for` loops.

Dictionaries

Dictionaries store key-value pairs. Keys can be any hashable value: strings, numbers, booleans, `None`, or tuples made only of hashable values. String keys are the most common for game data:

```
data = {"name": "Crystal", "value": 300}
empty = dict()
```

Access and modify:

```
print(data["name"])    # Crystal
data["quality"] = "high"
coords = {}
coords[(0, 0)] = "base"
```

Check if a key exists:

```
if "name" in data:
  print(data["name"])
```

Iterate:

```
for key in data.keys():
  print(key, data[key])

for pair in data.items():
  key = pair[0]
  val = pair[1]
  print(f"{key}: {val}")
```

MERGE

Use `left | right` to make a new merged dictionary. Use `left |= right` to update the existing dictionary in place. When the same key appears in both, the right-hand value wins.

```
thing = {"a": 1, "b": 2}
other = {"a": 3, "c": 4}
print(thing | other) # {"a": 3, "b": 2, "c": 4}
```

METHODS

- `.keys()`, list of all keys
- `.values()`, list of all values
- `.items()`, list of [key, value] pairs
- `.get(key, default)`, get value or default if missing
- `.has(key)`, check if key exists
- `.pop(key, default)`, remove key and return value
- `.popitem()`, remove and return the last inserted [key, value] pair
- `.update(dict)`, merge another dict in
- `.setdefault(key, default)`, get or set default
- `.clear()`, remove all entries

Built-in Functions Overview

These functions are always available.

OUTPUT

- `print(values...)`, print normal text to console
- `warn(values...)`, print an amber warning line to the console's WARNINGS view
- `debug(values...)`, print low-priority telemetry, hidden from ALL unless debug output is enabled
- `notify(text, level?, duration?)`, show a toast and archive it in Notifications

TYPES

- `type(value)`, returns value-based categories such as `int` for whole numbers, `float` for fractional numbers, `str`, and `bool`. Numeric categories use the current value, so `type(4.0)` is `int`.
- `isinstance(value, type)`, check type/category: `isinstance(name, str)`, `isinstance(n, int)`, `isinstance(n, int | float)`, `isinstance(x, (int, str))`, or broad `isinstance(n, "number")`
- `callable(value)`, `True` if value is a function, method, class, or object with a callable `__call__`
- `len(value)`, length of string, `list`, `tuple`, `dict`, or `set`
- `bool(value)`, convert to `True` / `False`

CONVERSION

- `str(value)`, `int(value, base?)`, `float(value)`
- `list(value)`, `tuple(value)`, `set(value)`, `dict(...)`, type constructors
- `chr(code)`, `ord(char)`
- `hex(n)`, `bin(n)`, `oct(n)`, integer to base-prefixed string
- `repr(value)`, debug-friendly string representation
- `format(value, spec)`, format one value the way an f-string field does, such as `format(3.14159, ".2f")`

MATH

- `abs(n)`, `round(n, digits?)`, `pow(base, exp)`

- `isclose(a, b, rel_tol=0.000000001, abs_tol=0.0)`, compare floating-point results with an accepted difference
- `min(values...)` / `max(values...)`, args, a `list`, or a `tuple`
- `sum(sequence, start=0)` / `prod(sequence, start=1)`, add or multiply numeric items
- `random()` / `rand()`, random `float` from 0 up to but not including 1
- `randint(min, max)`, random integer with inclusive bounds
- `import random`, module form with `random.random()`, `random.rand()`, and `random.randint(min, max)`
- `floor(n)`, `ceil(n)`, `trunc(n)`, `sign(n)`, `divmod(a, b)`
- `sqrt(n)`, `exp(n)`, `log(n)`, `log2(n)`, `log10(n)`
- `sin(n)`, `cos(n)`, `tan(n)`, `asin`, `acos`, `atan`, `atan2(y, x)`
- `degrees(rad)`, `radians(deg)`
- `inf`, positive infinity; use `-inf` for negative infinity
- `pi`, `tau`, constants

SEQUENCES

- `range(stop)` / `range(start, stop, step)`
- `sorted(sequence, key=fn, reverse=False)`, new sorted list. Pass `key=lambda x: x.value` to sort by a computed field; `reverse=True` for descending.
- `reversed(sequence)`, new reversed list
- `enumerate(sequence, start=0)`, list of `(index, value)` pairs; over a generator it stays lazy, as do `zip`, `map`, and `filter`
- `zip(sequence1, sequence2, ..., strict=False)`, combine corresponding elements into tuples with one element from each sequence; `strict=True` raises if lengths differ
- `map(fn, sequence)` / `filter(fn, sequence)` / `reduce(fn, sequence, initializer?)`, transform, select, or fold items
- `pairwise(sequence)`, neighboring `(a, b)` pairs
- `batched(sequence, size)`, group elements into tuples of up to the requested size; the last tuple can be shorter
- `starmap(fn, sequence)`, call `fn` with tuple/list items unpacked as arguments
- `flatten(sequence)`, flatten one nested level
- `count_by(sequence, key_fn?)`, count values or computed keys into a `dict`
- `iter(sequence)` / `next(iterator)`, manual iteration
- `chain(seq1, seq2, ...)`, concatenate sequences
- `accumulate(sequence)`, running totals

- `combinations(sequence, r)` / `permutations(sequence, r)`, combinatorics
- `product(seq1, seq2, ...)`, cartesian product

LOGIC

- `all(sequence)`, `True` if every item is truthy
- `any(sequence)`, `True` if any item is truthy

MODULES

- `import random`, random-number helpers: `choice`, `choices`, `shuffle`, `sample`, `randrange`, `uniform`, `gauss`, and `seed`
- `import math`, Python's math functions and constants: `gcd`, `lcm`, `factorial`, `comb`, `hypot`, `dist`, `isnan`, `e`, and the rest
- `import itertools`, lazy iterators: `count`, `cycle`, `repeat`, `islice`, `groupby`, `zip_longest`, `tee`, `takewhile`, `dropwhile`, and the rest of Python's module
- `import operator`, operators as functions: `add`, `mul`, `itemgetter`, `attrgetter`, `methodcaller`, and the rest
- `import string`, text constants: `ascii_letters`, `ascii_uppercase`, `digits`, `punctuation`, `whitespace`, plus `capwords`
- `from collections import Counter, defaultdict, OrderedDict, deque, namedtuple`, counting, dictionaries with a default, ordered dictionaries, double-ended queues and tuples with named fields
- `from functools import reduce`, reducer helper as a module import, plus `partial` for pre-bound callables, `lru_cache` / `cache` for memoization and `cmp_to_key` to sort with a compare function
- `import re`, regular expressions: `compile`, `search`, `match`, `fullmatch`, `findall`, `finditer`, `sub`, `subn`, `split`, and `escape`
- `from dataclasses import dataclass, field`, generated record-class construction and field configuration, plus `asdict`, `astuple`, `fields`, `replace`, `is_dataclass`, and the `MISSING` sentinel
- `import json`, JSON text: `dumps` and `loads`
- `import heapq`, priority queues: `heappush`, `heappop`, `heappushpop`, `heapreplace`, `heapify`, `nsmallest`, and `nlargest`
- `import traceback`, where a caught exception came from: `print_exc` and `format_exc`
- `from enum import Enum, auto`, named sets of constants: `Enum`, `IntEnum`, `StrEnum`, `Flag`, `IntFlag`, `auto`, and the `unique` and `verify` checks

TIMING

- `sleep(seconds)`, intentionally wait before continuing. Loops are paced automatically; use this only when you want game time to pass.

OTHER

- `hash(value)`, hash a string, number, `bool`, `None`, tuple-of-hashables, or class instance (identity by default, `__hash__` when defined)

Exceptions

Exceptions let a helper function stop with a named error when the caller gives it impossible input or an assumption has broken. Use them for bugs and malformed calls. Expected gameplay outcomes such as cargo full, nothing scanned, or queue empty belong in command result objects: branch on `.status` and read `.message`. These objects are normal return values, not exceptions.

RAISE AN ERROR

Call an exception class with a clear message, then `raise` it:

```
def normalize_sector(sector):
  if not isinstance(sector, str):
    raise TypeError(f"normalize_sector: expected str, got {type(sector)}")
  if len(sector) < 2:
    raise ValueError("normalize_sector: expected a sector like A1")
  return sector
```

`type("A1")` returns `str`, not `string`. Prefer `isinstance(value, str)` when you want a true/false type check.

API BOUNDARY

Gameplay APIs follow the same distinction. Wrong argument kinds raise `TypeError`; correctly typed but invalid values raise `ValueError` or another documented exception. Ordinary world states do not raise: a command returns its result object so your script can handle `"busy"`, `"no_cargo_space"`, `"not_found"`, and similar outcomes without guessing.

CATCH AND KEEP RUNNING

Use `try` / `except` when a script can recover and continue. The variable after `as` is the exception object: printing it shows its message, `error.args` holds the arguments it was created with, and `type(error)`, `isinstance(error, ValueError)`, identity checks, and `raise error` retain its exception class and identity.

```
target = None
try:
  target = normalize_sector(command.args["sector"])
except KeyError as error:
  warn("command missing sector", error)
except (TypeError, ValueError) as error:
  warn(error)

if target is not None:
  print("target", target)
```

Catch the narrowest error that makes sense. `except Exception as error:` catches ordinary game-script exceptions derived from `Exception`, which is useful at a boundary but can hide mistakes if used everywhere. It does not catch control-flow exceptions derived directly from `BaseException`, such as `GeneratorExit`. A handler can name one supported exception, a dotted alias such as `errors.ValueError`, or a `tuple` such as `except (KeyError, TypeError):`. Computed expressions in an `except` clause, the part that selects the exception, are not part of the game-script surface. Handler names resolve when an error is caught, so aliases and shadowing behave normally. An `as error` target exists only inside that handler and is cleared on every exit; copy any detail you need later into another variable.

CLEANUP AND RE-RAISING

A `try` block may have `else`, which runs only when the `try` body finishes without an exception, and `finally`, which always runs before control leaves through success, error, `return`, `break`, or `continue`. A new error or control-flow exit from `finally` replaces the pending one, so keep cleanup small and predictable. Bare `raise` inside an active handler or its `finally` block re-raises the same exception identity.

`raise RuntimeError("message") from cause` stores the cause on the new exception's `__cause__` (and `from None` marks the chain as deliberately cut). An exception raised while another is being handled records that one on `__context__`. The game console shows only the raised exception; your handler can walk `__cause__` and `__context__` when the earlier failure matters.

BUILT-IN NAMES

Common choices are `ValueError` for a value with the right type but wrong contents, `TypeError` for the wrong kind of value, `KeyError` for a missing dictionary key, `IndexError` for a bad list index, `AttributeError` for a missing field or method, and `RuntimeError` for a failed assumption while running.

Other supported names include `NameError`, `UnboundLocalError`, `ImportError`, `ModuleNotFoundError`, `ZeroDivisionError`, `StopIteration`, `GeneratorExit`,

`AssertionError`, `NotImplementedError`, `RecursionError`, `IndentationError`, `OverflowError`, and `SyntaxError`. A completed generator raises `StopIteration`, whose `.value` is the generator's `return` value. `GeneratorExit` is used by `generator.close()` and normally belongs inside generator cleanup.

YOUR OWN EXCEPTION CLASSES

Derive a class from `Exception` (or from any built-in exception) when a failure deserves its own name, for example so a library can signal one condition and every caller can catch exactly that:

```
class LowBattery(Exception):
  def __init__(self, level):
    super().__init__(f"battery at {level}%")
    self.level = level

def check(rover):
  level = round(rover.battery.level() * 100)
  if level < 20:
    raise LowBattery(level)

try:
  check(get_component("rover_1"))
except LowBattery as err:
  print(err, err.level)
```

`class Empty(Exception): pass` is enough for a class with no extra data: `Empty("no cargo")` stores the message on `.args` and `str(err)` returns it. A bare `raise Empty` creates the object with no arguments. `except Exception as err:` catches every class derived from `Exception`, including yours, and `isinstance(err, LowBattery)` and `issubclass(LowBattery, Exception)` walk the hierarchy. Override `__str__` when the console text should differ from the message argument. Pick the closest built-in base so generic handlers keep working, and keep the message specific enough that the console tells you where the problem came from.

Utility Helpers

Small helper functions are always available in every script. Use this page when you need quick random numbers, bounds-safe coordinates, math, collection transforms, or console output without hunting through the full Built-in Functions reference.

RANDOM NUMBERS

Use the Python-shaped module form when you want code to read like normal Python:

```
import random as rng

roll = rng.randint(1, 6)
jitter = rng.random()
print("roll", roll, "jitter", jitter)
```

`random.random()` returns a `float` from **0** up to but not including **1**. `random.rand()` is the same helper with a shorter name. `random.randint(min, max)` returns a whole number between `min` and `max`, including both ends.

You can also call the same helpers globally:

```
roll = randint(1, 6)
jitter = rand()
```

If you import the module as plain `import random`, the name `random` refers to the module in that script. That is fine; call `random.random()` for the `float` helper.

RANDOM VALID COORDINATES

Planet bounds are a good partner for `randint`:

```
planet = get_component("nocturna")
b = planet.get_bounds()

x = randint(b.min_x, b.max_x)
y = randint(b.min_y, b.max_y)

if planet.contains(x, y):
  print("valid target", x, y)
```

NUMERIC HELPERS

Common math helpers are built in:

- `min(...)` / `max(...)`, choose the smallest or largest value
- `sum(items, start=0)` / `prod(items, start=1)`, add or multiply numeric values
- `round(n, digits?)`, `floor(n)`, `ceil(n)`, `trunc(n)`, shape numbers
- `abs(n)`, `sign(n)`, `divmod(a, b)`, distance, direction, and quotient/remainder
- `sqrt(n)`, `sin(n)`, `cos(n)`, `atan2(y, x)`, geometry and steering helpers
- `degrees(rad)` / `radians(deg)`, convert angle units

Example:

```
distance = 42.8
whole = ceil(distance)
hours, minutes = divmod(135, 60)
print(whole, hours, minutes)
```

COLLECTION HELPERS

These help with lists, tuples, strings, dictionaries, and sets:

- `len(value)`, length
- `range(...)`, integer sequences for loops
- `sorted(sequence, key=fn, reverse=False)`, sorted copy
- `reversed(sequence)`, reversed copy
- `enumerate(sequence)`, `(index, value)` pairs
- `zip(a, b, ..., strict=False)`, combine sequences; `strict=True` raises if lengths differ
- `map(fn, sequence)` / `filter(fn, sequence)` / `reduce(fn, sequence, initializer?)`, transform, select, or fold values
- `pairwise(sequence)`, neighboring pairs, useful for route segments
- `batched(sequence, size)`, chunks for page-sized work or repeated commands
- `starmap(fn, sequence)`, unpack `tuple` or `list` rows into a function call
- `flatten(sequence)`, one-level flattening for nested route or cargo lists
- `count_by(sequence, key_fn?)`, counts into a `dict`, optionally by a computed key
- `all(sequence)` / `any(sequence)`, boolean checks

Example:

```
items = ["iron_ore", "ice", "quartz"]
for index, item in enumerate(sorted(items)):
  print(index, item)

print(count_by(["ice", "ore", "ice"]))
```

CONSOLE HELPERS

Use `print()` for normal output, `warn()` for persistent warnings, `debug()` for low-priority telemetry, and `notify(text, level?)` for on-screen alerts. Use the Console component when a script intentionally owns a noisy display.

```
warn("storage nearly full")
debug("loop heartbeat")
notify("Rover battery low", "warn")

console = get_component("console")
console.clear("alarms")
```

For the exhaustive list and exact signatures, open **Built-in Functions**.

Imports & Libraries

Imports have several precise kinds: executable built-in modules, typing-only support modules, compiler directives, the explicit shared root, and your own Library scripts. Library scripts unlock with **Shared Library** research; the built-in and support namespaces do not require that research.

EXECUTABLE BUILT-IN MODULES

Use `import random`, then `random.randint(1, 10)`, `import re` for regular expressions, `import json` to turn records into text and back, `import heapq` for priority queues, `import traceback` to find out where an exception you caught came from, `import math` for Python's math functions and constants such as `math.gcd`, `math.hypot` and `math.pi`, `import itertools` for lazy iterators such as `itertools.count`, `itertools.islice` and `itertools.groupby`, `import operator` for operators as functions such as `operator.add` and `operator.itemgetter`, `import string` for text constants such as `string.ascii_uppercase` and `string.digits`, `from collections import Counter, defaultdict, OrderedDict, deque, namedtuple` for counting, dictionaries with a default, ordered dictionaries, double-ended queues and tuples with named fields, `from enum import Enum, IntEnum, StrEnum, Flag, auto` for named sets of constants, `from functools import reduce, partial, lru_cache, cache, cmp_to_key` for reducer-style algorithms, pre-bound callables, memoization and compare-function sorting, or `from dataclasses import dataclass, field` for generated record classes, with `asdict`, `astuple`, `fields`, `replace`, `is_dataclass` and the `MISSING` sentinel alongside them. You can also call `random()`, `rand()`, `randint(min, max)`, `reduce(fn, iterable, initializer?)` and the common math functions such as `sqrt(x)` and `floor(x)` directly as global helpers. Everything else stays on its own module so the code says where it came from: regex helpers on `re`, and `dataclass` and `field` likewise require their standard module import.

TYPING AND EDITOR SUPPORT

`typing`, `types`, `collections.abc`, and `user_stubs` provide names for annotations and editor analysis. Their type names are erased while a script runs; they are not a place for executable helpers. `typing.TYPE_CHECKING` is always `False` in the game. `typing.NamedTuple` is the one that runs: `class Point(NamedTuple):` builds a named tuple class, as `collections.namedtuple` does. `collections.abc` sits inside the executable `collections` module and holds only annotation names. Put executable shared code in a Library instead of `user_stubs.py`.

`from __future__ import annotations` is a compiler directive, not a normal binding. Put future directives at the beginning of a module, after an optional module docstring and before ordinary statements. The directive itself creates no `annotations` name.

`__builtins__` is the explicit import view of the shared interpreter and game root, for example `from __builtins__ import len, get_component`. Script-owner locals such as `self` and `panel` are not part of that shared module. Game type names such as `Smelter`, `Battery`, and `Component` are part of it too: `from __builtins__ import Smelter` binds the `Smelter` class. An annotation needs no import, because annotations never run; a place that does run, such as an `isinstance` test, a `TypedDict` field dictionary or a type alias, does need it. An external editor has those names in scope without the import, but the game still needs it wherever the name runs. `isinstance(machine, Smelter)` is `True` for a smelter, every machine is also a `Component`, and `issubclass(Smelter, Component)` is `True`; `type_id` names a machine's type as a string. The generated `builtins` and `code_terraform` stubs exist only for external-editor type checking and cannot be imported by a running game script.

PLAYER LIBRARIES

After **Ship Computer** and **Shared Library** are researched, open **Ship** → **Computer** → **Library** in the left sidebar and click **+ New**. A Library script is shared code: write a helper once, then import it from any machine script.

Create a Library script called `sensors`:

```
def temp():
  """Read the current thermometer value."""
  return get_component("thermometer").get_value()

def o2():
  """Read the current oxygen sensor value."""
  return get_component("oxygen_sensor").get_value()
```

Import specific functions:

```
from sensors import temp

print(temp())
```

Import all public names from a library:

```
from sensors import *

print(temp())
print(o2())
```

Names starting with `_` stay private and are not imported by `*`. Or import the whole module:

```
import sensors

print(sensors.o2())
```

Library scripts run in their own shared scope. Built-ins and top-level game functions are available, but caller-local names are not. `self` and `panel` are not defined inside a library, so pass a component, component id, or other context into a helper when it needs to act for the importing script.

Function docstrings from imported libraries show in hover and autocomplete, so shared helpers can document their own parameters, return values, exceptions, and fixed result contracts.

Docstrings

Docstrings are string literals placed at the top of a function body. They do not change how the function runs; they document what the function is for.

When you hover a user-defined function or see it in autocomplete, the editor shows its signature, its docstring, and its return annotation if it has one. Imported library functions show their docstrings too. Add a return annotation such as `-> str | None` when you want the tooltip's separate return line.

```
def next_needed(order) -> str | None:
  """Pick the next missing item for an order.

  [[helper]]

  ## Returns

  - a fragment id when one is still required
  - `None` when the order already has everything it needs
  """
  for item in order.requires.keys():
    if order.delivered.get(item, 0) < order.requires[item]:
      return item
  return None
```

FORMATTING

Docstrings use the same markdown-lite formatting as DOCS descriptions:

- Inline code: `status`, `"ok"`, `self.deliver()`
- Bold text: **important**
- Badges: HELPER or SELF ONLY
- Headings: `## Returns`
- Callouts: lines starting with `>`
- Bullets: `- item` or `• item`
- Numbered lists: `1. item`
- Tables: a header row, a `|---|---|` separator row, then one row per line
- Fenced code blocks with triple backticks

The list above is rendered, so the markers are invisible. This is what you type, one example per line:

```
Inline code: `status`, `"ok"`, `self.deliver()`
Bold text: **important**
Badges: [[helper]] or [[self only]]
Heading: ## Returns
Callout: > check the status first
Bullet: - item
Numbered: 1. item
```

A table needs the separator row or it renders as plain text:

```
| Sector | Row | Col |
|---|---|---|
| C4 | 3 | 4 |
| C14 | 3 | 14 |
```

Use blank lines between sections when you want the hover to render them as separate blocks.

VARIANTS

A triple-quoted string at the very top of a script is a script description, not a function docstring. The Variants tab uses the first non-empty line of that top-of-file docstring as the variant description. If there is no top-of-file docstring, the first `#` comment is used instead.

```
"""Night route: charge first, then collect ice."""

while True:
  print("working")
```

For function-level help, put the docstring immediately under `def`. For variant summaries, put it at the top of the file before any code.

Writing Classes

Classes bundle **data** (attributes) with **behavior** (methods) into one reusable shape. An instance is one filled-in copy.

```
class Counter:
  def __init__(self, start):
    self.n = start   # an instance attribute
  def inc(self):
    self.n += 1

c = Counter(10)
c.inc()
print(c.n)       # 11
```

`self`, `__init__`, and attributes

`__init__(self, ...)` runs once when you write `Counter(10)`, it sets up the new instance. Regular instance methods receive the instance as their first parameter, conventionally named `self`, and `self.x = ...` creates an instance attribute. A value assigned in the class body (`limit = 5`) is a **class attribute**, shared by all instances and readable as `Counter.limit` or `c.limit`.

`self` inside a machine script

In a machine's own script the top-level `self` is the machine (`self.mine()`). Inside an instance method, `self` is the **object**, they never collide, because a method's `self` is simply its first parameter.

> If a method needs to drive the machine, pass it in: `Planner(self)` at the top level (where `self` is the machine), store it (`self.bot = bot`), then call `self.bot.mine()` inside the method.

Inheritance and `super()`

`class Dog(Animal):` inherits `Animal`'s methods and attributes. Override any of them, and reach the base version with `super()`:

```
class Animal:
  def __init__(self, name):
    self.name = name
  def speak(self):
    return "..."

class Dog(Animal):
  def __init__(self, name, breed):
    super().__init__(name)  # run Animal's setup first
    self.breed = breed
  def speak(self):
    return self.name + " says woof"

d = Dog("Rex", "husky")
print(d.speak())        # Rex says woof
print(isinstance(d, Animal), issubclass(Dog, Animal)) # True True
```

Multiple inheritance resolves by Python's C3 method-resolution order, so cooperative `super()` works across a diamond, every `__init__` in the chain runs once.

OPERATORS ARE DUNDER METHODS

Operators and builtins are sugar for `__dunder__` method calls. Define them to make your objects act like built-in types:

```
class Vec:
  def __init__(self, x):
    self.x = x
  def __add__(self, o):
    return Vec(self.x + o.x)    # powers a + b
  def __eq__(self, o):
    return self.x == o.x      # powers a == b
  def __repr__(self):
    return "Vec(" + str(self.x) + ")"  # how print() shows it
```

`__lt__` powers `<`, `__len__` powers `len()` and truthiness, `__getitem__` powers `x[k]`, `__contains__` powers `in`, `__iter__` powers `for`, and `__call__` makes an instance callable.

> Operator overloads must be **pure**, they cannot `sleep()` or change game state. Regular methods, `__init__`, and `__call__` can.

`@property` and method decorators

`@property` turns a method into an attribute read **without parens**; add `@name.setter` to allow assignment:

```
class Tank:
  def __init__(self):
    self._level = 0
  @property
  def level(self):
    return self._level
  @level.setter
  def level(self, v):
    self._level = max(0, v)

t = Tank()
t.level = 42      # runs the setter
print(t.level)     # runs the getter
```

`@staticmethod` defines a method with no `self`; `@classmethod` receives the class as `cls`. Any function can be a decorator (`@deco` rewrites to `name = deco(name)`). `from functools import total_ordering` fills in the rest of the comparisons from `__eq__` plus one of `__lt__` / `__le__` / `__gt__` / `__ge__`.

Dataclasses

Use the built-in `dataclasses` module when a class is mainly a declaration of named fields but still needs class behavior. The decorator generates declaration-ordered construction, readable representation, value equality, and optional ordering; `__post_init__` is the place for validation after generated assignment.

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Job:
  priority: int
  label: str = ""
  tags: list[str] = field(default_factory=list, compare=False)

  def __post_init__(self):
    if self.priority < 0:
      raise ValueError("priority must be non-negative")

a = Job(2, "ice run")
b = Job(1, "ore run")
a.tags.append("cold")
print(b < a, a)
```

`default_factory` creates an independent mutable value for every instance. Mutable or otherwise unhashable direct defaults such as lists, dictionaries, and sets are rejected; use `default_factory` for them. `dataclass` supports `init`, `repr`, `eq`, `order`, `kw_only`, and `match_args`. When `match_args=True` (the default), it generates `__match_args__` from fields with `init=True` and `kw_only=False`, unless the class already defines it. `field` supports `default`, `default_factory`, `init`, `repr`, `compare`, and `kw_only`. Inherited annotated fields and explicit field overrides participate in the generated constructor.

A dataclass remains an ordinary user-class instance. It does not become a `dict` and cannot cross JSON-shaped game boundaries such as the Signal Bus or notebook APIs. Use `TypedDict` for mapping-shaped payloads and cached rows; use a dataclass for methods, validation, generated construction, value equality, or ordering.

Convert a dataclass instance with `dataclasses.asdict()` before sending or storing it as a record. `asdict()` produces a `dict` and `astuple()` produces a `tuple`; both recursively convert nested dataclass instances inside lists, tuples, and dictionaries. Those containers are rebuilt; sets receive a shallow copy. Other values stay the same objects rather than being deep-copied. `fields()` accepts a dataclass class or instance and returns its `Field` records in declaration order; `MISSING` marks an absent default or default factory. `is_dataclass()` recognizes dataclass classes and instances. `replace()` creates a new instance with the specified field changes through its constructor, including `__post_init__`.

The compatibility spellings `frozen=False`, `unsafe_hash=False`, `slots=False`, and `weakref_slot=False` may be passed, but their `True` behavior is not supported. `ClassVar`, `InitVar`, `KW_ONLY`, `FrozenInstanceError`, `make_dataclass`, constructing a `Field` directly, and public `__dataclass_fields__` introspection are not supported. Annotation types otherwise stay erased and are not enforced. Canonical `ClassVar` and `InitVar` annotations are recognized only so the decorator can reject those unsupported field forms clearly; all other fields come from executed annotated names.

CONTROLLING CONSTRUCTION WITH `__NEW__`

Building an instance has two steps. `__new__` decides **which object exists**, then `__init__` fills it in. Most classes only need `__init__`; reach for `__new__` when `Cls(...)` should hand back an object it already has.

```
class Settings:
  _instance = None
  def __new__(cls):
    if cls._instance is None:
      cls._instance = super().__new__(cls)
    return cls._instance
  def __init__(self):
    self.rate = 5

print(Settings() is Settings())  # True, always the same object
```

`__new__` takes the class as its first parameter (`cls`, not `self`) and must **return** the object. `super().__new__(cls)` makes a fresh one. `__init__` is skipped only if the returned value is not an instance of the class being constructed. Returning an instance of that class or a subclass runs the returned object's `__init__`.

> `__new__` must answer immediately: it cannot `sleep()` or take world actions. Allocation only picks the object; put the work in `__init__`, which can still do both.

ANSWERING UNKNOWN ATTRIBUTES WITH `__GETATTR__`

`__getattr__(self, name)` runs only when a name was **not** found the normal way, so it never slows down or shadows a real attribute:

```
class Readings:
  def __init__(self, values):
    self.values = values
  def __getattr__(self, name):
    if name in self.values:
      return self.values[name]
    raise AttributeError(name)

r = Readings({"pressure": 91})
print(r.pressure)   # 91
```

Raising `AttributeError` is how you say a name really is missing, and it is what `hasattr()` and `getattr(obj, name, default)` look for. Like operator dunders, `__getattr__` must be pure.

REGISTERING SUBCLASSES WITH `__INIT_SUBCLASS__`

`__init_subclass__` runs on a **base** each time a subclass is defined, which is the plain way to keep a registry:

```
class Job:
  registry = []
  def __init_subclass__(cls):
    Job.registry.append(cls)

class Haul(Job): pass
class Scan(Job): pass
print(len(Job.registry))  # 2
```

It receives the new class as `cls` and never fires for the class that defines it.

WHEN TO USE A CLASS

- **Class**, custom initialization, inheritance, or operator overloading.
- **`dataclass`**, named fields plus generated construction, display, equality, or ordering.
- **`TypedDict`**, a mapping-shaped record or JSON payload (the editor autocompletes its keys).
- **`dict`**, dynamic, data-driven keys.

WHAT'S NOT SUPPORTED

Reasonably-full Python classes, with these deliberate exclusions. Defining any of them is reported as an error naming the method, so nothing you write is silently ignored:

- Metaclasses, `__slots__`, `abc` / `@abstractmethod`.
- The general descriptor protocol (`__get__`, `__set__`, `__delete__`, `__set_name__`), only `@property` is exposed.
- `__del__`. Scripts have no reference counting and their memory is released all at once when the script stops, so a finalizer could never run at a meaningful moment. Release things in a method you call yourself.
- `__setattr__`, `__delattr__`, `__getattribute__`. Attribute assignment always stores directly; use a `@property` setter to run code when a value changes, and `__getattr__` for names the class does not already have.
- `for` advances `__iter__` / `__next__` and generators lazily, so `break` works with an endless iterator. Operations that need the entire result, such as `list(...)`, remain bounded by the collection limit.
- A class that defines `__eq__` without defining `__hash__` is unhashable, matching Python. Define an integer-returning `__hash__` to use equality-aware instances as `dict` or `set` keys. Classes that define neither use identity hashing and also work as keys.

Code Editor

Each machine with a script slot has a code editor window. Open it by clicking **MANAGE** on the machine card.

TABS

- **Info**, task description, what this script should do
- **Code**, write your code here
- **Variants**, named versions you can reuse on matching machines (see below)
- **Notes**, free-text scratchpad for your own notes

TITLE BAR CONTROLS

- ▶ **Run** (`Shift+Enter`), execute the active variant
- ⏸ **Pause** (`F6`), pause or resume a running script
- ■ **Stop** (`Shift+F5`), stop the script completely and reset the machine settings the script controls. Rover, Pioneer, and Drone movement cancels immediately.
- –: **Trace**, highlight where the script is parked each tick. A green bar marks the "resting" line, where execution was when the most recent tick ended. Honest about suspension points (for scripts with `sleep()` or `scan()`) and where the step limit tripped (for tight loops).
- ⊕ **Debug**, enable breakpoints, stepping, and the variable watch panel. See the Debug guide for the full flow. Off by default.
- 🔍 **Find**, search and replace within the script (`Ctrl+F`).

BOTTOM BAR

- **Duplicate Variant**, save the current code as a new named variant
- 🔊 **Mute**, mute this script's console output. Text gets a strikethrough when muted. Muted scripts don't print to the console; errors still show. Useful for silencing background scripts you're not actively working on.
- ⊞ **Dock**, switch to an IDE-style layout. The editor pins to the left, the console docks to the bottom-right, and the dashboard fills the remaining space. The sidebar hides automatically to maximize content area. Drag the dividers between panels to resize. Your dock layout is remembered between sessions. Click **Undock** to return to floating windows.
- ↓ **Auto-min**, when enabled (green), the editor minimizes automatically when you run the script. Useful for watching the dashboard while your code executes.

NAVIGATING FROM CODE TO DOCS

The editor can jump straight to matching DOCS entries while you write:

- **Hover over any method or function name**, a tooltip shows the signature, a one-line description, and the return type. Works on `self.X`, `self.X.Y`, top-level API calls (`get_component`, `sleep`, `print`), builtins (`len`, `range`, etc.), component methods accessed via `get_component(id).method`, and user-defined functions with docstrings.
- **Cmd+click (macOS) / Ctrl+click (Windows/Linux) on any method, function, component id, or variable**, opens DOCS and jumps directly to that entry, with the specific method highlighted for a second or two so you can see exactly where you landed. Works like "Go to Definition" in a professional IDE, but the target is the DOCS page since the game APIs don't have a source file.
- **Autocomplete as you type** (`Ctrl+Space` to force it), suggestions are type-aware. Move through the list with `↓` and `↑`, `Enter` inserts the highlighted one, and Tab takes the first match. Those three keys are yours to change in **Settings** → **Keybinds**, under Editor. After `self.` you'll see only the methods and sub-objects available on your machine. After `self.battery.` you'll see only the Battery sub-object's API. String arguments offer known-valid values (component ids, bin names, mineral ids) where available.
- **Parameter hints inside `(...)`**, as you type `self.drill.mine(`, a hint shows each parameter's name, type, and description.
- **`F2` to rename** a symbol (variable, user function, parameter). Renames every whole-word occurrence in the current script, skipping strings and comments. Reserved names (keywords, `self`, builtins) can't be renamed.

> Hover for signatures. Cmd/Ctrl+click opens the matching DOCS entry without leaving the editor.

ERRORS AND WARNINGS

The editor checks your code as you type and underlines what would fail or looks wrong. To silence a check you know is fine, end that line with `# type: ignore` or `# noqa`, as in other Python tools. A `# type: ignore` line at the very top of a script, before any code, quiets the whole script. Syntax errors always show, because a script that does not parse cannot run. External editors show the same checks and follow the same comments.

COMMANDS TAB

While a script runs, its **Commands** tab lets you send named commands with key/value arguments into the script's command mailbox, your script reads them with `self.next_command()` and decides how to react. Frequent commands can be saved as presets. See the **Script Commands** page for the full pattern.

VARIANTS

Named variants use layered catalogs. Exact-type variants belong to one precise machine type; compatible-family variants are shared only across tiers with the same scripting contract. **Main** is this machine's own fallback code and is never shared. Editing in the Code tab saves changes to the active variant. Changes to **Main** stay with this script; changes to a named variant update its shared catalog entry. Other scripts keep their loaded copies until you explicitly update them. Use **Duplicate Variant** to publish the current code as a new named variant and choose its scope when a compatible family exists. Click **Use** to load a catalog variant onto this machine. **Apply to all** copies the selected catalog version to either the exact type or every compatible family tier, according to the section that owns the variant, and always requires the same script slot. Main cannot be deleted. Tip: add a top-of-file `"""docstring"""` or first-line `#` comment and it will show as the variant's description. Open **Docstrings** for function hover docs and supported formatting.

SAVING

Your scripts auto-save every **30 seconds**. Press `Ctrl+S` to save and flush the current source immediately. All editor shortcuts, including undo, redo, comments, search, and run, can be changed in **Settings** → **Keybinds**.

Scripts run continuously if they contain a `while True:` loop. Otherwise they run once and complete.

RELOAD BEHAVIOR

A script saved as running starts again from the top when you load the game. Its local variables and current line are not restored. A paused script stays paused but also starts from the top when you resume it after loading; a stopped script stays stopped. Persistent world state and machine work remain where they were, so check current cargo, storage, and machine state before repeating actions. Use the **Data Archive** for script progress that must survive reloads.

The console window shows output from all unmuted scripts. Use **CLEAR** to empty it and **AUTO-SCROLL** to follow new output.

Vim Mode

OVERVIEW

Vim mode gives the in-game code editor vim keybindings. Turn it on in Settings → Editor → Keybindings → Vim. It is opt-in and affects ONLY the code editor; game menus, the map, and dashboard controls keep their normal keys.

WHAT WORKS

The editor uses a vim-flavored keymap, so the everyday toolkit is there: normal / insert / visual modes, motions (`h` `j` `k` `l`, `w`, `b`, `e`, `gg`, `G`, `f`, `%`), operators (`d`, `c`, `y`, `p`) with counts and text objects, search (`/`, `?`, `n`, `N`), registers, marks, and undo / redo (`u`, `Ctrl-r`). It is vim-**flavored**, not a 100% feature-complete vim, most muscle memory carries over, but a few exotic commands and plugins will not.

SAVE AND QUIT

These ex-commands are wired to the game's real save and window controls:

- `:w`, save now. Flushes your script to disk immediately (it is auto-saved anyway, so this just makes the reflex real).
- `:wq` / `:x`, save, then close the editor window.
- `:q`, close the editor window. Your work is already saved.

BRING YOUR OWN VIM

Prefer your actual vim setup? See the External Editor page, your scripts live as real files on disk that you can edit in any editor, full vim included, and the game picks up changes live.

Script Commands

OVERVIEW

Every machine script has a **command mailbox**, a small queue of named commands you send by hand while the script runs. Commands let you steer a running script without editing it: the script keeps its own logic and decides how to handle each request.

This is the player-to-script channel. For script-to-script coordination, use the **Signal Bus** instead.

SAVING REUSABLE COMMANDS

Open a script's **Commands** tab. Add a command with a **Name** (the button label), a **Command** id (what your code checks for), and any **arguments** as simple key/value fields, no JSON.

Choose the command's **Availability**:

- **All machines of this type** makes the definition available on current and future machines of the same exact type.
- **All compatible family machines** appears when the machine registry declares a compatible family. It shares the definition across that family.
- **Only this machine** keeps instance-specific coordinates, target ids, or other one-off data local.

Then **Send** it once, or **Save** it as a reusable one-click command. Existing local commands have a **Share** action that moves them into an exact-type or compatible-family collection. Shared edits and deletion affect every matching machine.

A saved definition is reusable, but execution is always local: **Send** queues a fresh message only on the machine whose Commands tab is open. Pending queues and recent history are never copied or broadcast. They remain per machine.

Commands are defined in the tab, not in your script, and they do not belong to a script variant. Switching or copying a variant does not duplicate command definitions. Each machine's mailbox holds up to **40** pending commands; further sends are rejected until the script consumes or clears some.

READING COMMANDS IN YOUR SCRIPT

Four methods are available on `self` in every machine script:

- `self.next_command()` consumes the oldest command and returns a `CommandResult`. `.status` is `"ok"` with the command in `.command`, or `"empty"` with `.command is None`.

- `self.peek_command()` reads the next `ScriptCommand` without consuming it, or `None` when the mailbox is empty.
- `self.command_count()` reports how many commands are waiting.
- `self.clear_commands()` drops everything and returns a `CountResult`; `.count` is exactly how many commands were removed.

A consumed command is a `ScriptCommand` in `CommandResult.command`, with `.name` (the command id as a string, not the button's Name label; your script decides what each id means) and `.args` (a `dict`; use `.args.get(key, default)` for optional arguments).

```
while True:
  next_result = self.next_command()
  if next_result.status == "ok":
    cmd = next_result.command
    if cmd.name == "goto":
      self.nav.set_target(cmd.args["x"], cmd.args["y"])
      self.nav.set_throttle(0.6)
    elif cmd.name == "stop":
      self.nav.brake()
```

> Commands are requests, not remote control. A script that never calls `next_command()` ignores its mailbox. Handling commands is part of the script's design, and unknown names should be ignored or reported with `print()`.

COMMANDS VS SIGNAL BUS

- **Command mailbox**: you (the player) to one machine script, from the editor. Best for manual steering, testing a mode, or one-off orders.
- **Signal Bus** (`get_component("comms")`): script to script, on named channels. Best for automation that coordinates machines while you are away from the keyboard.

Script Variants

Variants are named script snapshots you can reuse across a fleet without copying code by hand.

MAIN AND NAMED VARIANTS

Every script starts with **Main**. Main is this machine's own fallback code; it is not shared and cannot be renamed or deleted.

A machine can expose two named catalogs:

- **Exact type**, specialized for that precise machine model or tier.
- **Compatible family**, shared only by models that expose the same scripting contract. Small, Medium, and Large Drones share Drone family variants, for example. A Water Pump and Oil Pump do not share merely because both extract fluids.

The Variants tab always lists exact-type variants first and compatible-family variants below them. Machines without an explicitly compatible family show only the exact-type catalog. When you save a variant on a compatible machine, you choose which catalog owns it.

EDITING THE ACTIVE VARIANT

The Code tab edits whichever variant is active. Changes to **Main** stay with this script. Changes to a named variant update its shared catalog entry, while other scripts keep their loaded copies until you explicitly update them. An older loaded copy is marked **outdated**; click **Update** to load the latest catalog version, or **Keep mine** to save that copy as a separate named variant.

Use **Duplicate Variant** or **Create Variant from Selection...** to publish the current code as a new named catalog entry. Duplicate names across the visible catalogs are blocked so variant selection stays unambiguous.

USE AND APPLY TO ALL

Use loads a catalog variant into this machine's running code and makes it active here.

Apply to all respects the catalog that owns the variant. An exact-type variant targets that precise type. A family variant targets every explicitly compatible tier with the same script slot. If the selected named variant is active here and you edited it, **Apply to all** uses the updated catalog source.

When the applied source changes, a running target restarts automatically with the new code. A manually paused target stays paused, a power-paused target waits for power and then resumes, and a stopped

target stays stopped. A target that already has the same source keeps its current evaluator without a restart.

Apply to all is a one-time copy, not a permanent shared link. Future edits to a named variant update its catalog entry but do not change other scripts' loaded copies. Use Apply to all again when you intentionally want those scripts to load the improved version. Changes to Main remain private to the script that owns it.

CATALOG CHANGES

Renaming or deleting a named variant changes only its owning catalog. When a saved variant is deleted, scripts using it keep their current working code and lifecycle, then detach to Main. A running script keeps its evaluator, a paused script stays paused, and a stopped script stays stopped.

A top-of-file `"""docstring"""` or first-line `#` comment becomes the variant description shown in the list. Function docstrings are separate: they appear in hover and autocomplete, and the **Docstrings** page lists the supported formatting.

SHARING REAL CODE

Compatible family sharing is explicit machine metadata, not a guess based on names or appearance. If different scripting contracts need common helper functions, put those helpers in a Library script and import them instead of forcing those machines into one variant family.

UNASSIGNED SCRIPTS

Undeploying or deconstructing a machine, or deleting a Control Room card, stops its script. Authored code stays in **Computer > Scripts > Unassigned**, with its notes, private Main, selected variant, and run history. **Current** shows assigned scripts by default; **All** includes both views.

Unassigned scripts cannot run. Use their actions menu to **Copy to machine** (**Copy to panel** for a card's script), **Copy to Playground**, or **Delete**. You can select several unassigned scripts for deletion. Deletion removes their private data but keeps shared machine variants.

Copy to machine copies into a compatible machine slot, whose script keeps the slot's existing ID. It stops the target and preserves the previous code in Variants before replacing Main. Compatibility requires a recorded former machine type and slot; older unassigned scripts without that information can still be reviewed and copied to Playground. **Copy to panel** works the same way on any card you choose, since every card's script uses the same `panel` drawing API.

Copy to Playground creates a new named document without replacing Scratch or running the code. The original script stays available with all its metadata. Copied code is unchanged, including machine IDs and `self` references.

Machine IDs are never recycled by deleting an unassigned script.

Debug Mode

Debug mode adds breakpoints, uncaught-exception pauses, three-mode stepping, conditional breakpoints, logpoints, and a live variable watch panel to any script. It's off by default, toggle the ⊕ **Debug** button in the editor's title bar to turn it on.

WHAT DEBUG MODE GIVES YOU

- A **breakpoint gutter** that lets you click any line to pause execution there
- A **Watch panel** on the right showing current values of all local variables
- **Step Over**, **Step Into**, **Step Out**, and **Continue** controls when the script is halted
- A **Call Stack** section for jumping between function frames and inspecting each frame's locals
- A persistent **breakpoint list** in the Watch panel for jumping between breakpoints and editing their conditions / log messages

SETTING BREAKPOINTS

Click a line number's left gutter. A red dot appears on that line; the same line also shows up in the **Breakpoints** section of the Watch panel. Click the gutter again to remove, or click the `×` next to it in the Watch panel.

Breakpoints persist when you toggle Debug off and on, and across game sessions. The Debug button shows a small `·` when dormant breakpoints exist on a script that isn't in debug mode.

CONDITIONAL BREAKPOINTS AND LOGPOINTS

Click the `···` button next to a breakpoint in the Watch panel to edit it.

- **Condition**, a read-only Python expression. If set, the breakpoint halts **only when the condition is truthy**. Use variables, attributes, indexes, and arithmetic; function and method calls are blocked so debug expressions can never change the game. Example: `heat > 80`.
- **Log message**, a template with read-only `{expr}` placeholders. If set, the breakpoint **prints the message to the console and keeps running** instead of pausing. Like sprinkling `print()` without editing the script. `scan hit: {site.id}` prints the current site id each time execution reaches that line.

You can combine them, a condition + log message will log only when the condition matches.

Breakpoint color cues: red = plain, yellow = conditional, accent = logpoint.

WHEN EXECUTION HALTS

A breakpoint pauses before the marked line runs. An uncaught exception pauses on the line that failed and shows the error message. In both cases the line gets an orange highlight. The **Watch panel** shows:

- **Paused at line N** or **Paused on exception at line N** header with debugger controls
- **Local variables** for the selected call-stack frame; dicts, lists, tuples, and sets show compact previews first and expand on click
- **Call Stack** rows you can click to jump to that frame and inspect that frame's locals
- **Breakpoints** list with jump-to, edit, and remove controls

STEP OVER / INTO / OUT

Three step modes, following the industry standard:

- **Over (** ⤵ **)**, runs the next statement. If it contains a function call, the call runs atomically (doesn't descend). Most-used step.
- **Into (** ↓ **)**, descends into the next function call. Stops on the first statement inside. Use when you want to debug a helper function.
- **Out (** ↥ **)**, runs until the current function returns. Use when you stepped Into something boring and want to bail back to the caller.
- **Continue (** ▸ **)**, resumes until the next breakpoint or completion. From an uncaught exception pause, Continue lets the script fail normally and clears it.

THE WORLD KEEPS RUNNING

Breakpoints and exception pauses pause **only the debugged script**. Everything else continues:

- The clock keeps ticking
- Other scripts on other machines keep running
- Atmosphere, power, and all systems tick normally
- The machine controlled by the paused script keeps its last-commanded state while paused. If the script errors, stops, or completes, the machine settings the script controls reset; Rover, Pioneer, and Drone movement cancels.

This means time spent stepping through code advances the game world. If you're debugging a solar tracker and take two minutes to inspect variables, the sun has moved in that time. For scripts with strict timing, factor this in.

INSPECTING VARIABLES

The Watch panel updates live every tick when the script is running, or freezes at the current values when the script is halted.

- Numbers, strings, booleans show their values directly
- Dicts, lists, tuples, and sets show their size plus a shallow preview; nested values are abbreviated until expanded
- Small objects may show a short field preview; larger objects show their type first and expand on click
- The current line number is shown at the bottom of the panel

PINNED WATCH VALUES ARE OPTIONAL

You do not need to pin anything to inspect the current frame. When the script pauses, **Local variables** and **Call Stack** update automatically. Use **+ Pin** in Watch only when you want to keep a custom value or calculation visible across steps, such as `co2 / 10`, `self.waste()`, or `atmo.get_co2()`.

> The Watch panel samples variables at the script's last suspension point. For a script with `sleep(1)` at the end of each loop, that's the value at the moment `sleep()` was called. For a tight loop with no deliberate pauses, it's wherever the tick's step limit tripped.

ISOLATION GUARANTEE

Debug mode is a read-only-plus-pause tool. Enabling Debug on one script **cannot affect another script's behavior**. Breakpoints, exception pauses, step state, and watch inspection are stored per-script, turning debug on a heater script has zero impact on an oxygen generator script running in parallel.

Control Room

The Control Room is a top-level page for **cards**: script-driven panels that draw live dashboards and can issue commands through shared game APIs. Open the **Control Room** page, click **+ New Card**, and a fresh panel script is bound to a canvas.

A panel script runs every game tick. The pattern is always the same:

```
while True:
  panel.clear()
  # ... call panel.draw_X(...) and panel.widget(...) here ...
```

Your script controls the layout, colors, data shown, and update cadence. The same read APIs you use from machine scripts work here too: `get_component(...)`, `clock.get_elevation()`, `orders.list_orders()`. Panels can read component state; they change the base only through shared authorities such as `power_control`, `shop`, `comms`, `inventory`, `atmosphere`, or `computer`.

COMMON USES

Panels can read any component exposed to scripts. Common uses include:

- **Atmospheric history**, three progress bars for O_2 / N_2 / pressure with a 50-tick spark line below
- **Fleet status board**, one row per vehicle with a status dot, name, current activity, and battery bar
- **Sun-tracker monitor**, three gauges for sun elevation, panel tilt, and solar efficiency
- **Tank levels**, a bar chart showing every fluid buffer at the base, with net flow text below
- **Active order briefing**, wrapped text of the contractor's brief plus an iron-ingot icon and progress bar
- **Storage inventory**, every storage bin's icon, name, fill percent, and count

The data comes from existing APIs; panels provide a canvas and widgets for displaying it.

THE WIDGET SET

Twenty-one named widgets for common patterns:

- **Layout:** `card`, `divider`, `label`
- **Indicators:** `status_dot`, `toggle`, `pill`, `counter`
- **Bars:** `progress_bar`, `vertical_bar`, `bar_chart`
- **Curves:** `gauge`, `spark_line`

- **Interactive (the player clicks them):** `button`, `icon_button`, `switch`, `checkbox`, `slider`, `radio_group`, `combo`, `text_field`, `list`, read them each tick to act on the player's input

A radio group, a combo box and a list each take their choices as one list and store one value, so a five-way choice costs the card one saved key rather than five. `combo` and `text_field` open the game's own menu and a real text editor over the card, which is why they stay readable at any card size and why typing into a field has working selection, clipboard and input method.

Seventeen primitives for everything the named widgets don't cover:

- `draw_text` (with optional `wrap`), `draw_icon`
- `draw_rect` / `fill_rect`, `draw_circle` / `fill_circle`, `draw_line`
- `draw_polygon` / `fill_polygon`, `polyline`
- `texture` / `draw_texture`, for pixel art and backgrounds
- `clip_rect` / `clear_clip`
- `clear`, `width`, `height`

Driving the widgets from code: `set_switch`, `set_slider`, `set_selected` and `set_text` force a control to a value without waiting for a click, and `get_switch`, `get_slider`, `get_selected` and `get_text` read one without drawing it, for a control on a page the card is not showing; `forget` drops one stored value and `clear_inputs` drops them all. Stored keys are never swept for you, because a card that paints one page at a time would lose the other page's state, so a card is capped at 512 of them.

Reading the player directly: `mouse()` gives the cursor position on this card and whether the left button went down or came back up since you last asked, which is what dragging needs, `clicks()` hands you every click that missed a widget so you can hit-test your own drawing, and `capture_keys()` plus `keys()` give a focused card the keyboard. The cursor is sampled once per tick and the card repaints at the same rate, so anything you draw from `mouse()` follows the pointer about a frame behind: right for showing what is under it, wrong for anything that must track it exactly.

Laying things out: a widget is placed by the point you pass, but not every widget uses it the same way. Boxes, bars, pills and controls start at their top-left corner; `draw_text` and `label` are centered on `y`; `counter` stands on `y` as its baseline; circles, dots and gauges are centered on the point. `last_bounds()` gives you the box the last widget took up, top-left whatever it was placed by, so you can put the next thing after it, and `measure_text()` gives you the size of text before you draw it.

Every widget that takes a `color` parameter accepts theme tokens: `"accent"`, `"success"`, `"warning"`, `"error"`, `"text-bright"`, `"text-secondary"`, `"text-muted"`, `"text-value"`. It also accepts any CSS color: hex (`"#ff8800"`), `"rgb(255, 136, 0)"`, `"hsl(30, 100%, 50%)"`, or a name like `"orange"`. Theme tokens follow the player's theme; CSS colors stay exactly as written. Cards can also paint with **surface tokens**, `"bg-base"`, `"bg-surface"`, `"bg-panel"`, `"border"`, `"border-dim"`. `progress_bar` and `vertical_bar` also take three **gradient presets**, the meters on machine cards: `"good-high"` (red to green), `"good-low"` (green to red) and `"neutral"` (accent to green).

Wrap a `card(x, y, w, h, title)` for the bordered+titled frame, fill zones with `fill_rect(x, y, w, h, "bg-surface")`, and a manage-style row is `status_dot` (green) + `draw_text` (name) + `button` (play/manage). Theme tokens resolve from the active theme whenever the script redraws the card.

See the `Panel` entry in the API reference (left sidebar) for the full method list with live previews of each widget.

A COMPLETE WORKING EXAMPLE

A fleet status panel:

```
rover = get_component("rover_1")
pioneer = get_component("pioneer_1")

while True:
  panel.clear()
  panel.label(12, 22, "FLEET", "caption")

  panel.status_dot(20, 50, 5, "running")
  panel.draw_text(38, 50, rover.name, 13)
  panel.progress_bar(360, 44, 110, 10, rover.battery.level(), "success")

  panel.status_dot(20, 80, 5, "running")
  panel.draw_text(38, 80, pioneer.name, 13)
  panel.progress_bar(360, 74, 110, 10, pioneer.battery.level(), "success")
```

No `sleep()` needed, the interpreter paces the loop automatically. The panel repaints every game tick.

WATCH FOR...

> Cards can command only through shared authorities. A card has no `self` over a machine, so a machine's own actions (`set_throttle`, `set_recipe`, `move_to`, `mine`) don't work from a card. It can still call shared authorities such as `power_control` (breakers), `shop` (`buy()` and `sell()`), `comms` (Signal Bus), `inventory`, `atmosphere`, and `computer` (`deploy()`, `upgrade()`, `undeploy()`, `decommission()`, `rename()`). Treat cards as trusted automation because they can change base state.

> The canvas does not auto-clear. Always call `panel.clear()` at the top of every loop iteration, or old paint will pile up under new paint and the panel will look smeared.

> Coordinates are in logical pixels. A 1×1 card is 500×200; two-column cards are 1000 pixels wide and two-row cards are 400 pixels high. CSS scales the visual to the screen, so use `panel.width()` / `panel.height()` when the script must adapt to the card's span.

LIMITS

- **50 cards per save**, max. Past that, **+ New Card** is grayed out. Delete one to make room.
- **Board density**, choose 2-6 columns from the selector above the board. The untouched default uses 2 for ordinary workspaces, 4 for 4K-class workspaces, and 6 for sufficiently wide ultrawides. A manual choice persists.
- **Four card sizes**, 1×1, 2×1 (wide), 1×2 (tall), and 2×2 (big), selected directly from the card-size menu. The logical canvas scales with the span, so **draw relative to `width()` / `height()`** and your card reflows at any size.
- **One script per card.** Want separate concerns? Make multiple cards.

External Editor

Your scripts are saved as `.py` files on disk. You can edit them in any text editor, VS Code, Sublime, Notepad++, or anything else.

Changes sync **live** in both directions, while the game is running:

- Edit in-game, the file updates on disk within ~half a second
- Edit externally, the game picks up your save within ~250ms and the in-game editor's buffer refreshes (cursor preserved as best it can)

If the operating system cannot watch the folder (Linux inotify limits, some network drives), the game rescans it every two seconds instead and says so once in its log. A file another program is still holding (an editor's own save, an on-access scanner on Windows) is retried for a moment before that pass gives up on it.

WHERE THE FILES LIVE

Scripts for an active save sit in a folder named `<save_id>_scripts/` under your platform's app-data directory:

- **Windows:** `%APPDATA%\io.codeterraform.game\<save_id>_scripts\`
- **macOS:** `~/Library/Application Support/io.codeterraform.game/<save_id>_scripts/`
- **Linux:** `$XDG_DATA_HOME/io.codeterraform.game/<save_id>_scripts/` or `~/.local/share/io.codeterraform.game/<save_id>_scripts/`

`<save_id>` is auto-generated when you first create the save (something like `save_lk2j8x_a1b2c3`). It matches the `.json` save file beside the script folder; if you are unsure, sort the app-data folder by modified time and use the newest `save_..._scripts/` folder for the save you just opened.

Each machine or panel script uses the same filename shown in the in-game editor, such as `o2gen_1.py`, `solar_1.py`, or `rover_1.py`. Library scripts live in a `lib/` subfolder, named normally, a library called `sensors` is `lib/sensors.py`. The `.py` file contains only code; `codeterraform-scripts.json` is a small sync manifest that tracks safe filename aliases after renames. Files that used older generated names, such as `oxygen_gen_1.py`, remain tracked as safe aliases after migration; use the canonical id-named file going forward.

EVERY FILE IN THE FOLDER

- `<name>.py`: one file per machine, panel or contract script; `lib/<name>.py`: Library scripts.

- `user_stubs.py`: yours, never overwritten. Type aliases, `TypedDict` types and re-exported Library types declared here are read by the in-game editor and the extension.
- `logs/`: mirrored console output, `all.log` plus one file per script, rotated by size.
- `codeterraform-workspace.json`: the read-only analysis snapshot the language server reads. `codeterraform-scripts.json`: the sync manifest. Do not edit either.
- `CODE-TERRAFORM-IDE.txt`: the setup guide, with the exact language-server path on your machine and editor configurations.
- `*.pyi` and `pyrightconfig.codeterraform.json`: fallback stubs for generic Python editors (see below).
- `.codeterraform/`: transient files the editor and the game exchange while a command runs (Run, Stop, Library operations, debugging). Safe to ignore.
- `codeterraform-fleet.json`: the machine panel's data (owners, scripts and their status). Do not edit.
- `.vscode/settings.json`, `.zed/settings.json`, `.gitignore`: written once if missing and yours afterwards. The first folds the helper files under one collapsed row in VS Code's Explorer and hides the transient folder; the second makes the game's server Zed's Python server and formatter for this folder only; the third ignores everything machine-specific or regenerated, so `git init` here tracks your scripts, Libraries, `user_stubs.py` and your own configuration.
- `*.codeterraform-retired-<token>.bak`: recovery archives. The game keeps one whenever a file is renamed, deleted, replaced or in conflict, so an editor that writes through a stale handle can never lose code. They are yours to delete whenever you like; `*.codeterraform-write.bak` is the crash-safety copy of an atomic write and disappears by itself.
- Only each script's running source is a file. The **Variants** and **Notes** tabs live in the save, not in this folder, so named variants cannot be listed, switched or edited from outside the game.

VS CODE SETUP

Launch Code Terraform normally from Steam and open your save. In Settings > Editor > External Editor, choose **Set Up VS Code**. The game installs its bundled extension and opens the scripts folder. Later the button becomes **Open in VS Code** and also installs any updated client. VS Code must already be installed; the game never installs an editor or extension at startup. **Show Extension Package** remains available for custom setups. No Python or separate Node installation is needed for VS Code.

Install the extension once. It is a thin client: the game installs the language server that matches its own build under `external-ide/server/` in the app-data folder and records the path in each save's `codeterraform-workspace.json`, so the extension always launches the server that belongs to the game you are running. After a game update, open the save once and VS Code switches to the new server by itself. When the extension itself changes, use the same Settings button to update it.

Game files automatically use the **Code Terraform** language mode. Completion, diagnostics, hover, signature help and Go to Definition use the same analysis as the in-game editor, including each script's own `self` or `panel`. Other Python projects keep their Python tooling.

Save edits to sync them into the game. The VS Code Command Palette includes `Create Library in Game`, `Import File as Game Library`, `Rename Library and Update Imports`, `Apply Library in Game`, `Apply All Libraries in Game`, `Run Script in Game` and `Stop Script in Game` under Code Terraform. Apply deploys saved Library edits and restarts the scripts that import them, the same as Apply in the game; it never runs on save. Unknown files offer an Import action. Save files and resolve in-game source conflicts first. Rename updates game imports and saved variants through the normal game command. Attach and delete scripts in the game. No command runs host Python. Its status item reports context availability; click it to restart the service.

`codeterraform-workspace.json` contains a read-only analysis snapshot. Keep this save open to refresh owners, names and research. Offline analysis uses the most recent snapshot. If the folder was written by a different game build than the server in use, open the save in the game once to refresh both. Do not edit the generated context or the sync manifest.

DEBUGGING IN THE GAME

Keep this save open and the game unpaused. In VS Code, open a machine, panel, or contract script, set a breakpoint, and press F5. An idle script starts; a running script attaches without restarting. Breakpoints, conditions, logpoints, call stacks, locals, watches, hover inspection, and Step Over/Into/Out use the game's interpreter. Library breakpoints are reached through a script that imports that Library. Each session controls one script.

Shift+F5 or closing the debug session disconnects and leaves the script running. Use `Stop Script in Game` to stop it. Watches and the Debug Console accept read-only expressions; they cannot execute world actions or assign variables. Save your files first. To run changed main-script code, use `Run Script in Game` before attaching again. Apply edited Libraries, from the editor or in the game, and resolve any source conflicts. If you enabled **Pause When Inactive**, turn it off when you want execution to continue while the game is minimized.

Other editors with a Debug Adapter Protocol client can launch `node /path/to/debug-adapter.cjs`. That file lives beside the installed `server.cjs`. Use a `launch` request to attach and run an idle script, or `attach` to inspect an existing runtime. Set `workspace` to this save's scripts directory and `script` to the script's file path. The adapter uses standard input/output. It needs Node.js 20 or later outside VS Code.

FORMATTING WITH RUFF

Format Document, Format Selection and format on save work on game scripts in VS Code and in every other editor that uses the game's language server. The server is the formatter and Ruff is the engine: it runs the Ruff you already have (the VS Code Ruff extension's own binary, `ruff` on PATH, or a pipx, uv or cargo

install) and hands back only the lines that changed, so your cursor and folds stay put. A `ruff.toml`, `.ruff.toml` or `pyproject.toml` in the scripts folder applies; without one you get Ruff's default, Black-compatible style. Nothing is downloaded: when Ruff is missing, the format action says how to get it, and in VS Code offers to install the Ruff extension. Ruff's own extension cannot format game files directly, because they use the Code Terraform language mode that keeps Pylance from flagging `self` and the game builtins.

COLORING, RENAME, REFERENCES AND OUTLINE

Game files get semantic coloring the way Python files do: variables, parameters, functions, classes, methods, properties, imported modules and the script's `self` or `panel`, all from the same analysis that answers completion. An identifier the analysis cannot place keeps its plain grammar color rather than a guessed one. Rename Symbol (F2 in VS Code) follows the same scoping rules as the in-game editor, refuses the same reserved names, and stays inside the one file; Find All References and the Outline, breadcrumbs and Go to Symbol views use the same source. Cross-file rename is not offered.

THE MACHINES PANEL IN VS CODE

The Code Terraform view in the activity bar lists this save's outposts and machines, each with its scripts and their live status (running, paused, error with the line, idle, unpowered), plus Panels, Contracts and Libraries. Click a script to open it; use the inline Run and Stop actions or the context menu's Debug. The panel reads `codeterraform-fleet.json`, which the game rewrites whenever a script starts, stops or fails, so it stays current while the save is open.

OTHER EDITORS

Any editor that speaks the Language Server Protocol (neovim, Helix, Sublime Text with the LSP package, Emacs with eglot or lsp-mode) can use the same language server. Node.js 20 or later is required outside VS Code. The game installs the server at:

- **Windows:** `%APPDATA%\io.codeterraform.game\external-ide\server\server.cjs`
- **macOS:** `~/Library/Application Support/io.codeterraform.game/external-ide/server/server.cjs`
- **Linux:** `~/.local/share/io.codeterraform.game/external-ide/server/server.cjs`

Launch it as `node <path> --stdio` with the scripts folder as the workspace root, one server per save; the root marker is `codeterraform-workspace.json`. The game rewrites the file on every launch, so a configured editor keeps working across game updates. `CODE-TERRAFORM-IDE.txt` in the scripts folder repeats these configurations with the exact path on your machine.

neovim (Neovim 0.11 or later, nvim-lspconfig):

```
vim.lsp.config("codeterraform", {
  cmd = { "node", "/path/to/server.cjs", "--stdio" },
  filetypes = { "python" },
  root_markers = { "codeterraform-workspace.json" },
})
vim.lsp.enable("codeterraform")
```

Helix (`languages.toml`; listing the server on `python` replaces Helix's default Python server for those files):

```
[language-server.codeterraform]
command = "node"
args = ["/path/to/server.cjs", "--stdio"]

[[language]]
name = "python"
roots = ["codeterraform-workspace.json"]
language-servers = ["codeterraform"]
```

Sublime Text (LSP package, `LSP.sublime-settings`):

```
{
  "clients": {
    "codeterraform": {
      "enabled": true,
      "command": ["node", "/path/to/server.cjs", "--stdio"],
      "selector": "source.python"
    }
  }
}
```

On Windows, write the path with forward slashes inside these configurations.

ZED

Zed has no one-click install for an extension outside its registry, so the game's extension is installed once from the source the game ships: install Rust with rustup (rustup.rs; Zed builds with the `cargo` your shell finds first, and rustup's carries the WebAssembly target), then in Zed run **zed: install dev extension** and choose `<app-data>/external-ide/zed/code-terraform` (**Open in Zed** in Settings > Editor > External Editor writes that folder, and **Show Extension Package** opens the folder beside it). Zed compiles it in a few seconds; after a game update, run the same command on the same folder again. The Settings row shows whether Zed lists the extension. Then open a save's scripts folder in Zed, or choose **Open in Zed**, and trust the folder when Zed asks. The extension reads `codeterraform-workspace.json` in the folder you

opened and starts the game's installed server with Zed's own Node; the folder's `.zed/settings.json` makes that server Zed's Python server and formatter for this folder only.

PYCHARM

Set Up PyCharm in Settings > Editor > External Editor installs the game's own Code Terraform plugin, and the LSP4IJ plugin it builds on, into PyCharm and opens the scripts folder there. Close PyCharm first: its plugins can only change while it is not running, and the button says so if it is. Approve the third-party plugin once when PyCharm asks. Node.js 20 or later must be installed, because PyCharm supplies no Node and the language server and the debugger run on it; the setup says so when it is missing, and the executable can be set in PyCharm's **Settings > Tools > Code Terraform**. After that everything is in the editor. Completion, diagnostics, hover, signatures, definitions, rename, symbols, formatting through Ruff (install Ruff so it is on PATH) and semantic coloring come from the game's server, and PyCharm's own Python inspections stay off for game scripts, so `self` and the game builtins are never underlined. The **Code Terraform** tool window holds the Machines view, every script with its status and Run, Stop and Debug, and Game Output, the game's log with tracebacks linked to their lines; Run, Stop and Debug also sit in the gutter of every script. Debugging needs no configuration: Debug starts the game's debug adapter for that script, with breakpoints, stepping and variables in PyCharm's Debug tool window. The line under the button reports whether the plugin is installed and current; after a game update, Set Up PyCharm (with PyCharm closed) updates it. The **?** beside the button has the manual steps: LSP4IJ from the Marketplace, then **Install Plugin from Disk...** with the zip the game keeps under its app-data folder (**Show Extension Package** opens the folder that holds it).
Fallback stubs

Generic editors can still use the generated `.pyi` files for partial API help; this fallback does not provide full game-aware analysis.

`pyrightconfig.codeterraform.json` is the generated fallback configuration. New `pyrightconfig.json` files extend it so you can add your own overrides. Byte-identical old generated defaults migrate automatically. Customized configurations are preserved; to adopt updated defaults, add `"extends": "./pyrightconfig.codeterraform.json"` to your config and retain the overrides you need. PyCharm does not read Pyright configuration; mark `lib/` as a Sources Root for its own Library import resolution.

`user_stubs.py` is player-owned and is never overwritten. Put type declarations there, not executable helpers: aliases such as `MineralId = Literal["iron_ore", "copper_ore"]`, `TypedDict` types and re-exported Library types. The in-game editor and the extension both read them, so a function typed with one of them completes and lints like any other, while the game treats every import from the file as a no-op. Settings > Editor > External Editor has **Freeze Stubs** and **Regenerate Stubs** controls. Freezing fallback stubs does not freeze the game-aware language service.

CONSOLE OUTPUT IN YOUR EDITOR

The game mirrors visible console output into `<save_id>_scripts/logs/` from the first time you open the scripts folder or the extension package (Settings → Editor → External Editor keeps the toggle). `all.log` holds the combined console stream; script-specific files such as `rover_1.log` hold only that script's output. In VS Code the extension follows `all.log` live in its Output panel: choose `Code Terraform: Open Game Output`, or pick the Code Terraform channel in the Output view. Traceback locations in the channel and in the log files link to the script lines. Other editors can follow `logs/all.log` with their own tail. Logs append while the game runs and rotate by size, so noisy print loops do not grow forever. Use Clear Logs in settings when you want a fresh transcript.

WHEN EDITS COLLIDE

If a script's file changes on disk while you are working on that script in the in-game editor, nothing is replaced and nothing is locked. Your version stays in the editor and keeps saving with the game. A bar at the top of that editor says the script also changed elsewhere and names where the game kept the other version: a variant of the script, or a recovery file in the scripts folder.

- **Keep mine** keeps your version and writes it to the file.
- **Use other version** switches the editor to the version from disk. Your version stays in the copy the game kept of it.
- **Merge both** combines the two when they changed different parts of the script. Undo restores what you had.
- **Compare** opens the Variants tab on the other version.

If no copy of the other version could be saved yet, the game leaves the file alone, because it is the only place that version exists: **Save a copy again** retries, and **Overwrite the file** writes your version over it. Saving and quitting keep working the whole time. If you have no work of your own on that script, the editor simply picks up the change.

USE CASES

- Use game-aware language assistance in VS Code
- Version-control your scripts with git
- Share scripts with other players
- Write longer, more complex programs comfortably

Keyboard Shortcuts

Every application-level keyboard shortcut the game uses. Click any customizable binding in the Keybinds settings tab to change it. For multi-line cursors, hold `Alt` (`Option` on macOS) and drag in the editor.

Contracts

Earth posts contracts, engineering tasks they need done on the planet. You write code to solve them and transmit the answer.

Each contract has:

- A briefing explaining what Earth needs
- Input data in `self.contract`
- A reward in credits

To complete a contract:

1. Go to the Contracts page and open one with View Contract
2. Read the briefing in the Info tab
3. Print `self.contract` to see available input fields
4. Transmit the answer via the transmitter:

```
c = self.contract
print(c)

transmitter = get_component("transmitter")
transmitter.connect("earth")
transmitter.transmit(c.id, answer)
```

If your answer is correct, you get paid immediately.

Commander and Shop are also accessible:

```
me = get_component("me")
print(me.get_credits())

shop = get_component("shop")
shop.sell("soil_sample")
shop.buy("solar_generator")
```

Vehicle Proximity & Service Areas

Ground vehicles use a forgiving service-area model. You do not need to land on one exact coordinate, but physical work requires the vehicle to be parked.

THE FIVE TERMS

- **Near**, the vehicle is inside a machine or outpost service area. It may still be driving through.
- **Stopped**, live speed is zero. A dead battery can stop a vehicle without canceling its route.
- **Parked**, the vehicle is stationary with no powered movement command. A stationary vehicle with no drive energy counts as parked even if its throttle or route remains set. Rescue and charging holds also keep a vehicle parked. `self.nav.brake()` parks it immediately.
- **Docked**, the vehicle is parked inside the service area required by an outpost, machine, or station.
- **Connected**, an item endpoint, pipe, or power relationship has been configured. A connection does not prove that a vehicle is docked and does not move cargo by itself.

The short rule is: **get near, park, then perform physical work**.

SERVICE AREAS

Machine and outpost service areas include a **~2 m** margin, so continuous vehicle movement does not need exact floating-point coordinates. An outpost's full footprint is its common service area. A field machine uses its own position and service area. Being near an outpost never makes a field machine part of that outpost.

PARKING BEFORE SERVICE

```
x = 25
y = -10
self.nav.set_target(x, y)
self.nav.set_throttle(0.5)
while self.nav.get_distance_to(x, y) > 2:
  sleep(1)
self.nav.brake()
```

After `brake()`, cargo transfer, charging, and mounting or installing hardware can use the matching service area. A cargo handoff takes one handling cycle and keeps the vehicle parked until it ends; movement commands return `busy` during that cycle.

`fleet.vehicles()` reports current activity separately from docking. `.status` can be `"moving"`, `"charging"`, `"drilling"`, or another activity. `.is_docked` means the ground vehicle was parked at an outpost when that snapshot was created; `.docked_at` gives that outpost id.

CHARGING STATIONS

A parked vehicle is docked at one Vehicle Charging Station at a time. Every station inside an outpost covers the whole outpost, so when an outpost has several, the vehicle's own script picks one with `self.dock("charging_station_2")`. Without a choice, it uses the eligible station closest to the vehicle, measured to the station or its outpost's center, whichever is closer, with equal-distance ties broken alphabetically by station id. The choice lasts until the vehicle drives away. `self.current_station()` and the fleet ref's `.current_station` report the station.

Drones use explicit station docking. A drone at a field Mining Drill must have completed its route before it can load ore.

Earth Orders

Earth requests materials through **contractor campaign orders** and the separate **Weekly Earth Orders** board. Both use Supply Docks and the same `Order` API. Orders are supply lines; Contracts are puzzles.

CONTRACTOR CAMPAIGN ORDERS

Helios Orbital, Spire Research, and Vestibule Logistics each expose the first unfinished order in their authored queue. These orders never expire. `orders.list_orders()` returns only the three current contractor slots, and completed campaign orders enter the permanent Completed Orders ledger. Contractor Reputation can increase their credit payout, and some campaign orders unlock technology or recipes.

PLANNING AHEAD

The Order Manifest and `orders.list_upcoming_orders()` reveal upcoming campaign requirements and rewards. The list preserves campaign declaration order, including each contractor's queue sequence, and excludes current orders, completed orders, and Weekly Earth Orders. Use `.contractor_id` to focus on one reward path. `orders.get_order(id)` also reads an upcoming order directly.

Upcoming orders have `.status == "upcoming"` and empty `.shipped` counts. Their credit rewards are quoted at your current reputation and may increase before fulfillment. Reading an order does not make it assignable: Supply Docks still accept only current orders.

WEEKLY EARTH ORDERS

The Weekly Orders tab is a dedicated full-width board with **five** progressive offers. It waits until you have uncovered an eligible mining or production chain, so `orders.list_weekly_orders()` can initially be empty. Once available, higher production tiers appear as your save uncovers their chains, and unavailable tiers fall back to earlier materials. An offer is an opportunity, not a guaranteed weekly completion: a known chain may still require machinery or infrastructure you have not built yet. The board starts on the day that an eligible chain appears and refreshes as a whole when its **seventh later day begins**. For example, a board created on Day 40 refreshes when Day 47 begins.

Weekly orders are credits-only. Their combined base payout stays at or below **20,000 credits**, or **40,000 credits** once **Bulk Orders** research makes every weekly order ask for twice the goods and pay twice the credits, Contractor Reputation does not multiply it, and fulfilled weekly rows remain visible until refresh. They do not enter the permanent campaign ledger. Read the board with `orders.list_weekly_orders()`; weekly objects have `.kind == "weekly"`, an `.expires_day`, and no contractor.

At refresh, every unfinished weekly order and all shipped progress expire permanently. Supply Docks assigned to the old board are unassigned and disabled. Cargo that was loaded but had not shipped remains inside its dock. The game posts a warning toast only when at least one unit actually shipped toward an unfinished weekly order; assigning or loading a dock without shipping stays silent. The toast remains available in **Computer > Notifications**, and scheduled refreshes do not create navigation badges.

SUPPLY DOCK WORKFLOW

1. **Pick an order.** Read `orders.list_orders()` or `orders.list_weekly_orders()`, then call `self.set_order(order.id)`.
2. **Load it.** Iterate `self.current_order().requires` and call `self.input.take(item_id, count)`. The dock refuses overshoot.
3. **Enable dispatch.** Call `self.set_enabled(True)`. Several docks may serve the same order and share shipped progress.
4. **React to release.** Completion or weekly expiry clears `current_order()` and disables dispatch, so the script must choose again and re-enable.

Each dock dispatches continuously at `self.dispatch_rate()` units/h. `self.current_dispatch()` reports the item leaving now and `self.dispatch_progress()` reports the next-unit fraction. Loaded units always remain physical cargo. `clear_order()` releases the assignment but leaves those units in the dock; a different `set_order(...)` produces an `ActionResult` with `.status == "cargo_present"` until a local machine or vehicle drains them.

REWARDS AND IDENTITY

Credits and unlocks apply automatically when the final required unit lands. Recipe rewards unlock the named recipe at its stated machine. Technology rewards make the named upgrade pack available for purchase in the Shop; they do not place a free pack in Inventory. Campaign objects have `.kind == "campaign"`, contractor identity, and no expiry. Weekly objects have `.kind == "weekly"`, `contractor_id == None`, `contractor_name == None`, and a board expiry day. Use `orders.get_order(id)` for upcoming or current orders and permanent campaign completions; unknown or expired weekly ids return `None`.

Signal Bus

OVERVIEW

Signal Bus is explicit shared state for scripts and the commander. Libraries share code; Signal Bus shares live values on named channels. For steering only one running script by hand, use its command mailbox (`self.next_command()`) instead.

Signal Bus unlocks with **Signal Bus** research. Before that, `get_component("comms")` returns `None`.

MANUAL OPERATOR

Open **Ship Computer > Signal Bus** to send traffic without writing a scratch script. Choose **Queue** when one receiver should consume the message, or **Broadcast** when many readers should see the current value. Plain input is sent as text; valid JSON sends structured objects, arrays, strings, numbers, booleans, or `null`. Manual traffic shows your commander name as its sender.

Use **Inspect** on a channel to review its queued messages. Delete one message or clear the currently shown queue. Clearing targets the exact message ids you inspected, so a script message that arrives afterward is preserved. Queue deletion does not remove the channel's broadcast value.

QUEUED MESSAGES

Use `send(channel, value)` and `receive(channel)` when each message should be handled once.

```
comms = get_component("comms")
sent = comms.send("jobs.mine", {
  "sector": "E14",
  "resource": "iron_ore",
  "priority": 2,
})
if sent.status != "ok":
  print(sent.message)
```

`send()` returns a `SendResult` with `.status`, `.message`, and `.message_id`. On `"ok"`, `.message_id` is the positive integer id of the message just queued. It is `None` for `"invalid_channel"`, `"invalid_value"`, `"queue_full"`, `"channel_limit"`, and `"id_exhausted"`, because those outcomes send nothing.

```
received = comms.receive("jobs.mine")
if received.status == "ok":
  msg = received.packet
  job = msg.value
  print("job from", msg.sender, job["sector"])
else:
  print(received.message)
```

`receive()` returns a `ReceiveResult`. `.packet` is a consumed `CommsMessage` only when `.status == "ok"`; it is `None` for `"empty"`, `"not_found"`, and `"invalid_channel"`. Omitting the id or passing `None` takes the oldest message. Supplying an id takes only that message. In short: `receive()` consumes; `latest()` does not.

WAIT FOR WORK

Use `wait(channel)` for a worker that should stay idle until it receives a request. It takes the oldest queued message immediately if one is available. Otherwise, only this script waits; the game and other scripts keep running. It does not need `async`, `await`, or a checking loop.

```
comms = get_component("comms")
while True:
    received = comms.wait("jobs.mine")
    if received.status != "ok":
        print(received.message)
        break
    job = received.packet
    print("Received", job.id, job.value)
    # Perform the requested work here before waiting again.
```

`wait()` returns a `ReceiveResult` with `.status`, `.message`, and `.packet`. A valid call waits for `"ok"` with a consumed `CommsMessage`. A malformed channel returns `"invalid_channel"` immediately with `.packet` set to `None`. Missing, empty, and broadcast-only channels keep the script waiting. There is no timeout. Broadcasts do not wake a worker with a job.

Messages stay queued until an active worker takes them. Each message can be taken by only one receiver. If another receiver or a Cancel button gets there first, the waiting worker stays waiting for the next message. Workers follow the normal script execution order; waiting does not reserve a message or grant priority over `receive()`.

Pausing the script preserves its wait without taking work. Stopping or restarting it abandons that wait without removing queued messages. Save/load preserves queued messages and follows normal script startup; it does not save the suspended call or local variables. Once a message has been received, the worker owns completing the job and reporting its outcome.

Use `receive()` when the same script must keep checking sensors, commands, or Control Room buttons while no job is available. A card using `wait()` pauses its own drawing and input handling too.

KEEP A SEND RECEIPT

Keep the successful send's `.message_id` when your script may need to identify or cancel its own request later. This is the same id shown by `pending()` and `receive().packet`, and identical payloads still receive different ids.

```
sent = comms.send("jobs.water", {"outpost": "A"})
if sent.status == "ok":
    request_id = sent.message_id
    print("My delivery request", request_id)
else:
    print(sent.message)
```

After a successful send, when that request is no longer needed, pass the saved `request_id` to `comms.cancel("jobs.water", request_id)`. A receipt records which message was sent; it does not promise that the message is still waiting or that the work has been completed. Match the id against a fresh `pending()` list to check whether it is still queued. If a worker already received it, cancellation returns `"not_found"`.

INSPECT PENDING WORK

Use `pending(channel)` to inspect waiting messages without consuming them. It returns a list of `CommsMessage` entries, oldest first, with the same `.id`, `.sender`, `.tick`, and `.value` fields as a received message.

```
comms = get_component("comms")
for msg in comms.pending("jobs.mine"):
    print(msg.id, msg.sender, msg.value)
```

Use this list in a Control Room card to display the backlog while worker scripts continue receiving jobs. Reading `pending()` does not take or reserve any work. The list, each message, and nested values are copies; changing them does not change the queue.

The snapshot contains only messages waiting when you called it. It excludes broadcasts and jobs already taken by `receive()` or `wait()`, even if their workers are still busy. Read again to refresh after messages arrive or are received. An empty queue, a missing channel, or a channel with only a broadcast returns `[]`. A malformed channel id raises `ValueError`.

CHOOSE AND TAKE A JOB

Pass a message id to `receive(channel, message_id)` to take exactly the job you chose from `pending()`. Workers can choose by priority, location, or capability, and a Control Room card can offer a Run button for a specific job.

```
for candidate in comms.pending("jobs.mine"):
    if candidate.value["sector"] == "E14":
        received = comms.receive("jobs.mine", candidate.id)
        if received.status == "ok":
            job = received.packet
            print("Claimed", job.id, job.value)
            break
        elif received.status == "not_found":
            continue
        else:
            print(received.message)
            break
```

Checking for the selected id and removing its message happen in one operation. If two workers choose the same message, only one can receive it. The other gets `"not_found"` with `.packet` set to `None`, even if other jobs remain in the queue. It never takes a different job in place of the selected one. Start work only after `"ok"` and use the returned `.packet` for the job's data.

The id can also come from a successful send receipt. It must be a positive whole number within the supported integer range. A wrong channel or an id already received, canceled, or cleared produces `"not_found"`; a malformed channel produces `"invalid_channel"`. Invalid numeric ids raise the documented exceptions. Without an id, an empty queue produces `"empty"`.

Other queued messages, new arrivals, and broadcasts are preserved. Earlier `pending()` snapshots remain copies; read again to refresh. Receiving removes the job from the queue immediately. Your worker script still owns completing the work and reporting its outcome.

CANCEL PENDING WORK

Use `cancel(channel, message_id)` to remove one obsolete request. Pass the successful send's `.message_id` or the message's `.id` from `pending()`, not its position in the list. A Control Room card can give each waiting job its own Cancel button.

```
for msg in comms.pending("jobs.mine"):
    if msg.value["sector"] == "E14":
        result = comms.cancel("jobs.mine", msg.id)
        if result.status == "ok":
            print("Canceled", msg.id)
        elif result.status == "not_found":
            print("This job is no longer waiting")
        else:
            print(result.message)
```

`cancel()` returns an `ActionResult`. `"ok"` means that exact waiting message was removed. `"not_found"` means the id is not in this channel's queue, including when another script already received or canceled it. A malformed channel returns `"invalid_channel"`. The message id must be a positive whole number within the supported integer range; malformed numeric arguments raise the documented exceptions.

Other waiting messages keep their ids, payloads, and order. New arrivals and the latest broadcast are preserved. Cancellation does not stop a worker that already received the job. An earlier `pending()` snapshot stays unchanged; read again to update the panel.

WAIT ON SEVERAL CHANNELS

Use `wait_any(channels)` when one worker handles several kinds of requests:

```
comms = get_component("comms")
result = comms.wait_any(["urgent_deliveries", "deliveries"])
if result.status == "ok":
    print(result.channel, result.packet.value)
```

The first listed channel with queued work supplies exactly one message, oldest first within that channel. Priority is checked when the script can take work, including after waking or resuming from pause. If all listed queues are empty, only this script waits until one has work. Unrelated channels and broadcasts do not supply a message. There is no timeout, and missing channels do not need to be created in advance.

The result is a `WaitAnyResult` with `status`, `message`, `channel`, and `packet`. On `"ok"`, `channel` names the selected channel and `packet` is the consumed `CommsMessage`. If any channel id is invalid, `"invalid_channel"` returns immediately with `channel` and `packet` both `None`; no message is taken from any channel.

Pass a list containing 1-128 string entries. Other container types and non-string entries raise `TypeError`; an empty list or more than 128 entries raises `ValueError`. The whole list is validated before receiving. The list is copied when called; duplicate names are considered once at their first position.

Priority chooses the next request; it does not interrupt work already underway. A continuously busy earlier channel can keep later channels waiting. Competing receivers still take each message at most once. Pause, stop, save/load, and Control Room drawing follow the same rules as `wait(channel)`.

WAIT FOR A NEW BROADCAST

Use `wait_broadcast(channel)` to let several scripts react to the same new signal:

```
comms = get_component("comms")
result = comms.wait_broadcast("factory.phase")
if result.status == "ok":
    print(result.broadcast.value)
    print(result.broadcast.sender)
```

After the workers are waiting, another script or the Computer Signal Bus operator can broadcast `"start"` on `"factory.phase"`. Every script already waiting captures that publication and resumes at its next execution opportunity. It does not consume the shared broadcast or any queued messages. `send()`, queue edits, cancellation, and clearing a channel do not satisfy this wait.

A call waits for the first successful broadcast published after it starts. A value already available through `latest()` does not satisfy a new wait; use `latest_info()` when you want the current value immediately. Publishing the same value again still counts, including `None`. Failed publications and broadcasts on other channels do not wake it. There is no timeout, and waiting does not create a channel or use its queue capacity.

The fixed `WaitBroadcastResult` has `status`, `message`, and `broadcast`. On `"ok"`, `broadcast` is a `BroadcastInfo` with a copied `value`, `sender`, and `age_seconds` sampled when the script resumes. On `"invalid_channel"`, `broadcast` is `None`. Incorrect argument types or arity raise `TypeError`.

The first captured publication is retained if later broadcasts arrive before the script resumes, even in the same tick, or the channel is cleared. Each script receives an independent copy. Pausing a waiting script retains its first signal until resume; stopping or restarting discards it. Save/load preserves the bus's latest value, but not active waits or their captured signals; a restarted wait listens for a future publication. Signals published while a script is doing other work are not buffered for its next wait.

EDIT A WAITING REQUEST

Use `update(channel, message_id, value)` to replace a request while it is still queued. A Control Room job board can let the player adjust an amount, destination, or priority without canceling and resending the job.

```
sent = comms.send("deliveries", {"item": "iron_ingot", "amount": 50})
if sent.status == "ok":
    updated = comms.update("deliveries", sent.message_id, {
        "item": "iron_ingot",
        "amount": 100,
    })
    print(updated.status, updated.message)
```

`update()` returns an `ActionResult` with `.status` and `.message`. `"ok"` means the complete replacement value was applied. `"not_found"` means that id is not waiting on the named channel, including after receipt, cancellation, or clearing. `"invalid_channel"` rejects a malformed channel; `"invalid_value"` rejects a replacement outside the payload rules below. Rejections leave the bus unchanged and never recreate a missing message. The id follows the same positive whole-number rules as `cancel()`.

The message keeps its id, queue position, original sender, and original send time. Other messages, new arrivals, and the latest broadcast stay unchanged. Updates require no new queue slot or message id, so a full queue can still be edited. A successful update is not a new send.

The entire value is replaced; dictionary fields are not merged. Earlier `pending()` snapshots remain unchanged. Read again to refresh your panel. If several scripts update the same waiting message, the last successful update wins.

Updating and receiving each happen in one operation. A worker receives either the complete original value or the complete replacement. If the worker receives first, an update returns `"not_found"` and cannot change the worker's copy or its work. Workers should use the packet returned by `receive()` or `wait()` for the actual job data, since an earlier snapshot may predate an edit.

BROADCASTS

Use `broadcast(channel, value)` and `latest(channel)` when many scripts should read the same current value. Broadcasts are not consumed; every reader sees the current value until a later broadcast replaces it or the channel is cleared.

```
published = comms.broadcast("fleet.mode", "return_home")
if published.status != "ok":
  print(published.message)

mode = comms.latest("fleet.mode")
if mode == "return_home":
  self.nav.set_target(0, 0)
```

`broadcast()` returns an `ActionResult`. `latest()` is a read-only query: it returns the current value directly, or `None` when there is no broadcast. `latest()` and `queue_size()` raise `ValueError` for a

malformed channel id instead of disguising it as missing data.

CHECK BROADCAST FRESHNESS

Use `latest_info(channel)` when a stored value is only useful while its publisher keeps reporting. It returns a `BroadcastInfo` snapshot with `.value`, `.sender`, and `.age_seconds`.

```
comms = get_component("comms")
info = comms.latest_info("worker.status")
if info is None:
    print("No report yet")
elif info.age_seconds is None:
    print("Report age is unknown")
elif info.age_seconds > 30:
    print("Worker report is outdated")
else:
    print(info.sender, info.value)
```

Age is measured in simulation seconds, the same time base as `sleep(seconds)` and `clock.elapsed_seconds()`. It advances as the simulation runs, freezes while paused, and does not count time spent with the save closed. Choose a threshold that matches how often your worker broadcasts.

No broadcast means `None`, including a missing channel or a channel with only queued messages. A broadcast of `None` still returns a snapshot whose `.value` is `None`. For older saved broadcasts without sender or valid timestamp information, the corresponding `.sender` or `.age_seconds` is `None`; publishing again supplies fresh information. Malformed channel ids raise `ValueError`.

The value and nested data are copies. The age is captured when you call the API, so read again to refresh your panel. Queued messages do not change the broadcast's sender or age, and reading does not consume anything. A new broadcast resets its age even when the value is unchanged. The API reports elapsed time; your script decides when a report is outdated.

MONITORING AND CLEARING

```
for channel in comms.channels():
  print(channel, comms.queue_size(channel))

cleared = comms.clear("jobs.mine")
print(cleared.status, cleared.count, cleared.message)
```

`clear()` returns a `CountResult`; `.count` is the number of entries removed: queued messages plus the current value `latest()` reads, and `"invalid_channel"` is distinct from a valid `"no_op"`. Unlike the Computer's queue-only deletion, script `clear()` removes both the queue and current broadcast. Use it only when your script intentionally owns that channel.

PAYLOAD RULES

Values must be JSON-safe: `None`, booleans, finite numbers, strings, lists/tuples, and dictionaries with string keys. Functions, components, sets, class instances, and dictionaries with non-string keys produce `"invalid_value"`. A payload supports up to **8** nested levels, **1,024** total values, and **4,096** characters in each string or dictionary key.

Channel ids contain **1-64** letters, numbers, `_`, `.`, `:`, or `-`. Reserved object-field names such as `"__proto__"` are rejected. A queue holds up to **64** messages and a save up to **512** channels. The Computer operator uses these same limits. Always branch on the returned `.status`; `.message` is the authoritative reason text.

Automations

OVERVIEW

An automation is a script that belongs to no machine. Use it for work that spans the whole base: managing power, buying from the shop, deploying equipment, planning construction.

Automations unlock with **Automations** research. Open **Ship Computer > Automations** and click **+ New Automation**.

HOW IT RUNS

An automation runs whichever page is open. It has no power supply of its own, so a brownout never pauses it, and it restarts with the game like any other running script. Start, stop and edit it from the Automations tab, the Scripts tab, or its editor window.

WHAT IT CAN DO

It reads every component through `get_component()`. It can give orders to the shared systems that accept them from any script: `power_control`, `run_control`, `shop`, `inventory`, `computer`, `construction_blueprint`, `comms`, and the other components whose DOCS entries say so.

```
power = get_component("power_control")

while True:
    total = power.total()
    if total.capacity > 0 and total.stored < total.capacity * 0.2:
        print("Battery low:", total.stored, "of", total.capacity, "Wh")
    sleep(60)
```

WHAT IT CANNOT DO

There is no `self`. A machine's own actions, such as moving a vehicle, mining, or setting a recipe, work only from that machine's script. To have a machine act, send it a message on the Signal Bus and let its script read it.

LIMITS

- **50 automations per save.** Past that, **+ New Automation** is grayed out.
- **One script per automation.** Want separate jobs? Make separate automations.

- Deleting an automation keeps the code you wrote in **Computer > Scripts > Unassigned**.

Data Archive

OVERVIEW

Data Archive is persistent shared knowledge for scripts. Libraries share code, Signal Bus shares live coordination, and Data Archive stores learned data across script stops, saves, and reloads.

Data Archive unlocks with **Data Archive** research. Before that, `get_component("notebook")` returns `None`.

STORE AND READ

```
notebook = get_component("notebook")
fuel_table = {"slow": 0.8, "cruise": 1.0, "fast": 1.7}
written = notebook.set("rover.fuel_table", fuel_table)
if written.status != "ok":
  print(written.message)

saved = notebook.get("rover.fuel_table", {})
print(saved["cruise"])
```

`set()` is a gameplay command and returns `ActionResult`. `get()`, `has()`, and `keys()` are read-only queries and return their natural values directly.

ATOMIC UPDATES

`transaction(key, default, updater)` atomically reads, updates, and writes one key. It returns `ActionResult`; `.status == "busy"` means another transaction already owns that key.

```
def add_one(count):
  return count + 1

result = notebook.transaction("stats.samples", 0, add_one)
if result.status != "ok":
  print(result.message)
```

Updaters must be small pure functions: no `sleep()`, yielding actions, or world-mutating API calls.

MANAGE ENTRIES

In **Computer > Data Archive**, use **View** to inspect or copy a complete value, **Edit** to change it, or **New entry** to add a key. The value field uses JSON: strings are quoted, and booleans and empty values use `true`, `false`, and `null`. Manual edits have the same storage limits as scripts and record your commander name as the writer.

Save replaces the complete value only if the entry has not changed since you opened it. A conflicting script or window update preserves your draft; **Copy value** copies it and **Refresh value** opens the current stored value. Scripts continue running and may write again after a successful manual save.

```
deleted = notebook.delete("rover.old_table")
if deleted.status == "not_found":
  print(deleted.message)

cleared = notebook.clear("scratch.")
print(cleared.status, cleared.count)
```

`delete()` returns `ActionResult`. `clear()` returns `CountResult`, whose `.count` is the exact number of entries removed.

KEY RULES

A key is 1 to **96** characters: ASCII letters, digits, `_`, `.`, `:`, or `-`. Spaces, `/`, and every other character are rejected, and so are the reserved names `__proto__`, `prototype`, and `constructor`. For such a key, `set()`, `transaction()`, and `delete()` return `.status == "invalid_key"` with the rule in `.message`, and `get()` and `has()` raise `ValueError`. A prefix for `keys()` or `clear()` uses the same characters and length, and an empty prefix matches every key. It matches the start of the key as text, so `clear("test")` also removes `"testing.keep"`; end the prefix with `.` to clear one group, as in `clear("test.")`. An invalid prefix makes `keys()` raise `ValueError`, and `clear()` returns `.status == "invalid_key"`.

Name keys by subsystem, such as `"rover.fuel_table"` or `"plants.recipe_scores"`. Use Signal Bus for live coordination and Data Archive for knowledge you want to keep.

VALUE RULES

Values must be JSON-safe: `None`, booleans, finite numbers, strings, lists/tuples, and dictionaries with string keys. Each stored value may contain up to **8** nested levels, **16,384** total nodes, and **4,096** characters per string or dictionary key. The root value, every contained value, and every list, tuple, or dictionary container each count as one node; dictionary keys do not. The exact result statuses are `"ok"`, `"invalid_key"`, `"entry_limit"`, `"invalid_value"`, `"not_found"`, and `"busy"`; every

applicable command returns a precise `.message`. The **2,048**-entry limit and transaction contention are programmatically distinct.

Map Markers

OVERVIEW

Map markers are annotations you leave on the Planet Map: a glyph, a color, a label, and a short note pinned to a coordinate. You place them by hand in Plan Mode, and your scripts place them with `get_component("markers")`. Both paths write the same records.

A marker is a note, not construction. Placing one is instant and free: no materials, no Pioneer, no build time, no collision. Markers overlap buildings, sites, and each other freely.

Markers unlock with **Cartography** research. Before that, `get_component("markers")` returns `None`.

PLACE FROM A SCRIPT

```
markers = get_component("markers")

result = markers.place("survey.rover_1.empty:120:-40", 120, -40,
                       "No contact", "x", "neutral",
                       "Sonar sweep found nothing here.")
if result.status != "ok":
  print(result.message)
```

Reusing an id rewrites that marker, position included, so there is no separate move or rename call. That matters because a script restarts from its first line after a save and reload: a create-only API would fill the map with duplicates.

ORGANIZE BY ID PREFIX

There are no tags. Ids carry the structure, the same way Data Archive keys and Signal Bus channels do.

```
for marker in markers.list("build."):
  print(marker.label, marker.x, marker.y)
```

Include the controlling machine in the prefix, as in `"survey.rover_1."` rather than `"survey."`. Two scripts that share a bare prefix will overwrite and clear each other's markers.

KEEPING A FAMILY CURRENT

```
prefix = "survey.rover_1."
markers.clear(prefix)
for site in get_component("journal").surveyed_sites("nocturna"):
  if site.kind() == "mineral":
    markers.place(prefix + site.id, site.x, site.y, site.item_id, "resource")
```

Clearing first makes the family a picture of what your script believes right now. The trade-off is that an error midway through leaves the map half-populated. Plain `place()` calls without the clear are safe against that, but markers for conclusions that are no longer true will pile up. Pick per script.

`markers.clear("")` deletes every marker on the planet, including the ones you placed by hand, and nothing records who placed what. The prefix argument is required so a full wipe has to be written out deliberately.

ACTING ON A MARKER

Markers hold coordinates; they do not build anything. To turn one into real work, pass its position to the construction queue and let a Pioneer do the job.

```
for marker in markers.list("build.outpost"):
  plan = get_component("construction_blueprint").plan_structure("outpost", marker.x,
marker.y)
  if plan.status != "ok":
    print(marker.id, plan.message)
```

LIMITS

A planet holds up to **256** markers. Ids are 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`. Labels are up to **48** characters and notes up to **240**. Coordinates are world meters and keep their fractions; anything outside the planet is rejected.

Markers are for the map. Anything structured belongs in the Data Archive under the same id.

Drones

OVERVIEW

Drones are aerial logistics and field-work units. Every drone can use `self.go_to(x, y)` from the beginning; specialist modules determine what it can do when it arrives, not whether coordinate flight exists. Electric drones use Battery Packs and trade speed for range. Heli drones use Oil Tanks and fly much faster. Cargo Pods add single-material storage.

BLUEPRINT UNLOCKS AND DEPLOYMENT

Basic Drone Operations opens the system. Hardware recipes come through Earth Orders. Fabricate a Drone Depot Kit and deploy it into an outpost. Keep each fabricated chassis in Inventory, select its item card, and use **Deploy** to commission it into a free Depot bay. Once the drone is docked, mount its thruster, energy storage, and role modules through that drone's service controls. Commissioning a chassis and mounting its modules are service orders; do not freight the chassis or modules into the Depot stockpile.

Drone Depots commission chassis and handle local cargo. A larger Depot handles cargo faster: a Medium Depot moves items **2×** and a Large Depot **4×** as fast as a base Depot. Drone Service Stations charge electric drones, refuel heli drones from `oil_in`, decontaminate working docked drones by **10 exposure per hour**, and rescue stalled or scrambled drones.

FIELD WORK

Ground Rover/Pioneer Sonar discovers mineral sites, vents, wells, and exotic deposits. Portable Bio Scanners and Extractors handle permanent biological coordinates. Weather aftermath collection uses the same coordinate flight and `collect()` interface; the Weather DOCS define the information available to your programs.

`collect()` finds only a live aftermath at the drone's exact coordinate. Wrong, nearby, expired, and exhausted targets return `nothing_here` without a direction clue. Storm Glass is ordinary cargo. Raw Uranium is radioactive cargo: one call takes at most **5**, adds **40 exposure** without Shield Plating or **0** with it, and remains in cargo if that batch scrambles the drone at 100. Plating also halves Cargo Pod capacity and raises fuel burn **1.5×**.

READING THE FLEET

```
fleet = get_component("fleet")
for drone in fleet.drones():
    print(drone.id, drone.name, drone.status, drone.battery_level, drone.oil_level)
```

Use stable ids when passing a drone to station and recovery APIs. Use `self.cargo.space_for(item_id)` before a harvest or transfer, and route through service stations before fuel, oil, or exposure becomes a problem.

CROSS-REFERENCES

- Weather, programmable signals and aftermath collection
- Biosphere / Biomass, Portable Bio Scanner and Extractor work
- Drone Depot and Drone Service Station component APIs
- Orders, the source of drone blueprint rewards
- Flow Networks and Refinement, oil supply for heli drones

See `mining_guide` for collecting a field drill's output, `remote_logistics` for delivery planning and `nuclear_power` for handling radioactive cargo.

Batteries & Charging

Vehicles run on batteries. A parked vehicle costs nothing, battery only drains during actions.

WHAT DRAINS BATTERY

- **Moving**, the main cost. The Rover gets much worse at high speeds; the Pioneer is more efficient, but top speed still costs extra watt-hours per meter.
- **Drilling**, draws **10-30 W** while the drill is running, depending on the drill variant (basic / Industrial / Heavy). Watts are the rate and watt-hours are the amount used: **10 W** for one hour uses **10 Wh**, and for **15** minutes uses **2.5 Wh**. See **What mining costs** below.
- **Sonar**, a small flat cost (**0.5-3 Wh**, by sonar tier) per scan or paid survey. See sonar guide for the comparison.
- **Stationary**, free. No drain.

WHAT MINING COSTS

Each `self.drill.mine()` digs **1** unit, and the drill draws its power for the whole dig, so one unit costs **drill watts × mining time**. Mining time is the mineral's base time, multiplied by the drill's speed multiplier, divided by the site's purity.

Mineral	Base time per unit
Iron, Silicon	15 min
Lead	18 min
Titanium, Cobalt	20 min
Rare Earth	25 min
Neutronium	30 min

A **rich** site digs **2×** as fast and a **pure** site **3×** as fast as a standard one.

Drill	Power	Speed multiplier	Iron, standard site	Iron, pure site
Basic	10 W	1.0×	15 min, 2.5 Wh	5 min, 0.83 Wh
Industrial	20 W	0.75×	11.25 min, 3.75 Wh	3.75 min, 1.25 Wh
Heavy	30 W	0.6×	9 min, 4.5 Wh	3 min, 1.5 Wh

A faster drill finishes sooner but uses more energy per unit. `mine()` does not start a dig the battery cannot finish.

THE ROVER'S BATTERY

- Integrated, fixed **100 Wh**. Cannot be upgraded.
- At full speed the rover burns through battery fast. At half speed, it's much more efficient. Lower throttle is better for long trips.
- A fully-charged Rover at full throttle lasts about **5 hours**. At half throttle, closer to **20**.

THE PIONEER

- No built-in battery, mount battery modules for capacity.
- More efficient drivetrain, the high-throttle penalty is gentler than the Rover's, but slower cruises still stretch range.
- Each mounted module adds to the power draw while moving.

QUERYING THE BATTERY

```
level = self.battery.level()    # 0-1
wh = self.battery.wh()       # current Wh stored
cap = self.battery.capacity()   # max Wh
```

Check `level` before driving far.

CHARGING & RESCUE

Park anywhere inside a Vehicle Charging Station's service area to charge directly; a station inside an outpost serves the whole outpost footprint plus a margin of **~2 m**. The station charges docked vehicles in the order they were queued, each to its own target level: `self.charge(vehicle_id, 0.8)` charges that vehicle until it reaches **80%**. `dispatch_rescue(vehicle, target_level)` can also send a slow field-service drone to any rover or Pioneer in the field. Your script decides the threshold and target: `0.2` for a safety bump, `0.8` before a long return trip, or `1.0` for full remote charging. `cancel_rescue()` recalls that station's active rescue drone; any charge already delivered stays on the vehicle, and the vehicle is released while the drone flies home.

A dead battery **pauses** the current route while the script is still running; it does not cancel the script's intent. If the vehicle is recharged before the script stops, it resumes course. The vehicle stops and clears its route when you press Stop, the script reaches its end, or an uncaught error occurs. A vehicle being rescue-charged stays parked until the drone finishes, detaches, or is recalled by `cancel_rescue()`.

CHARGE THE VEHICLE, OR SWAP ITS BATTERIES?

A **Vehicle Charging Station** charges a parked Rover or Pioneer directly and can send its service drone to a stranded vehicle. A **Battery Charger** charges loose Portable Batteries for a Pioneer to collect during a stop. The Rover's fixed battery cannot be swapped.

Swapping is useful on a regular delivery route: the Pioneer takes ready batteries and leaves its drained ones behind, while the charger works during the next trip. You need a spare set of the same battery types installed in that Pioneer.

SET UP A BATTERY CHARGER

Research Battery Charger, buy one and deploy it at home or a founded outpost. Supply it from the local grid, then load loose **Portable Batteries (50 Wh)** or **Heavy Portable Batteries (100 Wh)** through its input. Use Inventory at home or a local Warehouse at a remote outpost. Automated item transfers need Auto Feeders.

Charger	Shared battery slots	Charging bays	Total charging power
Mk I	4	1	30 W
Mk II	8	2	60 W

Slots hold all batteries, whether waiting, charging or ready. Bays determine how many charge at once. At Mk II, one active cell can use the full **60 W**; two share **30 W** each before any outpost overcrowding penalty. The charger draws no power while idle.

Charging is script-controlled. This loop belongs on the **Battery Charger** and queues newly arrived cells, including batteries left by a returning Pioneer:

```
while True:
  for cell in self.slots():
    if cell.item_id and cell.level < 1.0 and cell.target is None:
      result = self.charge(cell.index)
      if result.status != "queued":
        print(result.message)
  sleep(1)
```

A charging job continues after its script stops and pauses through a power outage. When it reaches its target, the cell is released for collection. An unfinished job reserves its cell: wait for it or call `self.stop(slot_index)` to release it with its current charge. The incoming battery does not inherit the outgoing battery's job, which is why a repeating queue script is useful.

EXCHANGE FROM THE PIONEER

Drive to the charger's service area and brake. Call the exchange from the **Pioneer's** script, using your charger's actual id or display name:

```
result = self.swap_batteries("battery_charger_1", None, 0.9)
print(result.status, result.message)
```

Here `None` selects all installed batteries and `0.9` requires each replacement to be at least **90%** charged. Omit those optional arguments to request fully charged replacements for all installed batteries. Call when the Pioneer needs replenishing; the minimum is your chosen acceptance threshold.

Every selected battery needs an unqueued replacement of the **same type**. If one is missing or below the requested charge, the whole exchange is refused and nothing changes. Drained batteries take the vacated charger slots, so a full charger can still exchange a complete matching set. The transfer takes handling time; let it finish before driving away.

Ready, unqueued batteries remain usable during a grid outage, although the charger cannot replenish them until power returns. If a swap fails, check parking, matching types, charge levels, reserved charging jobs and whether either endpoint is still handling another transfer.

TIPS

- Plan trips around battery. Round trip + drilling must fit in your charge.
- Throttle down for long expeditions, lower speed extends range.
- Write a fleet manager that auto-dispatches rescues before vehicles are dead.

Refinement & Storage

Once you're mining, the refinement pipeline turns raw ore into stock and stages it for crafting.

SETTING UP A PIPELINE

`"inventory"` is a freight endpoint only at home; remote outposts use local storage and vehicles.

```
input_link = self.input.connect("Iron Ore Bin")
output_link = self.output.connect("Iron Ingot Bin")
recipe = self.set_recipe("smelt_iron_ingot")

if input_link.status == "ok" and output_link.status == "ok" and recipe.status == "ok":
  while True:
    ready = self.get_output_count()
    if ready > 0:
      sent = self.output.send("iron_ingot", ready)
      if sent.status not in ["ok", "partial", "no_op"]:
        print(sent.message)

    room = self.input.capacity() - self.input.count()
    if room > 0:
      taken = self.input.take("iron_ore", min(10, room))
      if taken.status not in ["ok", "partial", "no_op"]:
        print(taken.message)
```

`connect()` and recipe commands return `ActionResult`. `take()` and `send()` return `TransferResult` with `.requested` and `.moved`. Processing is asynchronous, so completed output appears later.

`self.input.flush()` returns `TransferResult`; `.moved` is the number of units permanently discarded. On processing machines it also cancels current progress. Call it intentionally.

See `mining_guide` for bringing ore in from the field and `remote_logistics` for supplying a factory away from home.

Input & Output

Machines process items, while I/O ports decide where those items come from and where they go.

CONNECTIONS

A routed machine port connects to the display name/id of a compatible machine or store at the same outpost. `"inventory"` is a freight endpoint only at home.

```
self.input.connect("Raw Materials")
self.output.connect("inventory")
self.input.take("iron_ore", 10)
self.output.send("iron_ingot", self.output.count())
```

Connections persist. Transfers require **Auto Feeders**, take time proportional to units moved, and are transactional: rejected or blocked transfers leave the exact source items unchanged. **Fast Feeders** unlocks at **400,000 Terraform Points** and halves the duration of newly started timed item transfers throughout the logistics system. Transfers already in progress keep the duration they started with. A Drone Depot handles its own transfers faster by size: a Medium Depot at **2×** and a Large Depot at **4×** a base Depot's pace. **High-Capacity Depot Handling** at **580,000 Terraform Points** doubles every Depot, stacking with Fast Feeders.

A Plant Terraformer's receiving feeder handles **16 items per handling step** at Mk I and **80** at Mk II. The same capacity applies whether its input pulls from a source or another machine or store sends to it. Both physical endpoints remain occupied for the resulting transfer duration.

Field Mining Drills and Water Pumps use a pickup-only `PickupOutputSlot` instead. It exposes `count()`, `capacity()`, and `stacks()` but no connection or `send()` methods. A physically present carrier initiates the transfer through its own cargo API.

PROPERTY-BEARING ITEMS

The canonical value is an `ItemStack`: `.id`, whole-number `.count`, and opaque `.properties`. Properties belong to the item and survive Inventory, bins, warehouses, ports, and cargo. Equal ids merge only when their properties are deeply equal.

`stacks()` returns property-distinct snapshots:

```
for stack in self.output.stacks():
  print(stack.id, stack.count, stack.properties)
```

Use the optional property `dict` on `take` or `send` to select stacks containing that subset:

```
self.output.send(fragment_id, 1, {"forged": True})
```

The fourth argument controls matching. `"exact"` selects one full property identity, including `None` for ordinary propertyless items. `"subset"` selects `dict` subsets, and `"any"` ignores properties. When omitted, a `dict` uses subset matching and `None` uses any-variant matching.

```
self.output.send(fragment_id, 1, stack.properties, "exact")
self.output.send(fragment_id, 1, None, "exact")
```

Without a selector, transfers are deterministic. Most sources take matching stacks in storage order. A Warehouse keeps property-variant groups in storage order, but within the selected exact variant it drains the smallest physical stack first so partially filled duplicate slots are released sooner. Different Warehouse property variants still occupy different physical slots. A Storage Bin latches to one item id but may hold several property-distinct stacks of that id.

PORT API

- `connect(name)` / `disconnect()` / `connected_to()` on routed ports
- `take(item_id, count)` on inputs, with optional property and matching arguments
- `send(item_id, count)` on routed outputs, with optional property and matching arguments
- `count()` / `capacity()`
- `stacks()`

Manual UI actions are separate from port automation. At Nocturna Base, manual Biology uses Inventory. At another outpost, its Manual Stockroom selector binds every card to one local Warehouse. Habitat revival reagents use their own local staging. Port automation follows freight locality like every other machine.

Vehicle handoffs additionally require both vehicles stopped within about 2 m. Inventory freight requires a vehicle at home; stationary-store transfers require it inside the target service area. If a target rejects a transfer, its result names the reason and nothing is lost.

Battery Holder

A **Battery Holder** is a module that mounts in a modular vehicle's Universal slot and hosts **Portable Battery** items. The holder itself stores no energy; it is a rack. The contained batteries hold the charge.

Every battery installed in any holder contributes to the vehicle's single shared battery pool. Uninstalling a battery shrinks the pool immediately and transfers that cell's proportional share of stored charge back onto the battery item. Reinstalling restores only that retained charge, so moving cells never creates free energy. Scripts query the pool through `self.battery`:

```
level = self.battery.level()   # 0-1
wh = self.battery.wh()      # current Wh
cap = self.battery.capacity()   # max Wh = sum of every battery's rated Wh
```

For per-holder detail:

```
for h in self.battery.holders():
  print(h.size, h.wh, "/", h.capacity)
  for b in h.batteries:
    if b is not None:
      print(" ", b.id, b.wh(), "/", b.capacity())
```

VARIANTS

- **Small Battery Holder**, **1** bay
- **Medium Battery Holder**, **2** bays
- **Large Battery Holder**, **3** bays

Each holder occupies **1 Universal slot** on a compatible modular vehicle. Mount/unmount at base or any outpost.

Cargo Rack

A **Cargo Rack** is a module that mounts in a modular vehicle's Universal slot and hosts **Portable Storage Bin** items. The rack itself stores nothing; it is a frame. The contained bins hold the materials.

Every bin installed in any rack contributes to the vehicle's cargo. Each bin holds exactly **one material or item id**: the first unit loaded into it latches that id; subsequent units must match. Property-distinct variants of that id remain separate logical stacks in the bin, so transferring an item never erases its properties.

```
print(self.cargo.count(), "/", self.cargo.capacity())

for r in self.cargo.racks():
  for bin in r.bins:
    if bin is not None:
      print(bin.id, bin.item_id, bin.count, "/", bin.capacity)
      for stack in bin.stacks:
        print(stack.id, stack.count, stack.properties)
```

A bin drains back to **unassigned** when `send()` empties it to zero, ready for any material or item on the next trip.

VARIANTS

- **Small Cargo Rack**, **1** bin slot
- **Medium Cargo Rack**, **2** bin slots
- **Large Cargo Rack**, **3** bin slots

Each rack occupies **1 Universal slot** on a compatible modular vehicle. Mount/unmount at base or any outpost.

Portable Battery

A **Portable Battery** is a rechargeable cell that installs into a Battery Holder's internal slot. Shop-purchased cells arrive at **100% charge**. Installed, the cell transfers its stored Wh into the vehicle's shared battery pool.

All installed batteries discharge together as one pool. Individual `.wh()` tracks the pool's fill percentage scaled by the battery's rated capacity, not an independent charge. Uninstalling a cell transfers its proportional share of the pool back onto that battery item, so reinstalling never creates free energy.

VARIANTS

- **Portable Battery**, **50 Wh** rated capacity
- **Heavy Portable Battery**, **100 Wh** rated capacity

Install or uninstall at base or any outpost.

Portable Storage Bin

A **Portable Storage Bin** is a single-material container that installs into a Cargo Rack's internal slot. On its own, a portable bin stores nothing. Installed, it raises the vehicle's cargo capacity by its rated capacity and accepts one material or item id at a time.

Use item id `"portable_bin"` for the standard bin and `"heavy_portable_bin"` for the heavy bin when calling `self.install(...)`.

The first unit loaded into the bin sets which material or item id it holds; subsequent units must match. When `send()` empties the bin to zero, it unassigns and is ready for any material or item on the next trip.

VARIANTS

- **Portable Storage Bin**, **25** unit capacity
- **Heavy Portable Storage Bin**, **50** unit capacity

Install/uninstall at base or any outpost.

Biosphere, Biomass Tier

The **Biosphere** is the second major terraforming track. Its first tier is **Biomass**: drone crews discover and harvest native life, then outpost machines convert it into planetary biomass.

UNLOCK

Basic Drone Operations opens at 180k Terraform Index, before Biosphere. It exposes drone deployment and control, but every chassis, thruster, station, battery, and Cargo Pod still needs a blueprint earned from Earth Orders and must be fabricated. **Biosphere** opens at 210k and adds the Portable Bio Scanner, Portable Bio Extractor, Essence Liquifier, and Biomass Mixer.

PERMANENT BIOSITES

The Planet Map contains **35 permanent biological sites**: **7 total per biome type across the entire map**, shared among all disconnected patches of that biome. They begin as the same question-mark contacts used for other physical sites and block construction. Each contains **1-3** native life forms; every biome's six life forms are guaranteed somewhere among its seven sites. Locations and contents are identical in every game.

Biological field tools mount only on drones. You don't hunt for the question marks by eye: `get_component("nocturna").points_of_interest()` returns every "?" contact on the map (each with `.x` / `.y` / `.scanned`), so a scanning drone routes straight to the unscanned ones instead of wandering. Fly to a contact with `go_to(x, y)`, then call `self.bio_scanner.scan()`. The question mark becomes a biological marker and the result persists in the Journal. A contact's `.kind` stays `"unknown"` until scanned, so bio-scan each unknown to find the biosites (minerals, vents, and wells share the same map and scan empty for a bio scanner). A harvester drone returns to a biosite coordinate and calls `self.bio_extractor.extract()`.

A Small electric drone uses its Thruster slot for an Electric Thruster and one of its two Module slots for a Battery Pack. That leaves one specialist slot: fit a Scanner or an Extractor for the trip, swap roles while docked, or field two drones. Medium and Large chassis add room later.

The extractor includes a **25 t sealed chamber** that holds one life-form type at a time. Each Cargo Pod holds ONE material: Small 100, Medium 250, Large 500 units, so a drone carries as many materials as it has pods. Use `self.cargo.space_for(item_id)` to check room for a specific material before a trip. `get_component("journal").biomass_coords()` is the restart-safe list of discovered routes; each sample exposes peak `.tons` and current `.remaining_tons`.

DEPLETION AND COOLDOWN

A harvest removes only what the drone can carry. Remaining stock stays at the site for another trip. Cooldown starts only after every life form there is depleted. Common, uncommon, and rare sites use progressively longer cooldowns, then replenish at the same permanent coordinate. Use `journal = get_component("journal")`, then `journal.is_ready(x, y)` and `journal.next_ready_at(x, y)` to schedule rotations.

LOCAL PROCESSING

A Drone Depot stockpile belongs to one outpost. Dock there to unload, then route the sample into an **Essence Liquifier at that same outpost**. The specimen must be native to the outpost's biome. The Liquifier produces only that biome's essence.

Nothing is shared between outposts. Move life forms with drones, and move essence through a physical liquid network of ports, pipes, and tanks. A **Biomass Mixer** consumes the required biome essences and produces biomass tons. Later phases require a larger number of different essences, so progression expands the local station and Liquifier chain into more biomes and brings those liquid lines together at a Mixer.

> The extractor chamber makes a basic harvester functional. Cargo Pods reduce trips but do not replace the local outpost chain.

Plants and Wildlife are the next Biosphere pillars.

Biosphere, Plants

Plants is an active crop chain. The field grows one-shot **Forage** harvests for Wildlife and industry. The **Plant Terraformer fleet** processes that physical stock in visible batches to create permanent Plants km². Nothing in the field increases km² by itself.

1. DISCOVER AND MAKE SEEDS

At **500 t Biomass**, fabricate and deploy a **Seed Maker**. Each `combine(...)` trial consumes **1 t each of three different life forms**. A successful blend places one physical species seed in its result bay and records the recipe in the Flora journal. Repeat known blends whenever you want more seeds.

2. PLANT AND MAINTAIN THE FIELD

Send seeds to home Inventory. Call `load_seed(seed_id)` anywhere on the local grid to put one physical seed in the Harvester's held slot, move to an empty cell, and call `plant(seed_id)`. Sowing finishes after **0.5 hours**. Every species has visible requirements such as light, shade, water, salt, spacing, clusters, companions, or antagonists. Growth pauses safely while any requirement is missing.

Beside means orthogonally adjacent: directly above, below, left, and right. Diagonals never count for plant relationships. Every rule about neighbouring plants reads those four and only those four, so a **cluster** species needs a **2×2 block**, the smallest shape where all four cells have two of their own kind beside them. A row of three leaves both ends stalled forever.

A **Spacer** plant needs all four neighboring cells to be free of **other plants**. Machines, including Grow Lamps, Sprinklers, Dispensers, and Crop Automators, are allowed beside it. For example, a Sprinkler directly to its left does not block growth as long as the plant's other requirements are met. Diagonal plants are allowed, so Spacer plants can form a checkerboard.

Companion and **antagonist** read the same four cells. Fixed Grow Lamps, Sprinklers, and Dispensers also serve the four orthogonally adjacent cells (directly above, below, left, and right); diagonals do not count.

Each crop's Forage rises across its visible growth stages and freezes when ready. The Harvester can provide early manual light, water, and salt care for 24 hours per visit. Later, Grow Lamps, Sprinklers, and Dispensers maintain crop conditions while Crop Automators harvest, plant, and apply treatments. Hover any of these machines on the field to see its service area.

3. BUILD PRODUCTIVE DIVERSITY

Each distinct supported species multiplies crop yield. One productive species gives **×1**, five give **×5**, ten give **×10**, and all fifteen give **×15**. Harder species also have larger base harvest values.

4. GROW LAMP AND SPRINKLER UPGRADES

Each upgrade pack improves one deployed provider. Both machines keep the same four-cell coverage: directly above, below, left, and right. Tiers increase Forage accumulated during growth; they do not change growth speed or add diagonal coverage.

Tier	Forage factor on its own	Power per provider	Sprinkler Water use
Mk I	×1	5 W	2 t/h
Mk II	×2	25 W	4 t/h
Mk III	×4	100 W	8 t/h
Mk IV	×8	500 W	16 t/h

Power draw applies while enabled. A Sprinkler consumes Water while supplied, even with no crops beside it. The crop must require the provider's service: a Grow Lamp boosts light-requiring crops and a Sprinkler boosts water-requiring crops. All of the crop's other requirements must also be met. Providers must be powered, enabled, and supplied. The Dispenser stays at Mk I and supplies salt with no yield bonus.

Only the strongest covering provider for each required condition counts. Two sprinklers do not stack. Bonuses from different required services add above the ×1 baseline, with a **×12 cap on the combined provider factor**. For a crop requiring both light and water, a Mk II lamp plus a Mk II sprinkler gives **×3**: one baseline plus two +100% bonuses. A Mk IV lamp plus a Mk IV sprinkler reaches the **×12** cap.

5. FERTILIZER, GROWTH ACCELERANT, AND YIELD AMPLIFIER

Fertilizer Mk I, II, and III give **×2, ×3, and ×5** base Forage yield on their own for **8 hours per dose**. More of the same tier extends duration; wait for the active dose to expire before switching tiers. These field bonuses are separate from the **10, 30, and 50 potency** values used in Plant Terraformer recipes.

Growth Accelerant gives **×2 growth speed for 8 hours per dose**, reaching the same yield sooner while the crop's requirements are met. Fertilizer and Growth Accelerant time drains while the crop is immature, including while missing requirements stall it, and stops draining at maturity. Yield Amplifier gives **×3** base Forage yield on its own to growing crops across the whole field for **24 hours per dose**; its timer runs continuously. Additional doses extend duration.

Provider, Fertilizer, and Yield Amplifier bonuses add above the ×1 baseline. For example, a Mk II sprinkler (+100%), Mk III Fertilizer (+400%), and Yield Amplifier (+200%) give **×8**, before species diversity multiplies the result. This example requires a water-requiring crop with all its other conditions met. Bonuses apply only to Forage earned while they are active. Already banked Forage stays unchanged, and a mature crop gains no more until it is harvested and replaced.

6. HARVEST AND REPLANT

A ready crop stays in its cell until explicitly harvested. The mobile Harvester works for **0.5 hours**. Each Crop Automator job works for **0.1 hours** at 100% outpost efficiency. Submitting a job returns immediately; one physical executor then holds at most 50 jobs at a time, active and pending together, and processes them in strict FIFO order (first in, first out, so the oldest job runs first). It never runs two jobs at once or bypasses a blocked queue head on its own, and it pauses about **1 second** between jobs while its arm resets. A script can inspect the active and pending work, then use `move_job(job_id, position)` to implement another scheduling policy. Jobs whose targets no longer match fail immediately when they reach the executor and do not spend work time. The mobile Harvester transfers as much as Base Inventory can accept and leaves any remainder banked on the crop. A Crop Automator moves whatever fits in its local output, discards the remainder, and clears the cell. A completely full output blocks the job without changing the crop. Several automators can still work their separate service areas in parallel.

7. RUN THE PLANT TERRAFORMER FLEET

At **2,000 t Biomass**, fabricate and deploy one or more **Plant Terraformers** at operational outposts. Connect each Terraformer's standard `input` to a local Crop Automator output, Storage Bin or Warehouse, or home Inventory, then use `take(...)` or `send(...)` from the source. Connect Water through `water_in`, then call `self.set_enabled(True)`. Item transfers use ordinary feeder cooldowns; the Terraformer handles **16 items per step** at Mk I and **80** at Mk II. Each machine commits the largest proportional loaded batch, performs **3 hours** of work, and delivers km². Mk I stops at **2,250,000 km²**; Mk II handles the final two bands.

> The complete flow is: discover seeds, load and plant, maintain conditions, diversify, harvest and clear, replant, then transfer Forage into the Terraformer.

Plant Terraformer

The **Plant Terraformer** is the only machine that raises permanent Plants km². Deploy as many as you can feed across operational outposts. Every supplied machine adds its tier rate multiplied by that outpost's live efficiency.

INPUT AND CYCLES

After Auto Feeders research, connect `self.input` to home Inventory, a local Storage Bin or Warehouse, or a local machine output such as a Crop Automator. Use `self.input.take(item_id, count)`, or `send(...)` from the source's output. This is the same timed item-transfer model used by other machines: both endpoints stay occupied for the transfer duration. The input has one source connection at a time, so scripts may reconnect it when changing sources.

The Terraformer's destination feeder handles **16 items per handling step** at Mk I and **80** at Mk II, regardless of whether the source pushes or the Terraformer pulls. **Fast Feeders** unlocks at **400,000 Terraform Points** and halves the duration of all newly started timed item transfers. A full Forage load therefore takes about **18.8 seconds** at Mk I and **20.6 seconds** at Mk II before that research, or **9.4** and **10.3 seconds** afterward, when the source is free. Direct Crop Automator to Terraformer transport avoids the extra Warehouse leg; Warehouses remain useful player-controlled staging.

The Terraformer's own onboard holders fit one full Forage batch, the Salt that batch needs, and up to **10 each** of Fertilizer Mk I, Mk II, Mk III, and Growth Accelerant. There is no separate intake buffer. When a cycle can start, the machine commits the largest whole Forage batch supported by every loaded material. For example, 10 Forage with Water for 8 loads a batch of 8. If any required material cannot support one Forage, the machine waits.

All selected items and matching Water are consumed exactly once when the cycle starts. The batch then performs **3 hours** of work at 100% outpost efficiency and delivers its km² in one step. Power loss or `set_enabled(False)` pauses the batch in place. When a cycle finishes, the next affordable loaded batch starts immediately.

TIERS AND FLEET COST

At 100% outpost efficiency, Mk I loads up to **1,200 Forage**, averages **400 Forage/h**, and draws **180 W** while enabled. It handles the first three conversions, then reports `"needs_mk2"` at **2,250,000 km²**. Mk II loads up to **6,600 Forage**, averages **2,200 Forage/h**, draws **900 W**, and adds the Fertilizer and Growth Accelerant injectors. Each machine needs its own fabricated Mk II pack.

CUMULATIVE MATERIAL LADDER

Progress	km² per Forage	Band Forage	Added support
0 to 500,000 km²	20	25,000	none
500,000 to 1,250,000 km²	5	150,000	0.05 t Water
1,250,000 to 2,250,000 km²	5 / 3	600,000	0.002 Salt
2,250,000 to 3,500,000 km²	1	1,250,000	0.004 Fertilizer potency, Mk II
3,500,000 to 5,000,000 km²	1 / 3	4,500,000	0.0001 Growth Accelerant, Mk II

Fertilizer Mk I supplies **10 potency**, Mk II supplies **30**, and Mk III supplies **50**. Each cycle spends stored potency first. To cover a shortfall, the machine consumes a whole-item combination with the least excess potency, then the fewest items. Unused potency stays in the machine for later batches and survives save/load. One Mk III item leaves **23 potency** after a **27-potency** batch. The card includes stored potency in its total; item counts show only unopened items. A full Mk II batch asks for **27 potency**. Growth Accelerant is counted and consumed as whole items; a full Mk II batch asks for **1**.

SCRIPT CONTROL

```
self.input.connect("crop_automator_1")
self.water_in.connect("Water Reservoir")
self.set_enabled(True)

while True:
  result = self.input.take("forage", self.batch_requirements()["forage"])
  print(result.status, self.status(), self.batch_size())
  sleep(1)
```

Salt is consumed as whole items, **1 per 500 Forage rounded up per batch**. A full Mk I cycle consumes **3 Salt**; a full Mk II cycle consumes **14 Salt**. Unused coverage of a rounded-up Salt item does not carry over; unused Fertilizer potency does.

`batch_requirements()` reports Fertilizer as `fertilizer_potency` and Growth Accelerant as an item count. Use `fertilizer_potency(item_id)` to inspect a Fertilizer tier. Item transfers are local to the Terraformer's outpost. Water uses the dedicated `water_in` relationship and, for remote sources, a compatible completed Liquid Pipe route.

Wildlife, Overview

Wildlife counts every established colony's population, housed or awaiting housing: individuals, once born, are on the planet for good, and the sensor never falls. A powered-off or undeployed Habitat pauses that colony's growth, never its count.

END-TO-END LOOP

1. Catalog all 5 fragments of a creature and unlock its feed recipe through its Bio Order.
2. Select it with `self.set_revival_target(creature_id)`. This exposes that creature's two-node tree before revival.
3. Stage the exact species feed and rarity-scaled Biology reagents shown by the Habitat or Journal. Creature bonuses never change this one-time reagent recipe.
4. Call `self.revive()`, keep every active need healthy through rearing, then regulate later gas and liquid ranges as the colony advances.
5. Each completed breeding cycle normally adds **1 individual**. Brood bonuses build toward guaranteed extra offspring across breeding cycles. Other nodes can accelerate a part of the population curve, reward uninterrupted support, reduce feed demand, widen fluid bands, keep the colony on its base fluid instead of a rarer later one, or improve the founding population.
6. Positive population change automatically produces shared **Insight**. Static population and elapsed time produce none. Use `unlock_bonus(node_id)` for the real spending decision; there is no purchase control outside code.
7. At **600,000 Wildlife**, fabricate Habitat Mk II packs. Mk I stops at **175,000**, the top of Thriving. Mk II raises the ceiling to **350,000**, allowing the colony to enter and complete Abundant without changing any breeding statistic.

One Habitat houses one species, with no Wildlife-specific limit on how many Habitats an outpost may hold. Habitats follow the ordinary outpost overcrowding rules, so concentrating them is allowed while distributing them can improve throughput. The species owns its colony progress; the enclosure is replaceable. Undeploying an established colony's Habitat preserves population, life stage, brood progress, Insight history, and bonuses, while pausing growth; its population and species breadth keep counting. Call `self.rehouse(creature_id)` from an empty Habitat whose current capacity fits the whole colony, or use the same call to transfer it directly from another Habitat. Fifteen full Mk II species reach **5,250,000** Wildlife while fourteen reach **4,900,000**. All 16 full Mk II species generate **112 Insight** against the **80 Insight** that all nodes cost together, leaving 32 spare. Feed, gas, and liquid are consumed only for individuals actually produced; a capped colony idles.

Wildlife, Supply (Gases & Liquids)

The harder a colony climbs, the rarer the gas and liquid it demands. Exotic fluids come from a **prospect** → **capture** → **refine** → **meter** chain. Common fluids skip the Refiner; uncommon and rare fluids use raw feedstock plus tar.

FLUIDS BY RARITY

Four gases and three liquids appear in Habitat requirements:

- **Common (tap and use directly):** `swamp_gas`, `ammonia` (gases); `brine` (liquid).
- **Uncommon (raw → Refiner + 2 Tar per batch):** `sulfur_gas` (gas); `cryofluid` (liquid).
- **Rare (raw → Refiner + 5 Tar per batch):** `chlorine` (gas); `quicksilver` (liquid).

Uncommon and rare exotics have a **raw** deposit form (`raw_sulfur_gas`, `raw_cryofluid`, `raw_chlorine`, `raw_quicksilver`) and a **refined** Habitat-ready form. Most colonies never need the rare pair `chlorine` + `quicksilver`, so demand for those deposits stays low.

1. PROSPECT

Exotic deposits follow the same model as thermal vents:

- discover and **survey by sonar** (`"basic"` = active?, `"wide"` = rate, `"deep"` = cycle timing),
- collect with a **cap** (gas vents) or **tap** (liquid springs),
- pipe the output into gas or liquid storage.

Deposits alternate active and dormant phases. Stockpile during the active phase, then feed Habitats from tanks during dormancy. Deep-sonar timing lets you predict dormant phases and size the buffer first. Deposits do not deplete; multiple deposits help cover each other's dormant gaps.

2. REFINE (UNCOMMON / RARE ONLY)

Raw feedstock is not ready for Habitats. The **Refiner** converts raw feedstock + tar → refined exotic via a recipe (unlocked through Biolab orders). Pick the exotic with `self.set_recipe(...)`; wire raw feedstock into `gas_in` / `liquid_in`, tar into `self.input`, and the refined product out of `gas_out` / `liquid_out`.

Tar, the refining reagent

Refining eats **tar** alongside the raw feedstock. Tar is an oil-refining byproduct that otherwise mostly goes to the Waste Processor. Tar cost scales with rarity:

- **Common**, no tar (tapped direct, no Refiner).
- **Uncommon**, 2 Tar per batch.
- **Rare (Chlorine / Quicksilver)**, 5 Tar per batch.

Tar, an item made as a byproduct of oil recipes, is a running cost, not a hard gate. If you run short, expand oil production or slow the Habitat intake until the Refiner catches up.

3. METER

The refined gas/liquid flows into a Gas/Liquid Tank. Your Habitat script meters it in with `self.set_gas_intake(...)` / `self.set_liquid_intake(...)` to hold the band. See the **Husbandry** page.

WHEN EACH STEP MATTERS

A freshly-founded colony needs only **feed**. Mid-stage colonies add a **basic gas/liquid**. Thriving colonies can demand **rare refined exotics**, which pushes you into farther prospecting and heavier refining. Research **Exotic Fluids** unlocks survey/cap/tap + Refiner for uncommon exotics. **Rare Exotics** unlocks at **500,000 Wildlife** and opens the rarest extraction and refining. An early legendary colony may pause at its rare-fluid transition until the broader menagerie reaches that total; this is recoverable and costs no population. Read the Habitat's exact next-stage methods and grow other species while preparing the rare supply.

Wildlife, Husbandry

Husbandry means keeping every active life-support input ready. The worst active input sets `breeding_efficiency()`, a life-support multiplier rather than the population growth rate. Established populations never decay.

REVIVAL SETUP CHECK

Call `set_revival_target(creature_id)` first and branch on its `ActionResult`. It checks the creature, catalog, and one-colony-per-species rule, then saves the target without spending materials. Stage at least **2** correct feed units in `input` and every rarity-scaled Biology reagent in the separate `reagents` input. Then call parameterless `revive()` and branch on `.status` (`"ok"`, `"occupied"`, `"no_target"`, `"species_exists"`, `"not_cataloged"`, `"wrong_feed"`, `"insufficient_feed"`, or `"insufficient_reagents"`). Any rejected result spends nothing and keeps the target selected. Once revival starts, every active input must stay healthy for **12 world hours** of rearing.

RELOCATE AN ESTABLISHED COLONY

The colony belongs to its species, not its enclosure. Undeploying an established colony's Habitat leaves that colony awaiting housing while its population, life stage, brood progress, Insight history, and bonuses remain intact. Its population and species breadth keep counting; only its breeding pauses until it is rehoused. From an empty Habitat, call `result = self.rehouse(creature_id)` and branch on the returned `ActionResult`. The same call can transfer a colony directly from another deployed Habitat. The destination's current capacity must fit the whole colony; `"insufficient_capacity"` moves nothing.

FEED IS AUTOMATIC

`feed_ok()` is `True` when the input bin holds the creature's exact feed. Revival spends one feed and leaves one for rearing. Established colonies consume feed only when individuals are actually born; a full colony idles without consuming it. `feed_level()` reports usable feed after partial consumption, so scripts can restock from local storage without guessing.

GAS AND LIQUID ARE TWO-SIDED BANDS

`gas_band()` and `liquid_band()` return `[low, high]` in tons when active, or `[]` before that input opens. Too little and too much both reduce efficiency. The fluid identity must also match: compare `gas_fluid()` with `required_gas()` and the liquid equivalents. Cutting intake lets an overfilled enclosure bleed back down. For a large excess, explicitly empty that reserve with

`purge_reserve("gas")` or `purge_reserve("liquid")`, then refill it correctly. The discarded fluid is lost, while the colony and its other supplies stay intact.

COMPLETE REGULATOR PATTERN

```
def regulate_fluid(band, level, set_intake):
  if len(band) == 0:
    set_intake(0)
  elif level < band[0]:
    set_intake(5)
  elif level > band[1]:
    set_intake(0)
  else:
    set_intake(1)

while True:
  regulate_fluid(self.gas_band(), self.gas_level(), self.set_gas_intake)
  regulate_fluid(self.liquid_band(), self.liquid_level(), self.set_liquid_intake)

  if not self.feed_ok():
    print("Feed missing or wrong; need " + self.required_feed())
  if self.gas_band() and self.gas_fluid() != self.required_gas():
    print("Wrong gas, need " + self.required_gas())
  if self.liquid_band() and self.liquid_fluid() != self.required_liquid():
    print("Wrong liquid, need " + self.required_liquid())

  print("efficiency", self.breeding_efficiency())
  sleep(1)
```

Connect and stock the Habitat's feed and dedicated `reagents` inputs from local storage. Connect the gas and liquid ports to tanks containing the exact required exotic. Fluid intake setters change policy immediately, but the buffer and efficiency change only after simulation advances and material arrives.

READ THE TRANSITION BEFORE IT HAPPENS

`next_stage_population()` gives the exact threshold and `next_requirement()` names the kind of change. For the actual supply plan, `next_required_gas()` and `next_required_liquid()` name the fluids that will be required immediately after the transition. `next_gas_band()` and `next_liquid_band()` return their exact future `[low, high]` windows. Empty strings or lists mean that input will not be active. This makes prospecting, refining, piping, and regulator tuning possible before the crossing.

RECOVERY

A failed rearing attempt costs the staged materials but never the Habitat. Fix the supply or the script, then retry `revive()`.

For excess gas already inside the enclosure, stop intake and explicitly discard that reserve:

```
self.set_gas_intake(0)
result = self.purge_reserve("gas")
print(result.status, result.message)
```

Then refill with the required gas and resume regulation. For liquid, use `set_liquid_intake(0)` and `purge_reserve("liquid")`. Each purge destroys only the chosen enclosure reserve and clears its fluid identity. It preserves the other reserve, feed, colony progress, inlet buffers, connections, and intake settings. `purge_intake("gas_in")` or `purge_intake("liquid_in")` remains the separate tool for clearing an inlet that holds the wrong fluid.

Habitat Development

Every established colony learns from successful generations. Positive population change adds **Insight** directly to one shared Wildlife balance; elapsed time and static population add nothing. `self.get_insight()` reports the shared balance, this Habitat's current rate, and its lifetime contribution.

TWO PERMANENT NODES PER CREATURE

Each creature has exactly two predefined nodes. Its early **Adaptation** affects that species only and gives strong recurring relief tailored to its life support or breeding pace. Its later **Breakthrough** affects every current and future species. The complete set is deliberately bounded: it improves a well-supported late colony by a few times its untreated output, not by an order of magnitude. Bonuses include speed boosts for different parts of the growth curve, brood bonuses that guarantee extra offspring over time, a reward for one full day of uninterrupted support, feed efficiency, wider fluid bands, a larger founding population, and a higher late-stage rate limit. Rearing always takes 12 world hours. Scripts inspect `.scope` and `.source_species` through `self.get_bonus_tree()` and purchase with `self.unlock_bonus(node_id)`; there is no purchase control outside code.

COSTS AND GATES

The species-only Adaptation has exactly one requirement: **1 shared Insight**. It becomes purchasable after `set_revival_target(...)`, before revival. The all-species Breakthrough has exactly two requirements: **4 Insight** and its source creature reaching **10,000 population**. It does not require the first node, research, a planet-wide Wildlife total, or purchase materials. Every check is atomic; a rejected purchase spends nothing.

WHY MORE HABITATS MATTER

The cumulative Insight curve has diminishing per-individual yield. A creature has generated **3 Insight** at 10,000 population, while its two nodes cost **5** total, so no creature can self-fund its entire tree at the breakthrough gate. The player must pool growth from several species. Full Mk II populations across all 16 creatures generate **112 Insight** against **80** needed to own all 32 nodes, leaving 32 spare. That is a 1.4 times lifetime budget, while 15 full Mk II species still provide 105 Insight. A full or stalled colony cannot mint currency forever.

Wildlife, Progression

Wildlife has three linked progress scales: each colony's life stage, shared Insight, and planet-wide established population.

COLONY STAGES

Current stage	Colony population range	Habitat access
Founded	4-249	Mk I and Mk II
First Breeding	250-2,499	Mk I and Mk II
Self-Sustaining	2,500-24,999	Mk I and Mk II
Thriving	25,000-175,000	Mk I and Mk II
Abundant	175,001-350,000	Mk II only

Both Habitat tiers display this same five-stage ladder. Mk I reaches its **175,000** capacity at the end of Thriving. Mk II unlocks at **600,000 Wildlife** after Rare Exotics and raises capacity to **350,000**, allowing the colony to cross into Abundant. The upgrade changes capacity only. Mature draw is **40-64 W** at Mk I and **144-168 W** at Mk II.

BREEDING

Each creature completes breeding cycles. Population supplies natural momentum: a Common colony produces about **28 individuals/h** naturally at 5,000 population, then the population contribution smoothly flattens toward **150 individuals/h**. Life-support efficiency and rarity change that natural rate: Common **1.0x**, Uncommon **0.75x**, Rare **0.50x**, Legendary **0.30x**. Purchased bonuses improve selected parts of this curve while hard caps keep even a complete tree to a few times untreated output. Some brood bonuses save fractional progress across cycles and eventually guarantee an extra individual. Full-support momentum takes **24 world hours** to reach maximum and loses progress twice as quickly while support is imperfect. Capacity never slows breeding; it only stops growth when the Habitat is full. `breeding_efficiency()` is only the life-support multiplier, not the population rate. Read `breeding_rate()` for expected individuals per hour.

ADAPTATIONS AND INSIGHT

Every species has two independent permanent nodes. Its **1 Insight Adaptation** affects that species only. Its **4 Insight Breakthrough** affects every species and also requires its source colony to reach **10,000 population**. Neither node requires research, Bio Enrichment Compound, planet-wide Wildlife, or the other node. Insight accrues from population growth with diminishing returns per individual; the totals by

milestone are: **1.00** by 10, **1.50** by 100, **2.25** by 1,000, **3.00** by 10,000, **4.00** by 50,000, **5.50** by 175,000, and **7.00** by 350,000.

Only established colonies count toward the planet-wide phases. Failed rearing never touches the sensor, and established population never decreases.

Wildlife, Creature Profiles

Every creature has a feed recipe and, as it climbs, a gas (and for rarer ones, a liquid) requirement. Only one living colony of each species can exist. A colony can be housed at any outpost and later moved without losing progress when the destination Habitat has enough capacity.

FORAGE, THE CALORIE BASE

Every feed builds on **Forage**: physical yield collected when a ready crop is harvested and its field cell is cleared. Keeping feed supplied therefore depends on continued seed production, planting, cultivation, harvesting, and replanting. Recipes use **cross-biome combinations**; across the 16 creatures they use all 30 life-forms, so every biome is worth harvesting. A creature's biome tag does not affect placement or requirements.

HOW TO READ A CREATURE'S NEEDS

- **Feed**, complete the creature's **Biolab order** to unlock its feed recipe, then build it at the **Feed Maker**, whose recipe lists the exact life-form ingredients.
- **Current gas / liquid**, `self.required_gas()` / `self.required_liquid()` name the exact exotic and `self.gas_band()` / `self.liquid_band()` give its target window.
- **Next gas / liquid**, `self.next_required_gas()` / `self.next_required_liquid()` and `self.next_gas_band()` / `self.next_liquid_band()` reveal the exact post-transition supply before the colony crosses its next threshold. Empty strings or lists mean that input will not be active.

RARITY AND REQUIREMENTS

How far a creature climbs is set by its **rarity** (read from the Biology Lab catalog):

- **Common**, mostly feed-based; one common gas; no liquid.
- **Uncommon**, one common gas; a liquid requirement starts at Thriving.
- **Rare**, its gas escalates to a refined gas and its liquid tightens.
- **Legendary**, reaches the rarest exotics at the tightest bands.

The full stage-by-stage table lives on **Wildlife, Progression**. The two legendaries are the hardest to keep in-band, bring one to **Abundant** for the *Apex Husbandry* achievement, and revive, establish, and house all 16 for *Nocturna Reborn*.

Weather System

Weather is a timed resource system operated through Weather Stations and drones. Active storms are visible on Weather and the Planet Map, but an aftermath coordinate must be recovered from live transmissions.

STORM LIFECYCLE

Dust storms and thunderstorms move across the planet and may overlap. Heat and Pressure increase intensity, reward yield, and lightning energy; they do not make storms occur more often.

A storm's transmissions become audible when it starts and remain available until **4 world-clock hours after it ends**. Its aftermath appears only when the storm ends:

- A dust storm leaves **14-28 Raw Uranium** for **48 world-clock hours**.
- An eligible thunderstorm leaves **2-4 Storm Glass** for **96 world-clock hours**. A thunderstorm without an aftermath transmits no Storm Glass message.

The reward remains hidden until a drone reaches its exact coordinate. A nearby or expired location returns `nothing_here` and gives no directional hint.

BUILD RECEPTION COVERAGE

Weather Program installs one completed Weather Station at Nocturna Base. Each powered station draws **50 W**, monitors a **300 m** local radius, and receives broadcast traffic plus the dust channel for its own biome. Additional stations are deployed at outposts.

Dust movement records are divided among all five biome channels, so a complete dust message requires powered reception in every biome. Thunder movement records are broadcast, so one powered station can hear a complete Storm Glass message.

RECOVER THE COORDINATE

`signal_receiver.transmissions()` returns only records audible to that station now. It stores no history, several events may be present at once, and corrupted copies may declare the same packet number as a valid record. Player code must retain what it needs before reception closes.

Records with the same `event_id` belong to one event. Each record declares its `number`, expected `total`, encoded `data`, and `checksum`. The data format is `event_id|number|total|dx|dy`. A valid checksum equals the full sum of the ASCII code of every character in `data`, with no modulo reduction.

A complete set, with every number from 1 to `total` present as a checksum-valid record, describes movement from `(0, 0)`. Apply its `dx` and `dy` records in number order; the endpoint is the exact aftermath coordinate. DOCS entries for `SignalTransmission` define every field. Player code decides how to retain records, share them between stations, and track incomplete events.

COLLECT THE AFTERMATH

After the storm ends, send a drone with cargo space to the recovered coordinate and call `collect()`. One successful collection transfers at most **5 units**, so a larger aftermath may need repeated collections or more than one trip. Raw Uranium adds exposure unless the drone has Shield Plating; Storm Glass is not radioactive. A discovered live aftermath then appears on the maps until it is exhausted or expires.

WHAT EACH SURFACE DOES

- **Incoming** automatically summarizes storms expected to enter powered-station coverage. It does not require `observe()` and never reveals an aftermath.
- `observe()` measures local active storms and forecasts. `last_report()` reads that station's latest report.
- `signal_receiver` supplies the raw records used to recover an aftermath's coordinate.
- `signal_board` is an optional per-station display. It presents only records and labelled values published by player code; it does not validate, merge, store, or decode them.
- `strikes()` provides bounded local lightning history. Lightning Rod capture works independently of Weather Station scripts.

The required process is reception, retention, validation, assembly, coordinate recovery, and collection. The storage, coordination, presentation, and dispatch program remain yours to design.

Flow Networks, Fluids

THE COMPLETE MENTAL MODEL

Gas and liquid use the same transport model. Gas Pipes carry registered gases; Liquid Pipes carry registered liquids. Always answer three questions in order:

1. **Can the two locations reach each other?** Completed physical topology answers this.
2. **Which machines intend to exchange which exact substance?** `FluidPort.connect(...)` relationships answer this.
3. **How much can move now?** Supply, demand, power, throttle, buffer headroom, and shared throughput answer this.

A useful shorthand is: compatible connection + reachable topology + live supply + live demand = flow. The connection and topology establish identity. Live conditions only change the current t/h rate, so zero flow does not make a pipe neutral.

WHAT `CONNECT()` MEANS

Players connect machine ports to machines, never ports to pipe ids:

```
# Water Pump script
self.water_out.connect("bio_caster_1")
```

```
# Equivalent when written in the Bio Caster script
self.water_in.connect("water_pump_1")
```

Either side may declare the provider-consumer relationship, and only one side is required. The target can be a stable machine id or display name. The target must expose a compatible opposite-direction port: `water_out` matches `water_in` or generic `liquid_in`; `steam_in` matches `steam_out` or generic `gas_out`.

Each port stores one declared target. Calling `connect()` again on that port replaces its previous target. `result.status == "ok"` means the target exists and has a compatible port. It does not guarantee that a remote pipe is complete, that the source has supply, or that flow is nonzero.

`disconnect()` clears only the declaration owned by that port and leaves buffered fluid in place. `connected_to()` reports only the target declared by that port; it does not list reverse declarations made by peers. If both ends independently call `connect()` for the same pair, transport is deduplicated, but both declarations must be disconnected to remove the relationship completely.

There is no `connect_pipe`, `disconnect_pipe`, or `connected_pipe` API. Pipe ids are for construction and diagnostics.

LOCAL VERSUS REMOTE

Machines in the same completed outpost transfer directly after a compatible `connect()` call. They do not need a local pipe piece, but they still need the machine connection.

Machines at different locations need at least one completed conflict-free pipe network of the correct medium reaching both location anchors. A map machine such as a Pump or Thermal Cap uses its own visible utility footprint. An internal machine uses its completed outpost footprint as its anchor.

A remote `connect()` call may produce `.status == "ok"` before the pipe exists. That records durable intent. Flow begins when suitable topology is complete. Cutting the pipe makes the route unreachable but does not erase the connection; restoring topology resumes it.

FOOTPRINTS EXPOSE AVAILABILITY, NOT STORAGE

A pipe touching an outpost service footprint exposes that pipe network to compatible connections owned by machines there. Personnel distribution does not change utility reachability. The outpost stores nothing, mixes nothing, and selects no source or tank automatically. Every local machine still needs an explicit `connect()` relationship.

Separate pipe networks touching the same outpost service footprint remain separate; footprint contact is an availability relation and never a fluid junction. This lets water, oil, and volcanic essence reach one outpost through independent liquid pipe networks without becoming one pool. A map readout listing several liquids means those identified pipe networks reach the footprint; it does not mean the liquids are mixed.

One remote relationship uses at most one pipe network. An existing valid non-conflicting assignment is retained. A new relationship uses a pipe network already carrying that exact fluid, or one neutral pipe network. If exactly one pipe network serves simultaneous different-fluid relationships, both connector claims conflict there. Independent pipe networks never combine capacity automatically.

HOW A PIPE GETS EXACT CONTENTS

Pipe hardware establishes only `gas` or `liquid`. A completed pipe network gets an exact substance only from complete compatible provider-consumer relationships assigned to that pipe network.

- No connection: neutral.
- Provider present but unconnected: neutral.
- Consumer present but unconnected: neutral.

- One complete water connection: contents are `water`.
- Several water connections: compatible, they share supply and throughput.
- Water and oil relationships simultaneously have only one eligible pipe network: both claims conflict and all flow on that pipe network stops.
- Separately designated Water and Oil pipe networks are physically joined: conflict, all flow on the joined pipe network stops.
- A water connection with throttle at zero, no power, no supply, or a full sink: contents remain `water`, live flow is zero.
- Remove the final complete connection: the pipe network becomes neutral again.

The same rule applies to gases. A dormant Steam source still designates its pipe network while its connection exists. If a new Ammonia relationship has only that Steam pipe network, the overlapping connector claims conflict. If separately designated Steam and Ammonia pipe networks are physically joined, the joined pipe network also conflicts even while both sources are idle.

If several pipe networks reach the same two anchors, each relationship selects only one. A second exact fluid can designate another neutral pipe network without affecting the first. If exactly one pipe network is eligible for both exact fluids, their simultaneous connector claims conflict on it. Independently assigned fluids also conflict if their pipe geometries are later physically joined.

PHYSICAL JOINS

- Only completed pipe and bridge hardware transports fluid.
- Gas and liquid hardware never join.
- Same-medium ground pieces join wherever their real geometry touches.
- T shapes and ordinary ground crossings are real junctions.
- Components contacting one service footprint remain physically independent unless their own ground geometry touches.
- Deliberately building a real ordinary junction in the ground geometry joins the routes.
- A bridge joins its matching ramps along the bridge axis.
- A perpendicular ground line below the bridge's middle tile remains separate.

`Pipe.start()` and `Pipe.end()` report construction geometry only. They do not force live flow direction. Port direction, supply, and demand decide live movement.

TANKS ARE EXPLICIT STORAGE

An empty Gas Tank accepts any gas; an empty Liquid Tank accepts any liquid. Connecting an exact source to an empty tank establishes the route identity before delivery. The first exact fluid actually deposited latches the tank. It then accepts and provides only that fluid until it drains fully to zero and unlatches.

A tank is not inserted automatically by the outpost. A source-to-tank-to-consumer chain needs two relationships:

```
# Water Pump script, fill storage
self.water_out.connect("Water Reserve")
self.set_throttle(1.0)
```

```
# Consumer script, draw from storage
self.water_in.connect("Water Reserve")
```

The tank itself is passive and needs no control loop. Draining it to zero does not erase its port declarations. If it later latches to a different liquid or gas, an old incompatible declaration cannot flow and should be reconnected.

Two empty generic tanks connected only to each other remain neutral because neither side identifies an exact substance.

ONE PROVIDER, SEVERAL CONSUMERS

Each consumer can connect its own input to the same provider:

```
# Bio Caster script
self.water_in.connect("water_pump_1")

# Sprinkler script
self.water_in.connect("water_pump_1")

# Reactor script
self.water_in.connect("water_pump_1")
```

All three relationships participate. Available supply, destination headroom, and physical throughput are shared without list-order priority. Use separate tanks, topology, connection choices, or throttles when one consumer must have deliberate priority.

PIPE THROUGHPUT CEILING

Individual pipe pieces have no throughput capacity of their own. A pipe network starts with **2,000 t/h** of shared throughput for each matched source-side and sink-side attachment link. The common layout with one source link and one sink link therefore starts at **2,000 t/h**. **High-Pressure Fluid Transport** research retrofits every completed and future Gas Pipe and Liquid Pipe attachment link to **6,000 t/h**. Pipe length and extra pieces within the same route do not reduce or increase that limit.

Every compatible relationship assigned to the pipe network shares this budget. Multiple attachment links on both sides can raise aggregate capacity, but extra length or interior branches alone add nothing. One logical connection never pools independent pipe networks. Machine production, throttle, supply, consumer demand, and buffer headroom may set a lower live rate; stored bursts or several producers can still reach the ceiling.

READING THE APIS

`FluidPort.level()` is tons buffered at that port. Some pass-through outputs, including Pumps, store nothing and therefore read **0**. `capacity()` is the port's buffer limit. `flow_rate()` is the current live rate in t/h; **0** can mean idle, without supply, full, unreachable, conflicted, or merely waiting across the simulation timing boundary.

```
print(self.water_in.connected_to())
print(self.water_in.level(), self.water_in.capacity())
print(self.water_in.flow_rate())
```

Pipe diagnostics show the physical side:

```
for pipe in list_pipes():
    print(pipe.id, pipe.type(), pipe.contents(), pipe.state())
    print("rate", pipe.flow_rate())
    print("connections", pipe.connections())
    print("conflicts", pipe.conflicting_contents())
```

`type()` reports `"gas"` or `"liquid"`. `contents()` reports the exact established substance, or `None` while neutral or conflicted. Because both cases return `None`, check `state()` and `conflicting_contents()` too. `connections()` lists the complete logical routes using the pipe network; the representative pipe id in each diagnostic row is not a connection handle.

TROUBLESHOOTING ZERO FLOW

Check these in order:

1. Does the expected port's `connected_to()` show its own declaration?
2. Did the `connect()` result have `.status == "ok"` instead of `"not_found"` or `"incompatible"`?
3. For remote machines, is every intended pipe or bridge piece complete?
4. Does a pipe network of the correct medium reach both anchors?
5. Does `Pipe.state()` report `"conflict"`?
6. Is the source powered, supplied, active, and throttled above zero?
7. Does the destination have headroom or current demand?
8. Is shared throughput already being consumed by other routes?

Stored fluid redistributes before power, so generators may use tank delivery in that power tick. Powered Pumps and Caps release after the grid check. Later processing may use that release in the same tick, but a generator uses newly pumped or capped fluid on the following power tick. A one-tick delay can therefore be normal.

INVALID MACHINE LOCATIONS

A machine with a missing, stale, or unfinished outpost location fails closed. It does not borrow home connectivity. Outpost deconstruction should never leave such a machine behind; this rule prevents corrupted state from receiving hidden utility access.

CROSS-REFERENCES

- Infrastructure & Pipes, construction, physical joins, and bridges
- `FluidPort`, exact API return values
- `Pipe`, physical diagnostics
- Thermal Vents, steam source behavior
- Wells, water and oil source behavior

Infrastructure & Pipes

OVERVIEW

Infrastructure is physical hardware built on the Planet Map. Gas pipes carry gases and consume `gas_pipe_segment` items. Liquid pipes carry liquids and consume `liquid_pipe_segment` items. Hardware establishes only the medium; complete player-authored provider-consumer connections establish exact contents.

BUILDING A PIPE

Plan Mode and `construction_blueprint.plan_pipe(...)` create tile-based Construction jobs. A Pioneer with a Constructor executes each job:

```
bpq = get_component("construction_blueprint")
bpq.plan_pipe("liquid", 0, 0, 80, 0)

for job in bpq.pending_constructions():
    if job.kind == "pipe":
        self.nav.set_target(job.position.x, job.position.y)
        self.nav.set_throttle(1)
        while self.nav.get_distance_to(job.position.x, job.position.y) > 2:
            pass
        self.nav.brake()
        build = self.constructor.execute(job.id)
        print(build.status, build.message)
```

Unbuilt pieces are ghosts and transport nothing.

PHYSICAL TOPOLOGY

Touching geometry connects. Same-medium segments connect wherever their tile-aligned paths touch. Ordinary T and cross shapes are real junctions.

Service-footprint contacts stay independent. Two fluid pipe networks do not merge merely because both touch the same outpost. The outpost footprint exposes each pipe network independently to connected local machines. This allows water, oil, and other routes to touch one outpost without becoming one physical network.

Connections define contents. Provider presence alone and consumer presence alone are neutral. Each complete compatible remote `connect()` relationship selects at most one pipe network reaching both locations and establishes its exact fluid there, including while idle, full, powered off, or throttled to zero.

Conflicts halt flow. If exactly one pipe network serves simultaneous water and oil relationships, both connector claims conflict and that pipe network stops. Separately designated pipe networks produce the same conflict if their physical geometry is joined. The same applies to different gases. Same-fluid connections may share one pipe network and its capacity.

Machines do not connect to pipe ids. Scripts connect providers and consumers. Local machines transfer directly; each remote relationship keeps its current valid, non-conflicting completed pipe network reaching both locations; failing that, it selects one already carrying the same fluid, and failing that, a neutral one. Independent pipe networks never pool capacity automatically.

Bridges pass over lines without joining them. Gas, liquid, and power bridges are 3-tile overpasses. Their own axis connects; a perpendicular line beneath the middle tile remains a different network.

THROUGHPUT

Throughput belongs to the pipe network, not its segments. Individual pipe segments have no capacity stat. The common layout with one source link and one sink link starts at **2,000 t/h**, shared by every compatible relationship assigned to that pipe network. **High-Pressure Fluid Transport** research retrofits every completed and future Gas Pipe and Liquid Pipe attachment link to **6,000 t/h**. Pipe length and extra pieces within the same route do not reduce or increase the limit. Multiple attachment links on both sides can raise aggregate capacity, while machine rates, throttle, supply, demand, and headroom may impose a lower live rate.

INSPECTING PIPES

```
for pipe in list_pipes():
    print(pipe.id, pipe.type(), pipe.contents(), pipe.state())
    print("connections", pipe.connections())
    print("conflicts", pipe.conflicting_contents())
```

These pipe ids are inspection and construction identities, not player fluid-port handles. On the Planet Map, flowing networks use the exact fluid color, stalled networks are amber, and conflicts are red.

MATERIALS AND RECOVERY

Pipe segment cost is based on route length, approximately one segment per 10 meters. If construction pauses, query `get_component("construction_blueprint").paused_constructions()` and execute the same job again within range. Splitting or removing a route recomputes physical reachability immediately while preserving player-authored machine connection intent.

CROSS-REFERENCES

- Flow Networks, transport, throughput, and backpressure behavior
- Power Networks, wire topology and power bridges
- Constructor Module, `self.constructor.execute(...)`
- Construction Blueprint, script-side planning
- Thermal Vents and Wells, physical producers

Power Networks

OVERVIEW

Completed power lines divide the planet into connected subnets. Every generator, consumer, and battery in one subnet shares that subnet's generation and stored energy. A subnet may run from generation, stored energy, or both; it does not require a local or currently active generator while battery energy remains.

PHYSICAL TOPOLOGY

Touching lines connect. Power-line pieces join wherever their tile-aligned geometry touches. T-junctions and ordinary crossings form one subnet.

Outposts expose power service footprints. Every completed power piece touching an outpost's power service footprint joins that structure bus and therefore the same subnet. Every valid completed machine belonging to that outpost uses the same subnet automatically. Personnel distribution does not affect utility connectivity.

Power Bridges pass over lines without joining them. A Power Bridge connects along its own axis. A perpendicular ground-level power line passing under the middle tile remains a separate subnet.

Lines have no direction, loss, or per-line watt limit. Generation, storage, and demand are pooled across the connected subnet. Direction emerges from which machines generate and consume power.

Incomplete work carries no power. Blueprint ghosts and partially built pieces are visible planning state only. Machines with a missing or stale outpost location are invalid and receive no power at all; they are never moved to another subnet.

ALLOCATION AND STORAGE

Generation serves the subnet's complete live demand first. Powered conventional batteries form one capacity-weighted pool: they charge from surplus and discharge before Lightning Rod reserve. Lightning Rods never charge from ordinary grid surplus and are not included in the main battery Stored total.

There is no consumer priority or partial load shedding. If generation plus conventional batteries and Lightning reserve cannot fund the full live load for the current tick, every live consumer on that subnet pauses before work runs. Canceled work does not spend partial storage. If available energy covers the tick exactly, that tick completes and the next unfunded tick pauses the grid. Displayed W and Wh values may be rounded, but small internal charge and drain increments continue to accumulate.

BUILDING AND RECOVERY

Plan power lines and Power Bridges in Plan Mode or by using `construction_blueprint.plan_power_line()` and `construction_blueprint.plan_bridge()`, then have a Constructor-equipped Pioneer drive within interaction range of each job and call `self.constructor.execute(job.id)`. If a route is split by deconstruction, the surviving pieces immediately become separate subnets. Machines paused by an outage return together when live generation covers the full returning load, or when conventional batteries plus Lightning reserve hold one hour of the remaining deficit. Newly banked Lightning energy can recover a dark subnet directly. Manually powered-off machines do not join automatic recovery.

DIAGNOSING A SUBNET

Use the Planet Map to follow highlighted connected pieces and the Power dashboard to compare generation, demand, and battery storage. Lightning Rod cards and their category total show the separate deep reserve. If a remote outpost is dark, verify that every intended wire piece and bridge is complete, that the ground-level wire pieces touch both service footprints, and that the subnet has enough current generation or stored energy for its complete live load.

CROSS-REFERENCES

- Infrastructure & Pipes, construction and bridge crossing rules
- Batteries, storage behavior
- Constructor Module, executing Planet Map jobs

Thermal Vents

OVERVIEW

A **thermal vent** is a fracture in the planet's crust where internal heat leaks through as steam. Vents are discovered by sonar and harvested by placing a Thermal Cap blueprint on them. Each vent cycles between active and dormant phases.

DISCOVERY

`self.sonar.scan()` returns `SonarScanResult`. When `.status == "ok"`, iterate `.sites` and select contacts whose `kind() == "thermal"`; `.message` explains any rejected scan. Survey the selected contact with `self.sonar.survey(site)` and read the `SurveyResult.site` payload only after its status is `"ok"`. Basic Sonar reveals phase, Extended adds steam rates, and Deep adds cycle timing.

HARVESTING STEAM

Plan a Thermal Cap on the vent, carry its kit in the Pioneer, and execute the queued blueprint with the Constructor. The completed cap captures steam into its chamber while powered.

ROUTING AFTER CAPTURE

Establish the connection either by calling `self.steam_out.connect(...)` in the cap script with a compatible consumer's stable machine id or display name, or by connecting a compatible consumer's steam input to the cap's stable machine id or display name. Nothing leaves until the cap script opens its throttle with `self.set_throttle(value)`, where `value` is **0-1**. Every target needs a compatible completed gas-pipe route from the cap to the target or its outpost, even when it is nearby. The game selects the route automatically; scripts never select an individual pipe. The complete connection establishes steam identity even before flow starts and keeps it while idle, full, off, or throttled to zero.

Under-release and the chamber climbs. At **100%** it overpressurizes and blows off the whole chamber, then refills from empty. Read `pressure()`, `is_overpressured()`, and `is_stalled()` to control release, add storage, or restore physical reachability.

CROSS-REFERENCES

- Flow Networks, what happens after capture
- Infrastructure & Pipes, physical routing
- Construction Blueprint, queued build work

Tier 3 Progression

OVERVIEW

Tier 3 introduces **input-gated** upgrades. A supplied Mk III Oxygen or Pressure Generator runs at **200x** Mk I output, while a supplied Mk III Heat Generator runs at **208x**. Starved machines fall back to their Mk II base output: **5x** for Oxygen, **25x** for Pressure, or **4.7x** for Heat.

The gate is enforced per tick: every frame the machine checks its input buffer. If the required fluid is unavailable, `is_degraded()` reads `True` and `effective_tier()` falls back to the previous tier for that tick. The installed tier reported by `tier()` itself never changes.

THE THREE MK III PACKS

Each pack adds a required fluid port and consumption rate:

- **Oxygen Mk III**, `water_in`, **8 t/h**
- **Heat Mk III**, `steam_in`, **12 t/h** (direct steam, not water)
- **Pressure Mk III**, `water_in`, **5 t/h**

After applying a pack, the machine gains the corresponding port:

```
oxy = get_component("o2gen_1")
oxy.water_in.connect("liquid_tank_1")
print(oxy.water_in.level(), "/", oxy.water_in.capacity())
```

DEGRADATION SIGNALS

- `machine.is_degraded()` returns `True`.
- `machine.effective_tier()` returns one less than `machine.tier()`, the tier actually in effect right now.
- `machine.tier()` itself never changes, the pack is permanent; degradation is runtime.

When the input buffer again holds one tick's required consumption, all of the above flip back on the next tick.

CONTENTION

Early water comes from **Water Pumps on wells** (a well yields **10 / 20 / 30 t/h** by tier). Your downstream consumers at full Mk III:

- 1 Oxygen Mk III at full rate consumes 8 t/h of water.
- 1 Pressure Mk III consumes 5 t/h of water.
- Combined at full rate: 13 t/h of water.

A single standard well (10 t/h) can't feed both at once. Your options:

1. Tap a **richer well** (rich / pure yield 20 / 30 t/h) or add a **second well** with its own pump.
2. Buffer with a **Liquid Tank** so short demand spikes draw from storage.
3. Allocate via script, prioritize whichever Mk III matters more right now, let the other degrade to Mk II temporarily.
4. After **Steam Condensation** unlocks at 1,000,000 Plants, route stored vent steam through a **Steam Condenser** for another 250 t/h Water at full throttle.

ALLOCATION SCRIPT

A water allocator runs forever and decides which Mk III gets fresh water based on terraforming progress:

```
o2_tank = get_component("liquid_tank_1")
oxy = get_component("o2gen_1")
pres = get_component("pressure_1")
atmo = get_component("atmosphere")

while True:
  # Prioritize whichever pillar has the larger gap to next phase
  oxy_gap = ...  # compute from atmo.oxygen and next phase threshold
  pres_gap = ...
  if oxy_gap > pres_gap:
    # Favor oxygen, route the current water supply to its tank first
    ...
```

Choose the allocation rule that matches your current terraforming goal. Add `sleep()` only if you intentionally want a slower allocator cadence.

HEAT IS DIFFERENT

Heat Mk III takes steam **directly**, not water. Captured steam can feed any combination of three sinks:

- A **Steam Turbine** for grid power.
- **Heat Gen Mk III** for maximum heat output.

- A **Steam Condenser** for clean water used by Oxygen, Pressure, Plants, and other consumers.

One vent may supply several connected sinks when its average output and stored reserve can cover them. Reachable consumers share available flow; their throttle, demand, and buffer headroom determine what they accept. Gas Tanks are the practical way to bank active-phase steam and carry every branch through dormancy. Condenser-fed Oxygen and Pressure therefore compete indirectly with turbines and Heat for the same captured steam.

GRACEFUL FALLBACK

Degradation is a normal running state. The machine doesn't stop; output continues at the previous tier until the input buffer again holds one tick's required consumption. Scripts can observe `is_degraded()`, inspect the input port, and correct the supply chain.

CROSS-REFERENCES

- Flow Networks, the input system Mk III uses
- Thermal Vents, steam capture, storage, power, and condensation

Water & Oil Wells

OVERVIEW

The planet has deterministic **water wells** and **oil wells**. Water is steady; oil pulses between active and dormant phases.

Water wells produce **10 / 20 / 30 t/h** by yield tier. Active oil wells produce **8 / 16 / 24 t/h** and **0** while dormant.

READING WELLS IN SCRIPTS

```
scan = self.sonar.scan()
if scan.status == "ok":
  for contact in scan.sites:
    if contact.kind() == "water" or contact.kind() == "oil":
      survey = self.sonar.survey(contact)
      if survey.status == "ok":
        site = survey.site
        if site.kind() == "water" or site.kind() == "oil":
          print(site.id, site.kind(), site.yield_tier(), site.flow_rate())
      else:
        print(survey.message)
else:
  print(scan.message)
```

`scan()` returns `SonarScanResult`; `survey()` returns `SurveyResult`. Their payload fields are available only for the appropriate `.status`.

A deployed Pump starts at throttle **0**. Connect its output port, check the connection `ActionResult`, then set throttle. Local targets transfer directly; remote targets use a completed Liquid Pipe route. A Pump exposes its bound well through the read-only `well()` query.

Components

Everything on this planet is a component: sensors, generators, scanners, vehicles, storage bins, smelters, the planet itself, your field journal.

There are two ways to access them:

`self`, your own machine. Every script has a `self` variable that refers to the machine it runs on. You can read state and control it.

`get_component(id)`, a shared component or another machine by its concrete instance id. Read shared state from anywhere:

```
clock = get_component("clock")
print(clock.get_elevation())
```

For instanced machines, the id is the stable programmatic identifier such as `"bio_collector_1"`, `"rover_1"`, or `"storage_bin_2"`. The card title is the display name and can be renamed; use the id in long-running scripts so renames do not break references:

```
rover_id = "rover_1"
rover = get_component(rover_id)
print(rover.nav.get_position())
```

Use the exact id from the machine card. Type names like `"bio_collector"` describe a kind of machine; they are not the same thing as the deployed machine id `"bio_collector_1"`.

EACH CALL IS A NEW INSTANCE

`get_component(id)` is a factory, every call creates a fresh wrapper, like instantiating a Python class. For machines tied to a physical entity (rover, smelter, generator), wrappers all read and write through the same backing data, so it feels singleton-ish in practice: setting a value from one wrapper is visible from another.

For purely virtual components like the `transmitter`, each wrapper is independent:

```
# Fails, two separate transmitters, only the first is connected:
get_component("transmitter").connect("earth")
get_component("transmitter").transmit("weather", 42)

# Works, one transmitter, used twice:
t = get_component("transmitter")
t.connect("earth")
t.transmit("weather", 42)
```

> Wrappers are just objects. Make one, make ten, each is independent.

STORAGE BINS BY NAME

Storage bins are special, you label them with the material they hold (`"Iron Ore Bin"` , `"Copper Ingot Bin"`), so querying by that label reads naturally:

```
ore_bin = get_component_by_name("Iron Ore Bin")
if ore_bin.fill_percent() < 0.2:
  print("Ore running low!")
```

`get_component_by_name()` works for any uniquely named machine or outpost, but names are editable. Custom Panels share the rename namespace but are not components, so panel names are not returned here. For long-running scripts, store and use the stable id when you can.

FLEET LOOKUP

Use `fleet` when a script needs to monitor all vehicles or drones without hardcoding names:

```
fleet = get_component("fleet")
for unit in fleet.vehicles():
  print(unit.id, unit.name, unit.battery_level)
```

Fleet refs are read-only snapshots. Station APIs accept either `.id` or `.name` ; ids are safer for rename-proof scripts. Use `get_component(id)` when you need the full component.

THE PLANET AND YOUR JOURNAL

Two components represent the world you're exploring: the **planet** and your **journal**.

`nocturna` is the planet itself, intrinsic data, the same for any observer:

```
planet = get_component("nocturna")
print(planet.get_name())       # "Nocturna"
b = planet.get_bounds()
print(b.min_x, b.max_x, b.min_y, b.max_y)
```

`journal` is *your* record of what you've explored, sonar writes to it, you read from it:

```
log = get_component("journal")
for site in log.discovered_sites("nocturna"):
  print(site.id, "at", site.x, site.y)

for site in log.surveyed_sites("nocturna"):
  if site.kind() == "mineral":
    print(site.id, site.item_id, site.purity)
```

Passing the planet id scopes the query, future planets will work the same way.

READ VS CONTROL

You can read shared components from any script, check the clock, inspect inventory, or monitor a generator by its instance id.

But hardware actions belong to the machine's own script via `self`. Methods marked SELF ONLY in DOCS won't work on another machine through `get_component()`, you'll get an error explaining why.

Every component has `.id` (programmatic identifier) and `.name` (display name). Check the Components section below for all available components and their methods.

COMMANDS

System

`set_status(message: str, level: str = "info") → None`

Show a status message for the current script run. Accepts non-empty plain text up to 240 characters and a level of `info`, `warn`, or `error`. Replaces the previous message immediately without waiting, logging, changing machine behaviour, or raising a game warning. Paused, completed, and failed scripts show their last report as inactive. Stop, restart, and save loading clear it. Invalid text or levels raise `ValueError`.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

```
set_status("Waiting for iron ore", "warn")
```

`clear_status() → None`

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`get_status_report(script_id: str | None = None) → ScriptStatusReport | None`

Read a snapshot of a script's latest status report. Omitting `script_id` reads the current script. Returns `None` when the script has no report, has stopped, or does not exist. Reports from paused, completed, or failed runs have `active` set to `False`.

PARAMETERS

Name	Type	Description
`script_id`	`str` | `None`	Stable script id, or `None` for the current script.

RETURNS

`ScriptStatusReport | None`

`get_game_version() → str`

Read the current game's build identifier, matching the version at the bottom right of Settings and in feedback reports. Available before boot and from shared Libraries on every platform. Use it to identify an exact build when sharing scripts; build hashes cannot be compared as newer or older versions.

RETURNS

`str`

The seven-character commit hash, or `"dev"` in an unstamped development or test environment.

```
print(get_game_version())
```

`boot() → ActionResult`

Initialize the system and run diagnostics.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"booting"`	● success	The boot sequence has started.
`"already_booted"`	● success	The system is already booted.

```
boot()
```

`activate_power() → ActionResult`

Turn on the power grid after the station has finished booting.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"activating"`	● success	Activation has started.
`"already_online"`	● success	The system is already online.
`"boot_required"`	● rejection	The station must finish booting before this operation is available.

```
activate_power()
```

`activate_sensors() → ActionResult`

Bring the sensor array online. Requires power.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"initializing"`	● success	Initialization has started.
`"already_online"`	● success	The system is already online.
`"power_required"`	● rejection	The operation requires an online power system.

```
activate_sensors()
```

`get_component(name: str) → Component | None`

Access a player-owned entity by its **immutable id** (e.g. `"solar_3"`, `"outpost_home"`). The id is auto-generated on creation and never changes: use this in scripts that need to outlive renames. To look up by display name (mutable), use `get_component_by_name(name)`. Returns `None` if no entity has the given id.

PARAMETERS

Name	Type	Description
`name`	`str`	Entity id (machine or outpost)

RETURNS

`Component | None`

```
clock = get_component("clock")
time = clock.get_time()
print(time)
```

`get_component_by_name(name: str) → Component | None`

Look up any addressable player-owned component (machine or outpost) by its display name. Names default to the entity's id but can be freely renamed from the Computer System tab; uniqueness is enforced across the shared rename namespace. Custom Panels share that namespace but are not components, so they are not returned here. Mutable: `get_component(id)` is the stable form for long-running scripts. Returns `None` if no component has the given name.

PARAMETERS

Name	Type	Description
`name`	`str`	The entity's display name

RETURNS

`Component | None`

A machine or an outpost. `None` when no component has that name.

```
ore_bin = get_component_by_name("Iron Stockpile")
print(ore_bin.count("iron_ore"))
```

COMMANDS

Infrastructure

`get_pipe(pipe_id: str) → Pipe | None`

Look up an infrastructure pipe by id. Returns a live read-only `Pipe` handle, or `None` if no pipe by that id exists.

PARAMETERS

Name	Type	Description
`pipe_id`	`str`	Pipe id (from `list_pipes()`)

RETURNS

`Pipe | None`

A live, read-only handle.

```
pipe = get_pipe("pipe_1")
if pipe != None:
    print(pipe.state())
```

`list_pipes() → list[Pipe]`

List every infrastructure pipe currently laid (complete or in-progress) as live read-only `Pipe` handles.

RETURNS

`list[Pipe]`

Live, read-only handles.

```
for pipe in list_pipes():
    print(pipe.id, pipe.state())
```

Built-in Functions

OUTPUT

`print(*values: object, sep: str = " ", end: str = "\n") → None`

Output text to the console. Multiple values are joined by `sep` (default a single space). `end` is appended after the last value (default a newline). Console output is a character stream and the newlines in it are what break lines, so `end=""` leaves the line open and the next `print()` continues it: use that to build a row from several calls, then close it with a bare `print()`. Every call updates an unfinished line immediately. Within that row, `\r` returns the write position to the start and `\b` moves it back one visible character. Neither control erases text by itself; following text overwrites existing text. Neither control can enter an earlier row, and ANSI escape sequences are not interpreted.

RETURNS

`None`

`warn(*values: object, sep: str = " ", end: str = "\n") → None`

Output an amber warning line to the persistent console. Same argument behavior as `print()`, but routed to the WARNINGS filter. Use it for background monitors that need attention without showing a toast. Use `notify(text, "warn")` when the player should be interrupted.

RETURNS

`None`

`debug(*values: object, sep: str = " ", end: str = "\n") → None`

Output low-priority telemetry to the persistent console. Same argument behavior as `print()`, but hidden from the ALL view unless debug output is enabled in console options. Use it for noisy tuning data that should not crowd normal logs.

RETURNS

`None`

`notify(text: str, /, level: str = "info", duration_seconds?: float, dismissible: bool = True) → None`

Show a toast to the player and add it to the **Computer → Notifications** archive. Use sparingly: for events that genuinely need the operator's attention (battery critical, contract solved, drone stranded); prefer `print()` for ongoing telemetry. `level` is `"info"` (default), `"warn"`, or `"error"` and drives the toast's color + the history entry's color dot. `duration_seconds` sets how long the toast stays before auto-dismissing: clamped to **0.5-30s**; omitted uses the default (**5s** `info` / `warn`, **6s** `error`). Pass **0** as the duration to make the toast **sticky**: it never auto-dismisses and stays until the player clicks it. Use this for fatal errors that must be acknowledged. `dismissible` defaults to `True`; pass `False` as the fourth argument for a forced-read toast with no early close. A sticky toast is always dismissible, so it can never pin the screen. Identical consecutive notifications from the same script collapse inside a 1-second window, and a sticky already on screen is never duplicated, so a tight loop can't spam the screen.

PARAMETERS

Name	Type	Description
`text`	`str`	Notification text
`level`	`str`	Notification severity
`duration_seconds`	`float`	Visible duration in real seconds; 0 is sticky
`dismissible`	`bool`	Whether the player may close the toast early

RETURNS

`None`

CONTROL

`sleep(seconds: float, /) → None`

Wait before continuing. **The argument is in real seconds**, not world-clock hours. The day cycle compresses **24** world-clock hours into a shorter real-time window, so `sleep(25)` is about 1 world-clock hour at the default 10-min-per-day pacing. For planet-aware delays, query the conversion: `sleep(get_component("clock").real_seconds_per_hour() * 2)` waits exactly two world-clock hours regardless of pacing. Loops are paced automatically; `sleep()` is for deliberate delays.

PARAMETERS

Name	Type	Description
`seconds`	`float`	How long to wait, in seconds; zero or more

RETURNS

`None`

RAISES

Exception	Condition
`TypeError`	`sleep(seconds)` requires one numeric duration.
`ValueError`	`sleep(seconds)` requires a finite duration greater than or equal to zero.
`OverflowError`	`sleep(seconds)` cannot represent the requested duration within the simulation tick range.

UTILITY

`len(value: Sized, /) → int`

Length of a `list`, `tuple`, string, `dict`, or `set`.

PARAMETERS

Name	Type	Description
`value`	`Sized`	String, `list`, `tuple`, `dict`, `set`, `range`, or object to measure

RETURNS

`int`

`range(stop: int, /) / range(start: int, stop: int, /) / range(start: int, stop: int, step: int, /) → list[int]`

Materialize the bounded integer range from `start` to `stop` (exclusive), stepping by `step`. `range(5)` → `[0,1,2,3,4]`. Endpoints and steps retain arbitrary-size exact integers, while the produced list must fit the interpreter's collection limit. All arguments must be integers; `step=0` and fractional values are rejected. Negative `step` counts down: `range(5, 0, -1)` → `[5,4,3,2,1]`.

RETURNS

`list[int]`

`slice(stop: int | None, /) / slice(start: int | None, stop: int | None, step: int | None = None, /) → slice`

Build a reusable slice object for `list`, `tuple`, and string subscripts. `slice(None, None, -1)` is the reusable form of `[::-1]`; `seq[s]` follows the same bounds and step rules as `seq[start:stop:step]`.

RETURNS

`slice`

`type(value: object, /) → str`

Returns the player's value-based type category as a string: a user class instance returns its class name, `"int"` covers whole numbers (including exact huge integers), `"float"` covers fractional / non-finite numbers, and built-ins report `"str"`, `"bool"`, `"NoneType"`, `"list"`, `"tuple"`, `"dict"`, `"set"`, or `"slice"`. Game API values retain the established `"object"` category so existing scripts remain unchanged; use `object_type(value)` for a concrete registered data-object name. Numeric categories are based on the current value, not literal spelling, so `type(4.0)` returns `"int"` because the value is whole. Parameter types follow Python instead: a `float` parameter takes any number, an `int` parameter only a whole one.

PARAMETERS

Name	Type	Description
`value`	`object`	Value whose type name you want

RETURNS

`str`

`object_type(value: object, /) → str`

Return a concrete registered name such as `"MiningSite"`, `"CatalogedFragment"`, or `"Position"` for a game data object without changing the established result of `type(value)`. Component references return `"Component"`; other values use the same public category as `type(value)`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value whose concrete game data-object name is needed

RETURNS

`str`

`vars(object: object, /) → dict[str, object]`

Return a detached, shallow dictionary of an object's public data fields. Structured results, positions, journal entries, and other game data values include their documented attribute fields but not methods; class instances include their own stored attributes. Changing the returned dictionary does not change the object, although nested lists and dictionaries are shared. Use `object_type(object)` separately when the concrete registered name is needed. Pass exactly one object; the no-argument local-scope form is not supported.

PARAMETERS

Name	Type	Description
`object`	`object`	Game API value or class instance to snapshot

RETURNS

`dict[str, object]`

`object() → object`

Construct a new identity-only base object. It takes no arguments and has no writable game-script attributes. User classes inherit from `object` implicitly.

RETURNS

`object`

`TypedDict(name: str, fields: dict[str, object], /) → type`

Declare a fixed-key `dict` shape for the editor. Use the functional form (`State = TypedDict("State", {"mode": str})`) and annotate records with that name (`state: State = {...}`). Runtime treats this as an inert type marker; the editor uses it for key autocomplete, field type flow, and typo lint. The field dictionary is ordinary code, so a game type used as a field type needs its import first (`from __builtins__ import Site`, then `"site": Site`) or the name in quotes (`"site": "Site"`); both give the same editor type flow.

PARAMETERS

Name	Type	Description
`name`	`str`	Declared type name
`fields`	`dict[str, object]`	Dictionary mapping field names to type expressions. Every value is evaluated, so a game type name must be imported or quoted.

RETURNS

`type`

`hasattr(value: object, name: str, /) → bool`

Return `True` if `value.name` is readable, otherwise `False`. The attribute name must be a string. Useful in duck-typed helpers that accept several documented object shapes, e.g. `hasattr(value, "required_recipe")`. Gameplay result objects keep their fixed fields, so branch on `.status` before reading a documented payload such as `analysis.info`. Missing mounted sub-objects such as `self.sonar` return `False` when the module is not installed.

RETURNS

`bool`

`getattr(value: object, name: str, default?: object, /) → object`

Read an attribute by string name, exactly like `value.name`. Without `default`, a missing attribute raises `AttributeError`; with `default`, missing attributes return that fallback. Works for game object properties/methods and built-in methods such as `getattr([1], "append")`. Use with `hasattr()` for duck-typed helpers, not as a replacement for a gameplay result's documented `.status`, `.message`, and payload fields.

RETURNS

`object`

`setattr(value: object, name: str, new_value: object, /) → None`

Set the attribute called `name` on `value`, exactly as `value.name = new_value` does, when the name is only known while the script runs: `setattr(state, field, 0)`.

PARAMETERS

Name	Type	Description
`value`	`object`	Object to change
`name`	`str`	Attribute name
`new_value`	`object`	Value to store

RETURNS

`None`

`sorted(sequence: Iterable[T], /, *, key: Callable | None = None, reverse: bool = False) → list[T]`

Return a new list sorted using `<` between mutually comparable keys. With `key=None` (the default), each value is its own key. Numeric and boolean keys compare across that family, strings compare lexicographically, and user objects may define the exact rich-comparison slots needed by `<`; unsupported pairs raise. `key=` is called once per item and must be pure: it cannot suspend or mutate game state. `reverse` is truth-tested, and the finite input is handled eagerly within the interpreter's collection limit.

PARAMETERS

Name	Type	Description
`sequence`	`Iterable[T]`	Values to sort
`key`	`Callable` | `None`	Function returning the value to compare for each item
`reverse`	`bool`	`True` to sort from largest to smallest

RETURNS

`list[T]`

`reversed(iterable: Iterable[T], /) → list[T]`

Return a reversed list copy of any finite iterable. This consumes the input fully, so an infinite generator exceeds the collection limit; dictionaries (their keys), sets (deterministic insertion order), generators, and custom iterator-protocol classes are accepted.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Sequence to reverse

RETURNS

`list[T]`

`enumerate(iterable: Iterable[T], /, start: int = 0) → list[tuple[int, T]]`

List of (index, value) pairs. Optional integer `start=N` shifts the index: `enumerate(items, start=1)` for 1-based indexing. Arbitrarily large integer starts remain exact. `start` may be positional or keyword, but not both. Given a generator, it returns a lazy iterator instead of a list, taking each item only when the loop asks for it.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to number
`start`	`int`	Number given to the first item

RETURNS

`list[tuple[int, T]]`

`zip(*iterables: Iterable, strict: bool = False) → list[tuple]`

Combine iterables into tuples. With no args, returns empty list. With one arg, returns a list of 1-tuples. Stops at the shortest input unless `strict=True`, which raises if input lengths differ. When an input is a generator, it returns a lazy iterator instead of a list.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable`	Sequences to pair up
`strict`	`bool`	`True` to raise when the lengths differ

RETURNS

`list[tuple]`

`isinstance(value: object, type_or_tuple: type | tuple, /) → bool`

Check if a value is of the given type. Accepts a builtin type callable (`object`, `int`, `float`, `str`, `list`, `dict`, `set`, `tuple`, `slice`, `bool`), a class of your own, a game class imported from `__builtins__` (after `from __builtins__ import Smelter`, a smelter is an instance of `Smelter` and of `Component`), a string type name, a tuple of types, or a type union like `int | float` (matches any). `int` matches whole numbers and booleans, `float` matches fractional / non-finite numbers, and string `"number"` remains the broad numeric family for scripts that intentionally accept either.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to test
`type_or_tuple`	`type` | `tuple`	A type, or a tuple of types

RETURNS

`bool`

`callable(value: object, /) → bool`

`True` when a value has a call surface: user-defined `def` / `lambda`, built-in functions, classes, bound methods, and class instances whose type defines `__call__`. Like Python, this checks the type-level call slot without executing its descriptor; an actual call can still raise if that slot resolves to a non-callable value.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to test

RETURNS

`bool`

`dir(value: object, /) → list[str]`

Lists the method and attribute names available on `value`, sorted: runtime introspection for discovering what you can do with something. On a component or game object (`dir(self)`, `dir(get_component("smelter_1"))`) it returns that object's callable methods and properties, straight from the console. On a built-in container it returns the type's methods: `dir([1, 2])` → `"append"`, `"pop"`, ...; `dir("hi")` → `"upper"`, `"split"`, ... Values with no members (numbers, booleans, `None`) return an empty list. Pass exactly one value: the no-argument `dir()` that lists current-scope names is not supported.

PARAMETERS

Name	Type	Description
`value`	`object`	Value whose attribute names you want

RETURNS

`list[str]`

`hash(value: object, /) → int`

Hash any hashable value to an integer: strings, numbers, booleans, `None`, tuples of hashable values, functions, classes, and hashable user instances. Instances are identity-hashable by default; defining `__eq__` without `__hash__` makes them unhashable, and a custom `__hash__` controls `hash(obj)`, `dict` keys, and `set` membership.

PARAMETERS

Name	Type	Description
`value`	`object`	Hashable value

RETURNS

`int`

`super() / super(type: type, object: object, /) → object`

Return a proxy that searches the receiver's MRO after a chosen class. Inside a method, use `super()` for cooperative parent calls such as `super().__init__(...)`. The explicit `super(type, object)` form accepts an instance or subclass of `type`; the one-argument form is not supported.

RETURNS

`object`

`issubclass(cls: type, class_or_tuple: type | tuple, /) → bool`

`True` if `cls` is the given class or a subclass of it (or of any class in the tuple), per the MRO. `issubclass(Dog, Animal)` is `True`; every class is a subclass of `object`. Game classes imported from `__builtins__` work the same way: `issubclass(Smelter, Component)` and `issubclass(MiningSite, Site)` are `True`.

PARAMETERS

Name	Type	Description
`cls`	`type`	Class to test
`class_or_tuple`	`type` | `tuple`	A class, or a tuple of classes

RETURNS

`bool`

`property(fget: Callable | None = None, fset: Callable | None = None, fdel: Callable | None = None, doc: str | None = None) → property`

Build a property descriptor. Use `@property` for the common getter form, or call `property(fget, fset, fdel, doc)` directly; every argument is optional and may also be named. Instance reads call `fget`, writes call `fset`, and deletion calls `fdel`; an already-bound hook stays bound and receives the object being read, written, or deleted as an additional argument. `.getter(fn)`, `.setter(fn)`, and `.deleter(fn)` return cloned descriptors. An omitted/`None` `doc` follows `fget.__doc__` through normal attribute lookup; an explicit non-`None` `doc` is preserved by clones.

PARAMETERS

Name	Type	Description
`fget`	`Callable` | `None`	Optional instance getter
`fset`	`Callable` | `None`	Optional instance setter
`fdel`	`Callable` | `None`	Optional instance deleter
`doc`	`str` | `None`	Optional explicit descriptor doc value

RETURNS

`property`

`classmethod(func: Callable, /) → Callable`

Wrap a function as a class method. Usually written `@classmethod`; the decorated method receives the class (conventionally `cls`) as its first argument and can be called on the class or an instance.

PARAMETERS

Name	Type	Description
`func`	`Callable`	Function to bind to the class

RETURNS

`Callable`

`staticmethod(func: Callable, /) → Callable`

Wrap a function as a static method. Usually written `@staticmethod`; the decorated method receives neither an instance nor the class and behaves as a plain function namespaced under the class.

PARAMETERS

Name	Type	Description
`func`	`Callable`	Function to store without receiver binding

RETURNS

`Callable`

`NotImplemented: NotImplemented`

The immutable singleton returned by an operator method to ask Python to try reflected dispatch or the normal fallback.

RETURNS

`NotImplemented`

`Ellipsis: object`

The immutable singleton also written `...`. As a statement it stands in for a body you have not written yet, and inside a type annotation it means "any number of": `tuple[float, ...]` is a tuple of any length and `Callable[..., Site]` takes any arguments.

RETURNS

`object`

`__name__: str`

The name of the module the code runs in: `"__main__"` in the script you run, and the library's name when the code runs because another script imported it. `if __name__ == "__main__":` runs a block only when the file itself is run.

RETURNS

`str`

CONVERSION

`str(value: object = '', /) → str`

Convert a value to its string form. With no arguments, returns the empty string `''`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to turn into text

RETURNS

`str`

`int(value: str | float = 0, /, base?: int) → int`

Convert to integer. With no args returns `0`. Numbers truncate toward zero. String conversion consumes the whole trimmed value and accepts Python underscore separators and Unicode decimal digits. The optional `base` is 2-36 or `0`; `base=0` detects `0x` / `0b` / `0o` prefixes. Prefix and `base` forms retain arbitrary-size exact integers within the interpreter's integer budget.

PARAMETERS

Name	Type	Description
`value`	`str` | `float`	Number, `True` / `False`, or numeric text to convert
`base`	`int`	Number base for text, from 2 to 36, or 0 to detect numeric prefixes

RETURNS

`int`

`float(value: str | float = 0.0, /) → float`

Convert to `float`. With no arguments, returns `0.0`. String conversion consumes the whole trimmed value and accepts Python decimal/exponent syntax, underscore separators, Unicode decimal digits, and case-insensitive `inf` / `nan`. Trailing garbage raises; booleans coerce to `0.0` / `1.0`.

PARAMETERS

Name	Type	Description
`value`	`str` | `float`	Number, `True` / `False`, or numeric text to convert

RETURNS

`float`

`chr(code: int, /) → str`

Character from Unicode code point (e.g. `chr(65)` → `'A'`).

PARAMETERS

Name	Type	Description
`code`	`int`	Unicode code point

RETURNS

`str`

`ord(char: str, /) → int`

Unicode code point from character (e.g. `ord('A')` → 65).

PARAMETERS

Name	Type	Description
`char`	`str`	A single character

RETURNS

`int`

`bool(value?: object, /) → bool`

Convert to boolean (`True` / `False`). With no args returns `False`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to test for truth

RETURNS

`bool`

`list(iterable?: Iterable[T], /) → list[T]`

Convert any iterable to a list, or create an empty list with `list()`. Methods: `.append(x)`, `.pop([i])`, `.insert(i, x)`, `.remove(x)`, `.index(x [, start [, end]])`, `.count(x)`, `.sort()`, `.reverse()`, `.copy()`, `.extend(iter)`, `.clear()`, `.length` (property). Subscript with `lst[i]` and slice with `lst[start:end:step]`. See individual entries below for full Python-compliant semantics.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to copy into a new list

RETURNS

`list[T]`

`dict(**kwargs: V) / dict(source: dict[K, V] | Iterable[tuple[K, V]], /, **kwargs: V) → dict[K, V]`

Build a dictionary. Empty form `dict()`. From pairs: `dict([("a", 1), ("b", 2)])`. Shallow-copy another `dict`: `dict(d)`. Keyword form: `dict(name="Mars", temp=-63)`. Keys may be any hashable value, including tuples of hashables and properly hashable user instances. Methods: `.keys()`, `.values()`, `.items()`, `.has(k)`, `.get(k, default?)`, `.pop(k, default?)`, `.popitem()`, `.setdefault(k, default?)`, `.update(other)`, `.copy()`, `.clear()`, `.length` (property). Operators: `a | b` returns a merged copy with right-hand values winning; `a |= b` updates `a` in place. Subscript with `d[k]`; missing key raises. Use `.get(k)` or `.has(k)` for safe lookup.

RETURNS

`dict[K, V]`

`set(iterable?: Iterable[T], /) → set[T]`

Build a set of unique members from any iterable, or `set()` for empty. Members may be any hashable values, including tuples of hashables and properly hashable user instances. Use `{1, 2, 3}` for a literal: empty `{}` is a `dict`, not a `set`, so empty set is always `set()`. Methods: `.add(x)`, `.remove(x)`, `.discard(x)`, `.pop()`, `.clear()`, `.copy()`, `.union(s)`, `.intersection(s)`, `.difference(s)`, `.symmetric_difference(s)`, `.update(s)`, `.issubset(s)`, `.issuperset(s)`, `.isdisjoint(s)`. Operators: `in`, `|` (union), `&` (intersection), `-` (difference), `^` (symmetric difference), `<` `<=` `>=` `>` (subset/superset), `==`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to copy into a new set

RETURNS

`set[T]`

`tuple(iterable?: Iterable[T], /) → tuple[T, ...]`

Build a tuple from any iterable, or `tuple()` for empty. Tuples are like lists but immutable (no `append` / `pop` / `sort`): useful for fixed records and as hashable keys. Methods: `.index(x)`, `.count(x)`, `.length` (property). Subscript and slice with `t[i]` / `t[a:b]` just like lists.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to copy into a new tuple

RETURNS

`tuple[T, ...]`

`hex(integer: int, /) → str`

Render an integer as a Python-style hex string with `0x` prefix. `hex(255)` → `'0xff'`, `hex(-16)` → `'-0x10'`. Floats reject.

PARAMETERS

Name	Type	Description
`integer`	`int`	Whole number to convert

RETURNS

`str`

`bin(integer: int, /) → str`

Render an integer as a binary string with `0b` prefix. `bin(10)` → `'0b1010'`. Floats reject.

PARAMETERS

Name	Type	Description
`integer`	`int`	Whole number to convert

RETURNS

`str`

`oct(integer: int, /) → str`

Render an integer as an octal string with `0o` prefix. `oct(8)` → `'0o10'`. Floats reject.

PARAMETERS

Name	Type	Description
`integer`	`int`	Whole number to convert

RETURNS

`str`

`repr(value: object, /) → str`

Developer-readable string for a value, with quotes around strings and nested `repr()` for containers. `repr([1, "a"])` → `"[1, 'a']"` (note the quotes around `'a'`). Use when you want to see the value's structure, not its display form. Also reached via f-string `!r` conversion: `f"{name!r}"`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to describe

RETURNS

`str`

`format(value: object, format_spec: str = '', /) → str`

Format one value by a format spec, the same text an f-string field produces: `format(3.14159, ".2f")` → `"3.14"`, `format(42, ">6")` → `" 42"`. With no spec it is `str(value)`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to format
`format_spec`	`str`	Format spec, such as `".2f"` or `">10"`

RETURNS

`str`

FUNCTIONAL

`total_ordering(cls: type, /) → type`

Class decorator that preserves the class object and fills missing ordering methods from `__eq__` plus one of `__lt__`, `__le__`, `__gt__`, or `__ge__`. Explicit methods are never replaced. Applying it mutates the local class namespace.

PARAMETERS

Name	Type	Description
`cls`	`type`	Class to complete

RETURNS

`type`

`wraps(wrapped: Callable, /) → Callable`

Return a decorator that preserves the wrapped callable's supported name, qualified name, docstring, custom attributes, and `__wrapped__` link while keeping the wrapper's call behavior. Applying the returned decorator mutates only that local wrapper function.

PARAMETERS

Name	Type	Description
`wrapped`	`Callable`	Callable whose metadata should be copied

RETURNS

`Callable`

MATH

`abs(number: float, /) → float`

Absolute value.

PARAMETERS

Name	Type	Description
`number`	`float`	Number whose distance from zero you want

RETURNS

`float`

`isclose(a: float, b: float, rel_tol: float = 0.000000001, abs_tol: float = 0.0) → bool`

Return `True` when two numbers are close enough to treat as equal. `rel_tol` scales with the compared values; `abs_tol` sets a fixed accepted difference in the same unit, useful for values near zero and physical readings such as coordinates. Tolerances must be non-negative. This is the directly available equivalent of Python's `math.isclose()`.

PARAMETERS

Name	Type	Description
`a`	`float`	First number
`b`	`float`	Second number
`rel_tol`	`float`	Maximum relative difference
`abs_tol`	`float`	Maximum absolute difference in the values' unit

RETURNS

`bool`

`min(iterable: Iterable[T], /, *, key: Callable | None = None, default?: object) / min(a: T, b: T, /, *args: T, key: Callable | None = None) → T`

Return the selected original value whose key is smallest under `<`. Accepts multiple values or one finite iterable, handled eagerly within the collection limit. With `key=None` (the default), each value is its own key; otherwise the pure `key=` callback runs once per item and cannot suspend or mutate game state. Keys must be mutually comparable: numeric and boolean keys compare across that family, strings compare lexicographically, and user objects may supply the rich-comparison slots required by `<`. In single-iterable form, `default=v` supplies the result for an empty input; without it, empty input raises.

RETURNS

`T`

`max(iterable: Iterable[T], /, *, key: Callable | None = None, default?: object) / max(a: T, b: T, /, *args: T, key: Callable | None = None) → T`

Return the selected original value whose key is largest under `>`. Accepts multiple values or one finite iterable, handled eagerly within the collection limit. With `key=None` (the default), each value is its own key; otherwise the pure `key=` callback runs once per item and cannot suspend or mutate game state. Keys must be mutually comparable: numeric and boolean keys compare across that family, strings compare lexicographically, and user objects may supply the rich-comparison slots required by `>`. In single-iterable form, `default=v` supplies the result for an empty input; without it, empty input raises.

RETURNS

`T`

`round(number: float, ndigits: int | None = None) → float`

Round with ties to even. Without `ndigits` (or with `None`), returns the nearest whole value and rejects NaN/infinity. Integer and boolean inputs remain exact; negative `ndigits` rounds exact integers in decimal. Both arguments accept keyword form.

PARAMETERS

Name	Type	Description
`number`	`float`	Value to round
`ndigits`	`int` | `None`	Decimal digits or `None`

RETURNS

`float`

`pow(base: float, exp: float, mod: int | None = None) → float`

`base` raised to `exp`, exactly like `base ** exp`, so your own classes that define `__pow__` or `__rpow__` work too. Optional `mod` performs exact modular exponentiation and accepts negative exponents when the base has a modular inverse; a class base receives it as `__pow__(exp, mod)`. `base`, `exp`, and `mod` accept positional or keyword form.

PARAMETERS

Name	Type	Description
`base`	`float`	Base value
`exp`	`float`	Exponent
`mod`	`int` | `None`	Optional integer modulus

RETURNS

`float`

`random() → float`

Random floating-point number `>= 0` and `< 1`. Each successful run begins a new automatic sequence. Also available as `random.random()` after `import random`.

RETURNS

`float`

`rand() → float`

Short alias for `random()`: a random floating-point number `>= 0` and `< 1`. Each successful run begins a new automatic sequence. Also available as `random.rand()` after `import random`.

RETURNS

`float`

`randint(min: int, max: int, /) → int`

Random integer `N` where `min <= N <= max`. Bounds are inclusive and must be whole numbers. Each successful run begins a new automatic sequence. Useful with `planet.get_bounds()` for random valid coordinates. Also available as `random.randint(min, max)` after `import random`.

PARAMETERS

Name	Type	Description
`min`	`int`	Smallest possible result
`max`	`int`	Largest possible result

RETURNS

`int`

`sqrt(number: float, /) → float`

Square root. Errors on negative input.

PARAMETERS

Name	Type	Description
`number`	`float`	Number zero or greater

RETURNS

`float`

`floor(number: float, /) → int`

Round down to the nearest integer.

PARAMETERS

Name	Type	Description
`number`	`float`	Number to round down

RETURNS

`int`

`ceil(number: float, /) → int`

Round up to the nearest integer.

PARAMETERS

Name	Type	Description
`number`	`float`	Number to round up

RETURNS

`int`

`trunc(number: float, /) → int`

Drop the fractional part (round toward zero).

PARAMETERS

Name	Type	Description
`number`	`float`	Number to cut toward zero

RETURNS

`int`

`divmod(a: float, b: float, /) → tuple[float, ...]`

Divides `a` by `b` and returns two values: the quotient rounded down, and the remainder left over. For example, `divmod(19, 2)` returns `(9, 1)` because 2 fits into 19 nine times with 1 left over.

PARAMETERS

Name	Type	Description
`a`	`float`	Number to divide
`b`	`float`	Number to divide by

RETURNS

`tuple[float, ...]`

`sign(number: float, /) → int`

Returns `-1`, `0`, or `1` for negative, zero, or positive input.

PARAMETERS

Name	Type	Description
`number`	`float`	Number to test

RETURNS

`int`

`exp(number: float, /) → float`

`e` raised to the power of the argument.

PARAMETERS

Name	Type	Description
`number`	`float`	Power of e to compute

RETURNS

`float`

`log(number: float, base?: float, /) → float`

Natural log, or log with the given base.

PARAMETERS

Name	Type	Description
`number`	`float`	Number greater than zero
`base`	`float`	Logarithm base; the natural logarithm when omitted

RETURNS

`float`

`log2(number: float, /) → float`

Base-2 logarithm.

PARAMETERS

Name	Type	Description
`number`	`float`	Number greater than zero

RETURNS

`float`

log10(number: float, /) → float

Base-10 logarithm.

PARAMETERS

Name	Type	Description
number	float	Number greater than zero

RETURNS

float

sin(radians: float, /) → float

Sine of an angle in radians.

PARAMETERS

Name	Type	Description
radians	float	Angle in radians

RETURNS

float

cos(radians: float, /) → float

Cosine of an angle in radians.

PARAMETERS

Name	Type	Description
radians	float	Angle in radians

RETURNS

float

tan(radians: float, /) → float

Tangent of an angle in radians.

PARAMETERS

Name	Type	Description
radians	float	Angle in radians

RETURNS

float

asin(number: float, /) → float

Arc sine. Input must be from -1 to 1.

PARAMETERS

Name	Type	Description
number	float	Value from -1 to 1

RETURNS

float

acos(number: float, /) → float

Arc cosine. Input must be from -1 to 1.

PARAMETERS

Name	Type	Description
number	float	Value from -1 to 1

RETURNS

float

`atan(number: float, /) → float`

Arc tangent.

PARAMETERS

Name	Type	Description
`number`	`float`	Tangent value

RETURNS

`float`

`atan2(y: float, x: float, /) → float`

Arc tangent of `y/x`, correctly choosing the quadrant.

PARAMETERS

Name	Type	Description
`y`	`float`	Vertical component
`x`	`float`	Horizontal component

RETURNS

`float`

`degrees(radians: float, /) → float`

Convert radians to degrees.

PARAMETERS

Name	Type	Description
`radians`	`float`	Angle in radians

RETURNS

`float`

`radians(degrees: float, /) → float`

Convert degrees to radians.

PARAMETERS

Name	Type	Description
`degrees`	`float`	Angle in degrees

RETURNS

`float`

`sum(iterable: Iterable[float] | Iterable[list] | Iterable[object], /, start: float | list | object = 0) → float | list`

Adds up the items of an iterable with `+`, starting from `start` (default `0`). Works for numbers, lists (`sum(list_of_lists, [])` flattens), tuples, and your own classes that define `__add__` or `__radd__`. Strings are refused: use `"".join(items)`. `start` may be positional or keyword, but not both.

RETURNS

`float | list`

`prod(iterable: Iterable[float], /, start: float = 1) → float`

Multiplies the items of an iterable with `*`, starting from `start` (default `1`). Works for numbers and for your own classes that define `__mul__` or `__rmul__`. Empty iterables return `start`. `start` may be positional or keyword, but not both.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[float]`	Values to multiply
`start`	`float`	Value the product starts from

RETURNS

`float`

inf: float

Positive infinity, larger than every finite number. Use `-inf` for negative infinity, including as an initial best or worst value in search and pathfinding algorithms.

RETURNS

`float`

pi: float

Mathematical constant π ≈ 3.14159.

RETURNS

`float`

tau: float

Mathematical constant τ = 2π.

RETURNS

`float`

LOGIC

all(iterable: Iterable, /) → bool

`True` if every item in the iterable is truthy. Stops at the first falsy item, so generators are consumed only as far as needed.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable`	Values to test

RETURNS

`bool`

`any(iterable: Iterable, /) → bool`

`True` if any item in the iterable is truthy. Stops at the first truthy item, so generators are consumed only as far as needed.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable`	Values to test

RETURNS

`bool`

ITERATION

`iter(iterable: Iterable[T] | Callable, sentinel?: object, /) → list | generator`

Return an iterator. Built-in iterables use a compatibility list-shaped iterator; protocol iterators and generators keep their identity, so `iter(iterator) is iterator`. `for` advances generators and custom iterators one item at a time, so it can break out of an infinite iterator. Consumers that must finish, such as `list(...)`, remain bounded. With a `sentinel`, `iter(read, None)` calls `read()` for each item and stops at the first result equal to `None`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]` | `Callable`	Values to iterate, or a function to call when `sentinel` is given
`sentinel`	`object`	Value that ends the iteration when the function returns it

RETURNS

`list | generator`

`next(iterator: Iterator[T], default?: object, /) → T`

Advance an iterator by one item. Iterators returned by `iter()` and custom `__next__` iterators retain their cursor across calls. Ordinary lists keep the compatibility behavior of popping the front; other non-iterator iterables return their first materialized item. Exhaustion raises `StopIteration` unless `default` is given.

PARAMETERS

Name	Type	Description
`iterator`	`Iterator[T]`	Iterator to advance
`default`	`object`	Returned instead of raising `StopIteration` once the iterator is exhausted

RETURNS

`T`

`pairwise(iterable: Iterable[T], /) → list[tuple[T, T]]`

Return neighboring pairs from an iterable. `pairwise([1,2,3])` → `[(1,2), (2,3)]`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Sequence to walk in neighbouring pairs

RETURNS

`list[tuple[T, T]]`

`batched(iterable: Iterable[T], size: int, /) → list[tuple[T, ...]]`

Split an iterable into batches of `size`, each one a tuple. The final batch may be shorter. `batched([1,2,3,4,5], 2)` → `[(1,2), (3,4), (5,)]`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to group
`size`	`int`	Items per batch; 1 or more

RETURNS

`list[tuple[T, ...]]`

`starmap(fn: Callable, iterable: Iterable, /) → list`

Call pure `fn` with each tuple/list item unpacked as arguments. The callback cannot suspend the script or mutate game state. `starmap(pow, [(2,3), (3,2)])` → `[8,9]`. An empty input never inspects or calls `fn`.

PARAMETERS

Name	Type	Description
`fn`	`Callable`	Function called with each item unpacked as its arguments
`iterable`	`Iterable`	Tuples or lists of arguments

RETURNS

`list`

`flatten(iterable: Iterable, /) → list`

Flatten one level of nested iterables into a list. `flatten([[1,2], (3,4)])` → `[1,2,3,4]`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable`	Sequence of sequences to flatten one level

RETURNS

`list`

`count_by(iterable: Iterable[T], key_fn?: Callable, /) → dict[object, int]`

Count items into a `dict`. Without `key_fn` (or with `None`), counts each item. With a pure `key_fn`, counts the computed key; callbacks cannot suspend the script or mutate game state: `count_by(items, lambda x: x.kind)`. An empty input never inspects or calls the key function.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to count
`key_fn`	`Callable`	Optional function that maps each item to the key to count

RETURNS

`dict[object, int]`

`chain(*iterables: Iterable[T]) → list[T]`

Flattens any number of iterables into one list, in order. `chain([1,2], [3,4])` → `[1,2,3,4]`. Accepts lists, tuples, strings, sets.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable[T]`	Sequences to join end to end

RETURNS

`list[T]`

`accumulate(iterable: Iterable[float], /) → list[float]`

Running totals over any finite iterable: the first item, then each total so far plus the next item with `+`. `accumulate([1,2,3,4])` → `[1,3,6,10]`. Works for anything `+` works on, including strings, lists and your own classes that define `__add__`. Arbitrarily large integers remain exact.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[float]`	Values to total as you go

RETURNS

`list[float]`

`combinations(iterable: Iterable[T], k: int, /) → list[tuple[T, ...]]`

All `k`-element combinations from an iterable, in input order, no repeats. Returns a list of tuples. `combinations([1,2,3], 2)` → `[(1,2), (1,3), (2,3)]`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to choose from
`k`	`int`	How many to choose

RETURNS

`list[tuple[T, ...]]`

`permutations(iterable: Iterable[T], k?: int, /) → list[tuple[T, ...]]`

All `k`-length ordered arrangements from an iterable. `k` defaults to the full length. `permutations([1,2,3])` → all 6 orderings as tuples.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to arrange
`k`	`int`	How many to arrange; all of them when omitted

RETURNS

`list[tuple[T, ...]]`

`product(*iterables: Iterable, repeat: int = 1) → list[tuple]`

Cartesian product. `product([0,1], [0,1])` → `[(0,0), (0,1), (1,0), (1,1)]`. Each input must be iterable; result is a list of tuples. Optional keyword `repeat=N` repeats the input pools, matching `itertools.product([0,1], repeat=2)`.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable`	Sequences to combine
`repeat`	`int`	How many times to repeat the sequences

RETURNS

`list[tuple]`

`map(fn: Callable, iter1: Iterable, iter2?: Iterable, ..., /, strict: bool = False) → list`

Apply pure `fn` to corresponding items of each iterable. Callbacks cannot suspend the script or mutate game state. Single-iter form calls `fn(x)`; multi-iter form calls `fn(x, y, ...)` and stops at the shortest input unless `strict=True`, which raises if input lengths differ. If no row is produced, `fn` is never inspected or called. When an input is a generator, it returns a lazy iterator instead of a list.

PARAMETERS

Name	Type	Description
`fn`	`Callable`	Function called with one item from each sequence
`iter1`	`Iterable`	First sequence
`iter2`	`Iterable`	Second sequence, and any further ones
`strict`	`bool`	`True` to raise when the lengths differ

RETURNS

`list`

`filter(fn: Callable | None, iterable: Iterable[T], /) → list[T]`

Keep items for which pure `fn(item)` is truthy; callbacks cannot suspend the script or mutate game state. `filter(None, iter)` keeps every truthy item without a callback. An empty non-generator input never inspects or calls `fn`. Given a generator, it checks that `fn` is callable or `None` immediately and returns a lazy iterator instead of a list.

PARAMETERS

Name	Type	Description
`fn`	`Callable` | `None`	Function returning `True` for items to keep, or `None` to keep truthy items
`iterable`	`Iterable[T]`	Values to filter

RETURNS

`list[T]`

`reduce(fn: Callable, iterable: Iterable[T], initializer?: object, /) → object`

Combine an iterable into one value by repeatedly calling pure `fn(total, item)`; callbacks cannot suspend the script or mutate game state. With no initializer, the first item becomes the initial total; empty iterables then raise. Empty-with-initializer and singleton-without-initializer perform no callback call. Also available as `from functools import reduce`.

PARAMETERS

Name	Type	Description
`fn`	`Callable`	Function combining the running value with the next item
`iterable`	`Iterable[T]`	Values to combine
`initializer`	`object`	Starting value

RETURNS

`object`

EXCEPTIONS

`BaseException(*args: object) → BaseException`

Root of the supported exception hierarchy.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`BaseException`

`BaseException.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`BaseException.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`BaseException.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`BaseException.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`Exception(*args: object) → Exception`

Base class matched by ordinary `except Exception:` handlers.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`Exception`

`Exception.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`Exception.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`Exception.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`Exception.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ArithmeticError(*args: object) → ArithmeticError`

Base class for numeric calculation failures.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ArithmeticError`

`ArithmeticError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ArithmeticError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ArithmeticError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ArithmeticError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ValueError(*args: object) → ValueError`

A value has the right type but an invalid value.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ValueError`

`ValueError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ValueError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ValueError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ValueError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`TypeError(*args: object) → TypeError`

An operation received a value of an inappropriate type.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`TypeError`

`TypeError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`TypeError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`TypeError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`TypeError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`PermissionError(*args: object) → PermissionError`

An operation is prohibited by the caller's ownership or access contract.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`PermissionError`

`PermissionError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`PermissionError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`PermissionError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`PermissionError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ReferenceError(*args: object) → ReferenceError`

A captured component or module handle is no longer valid.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ReferenceError`

`ReferenceError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ReferenceError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ReferenceError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ReferenceError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`LookupError(*args: object) → LookupError`

Base class for invalid mapping keys and sequence indexes.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`LookupError`

`LookupError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`LookupError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`LookupError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`LookupError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`KeyError(*args: object) → KeyError`

A dictionary key is not present.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`KeyError`

KeyError.args: tuple

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

KeyError.__cause__: BaseException | None

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

KeyError.__context__: BaseException | None

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

KeyError.__suppress_context__: bool

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`IndexError(*args: object) → IndexError`

A sequence index is out of range.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`IndexError`

`IndexError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`IndexError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`IndexError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`IndexError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`AttributeError(*args: object) → AttributeError`

An attribute reference failed.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`AttributeError`

`AttributeError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`AttributeError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`AttributeError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`AttributeError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`RuntimeError(*args: object) → RuntimeError`

A runtime failure does not fit a more specific category.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`RuntimeError`

`RuntimeError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`RuntimeError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`RuntimeError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`RuntimeError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`NotImplementedError(*args: object) → NotImplementedError`

A required operation or override is not implemented.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`NotImplementedError`

`NotImplementedError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`NotImplementedError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`NotImplementedError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`NotImplementedError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`RecursionError(*args: object) → RecursionError`

The interpreter's call-depth limit was exceeded.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`RecursionError`

`RecursionError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`RecursionError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`RecursionError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`RecursionError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`NameError(*args: object) → NameError`

A local, free, or global name could not be resolved.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`NameError`

`NameError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`NameError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`NameError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`NameError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`UnboundLocalError(*args: object) → UnboundLocalError`

A statically local name was read before it was bound; this is a `NameError` subclass.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`UnboundLocalError`

`UnboundLocalError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`UnboundLocalError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`UnboundLocalError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`UnboundLocalError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ImportError(*args: object) → ImportError`

An import could not provide the requested binding.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ImportError`

`ImportError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ImportError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ImportError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ImportError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ModuleNotFoundError(*args: object) → ModuleNotFoundError`

An imported module could not be found; this is an `ImportError` subclass.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ModuleNotFoundError`

`ModuleNotFoundError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ModuleNotFoundError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ModuleNotFoundError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ModuleNotFoundError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`OverflowError(*args: object) → OverflowError`

A numeric conversion or bounded allocation exceeded its supported range.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`OverflowError`

`OverflowError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`OverflowError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`OverflowError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`OverflowError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`ZeroDivisionError(*args: object) → ZeroDivisionError`

Division or modulo used a zero divisor.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`ZeroDivisionError`

`ZeroDivisionError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`ZeroDivisionError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`ZeroDivisionError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`ZeroDivisionError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`StopIteration(*args: object) → StopIteration`

An iterator was exhausted. Its `.value` contains a generator's return value, or `None`.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`StopIteration`

`StopIteration.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`StopIteration.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`StopIteration.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`StopIteration.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`StopIteration.value: object`

The return value carried by a completed generator, or `None`.

RETURNS

`object`

`GeneratorExit(*args: object) → GeneratorExit`

Raised inside a generator when `close()` asks it to stop. It derives directly from `BaseException`.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`GeneratorExit`

`GeneratorExit.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`GeneratorExit.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`GeneratorExit.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`GeneratorExit.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`AssertionError(*args: object) → AssertionError`

An `assert` statement failed.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`AssertionError`

`AssertionError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`AssertionError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`AssertionError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`AssertionError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`SyntaxError(*args: object) → SyntaxError`

Source could not be compiled.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`SyntaxError`

`SyntaxError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`SyntaxError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`SyntaxError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`SyntaxError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

`IndentationError(*args: object) → IndentationError`

Source indentation is inconsistent; this is a `SyntaxError` subclass.

PARAMETERS

Name	Type	Description
`args`	`object`	Exception arguments, usually one message. They are stored on `.args`; keyword arguments are rejected.

RETURNS

`IndentationError`

`IndentationError.args: tuple`

The tuple of arguments the exception was created with. One message argument makes `str(error)` that message.

RETURNS

`tuple`

`IndentationError.__cause__: BaseException | None`

The exception named by `raise ... from cause`, or `None`.

RETURNS

`BaseException | None`

`IndentationError.__context__: BaseException | None`

The exception that was being handled when this one was raised, or `None`.

RETURNS

`BaseException | None`

`IndentationError.__suppress_context__: bool`

`True` after `raise ... from ...` (including `from None`), which marks the implicit context as deliberately replaced.

RETURNS

`bool`

Argument Types

The types parameters and results are written in. Most are ordinary Python types (`str` , `int` , `float` , `bool` , `list[str]` , `dict[str, int]`) and game types such as `Site` . The names below cover the rest: shapes several calls accept, and the Python words for 'any value' and 'anything a loop can walk'.

`JsonValue = str | int | float | bool | None | list[JsonValue] | dict[str, JsonValue]`

Any value the Signal Bus, the Data Archive and `json` can carry: text, a number, `True` or `False` , `None` , or a list or `dict` of those, with string keys.

`ItemProperties = dict[str, JsonValue]`

The properties of one item, as `ItemStack.properties` returns them: a `dict` from property name to a `JsonValue` .

`IdRecord`

A `dict` or an object with a string `id` , such as a `Site` , a record saved with `vars()` , or an instance of your own class. Only `id` is read; other fields are ignored.

`TransmissionRecord`

A `SignalTransmission` , or a `dict` or object with the same fields: string `event_id` , `channel` and `data` , and whole-number `number` and `total` .

`SiteRef = str | Site | IdRecord`

A site to act on: its id, a `Site` from `scan()` , or an `IdRecord` carrying the site's id.

`RecipeRef = str | Recipe | IdRecord`

A recipe to select: its id, a `Recipe`, or an `IdRecord` carrying the recipe's id.

`object`

Any value at all.

`Callable`

Something you can call: a function, a lambda, a method, or a class.

`Iterable`

Anything a `for` loop can walk: a `list`, `tuple`, `set`, `dict`, string, `range`, or generator. `Iterable[str]` means one whose items are strings.

`Iterator`

An iterable that hands out one item per `next()` call, such as the result of `iter()` or a generator.

`Sized`

Anything `len()` accepts: a string, `list`, `tuple`, `dict`, `set`, `range`, or a class with `__len__`.

`T`

The item type of the container on this page: in `list[T]`, `T` is whatever the list holds. `K` and `V` are a `dict`'s key and value types. Hover shows the real type when the editor knows it.

random

Built-in random-number helpers. Each successful run gets a new automatic sequence; use `random.seed(value)` when you want a reproducible sequence. Works without Shared Library research.

`random.random() → float`

Random floating-point number `>= 0` and `< 1`. Each successful run begins a new automatic sequence. Also available as `random.random()` after `import random`.

RETURNS

`float`

`random.rand() → float`

Short alias for `random()`: a random floating-point number `>= 0` and `< 1`. Each successful run begins a new automatic sequence. Also available as `random.rand()` after `import random`.

RETURNS

`float`

`random.randint(min: int, max: int, /) → int`

Random integer `N` where `min <= N <= max`. Bounds are inclusive and must be whole numbers. Each successful run begins a new automatic sequence. Useful with `planet.get_bounds()` for random valid coordinates. Also available as `random.randint(min, max)` after `import random`.

PARAMETERS

Name	Type	Description
`min`	`int`	Smallest possible result
`max`	`int`	Largest possible result

RETURNS

`int`

`random.seed(a: int | float | str | bool | None = None) → None`

Initialize random-number generation. Omit `a` or pass `None` to select another automatic sequence. Pass a number, boolean, or string to make later draws reproducible; the same value produces the same sequence.

PARAMETERS

Name	Type	Description
`a`	`int` | `float` | `str` | `bool` | `None`	Optional number, boolean, string, or `None` seed

RETURNS

`None`

`random.randrange(start: int, stop: int | None = None, step: int = 1, /) → int`

Random whole number from `range(start, stop, step)`: `randrange(10)` is 0 to 9, `randrange(5, 20, 5)` is 5, 10 or 15. The stop value is never picked.

PARAMETERS

Name	Type	Description
`start`	`int`	First possible value, or the stop value when it is the only argument
`stop`	`int` | `None`	End of the range, never picked
`step`	`int`	Distance between possible values (default 1)

RETURNS

`int`

`random.choice(seq: list[T] | tuple[T, ...] | str, /) → T`

One random item of a list, tuple or string: `random.choice(["iron", "copper"])`. An empty one is an `IndexError`.

PARAMETERS

Name	Type	Description
`seq`	`list[T]` | `tuple[T, ...]` | `str`	List, tuple or string to pick from

RETURNS

`T`

`random.choices(population: list[T] | tuple[T, ...] | str, weights: Iterable[float] | None = None, *, cum_weights: Iterable[float] | None = None, k: int = 1) → list[T]`

`k` random items picked with repetition, as a list. `weights` makes some items likelier: `random.choices(["ore", "ice"], weights=[3, 1], k=5)`.

PARAMETERS

Name	Type	Description
`population`	`list[T]` | `tuple[T, ...]` | `str`	List, tuple or string to pick from
`weights`	`Iterable[float]` | `None`	How likely each item is, in the same order
`cum_weights`	`Iterable[float]` | `None`	Running totals of the weights, instead of `weights`
`k`	`int`	How many items to pick (default 1)

RETURNS

`list[T]`

`random.shuffle(x: list[object], /) → None`

Put a list's items in random order, in place. Returns `None`; the list itself changes.

PARAMETERS

Name	Type	Description
`x`	`list[object]`	List to reorder

RETURNS

`None`

`random.sample(population: list[T] | tuple[T, ...] | str, k: int, *, counts: Iterable[int] | None = None) → list[T]`

`k` different random items, as a list, never picking one place twice: `random.sample(range(100), 3)`. `counts` repeats items: `counts=[3, 2]` puts the first item in the pool three times and the second twice.

PARAMETERS

Name	Type	Description
`population`	`list[T]` | `tuple[T, ...]` | `str`	List, tuple or string to pick from
`k`	`int`	How many items to pick, at most the population size or the expanded pool size when `counts` is supplied
`counts`	`Iterable[int]` | `None`	How many times each item is in the pool

RETURNS

`list[T]`

`random.uniform(a: float, b: float, /) → float`

Random decimal number between `a` and `b`: `random.uniform(-1, 1)`.

PARAMETERS

Name	Type	Description
`a`	`float`	One end of the range
`b`	`float`	The other end of the range

RETURNS

`float`

`random.gauss(mu: float = 0.0, sigma: float = 1.0) → float`

Random number from a bell curve around `mu` with spread `sigma`: most results fall within `sigma` of `mu`.

PARAMETERS

Name	Type	Description
`mu`	`float`	Center of the bell curve (default 0)
`sigma`	`float`	Spread of the bell curve (default 1)

RETURNS

`float`

functools

Built-in functional helpers. Works without Shared Library research.

`functools.reduce(fn: Callable, iterable: Iterable[T], initializer?: object, /) → object`

Combine an iterable into one value by repeatedly calling pure `fn(total, item)`; callbacks cannot suspend the script or mutate game state. With no initializer, the first item becomes the initial total; empty iterables then raise. Empty-with-initializer and singleton-without-initializer perform no callback call. Also available as `from functools import reduce`.

PARAMETERS

Name	Type	Description
`fn`	`Callable`	Function combining the running value with the next item
`iterable`	`Iterable[T]`	Values to combine
`initializer`	`object`	Starting value

RETURNS

`object`

`functools.total_ordering(cls: type, /) → type`

Class decorator that preserves the class object and fills missing ordering methods from `__eq__` plus one of `__lt__`, `__le__`, `__gt__`, or `__ge__`. Explicit methods are never replaced. Applying it mutates the local class namespace.

PARAMETERS

Name	Type	Description
`cls`	`type`	Class to complete

RETURNS

`type`

`functools.wraps(wrapped: Callable, /) → Callable`

Return a decorator that preserves the wrapped callable's supported name, qualified name, docstring, custom attributes, and `__wrapped__` link while keeping the wrapper's call behavior. Applying the returned decorator mutates only that local wrapper function.

PARAMETERS

Name	Type	Description
`wrapped`	`Callable`	Callable whose metadata should be copied

RETURNS

`Callable`

`functools.partial(func: Callable, /, *args: object, **keywords: object) → Callable`

Return a new callable that calls `func` with some arguments already filled in. `partial(move, rover)` is a one-argument function; later positional arguments follow the bound ones and later keywords replace bound keywords of the same name. The result carries `.func`, `.args` and `.keywords`.

PARAMETERS

Name	Type	Description
`func`	`Callable`	Function to bind arguments to
`*args`	`object`	Positional arguments to bind now; arguments passed later follow them
`**keywords`	`object`	Keyword arguments to bind now; a keyword passed later replaces the bound one

RETURNS

`Callable`

`functools.lru_cache(maxsize: int | Callable | None = 128, typed: bool = False) → Callable`

Decorator that remembers what the function returned for each set of arguments, so a repeat call returns the stored value instead of running the body. Use it on pure calculations that a loop repeats, never on a function that reads the world. A cached reading is frozen at the first call and will not follow the machine. The wrapped function gains `.cache_clear()` and `.cache_info()`. Arguments must be hashable, exactly as dict keys are.

PARAMETERS

Name	Type	Description
`maxsize`	`int` | `Callable` | `None`	How many results to keep, **128** by default. Past that the least recently used one is dropped. `None` asks for no limit, which this game caps anyway so a script that runs all session cannot grow a cache forever. A function here, `lru_cache(f)`, wraps it with the default size
`typed`	`bool`	Also key results by argument type, so arguments that compare equal but have different types never share a result

RETURNS

`Callable`

`functools.cache(user_function: Callable, /) → Callable`

Decorator that remembers every result, the same as `lru_cache(maxsize=None)`. The same warning applies: cache calculations, never world readings.

PARAMETERS

Name	Type	Description
`user_function`	`Callable`	Function to memoize

RETURNS

`Callable`

`functools.cmp_to_key(mycmp: Callable, /) → Callable`

Turn a compare function into a `key=` function: `sorted(items, key=cmp_to_key(compare))`, where `compare(a, b)` returns a negative number when `a` comes first, zero when they tie and a positive number when `b` comes first.

PARAMETERS

Name	Type	Description
`mycmp`	`Callable`	Compare function taking two items

RETURNS

`Callable`

re

Built-in regular-expression helpers. Works without Shared Library research. Patterns use a linear-time safe regular expression subset.

re.IGNORECASE: int

Case-insensitive matching flag. Short alias: `re.I`.

RETURNS

`int`

re.MULTILINE: int

`^` and `$` also match line boundaries. Short alias: `re.M`.

RETURNS

`int`

re.DOTALL: int

`.` also matches newline characters. Short alias: `re.S`.

RETURNS

`int`

re.I: int

Short alias for `re.IGNORECASE`.

RETURNS

`int`

re.M: int

Short alias for `re.MULTILINE`.

RETURNS

`int`

re.S: int

Short alias for `re.DOTALL`.

RETURNS

`int`

re.VERBOSE: int

Whitespace and `#` comments in the pattern are layout, so a long pattern can span lines. Short alias: `re.X`.

RETURNS

`int`

re.ASCII: int

`\d`, `\w` and `\s` match only ASCII characters instead of all Unicode ones. Short alias: `re.A`.

RETURNS

`int`

`re.X: int`

Short alias for `re.VERBOSE`.

RETURNS

`int`

`re.A: int`

Short alias for `re.ASCII`.

RETURNS

`int`

`re.search(pattern: str | Pattern, string: str, flags: int = 0) → Match | None`

Search anywhere in `string` for `pattern`. Returns a `Match` object, or `None` if there is no match. Patterns use the documented safe regular expression subset.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to search
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`Match | None`

`re.match(pattern: str | Pattern, string: str, flags: int = 0) → Match | None`

Match `pattern` at the start of `string`. Returns a `Match` object, or `None` if the start does not match.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to check
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`Match | None`

`re.fullmatch(pattern: str | Pattern, string: str, flags: int = 0) → Match | None`

Match the whole `string` against `pattern`. Returns a `Match` object, or `None` if any part is left unmatched.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to check
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`Match | None`

`re.findall(pattern: str | Pattern, string: str, flags: int = 0) → list`

Return all non-overlapping matches. With no capture groups, the result is a list of matched strings. With one capture group, the result is that group. With multiple capture groups, the result is tuples.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to search
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`list`

`re.sub(pattern: str | Pattern, repl: str | Callable, string: str, count: int = 0, flags: int = 0) → str`

Replace matches of `pattern` in `string` with `repl`. `count=0` replaces all matches; a positive count limits replacements. Replacement text supports backreferences like `\1`, `\g<1>` and `\g<name>`, and `repl` may instead be a function that gets each `Match` and returns its replacement.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`repl`	`str` | `Callable`	Replacement text, or a function from `Match` to text
`string`	`str`	String to transform
`count`	`int`	Maximum replacements; 0 means all
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`str`

`re.split(pattern: str | Pattern, string: str, maxsplit: int = 0, flags: int = 0) → list`

Split `string` wherever `pattern` matches. `maxsplit=0` means no limit. Capturing groups are included in the output, matching Python's `re.split` behavior.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to split
`maxsplit`	`int`	Maximum splits; 0 means all
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`list`

`re.compile(pattern: str | Pattern, flags: int = 0) → Pattern`

Compile `pattern` once into a `Pattern` whose own `search`, `findall`, `sub` and other methods reuse it: `digits = re.compile(r"\d+")`.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`Pattern`

`re.finditer(pattern: str | Pattern, string: str, flags: int = 0) → Iterator[Match]`

Every non-overlapping match of `pattern` in `string`, one `Match` at a time, for `for m in re.finditer(...)`.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`string`	`str`	String to search
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`Iterator[Match]`

`re.subn(pattern: str | Pattern, repl: str | Callable, string: str, count: int = 0, flags: int = 0) → tuple[str, int]`

Like `re.sub()`, but returns `(new_string, number_of_replacements)`.

PARAMETERS

Name	Type	Description
`pattern`	`str` | `Pattern`	Regular expression pattern
`repl`	`str` | `Callable`	Replacement text, or a function from `Match` to text
`string`	`str`	String to transform
`count`	`int`	Maximum replacements; 0 means all
`flags`	`int`	Optional flags such as `re.IGNORECASE`

RETURNS

`tuple[str, int]`

`re.escape(pattern: str, /) → str`

Put a backslash before every character that means something in a pattern, so `re.escape("1.5")` matches the text `1.5` exactly.

PARAMETERS

Name	Type	Description
`pattern`	`str`	Text to match literally

RETURNS

`str`

`re.purge() → None`

Clear the cache of compiled patterns. Patterns are compiled and cached automatically, so this is rarely needed.

RETURNS

`None`

dataclasses

Built-in record-class helpers. Works without Shared Library research. Generates concise user-class value objects; use `TypedDict` for JSON-shaped records.

dataclasses.MISSING: object

Sentinel telling a field with no default apart from one that defaults to `None`. Test it with `field.default is MISSING`.

RETURNS

`object`

dataclasses.dataclass(cls?: type | None, /, *, init: bool = True, repr: bool = True, eq: bool = True, order: bool = False, kw_only: bool = False, match_args: bool = True, unsafe_hash: bool = False, frozen: bool = False, slots: bool = False, weakref_slot: bool = False) → type

Decorate a class to generate declaration-ordered construction, representation, value equality, and optional ordering. Supports both `@dataclass` and `@dataclass(...)`, inherited fields, `__post_init__`, and explicit-method preservation. Generated behavior is controlled by `init`, `repr`, `eq`, `order`, `kw_only`, and `match_args`. The `match_args` option controls whether `__match_args__` is generated. Compatibility flags `unsafe_hash`, `frozen`, `slots`, and `weakref_slot` may be passed only as `False`; their `True` behavior and every unlisted standard-library option are rejected rather than ignored. Dataclass instances remain ordinary user objects and cannot cross JSON-shaped game API boundaries.

PARAMETERS

Name	Type	Description
`cls`	`type` | `None`	Optional class for functional or bare-decorator use
`init`	`bool`	Generate `__init__` (default `True`)
`repr`	`bool`	Generate `__repr__` (default `True`)
`eq`	`bool`	Generate exact-class `__eq__` (default `True`)
`order`	`bool`	Generate ordering methods (default `False`)
`kw_only`	`bool`	Make the generated `__init__` take every field as a keyword-only argument (default `False`)
`match_args`	`bool`	Set `__match_args__` to the fields `__init__` takes positionally, so `case Point(x, y):` matches them in order (default `True`)
`unsafe_hash`	`bool`	Compatibility flag; only `False` is supported
`frozen`	`bool`	Compatibility flag; only `False` is supported
`slots`	`bool`	Compatibility flag; only `False` is supported
`weakref_slot`	`bool`	Compatibility flag; only `False` is supported

RETURNS

`type`

`dataclasses.field(*, default?: object, default_factory?: Callable, init: bool = True, repr: bool = True, compare: bool = True, kw_only?: bool) → object`

Configure one annotated dataclass field. Use `default` for a shared value, or `default_factory` for a zero-argument factory that creates an independent value per instance; supplying both is an error. Instances of `list`, `dict`, `set`, or a class with `__hash__ = None` are rejected as direct defaults. Mutable hashable objects are accepted and shared; use `default_factory` for independent mutable values. `init`, `repr`, `compare`, and `kw_only` control construction, display, equality/ordering, and keyword-only arguments. `metadata`, `hash`, and unlisted options are unsupported.

PARAMETERS

Name	Type	Description
`default`	`object`	Optional shared value. Instances of `list`, `dict`, `set`, or a class with `__hash__ = None` are rejected; use `default_factory` for independent mutable values.
`default_factory`	`Callable`	Optional zero-argument factory
`init`	`bool`	Include this field in the generated constructor (default `True`)
`repr`	`bool`	Include this field in generated representation (default `True`)
`compare`	`bool`	Include this field in generated equality and ordering (default `True`)
`kw_only`	`bool`	Make this constructor field keyword-only

RETURNS

`object`

`dataclasses.asdict(obj: object, /) → dict[str, JsonValue]`

Return a record-class instance as a `dict`, recursively converting nested record classes and `list`, `tuple` and `dict` values. A `set` is copied as a `set` with its members unchanged. `dict` keys are converted too, so a record class used as a key raises `TypeError` when its converted `dict` cannot be used as a key. The result can be passed to `json.dumps()`, `comms.send()` or the Data Archive only when its values and keys meet that destination's rules. `list`, `tuple` and `dict` values are rebuilt; other leaf values are reused rather than deep-copied.

PARAMETERS

Name	Type	Description
`obj`	`object`	Record-class instance to convert

RETURNS

`dict[str, JsonValue]`

`dataclasses.astuple(obj: object, /) → tuple`

Return a record-class instance as a `tuple` of its field values, recursing the same way `asdict()` does. Useful as a sort key or a `dict` key when every field is hashable.

PARAMETERS

Name	Type	Description
`obj`	`object`	Record-class instance to convert

RETURNS

`tuple`

`dataclasses.fields(obj: object, /) → tuple[Field, ...]`

Return one `Field` per declared field, in declaration order. Accepts a record class or one of its instances.

PARAMETERS

Name	Type	Description
`obj`	`object`	Record class or instance to inspect

RETURNS

`tuple[Field, ...]`

`dataclasses.replace(obj: object, /, **changes: object) → object`

Return a new instance with the named fields changed and every other field copied from `obj`. The class is constructed normally, so `__init__` runs again. A field declared `init=False` cannot be replaced.

PARAMETERS

Name	Type	Description
`obj`	`object`	Record-class instance to copy
`**changes`	`object`	Fields to set by name; every other field is copied from `obj`

RETURNS

`object`

`dataclasses.is_dataclass(obj: object, /) → bool`

`True` when the value is a record class or an instance of one.

PARAMETERS

Name	Type	Description
`obj`	`object`	Value to test

RETURNS

`bool`

enum

Built-in enumerations: named constants, flags, and `int` or `str` members. Works without Shared Library research.

class Enum

Base class for enumerations: a fixed set of named members. Each assignment in the class body makes a member with `.name` and `.value`, in definition order. `Color(value)` finds a member by value, `Color[name]` by name, and iterating the class, `len(Color)` and `value in Color` cover every member. Members are singletons compared by identity and cannot be reassigned.

class ReprEnum(Enum)

An `Enum` whose `str()` and `format()` are those of its mixed-in data type, while `repr()` stays `<Color.RED: 1>`. `IntEnum`, `StrEnum` and `IntFlag` are built on it.

class IntEnum(int, ReprEnum)

An enumeration whose members are ints: they work in arithmetic, comparisons, indexing, `json.dumps` and any game function that takes a number. `str()` of a member is its number.

class StrEnum(str, ReprEnum)

An enumeration whose members are strings: they work with every string method, as dict keys equal to the plain string, in `json.dumps` and in any game function that takes a string. `auto()` gives the member name in lower case.

class Flag(Enum)

An enumeration of bit flags. Combine members with `|`, `&`, `^` and `~`; `in` tests whether one flag is set in another, iterating a combination yields its single flags, and `len` counts them. `auto()` gives successive powers of two.

class IntFlag(int, ReprEnum, Flag)

A flag enumeration whose members are ints, so a combination is also a plain number. Bits the class does not name are kept rather than refused.

class EnumType(type)

The type of every enumeration class, for `isinstance(cls, EnumType)`. Enumeration classes are built with a `class` statement or the functional form `Enum("Name", names)`, never by calling `EnumType` directly.

EnumMeta = EnumType

Another name for `EnumType`.

class FlagBoundary(StrEnum)

How a flag enumeration treats bits it does not name, given as `boundary=` in the `class` statement: `STRICT` raises, `CONFORM` drops them, `EJECT` returns a plain `int`, and `KEEP` keeps them. `Flag` defaults to `STRICT` and `IntFlag` to `KEEP`.

class EnumCheck(StrEnum)

The checks `verify` accepts: `UNIQUE`, `CONTINUOUS` and `NAMED_FLAGS`.

enum.STRICT: str

Flag boundary: a value with bits the class does not name raises `ValueError`.

RETURNS

`str`

enum.CONFORM: str

Flag boundary: bits the class does not name are dropped.

RETURNS

`str`

enum.EJECT: str

Flag boundary: a value with bits the class does not name comes back as a plain `int`.

RETURNS

`str`

enum.KEEP: str

Flag boundary: bits the class does not name are kept in the flag value.

RETURNS

`str`

enum.CONTINUOUS: str

`verify` check: no integer (or, for a flag, no bit) is missing between the smallest and largest member value.

RETURNS

`str`

enum.NAMED_FLAGS: str

`verify` check: every bit of a multi-bit flag alias belongs to a named member.

RETURNS

`str`

enum.UNIQUE: str

`verify` check: no two members share a value.

RETURNS

`str`

enum.auto(value: object = _auto_null) → object

A placeholder for a member's value. The enumeration's `_generate_next_value_` fills it in at the assignment: `Enum` and `IntEnum` count up from 1 after the largest earlier value, `StrEnum` uses the member name in lower case, and `Flag` and `IntFlag` use the next power of two. Pass a value to set it yourself.

PARAMETERS

Name	Type	Description
`value`	`object`	The member's value. Omit it to have the enumeration generate one.

RETURNS

`object`

`enum.member(value: object) → object`

Makes a class-body value a member even when it would not be one, such as a function: `@member` above a `def`, or `x = member(f)`.

PARAMETERS

Name	Type	Description
`value`	`object`	The value that becomes a member.

RETURNS

`object`

`enum.nonmember(value: object) → object`

Keeps a class-body value out of the members: `SIZE = nonmember(3)` stays an ordinary class attribute.

PARAMETERS

Name	Type	Description
`value`	`object`	The value to keep as an ordinary class attribute.

RETURNS

`object`

`enum.property(fget: Callable | None = None, fset: Callable | None = None, fdel: Callable | None = None, doc: str | None = None) → Callable`

A property for enumerations. On a member it reads like `property`; read through the class, it looks up the member of the same name. Use it for a member attribute whose name is also a member name.

PARAMETERS

Name	Type	Description
`fget`	`Callable` | `None`	Function that returns the attribute's value for a member.
`fset`	`Callable` | `None`	Function that stores a new value, or `None` to make the attribute read-only.
`fdel`	`Callable` | `None`	Function that deletes the attribute, or `None`.
`doc`	`str` | `None`	Documentation text for the attribute.

RETURNS

`Callable`

`enum.unique(enumeration: type, /) → type`

Class decorator that raises `ValueError` naming every alias when two members share a value, and returns the enumeration unchanged when every value is different.

PARAMETERS

Name	Type	Description
`enumeration`	`type`	The enumeration class to check.

RETURNS

`type`

`enum.verify(*checks: EnumCheck) → Callable`

Class decorator that checks an enumeration against `UNIQUE` (no aliases), `CONTINUOUS` (no missing integers, or no missing bits for a flag) and `NAMED_FLAGS` (every bit of a multi-bit flag alias is a member), and raises `ValueError` describing the first check that fails.

PARAMETERS

Name	Type	Description
`checks`	`EnumCheck`	The checks to apply: any of `UNIQUE`, `CONTINUOUS` and `NAMED_FLAGS`.

RETURNS

`Callable`

`enum.global_enum(cls: type, update_str: bool = False) → type`

Class decorator that copies every member into the global names of the module that defines the enumeration, and makes `repr()` show module-style names such as `__main__.RED`.

PARAMETERS

Name	Type	Description
`cls`	`type`	The enumeration class.
`update_str`	`bool`	Also change `str()` of an `IntEnum` or `StrEnum` member to the bare name.

RETURNS

`type`

enum.global_enum_repr(self: Enum) → str

The `repr()` that `global_enum` installs on an enumeration: the module name and the member name, such as `__main__.RED`.

PARAMETERS

Name	Type	Description
`self`	`Enum`	An enumeration member.

RETURNS

`str`

enum.global_flag_repr(self: Enum) → str

The `repr()` that `global_enum` installs on a flag enumeration, joining module-style names with `|`.

PARAMETERS

Name	Type	Description
`self`	`Enum`	An enumeration member.

RETURNS

`str`

enum.global_str(self: Enum) → str

The `str()` that `global_enum` installs: the bare member name.

PARAMETERS

Name	Type	Description
`self`	`Enum`	An enumeration member.

RETURNS

`str`

`enum.show_flag_values(value: int) → list[int]`

The single-bit values set in a positive integer or flag member, lowest first, as a list. Raises `ValueError` for a negative number.

PARAMETERS

Name	Type	Description
`value`	`int`	A non-negative integer or a flag member.

RETURNS

`list[int]`

`enum.pickle_by_global_name(self: Enum, proto: int) → str`

Returns the member's name. Provided so code written for CPython runs unchanged; the game has no `pickle`, so nothing calls it for you.

PARAMETERS

Name	Type	Description
`self`	`Enum`	An enumeration member.
`proto`	`int`	The `pickle` protocol number, which the function ignores.

RETURNS

`str`

`enum.pickle_by_enum_name(self: Enum, proto: int) → tuple`

Returns `(getattr, (cls, name))` for the member. Provided so code written for CPython runs unchanged; the game has no `pickle`, so nothing calls it for you.

PARAMETERS

Name	Type	Description
`self`	`Enum`	An enumeration member.
`proto`	`int`	The `pickle` protocol number, which the function ignores.

RETURNS

`tuple`

json

Built-in JSON text helpers. Turns supported values into text and back. Supported non-string dictionary keys become JSON key text; the Signal Bus and Data Archive instead require string keys and apply their own payload limits. Works without Shared Library research.

`json.dumps(obj: JsonValue, /, *, indent: int | str | None = None, sort_keys: bool = False, ensure_ascii: bool = True, separators: tuple[str, str] | None = None, allow_nan: bool = True, skipkeys: bool = False) → str`

Return `obj` as JSON text. Accepts `None`, booleans, numbers, strings, `list` and `tuple` values, and `dict` values whose keys are strings, numbers, booleans or `None`. Those non-string keys are converted to text. The Signal Bus and Data Archive instead require string dictionary keys and apply their own payload limits. A `set`, a class instance or a function raises `TypeError`; convert it first, for example with `dataclasses.asdict()`. A container that contains itself raises `ValueError`, and one nested deeper than **256** levels raises `RecursionError`, which is the same depth `loads()` reads back.

PARAMETERS

Name	Type	Description
`obj`	`JsonValue`	Value to write as JSON text
`indent`	`int` | `str` | `None`	`None` for one compact line, a number of spaces, or the literal text to indent each level with
`sort_keys`	`bool`	Write object keys in sorted order, so two equal `dict` values built in a different order produce identical text. Use this whenever the text is a cache key. The keys themselves are sorted, so numeric keys order numerically (**1, 2, 10**) and a `dict` mixing key types Python cannot compare raises `TypeError`
`ensure_ascii`	`bool`	Escape every non-ASCII character as `\uXXXX`. Pass `False` to write the characters directly
`separators`	`tuple[str, str]` | `None`	A two-item tuple of strings, the item separator and then the key separator, replacing the defaults `(", ", ": ")`
`allow_nan`	`bool`	Write `nan` and infinities as `NaN`, `Infinity` and `-Infinity`. Pass `False` to raise `ValueError` instead. Note that those three spellings are not standard JSON, and a value that round-trips here can still be refused by `comms.send()` and the Data Archive
`skipkeys`	`bool`	Silently drop `dict` entries whose key has no JSON spelling instead of raising `TypeError`

RETURNS

`str`

`json.loads(s: str, /) → JsonValue`

Read JSON text and return the value: `None`, a boolean, a number, a string, a list, or a `dict` with string keys. Malformed text raises `ValueError` naming the line, column and character position. Note that a whole number always comes back as an `int`, because in this language a whole `float` IS an `int`.

PARAMETERS

Name	Type	Description
`s`	`str`	JSON text to read

RETURNS

`JsonValue`

heapq

Built-in priority-queue helpers. Keeps an ordinary list arranged so the smallest item is always first. Works without Shared Library research.

heapq.heappush(heap: list[T], item: object, /) → None

Add `item` to `heap`, keeping the smallest item at `heap[0]`. The heap is an ordinary list, so `len()` and `heap[0]` work as usual, and only the ordering of the rest is the heap's business.

PARAMETERS

Name	Type	Description
`heap`	`list[T]`	List arranged as a heap by the other `heapq` functions
`item`	`object`	Value to add

RETURNS

`None`

heapq.heappop(heap: list[T], /) → T

Remove and return the smallest item, keeping the heap arranged. Raises `IndexError` on an empty heap, so check `len(heap)` first.

PARAMETERS

Name	Type	Description
`heap`	`list[T]`	List arranged as a heap by the other `heapq` functions

RETURNS

`T`

heapq.heappushpop(heap: list[T], item: object, /) → T

Add `item` and return the smallest item, in one pass. Faster than a push followed by a pop, and never grows the heap.

PARAMETERS

Name	Type	Description
`heap`	`list[T]`	List arranged as a heap by the other `heapq` functions
`item`	`object`	Value to add

RETURNS

`T`

heapq.heapreplace(heap: list[T], item: object, /) → T

Return the smallest item and add `item`, in one pass. The heap keeps its size. Raises `IndexError` on an empty heap.

PARAMETERS

Name	Type	Description
`heap`	`list[T]`	List arranged as a heap by the other `heapq` functions
`item`	`object`	Value to add

RETURNS

`T`

heapq.heapify(x: list, /) → None

Rearrange an existing list into a heap, in place. Cheaper than pushing the items one at a time.

PARAMETERS

Name	Type	Description
`x`	`list`	List to rearrange in place

RETURNS

`None`

`heapq.nsmallest(n: int, iterable: Iterable[T], /, key: Callable | None = None) → list[T]`

Return the `n` smallest items as a sorted list. Pass `key=` to compare something derived from each item, exactly as `sorted()` does.

PARAMETERS

Name	Type	Description
`n`	`int`	How many items to return
`iterable`	`Iterable[T]`	Values to choose from
`key`	`Callable` | `None`	Optional function called once per item; the returned values are compared instead of the items

RETURNS

`list[T]`

`heapq.nlargest(n: int, iterable: Iterable[T], /, key: Callable | None = None) → list[T]`

Return the `n` largest items as a list, largest first. Pass `key=` to compare something derived from each item, exactly as `sorted()` does.

PARAMETERS

Name	Type	Description
`n`	`int`	How many items to return
`iterable`	`Iterable[T]`	Values to choose from
`key`	`Callable` | `None`	Optional function called once per item; the returned values are compared instead of the items

RETURNS

`list[T]`

traceback

Built-in traceback helpers. Report where a caught exception came from, frame by frame. Works without Shared Library research.

`traceback.format_exc() → str`

Return the traceback of the exception currently being handled, as a string: the chain of calls that led to it, innermost last, each with its file, line and function. Call it inside an `except` block. Outside one it returns `NoneType: None`.

RETURNS

`str`

`traceback.print_exc() → None`

Write the traceback of the exception currently being handled to the console's error output. Same text as `format_exc()`, printed instead of returned. This is the answer to "the message says what broke, but where?" for an exception your own code caught.

RETURNS

`None`

BUILT-IN MODULES

math

Python's math functions and constants under `math.`, such as `math.sqrt`, `math.floor`, `math.gcd` and `math.pi`. The common ones also work without the prefix. Works without Shared Library research.

`math.pi: float`

Mathematical constant π ≈ 3.14159.

RETURNS

`float`

`math.tau: float`

Mathematical constant τ = 2π.

RETURNS

`float`

`math.e: float`

Euler's number, `2.718281828459045`.

RETURNS

`float`

`math.inf: float`

Positive infinity, larger than any number.

RETURNS

`float`

`math.nan: float`

Not a number: the result of an undefined calculation. Test for it with `math.isnan`, since `nan == nan` is `False`.

RETURNS

`float`

`math.acos(number: float, /) → float`

Arc cosine. Input must be from `-1` to `1`.

PARAMETERS

Name	Type	Description
`number`	`float`	Value from -1 to 1

RETURNS

`float`

`math.asin(number: float, /) → float`

Arc sine. Input must be from `-1` to `1`.

PARAMETERS

Name	Type	Description
`number`	`float`	Value from -1 to 1

RETURNS

`float`

`math.atan(number: float, /) → float`

Arc tangent.

PARAMETERS

Name	Type	Description
`number`	`float`	Tangent value

RETURNS

`float`

`math.atan2(y: float, x: float, /) → float`

Arc tangent of `y/x`, correctly choosing the quadrant.

PARAMETERS

Name	Type	Description
`y`	`float`	Vertical component
`x`	`float`	Horizontal component

RETURNS

`float`

`math.ceil(number: float, /) → int`

Round up to the nearest integer.

PARAMETERS

Name	Type	Description
`number`	`float`	Number to round up

RETURNS

`int`

`math.cos(radians: float, /) → float`

Cosine of an angle in radians.

PARAMETERS

Name	Type	Description
`radians`	`float`	Angle in radians

RETURNS

`float`

`math.degrees(radians: float, /) → float`

Convert radians to degrees.

PARAMETERS

Name	Type	Description
`radians`	`float`	Angle in radians

RETURNS

`float`

`math.exp(number: float, /) → float`

`e` raised to the power of the argument.

PARAMETERS

Name	Type	Description
`number`	`float`	Power of e to compute

RETURNS

`float`

`math.floor(number: float, /) → int`

Round down to the nearest integer.

PARAMETERS

Name	Type	Description
`number`	`float`	Number to round down

RETURNS

`int`

`math.isclose(a: float, b: float, rel_tol: float = 0.000000001, abs_tol: float = 0.0) → bool`

Return `True` when two numbers are close enough to treat as equal. `rel_tol` scales with the compared values; `abs_tol` sets a fixed accepted difference in the same unit, useful for values near zero and physical readings such as coordinates. Tolerances must be non-negative. This is the directly available equivalent of Python's `math.isclose()`.

PARAMETERS

Name	Type	Description
`a`	`float`	First number
`b`	`float`	Second number
`rel_tol`	`float`	Maximum relative difference
`abs_tol`	`float`	Maximum absolute difference in the values' unit

RETURNS

`bool`

`math.log(number: float, base?: float, /) → float`

Natural log, or log with the given base.

PARAMETERS

Name	Type	Description
`number`	`float`	Number greater than zero
`base`	`float`	Logarithm base; the natural logarithm when omitted

RETURNS

`float`

`math.log10(number: float, /) → float`

Base-10 logarithm.

PARAMETERS

Name	Type	Description
`number`	`float`	Number greater than zero

RETURNS

`float`

`math.log2(number: float, /) → float`

Base-2 logarithm.

PARAMETERS

Name	Type	Description
`number`	`float`	Number greater than zero

RETURNS

`float`

`math.prod(iterable: Iterable[float], /, start: float = 1) → float`

Multiplies the items of an iterable with `*`, starting from `start` (default `1`). Works for numbers and for your own classes that define `__mul__` or `__rmul__`. Empty iterables return `start`. `start` may be positional or keyword, but not both.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[float]`	Values to multiply
`start`	`float`	Value the product starts from

RETURNS

`float`

`math.radians(degrees: float, /) → float`

Convert degrees to radians.

PARAMETERS

Name	Type	Description
`degrees`	`float`	Angle in degrees

RETURNS

`float`

`math.sin(radians: float, /) → float`

Sine of an angle in radians.

PARAMETERS

Name	Type	Description
`radians`	`float`	Angle in radians

RETURNS

`float`

`math.sqrt(number: float, /) → float`

Square root. Errors on negative input.

PARAMETERS

Name	Type	Description
`number`	`float`	Number zero or greater

RETURNS

`float`

`math.tan(radians: float, /) → float`

Tangent of an angle in radians.

PARAMETERS

Name	Type	Description
`radians`	`float`	Angle in radians

RETURNS

`float`

`math.trunc(number: float, /) → int`

Drop the fractional part (round toward zero).

PARAMETERS

Name	Type	Description
`number`	`float`	Number to cut toward zero

RETURNS

`int`

`math.fabs(x: float, /) → float`

Absolute value as a float: `math.fabs(-2)` → `2.0`.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.cbrt(x: float, /) → float`

Cube root, negative numbers included: `math.cbrt(-8)` → `-2.0`.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.exp2(x: float, /) → float`

`2` raised to `x`.

PARAMETERS

Name	Type	Description
`x`	`float`	Power of two

RETURNS

`float`

`math.expm1(x: float, /) → float`

`e ** x - 1`, accurate even when `x` is tiny.

PARAMETERS

Name	Type	Description
`x`	`float`	Exponent

RETURNS

`float`

`math.log1p(x: float, /) → float`

Natural log of `1 + x`, accurate even when `x` is tiny.

PARAMETERS

Name	Type	Description
`x`	`float`	Number above -1

RETURNS

`float`

`math.sinh(x: float, /) → float`

Hyperbolic sine.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.cosh(x: float, /) → float`

Hyperbolic cosine.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.tanh(x: float, /) → float`

Hyperbolic tangent, always between `-1` and `1`.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.asinh(x: float, /) → float`

Inverse hyperbolic sine.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`float`

`math.acosh(x: float, /) → float`

Inverse hyperbolic cosine.

PARAMETERS

Name	Type	Description
`x`	`float`	Number, at least 1

RETURNS

`float`

`math.atanh(x: float, /) → float`

Inverse hyperbolic tangent.

PARAMETERS

Name	Type	Description
`x`	`float`	Number between -1 and 1

RETURNS

`float`

`math.isfinite(x: float, /) → bool`

`True` unless `x` is infinite or `nan`.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`bool`

`math.isinf(x: float, /) → bool`

`True` when `x` is positive or negative infinity.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`bool`

`math.isnan(x: float, /) → bool`

`True` when `x` is `nan`, which is not even equal to itself.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`bool`

`math.copysign(x: float, y: float, /) → float`

`x`'s size with `y`'s sign: `math.copysign(3, -0.5)` → `-3.0`.

PARAMETERS

Name	Type	Description
`x`	`float`	Size of the result
`y`	`float`	Number whose sign the result takes

RETURNS

`float`

`math.fmod(x: float, y: float, /) → float`

Remainder with the sign of `x`, unlike `%`, which takes the sign of `y`: `math.fmod(-7, 3)` → `-1.0`.

PARAMETERS

Name	Type	Description
`x`	`float`	Dividend
`y`	`float`	Divisor

RETURNS

`float`

`math.remainder(x: float, y: float, /) → float`

Distance from `x` to the nearest multiple of `y`, which can be negative: `math.remainder(7, 4)` → `-1.0`.

PARAMETERS

Name	Type	Description
`x`	`float`	Dividend
`y`	`float`	Divisor

RETURNS

`float`

`math.modf(x: float, /) → tuple[float, float]`

The fractional and whole parts of `x`, both with its sign: `math.modf(-3.25)` → `(-0.25, -3.0)`.

PARAMETERS

Name	Type	Description
`x`	`float`	Number

RETURNS

`tuple[float, float]`

`math.hypot(*coordinates: float) → float`

Length of the vector from the origin: `math.hypot(3, 4)` → `5.0`.

PARAMETERS

Name	Type	Description
`coordinates`	`float`	Coordinates

RETURNS

`float`

`math.dist(p: Iterable[float], q: Iterable[float], /) → float`

Straight-line distance between two points: `math.dist((0, 0), (3, 4))` → `5.0`.

PARAMETERS

Name	Type	Description
`p`	`Iterable[float]`	First point
`q`	`Iterable[float]`	Second point, same number of coordinates

RETURNS

`float`

`math.fsum(iterable: Iterable[float], /) → float`

Sum of floats without rounding error building up: `math.fsum([0.1] * 10)` → `1.0`, where `sum` gives `0.9999999999999999`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[float]`	Numbers to add

RETURNS

`float`

`math.gcd(*integers: int) → int`

Greatest common divisor: `math.gcd(12, 18)` → `6`.

PARAMETERS

Name	Type	Description
`integers`	`int`	Whole numbers

RETURNS

`int`

`math.lcm(*integers: int) → int`

Least common multiple: `math.lcm(4, 6)` → `12`.

PARAMETERS

Name	Type	Description
`integers`	`int`	Whole numbers

RETURNS

`int`

`math.isqrt(n: int, /) → int`

Whole-number square root, rounded down: `math.isqrt(10)` → `3`.

PARAMETERS

Name	Type	Description
`n`	`int`	Whole number, at least 0

RETURNS

`int`

`math.factorial(n: int, /) → int`

`n!`, the product of `1` to `n`: `math.factorial(5)` → `120`.

PARAMETERS

Name	Type	Description
`n`	`int`	Whole number, at least 0

RETURNS

`int`

`math.perm(n: int, k: int | None = None, /) → int`

Ways to pick `k` of `n` items in order: `math.perm(5, 2)` → `20`.

PARAMETERS

Name	Type	Description
`n`	`int`	How many items
`k`	`int` | `None`	How many to arrange (default all)

RETURNS

`int`

`math.comb(n: int, k: int, /) → int`

Ways to choose `k` of `n` items, order ignored: `math.comb(5, 2)` → `10`.

PARAMETERS

Name	Type	Description
`n`	`int`	How many items
`k`	`int`	How many to choose

RETURNS

`int`

itertools

Python's iterator building blocks under `itertools.`, such as `itertools.count`, `itertools.islice` and `itertools.groupby`. Each returns a lazy iterator, so an endless one is fine in a loop with a `break`. Works without Shared Library research.

`itertools.count(start: int | float = 0, step: int | float = 1) → Iterator[int | float]`

Count up forever from `start` by `step`: `for i in count(1):` numbers loop passes from 1. It never ends on its own, so stop with `break` or take a few with `islice`.

PARAMETERS

Name	Type	Description
`start`	`int` | `float`	First number
`step`	`int` | `float`	Added each time

RETURNS

`Iterator[int | float]`

`itertools.cycle(iterable: Iterable[T], /) → Iterator[T]`

Repeat the items of `iterable` forever, in order: `cycle(["north", "east", "south"])` for a patrol route. An empty input gives nothing.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to repeat

RETURNS

`Iterator[T]`

`itertools.repeat(object: T, times?: int) → Iterator[T]`

Give the same value `times` times, or forever when `times` is omitted: `repeat(0, 5)` yields five zeros. An explicit `None` is invalid.

PARAMETERS

Name	Type	Description
`object`	`T`	Value to give
`times`	`int`	Integer repetition count; omit for forever. An explicit `None` is invalid.

RETURNS

`Iterator[T]`

`itertools.accumulate(iterable: Iterable[T], func: Callable | None = None, *, initial: T | None = None) → Iterator[T]`

Running totals: `accumulate([1, 2, 3])` yields `1, 3, 6`. Pass `func` to combine with something other than `+`, such as `max` for a running best, and `initial=` to start from a value.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Values to combine
`func`	`Callable` | `None`	Pure function combining the total so far with the next value; `+` when omitted
`initial`	`T` | `None`	Value to start from, given first

RETURNS

`Iterator[T]`

`itertools.batched(iterable: Iterable[T], n: int, *, strict: bool = False) → Iterator[tuple[T, ...]]`

Group items into tuples of `n`: `batched("abcdefg", 3)` yields `("a", "b", "c")`, `("d", "e", "f")`, `("g",)`. With `strict=True`, a short last group raises `ValueError`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to group
`n`	`int`	Items per group, at least 1
`strict`	`bool`	Raise instead of giving a short last group

RETURNS

`Iterator[tuple[T, ...]]`

`itertools.chain(*iterables: Iterable[T]) → Iterator[T]`

Go through several iterables one after another: `chain(ore_sites, ice_sites)`.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable[T]`	Iterables to go through in order

RETURNS

`Iterator[T]`

`itertools.chain.from_iterable(iterable: Iterable[Iterable[T]], /) → Iterator[T]`

Like `chain`, reading the iterables from one iterable: `chain.from_iterable(rows)` flattens rows lazily.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[Iterable[T]]`	Iterable of iterables

RETURNS

`Iterator[T]`

`itertools.compress(data: Iterable[T], selectors: Iterable) → Iterator[T]`

Keep the items of `data` whose matching selector is truthy: `compress("abcd", [1, 0, 1, 0])` yields `"a"`, `"c"`. Stops at the shorter input.

PARAMETERS

Name	Type	Description
`data`	`Iterable[T]`	Items to pick from
`selectors`	`Iterable`	Truth values, one per item

RETURNS

`Iterator[T]`

`itertools.dropwhile(predicate: Callable, iterable: Iterable[T], /) → Iterator[T]`

Skip items while pure `predicate(item)` is truthy, then give every remaining item.

PARAMETERS

Name	Type	Description
`predicate`	`Callable`	Pure test deciding what to skip at the start
`iterable`	`Iterable[T]`	Items to read

RETURNS

`Iterator[T]`

`itertools.filterfalse(predicate: Callable | None, iterable: Iterable[T], /) → Iterator[T]`

Keep the items for which pure `predicate(item)` is falsy, the opposite of `filter`. With `None`, keeps the falsy items.

PARAMETERS

Name	Type	Description
`predicate`	`Callable` | `None`	Pure test, or `None` to test the items themselves
`iterable`	`Iterable[T]`	Items to read

RETURNS

`Iterator[T]`

`itertools.groupby(iterable: Iterable[T], key: Callable | None = None) → Iterator[tuple[object, Iterator[T]]]`

Group consecutive items that share a key, as `(key, group)` pairs; each group is an iterator over its items. Sort by the same key first to group all equal items together. Reading the next pair ends the previous group.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to group
`key`	`Callable` | `None`	Pure function giving each item's key; the item itself when omitted

RETURNS

`Iterator[tuple[object, Iterator[T]]]`

`itertools.islice(iterable: Iterable[T], start: int | None, stop: int | None = None, step: int | None = 1, /) → Iterator[T]`

Take part of an iterator without building a list: `islice(it, 3)` gives the first three items, `islice(it, 2, 10, 2)` every second item from index 2 up to 10. Skipped items are consumed; if `start` is greater than `stop`, it can still consume the first `start` items.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to read
`start`	`int` | `None`	With one number, where to stop; otherwise the first index to give
`stop`	`int` | `None`	Index to stop before, or `None` for no end
`step`	`int` | `None`	Distance between given indexes, at least 1

RETURNS

`Iterator[T]`

`itertools.pairwise(iterable: Iterable[T], /) → Iterator[tuple[T, T]]`

Neighboring pairs, lazily: `pairwise(route)` yields each `(a, b)` leg of a route.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to pair

RETURNS

`Iterator[tuple[T, T]]`

`itertools.starmap(function: Callable, iterable: Iterable, /) → Iterator[object]`

Call `function` with each item unpacked as its arguments: `starmap(pow, [(2, 3), (3, 2)])` yields `8, 9`.

PARAMETERS

Name	Type	Description
`function`	`Callable`	Pure function to call
`iterable`	`Iterable`	Rows of arguments

RETURNS

`Iterator[object]`

`itertools.takewhile(predicate: Callable, iterable: Iterable[T], /) → Iterator[T]`

Give items while pure `predicate(item)` is truthy, and stop at the first one that is not.

PARAMETERS

Name	Type	Description
`predicate`	`Callable`	Pure test deciding when to stop
`iterable`	`Iterable[T]`	Items to read

RETURNS

`Iterator[T]`

`itertools.tee(iterable: Iterable[T], n: int = 2, /) → tuple[Iterator[T], ...]`

Split one iterator into `n` independent iterators that each give every item. Use the copies, not the original, afterwards.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Iterator to split
`n`	`int`	How many copies

RETURNS

`tuple[Iterator[T], ...]`

`itertools.zip_longest(*iterables: Iterable, fillvalue: object = None) → Iterator[tuple]`

Like `zip`, but runs to the longest input and fills the missing places with `fillvalue`.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable`	Iterables to combine
`fillvalue`	`object`	Value for an input that has run out

RETURNS

`Iterator[tuple]`

`itertools.product(*iterables: Iterable, repeat: int = 1) → Iterator[tuple]`

Every combination taking one item from each input, as tuples: `product("ab", [1, 2])` yields `("a", 1)`, `("a", 2)`, `("b", 1)`, `("b", 2)`. `repeat=n` uses the inputs `n` times.

PARAMETERS

Name	Type	Description
`iterables`	`Iterable`	Inputs to combine
`repeat`	`int`	How many times to use the inputs

RETURNS

`Iterator[tuple]`

`itertools.permutations(iterable: Iterable[T], r: int | None = None) → Iterator[tuple[T, ...]]`

Every ordering of `r` items, as tuples; all items when `r` is omitted. Items are told apart by position, not value.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to order
`r`	`int` | `None`	Items per ordering

RETURNS

`Iterator[tuple[T, ...]]`

`itertools.combinations(iterable: Iterable[T], r: int) → Iterator[tuple[T, ...]]`

Every choice of `r` items, in input order and without repeats: `combinations("abc", 2)` yields `("a", "b")`, `("a", "c")`, `("b", "c")`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to choose from
`r`	`int`	Items per choice

RETURNS

`Iterator[tuple[T, ...]]`

`itertools.combinations_with_replacement(iterable: Iterable[T], r: int) → Iterator[tuple[T, ...]]`

Every choice of `r` items where an item may be chosen more than once: `combinations_with_replacement("ab", 2)` yields `("a", "a")`, `("a", "b")`, `("b", "b")`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to choose from
`r`	`int`	Items per choice

RETURNS

`Iterator[tuple[T, ...]]`

operator

Python's operators as functions under `operator.`, such as `operator.add` to pass to `reduce` and `operator.itemgetter` to pass as a `key=`. Works without Shared Library research.

`operator.abs(a: object, /) → object`

The built-in `abs(a)`: a number without its sign, or what a class's `__abs__` returns.

PARAMETERS

Name	Type	Description
`a`	`object`	Operand

RETURNS

`object`

`operator.lt(a: object, b: object, /) → object`

`a < b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.le(a: object, b: object, /) → object

`a <= b` as a function.

PARAMETERS

Name	Type	Description
a	object	Left operand
b	object	Right operand

RETURNS

`object`

operator.eq(a: object, b: object, /) → object

`a == b` as a function.

PARAMETERS

Name	Type	Description
a	object	Left operand
b	object	Right operand

RETURNS

`object`

operator.ne(a: object, b: object, /) → object

`a != b` as a function.

PARAMETERS

Name	Type	Description
a	object	Left operand
b	object	Right operand

RETURNS

`object`

operator.ge(a: object, b: object, /) → object

`a >= b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.gt(a: object, b: object, /) → object

`a > b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.not_(a: object, /) → bool

`not a` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Value to test

RETURNS

`bool`

operator.truth(a: object, /) → bool

`True` when `a` is truthy, the same test `if a:` makes.

PARAMETERS

Name	Type	Description
`a`	`object`	Value to test

RETURNS

`bool`

operator.is_(a: object, b: object, /) → bool

`a is b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`bool`

operator.is_not(a: object, b: object, /) → bool

`a is not b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`bool`

operator.is_none(a: object, /) → bool

`a is None` as a function.

PARAMETERS

Name	Type	Description
a	object	Value to test

RETURNS

bool

operator.is_not_none(a: object, /) → bool

`a is not None` as a function.

PARAMETERS

Name	Type	Description
a	object	Value to test

RETURNS

bool

operator.add(a: object, b: object, /) → object

`a + b` as a function: `reduce(operator.add, [1, 2, 3])` → `6`.

PARAMETERS

Name	Type	Description
a	object	Left operand
b	object	Right operand

RETURNS

object

operator.sub(a: object, b: object, /) → object

`a - b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.mul(a: object, b: object, /) → object

`a * b` as a function: `reduce(operator.mul, [2, 3, 4])` → `24`.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.truediv(a: object, b: object, /) → object

`a / b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.floordiv(a: object, b: object, /) → object

`a // b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.mod(a: object, b: object, /) → object

`a % b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.pow(a: object, b: object, /) → object

`a ** b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.lshift(a: object, b: object, /) → object

`a << b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.rshift(a: object, b: object, /) → object

`a >> b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.and_(a: object, b: object, /) → object

`a & b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.or_(a: object, b: object, /) → object

`a | b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.xor(a: object, b: object, /) → object

`a ^ b` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Left operand
`b`	`object`	Right operand

RETURNS

`object`

operator.neg(a: object, /) → object

`-a` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Operand

RETURNS

`object`

operator.pos(a: object, /) → object

`+a` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Operand

RETURNS

`object`

operator.invert(a: object, /) → object

`~a` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Operand

RETURNS

`object`

operator.inv(a: object, /) → object

`~a` as a function, the same as `invert`.

PARAMETERS

Name	Type	Description
`a`	`object`	Operand

RETURNS

`object`

`operator.index(a: object, /) → int`

`a` as a whole number, the conversion a list index makes: `operator.index(True)` → `1`.

PARAMETERS

Name	Type	Description
`a`	`object`	Value to convert

RETURNS

`int`

`operator.concat(a: object, b: object, /) → object`

`a + b` for two sequences, such as lists or text.

PARAMETERS

Name	Type	Description
`a`	`object`	First sequence
`b`	`object`	Sequence to add

RETURNS

`object`

`operator.contains(a: object, b: object, /) → bool`

`b in a` as a function. The container comes first.

PARAMETERS

Name	Type	Description
`a`	`object`	Container to search
`b`	`object`	Value to look for

RETURNS

`bool`

`operator.countOf(a: Iterable, b: object, /) → int`

How many items of `a` equal `b`.

PARAMETERS

Name	Type	Description
`a`	`Iterable`	Items to count in
`b`	`object`	Value to count

RETURNS

`int`

`operator.indexOf(a: Iterable, b: object, /) → int`

Position of the first item of `a` that equals `b`. Raises `ValueError` when none does.

PARAMETERS

Name	Type	Description
`a`	`Iterable`	Items to search
`b`	`object`	Value to find

RETURNS

`int`

`operator.getitem(a: object, b: object, /) → object`

`a[b]` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Container
`b`	`object`	Index or key

RETURNS

`object`

operator.setitem(a: object, b: object, c: object, /) → None

`a[b] = c` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Container
`b`	`object`	Index or key
`c`	`object`	Value to store

RETURNS

`None`

operator.delitem(a: object, b: object, /) → None

`del a[b]` as a function.

PARAMETERS

Name	Type	Description
`a`	`object`	Container
`b`	`object`	Index or key

RETURNS

`None`

`operator.attrgetter(attr: str, /, *attrs: str) → Callable`

A function that reads the named attribute, for `key=`: `sorted(points, key=attrgetter("x"))`. Several names give a `tuple`.

PARAMETERS

Name	Type	Description
`attr`	`str`	Attribute name; a dotted name reads through, such as `"pos.x"`
`*attrs`	`str`	More attribute names

RETURNS

`Callable`

`operator.itemgetter(item: object, /, *items: object) → Callable`

A function that reads the given index or key, for `key=`: `sorted(pairs, key=itemgetter(1))` sorts by each pair's second item. Several keys give a `tuple`.

PARAMETERS

Name	Type	Description
`item`	`object`	Index or key
`*items`	`object`	More indexes or keys

RETURNS

`Callable`

`operator.methodcaller(name: str, /, *args: object, **kwargs: object) → Callable`

A function that calls the named method on its argument: `map(methodcaller("strip"), lines)`.

PARAMETERS

Name	Type	Description
`name`	`str`	Method name
`*args`	`object`	Arguments for the method
`**kwargs`	`object`	Keyword arguments for the method

RETURNS

`Callable`

`operator.call(obj: Callable, /, *args: object, **kwargs: object) → object`

`obj(*args, **kwargs)` as a function.

PARAMETERS

Name	Type	Description
`obj`	`Callable`	Function to call
`*args`	`object`	Arguments
`**kwargs`	`object`	Keyword arguments

RETURNS

`object`

string

Python's text constants under `string.`, such as `string.ascii_uppercase` and `string.digits`, plus `string.capwords`. Works without Shared Library research.

string.ascii_letters: str

`ascii_lowercase` followed by `ascii_uppercase`.

RETURNS

`str`

string.ascii_lowercase: str

The letters `abcdefghijklmnopqrstuvwxyz`.

RETURNS

`str`

string.ascii_uppercase: str

The letters `ABCDEFGHIJKLMNOPQRSTUVWXYZ`.

RETURNS

`str`

string.digits: str

The text `0123456789`.

RETURNS

`str`

string.hexdigits: str

The text `0123456789abcdefABCDEF`.

RETURNS

`str`

string.octdigits: str

The text `01234567`.

RETURNS

`str`

string.punctuation: str

Every ASCII punctuation character, `!` through `~`.

RETURNS

`str`

string.printable: str

`digits`, `ascii_letters`, `punctuation` and `whitespace` together.

RETURNS

`str`

string.whitespace: str

Space, tab, newline, carriage return, vertical tab and form feed.

RETURNS

`str`

`string.capwords(s: str, sep: str | None = None) → str`

Capitalize every word: `capwords("hello world")` → `"Hello World"`. Without `sep`, words split on any whitespace and rejoin with one space; with `sep`, they split and rejoin on it.

PARAMETERS

Name	Type	Description
`s`	`str`	Text to capitalize
`sep`	`str` | `None`	Word separator (default any whitespace)

RETURNS

`str`

collections

Python's container types under `collections.`: `defaultdict`, `Counter`, `OrderedDict`, `deque` and `namedtuple`. `collections.abc` still names the abstract types for annotations. Works without Shared Library research.

`collections.Counter(iterable: object = None, /, **kwargs: int) → Counter`

A `dict` that counts things: `Counter("banana")` counts each letter, and a missing key reads as `0`. `most_common(n)` lists the biggest counts.

PARAMETERS

Name	Type	Description
`iterable`	`object`	Items to count, or a mapping of counts
`**kwargs`	`int`	Counts by name

RETURNS

`Counter`

`collections.defaultdict(default_factory: Callable | None = None, /, *args: object, **kwargs: object) → defaultdict`

A `dict` that makes a value for a missing key: with `defaultdict(list)` every new key starts as `[]`, so `groups[key].append(x)` just works.

PARAMETERS

Name	Type	Description
`default_factory`	`Callable` | `None`	Function that makes a missing key's value
`*args`	`object`	A mapping or iterable of pairs to start from
`**kwargs`	`object`	Items by name

RETURNS

`defaultdict`

`collections.OrderedDict(iterable: object = (), /, **kwargs: object) → OrderedDict`

A `dict` with order-aware extras: `move_to_end(key)` and `popitem(last=False)`, and `==` between two of them also compares their order.

PARAMETERS

Name	Type	Description
`iterable`	`object`	A mapping or iterable of pairs to start from
`**kwargs`	`object`	Items by name

RETURNS

`OrderedDict`

`collections.deque(iterable: Iterable[T] = (), maxlen: int | None = None) → deque`

A list-like queue with fast adds and removes at both ends: `append`, `appendleft`, `pop`, `popleft`. With `maxlen`, adding past the limit drops from the other end.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to start with
`maxlen`	`int` | `None`	Size limit (default `None`)

RETURNS

`deque`

`collections.namedtuple(typename: str, field_names: str | Iterable[str], *, rename: bool = False, defaults: Iterable | None = None, module: str | None = None) → type`

Make a `tuple` class with named fields: `Point = namedtuple("Point", "x y")`, then `Point(1, 2).x`. It still indexes and unpacks like a `tuple`.

PARAMETERS

Name	Type	Description
`typename`	`str`	Class name
`field_names`	`str` | `Iterable[str]`	Field names, as a list or one string such as `"x y"`
`rename`	`bool`	Replace invalid field names with `_0`, `_1` and so on
`defaults`	`Iterable` | `None`	Defaults for the last fields
`module`	`str` | `None`	Accepted and ignored

RETURNS

`type`

Variables

Store values for later use. No type declarations needed.

```
x = 10
name = "oxygen"
active = True
```

Print

Output text to the console.

```
print("Hello")
print(42)
print("Temp:", temp, "°C")
```

Type Annotations

Optional type hints on assignments and function signatures, in the standard Python (PEP 526) form. `thermometer: Thermometer = get_component("thermometer")` documents the variable's type inline; the editor uses the annotation for hover, autocomplete, and Cmd+click navigation. Function params accept the same form (`def f(x: int):`) along with a return annotation (`-> Site:`). Generic shapes work: `xs: list[Site] = []`, `m: dict[str, int] = {}`, `b: Optional[Battery] = None`. The game's id types `ItemId`, `RecipeId`, `FluidId` and `MachineTypeId` stand for every id of that kind, so with `def restock(item: ItemId):` the editor suggests item ids in each call and flags one that does not exist. Annotations help the editor understand your code. Annotation types are not evaluated and do not check or convert values at runtime. The structural exception is `@dataclass`: annotated class-attribute names become fields used to generate its constructor and other methods.

```
thermometer: Thermometer = get_component("thermometer")
xs: list[int] = []

def pick(sites: list[Site]) -> Site:
    return sites[0]
```

Ellipsis

`...` is a value. It is written as three dots or by its name `Ellipsis`, and there is exactly one of it, so `x is Ellipsis` is how you test for it. It does two jobs. As a statement it stands in for a body you have not written yet: `def plan(): ...` runs and returns `None`, the same as `pass`. Inside a type annotation it means "any number of": `tuple[float, ...]` is a `tuple` of any length, and `Callable[..., Site]` is a function that takes any arguments and returns a `Site`. It is truthy, it prints as `Ellipsis`, and it works as a `dict` key or a `set` member. Annotations are erased when a script runs, so the annotation forms cost nothing at runtime; the editor reads them for hover and autocomplete.

```
def plan_route(sites):
    ...          # a body you have not written yet

readings: tuple[float, ...] = (1.0, 2.5, 4.0)   # a tuple of any length
pick: Callable[..., Site]                       # any arguments, returns a Site

print(...)               # Ellipsis
print(... is Ellipsis)   # True
```

Tracebacks

When you catch an exception, the message tells you what went wrong and nothing about where. Import the built-in `traceback` module and call `traceback.print_exc()` inside an `except` block to print the whole chain of calls that led to it, innermost last, each line naming the script or Library, the line number and the function. `traceback.format_exc()` returns the same text as a string instead of printing it, so you can put it in a `notify()`, write it to a Signal Bus channel, or keep it for later. Outside an `except` block both report `NoneType: None`, because there is no exception to describe. An uncaught error already prints its file and line by itself; this is for the ones your own code handles, which is most of them once a script grows a Library.

```
import traceback

try:
    site = self.sonar.best_site()
    self.nav.move_to(site.id)
except Exception:
    traceback.print_exc()          # where it broke, frame by frame

try:
    risky()
except Exception:
    notify(traceback.format_exc(), "error")
```

Regular Expressions

Import the built-in `re` module for pattern matching, captures, replacements, and regex-based splitting. It works without Shared Library research and returns `Match` objects from `search`, `match`, and `fullmatch`. Pair it with raw strings (`r"..."`) so you don't have to double every backslash in a pattern.

```
import re

m = re.search(r"ore_(\d+)", "ore_42")
if m:
    print(m.group(1))
```

Raw Strings

Prefix a string with `r` to keep backslashes literal instead of treating them as escapes, handy for `re` patterns so you don't have to double every backslash. `r"\d+"` is the three characters backslash, `d`, `+`. Combine with `f` for raw f-strings (`rf"..."`), where `{...}` interpolation still works but backslashes stay literal. A backslash still escapes the closing quote, so a raw string can't end in an odd number of backslashes.

```
import re

# r"..." keeps backslashes literal: no need to double them
m = re.search(r"ore_(\d+)", "ore_42")
print(m.group(1))

label = "iron"
print(rf"\d items of {label}")
```

Bitwise & Set Operators

The `|`, `&`, `^` operators are polymorphic. On two **sets**: union, intersection, symmetric difference (`{1,2} | {2,3}` → `{1,2,3}`). On two **numbers**: bitwise OR / AND / XOR (`0xF0 & 0x33` → `0x30`). Mixed-type operands raise. Shifts `<<` / `>>` and unary `~` are integer-only. Bitwise math uses Python-style arbitrary-size integers with a safety cap for runaway values, `1 << 61` prints `2305843009213693952`, not a rounded JavaScript number. Augmented forms (`x |= 1`, `x <<= 8`) work on both shapes.

```
flags = (1 << 8) | 0xFF
masked = flags & 0xF0
toggled = flags ^ 0b1010

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)  # {1, 2, 3, 4}
print(a & b)  # {2, 3}
```

If / Else

Branch on a condition. A chain is `if`, then any number of `elif` branches, then an optional `else` that catches everything left. The first branch whose condition is true runs, and the rest are skipped, so order the branches from most specific to least. A condition is any expression: a comparison, `and` / `or` / `not`, or a value tested for truth, where `0`, `""`, an empty list or dict, and `None` are all false. Use `x if condition else y` to choose one value inline. Reach for `match` instead when many branches test the same value against fixed shapes.

```
if temp > 20:
    print("Warm")
elif temp > 0:
    print("Above freezing")
elif temp > -40:
    print("Cold")
else:
    print("Dangerously cold")

status = "warm" if temp > 0 else "cold"
```

While Loop

Repeat a block for as long as its condition is true. The condition is checked before every pass, so something inside the block has to move it toward false or the loop never ends. `break` stops the loop immediately and carries on below it. `continue` skips the rest of this pass and starts the next one. An optional `else:` block runs when the loop ends without a `break`. Loop control belongs to the current function/class-free loop body; a nested helper or class cannot break or continue its caller's loop.

```
i = 0
while i < 5:
    print(i)
    i = i + 1
```

For Loop

Walk a list, tuple, or range, running the block once per item with the current item bound to a name you choose. `break` stops the loop immediately and carries on below it. `continue` skips the rest of this pass and starts the next one. Tuple targets unpack (`for k, v in d.items():`), and an optional `else:` block runs when the loop ends without a `break`. Loop control belongs to the current function/class-free loop body; a nested helper or class cannot break or continue its caller's loop.

```
for i in range(5):
    print(i)
```

Match / Case

Branch on the shape of a value. Supports literal cases, `_` wildcard, variable captures, `|` alternatives, `if` guards, list/tuple patterns with `*rest`, and dict-style patterns. An unguarded capture or `_` always matches and must be the final reachable case; a guard may still fall through. Mapping-pattern keys must be unique (`True` and `1` count as the same key). Dict-style patterns also work with fixed result objects, e.g. `result = self.input.take("iron_ore", 10)` followed by `case {"status": "ok", "moved": moved}:`.

```
result = self.input.take("iron_ore", 10)
match result:
    case {"status": "ok", "moved": moved}:
        print("moved", moved)
    case {"status": status, "message": message}:
        print(status, message)
```

Lists & Tuples

Ordered collections and fixed pairs of values.

```
readings = [10, 20, 30]
point = (12, 8)
print(point[0])
readings.append(40)
```

Dictionaries

Key-value pairs for named data.

```
planet = {"name": "Mars", "temp": -63}
print(planet["name"])
```

`TypedDict` (record shapes)

Use `TypedDict("Name", {...})` for small record-shaped structures: controller state, route records, cached scan rows, or the simple data objects you might otherwise reach for a class/decorator to model. Annotate a variable with it (`state: Name`) and the editor autocompletes keys, flows each field's type through both `state["key"]` and `state.key`, and flags a typo'd key **before you run**. At runtime, `state.mode` and `state["mode"]` are equivalent for a dict containing the ordinary string key `"mode"`; built-in dict method names still resolve as methods. `TypedDict` adds a stable declared shape so the editor can autocomplete and type-check those fields even when the value flows through other code.

The field dictionary is ordinary code that runs, unlike an annotation, so a game type named there has to exist as a value. Import it first (`from __builtins__ import Site`, then `"site": Site`) or write the name in quotes (`"site": "Site"`). Both give identical editor type flow; built-in types such as `str`, `int` and `tuple[float, float]` need neither.

```
State = TypedDict("State", {
    "mode": str,
    "scans": int,
})

s: State = {"mode": "sonar", "scans": 0}
s.mode           # or s["mode"]: autocompletes; typos flagged
s["scans"] += 1
```

JSON

Import the built-in `json` module to turn a record into text and back. `json.dumps(value)` writes `None`, booleans, numbers, strings, lists, tuples, and dictionaries with scalar keys, converting non-string keys to text. The Signal Bus and the Data Archive accept a narrower shape (string `dict` keys, finite numbers, and their own size and nesting limits), so a value that serializes is not automatically one you can send or store. `json.dumps(value, sort_keys=True)` produces the same text for two equal dictionaries built in a different order, which is what makes it usable as a cache key. `json.loads(text)` reads it back and raises `ValueError` naming the line, column, and character when the text is malformed. A set, a class instance, or a function is refused; convert it to supported values first. `dataclasses.asdict()` converts a dataclass instance to a `dict`. Note that a whole number always comes back as an `int`, because in this language a whole `float` IS an `int`.

```
import json

plan = {"site": "b4", "tons": 12}
text = json.dumps(plan, sort_keys=True)
print(text)
print(json.loads(text)["site"])
```

Priority Queues

Import the built-in `heapq` module to keep an ordinary list arranged so the smallest item is always `heap[0]`. Push with `heapq.heappush(heap, item)` and take the smallest with `heapq.heappop(heap)`, both far cheaper than re-sorting the whole list every tick. Push `(priority, payload)` tuples when you want an explicit priority, since tuples compare by their first element. `heapq.heapify(existing_list)` arranges a list you already have, and `heapq.nsmallest(n, items, key=...)` / `heapq.nlargest(n, items, key=...)` pick the best few without sorting everything. Your own classes work too, as long as they define `__lt__`. Popping an empty heap raises `IndexError`, so check `len(heap)` first.

```
import heapq

queue = []
heapq.heappush(queue, (2, "refuel"))
heapq.heappush(queue, (1, "rescue"))

while len(queue) > 0:
    priority, job = heapq.heappop(queue)
    print(priority, job)
```

Functions

Define reusable blocks of code. Parameters can have default values (`def f(x, n=10): ...`), and callers can use either positional args (`f(5, 20)`) or keyword args (`f(5, n=20)`), same as Python. All positional args must come before any keyword args at the call site. Optional type annotations document intent: `def scan(site: MiningSite) -> Site:`. They help the editor with hover and autocomplete, but they do not change how the script runs.

```
def double(x):
    return x * 2

print(double(5))
```

Generators (`yield`)

A `def` that contains `yield` is a generator: calling it returns a lazy iterator that produces one value each time it is asked, instead of building the whole list up front. Loop over it with `for x in gen():`, pull one value with `next(it)` (raises `StopIteration` when spent, or returns a default with `next(it, fallback)`), or materialize it with `list(gen())`. `yield from other()` re-emits every value from another iterable/generator and evaluates to that generator's `return` value. Advanced control: `gen.send(v)` resumes the paused `yield` with `v`, `gen.throw(error)` raises an exception at that `yield`, and `gen.close()` stops it (running any `finally`). A generator is one-shot: once exhausted it stays empty. Note: a bare generator expression `(x for x in xs)` is now lazy too, so wrap it in `list(...)` if you need a reusable list. Generators live only in the running script and cannot be sent over the Signal Bus or stored in the Data Archive: `list(...)` them first.

```
def route(points):
    for point in points:
        yield point[0], point[1]

# Values are produced one at a time, on demand:
waypoints = [(10, 4), (14, 9), (20, 12)]
for x, y in route(waypoints):
    print("next", x, y)

# yield from re-emits another iterable's values:
def full_route(near, far):
    yield from route(near)
    yield from route(far)
```

Caching Results

Decorate a function with `@lru_cache(maxsize=...)` or `@cache` from `functools` and it remembers what it returned for each set of arguments, so a repeat call skips the body. Each script gets a limited number of steps per tick, so this matters when a loop recomputes the same score or distance for the same inputs. Only cache **pure calculations**. Put it on a function that reads a machine and the first reading is frozen forever, which is a bug that looks like the machine stopped changing. Arguments must be hashable, exactly as `dict` keys are. The wrapped function gains `.cache_info()` and `.cache_clear()`. `maxsize` bounds how many results are kept, dropping the least recently used one first; `@cache` asks for no bound, which the game caps anyway so a script that runs all session cannot grow a cache forever.

```
from functools import lru_cache

@lru_cache(maxsize=128)
def travel_cost(from_x, from_y, to_x, to_y):
    return sqrt((to_x - from_x) ** 2 + (to_y - from_y) ** 2)

print(travel_cost(0, 0, 3, 4))
print(travel_cost(0, 0, 3, 4))
print(travel_cost.cache_info().hits)
```

Classes

Bundle data and behavior together. `__init__(self, ...)` sets up each instance (`self.count = 0`), and methods take `self` first. **In a machine script the top-level `self` is the machine; inside a method `self` is the object**, pass the machine in if a method needs to drive it. `__new__(cls, ...)` decides which object a constructor hands back, for singletons and cached instances; `__getattr__(self, name)` answers names the class does not already have; `__init_subclass__(cls)` runs on a base each time a subclass is defined. The editor autocompletes `instance.` members and flows method return types through chains.

```
class Counter:
    def __init__(self, start):
        self.n = start
    def inc(self):
        self.n += 1

c = Counter(10)
c.inc()
print(c.n)
```

`@dataclass`

Import `dataclass` and `field` from `dataclasses` to turn annotated class attributes into declaration-ordered constructor fields. `dataclass` supports `init`, `repr`, `eq`, `order`, `kw_only`, and `match_args`; `field` supports `default`, `default_factory`, `init`, `repr`, `compare`, and `kw_only`. Mutable or otherwise unhashable direct defaults are rejected; use `default_factory` to create a separate value for each instance. Generated behavior includes inherited fields, `__init__`, `__repr__`, `__match_args__`, exact-class equality, optional ordering, and `__post_init__`. The compatibility spellings `frozen=False`, `unsafe_hash=False`, `slots=False`, and `weakref_slot=False` are accepted, but their `True` behavior is not supported. `ClassVar` / `InitVar` field semantics are unsupported; canonical uses receive a clear error. `asdict`, `astuple`, `fields`, `replace`, `is_dataclass`, the `MISSING` sentinel, and the `Field` records `fields()` returns are all available; `asdict()` is the supported bridge from an instance to a JSON-shaped value. `KW_ONLY`, `FrozenInstanceError`, `make_dataclass`, and public `__dataclass_fields__` introspection are unavailable; annotation types stay erased. A dataclass is still an ordinary user-class instance: use it when a record needs methods, validation, value equality, or ordering. Use `TypedDict` when the value is fundamentally a mapping or must cross a JSON-shaped boundary such as the Signal Bus or `notebook` APIs.

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Job:
    priority: int
    label: str = ""
    tags: list[str] = field(default_factory=list, compare=False)

job = Job(2, "ice run")
print(job)
```

Enumerations (enum)

Import from `enum` to give a fixed set of named constants one type: `class Mode(Enum):` with members such as `IDLE = auto()` and `MINING = auto()`. Each member is a single object with `.name` and `.value`. Compare members with `is` or `==`, look one up by value with `Mode(1)` or by name with `Mode["IDLE"]`, and loop over the class to visit every member in definition order. A plain `Enum` member is not a string, so `mode == "IDLE"` is always `False`: compare with `Mode.IDLE`, or with `mode.name`. In `match`, write `case Mode.IDLE:`, because a bare `case IDLE:` captures every value instead of checking for the member. `IntEnum` and `StrEnum` members are real numbers and strings, so they compare equal to plain values and travel on the Signal Bus and through `json.dumps` as those values; send a plain member's `.name` or `.value` instead. `Flag` and `IntFlag` members combine with `|`, `&`, `^` and `~` into sets of options you test with `in` and loop over. `@unique` refuses aliases and `@verify(...)` checks the values; `_missing_`, `_generate_next_value_`, `_ignore_`, `member()`, `nonmember()`, `@property`, `@global_enum` and the functional form `Enum("Mode", "IDLE MINING")` behave as in Python. Hooks the class calls for you (`_missing_`, `_generate_next_value_`, `__new__`) finish at once, so they cannot `sleep()` or run machine actions. Only `int`, `str` and `float` can be mixed in as a member data type. The editor completes members, knows each member's `name` and `value`, and warns on both mistakes above.

```
from enum import Enum, IntFlag, auto

class Mode(Enum):
    IDLE = auto()
    MINING = auto()

class Need(IntFlag):
    POWER = auto()
    WATER = auto()

mode = Mode.MINING
match mode:
    case Mode.IDLE:
        print("waiting")
    case Mode.MINING:
        print("digging", mode.name)

needs = Need.POWER | Need.WATER
print(Need.WATER in needs)
```

Inheritance & `super()`

`class Dog(Animal):` inherits `Animal`'s methods and attributes. Override any of them, and call the base version with `super().method(...)`. Multiple inheritance resolves by Python's C3 MRO, so cooperative `super()` works across a diamond. `isinstance(x, Animal)` and `issubclass(Dog, Animal)` walk the chain.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):
    def speak(self):
        return "woof"

print(Dog("Rex").speak())
```

Custom exceptions

Derive your own exception classes from `Exception` (or any built-in exception) with `class LowBattery(Exception):`. Raise them with `raise LowBattery(level)`, catch them by class with `except LowBattery as err:`, and `except Exception:` still catches them through the hierarchy. An exception object keeps its constructor arguments on `.args` (one message argument makes `str(err)` that message), any extra attributes your `__init__` sets, `.__cause__` from `raise ... from`, and `.__context__` for the exception that was being handled. Override `__str__` to control the console text. The editor autocompletes your exception's members after `except ... as err:` and flags a `raise` or `except` target that is not an exception class.

```
class LowBattery(Exception):
    def __init__(self, level):
        super().__init__(f"battery at {level}%")
        self.level = level

def check(rover):
    level = rover.battery.percent()
    if level < 20:
        raise LowBattery(level)

try:
    check(get_component("rover_1"))
except LowBattery as err:
    print(err, err.level, err.args)
except Exception as err:
    print("other failure:", err)
```

Operators & dunder methods

Define what operators and builtins do on your objects. `__eq__` / `__lt__` power `==` and `<`; `__add__` (and reflected `__radd__`) power `+`; `__len__`, `__bool__`, `__getitem__`, `__contains__`, `__iter__`, `__call__`, and `__str__` / `__repr__` dispatch from `len()`, truthiness, `x[k]`, `in`, `for`, calling, and `print`. Operator overloads must be **pure**, they cannot `sleep()` or change game state.

```
class Vec:
    def __init__(self, x):
        self.x = x
    def __add__(self, o):
        return Vec(self.x + o.x)
    def __eq__(self, o):
        return self.x == o.x
    def __repr__(self):
        return "Vec(" + str(self.x) + ")"

print(Vec(1) + Vec(2))
print(Vec(3) == Vec(3))
```

Decorators

`@deco` above a `def` or `class` wraps it: `name = deco(name)`. Stack them, and pass arguments (`@deco(arg)`). Built-in method decorators: `@staticmethod` (no `self`), `@classmethod` (receives the class as `cls`), and `@property`. The standard `dataclasses` module provides `@dataclass` for generated record-class behavior.

```
def trace(fn):
    def wrapper(*args):
        print("calling", fn)
        return fn(*args)
    return wrapper

@trace
def step(n):
    return n * 2

print(step(5))
```

`@property`

`@property` turns a method into a computed attribute read **without parens** (`tank.level` , not `tank.level()`). Add `@name.setter` so `tank.level = 42` runs validation. Use it to expose derived or guarded state while keeping plain attribute syntax.

```
class Tank:
    def __init__(self):
        self._level = 0
    @property
    def level(self):
        return self._level
    @level.setter
    def level(self, v):
        self._level = max(0, v)

t = Tank()
t.level = 42
print(t.level)
```

Generic types

Import `TypeVar` and `Generic` from `typing` to describe functions and classes that preserve a caller's value type. `T = TypeVar("T")` defines a type variable; use it in annotations such as `list[T]` or a class base such as `Generic[T]`. Specializing `Box[str]` lets the editor carry that type through its methods. These annotations guide editor assistance; they do not convert or validate stored values at runtime.

```
from typing import Generic, TypeVar

T = TypeVar("T")

def first(values: list[T]) -> T:
    return values[0]

class Box(Generic[T]):
    def __init__(self, value: T):
        self.value = value

    def get(self) -> T:
        return self.value

label = first(["ice", "iron"])
box = Box[str](label)
print(box.get().upper())
```

Clock

The ship's clock. It tracks the time of day, the day count, and the sun's position, everything a script needs for day-night timing and solar tracking.

Type	Infrastructure

Access via: `get_component("clock")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_time() → list[int]`

Current time as a **3-element list** `[hours, minutes, seconds]` in 24-hour format. Index with `t[0]`, `t[1]`, `t[2]`. Use for time-of-day branches or to wait for specific hours.

RETURNS

`list[int]`

`[hours, minutes, seconds]`.

`.get_day() → int`

Current day number, starts at **1** and increments when the in-world clock rolls past midnight. Use for daily-budget logic (e.g. reset counters at the start of each day) or to detect day transitions for machines like the Heat Generator whose state changes per day.

RETURNS

`int`

The day count.

`.get_time_of_day() → str`

Current daylight phase as a string: `"dawn"`, `"day"`, `"dusk"`, or `"night"`. This is the simulation phase used by solar/day-night logic; the header may further label `"day"` as Morning/Afternoon/Evening for flavor.

RETURNS

`str`

POSSIBLE VALUES

`"dawn"`, `"day"`, `"dusk"`, `"night"`

`.get_elevation() → float`

Sun's elevation above the horizon (**0-90** degrees). **0** at night, rises to **90** at solar noon (equator), back to **0** at dusk. Solar panels peak when their tilt complements the current elevation.

RETURNS

`float`

0-90 degrees.

`.tick() → int`

Deterministic simulation tick since save start. At normal speed the simulation grants a fresh script step budget every tick (**10 ticks/sec**, so one tick is **0.1** simulation seconds). Use tick deltas for profiling script timing instead of wall-clock milliseconds.

RETURNS

`int`

Simulation tick since save start.
Deterministic; one normal-speed
tick is 100 ms of simulation time.

`.elapsed_seconds() → float`

Elapsed simulation seconds since save start. This is the same time base that `sleep(seconds)` waits against, not browser wall-clock time.

RETURNS

`float`

Elapsed simulation seconds since
save start.

`.elapsed_game_hours() → float`

Elapsed world-clock hours since save start. Useful for rate calculations and logs that should follow the compressed day/night cycle instead of real seconds.

RETURNS

`float`

Elapsed world-clock hours since save start.

`.real_seconds_per_hour() → float`

Number of real seconds in one world-clock hour. The day cycle compresses **24** world-clock hours into a fixed real-time window, so `sleep(clock.real_seconds_per_hour())` waits exactly one world-clock hour and multiplying by **24** waits a full day. Lets scripts express world-time delays without hardcoding the conversion.

RETURNS

`float`

Real seconds in one world-clock hour. Multiply by `24` for a full day. Use for planet-aware `sleep()` calls (`sleep(clock.real_seconds_per_hour())` waits exactly one world-clock hour regardless of pacing).

Commander

Read the player's name and current credits with `get_component("me")` or `get_component("commander")`. Scripts cannot change either value.

Access via: `get_component("commander")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

METHODS

`.get_name() → str`

Your commander name as a string. Set during initial character creation (or default). Use for personalized dashboard messages.

RETURNS

`str`

`.get_credits() → int`

Current credit balance. Changes when `shop.buy()` / `shop.sell()` run, Bio Exchanges pay out, contract transmissions succeed, and Orders complete. Use as a gate before expensive `shop.buy()` calls.

RETURNS

`int`

Console

Writes structured script output to the same Console used by `print()`. Access it with `get_component("console")`; no research is required. Messages can have a severity, named channel, color, and timestamp.

Access via: `get_component("console")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.print(message: object, level: str = "info", channel: str = "", color: str = "", timestamp: bool = False) → ActionResult`

Print a line with full control. `level` is `info` / `warn` / `error` / `debug` (which feed the WARNINGS / ERRORS filters), or any other non-empty string for a custom level shown as a colored badge. An empty level behaves like `info`. `channel` routes the line to a named tab (empty = the main stream). `color` is a theme token (`"warning"` , `"success"` , `"accent"`), which recolors with the theme, or any CSS color: hex (`"#aabbcc"`), `"rgb(255,100,0)"` , `"hsl(30,100%,50%)"` , or a name like `"orange"` . A `timestamp` of `True` prepends the game time-of-day. Example: `get_component("console").print("Overheat", "alert", "alarms", "warning", True)` .

PARAMETERS

Name	Type	Description
`message`	`object`	Value to print.
`level`	`str`	Severity or custom level. Built-ins: `info` / `warn` / `error` / `debug` . An empty string behaves like `info` ; any other non-empty string is a custom level shown as a colored badge.
`channel`	`str`	Channel/tab name. Empty = the main stream.
`color`	`str`	Theme token (recolors with the theme), or any CSS color: hex (`"#aabbcc"`), `"rgb(255,100,0)"` , `"hsl(...)"` , or a name like `"orange"` .
`timestamp`	`bool`	Prepend the game time-of-day.

RETURNS

`ActionResult`

RESULT FIELDS

`.status` , `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.info(message: object, channel: str = "", color: str = "", timestamp: bool = False) → ActionResult`

Print an info line (the default level). Its optional `channel`, `color`, and `timestamp` parameters behave like those on `print`. Equivalent to `print(message)`.

PARAMETERS

Name	Type	Description
`message`	`object`	Value to print.
`channel`	`str`	Channel/tab name. Empty = the main stream.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, or a named color).
`timestamp`	`bool`	Prepend the game time-of-day.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.warn(message: object, channel: str = "", color: str = "", timestamp: bool = False) → ActionResult`

Print a warning line, appears in the console's WARNINGS filter. Its optional `channel`, `color`, and `timestamp` parameters control routing and presentation. For an interruptive popup instead, use the global `notify(text, "warn")`.

PARAMETERS

Name	Type	Description
`message`	`object`	Value to print.
`channel`	`str`	Channel/tab name. Empty = the main stream.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, or a named color).
`timestamp`	`bool`	Prepend the game time-of-day.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.error(message: object, channel: str = "", color: str = "", timestamp: bool = False) → ActionResult`

Print an error line, appears in the console's ERRORS filter. Its optional `channel`, `color`, and `timestamp` parameters control routing and presentation. This is your own message at error severity, not an uncaught exception.

PARAMETERS

Name	Type	Description
`message`	`object`	Value to print.
`channel`	`str`	Channel/tab name. Empty = the main stream.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, or a named color).
`timestamp`	`bool`	Prepend the game time-of-day.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.debug(message: object, channel: str = "", color: str = "", timestamp: bool = False) → ActionResult`

Print a low-priority debug line, hidden from the ALL view unless the player enables debug output. Its optional `channel`, `color`, and `timestamp` parameters control routing and presentation.

PARAMETERS

Name	Type	Description
`message`	`object`	Value to print.
`channel`	`str`	Channel/tab name. Empty = the main stream.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, or a named color).
`timestamp`	`bool`	Prepend the game time-of-day.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.now() → str`

Return the current game time-of-day as a `"HH:MM:SS"` string, for building your own line prefixes when you want full control over formatting.

RETURNS

`str`

Game time of day as `"HH:MM:SS"`.

`.clear(channel: str = "") → ActionResult`

Clear output produced by this script. With a `channel` argument, clears only this script's lines in that channel; with no argument, clears all output from this script. Other scripts and system messages are preserved.

PARAMETERS

Name	Type	Description
`channel`	`str`	Optional channel from this script to clear. Empty = every channel from this script.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

GPS

A ship sensor that reports which outpost you're viewing, its name and coordinates, and how many buildings are deployed there. Switching outposts on the dashboard retargets it.

Type	Infrastructure

Access via: `get_component("gps")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.planet() → Nocturna`

Returns the current planet component. Use `gps.planet().id` for stable ids such as `"nocturna"` when calling Journal APIs, and `gps.planet().get_name()` for the display name `"Nocturna"`.

RETURNS

`Nocturna`

Use `.id` for stable journal ids and `.get_name()` for display.

`.site_name() → str`

Returns the current outpost's display name. `"Nocturna Base"` for the home outpost (default, players can rename); `"Outpost 1"`, `"Outpost 2"`, … for player-founded outposts.

RETURNS

`str`

The current outpost's name, e.g. `"Nocturna Base"`.

`.coords() → list[int]`

Returns the current outpost's world coordinates as a **2-element list** `[x, y]`. The home outpost sits at `[0, 0]`; founded outposts carry the position the player chose in Plan mode.

RETURNS

`list[int]`

`[x, y]` of the current outpost.

`.buildings_used() → int`

Returns the number of buildings deployed at the **current** outpost. Sensors, mobile units, structural hubs, and POI extraction machines don't count, only shop-purchased deployable buildings.

RETURNS

`int`

Buildings deployed at the current outpost.

`.buildings_capacity() → int`

Returns the soft building threshold at the current outpost. Each counted building above it reduces productive and service throughput. Nocturna Base has a few extra starter slots; founded outposts use the standard threshold.

RETURNS

`int`

Soft building threshold at the current outpost.

`.is_full() → bool`

Returns `True` when the current outpost has reached or exceeded its soft building threshold. The threshold itself does not block ordinary deployment.

RETURNS

`bool`

`True` if the soft building threshold is reached or exceeded.

`.is_home() → bool`

Returns `True` when the current outpost is the home outpost (the one the player started at, default name `"Nocturna Base"`). Useful for branching on whether you're managing the spawn site versus a remote outpost.

RETURNS

`bool`

`True` if the current outpost is the home base.

Nocturna

Provides stable planet-wide data for Nocturna through `get_component("nocturna")`, including map bounds, biomes, terraforming progress, and permanent map contacts. Hidden weather aftermaths are deliberately absent; their coordinates exist only in the storm packets your station network captures. Sonar discoveries and surveys are stored separately in `get_component("journal")`.

Access via: `get_component("nocturna")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_name() → str`

Display name of this planet, `"Nocturna"` in English. It follows the game's language setting, so use it for output and never compare it with fixed text.

RETURNS

`str`

The display name.

`.get_bounds() → Bounds`

Surface coordinate bounds as a `Bounds` object with `.min_x`, `.max_x`, `.min_y`, `.max_y`, all in meters from base. Use to clamp nav targets inside the operational surface, or to generate random valid coordinates: `import random; b = planet.get_bounds(); x = random.randint(b.min_x, b.max_x); y = random.randint(b.min_y, b.max_y)`. See `Bounds`.

RETURNS

`Bounds`

In meters.

`.contains(x: float, y: float) → bool`

`True` if the point `(x, y)` is inside Nocturna's surface bounds. Use as a safety check before `nav.set_target()`: `if nocturna.contains(x, y): self.nav.set_target(x, y)`. Targeting outside the bounds returns immediately with no drive.

PARAMETERS

Name	Type	Description
`x`	`float`	X coordinate in meters
`y`	`float`	Y coordinate in meters

RETURNS

`bool`

`True` if the point is inside the planet's surface bounds.

`.biome_at(x: float, y: float) → str`

Biome id at the world coordinate `(x, y)`. Returns one of `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`. Pure geography, same answer for the same coordinate forever. Use to script biome-aware behavior anywhere on the planet: `if nocturna.biome_at(x, y) == "volcanic": deploy_heat_resistant()`.

PARAMETERS

Name	Type	Description
`x`	`float`	World x coordinate.
`y`	`float`	World y coordinate.

RETURNS

`str`

POSSIBLE VALUES

`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.life_form_biome(item_id: str) → str | None`

Find the native biome for a life-form item: `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, or `"deep"`. Returns `None` for other items. A species always belongs to the same biome, regardless of where the specimen was found. Essence Liquifiers accept only life forms native to their outpost, so a drone can test `nocturna.life_form_biome(item) == self.outpost.biome` before unloading.

PARAMETERS

Name	Type	Description
`item_id`	`str`	A life-form item id, e.g. `"vent_fungus"`.

RETURNS

`str | None`

Native biome id for the life form, or `None` for another item type. The value is fixed per species and matches the biome whose Essence Liquifier accepts it.

POSSIBLE VALUES

`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.biomes() → list[str]`

Lists every biome id on the planet. Use it for planet-wide loops: `for biome in nocturna.biomes(): print(biome)`. The set never changes during play.

RETURNS

`list[str]`

Every biome id on the planet.

`.terraform_progress() → float`

Terraform Index progress on a **0-100%** scale: **0** is untouched; **100** means 1,000,000 TP and all six pillars are complete. Atmosphere owns 70% of the Index; biomass, plants, and wildlife each own a separate 10% that cannot substitute for another pillar. Gate progress logic with it: `if nocturna.terraform_progress() >= 50: ...`. For the raw TP value, use `total_tp()`.

RETURNS

`float`

Terraform Index progress as a percent (**0-100**).

`.total_tp() → float`

The finite Terraform Index in TP, in the **0-1,000,000** range. Temperature, oxygen, and pressure together own 700,000 TP; biomass, plants, and wildlife each own a non-substitutable 100,000 TP. Raw metrics may keep growing after their final phase, but a completed pillar contributes no additional Index. The value gates cross-system research.

RETURNS

`float`

Finite Terraform Index points (**0-1,000,000**) used by cross-system research.

`.points_of_interest() → list[PointOfInterest]`

Lists every permanent "?" contact on the Planet Map so scripts can route to real sites. Each `PointOfInterest` has whole-number coordinates, a `scanned` flag, and a `kind` that stays `"unknown"` until a scanner reaches the contact. Filter for `not point.scanned`, travel to its coordinates, and scan with Rover or Pioneer sonar or a drone Bio Scanner. A contact the instrument you brought cannot identify stays unscanned even after a sweep that succeeded, and the sweep lists it in `scan.blocked` with the same coordinates and a `reason`: record those or your loop reselects the same contact. See `PointOfInterest`.

RETURNS

`list[PointOfInterest]`

Every physical map contact (the "?" markers). `.kind` reads `"unknown"` until you scan the contact.

Research

Checks global research progress through `get_component("research")`. Use the public ids shown on the Research page, such as `"research_auto_feeders"`. Available from the beginning and read-only.

Access via: `get_component("research")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.is_unlocked(research_id: str) → bool`

Return `True` only when `research_id` is known and unlocked. Known but locked research returns `False`. An unknown id also returns `False` without printing, so scripts can handle every lookup result themselves.

PARAMETERS

Name	Type	Description
`research_id`	`str`	Public research id, for example `research_auto_feeders`.

RETURNS

`bool`

`True` only when the known research is unlocked and available in the current build.

`.unlocked() → list[str]`

Return a fresh list of public research ids that are unlocked and available in the current build. The list follows stable Research-page registry order, contains no internal capability ids, and can be modified without changing game state.

RETURNS

`list[str]`

Public research ids unlocked and available in the current build, in stable research-registry order.

Shop

Buys from and sells to Earth. Use `get_component("shop")` to automate surplus sales or purchases when a threshold is reached. The same catalogue and prices are used by the Shop UI.

Access via: `get_component("shop")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

METHODS

`.sell(item_id: str, quantity: int = 1) → SaleResult`

Sell a positive whole-number `quantity` of `item_id`, defaulting to **1**. The complete quantity is removed from the lowest-indexed matching Inventory slots in one transaction; if Inventory contains fewer units, nothing is sold. Battery products refund their charge percentage, with a **50% minimum**; fully charged batteries refund their full normal value. Use the Inventory page when you need to choose one exact battery instance.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to sell
`quantity`	`int`	Positive whole-number units to sell; defaults to 1

RETURNS

`SaleResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.item_id`, `.units`, `.credits`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Sale completed: `.item_id` × `.units`; received `.credits` cr.
`"not_sellable"`	● rejection	The requested item cannot be sold.
`"no_stock"`	● rejection	Inventory does not contain the requested quantity of the item.

RAISES

Exception	Condition
`TypeError`	`quantity` must be a finite whole number.
`ValueError`	`quantity` must be greater than zero.
`OverflowError`	`quantity` must fit within the supported whole-number range.

`.sell_all(item_id: str) → SaleResult`

Sell every unit of `item_id` currently in Inventory in one transaction. There is no per-unit cooldown. Each battery product is valued from its own retained charge, with a **50% minimum** and full normal value at full charge.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to sell every stack of

RETURNS

`SaleResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.item_id`, `.units`, `.credits`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Sale completed: `.item_id` × `.units`; received `.credits` cr.
`"not_sellable"`	rejection	The requested item cannot be sold.
`"no_stock"`	rejection	Inventory does not contain the requested quantity of the item.

`.buy(item_id: str, quantity: int = 1) → ActionResult`

Buy a positive whole-number `quantity` of `item_id`, defaulting to **1**. The complete quantity must be affordable and fit in base Inventory; otherwise nothing is charged or delivered. Purchases are placed in base Inventory, not delivered directly to a machine or remote outpost.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Shop item id (machine kit, module, pack, reagent, or equipment)
`quantity`	`int`	Positive whole-number units to buy; defaults to 1

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"insufficient_credits"`	● rejection	The available credits are below the total cost of the requested purchase.
`"inventory_full"`	● rejection	Inventory has no capacity for the result.

RAISES

Exception	Condition
`TypeError`	`quantity` must be a finite whole number.
`ValueError`	`quantity` must be greater than zero.
`OverflowError`	`quantity` must fit within the supported whole-number range.

`.get_catalogue() → list[ShopItem]`

Every available catalogue entry as a list of `{id, name, cost}` objects. Use to pick a target dynamically or to show a filtered picker in a script. The Earth shop never runs out of catalogue items; entries hidden by tech gates don't appear.

RETURNS

`list[ShopItem]`

Transmitter

Sends data to other planets. Use it to report sensor readings to Earth or submit contract answers. Call `connect()` to choose a planet, then `transmit(key, value)` to send data; `disconnect()` clears the connection. A connection lasts only for the current script run, so each transmitting script must connect first. Save `get_component("transmitter")` to a variable and reuse it for both calls.

Access via: `get_component("transmitter")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.list_planets() → list[Planet]`

Every available transmission destination as a list of `Planet` objects (each with `.id`, `.name`, etc.). Call once at script start to see what is available; pass a returned `.id` to `connect(id)`.

RETURNS

`list[Planet]`

`.connect(planet: str) → ActionResult`

Open a channel to the planet with the given id: `result = transmitter.connect("earth")`. The id must be lowercase (from `list_planets()`). Read `transmitter.get_info().target` after success. Connection lasts only for the current script run, if your script restarts, `connect()` again before `transmit()`.

PARAMETERS

Name	Type	Description
`planet`	`str`	Planet id to connect to

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.

`.disconnect()` → `ActionResult`

Close the current script-run channel. This does not affect contracts or any other script; it only clears this Transmitter object's active target so later `transmit()` calls must `connect()` again.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The Transmitter's current script-run channel is disconnected.

`.get_info()` → `TransmitterInfo`

Current connection status. Returns an object with `.connected` (boolean) and `.target` (connected planet id, or `"none"`). Use as a guard before `transmit()`: `if transmitter.get_info().connected: transmitter.transmit(...)`.

RETURNS

`TransmitterInfo`

`.transmit(key: str, value: JsonValue) → ActionResult`

Send a named value to the connected planet with `transmitter.transmit(key, value)`. Opening sensor telemetry is unavailable until the power and sensor onboarding steps are complete and the uplink step is active. For sensor readings, use the name requested by Earth, such as `"current_temperature"`. For contract answers, use `self.contract.id`. The data arrives in the same tick.

PARAMETERS

Name	Type	Description
`key`	`str`	A sensor-telemetry key such as `"current_temperature"` (what the opening uplink asks for), or a contract id read from `self.contract.id` inside a contract script.
`value`	`JsonValue`	The reading or computed answer to send.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"correct"`	success	The answer solved the active contract. The reward was paid and the contract is now complete.
`"incorrect"`	rejection	The submitted value is incorrect.
`"already_completed_correct"`	success	The contract is already completed. The value submitted now is correct.
`"already_completed_incorrect"`	rejection	The contract is already completed. The value submitted now is incorrect.
`"accepted"`	success	The reading matched the planet value for the requested key. This is the telemetry path used by the uplink step, not a contract answer.
`"rejected"`	rejection	The submitted value or request was rejected.
`"not_connected"`	rejection	The component has no active connection.
`"wrong_planet"`	rejection	The operation targets a different planet.
`"wrong_contract"`	rejection	The supplied contract is not active for this operation.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"unknown_contract"`	rejection	The supplied contract identifier does not exist.

Status	Kind	Meaning
`"key_is_planet"`	● rejection	The supplied key names a planet rather than a telemetry or contract key.
`"unknown_key"`	● rejection	The supplied telemetry or contract key does not exist.

Biomass Sensor

Reports cultivated biomass on the planet, in tons. Updates live as Biomass Mixers produce. Available after the Biosphere research lands; no calibration step.

Access via: `get_component("biomass_sensor")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → float`

Returns total biomass tonnage on the planet as a number. `0` before any Biomass Mixer has produced.

RETURNS

`float`

Tons of biomass.

```
component = get_component("biomass_sensor")
value = component.get_value()
print(value)
```

Oxygen Sensor

An atmospheric oxygen probe that landed broken. It reports a raw voltage until a script works out the calibration and repairs it, after which it reads oxygen directly.

Type	Sensors

Access via: `get_component("oxygen_sensor")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → float`

Before repair, read raw voltage from the uncalibrated probe as a small decimal value. This is not yet a ppt reading; compare it with a known reference to calculate the calibration factor. After repair, read the current atmospheric oxygen level in ppt directly.

RETURNS

`float`

Raw voltage before repair; atmospheric oxygen in ppt after repair.

`.calibrate(value: float) → ActionResult`

Start the black-box calibration suite with the processed value: `result = self.calibrate(raw_value * factor)`. The suite then checks the whole script against several readings. A fully correct suite repairs the sensor; failed test cases remain visible in the console.

PARAMETERS

Name	Type	Description
`value`	`float`	Calibration value to test

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"started"`	success	The operation started.
`"already_repaired"`	success	The target is already repaired.
`"no_source"`	rejection	No source is configured or available.
`"already_testing"`	transient	The requested test is already running.

`.set_status(message: str, level: str = "info") → None`

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None`

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

```
component = get_component("oxygen_sensor")
value = component.get_value()
print(value)
```

Plants Sensor

Reports permanent vegetated km² produced by the Plant Terraformer fleet. Available after the Biosphere research lands. Field growth alone does not change this value.

Access via: `get_component("plants_sensor")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → float`

Returns permanent Plants km² as a number. Plant Terraformers are the only writers; repeated crop cycles, diversity, providers, Fertilizer, and Yield Amplifier increase the physical Forage supply they process.

RETURNS

`float`

km² planted.

```
component = get_component("plants_sensor")
value = component.get_value()
print(value)
```

Pressure Sensor

An atmospheric pressure probe that landed broken. Its readings come out scrambled until a script stabilizes the repair signal, after which it reads pressure directly.

Type	Sensors

Access via: `get_component("pressure_sensor")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → int`

Current unstable repair reading as an integer. If the value is odd, add **1**; if it is even, use it unchanged. After repair, this method returns real atmospheric pressure in kPa.

RETURNS

`int`

A scrambled reading until the sensor is repaired; kPa after repair.

`.stabilize(value: float) → ActionResult`

Start the black-box stabilization suite with the corrected even reading: `result = self.stabilize(corrected_value)`. The suite then checks the whole script against several readings. A fully correct suite repairs the sensor; failed test cases remain visible in the console.

PARAMETERS

Name	Type	Description
`value`	`float`	Stabilization value to test

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"started"`	success	The operation started.
`"already_repaired"`	success	The target is already repaired.
`"no_source"`	rejection	No source is configured or available.
`"already_testing"`	transient	The requested test is already running.

`.set_status(message: str, level: str = "info") → None`

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None`

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

```
component = get_component("pressure_sensor")
value = component.get_value()
print(value)
```

Thermometer

Always-working surface temperature probe, no calibration needed. Reads the planet's current surface temperature in **°C** directly.

Type	Sensors

Access via: `get_component("thermometer")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → float`

Current surface temperature in **°C** as a number. Safe to call from any script; no repair step needed. This is the display °C, for the heat-units progression metric that research thresholds compare against, read `get_component("atmosphere").get_heat()`.

RETURNS

`float`

In °C.

`.set_status(message: str, level: str = "info") → None`

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None`

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

```
component = get_component("thermometer")
value = component.get_value()
print(value)
```

Wildlife Sensor

Reports total individual fauna across all established Habitat colonies. Available after Biosphere research lands; reads `0` until the first Wildlife colony establishes.

Access via: `get_component("wildlife_sensor")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_value() → int`

Returns the current Wildlife population count as a number, summed from established Habitat colonies.

RETURNS

`int`

Individuals.

```
component = get_component("wildlife_sensor")
value = component.get_value()
print(value)
```

Journal

Stores sites found or surveyed by sonar and fragments cataloged by Bio Labs. Access it with `get_component("journal")` to plan trips and Bio Orders without scanning again. Records are separated by planet and survive script restarts, vehicle changes, and save/load.

Access via: `get_component("journal")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.discovered_sites(planet_id: str) → list[Site]`

Lists every site classified by sonar on `planet_id`. Call `journal.discovered_sites("nocturna")` for Nocturna. Each entry is a `MiningSite`, `ThermalVent`, `WaterWell`, `OilWell`, `ExoticDeposit`, or `GeologicalAnomaly`, according to `kind()`. Unsurveyed productive sites leave their detailed fields as `None`; inert formations are resolved by scanning. Duplicate scans do not add duplicate entries. Returns an empty list before any sites are found. See `Site`.

PARAMETERS

Name	Type	Description
`planet_id`	`str`	Journal planet id; the current planet is usually `"nocturna"`.

RETURNS

`list[Site]`

Classified sites for that planet; each concrete type follows `kind()`. Unsurveyed productive sites leave detailed fields as `None`; inert formations are resolved by scanning. Query again for current survey, cycle, and cap data.

`.surveyed_sites(planet_id: str) → list[Site]`

Every fully-resolved site on `planet_id` as `list[Site]`, same shape as `discovered_sites()`, filtered to `surveyed == True`. This includes inert `GeologicalAnomaly` contacts because sonar resolves them without a second survey. Branch on `kind()` to access fields: `MiningSite` exposes `.item_id`, `.hardness`, `.purity`; `ThermalVent` exposes phase / rate / cycle timing (gated by sonar tier); `WaterWell` / `OilWell` expose `.yield_tier`, `.flow_rate`. See `Site`.

PARAMETERS

Name	Type	Description
`planet_id`	`str`	Journal planet id; the current planet is usually `"nocturna"`.

RETURNS

`list[Site]`

Fully resolved sites, including inert formations resolved by a scan: `discovered_sites()` filtered to `surveyed == True`. Query again for current cycle and cap data.

`.cataloged_fragments(planet_id: str) → list[CatalogedFragment]`

Lists fragments analyzed at a Bio Lab on `planet_id`, newest first. Each `CatalogedFragment` includes its stable fragment id, display name, biome, coordinates, and rarity. Match `entry.fragment_id` against `BioOrder.requires`, and pass `entry.coords` to `bio_collector.collect(...)`. Unanalyzed fragments and creature identity remain hidden. After all five fragments are cataloged, the completed creature appears in `journal.cataloged_creatures(planet_id)`. Returns an empty list for a different planet.

PARAMETERS

Name	Type	Description
`planet_id`	`str`	Journal planet id; the current planet is usually `"nocturna"`.

RETURNS

`list[CatalogedFragment]`

Every fragment you've analyzed at a Bio Lab on that planet. Sorted **most-recently-cataloged first**. `[]` for any planet other than the current one.

`.cataloged_creatures(planet_id: str) → list[CatalogedCreature]`

Lists creatures whose five fragments have all been analyzed on `planet_id`, most recently completed first. Each `CatalogedCreature` provides the stable creature id, its five fragment ids, required feed item and Feed Maker recipe, minimum startup feed, and exact rarity-scaled revival reagents. Use `.creature_id` with `habitat.set_revival_target(...)`. Use `.feed_recipe_id` to find the matching unlocked `Recipe` in `feed_maker.list_recipes()`; recipe ingredients remain owned by that `Recipe`. Returns an empty list for a different planet.

PARAMETERS

Name	Type	Description
`planet_id`	`str`	Journal planet id; the current planet is usually `"nocturna"`.

RETURNS

`list[CatalogedCreature]`

Every creature whose five fragments have been analyzed on that planet, with the ids and exact startup supplies a generic Habitat revival script needs. Sorted **most-recently-completed first**. `[]` for any planet other than the current one.

`.coord_info(x: int, y: int) → LifeFormScanResult | None`

Read the saved `LifeFormScanResult` for a discovered permanent biosite coordinate. Returns `None` for biosites the Journal has no scan of, and for scanned coordinates that are not sites. The query returns immediately.

PARAMETERS

Name	Type	Description
`x`	`int`	Whole-number world x
`y`	`int`	Whole-number world y

RETURNS

`LifeFormScanResult | None`

`None` if `(x, y)` is untouched or not a biosite. Cheap; no yield.

`.biomass_coords() → list[LifeFormScanResult]`

Lists every discovered permanent biosite as a `LifeFormScanResult`. This is the restart-safe route source for harvester drones: inspect `.coord`, each sample's `.remaining_tons`, and `is_ready(x, y)` before dispatching.

RETURNS

`list[LifeFormScanResult]`

Every discovered permanent biosite. Use each `.coord`, `.life_forms`, and `.remaining_tons` to schedule drone harvest routes.

`.has_scanned(x: int, y: int) → bool`

`True` after the whole-number coordinate `(x, y)` has been scanned. Use it to skip biosites already visited by a route that resumes across script restarts.

PARAMETERS

Name	Type	Description
`x`	`int`	Whole-number world x
`y`	`int`	Whole-number world y

RETURNS

`bool`

`True` if you've scanned this whole-number coordinate.

`.is_empty(x: int, y: int) → bool`

`True` only when this whole-number tile has been scanned and contained no life forms. Returns `False` for both occupied and untouched tiles, so pair it with `has_scanned()`.

PARAMETERS

Name	Type	Description
`x`	`int`	Whole-number world x
`y`	`int`	Whole-number world y

RETURNS

`bool`

`True` only when you scanned this whole-number coordinate and it was not a biosite.

`.is_ready(x: int, y: int) → bool`

`True` when a discovered biosite can be extracted now. Returns `False` while another drone is extracting there, after depletion, or during its rarity-based cooldown. The query uses current site state and returns immediately.

PARAMETERS

Name	Type	Description
`x`	`int`	Biosite world x
`y`	`int`	Biosite world y

RETURNS

`bool`

`True` if a discovered biosite has stock or its cooldown has elapsed, and no extraction is currently in progress there.

`.next_ready_at(x: int, y: int) → float | None`

When the extraction cooldown ends, as an absolute hour. A depleted site's timestamp remains even after that hour passes, until extraction replenishes it. Returns `None` if the biosite is not recorded, still has material, or has never been extracted. Use `is_ready(x, y)` to check whether extraction can begin now.

PARAMETERS

Name	Type	Description
`x`	`int`	Biosite world x
`y`	`int`	Biosite world y

RETURNS

`float | None`

Absolute hour when a depleted biosite's extraction cooldown ends. The deadline may already be in the past until extraction replenishes the site. `None` if the site is not recorded, material remains, or it has never been extracted.

Map Markers

Annotates the Planet Map from your scripts. Get it with `get_component("markers")` after Cartography unlocks, then `markers.place("survey.rover_1.empty:120:-40", 120, -40, "No contact", "x")` to drop a marker anywhere in the world, instantly, with no vehicle and no materials. Markers are notes, not blueprints: to actually build somewhere, pass the coordinates to `construction_blueprint.plan_structure(...)`. Store structured data in the Data Archive under the same id.

Access via: `get_component("markers")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.place(id: str, x: float, y: float, label: str = "", icon: str = "pin", color: str = "accent", note: str = "") → ActionResult`

Create or rewrite one marker. Reusing an id moves and restyles that marker instead of adding a second one, so a script that restarts after a save does not fill the map with duplicates. Coordinates are world meters and keep their fractions. Organize families of markers by id prefix, and include the controlling machine in the prefix, as in `"survey.rover_1."`, so two scripts cannot overwrite each other.

PARAMETERS

Name	Type	Description
`id`	`str`	Stable marker id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`. Reusing an id rewrites that marker.
`x`	`float`	World x coordinate in meters. Fractions are kept exactly.
`y`	`float`	World y coordinate in meters. Fractions are kept exactly.
`label`	`str`	Display text, up to 48 characters. Empty shows the id instead.
`icon`	`str`	Marker glyph.
`color`	`str`	Marker color.
`note`	`str`	Longer note shown on hover and in the Markers list, up to 240 characters.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"invalid_key"`	● rejection	The supplied key is invalid.
`"invalid_coords"`	● rejection	The supplied coordinates are invalid.
`"out_of_bounds"`	● rejection	The requested position lies outside the valid world bounds.
`"invalid_icon"`	● rejection	The supplied marker glyph is not one of the available glyphs.
`"invalid_color"`	● rejection	The supplied marker color is not one of the available colors.
`"invalid_text"`	● rejection	The supplied text is longer than the field allows.

Status	Kind	Meaning
`"limit_reached"`	● rejection	The collection already holds its maximum number of entries.

`.get(id: str) → Marker | None`

Read one marker by id.

PARAMETERS

Name	Type	Description
`id`	`str`	Marker id to look up.

RETURNS

`Marker | None`

`None` when no marker uses that id.

RAISES

Exception	Condition
`ValueError`	The marker id must be 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.list(prefix: str = "") → list[Marker]`

Read markers as a list sorted by id. Pass a prefix such as `"build."` to read one family. Loop the result to route a vehicle: `for m in markers.list("build."): self.nav.set_target(m.x, m.y)`.

PARAMETERS

Name	Type	Description
`prefix`	`str`	Optional id prefix such as `"survey.rover_1."`.

RETURNS

`list[Marker]`

Sorted by id, optionally narrowed to one id prefix. An empty prefix returns every marker.

RAISES

Exception	Condition
`ValueError`	The marker prefix must use letters, numbers, `_`, `.`, `:`, or `-` and be at most 64 characters.

`.remove(id: str) → ActionResult`

Delete one marker by id.

PARAMETERS

Name	Type	Description
`id`	`str`	Marker id to delete.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"invalid_key"`	● rejection	The supplied key is invalid.

`.clear(prefix: str) → CountResult`

Delete a whole family of markers by id prefix, then place the current ones again to keep a family in step with what your script now believes. The prefix is required: `markers.clear("")` deletes every marker on the planet, including the ones you placed by hand, and nothing records who placed what.

PARAMETERS

Name	Type	Description
`prefix`	`str`	Id prefix to delete. Pass `""` to delete every marker, including the ones you placed by hand.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Command completed. Affected entries or units: `.count`.
`"no_op"`	● success	The command affected no entries or units.
`"invalid_key"`	● rejection	The supplied key is invalid.

Scanner

Reveals the sectors of the Harvester grid around base so the Harvester knows where to collect. It maps the home grid only; exploring the wider planet is a job for a vehicle's sonar.

Type	Harvesting

Access via: `get_component("scanner_1")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

METHODS

`.scan(sector: str) → ScanResult` SELF ONLY

Scan one local sector with `self.scan("E14")`. The scan takes a few ticks and pauses the script. Malformed or out-of-bounds sector ids raise `ValueError`. This local-grid scanner finds surface items, not planetary `Site` contacts.

PARAMETERS

Name	Type	Description
`sector`	`str`	Local grid sector id (e.g. `"E14"`).

RETURNS

`ScanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.id`, `.name`, `.value`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The sector scan found `item_id`.
`"empty"`	● success	The sector scan completed and found no item.

RAISES

Exception	Condition
`ValueError`	The `sector` argument is not a valid local grid sector id.

`.get_scanned() → dict[str, ScanResult]`

Every previously scanned sector as a fresh `dict` `{sector_id: ScanResult}`. Iterate with `.keys()` / `.values()` / `.items()`, or index by sector id: `self.get_scanned()["E14"]`. A sector only needs to be physically scanned once, and that history persists across script runs. Each `get_scanned()` call reflects the current contents of those sectors, including items collected or dropped since the last call. A `dict` or `ScanResult` already saved in your script does not update itself, so call `get_scanned()` again before choosing another target. Returns an empty `dict` if nothing has been scanned yet.

RETURNS

`dict[str, ScanResult]`

Keyed by sector id.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Battery

Base-station energy storage. It fills on its own when generation runs a surplus and drains when the grid falls short. If it empties, machines shut off and their scripts pause.

Type	Power
Energy	500 Wh

HOW TO OBTAIN

1. Buy from the Shop for 300 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.get_level() → float`

Current stored energy in watt-hours (**Wh**). Drops when consumption exceeds generation, rises when generation exceeds consumption, and stays level when they are equal. Approaching **0** is a red flag, the grid is about to brown out.

RETURNS

`float`

In Wh.

`.get_capacity() → float`

Total battery capacity in watt-hours (**Wh**). Queryable rather than hardcoded so future upgrades don't break scripts. Use with `get_level()` for charge percent.

RETURNS

`float`

In Wh.

Battery Charger

Charges loose **Portable Batteries (50 Wh)** and **Heavy Portable Batteries (100 Wh)** from the local grid. Mk I has **4 shared slots**, **1 charging bay**, and a **30 W** budget. Place at Base or a founded outpost. Park a Pioneer in its service area for a timed battery exchange. Charging is script-controlled; idle draw is 0 W.

Type	Mining
Tiers	Mk II

HOW TO OBTAIN

1. Requires the **Battery Charger** research (Oxygen 25).
2. Buy from the Shop for 1,500 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Standard local input port. Both ports share the physical socket store. Only portable vehicle batteries are accepted. `eject()` recovers unqueued cells; `flush()` discards unqueued cells permanently.

RETURNS

`InputSlot`

`.output: OutputSlot`

Standard local output port. Only unqueued cells can be sent or pulled out. Item properties and stored charge are preserved.

RETURNS

`OutputSlot`

METHODS

`.slots() → list[BatteryChargerSlot]`

Read snapshots of every physical socket. Waiting, charging, and ready cells share the same capacity.

RETURNS

`list[BatteryChargerSlot]`

`.status(slot_index: int) → BatteryChargerSlot`

Read one socket snapshot. Raises `ValueError` for an out-of-range index.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Zero-based physical socket index, from 0 to the slot count minus one.

RETURNS

`BatteryChargerSlot`

`.charge(slot_index: int, target_level: float = 1.0) → ActionResult` SELF ONLY

Queue this exact cell to a target charge fraction. Repeating the call updates its existing target without duplicating or reordering the job. Completed jobs release their socket automatically. Power loss pauses work. Removing or swapping the cell never transfers its job to its replacement.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Zero-based physical socket index, from 0 to the slot count minus one.
`target_level`	`float`	Required charge fraction greater than 0 and at most 1; defaults to 1.0.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"queued"`	● success	The battery has an unfinished charging job.
`"target_reached"`	● success	The cell already meets the requested target.
`"empty"`	● rejection	The socket contains no battery.
`"under_construction"`	● rejection	Construction is not complete.

`.stop(slot_index: int) → ActionResult` SELF ONLY

Cancel this socket's charging job and release the cell, preserving all stored charge.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Zero-based physical socket index, from 0 to the slot count minus one.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed.
`"no_op"`	● success	No changes were needed.

`.clear_queue() → CountResult` SELF ONLY

Cancel all charging jobs, retain all cells and their stored charge, and return the number canceled.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Command completed. Affected entries or units: `.count`.
`"no_op"`	● success	The command affected no entries or units.

`.get_queue() → list[int]`

Read ordered unfinished socket indices, including active jobs.

RETURNS

`list[int]`

`.get_active() → list[int]`

Read the socket indices receiving power now. Empty during an outage or battery handling.

RETURNS

`list[int]`

`.tier() → int`

Read the installed tier (1 or 2).

RETURNS

`int`

`.get_slot_count() → int`

Read the shared physical storage capacity: 4 at Mk I or 8 at Mk II.

RETURNS

`int`

`.get_bay_count() → int`

Read simultaneous charging capacity: 1 at Mk I or 2 at Mk II.

RETURNS

`int`

`.get_bay_rate() → float`

Read the nominal power per bay in watts. Idle bay power is pooled among active cells.

RETURNS

`float`

`.get_charge_rate(slot_index: int) → float`

Read this socket's actual charging watts, including pooling and overcrowding. Zero when inactive.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Zero-based physical socket index, from 0 to the slot count minus one.

RETURNS

`float`

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

```
while True:
    for cell in self.slots():
        if cell.item_id and cell.level < 1.0 and cell.target is None:
            result = self.charge(cell.index)
            if result.status != "queued":
                print(result.message)
    sleep(1)
```

Lightning Rod

A **4,000 Wh** emergency reserve that catches lightning within **600 m** and discharges behind batteries. Condition falls **0.05 per day**, reducing capture to zero unless a running script repairs it with **1 Storm Glass**.

Type	Power
Energy	4,000 Wh
Input buffer	Units: 5

HOW TO OBTAIN

1. The recipe unlocks with the **Lightning Rods** research (Temperature 8,000).
2. Fabricate **Lightning Rod** on a **Fabricator**: 1× Machine Frame, 4× Battery Cell, and 2× Circuit Panel.
3. Deploy it from your Inventory.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

The rod's material slot. Feed it Storm Glass with your ordinary logistics, then spend one with `repair()` to restore full condition.

RETURNS

`InputSlot`

Holds the Storm Glass that `repair()` spends.

METHODS

`.bank() → float`

Wh currently banked, **0** up to `capacity()`. Rises only when a strike lands within catch range of this rod; falls automatically when its grid is short and the batteries are empty. It never charges from grid surplus.

RETURNS

`float`

Wh currently banked, from **0** up to `capacity()`. Fills only when a thunderstorm strike lands within catch range of this rod; drains automatically when its grid runs short and the batteries are empty. Never charges from grid surplus.

`.capacity() → float`

Bank capacity in Wh, several times a base battery. Query this instead of hardcoding the number.

RETURNS

`float`

Bank capacity, in Wh.

`.last_strike() → float`

Hour timestamp of the last strike from which this rod accepted energy, or **-1** if none. A strike that adds no energy, for example when the bank is full or integrity is zero, does not update this record. Compare with the clock's current time to see how long it has been since energy was last captured.

RETURNS

`float`

Hour timestamp of the last strike from which this rod accepted energy, or **-1** if none. A strike that adds no energy, for example when the bank is full or integrity is zero, does not update this record.

`.integrity() → float`

This rod's condition from **0** to **1**, which is also its capture efficiency. Continuous corrosion lowers it by **0.05 per day**; strikes do not cause separate damage. A rod at **0.5** banks half of every strike it catches, and one at **0** banks nothing while still standing and still repairable.

RETURNS

`float`

This rod's condition (**0-1**), which is also its capture efficiency. A rod at **0.5** banks half of every strike it catches; at **0** it banks nothing and still stands.

`.repair() → ActionResult` SELF ONLY

Restore this rod to full condition. If it is worn, one call consumes **1 Storm Glass** from its input and sets condition to **1**. At full condition, no material is consumed. Without Storm Glass in the input, condition does not change.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_op"`	success	The operation was already satisfied, so no state changed and nothing was consumed.
`"no_material"`	rejection	The required quantity of material is not available in the machine's input.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS
`None`
`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Oil Generator

Burns oil into strong power that bridges solar gaps. Oil wells pulse between active and dormant phases, so bank their output in Liquid Tanks for continuous generation. Idle until a script runs it.

Type	Power
Power out	+700 W (feeds the grid)
Consumes	Oil, buffer 10 t

HOW TO OBTAIN

1. Requires the **Oil Generator** research (Temperature 1,500).
2. Buy from the Shop for 2,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.oil_in: FluidPort`

Input flow port for oil. Wire with `self.oil_in.connect("Liquid Tank 1")` (recommended) or directly to an Oil Pump. See `FluidPort`.

RETURNS

`FluidPort`

METHODS

`.power_output() → float`

Watts fed to the grid on the last power tick. **0** when throttled to 0 or when the `oil_in` buffer is empty. The generator scales output proportionally to available oil, so a partially-starved generator produces partial power. Updates once per power tick, a fresh `set_throttle(...)` is reflected on the next tick.

RETURNS

`float`

Watts currently fed to the grid. **0** when idle (`throttle() == 0`) or starved (no oil in buffer).

`.oil_consumption() → float`

Actual oil consumed on the last power tick, in t/h. With enough oil supplied, demand scales linearly with throttle from **0 t/h** at 0 to **8 t/h** at 1. Partial or total oil starvation lowers the actual rate.

RETURNS

`float`

Oil consumed, in t/h. **0** when idle.

`.throttle() → float`

Current throttle (**0-1**). **0** by default, generator idles until scripted.

RETURNS

`float`

The throttle setting (**0-1**).

`.set_throttle(rate: float) → ActionResult` SELF ONLY

Set the generator throttle (**0-1**). Power output and oil consumption scale linearly with the throttle. This script-owned setpoint resets to **0** when the script stops, ends, or errors, so keep the control loop running while the Generator should operate.

PARAMETERS

Name	Type	Description
`rate`	`float`	Throttle setting (**0-1**).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Reactor

Generates up to **5,000 W** from Fuel Rods and cooling water. One rod lasts **72 hours** at heat **1.0**; fuel use follows commanded heat even while the core is warming or outside its efficient band.

Type	Power
Power out	+5,000 W (feeds the grid)
Consumes	Water, buffer 3 t
Input buffer	Units: 3

HOW TO OBTAIN

1. Requires the **Nuclear Program** research (Terraform Index 650,000).
2. Buy from the Shop for 750,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.water_in: FluidPort`

Automatic cooling-water input. The reactor consumes **0.5-1 t/h** whenever its heat setting is above **0**, and pauses safely if the supply runs dry.

RETURNS

`FluidPort`

The automatic cooling-water input.

`.input: InputSlot`

Normal Fuel Rod input. Rods arrive from a Lead Cask, and the reactor takes the next one automatically when needed.

RETURNS

`InputSlot`

Ordinary Fuel Rod supply. The reactor seats a rod automatically when needed. Fuel Rod recovery must target a compatible local Lead Cask.

METHODS

`.set_heat(value: float) → ActionResult` SELF ONLY

Set reactor heat from **0-1**. Values outside the range are clamped. The setting returns to **0** when the owning script stops.

PARAMETERS

Name	Type	Description
`value`	`float`	Heat setting in the **0-1** range.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.heat() → float`

Current heat setting from **0-1**.

RETURNS

`float`

The current heat setting (**0-1**).

`.temperature() → float`

Current temperature in °C. Output begins at **300**, peaks at **900**, then falls back to zero across the **900-950** red band. **950** triggers an automatic overheat shutdown.

RETURNS

`float`

Reactor temperature this tick, in °C. Output starts at **300**, peaks at **900**, falls back to zero across the **900-950** red band, and **950** overheats.

`.fuel_level() → float`

Active Fuel Rod life from **0-1**. One full rod lasts **72 hours** at heat **1.0**; lower heat extends it proportionally. The next rod is taken automatically from `input`.

RETURNS

`float`

The active Fuel Rod's remaining life (**0-1**).

`.power_output() → float`

Watts on the grid this tick.

RETURNS

`float`

Watts produced this tick (**0-5,000**).

`.status() → str`

Current operating state: `running`, `overheated`, `no_fuel`, or `no_coolant`. Shutdowns recover automatically after cooling or supplies return.

RETURNS

`str`

Overheat cools and recovers automatically; restored supplies resume automatically.

POSSIBLE VALUES

`"running"`, `"overheated"`, `"no_fuel"`, `"no_coolant"`

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Solar Generator

Turns sunlight into up to **50 W** for the grid when a script tracks the Sun. Poor tilt reduces daytime output, and night produces **0 W**.

Type	Power
Power out	+50 W (feeds the grid)

HOW TO OBTAIN

1. Buy from the Shop for 500 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.set_tilt(degrees: float) → ActionResult` SELF ONLY

Set the panel tilt angle in degrees. Range is **0°** (flat) to **90°** (vertical); values outside are clamped. A well-tuned tracker holds output near its maximum throughout the day; a fixed tilt wastes a large fraction. The panel returns to its resting tilt of **90°** when the script stops, ends, or errors.

PARAMETERS

Name	Type	Description
`degrees`	`float`	Panel tilt in degrees

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.tilt() → float`

Current panel tilt setpoint in degrees (**0-90**). Returns the value the script last wrote via `self.set_tilt(...)`, or the default rest angle for an idle panel. Use to verify your sweep loop or to step a tilt search against the previous value.

RETURNS

`float`

The current panel tilt setpoint, 0-90 degrees.

`.get_output() → float`

Current power output in watts for this specific generator. Returns **0** if powered off. Computed live from sun elevation vs panel tilt, use it to verify the tracker is holding peak (compare current output against the panel's rated max).

RETURNS

`float`

In watts.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Steam Turbine

Produces up to 108 W from 90 t/h Steam. Its script-owned throttle scales both consumption and output.

Type	Power
Power out	+108 W (feeds the grid)
Consumes	Steam, up to 90 t per hour, buffer 100 t

HOW TO OBTAIN

1. Requires the **Steam Turbine** research (Pressure 3).
2. Buy from the Shop for 7,500 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.steam_in: FluidPort`

Input flow port accepting steam from a Thermal Cap or Gas Tank. Call `self.steam_in.connect(...)` with the source's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free Gas Pipe route between both locations. `self.steam_in.level()` shows the buffer, low = about to run dry; full = source is backpressured. See `FluidPort`.

RETURNS

`FluidPort`

METHODS

`.power_output() → float`

Watts fed to the grid on the last power tick. Scales with `throttle()` and the steam actually available. **0** when idle or no steam is available. Insufficient but nonzero steam supply reduces output proportionally. Updates once per power tick, a fresh `set_throttle(...)` is reflected on the next tick, not the same one. Use for live dashboards or to compare against Oxygen / Heat consumption to balance the power budget.

RETURNS

`float`

Watts currently fed to the grid.

`.efficiency() → float`

Fraction of the throttle's desired steam actually drawn on the last power tick (**0.0-1.0**). **1.0** = the turbine got all the steam its throttle asked for. Below **1** means steam-starved (vent dormant, or upstream can't keep up). Use to detect whether the turbine is being fed enough.

RETURNS

`float`

Fraction of the throttle's desired steam actually drawn (**0-1**); below **1** means steam-starved.

`.is_stalled() → bool`

`True` when the throttle is up but no steam is arriving (the vent is dormant or the steam line is disconnected). Check `self.steam_in.level()` (upstream) and the feeding Cap's vent phase to diagnose. There is no water side to back up anymore.

RETURNS

`bool`

`True` when the throttle is up but no steam is arriving (vent dormant or disconnected).

`.throttle() → float`

Current throttle setting (**0.0-1.0**). **0** = off, the turbine consumes no steam and produces nothing (the default; idle until `set_throttle()` is called). Higher values draw more steam and produce more power, up to the peak at **1.0**. Read-only view of what `set_throttle()` last committed.

RETURNS

`float`

Current throttle setting (**0-1**).

`.set_throttle(t: float) → ActionResult` SELF ONLY

Set the turbine throttle (**0.0-1.0**, clamped). **0** switches the turbine off (no steam consumed, no power). **1.0** draws full steam for peak watts. `self.set_throttle(1.0)` runs it flat out; ease down when the steam buffer runs dry so it isn't spinning on empty, e.g. `if self.steam_in.level() < 5: self.set_throttle(0.3)`. This script-owned setpoint resets to **0** when the script stops, ends, or errors.

PARAMETERS

Name	Type	Description
`t`	`float`	**0** = idle (no steam, no power), **1** = full steam draw / max power.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Vehicle Charging Station

Grid-powered fleet charging: Mk I provides **1 bay / 30 W**, Mk II **2 bays / 120 W**, and Mk III **4 bays / 240 W**. Idle bays pool onto one vehicle; several vehicles share the budget. A script queues charging or dispatches rescue.

Type	Mining
Power in	Variable (draws from grid)
Tiers	Mk II and Mk III

HOW TO OBTAIN

1. Requires the **Vehicle Charging Station** research (Oxygen 9).
2. Buy from the Shop for 1,200 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.get_docked() → list[str]`

List of vehicle instance ids parked inside the station's local service pad plus its **~2 m** margin, or inside the owning Outpost's common service area. Each parked vehicle is docked at one station only: the one it picked with its own `dock()`, otherwise the eligible station closest to the vehicle, measured to the station or its outpost's center, whichever is closer, with equal-distance ties broken alphabetically by station id. Returns ids; read each vehicle with `get_component(id)`. Empty list means no vehicles docked. Call each iteration; the list can change between ticks as vehicles drive in or out.

RETURNS

`list[str]`

Instance ids of the vehicles docked at this station. Read each vehicle's state via `get_component(id)`.

`.charge(vehicle_id: str, target_level: float = 1.0) → ActionResult` SELF ONLY

Queue a docked rover or Pioneer to charge until its battery reaches `target_level` (greater than **0** and at most **1**, defaults to **1.0**). The station decides whether it starts immediately or waits behind another vehicle: Mk I charges one vehicle at a time, Mk II two, Mk III four. The fewer vehicles active, the faster each charges (idle bays pool). Example: `self.charge("pioneer_1", 0.8)` means 'charge this vehicle until it reaches 80%'.

PARAMETERS

Name	Type	Description
`vehicle_id`	`str`	Display name or id of a docked rover or pioneer
`target_level`	`float`	Optional battery target fraction greater than **0** and at most **1**. Defaults to **1.0**.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"charging"`	success	A charging job is active.
`"queued"`	success	The operation was added to the queue.
`"target_reached"`	success	The vehicle is already at or above the requested charge or fuel level.
`"not_docked"`	rejection	The requested vehicle is not docked at this station.
`"station_offline"`	transient	The station is offline.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.stop(vehicle_id: str) → ActionResult` SELF ONLY

Remove a vehicle from this station's charge queue. It does not move the vehicle or change its current battery.

PARAMETERS

Name	Type	Description
`vehicle_id`	`str`	Display name or id of a queued vehicle

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.clear_queue() → CountResult` SELF ONLY

Clear every queued charge job on this station. Rescue-drone missions are separate and are not cancelled by this.

RETURNS
`CountResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Command completed. Affected entries or units: `.count`.
`"no_op"`	success	The command affected no entries or units.

`.get_active() → list[str]`

List of vehicle ids currently occupying active charging bays. Mk I returns at most one id, Mk II two, Mk III four. These are the vehicles sharing the station's pooled budget this tick.

RETURNS

`list[str]`

Ids of the vehicles currently occupying charging bays.

`.get_queue() → list[str]`

List of vehicle ids in charge-queue order. The first `get_bay_count()` entries are the ones that can be active right now, assuming they are still docked and below their target.

RETURNS

`list[str]`

Queued vehicle ids in order; the first `get_bay_count()` entries can be active.

`.status(vehicle_id: str) → dict[str, JsonValue]`

Detailed status for one vehicle: a `dict` with `state` (`"charging"`, `"queued"`, `"docked"`, `"target_reached"`, `"not_docked"`, `"station_offline"`, or `"missing"`), `target_level`, `battery_wh`, `capacity_wh`, `rate_w` (the pooled watts this vehicle is actually receiving, rises as fewer vehicles share the bays), `bay_index`, and `queue_index`. Use this for dashboards or queue managers.

PARAMETERS

Name	Type	Description
`vehicle_id`	`str`	Display name or id of a vehicle

RETURNS

`dict[str, JsonValue]`

Keys: `state`, `target_level`, `battery_wh`, `capacity_wh`, `rate_w`, `bay_index`, `queue_index`.

`.tier() → int`

Permanently installed Charging Station tier as an integer (**1-3**). Mk II raises bay count and bay rate; Mk III raises bay count again.

RETURNS

`int`

Permanently installed Charging Station tier (**1-3**).

`.get_bay_count() → int`

Number of simultaneous vehicle charging bays. Mk I is **1**, Mk II **2**, Mk III **4**.

RETURNS

`int`

How many vehicles can charge at once.

`.get_bay_rate() → float`

Watts pushed by a single bay (**30 W** Mk I, **60 W** Mk II/III). With bay pooling a lone vehicle draws every idle bay, so its actual rate is up to `get_bay_count() × get_bay_rate()`, read `get_charge_rate(id)` for what a specific vehicle is really getting. The station's total grid draw is `get_bay_count() × get_bay_rate()` whenever any vehicle is charging, plus rescue-drone draw if a rescue is out.

RETURNS

`float`

Watts pushed by each active bay.

`.get_charge_rate(vehicle_id: str) → float`

Actual watts being pushed into the specified vehicle right now, the station's total budget (`get_bay_count() × get_bay_rate()`) split evenly across every active vehicle. A lone vehicle gets the whole budget (Mk III: **240 W**); the more vehicles charging, the lower each one's share. Returns **0** if the vehicle is not occupying an active bay.

PARAMETERS

Name	Type	Description
`vehicle_id`	`str`	Display name or id of a docked vehicle

RETURNS

`float`

Watts currently being pushed into that vehicle.

`.dispatch_rescue(vehicle_name: str, target_level: float = 1.0) → ActionResult`

`SELF ONLY`

Send a field-service drone to a Rover or Pioneer by display name or id. `target_level` is a battery fraction greater than **0** and at most **1** and defaults to **1.0**. Dispatch stops the target vehicle so the drone can reach it. The station must be powered at launch, but the drone can finish its mission through a later outage. If the target reaches a powered Charging Station first, the remaining request joins that station's queue. Only one rescue can run at a time; call `cancel_rescue()` to recall it.

PARAMETERS

Name	Type	Description
`vehicle_name`	`str`	Display name or instance id of the rover/Pioneer to service.
`target_level`	`float`	Optional battery target fraction greater than **0** and at most **1**. Defaults to **1.0**.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"already_dispatched"`	rejection	This station already has an active rescue, or the requested target is already assigned to a rescue.
`"station_offline"`	transient	The station is offline.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.cancel_rescue() → ActionResult` SELF ONLY

Recall this station's active field-service drone. If the drone was outbound or trickle-charging, the target vehicle is released immediately and keeps any charge already delivered while the drone returns to the station.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.

`.is_rescuing() → bool`

`True` while a rescue drone is deployed (out, at the target, or returning). A recalled drone still counts as rescuing until it reaches the station, but the target vehicle is released as soon as `cancel_rescue()` succeeds. Use before `dispatch_rescue()` to avoid the `"already_dispatched"` rejection: `if not self.is_rescuing(): self.dispatch_rescue(name)`. Exactly one drone at a time, queue rescues manually if you need more.

RETURNS
`bool`

`.get_rescue_target() → str`

Display name of the vehicle currently being rescued, or empty string if the drone is idle. Use for dashboards ("rescuing Rover 1") or to decide whether to wait vs send a different vehicle to pick up slack.

RETURNS
`str`
Display name of the vehicle being rescued, or empty.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Atmosphere

Planetary atmosphere, read gas composition, pressure, temperature, and the planet's heat-units progression metric (`get_heat()`). Oxygen and pressure reads require their sensors to be repaired.

Access via: `get_component("atmosphere")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.get_co2() → float`

Current CO2 level in parts per thousand (ppt). Oxygen generation consumes CO2 **1:1**, so this value falls as oxygen rises. CO2 is returned to the atmosphere by burning oil (Oil Generator), incinerating items (Waste Processor), and, at scale, by wildlife respiration (established colonies exhale CO2). A small volcanic outgassing trickle replenishes CO2 when reserves are low.

RETURNS

`float`

In ppt.

`.get_o2() → float`

Current oxygen level in parts per thousand (ppt). Requires the oxygen sensor to be repaired.

RETURNS

`float`

In ppt.

`.get_n2() → float`

Current nitrogen level in parts per thousand (ppt).

RETURNS

`float`

In ppt.

`.get_pressure() → float`

Current atmospheric pressure in kPa. Requires the pressure sensor to be repaired.

RETURNS

`float`

In kPa.

`.get_temperature() → float`

Current surface temperature in °C. This is a **display value**, a non-linear transform of the heat-units metric. For terraforming progress or heat cutoffs, use `get_heat()`, not this.

RETURNS

`float`

In °C.

`.get_heat() → float`

Current accumulated **heat units**, the temperature pillar's progression metric, the exact value temperature research and phase thresholds compare against (the Research page shows the current targets). Gate heat cutoffs on this, the way `get_o2()` / `get_pressure()` work for those pillars. Unlike `get_temperature()` (surface °C, a non-linear display value), a difference in heat units IS terraforming progress. Starts at **0**.

RETURNS

`float`

In heat units: the temperature pillar's progression metric, which research and phase thresholds compare against.

Bio Caster

Forges a volcanic fragment by holding the crucible in a target temperature band while the recipe's materials are loaded, then casting. Steam and water use separate 20 t internal process buffers. Connect each input port to a compatible source; local sources transfer directly, while remote sources also need a completed conflict-free pipe route between both locations. A script drives the heat and cooling; casting out of band or with the wrong materials burns the whole charge.

Type	Atmosphere
Power in	-15 W (draws from grid)
Consumes	Steam, buffer 20 t
Consumes	Water, buffer 20 t
Output buffer	Units: 30
Stockpile	30 units (mixed)

HOW TO OBTAIN

1. Requires the **Volcanic Forge-Casting** research (Oxygen 350).
2. Buy from the Shop for 150,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.input: InputSlot`

The multi-material `InputSlot` for both the raw volcanic sample and fabricated materials. Connect Inventory only at Nocturna Base; at another outpost connect a same-outpost Storage Bin/Warehouse. Call `take(...)` for every required item.

RETURNS

`InputSlot`

For the raw Volcanic sample and fabricated materials. Inventory is available only at home; at remote outposts route freight through local storage. Before `cast()`, recover a mistaken sample or material with `eject(...)`.

`self.output: OutputSlot`

The `OutputSlot` for Forged samples and non-destructively ejected items. Exact item properties are preserved.

RETURNS

`OutputSlot`

For Forged samples and non-destructively ejected items.

`self.steam_in: FluidPort`

Supplies the Caster's heat control through a **20 t** steam buffer. Call `self.steam_in.connect(...)` with a Thermal Cap or steam Gas Tank's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free Gas Pipe route between both locations. The connection remains while idle, full, or powered off. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

The Caster's **20 t** steam buffer. Call `connect(...)` with the provider's stable machine id or display name; completed gas-pipe networks carry steam between outposts.

`self.water_in: FluidPort`

Supplies the Caster's cooling control through a **20 t** water buffer. Call `self.water_in.connect(...)` with a Water Pump, Steam Condenser, water Liquid Tank, or Large Liquid Tank's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free Liquid Pipe route between both locations. The connection remains while idle, full, or powered off. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

The Caster's **20 t** water buffer. Call `connect(...)` with the provider's stable machine id or display name; completed liquid-pipe networks carry water between outposts.

METHODS

`self.list_recipes() → list[BioCasterRecipe]`

List all 16 forge recipes without changing the selected recipe, `self.list_recipes()`. Each `BioCasterRecipe` includes its fragment id, tier, exact material shopping list, and target temperature range. The read works through `get_component(...)`, so another machine can plan supplies without possessing every fragment.

RETURNS

`list[BioCasterRecipe]`

All 16 recipes, for read-only production planning.

`self.find_recipe(fragment_id: str) → BioCasterRecipe | None`

Look up one forge recipe by volcanic fragment id without selecting it, `self.find_recipe(fragment_id)`. Returns `None` for an unknown id.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	Volcanic fragment id from `catalog()`.

RETURNS

`BioCasterRecipe | None`

`None` for an unknown fragment id.

`self.catalog() → list[str]`

The 16 forgeable volcanic fragment ids, `self.catalog()`. Pass one to `set_recipe(...)`, or use `list_recipes()` when you also need every recipe's material and temperature requirements. Fixed hardware; read it once.

RETURNS

`list[str]`

The 16 forgeable Volcanic fragment ids. Use `list_recipes()` when you also need material and temperature requirements.

`self.recipe_tier(fragment_id: str) → int | None`

Derived production tier for one recipe from `catalog()`. Returns `None` for an unknown fragment id.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	Volcanic fragment id from `catalog()`.

RETURNS

`int | None`

Derived production tier for this Bio Caster recipe, or `None` for an unknown fragment id.

`self.set_recipe(fragment_id: str) → ActionResult` SELF ONLY

Select which volcanic fragment to forge, `self.set_recipe("sd_tail_barb")`. After this, `required_range()` and `required_materials()` describe that recipe. The selection persists like a Smelter recipe.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	A Volcanic fragment id from `catalog()`: the fragment you intend to forge.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_recipe"`	rejection	The supplied recipe is invalid.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.recipe() → str | None`

The recipe you've selected, `self.recipe()` returns the volcanic fragment id you're set to forge (also the raw fragment to `load`), or `None` if none is set.

RETURNS

`str | None`

The selected recipe = the Volcanic fragment id this Caster will forge (also the raw fragment to `load`), or `None` if no recipe is set.

POSSIBLE VALUES

`"ma_chitinous_seta"`, `"st_beak"`, `"ms_abdomen_sclerite"`, `"mh_chitin_node"`, `"gm_abdominal_sheath"`, `"vm_tail_barb"`, `"vc_walking_leg"`, `"oc_ink_sac"`, `"bw_hindlimb"`, `"vd_photophore"`, `"hs_abdomen_segment"`, `"hc_beak"`, `"fs_oral_tegmen"`, `"ce_great_appendage"`, `"gw_jaw_fang"`, `"sd_tail_barb"`

`self.required_range() → list[float] | None`

The selected recipe's target band `[low, high]` in °C, `self.required_range()`. `cast()` must fire with `temperature()` inside it (inclusive). `None` if no recipe is set.

RETURNS

`list[float] | None`

The selected recipe's target temperature band `[low, high]` in °C: `cast()` must fire with `temperature()` inside it (inclusive). `None` if no recipe is set.

`self.required_materials() → dict[str, int]`

Lists the fabricated materials required by the selected recipe as `{item_id: count}`. Load exactly those amounts before casting. Iterate with `.items()`. Returns an empty `dict` when no recipe is selected.

RETURNS

`dict[str, int]`

`{item_id: count}` of the fabricated materials the selected recipe needs: load EXACTLY these (no more, no less) before `cast()`. `{}` if no recipe is set. Iterate with `.items()`.

`self.required_fragment() → str | None`

The raw fragment id the recipe consumes, `self.required_fragment()` (identical to `recipe()`). `None` if no recipe is set.

RETURNS

`str | None`

The raw Volcanic fragment id the selected recipe consumes (identical to `recipe()`). `None` if no recipe is set.

POSSIBLE VALUES

`"ma_chitinous_seta"`, `"st_beak"`, `"ms_abdomen_sclerite"`, `"mh_chitin_node"`, `"gm_abdominal_sheath"`, `"vm_tail_barb"`, `"vc_walking_leg"`, `"oc_ink_sac"`, `"bw_hindlimb"`, `"vd_photophore"`, `"hs_abdomen_segment"`, `"hc_beak"`, `"fs_oral_tegmen"`, `"ce_great_appendage"`, `"gw_jaw_fang"`, `"sd_tail_barb"`

`self.temperature() → float`

Current crucible temperature in °C, `self.temperature()`, ranging **100** (cold baseline) to **1000** (max). Drive it into `required_range()` with the knobs before casting.

RETURNS

`float`

Current crucible temperature in °C (**100-1,000**). Drive it into `required_range()` with the knobs before casting.

`self.temp_rate() → float`

Net temperature change in °C/h right now, `self.temp_rate()`. Positive = heating and negative = cooling. Full heat alone reaches +2400 °C/h and full cool alone reaches -2400 °C/h with power, enough input fluid, and room within the 100-1000 °C limits. With both knobs at 0, an unlocked crucible cools naturally at up to -20 °C/h until it reaches 100 °C. Heating can raise it from 100 °C. Returns 0 while a cast is in progress.

RETURNS

`float`

Net rate in °C/h: positive = heating, negative = cooling. Full heat alone reaches +2400 °C/h and full cool alone -2400 °C/h with power, enough fluid, and room within 100-1000 °C. With both knobs at 0, an unlocked crucible cools at up to -20 °C/h to 100 °C. Heating can raise it from 100 °C. Returns 0 during a cast.

`self.heat() → float`

Current heat-knob setting **0-100 %**, `self.heat()`. At 100 % temperature rises **+2400 °C/h** (burning steam); use a lower setting near the target band.

RETURNS

`float`

Current heat-knob setting, **0-100%**. At 100 % temperature rises **+2400 °C/h** (burning steam); use a lower setting near the target band.

`self.cool() → float`

Current cool-knob setting **0-100 %**, `self.cool()`. At 100 % temperature falls **-2400 °C/h** (burning water); use a lower setting near the target band.

RETURNS

`float`

Current cool-knob setting, **0-100%**. At 100 % temperature falls **-2400 °C/h** (burning water); use a lower setting near the target band.

`self.fragment() → str | None`

The raw fragment id loaded in the chamber, `self.fragment()`, or `None` if the chamber is empty.

RETURNS

`str | None`

The raw Volcanic fragment id currently in the chamber (from `load`), or `None` if the chamber is empty.

POSSIBLE VALUES

`"ma_chitinous_seta"`, `"st_beak"`, `"ms_abdomen_sclerite"`, `"mh_chitin_node"`, `"gm_abdominal_sheath"`, `"vm_tail_barb"`, `"vc_walking_leg"`, `"oc_ink_sac"`, `"bw_hindlimb"`, `"vd_photophore"`, `"hs_abdomen_segment"`, `"hc_beak"`, `"fs_oral_tegmen"`, `"ce_great_appendage"`, `"gw_jaw_fang"`, `"sd_tail_barb"`

`self.materials() → dict[str, int]`

The materials currently loaded in the crucible, `self.materials()` returns `{item_id: count}`. Compare against `required_materials()` before `cast()`. Iterate with `.items()`.

RETURNS

`dict[str, int]`

`{item_id: count}` of the fabricated materials currently loaded in the crucible. Compare against `required_materials()` before `cast()`. Iterate with `.items()`.

`self.set_heat(pct: float) → ActionResult` SELF ONLY

Set the heat knob (steam to temperature up), `self.set_heat(100)` for +2400 °C/h, then use a lower percentage for the final approach. Range **0-100**, clamped. Open-loop: it keeps heating and burning steam until you set it back. With both knobs at 0, the crucible cools naturally at 20 °C/h, so cast after entering the band. Re-idles to 0 when the chamber empties or the script stops.

PARAMETERS

Name	Type	Description
`pct`	`float`	Heat level 0-100 (%). Both knobs may run at once; net rate = heat − cool.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.set_cool(pct: float) → ActionResult` SELF ONLY

Set the cool knob (water to temperature down), `self.set_cool(100)` for -2400 °C/h, then use a lower percentage for the final approach. Range **0-100**, clamped. Open-loop: keeps cooling and burning water until you set it back. Both knobs may run at once, but that burns both fluids for little movement. With both knobs at 0, the crucible still cools naturally at 20 °C/h. Re-idles to 0 when the chamber empties or the script stops.

PARAMETERS

Name	Type	Description
`pct`	`float`	Cool level 0-100 (%). Both knobs may run at once; net rate = heat − cool.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.load(fragment_id: str, properties: ItemProperties | None = None, property_match: str | None = None) → ActionResult` SELF ONLY

Pull a raw volcanic sample of `fragment_id` from `self.input` into the chamber, usually `self.load(self.recipe())`. Optional `properties` and `property_match` select a specific identity using the standard `any`, `subset`, or `exact` convention.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	A raw Volcanic fragment id staged in `self.input`. Usually `recipe()`.
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"chamber_occupied"`	● rejection	The processing chamber already contains an item.
`"not_in_input"`	● rejection	The requested item is not present in the input.
`"invalid_fragment"`	● rejection	The supplied fragment identifier is invalid.
`"invalid_properties"`	● rejection	The supplied item-property selector is invalid.
`"invalid_property_match"`	● rejection	The supplied property-matching mode is invalid.
`"busy"`	● transient	The component is already performing another operation.
`"output_full"`	● rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.eject()` → `ActionResult` SELF ONLY

Stage the chamber sample and all loaded materials in `self.output` without changing their properties. A single-material output may require a `send` / `eject` cycle for each item type.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`self.cast()` → `ActionResult` SELF ONLY

Forge the loaded fragment, `self.cast()`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"out_of_range"`	rejection	The chamber temperature was outside the selected recipe's required range, so the loaded fragment and materials were destroyed.
`"wrong_materials"`	rejection	The staged materials do not match the operation's requirements.
`"wrong_fragment"`	rejection	The loaded fragment does not match the requested fragment.
`"empty"`	rejection	The relevant source or queue is empty.
`"no_recipe"`	rejection	No recipe is currently selected.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Bio Collector

Automates the Collect step of the biology loop, fetching a field specimen into its cargo slot on its own. It does nothing until a script tells it where to collect.

Type	Atmosphere
Power in	-5 W (draws from grid)

HOW TO OBTAIN

1. Buy from the Shop for 3,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.cargo: Specimen | None`

The collected `Specimen` waiting for transfer, or `None` if the slot is empty. Pre-analyze, `cargo.fragment_id` / `cargo.rarity` / `cargo.recipe` are `None`; `cargo.coords` and `cargo.distance` are always available.

RETURNS

`Specimen | None`

`None` if empty. Cleared when a Bio Lab takes it with `self.take_from(collector)`.

METHODS

`self.scan() → list[FragmentLocation]`

Lists fragment locations in this outpost's biome, nearest first. Each `FragmentLocation` includes `coords`, `distance`, and whether it has been `cataloged`. Cataloged locations also reveal their `fragment_id`, `name`, and `rarity`; unknown locations leave those details as `None`. To identify an unknown location, collect it with `bio_collector.collect(location.coords)` and analyze it at a Bio Lab. Recipes and creature identity are not revealed here. Analyzed fragments also appear in `journal.cataloged_fragments(...)`.

RETURNS

`list[FragmentLocation]`

Every fragment in this outpost's biome, nearest first. A fragment's identity stays `None` until it has been analyzed; recipes and creature identity stay hidden.

`self.collect(coords: list[float]) → ActionResult` SELF ONLY

Retrieve the fragment at `coords` from `scan()` and place it in the collector's cargo slot. The trip takes **~0.1-0.3 h** in every biome, depending on distance; the script pauses until collection finishes.

PARAMETERS

Name	Type	Description
`coords`	`list[float]`	World coordinates `[x, y]` from `scan()`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"cargo_occupied"`	rejection	The cargo area already contains an incompatible item or material.
`"invalid_coords"`	rejection	The supplied coordinates are invalid.
`"no_fragment"`	rejection	No fragment is available for the operation.

`self.discard() → ActionResult` SELF ONLY

Discard the specimen currently held in collector cargo. Use this when the collector picked up a specimen you do not want to send to a Bio Lab.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Bio Conditioner

Inspects a deep-biome fragment against a fixed rulebook, quizzing your script on its properties one at a time. Judge each one correctly to pass the fragment; a single wrong call burns the whole specimen.

Type	Atmosphere
Power in	-25 W (draws from grid)
Input buffer	Units: 10
Output buffer	Units: 10

HOW TO OBTAIN

1. Requires the **Deep Fragment Conditioning** research (Temperature 1,200).
2. Buy from the Shop for 225,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.input: InputSlot`

The `InputSlot` for raw deep samples. It holds one sample item id at a time and stays latched to that id until `load()` consumes the remaining units or `flush()` discards them. `stacks()` lists property-distinct variants and does not mean the port accepts multiple sample types. Inventory is a source only at Nocturna Base; remote Conditioners use a same-outpost Storage Bin/Warehouse.

RETURNS

`InputSlot`

For raw Deep samples. Recover a mistaken property variant to an explicit local destination with `eject(...)` before loading it into the chamber.

`self.output: OutputSlot`

The `OutputSlot` for Conditioned or ejected samples. Exact sample properties are preserved.

RETURNS

`OutputSlot`

For Conditioned or ejected samples with exact properties.

METHODS

`self.report() → dict[str, JsonValue]`

The loaded fragment's full condition report, `self.report()` returns `{property: value}` for all 10 properties (`glow`, `brightness`, `smell`, `gunk`, `cracks`, `feel`, `twitch`, `bugs`, `weight`, `sound`). Read the whole thing: the combo rules need siblings (`brightness` reads `glow`, `weight` reads `gunk`, `sound` reads `cracks`). Word properties are strings, numbers are numbers. `{}` if nothing is loaded. Iterate with `.items()` or index `report["gunk"]`.

RETURNS

`dict[str, JsonValue]`

All ten condition properties for the loaded fragment, or `{}` when empty. Word values are strings and numeric values are numbers. Read the full report because `brightness` uses `glow`, `weight` uses `gunk`, and `sound` uses `cracks`.

`self.properties() → list[str]`

Lists all ten property ids in a fixed catalog order, from `"glow"` through `"sound"`. The list is the same for every Deep fragment. Each inspection quizzes five properties in a random order; use `self.current()` to identify the property at each stage.

RETURNS

`list[str]`

The 10 property ids in their fixed order (`glow` , `brightness` , … `sound`). The inspection rubric is the same for every Deep fragment.

`self.fragment() → str | None`

The raw deep fragment id loaded in the chamber, `self.fragment()` , or `None` if empty.

RETURNS

`str | None`

The raw Deep fragment id currently loaded in the chamber (from `load`), or `None` if empty.

POSSIBLE VALUES

`"gw_spinal_vertebra"` ,
`"vc_eye_stalk"` ,
`"oc_lens_eye"` ,
`"bw_tail_spike"` ,
`"vd_tendril"` ,
`"mh_stigmatic_disc"` ,
`"hs_compound_eye"` ,
`"ms_eye_cluster"` ,
`"hc_tentacle_crown"` ,
`"ma_luminous_ring"` ,
`"gm_antennal_whip"` ,
`"fs_holdfast_rootlet"` ,
`"sd_talon"` ,
`"ce_cephalic_photophore"` ,
`"st_carapace_neural"` ,
`"vm_ventral_photophore"`

`self.stage() → int`

The current QC stage, `self.stage()` returns **1-5** while a run is live, or **0** when none is active (nothing loaded, or the run just resolved). Each stage quizzes one property.

RETURNS

`int`

The current QC stage, **1-5** while a run is live, or **0** when no run is active (nothing loaded, or the run just resolved). Each stage quizzes one property.

`self.current() → str | None`

The property this stage is quizzing, `self.current()` returns one of the 10 ids (look its value up in `report()`, apply its rule, then `accept()` / `reject()`), or `None` if no run is active. You can't predict which 5 of the 10 come up, so encode every rule.

RETURNS

`str | None`

The property id this stage is quizzing (one of the 10): look its value up in `report()`, apply its rule, then `accept()` or `reject()`. `None` if no run is active. You can't predict which 5 of the 10 you'll be quizzed on, so know every rule.

POSSIBLE VALUES

`"glow"`, `"brightness"`, `"smell"`, `"gunk"`, `"cracks"`, `"feel"`, `"twitch"`, `"bugs"`, `"weight"`, `"sound"`

`self.lights() → list[str]`

The 5 stage results so far, `self.lights()` returns a list of `"green"` (correct call), `"red"` (a miss, run over), and `"pending"` (not reached). They light one at a time as you answer.

RETURNS

`list[str]`

The 5 stage results so far: `"green"` (a correct call), `"red"` (a miss: the run is over), or `"pending"` (not reached yet). Lights up one at a time as you answer.

`self.is_running() → bool`

`True` while a 5-stage run is live (a fragment is loaded with stages left), `self.is_running()`. Drive the gauntlet with `while cond.is_running(): prop = cond.current(); ...`. Goes `False` when the run finishes, when a wrong call burns the loaded fragment, or when nothing is loaded.

RETURNS

`bool`

`True` while a 5-stage run is in progress (a fragment is loaded and stages remain). Loop on it: `while cond.is_running(): ...`. Goes `False` when the run finishes, when a wrong call burns the loaded fragment, or when nothing is loaded.

`self.load(fragment_id: str, properties: ItemProperties | None = None, property_match: str | None = None) → ActionResult` SELF ONLY

Pull one raw deep sample from `self.input` into the chamber and start a fresh 5-stage run. Optional `properties` and `property_match` select a specific identity using the standard `any`, `subset`, or `exact` convention.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	A raw Deep fragment id staged in `self.input`.
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"chamber_occupied"`	● rejection	The processing chamber already contains an item.
`"not_in_input"`	● rejection	The requested item is not present in the input.
`"invalid_fragment"`	● rejection	The supplied fragment identifier is invalid.
`"invalid_properties"`	● rejection	The supplied item-property selector is invalid.
`"invalid_property_match"`	● rejection	The supplied property-matching mode is invalid.
`"busy"`	● transient	The component is already performing another operation.
`"output_full"`	● rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.eject()` → `ActionResult` SELF ONLY

Stage the unchanged chamber sample in `self.output` and end the run.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`self.accept()` → `ActionResult` SELF ONLY

Stamp the current property as passing.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The decision matched the rule book for this stage. The specimen remains in the chamber and the inspection advances to the next property.
`"conditioned"`	success	The decision matched the rule book on the final stage, so the inspection passed and the Conditioned fragment was delivered to the output.
`"burned"`	success	An incorrect conditioning decision burned the specimen and emptied the chamber.
`"no_run"`	rejection	There is no active run.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`self.reject() → ActionResult` SELF ONLY

Stamp the current property as damaged.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The decision matched the rule book for this stage. The specimen remains in the chamber and the inspection advances to the next property.
`"conditioned"`	success	The decision matched the rule book on the final stage, so the inspection passed and the Conditioned fragment was delivered to the output.
`"burned"`	success	An incorrect conditioning decision burned the specimen and emptied the chamber.
`"no_run"`	rejection	There is no active run.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Bio Exchange

Delivers biology samples to fulfill a Bio Order, the biology counterpart to the Supply Dock. A script assigns an order and delivers its required fragments until the reward pays out.

Type	Atmosphere
Power in	-8 W (draws from grid)
Input buffer	Units: 10
Output buffer	Units: 10

HOW TO OBTAIN

1. Buy from the Shop for 4,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.input: InputSlot`

The `InputSlot` for `deliver()` samples. Holds one item id until emptied by `deliver()` or `eject()`. `stacks()` lists property variants, not multiple fragment types. Connect local storage: Inventory at home, a Storage Bin or Warehouse elsewhere. Exact sample properties are preserved. Recover unused samples with `self.input.eject(destination, item_id, count, properties, "exact")`; the destination must be local. Active deliveries keep their samples reserved.

RETURNS

`InputSlot`

For property-bearing samples. Inventory is available only at Nocturna Base; remote Exchanges use compatible storage at their own outpost. Recover an unused exact sample with `eject(destination, item_id, count, properties, "exact")`.

`self.output: OutputSlot`

The `OutputSlot` that safely receives a surplus in-transit sample when another Exchange completes the shared order first. Drain it with `send(...)`; exact sample properties are preserved.

RETURNS

`OutputSlot`

For a surplus in-transit sample returned because another Exchange completed the shared order first.

METHODS

`self.orders() → list[BioOrder]`

Lists every `BioOrder`, including orders you cannot fill yet. Each order includes its `id`, `biome`, `requires`, `reward`, `status`, `delivered` samples, samples already `in_transit`, completion `percent`, and any required `target_glow`. Progress is shared by every Bio Exchange serving that order. Use `requires - delivered - in_transit` to avoid making samples that are already committed, then pass the chosen `order.id` to `set_order(...)`. From another script, call `get_component("bio_exchange_1").orders()`. `get_component("orders")` is for Earth Orders.

RETURNS

`list[BioOrder]`

Every Bio Order. Delivered and in-transit counts are shared across all Bio Exchanges. Other scripts can query a deployed Exchange; Earth Orders come from `get_component("orders")` instead.

`self.set_order(order_id: str) → ActionResult` SELF ONLY

Pick the Bio Order this Exchange will fill. `self.set_order("bio_order_03")`. Several Exchanges may activate the **same** Bio Order, they cooperate on one shared delivery count, so big Bio Orders can be served from multiple outposts at once.

PARAMETERS

Name	Type	Description
`order_id`	`str`	Id of a Bio Order from `orders()` (e.g. `"bio_order_01"`).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_order"`	rejection	The supplied order is unknown or currently unavailable for assignment.
`"completed"`	success	The requested Bio Order is already filled and cannot be assigned again. The order object's own `.status` reads `complete`, and `deliver()` reports `complete` for the delivery that finishes it; `completed` here is the assignment refusal.

`self.clear_order() → ActionResult` SELF ONLY

Clear this Exchange's active Bio Order assignment. Already delivered progress stays recorded on the Bio Order, completed Bio Orders stay completed, and no samples are moved.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.active_order() → BioOrder | None`

A snapshot of the active `BioOrder` object, its `requires`, completed `delivered` progress, live `in_transit` commitments, `percent`, and `target_glow` (coastal infusion target) at the moment you call it, or `None` if no order is set. `in_transit` includes qualifying samples already staged in serving Exchange inputs plus active timed deliveries. Call `active_order()` again to read fresh progress.

RETURNS

`BioOrder | None`

A snapshot, or `None` if no order is set. Re-query it to read fresh `delivered`, committed `in_transit`, and `percent` progress.

`self.matches_order(item_id: str, properties: ItemProperties | None = None) → bool`

Check whether one exact item matches this Exchange's active Bio Order without moving it. Pass the `id` and `properties` from an `ItemStack`. `True` means that variant satisfies the order's glow, genes, Forged, Conditioned, or plain-sample requirement. Use it with Inventory, Storage Bin, or Warehouse `stacks()` before an exact `self.input.take(...)`. Returns `False` when there is no active order, the order is complete, or the item does not qualify.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Candidate fragment id from an `ItemStack`.
`properties`	`ItemProperties` | `None`	Exact property `dict` from the same `ItemStack`, or `None` for a propertyless sample.

RETURNS

`bool`

`True` when this exact item-property identity satisfies the active Bio Order's current sample contract. Use it while iterating a storage component's `stacks()` before calling `self.input.take(..., properties, "exact")`. `False` with no active order, for completed orders, or for an invalid or wrong variant.

`self.deliver() → ActionResult` SELF ONLY

Send one matching sample from `self.input` toward the active Bio Order. `self.deliver()` moves a single sample per call and takes a short time, so a script calls it in a loop until the order is filled. A sample only counts if it satisfies the order's exact requirement, which for some biomes means a specific glow, gene set, or processed variant and not just the right fragment id; `self.matches_order(item_id, properties)` tests one stack before it is taken. Progress is **shared across every Bio Exchange** serving the same Bio Order, so another Exchange may land the final sample first and a surplus sample is refunded.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The sample was accepted toward the active Bio Order, which still requires more samples.
`"complete"`	● success	The active Bio Order is now filled and its reward has been paid. Shared progress means another Exchange may have contributed the final sample. The order can no longer be assigned.
`"no_input"`	● rejection	No input material is available.
`"output_full"`	● rejection	The output has no capacity for the result.
`"no_active"`	● rejection	There is no active operation of the requested kind.
`"busy"`	● transient	The component is already performing another operation.

`self.lifetime_credits() → int`

Total credits this Exchange has earned across every completed Bio Order.

RETURNS

`int`

Total credits this Exchange has earned across every completed Bio Order.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Bio Lab

Automates the Analyze and Extract steps of the biology loop, studying a specimen and pulling a usable sample from it. It stays idle until a script drives it.

Type	Atmosphere
Power in	-5 W (draws from grid)
Input buffer	Units: 30
Stockpile	30 units (mixed)

HOW TO OBTAIN

1. Buy from the Shop for 5,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.specimen: Specimen | None`

The `Specimen` in the lab chamber right now, exposed as `self.specimen`, or `None`. Read `self.specimen.stage` to distinguish `"collected"` from `"analyzed"`. Before analysis its identifying fields are hidden; after analysis its `fragment_id`, `rarity`, and `recipe` are populated.

RETURNS

`Specimen | None`

The specimen in the chamber, at `.stage` `"collected"` or `"analyzed"`, or `None` if empty.

`self.loaded_reagents: dict[str, int]`

A `dict` `{reagent_id: qty}` of reagents staged for the next `extract()`. Iterate `.items()` to inspect.

RETURNS

`dict[str, int]`

`{reagent_id: qty}` of reagents staged for the next `extract()`.

`self.input: InputSlot`

The `InputSlot` for scripted reagent routing. It holds one reagent item id at a time and stays latched to that id until `load()` consumes the remaining units or `flush()` discards them. `stacks()` lists property-distinct variants and does not mean the port accepts multiple reagent types. Connect Inventory only at Nocturna Base; at another outpost connect a same-outpost Storage Bin/Warehouse. Call `take(...)` before `load(...)`.

RETURNS

`InputSlot`

For scripted reagent routing. Inventory is available only at Nocturna Base; remote Labs use a local Storage Bin or Warehouse. Recover an unneeded staged reagent with `eject(...)`.

`self.output: OutputSlot`

The `OutputSlot` for extracted property-bearing samples and unloaded reagents. Connect any eligible local item store and drain it with `send(...)`.

RETURNS

`OutputSlot`

Holds extracted samples and unloaded reagents with exact properties intact.

METHODS

`self.take_from(collector: Component | IdRecord) → ActionResult` SELF ONLY

Pull the specimen out of a Bio Collector's cargo into this lab's specimen chamber. The source Collector must be at the same outpost as this Lab; pass an explicit collector reference: `self.take_from(get_component("bio_collector_1"))`.

PARAMETERS

Name	Type	Description
`collector`	`Component` | `IdRecord`	A `bio_collector` component reference, or a dictionary or class instance with its string `id` field. The collector must still exist in the same outpost; cargo is read from the current collector.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The specimen was transferred into the Bio Lab's specimen chamber.
`"busy"`	transient	The Bio Lab is currently analyzing or extracting.
`"input_occupied"`	rejection	The Bio Lab's specimen chamber already contains a specimen.
`"not_found"`	rejection	The supplied component reference does not exist.
`"source_empty"`	rejection	The Bio Collector's cargo contains no specimen.
`"source_busy"`	transient	The Bio Collector is still completing a collection trip.
`"wrong_outpost"`	rejection	The Bio Collector belongs to a different outpost.
`"invalid_source"`	rejection	The supplied component is not a valid Bio Collector.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.analyze() → AnalyzeResult` SELF ONLY

Identify the lab's current fragment and reveal its extraction recipe. Analysis takes **~0.1 h** in every biome; the script pauses until it finishes. A successful analysis adds the fragment to `journal.cataloged_fragments(planet_id)`. Creature identity stays hidden until all five fragments are cataloged, then the creature appears in `journal.cataloged_creatures(planet_id)`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`AnalyzeResult`	`.status`, `.message`	`.info`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The specimen analysis completed successfully.
`"busy"`	transient	The Bio Lab is already completing another action.
`"output_full"`	rejection	The Bio Lab is holding a finished sample that is waiting for space in its output, so nothing else can start.
`"input_empty"`	rejection	The Bio Lab has no specimen to analyze.
`"invalid_specimen"`	rejection	The loaded specimen has no recognizable fragment identity and cannot be analyzed.

`self.load(reagent_id: str, qty: int, properties: ItemProperties | None = None, property_match: str | None = None) → ActionResult` SELF ONLY

Stage a whole-number reagent quantity for the next `extract()` by consuming it from `self.input`. `self.load("alkaline_buffer", 4)`. Reagents are sold by the `shop`; both UI purchases and `shop.buy(reagent_id)` place them in base Inventory. Optional `properties` and `property_match` select a specific item identity using the standard `any`, `subset`, or `exact` convention. Fractional or negative quantities raise an argument error. Calling `extract()` with a mismatched recipe destroys the loaded reagents.

PARAMETERS

Name	Type	Description
`reagent_id`	`str`	Reagent item id to load from the connected input.
`qty`	`int`	Whole-number reagent units to load
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"busy"`	● transient	The component is already performing another operation.
`"invalid_reagent"`	● rejection	The supplied reagent is invalid for this operation.
`"invalid_qty"`	● rejection	The supplied quantity is invalid.
`"invalid_properties"`	● rejection	The supplied item-property selector is invalid.
`"invalid_property_match"`	● rejection	The supplied property-matching mode is invalid.
`"insufficient_input"`	● rejection	The input does not contain the required quantity.
`"output_full"`	● rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.unload_reagents() → ActionResult` SELF ONLY

Stage loaded reagents in `self.output` without touching the specimen, as many as the output can hold. Use this when you staged the wrong recipe.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	All loaded reagents were staged in the Bio Lab's output without changing the specimen.
`"empty"`	● rejection	The Bio Lab has no loaded reagents.
`"busy"`	● transient	The Bio Lab is currently taking, analyzing, or extracting.
`"output_full"`	● rejection	The output could not hold every loaded reagent; what fit was staged and the rest remain loaded.

`self.extract()` → `ActionResult` SELF ONLY

Consume `loaded_reagents` and place **1 sample** of the analyzed specimen in `self.output`, preserving its exact properties. Extraction takes **~0.1 + 0.05 × units h** in every biome; the script pauses until it finishes.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The loaded reagents were consumed and one sample preserving the analyzed specimen's exact properties was staged in the output.
`"output_full"`	rejection	The completed extraction is preserved because the output has no capacity for its sample.
`"recipe_mismatch"`	rejection	The loaded reagents do not match the analyzed specimen's recipe; the reagents are destroyed and the specimen is preserved.
`"busy"`	transient	The Bio Lab is currently taking, analyzing, or extracting.
`"input_empty"`	rejection	The Bio Lab has no specimen to extract.
`"not_analyzed"`	rejection	The loaded specimen has not been analyzed.
`"invalid_specimen"`	rejection	The loaded specimen has no recognizable fragment identity.

`self.discard() → ActionResult` SELF ONLY

Drop the current specimen and stage any loaded reagents in `self.output`. Use it after `analyze()` reveals a fragment you do not need.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"input_empty"`	rejection	The operation's input contains no applicable item or material.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Bio Luminizer

Tints a coastal fragment's glow to a target color using three built-in colored lamps. The lamps bleed into each other, so a script solves the brightness mix that lands on the exact target.

Type	Atmosphere
Power in	-12 W (draws from grid)
Input buffer	Units: 10
Output buffer	Units: 10

HOW TO OBTAIN

1. Requires the **Bioluminescent Infusion** research (Temperature 90).
2. Buy from the Shop for 60,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.chamber: ChamberSample | None`

The `ChamberSample` loaded right now, exposed as `self.chamber`, or `None`. Read `.glow` for its starting color and `.fragment_id` for its item id.

RETURNS

`ChamberSample | None`

`None` if the chamber is empty.

`self.input: InputSlot`

The `InputSlot` for raw coastal samples. It holds one sample item id at a time and stays latched to that id until `load()` consumes the remaining units or `flush()` discards them. `stacks()` lists property-distinct variants and does not mean the port accepts multiple sample types. Inventory is a source only at Nocturna Base; remote Luminizers use a same-outpost Storage Bin/Warehouse.

RETURNS

`InputSlot`

For raw coastal samples. Recover a mistaken property variant to an explicit local destination with `eject(...)` before loading it into the chamber.

`self.output: OutputSlot`

The `OutputSlot` for Luminous or ejected samples. Exact sample properties are preserved.

RETURNS

`OutputSlot`

For Luminous or ejected samples, preserving exact properties.

METHODS

`self.load(fragment_id: str, properties: ItemProperties | None = None, property_match: str | None = None) → ActionResult` SELF ONLY

Pull a raw glowing coastal sample of `fragment_id` from `self.input` into the chamber, `self.load("gw_caudal_fin")`. Optional `properties` and `property_match` select a specific identity using the standard `any`, `subset`, or `exact` convention. Read its start color with `self.chamber.glow`.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	A coastal fragment id staged in `self.input`.
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"chamber_occupied"`	● rejection	The processing chamber already contains an item.
`"not_in_input"`	● rejection	The requested item is not present in the input.
`"invalid_fragment"`	● rejection	The supplied fragment identifier is invalid.
`"invalid_properties"`	● rejection	The supplied item-property selector is invalid.
`"invalid_property_match"`	● rejection	The supplied property-matching mode is invalid.
`"busy"`	● transient	The component is already performing another operation.
`"output_full"`	● rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.lamp_signature(channel: str) → list[int] | None`

The RGB-per-unit `[r,g,b]` a lamp adds per brightness step, its impurity. `self.lamp_signature("red")` is roughly `[6, 1, 1]`: mostly red, but it bleeds a little into green and blue. Read all three (`"red"`, `"green"`, `"blue"`) to build the 3×3 you invert. Fixed hardware, read once and reuse. `None` for an unknown channel.

PARAMETERS

Name	Type	Description
`channel`	`str`	Lamp channel: `"red"`, `"green"`, or `"blue"`.

RETURNS

`list[int] | None`

The lamp's RGB-per-unit `[r,g,b]` (its impurity) for `channel` `"red"` / `"green"` / `"blue"`, or `None` for an unknown channel. Fixed hardware: read once and reuse.

`self.glow() → list[int] | None`

The chamber's **current** resulting glow `[r,g,b]` given the lamps set right now, it reflects `set_lamps(...)` immediately, so use it to verify your solve before committing: `if self.glow() == target: self.infuse()`. `None` when the chamber is empty.

RETURNS

`list[int] | None`

The chamber's **current** resulting glow `[r,g,b]` given the lamps set right now (reflects `set_lamps` immediately), or `None` if the chamber is empty. Compare against the order's `target_glow` before `infuse()`.

`self.set_lamps(r: int, g: int, b: int) → ActionResult` SELF ONLY

Set the three lamp brightnesses, `self.set_lamps(7, 13, 4)`. Each is a **whole number 0-40**; the exact answer is always an integer, so `round()` your computed values. Fractional or out-of-range values raise an argument error (not silently floored). Re-idles to 0 when the script stops.

PARAMETERS

Name	Type	Description
`r`	`int`	Red lamp intensity.
`g`	`int`	Green lamp intensity.
`b`	`int`	Blue lamp intensity.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.infuse() → ActionResult` SELF ONLY

Produce a **Luminous** sample in `self.output`, preserving its other properties and replacing its previous glow with the tuned glow.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"output_full"`	rejection	The output has no capacity for the result.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.

`self.discard() → ActionResult` SELF ONLY

Stage the unchanged chamber sample in `self.output`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

DNA Sequencer

Splices genes into geothermal fragments so they carry what an order needs. You work at the gene level, load a fragment, read the genes it has and the genes it needs, splice the target set in, the machine composes the DNA for you. One splice per fragment.

Type	Atmosphere
Power in	-20 W (draws from grid)
Input buffer	Units: 10
Output buffer	Units: 10

HOW TO OBTAIN

1. Requires the **Gene Sequencing** research (Pressure 120).
2. Buy from the Shop for 100,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`self.chamber: ChamberFragment | None`

The `ChamberFragment` loaded right now, exposed as `self.chamber`, or `None`. Read `.genes` and `.spliced`; a second splice destroys an already-spliced sample.

RETURNS

`ChamberFragment | None`

`None` if the chamber is empty.

`self.input: InputSlot`

The `InputSlot` for geothermal samples. It holds one sample item id at a time and stays latched to that id until `load()` consumes the remaining units or `flush()` discards them. `stacks()` lists property-distinct variants and does not mean the port accepts multiple sample types. Inventory is a source only at Nocturna Base; remote Sequencers use a same-outpost Storage Bin/Warehouse.

RETURNS

`InputSlot`

For geothermal samples. Recover a mistaken property variant to an explicit local destination with `eject(...)` before loading it into the chamber.

`self.output: OutputSlot`

The `OutputSlot` for spliced or ejected samples. Exact sample properties are preserved.

RETURNS

`OutputSlot`

For samples whose genes `splice()` changed, or ejected samples, preserving every property.

METHODS

`self.load(fragment_id: str, properties: ItemProperties | None = None, property_match: str | None = None) → ActionResult` SELF ONLY

Pull a geothermal sample of `fragment_id` from `self.input` into the chamber, `self.load("gw_cardiac_node")`. Optional `properties` and `property_match` select a specific identity using the standard `any`, `subset`, or `exact` convention.

PARAMETERS

Name	Type	Description
`fragment_id`	`str`	A geothermal fragment id staged in `self.input`.
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"chamber_occupied"`	● rejection	The processing chamber already contains an item.
`"not_in_input"`	● rejection	The requested item is not present in the input.
`"invalid_fragment"`	● rejection	The supplied fragment identifier is invalid.
`"invalid_properties"`	● rejection	The supplied item-property selector is invalid.
`"invalid_property_match"`	● rejection	The supplied property-matching mode is invalid.
`"busy"`	● transient	The component is already performing another operation.
`"output_full"`	● rejection	Finished work is waiting for space in this machine's output, so nothing else can start. Free the output and the wait ends on its own.

`self.genes() → list[str] | None`

Lists the genes currently carried by the chambered fragment, such as `["cold_tolerance", "pressure_tolerance"]`, or returns `None` when empty. `splice()` takes time. On completion, the fragment moves to output and the chamber becomes empty. If output is full, the spliced fragment stays in the chamber until space is available.

RETURNS

`list[str] | None`

The genes currently carried by the chambered fragment, e.g. `["cold_tolerance", "pressure_tolerance"]`, or `None` if the chamber is empty. `splice()` takes time. Successful delivery empties the chamber; a completed splice waiting for output space retains its new genes in the chamber.

`self.gene_catalog() → list[str]`

Every gene id the machine can splice, `self.gene_catalog()` returns the full list (e.g. `["heat_resistance", "acid_resistance", "cold_tolerance", "pressure_tolerance", "toxin_resistance", "radiation_shield"]`), the valid values to pass to `splice()`.

RETURNS

`list[str]`

Every gene id the machine can splice (e.g. `["heat_resistance", "acid_resistance", "cold_tolerance", ...]`): the valid values for `splice()`.

`self.splice(genes: list[str]) → ActionResult` SELF ONLY

Replace the chambered fragment's gene set with exactly `genes`, then place it in `self.output` while preserving every unrelated property.

PARAMETERS

Name	Type	Description
`genes`	`list[str]`	A list of gene ids the engineered fragment must carry (from `gene_catalog()`, or an order's `required_genes[fragment_id]`).

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"destroyed"`	success	The target was destroyed.
`"empty"`	rejection	The relevant source or queue is empty.
`"unknown_gene"`	rejection	The supplied gene identifier does not exist.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`self.discard() → ActionResult` SELF ONLY

Stage the chamber fragment in `self.output` without splicing.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"empty"`	rejection	The relevant source or queue is empty.
`"busy"`	transient	The component is already performing another operation.
`"output_full"`	rejection	The output has no capacity for the result.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Feed Maker

Crafts the creature feeds that Habitat colonies eat, working like the Fabricator from a recipe you choose. Each creature's recipe unlocks through a Biolab order, so it only makes what you've unlocked. Each machine is upgraded to Mk II with its own Fabricator-made pack, unlocked at 250,000 Wildlife: 1.5× crafting and input Auto Feeder speed, with 2× operating power. Crafting can advance while the feeder cools down.

Type	Biosphere
Power in	Variable (draws from grid)
Produces	Depends on the selected recipe
Output buffer	Units: 50
Stockpile	200 units (mixed)
Recipes	16 available
Tiers	Mk II

HOW TO OBTAIN

1. Requires the **Wildlife** research (Plants 2,250,000).
2. Buy from the Shop for 50,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Loads forage and life forms into the Feed Maker. Connect a source with `self.input.connect("Forage Bin")`, then pull materials with `self.input.take("forage", 8)`. See `InputSlot`.

RETURNS

`InputSlot`

Load forage and life-forms: `self.input.connect("Forage Bin")` then `self.input.take("forage", 8)`. This is a multi-material stockpile; use `eject(...)` to recover a staged material while the machine is idle.

`.output: OutputSlot`

Sends finished feed to a bin or Habitat supply chain. Connect a destination with `self.output.connect("Feed Bin")`, then send feed with `self.output.send("feed_salt_tortoise", 10)`. See `OutputSlot`.

RETURNS

`OutputSlot`

Push finished feed onward: `self.output.connect("Feed Bin")` then `self.output.send("feed_salt_tortoise", 10)`.

METHODS

`.tier() → int`

The machine's upgrade tier: 1 for Mk I or 2 for Mk II.

RETURNS

`int`

1 for Mk I or **2** for Mk II.

`.list_recipes() → list[Recipe]`

Lists the feed recipes unlocked through Bio Lab orders, one `Recipe` per creature whose feed you can craft. Locked recipes do not appear. Each recipe includes its tier, id, name, inputs, output, duration, and power draw. Use `for recipe in self.list_recipes(): print(recipe.tier, recipe.id)` to discover what is available.

RETURNS

`list[Recipe]`

One per creature feed whose recipe a Biolab order has unlocked.

`.find_recipe(recipe_id: str) → Recipe | None`

Find one unlocked feed recipe by id without looping through `list_recipes()`. Returns its `Recipe` object, or `None` when the id is unknown, locked, or belongs to another machine.

PARAMETERS

Name	Type	Description
`recipe_id`	`str`	Feed Maker recipe id

RETURNS

`Recipe | None`

`None` if this Feed Maker cannot currently run that id.

`.set_recipe(recipe_or_id: str | Recipe | IdRecord) → ActionResult` SELF ONLY

Pick which creature feed to craft by its recipe id or by passing a `Recipe` from `list_recipes()`, e.g. `self.set_recipe("craft_feed_salt_tortoise")`. Once set, ProcessingSystem crafts automatically whenever the stockpile holds the inputs and the output bin has room.

PARAMETERS

Name	Type	Description
`recipe_or_id`	`str` | `Recipe` | `IdRecord`	Feed recipe id, a `Recipe` object, or a dictionary or class instance with a string `id` field. The recipe must belong to this machine and be unlocked.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_recipe"`	rejection	The supplied recipe is unknown or does not apply to this machine.
`"offline"`	transient	The component is offline.
`"recipe_locked"`	rejection	The selected recipe is locked.
`"busy"`	transient	The component is already performing another operation.
`"material_mismatch"`	rejection	The existing material does not match the requested material.

`.clear_recipe() → ActionResult` SELF ONLY

Unset the selected feed recipe and leave the Feed Maker idle. The input stockpile stays loaded.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"material_present"`	rejection	Existing material prevents the requested configuration change.

`.get_recipe() → str`

Returns the current recipe id, or `""` when none is set (or the set recipe is no longer unlocked). Use to check state before re-setting: `if self.get_recipe() == "": self.set_recipe(...)`.

RETURNS

`str`

Empty when no recipe is set, or when the set one is no longer unlocked.

POSSIBLE VALUES

`""`, `"craft_feed_salt_tortoise"`, `"craft_feed_magmatic_annelid"`, `"craft_feed_mycelial_husk"`, `"craft_feed_mantle_strider"`, `"craft_feed_glasswing_mantis"`, `"craft_feed_veil_mantle"`, `"craft_feed_vault_crab"`, `"craft_feed_tidal_cephalopod"`, `"craft_feed_bone_walker"`, `"craft_feed_vent_drifter"`, `"craft_feed_hive_sentinel"`, `"craft_feed_hollow_choir"`, `"craft_feed_ferric_sea_lily"`, `"craft_feed_crustal_echo"`, `"craft_feed_glacial_wyrm"`, `"craft_feed_spire_drake"`

`.get_recipe_inputs() → dict[str, int]`

A `dict` mapping each input `item_id` → units consumed per craft for the current recipe (empty `dict` if no recipe set). Iterate it to know what to stock: `for item, qty in self.get_recipe_inputs().items(): self.input.take(item, qty * 5)`.

RETURNS

`dict[str, int]`

`{item_id: count}` consumed per craft, or empty if no recipe is set. Iterate it to stock the right life-forms and forage.

`.get_stockpile() → dict[str, int]`

A `dict` mapping each `item_id` currently in the input stockpile → its unit count. Read it to see what's loaded before crafting.

RETURNS

`dict[str, int]`

`{item_id: count}` currently in the input stockpile.

`.get_stockpile_used() → int`

Total units across every material in the input stockpile. Compare to `get_stockpile_capacity()` to avoid overfilling.

RETURNS

`int`

Total units across every material in the input stockpile.

`.get_stockpile_capacity() → int`

Combined unit cap across all materials in the input stockpile. The cap is shared, many materials sum against one limit.

RETURNS

`int`

Combined unit cap across all materials in the input stockpile.

`.is_running() → bool`

`True` while a craft is actively advancing this tick (recipe set, inputs present, output has room). `False` when starved, output-full, idle, or powered off. Poll to detect a stalled line.

RETURNS

`bool`

`True` while a craft is actively advancing this tick.

`.get_progress() → float`

Fraction **0-1** through the current craft. Resets to 0 each time a craft completes (a batch of feed lands in the output bin) and starts again if inputs remain.

RETURNS

`float`

Progress through the current craft (**0-1**).

`.get_output_count() → int`

Completed feed units waiting in the output bin for pickup. Push them onward with `self.output.send(...)` before the bin fills (a full output bin stalls crafting).

RETURNS

`int`

Completed feed units waiting in the output bin for pickup.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Habitat

Revives one species from Biology reagents staged in this Habitat's dedicated local input, then breeds it into a colony. One living colony is allowed per species. Established colonies keep their progress when moved between Habitats.

Type	Biosphere
Power in	Variable (draws from grid)
Input buffer	Units: 50
Stockpile	25 units (mixed)
Tiers	Mk II

HOW TO OBTAIN

1. Requires the **Wildlife** research (Plants 2,250,000).
2. Buy from the Shop for 100,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.gas_in: FluidPort`

Supplies the Habitat with gas. Connect a Gas Tank with `self.gas_in.connect("gas_tank_1")`, then use `set_gas_intake(...)` to meter gas from this port into the enclosure. See `FluidPort` for level, capacity, flow, and connection queries.

RETURNS

`FluidPort`

Wire to a Gas Tank: `self.gas_in.connect("gas_tank_1")`. The tank fills this port; `set_gas_intake(...)` meters it into the enclosure.

`.liquid_in: FluidPort`

Supplies the Habitat with liquid. Connect a Liquid Tank with `self.liquid_in.connect("liquid_tank_1")`. See `FluidPort` for level, capacity, flow, and connection queries.

RETURNS

`FluidPort`

Wire to a Liquid Tank: `self.liquid_in.connect("liquid_tank_1")`.

`.input: InputSlot`

Stores feed for the colony. Connect a source with `self.input.connect("Feed Bin")`, then pull feed with `self.input.take("feed_salt_tortoise", 20)`. Compare `self.input.stacks()` with `required_feed()` after revival; only the colony's own feed item is useful. See `InputSlot`.

RETURNS

`InputSlot`

Stock the colony's feed: `self.input.connect("Feed Bin")` then `self.input.take("feed_salt_tortoise", 20)`. Compare `stacks()` with `required_feed()` after revival; recover the wrong feed into compatible storage at this outpost with `eject(...)`.

`.reagents: InputSlot`

Dedicated local `InputSlot` for revival reagents. Select a revival target, then connect a Storage Bin or Warehouse at this Habitat's outpost and stage all five Bio Lab reagents shown on the card before `revive()`. A Habitat at Nocturna Base may connect Base Inventory; a remote Habitat cannot.

RETURNS

`InputSlot`

A dedicated local input for the five Biology reagents consumed by `revive()`. Select a target, then connect a store at this Habitat's outpost and stage the rarity-scaled quantity shown on the card. Base Inventory is a valid source only for a Habitat at Nocturna Base. Recover unused reagents with `eject(...)`.

METHODS

`.set_revival_target(creature_id: str) → ActionResult` SELF ONLY

Select which cataloged creature this Habitat is preparing with `self.set_revival_target("salt_tortoise")`. The validated target persists when the script stops and after a failed setup check or rearing attempt, so the Habitat card can show the creature and its live preparation requirements before a colony exists. Selecting another valid creature replaces the target without consuming materials.

PARAMETERS

Name	Type	Description
`creature_id`	`str`	Cataloged creature id to prepare in this Habitat (e.g. `"salt_tortoise"`).

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"occupied"`	● rejection	The requested cell, sector, or Habitat is already occupied.
`"species_exists"`	● rejection	A living colony of this species already exists.
`"unknown_creature"`	● rejection	The supplied creature identifier does not exist.
`"not_cataloged"`	● rejection	The requested biological identity has not been cataloged.

`.revive()` → `ActionResult` SELF ONLY

Bring this Habitat's selected revival target to life with `self.revive()`. Stock at least **2** of its feed first: revival spends one and one must remain for rearing. Stage the exact rarity-scaled Bio Lab reagents shown by the Habitat or `journal.cataloged_creatures(...)`. Creature bonuses do not change this one-time recipe. Failed checks spend nothing and keep the selected target.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"occupied"`	rejection	The requested cell, sector, or Habitat is already occupied.
`"no_target"`	rejection	No target has been selected for this operation.
`"species_exists"`	rejection	A living colony of this species already exists.
`"not_cataloged"`	rejection	The requested biological identity has not been cataloged.
`"wrong_feed"`	rejection	The Habitat input contains feed for a different creature.
`"insufficient_feed"`	rejection	The Habitat does not contain enough of the required feed to start revival.
`"insufficient_reagents"`	rejection	The staged reagents do not satisfy the required recipe.

`.rehouse(creature_id: str) → ActionResult` SELF ONLY

Attach an established species colony to this Habitat with `self.rehouse("salt_tortoise")`. The destination must be empty and its current capacity must fit the entire colony. The same call moves a colony directly from another Habitat or restores one after its former Habitat was undeployed. Population, life stage, brood progress, Insight history, and purchased bonuses are preserved, and the population keeps counting toward Wildlife. Only growth pauses while a colony is unhoused.

PARAMETERS

Name	Type	Description
`creature_id`	`str`	Established creature colony to attach to this Habitat (e.g. `"salt_tortoise"`).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"occupied"`	● rejection	The requested cell, sector, or Habitat is already occupied.
`"no_colony"`	● rejection	No living colony exists for the requested species.
`"not_established"`	● rejection	The requested colony has not completed rearing.
`"insufficient_capacity"`	● rejection	The destination does not have enough capacity for the requested operation.
`"unknown_creature"`	● rejection	The supplied creature identifier does not exist.

`.set_gas_intake(rate: float) → ActionResult` SELF ONLY

Set the gas inflow rate in **t/h**, pulled from the connected Gas Tank (wire it with `self.gas_in.connect("gas_tank_1")`) into the enclosure's gas reserve. This is the regulator actuator: read `gas_level()`, compare it with `gas_band()`, and raise intake below the band or lower it above the band. **0** stops the inflow. The rate returns to **0** when the script stops, ends, or errors. Clamps to `>= 0`.

PARAMETERS

Name	Type	Description
`rate`	`float`	Gas inflow rate in t/h, pulled from the connected Gas Tank into the enclosure's gas reserve. Meter this to hold `gas_band()`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.

`.purge_intake(port: str | None = None) → ActionResult` SELF ONLY

Vents a feedstock inlet so it can accept a different fluid. Pass `"gas_in"` or `"liquid_in"` to vent one, or omit to vent both, venting only what is actually blocked leaves a healthy buffer alone. The inlets take the first fluid that reaches them and then refuse any other, so an enclosure supplied the wrong gas ends up holding one it cannot use with no way to take the right one. Purge, rewire, and the next correct delivery replaces the enclosure air: `self.purge_intake()` then `self.gas_in.connect("Sulfur Refiner")`. The vented fluid is destroyed; feed and colony progress are untouched.

PARAMETERS

Name	Type	Description
`port`	`str` | `None`	Inlet to vent: `"gas_in"` or `"liquid_in"`. Omit to vent both.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"empty"`	● rejection	The relevant source or queue is empty.

`.purge_reserve(medium: str) → ActionResult` SELF ONLY

Empties the selected enclosure reserve immediately: `self.purge_reserve("gas")` or `self.purge_reserve("liquid")`. The fluid is destroyed. The other reserve, inlet buffers, connections, intake settings, feed, and colony progress stay intact. The reserve can refill on later ticks if intake remains open.

PARAMETERS

Name	Type	Description
`medium`	`str`	Enclosure reserve to empty: `"gas"` or `"liquid"`. The selected fluid is destroyed.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The selected enclosure reserve was emptied and its fluid discarded.
`"empty"`	● success	The selected enclosure reserve was already empty.

RAISES

Exception	Condition
`ValueError`	`medium` must be `"gas"` or `"liquid"`.

`.set_liquid_intake(rate: float) → ActionResult` SELF ONLY

Set the liquid inflow rate in **t/h**, pulled from the connected Liquid Tank (`self.liquid_in.connect("liquid_tank_1")`) into the enclosure's liquid reserve. Meter it to hold `liquid_band()`. **0** stops the inflow. The rate returns to **0** when the script stops, ends, or errors. Clamps to `>= 0`.

PARAMETERS

Name	Type	Description
`rate`	`float`	Liquid inflow rate in t/h, pulled from the connected Liquid Tank into the enclosure's liquid reserve. Meter this to hold `liquid_band()`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.unlock_bonus(node_id: str) → ActionResult` SELF ONLY

Permanently purchase one node from this creature's tree. Pass a node id from `get_bonus_tree().nodes`. The **1 Insight** Adaptation affects this species only. The **4 Insight** Breakthrough affects every species and also requires this source colony to reach **10,000** population. A rejected purchase spends nothing.

PARAMETERS

Name	Type	Description
`node_id`	`str`	Node id from `get_bonus_tree().nodes`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_node"`	rejection	The supplied Habitat bonus node identifier does not exist for this creature.
`"already_purchased"`	success	The Habitat bonus node is already permanently purchased.
`"population_locked"`	rejection	The local colony has not reached the required population.
`"insufficient_insight"`	rejection	The shared Wildlife Insight balance is below the node cost.

`.tier() → int`

Permanently installed Habitat tier as an integer (**1-2**). Mk II doubles carrying capacity only; breeding speed and biological costs come from adaptations.

RETURNS

`int`

Permanently installed Habitat tier (**1-2**).

`.population() → int`

Current colony head count (individuals). Reads **0** for an empty Habitat OR one still in the Founded rearing window, a colony only counts (here and on the Wildlife sensor) once it's established. Breeds up toward `carrying_capacity()` and never falls.

RETURNS

`int`

Current colony head count (individuals). **0** for an empty Habitat or one still in the Founded rearing window (not yet established or counted on the sensor).

`.species() → str`

The housed creature id (e.g. `"glacial_wyrm"`), or `""` when the Habitat is empty. Use to look the creature up or branch your regulator per species.

RETURNS

`str`

The housed creature id, or `""` when the Habitat is empty.

POSSIBLE VALUES

`""`, `"salt_tortoise"`, `"magmatic_annelid"`, `"mycelial_husk"`, `"mantle_strider"`, `"glasswing_mantis"`, `"veil_mantle"`, `"vault_crab"`, `"tidal_cephalopod"`, `"bone_walker"`, `"vent_drifter"`, `"hive_sentinel"`, `"hollow_choir"`, `"ferric_sea_lily"`, `"crustal_echo"`, `"glacial_wyrm"`, `"spire_drake"`

`.revival_target() → str`

The creature id explicitly selected with `set_revival_target(...)`, or `""` before selection and after establishment. Failed setup checks and rearing attempts retain this value, while `species()` remains empty until revival actually starts.

RETURNS

`str`

The creature id explicitly selected for preparation, or `""` when no target is selected or the colony is established. A failed setup check or rearing attempt keeps the target selected.

POSSIBLE VALUES

`""`, `"salt_tortoise"`, `"magmatic_annelid"`, `"mycelial_husk"`, `"mantle_strider"`, `"glasswing_mantis"`, `"veil_mantle"`, `"vault_crab"`, `"tidal_cephalopod"`, `"bone_walker"`, `"vent_drifter"`, `"hive_sentinel"`, `"hollow_choir"`, `"ferric_sea_lily"`, `"crustal_echo"`, `"glacial_wyrm"`, `"spire_drake"`

`.life_stage() → str`

The colony's stage as a string: `"empty"`, `"founded"`, `"first_breeding"`, `"self_sustaining"`, `"thriving"`, or `"abundant"`. The colony advances as its population crosses each stage threshold. Later stages can change support requirements; earlier advances raise base capacity, but entering Abundant does not. Branch on the stage to scale up your supply: `if self.life_stage() == "thriving": self.set_liquid_intake(...)`.

RETURNS

`str`

The colony climbs as its own population crosses each stage threshold.

POSSIBLE VALUES

`"empty"`, `"founded"`, `"first_breeding"`, `"self_sustaining"`, `"thriving"`, `"abundant"`

`.is_established() → bool`

`True` once the colony cleared the Founded rearing window and is counting on the sensor; `False` during rearing or when empty. Gate breeding logic on it: `if self.is_established(): regulate()`.

RETURNS

`bool`

`True` once the colony cleared the Founded rearing window and counts on the sensor; `False` during rearing or when empty.

`.rearing_progress() → float`

Fraction **0-1** of the Founded rearing window held in-band. Climbs only while every active band is satisfied; reaches **1.0** to establish. Reads **0** when empty/established, and resets to 0 if a band is lost (rearing fails). Watch it during a fresh revive to confirm your regulator is holding.

RETURNS

`float`

Fraction of the Founded rearing window held in-band (**0-1**). Reaches **1.0** to establish. **0** when empty or established. Resets to 0 on a band loss (rearing fails: see `rearing_failed()`).

`.rearing_failed() → bool`

`True` after a rearing attempt lost its bands and reverted the Habitat to preparation. The genome and selected target are kept, fix your regulator and call `revive()` again. Clears when a target is selected or revival restarts.

RETURNS

`bool`

`True` after a rearing attempt lost its bands and reverted the Habitat to preparation. The genome and selected target are kept; fix your regulator and call `revive()` again. Clears when a target is selected or revival restarts.

`.brood_size() → int`

Whole individuals produced by the next completed breeding cycle. Normally **1**; brood bonuses build toward a guaranteed extra individual and periodically make it **2**. Reads **0** without an established colony or carrying-capacity space.

RETURNS

`int`

Whole individuals produced by the next completed breeding cycle. Normally **1**; brood bonuses build toward a guaranteed extra individual and periodically make it **2**. Reads **0** without an established colony or carrying-capacity space.

`.breeding_rate() → float`

Expected individuals/h at the current population, remaining capacity (headroom), life support, rarity, adaptation effects, and available local inputs. Population adds less and less extra speed above 10 individuals, while brood yield and purchased speed effects stay within their overall limits. This is the rate readout; `breeding_efficiency()` is only the life-support factor.

RETURNS

`float`

Expected individuals/h at current conditions. **0** before establishment or while blocked.

`.feed_level() → float`

Usable feed units remaining after partial consumption. Feed is consumed automatically only when individuals are born. Top it up with `self.input.take(...)`; over-stocking is harmless.

RETURNS

`float`

Usable feed units remaining after partial consumption. Feed is drawn automatically only when individuals are born. Keep it above the next birth's cost; over-stocking is harmless.

`.gas_level() → float`

Current gas held in the enclosure, in **tons**. Compare it with `gas_band()` and meter `set_gas_intake(...)` to hold it inside the window. It drops as the colony consumes gas while breeding and rises with intake.

RETURNS

`float`

Gas held in the enclosure now, in tons. Compare to `gas_band()`; meter `set_gas_intake(...)` to hold it inside the window.

`.liquid_level() → float`

Current liquid held in the enclosure, in **tons**. Compare it with `liquid_band()` and meter `set_liquid_intake(...)` to hold the window.

RETURNS

`float`

Liquid held in the enclosure now, in tons. Compare to `liquid_band()`; meter `set_liquid_intake(...)` to hold it inside the window.

`.get_insight() → HabitatInsight`

Read the shared Wildlife Insight balance and this colony's current and lifetime contribution. Insight accrues directly from positive population change. Static population and elapsed time alone produce nothing.

RETURNS

`HabitatInsight`

The shared balance, this colony's current production rate, and its lifetime contribution.

`.get_bonus_tree() → HabitatBonusTree`

Read this creature's species-only Adaptation and all-species Breakthrough, including each node's `.scope`, `.source_species`, permanent purchase state, current effect activity, Insight cost, and unmet requirements. The tree appears after selecting a revival target and its nodes remain visible while locked.

RETURNS

`HabitatBonusTree`

One species-only Adaptation and one all-species Breakthrough, with scope, source species, costs, gate, activity, and purchase state.

`.get_active_bonuses() → list[HabitatBonusNode]`

Read every purchased node currently affecting this Habitat, including global Breakthroughs earned from other species. Each result exposes `.scope` and `.source_species`. A conditional node disappears while its local condition is unmet; use its source Habitat's `get_bonus_tree()` to inspect permanent ownership.

RETURNS

`list[HabitatBonusNode]`

Purchased bonuses currently affecting this Habitat, including global Breakthroughs earned from other species.

`.gas_band() → list[float]`

Safe gas-inventory range `[low, high]` in **tons** for the colony's current stage. Breeding receives full gas support inside the range; too little starves the colony and too much is toxic. Returns an empty list before gas is required. Read it each loop because the range tightens as the colony grows.

RETURNS

`list[float]`

`[low, high]` in tons: the safe gas-inventory window for the colony's CURRENT life-stage. `gas_level()` inside `[low, high]` = breeding at 100% for gas; outside (too low OR too high) throttles. `[]` when gas isn't an active requirement yet (early stages).

`.liquid_band() → list[float]`

Safe liquid-inventory range `[low, high]` in **tons** for the colony's current stage. Returns an empty list before liquid is required. Read it each loop because the range tightens as the colony grows.

RETURNS

`list[float]`

`[low, high]` in tons: the safe liquid-inventory window for the colony's CURRENT life-stage. `[]` when liquid isn't an active requirement yet.

`.feed_ok() → bool`

`True` when the feed named by `required_feed()` is stocked. Feed is consumed automatically as individuals are born. `False` means the bin is empty or contains the wrong feed. Missing feed stalls breeding but does not reduce the established population.

RETURNS

`bool`

`True` when the feed named by `required_feed()` is stocked. Feed is consumed automatically as individuals are born. `False` (*X* on the card) means the bin is empty or holds the wrong feed.

`.gas_ok() → bool`

`True` when gas requirements are met or gas is not required yet. `False` means the level is outside `gas_band()` or the enclosure holds the wrong gas. If the level is inside the range, compare `gas_fluid()` with `required_gas()`.

RETURNS

`bool`

`True` when gas is in-band (or not yet required). `False` means EITHER `gas_level()` is outside `gas_band()` (adjust `set_gas_intake(...)`) OR the enclosure holds the wrong gas (`gas_fluid()` ≠ `required_gas()`: re-pipe the right exotic). Check both.

`.liquid_ok() → bool`

`True` when liquid requirements are met or liquid is not required yet. `False` means the level is outside `liquid_band()` or the enclosure holds the wrong liquid. Compare `liquid_fluid()` with `required_liquid()` to tell which.

RETURNS

`bool`

`True` when liquid is in-band (or not yet required). `False` means EITHER `liquid_level()` is outside `liquid_band()` OR the wrong liquid is held (`liquid_fluid()` ≠ `required_liquid()`). Check both.

`.breeding_efficiency() → float`

Current life-support multiplier from **0-100%**, not the population growth rate. The weakest active input sets it; **100%** means every input is ready. Growth also scales with population momentum, rarity, and bonuses. Capacity stops growth only when full and never slows breeding beforehand. Poor conditions never reduce population.

RETURNS

`float`

Life-support multiplier from **0-100%**, not the population growth rate. The weakest active input sets it; **100%** means every input is ready. Growth also scales with population momentum, rarity, and bonuses. Capacity stops growth only when full and never slows breeding beforehand. Poor conditions never reduce population.

`.gas_fluid() → str`

The exotic gas currently held in the enclosure, such as `"chlorine"`, or `""` when empty. It must equal `required_gas()` for the gas range to count. If `gas_level()` is in range but `gas_ok()` is `False`, the enclosure holds the wrong gas.

RETURNS

`str`

The exotic gas currently HELD in the enclosure (the fluid you piped in), or `""` when empty. The band only counts when this equals `required_gas()`; if `gas_level()` reads in-band but `gas_ok()` is `False`, you're holding the WRONG gas: compare these two to spot it.

POSSIBLE VALUES

`""`, `"steam"`, `"ammonia"`, `"swamp_gas"`, `"raw_sulfur_gas"`, `"sulfur_gas"`, `"raw_chlorine"`, `"chlorine"`

`.liquid_fluid() → str`

The exotic liquid currently held in the enclosure, or `""` when empty. It must equal `required_liquid()` for the liquid range to count.

RETURNS

`str`

The exotic liquid currently HELD in the enclosure, or `""` when empty. Must equal `required_liquid()` for the band to count; compare them to diagnose a wrong-liquid supply.

POSSIBLE VALUES

`""`, `"water"`, `"oil"`, `"frozen_essence"`, `"coastal_essence"`, `"geothermal_essence"`, `"volcanic_essence"`, `"deep_essence"`, `"brine"`, `"raw_cryofluid"`, `"cryofluid"`, `"raw_quicksilver"`, `"quicksilver"`

`.required_feed() → str`

The exact feed item id required by the selected revival target or housed creature, or `""` when neither exists. Compare it with `self.input.stacks()` while staging or diagnosing feed.

RETURNS

`str`

The exact feed item id required by the selected revival target or housed creature, or `""` when neither exists. Compare it with `self.input.stacks()` while staging or diagnosing feed.

POSSIBLE VALUES

`""`,
`"feed_salt_tortoise"`,
`"feed_magmatic_annelid"`,
`"feed_mycelial_husk"`,
`"feed_mantle_strider"`,
`"feed_glasswing_mantis"`,
`"feed_veil_mantle"`,
`"feed_vault_crab"`,
`"feed_tidal_cephalopod"`,
`"feed_bone_walker"`,
`"feed_vent_drifter"`,
`"feed_hive_sentinel"`,
`"feed_hollow_choir"`,
`"feed_ferric_sea_lily"`,
`"feed_crustal_echo"`,
`"feed_glacial_wyrm"`,
`"feed_spire_drake"`

`.required_gas() → str`

The exotic gas required at the colony's current stage, such as `"swamp_gas"`, or `""` before gas is needed. The requirement becomes more demanding at later stages, so check it again as the colony grows. Other gases do not satisfy the required range.

RETURNS

`str`

The exotic gas this creature needs at its CURRENT stage (some species need a rarer exotic at later stages, e.g. `"sulfur_gas"` → `"chlorine"`; a purchased Adaptation may retain the base gas instead), or `""` when gas isn't required yet. Pipe exactly this fluid; `gas_fluid()` must match it.

POSSIBLE VALUES

`""`, `"swamp_gas"`, `"ammonia"`, `"sulfur_gas"`, `"chlorine"`

`.required_liquid() → str`

The exotic liquid this creature needs at its current stage, e.g. `"brine"`; escalates at later stages (`"brine"` → `"cryofluid"` → `"quicksilver"`). `liquid_fluid()` must match it. `""` when liquid isn't required yet.

RETURNS

`str`

The exotic liquid this creature needs at its current stage (some species need a rarer exotic at later stages, e.g. `"cryofluid"` → `"quicksilver"`; a purchased Adaptation may retain the base liquid instead), or `""` when liquid isn't required yet. `liquid_fluid()` must match it.

POSSIBLE VALUES

`""`, `"brine"`, `"cryofluid"`, `"quicksilver"`

`.next_required_gas() → str`

The gas required after the next life-stage transition, or `""` if the next stage needs no gas or there is no next stage. Read it with `next_gas_band()` before the population reaches the threshold.

RETURNS

`str`

The exact gas required immediately AFTER the next life-stage transition, or `""` when that next stage has no gas requirement or no next stage exists. Pair with `next_gas_band()` to prepare before crossing.

POSSIBLE VALUES

`""`, `"swamp_gas"`, `"ammonia"`, `"sulfur_gas"`, `"chlorine"`

`.next_required_liquid() → str`

The liquid required after the next life-stage transition, or `""` if the next stage needs no liquid or there is no next stage. Read it with `next_liquid_band()` before the population reaches the threshold.

RETURNS

`str`

The exact liquid required immediately AFTER the next life-stage transition, or `""` when that next stage has no liquid requirement or no next stage exists. Pair with `next_liquid_band()`.

POSSIBLE VALUES

`""`, `"brine"`, `"cryofluid"`, `"quicksilver"`

`.next_gas_band() → list[float]`

The next life stage's exact gas window as `[low, high]` in **tons**. Returns `[]` if that stage needs no gas or the colony is already Abundant. Pair with `next_required_gas()` to prepare the correct supply and regulator in advance.

RETURNS

`list[float]`

`[low, high]` in tons, the exact gas band immediately AFTER the next life-stage transition. `[]` when that stage has no gas requirement or no next stage exists.

`.next_liquid_band() → list[float]`

The next life stage's exact liquid window as `[low, high]` in **tons**. Returns `[]` if that stage needs no liquid or the colony is already Abundant. Pair with `next_required_liquid()`.

RETURNS

`list[float]`

`[low, high]` in tons, the exact liquid band immediately AFTER the next life-stage transition. `[]` when that stage has no liquid requirement or no next stage exists.

`.carrying_capacity() → int`

The colony's current ceiling. Population breeds toward it and plateaus there; a maxed colony consumes nothing. Earlier stage advances raise base capacity, but Thriving and Abundant share a base of **175,000**. Habitat Mk II doubles the base, while having too many buildings at the outpost can lower the live ceiling. Adaptations never change capacity. Reads **0** when empty.

RETURNS

`int`

The colony's current ceiling. Earlier stage advances raise base capacity, but Thriving and Abundant share a base of **175,000**. Habitat Mk II doubles the base, while having too many buildings at the outpost can lower the live ceiling. Adaptations never change capacity. **0** when empty.

`.headroom() → int`

Individuals still breedable before the current ceiling (`carrying_capacity() - population()`, floored at **0**). **0** means this colony is capped and idling. Upgrade a capped Thriving Mk I Habitat to Mk II to make the **175,001** Abundant threshold reachable. Reduce overcrowding if it lowers the ceiling below a stage threshold. When an Abundant colony is fully capped, grow another species.

RETURNS

`int`

Individuals still breedable before the colony hits its current ceiling (`carrying_capacity() - population()`, floored at **0**). **0** means this colony is capped and idling.

`.next_stage_population() → int`

Exact population that opens the next life stage: **250**, **2,500**, **25,000**, or **175,001**. Returns **0** when the Habitat is empty or already Abundant. Mk I stops at **175,000**, so entering Abundant requires Mk II. Compare the result with `population()` so your script can prepare the next supply before the transition.

RETURNS

`int`

Whole-number population threshold that opens the next life-stage (**250**, **2,500**, **25,000**, or **175,001**). **0** when empty or already Abundant. Compare with `population()` to plan the next supply change.

`.next_requirement() → str`

What changes at the colony's next stage: `"capacity"`, `"gas"`, `"liquid"`, `"switch_gas"`, `"switch_liquid"`, `"tighter_bands"`, or `"none"` at Abundant. Pair it with `next_stage_population()` to learn when the transition happens and what to prepare.

RETURNS

`str`

What changes at the next stage: more `"capacity"`, a new `"gas"` or `"liquid"`, `"switch_gas"`, `"switch_liquid"`, `"tighter_bands"`, or `"none"` at Abundant. Pair with `next_stage_population()` to prepare before the transition.

POSSIBLE VALUES

`"capacity"`, `"gas"`, `"liquid"`, `"switch_gas"`, `"switch_liquid"`, `"tighter_bands"`, `"none"`

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Heat Generator

Warms the planet surface by producing heat. The best power setting shifts with the day's weather, so a script reads the conditions and holds the heater at the right level.

MK I

Type	Atmosphere
Power in	Variable (draws from grid)
Produces	up to 0.252 heat / day at peak efficiency
Input buffer	Units: 4
Tiers	Mk II, Mk III, and Mk IV

MK II

Produces	up to 1.184 heat / day at peak efficiency

MK III

Produces	up to 52.416 heat / day at peak efficiency
Consumes	Steam, up to 12 t per hour, buffer 8 t

MK IV

Produces	up to 129.780 heat / day at peak efficiency
Consumes	Fuel Rod, 0.1 per day

HOW TO OBTAIN

1. Buy from the Shop for 800 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Fuel Rod magazine for the Mk IV tier. Connect a Lead Cask to keep spare rods staged here; the heater swallows one whole rod at a time and burns it down internally. Empty and unused below Mk IV, and a Mk IV with no rod stops producing instead of falling back to Mk III output.

RETURNS

`InputSlot`

Magazine for the Mk IV Fuel Rod supply. Empty and unused below Mk IV.

`.steam_in: FluidPort`

Steam supply port installed by the Mk III pack. Call `connect(...)` with a compatible steam provider, then inspect `level()`, `capacity()`, `flow_rate()`, or `connected_to()`. Mk IV uses Fuel Rods instead, but the installed port remains available.

RETURNS

`FluidPort`

The steam supply installed by the Mk III pack. Call `connect(...)` with a compatible steam provider. Mk IV operation uses Fuel Rods instead, but the installed port remains available.

METHODS

`.set_power(watts: float) → ActionResult` SELF ONLY

Set base heater power from **0-10**; values outside that range are clamped. `0` turns heating off. The best positive setting depends on the current `thermal_state()`, so update it with `self.set_power(value)` as conditions change. Higher Mk tiers multiply grid draw without changing the best base setting. A poor setting wastes energy and reduces heat output. The setting returns to **0** when the script stops, ends, or errors.

PARAMETERS

Name	Type	Description
`watts`	`float`	Power draw in watts

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.thermal_state() → str`

Current daily heater thermal state, one of `"clear"`, `"dust_storm"`, `"heat_bleed"`, `"dust_veil"`. Each state has its own optimal positive `set_power()` value. The state is stable throughout the current day and changes only on a new day, so read it at the start of each iteration and branch when the string changes: `if state == "clear": self.set_power(5)` etc. Your job is figuring out the four optimal values.

RETURNS

`str`

POSSIBLE VALUES

`"clear"`, `"dust_storm"`, `"heat_bleed"`, `"dust_veil"`

`.efficiency() → float`

Current heating efficiency (**0-100%**). Hits **100%** only when `set_power()` exactly matches the current `thermal_state()`'s optimal, and falls off *steeply* around it (not linearly): about **31%** one step away, then a **10%** floor for any setting two or more steps off. Reads **0%** only when power is `0`. Scan positive power values and take the setting that reads **100%** as each state's optimal.

RETURNS

`float`

0-100%.

`.output() → float`

Current heat-unit production rate per hour at the current settings. Heat accumulates to raise surface temperature over many days; the sensor rate display projects per-day totals. Reflects `efficiency()` × the tier multiplier. Reads `0` if unpowered or no script running. Recomputed live on every read, a fresh `set_power(...)` is reflected immediately.

RETURNS

`float`

Current heat output rate, in heat units/h. Derived live; a fresh `set_power(...)` is reflected immediately.

`.tier() → int`

Permanently installed Mk tier as an integer (**1-4**). Upgrade packs raise this value; temporary Mk III steam starvation does not. Compare with `effective_tier()` when diagnosing a supplied or degraded heater.

RETURNS

`int`

Permanently installed Mk tier.

`.is_degraded() → bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick. Check after applying a Mk III pack, if `True`, your tier-3 heater is temporarily running as Mk II; look at `self.steam_in.level()` and the upstream thermal cap.

RETURNS

`bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick.

`.effective_tier() → int`

The tier actually in effect this tick: `tier()` normally, previous tier while `is_degraded()` is `True`. Scripts that rebalance steam flow between heaters should compare `effective_tier()` with `tier()`.

RETURNS

`int`

The tier actually in effect this tick: `tier()` normally, the previous tier while `is_degraded()` is `True`.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Oxygen Generator

Draws CO2 from the atmosphere and turns it into breathable oxygen, one of the core steps toward a livable planet. It runs only when a script sets its intake.

MK I

Type	Atmosphere
Power in	-8 W (draws from grid)
Produces	up to 0.240 ppt / day at peak efficiency
Input buffer	Units: 4
Tiers	Mk II, Mk III, and Mk IV

MK II

Power in	-40 W (draws from grid)
Produces	up to 1.200 ppt / day at peak efficiency

MK III

Power in	-160 W (draws from grid)
Produces	up to 48.000 ppt / day at peak efficiency
Consumes	Water, up to 8 t per hour, buffer 5 t

MK IV

Power in	-800 W (draws from grid)
Produces	up to 120.000 ppt / day at peak efficiency
Consumes	Fuel Rod, 0.1 per day

HOW TO OBTAIN

1. Buy from the Shop for 1,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Fuel Rod magazine for the Mk IV tier. Connect a Lead Cask to keep spare rods staged here; the generator swallows one whole rod at a time and burns it down internally. Empty and unused below Mk IV, and a Mk IV with no rod stops producing instead of falling back to Mk III output.

RETURNS

`InputSlot`

Magazine for the Mk IV Fuel Rod supply. Empty and unused below Mk IV.

`.water_in: FluidPort`

Water supply port installed by the Mk III pack. Call `connect(...)` with a compatible water provider, then inspect `level()`, `capacity()`, `flow_rate()`, or `connected_to()`. Mk IV uses Fuel Rods instead, but the installed port remains available.

RETURNS

`FluidPort`

The water supply installed by the Mk III pack. Call `connect(...)` with a compatible water provider. Mk IV operation uses Fuel Rods instead, but the installed port remains available.

METHODS

`.set_intake(value: float) → ActionResult` SELF ONLY

Set the CO2 intake rate. It holds until you set it again and returns to **0** when the script stops, ends, or errors. Call `self.set_intake(atmosphere.get_co2() / 10)` each iteration, the chamber's peak-efficiency sweet spot is exactly **1/10th** of ambient CO2. Values above or below that point reduce efficiency smoothly; there is no precision-sensitive cutoff.

PARAMETERS

Name	Type	Description
`value`	`float`	CO2 intake rate to request

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.waste() → float`

Current carbon waste level (**0-100**). Production runs clean below **60**, drops linearly **60-100**, and stalls (zero output) at **100**. Check this each iteration before calling `dump_waste()`, dumping between **50-60** is penalty-free; dumping elsewhere costs efficiency.

RETURNS

`float`

0-100.

`.dump_penalty() → float`

Current efficiency penalty from the last `dump_waste()` call (**0-1**). `0` means no active waste-clearing penalty; `0.25` means output is reduced by 25%. A bad dump remains visible here until the next `dump_waste()` call.

RETURNS

`float`

Active waste-clearing efficiency penalty, 0-1.

`.dump_waste() → WasteDumpResult` SELF ONLY

Clear accumulated carbon waste. Call `result = self.dump_waste()` when `waste()` is in the **50-60** sweet spot for a clean dump. The penalty lingers until the next dump.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`WasteDumpResult`	`.status`, `.message`	`.penalty`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The waste chamber was emptied. This dump applied an efficiency penalty of `.penalty`.

`.efficiency() → float`

Current conversion efficiency (**0-100%**). Hits **100%** when intake matches **CO2/10**, CO2 is available, waste is at or below **60**, and no dump penalty is active. Waste above **60** reduces efficiency; waste at **100** stalls production. The **50-60** range is the clean window for `dump_waste()`, dumping outside it applies an efficiency penalty you can read with `dump_penalty()`.

RETURNS

`float`

0-100%.

`.output() → float`

Current O2 production rate in **ppt/h** at the current settings. Reflects `efficiency()` × the tier multiplier, capped by available CO2. Reads `0` if the machine is unpowered, no script is running, CO2 is exhausted, or waste has stalled production. Recomputed live on every read, a fresh `set_intake(...)` is reflected immediately. Use this for live dashboards or to detect degradation mid-loop.

RETURNS

`float`

Current O2 output rate, in ppt/h.
Derived live; a fresh
`set_intake(...)` is reflected
immediately.

`.tier() → int`

Permanently installed Mk tier as an integer (**1-4**). Upgrade packs raise this value; temporary Mk III fluid starvation does not. Compare with `effective_tier()` when diagnosing a supplied or degraded generator.

RETURNS

`int`

Permanently installed Mk tier.

`.is_degraded() → bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick. Check after applying a Mk III pack and before trusting `output()` to meet your Mk III projections, if `True`, check `self.water_in.level()` and the upstream pipe.

RETURNS

`bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick.

`.effective_tier() → int`

The tier actually in effect this tick: `tier()` normally, previous tier while `is_degraded()` is `True`. Scripts that decide whether to route more water toward this generator should compare `effective_tier()` with `tier()`.

RETURNS

`int`

The tier actually in effect this tick: `tier()` normally, the previous tier while `is_degraded()` is `True`.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Pressure Generator

Compresses the thin atmosphere to raise surface pressure. It runs best when a script catches each sync window as its gauge sweeps; missed windows cost efficiency.

MK I

Type	Atmosphere
Power in	-7 W (draws from grid)
Produces	up to 0.012 kPa / day at peak efficiency
Input buffer	Units: 4
Tiers	Mk II, Mk III, and Mk IV

MK II

Power in	-35 W (draws from grid)
Produces	up to 0.300 kPa / day at peak efficiency

MK III

Power in	-140 W (draws from grid)
Produces	up to 2.400 kPa / day at peak efficiency
Consumes	Water, up to 5 t per hour, buffer 5 t

MK IV

Power in	-700 W (draws from grid)
Produces	up to 6.000 kPa / day at peak efficiency
Consumes	Fuel Rod, 0.1 per day

HOW TO OBTAIN

1. Buy from the Shop for 900 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Fuel Rod magazine for the Mk IV tier. Connect a Lead Cask to keep spare rods staged here; the generator swallows one whole rod at a time and burns it down internally. Empty and unused below Mk IV, and a Mk IV with no rod stops producing instead of falling back to Mk III output.

RETURNS

`InputSlot`

Magazine for the Mk IV Fuel Rod supply. Empty and unused below Mk IV.

`.water_in: FluidPort`

Water supply port installed by the Mk III pack. Call `connect(...)` with a compatible water provider, then inspect `level()`, `capacity()`, `flow_rate()`, or `connected_to()`. Mk IV uses Fuel Rods instead, but the installed port remains available.

RETURNS

`FluidPort`

The water supply installed by the Mk III pack. Call `connect(...)` with a compatible water provider. Mk IV operation uses Fuel Rods instead, but the installed port remains available.

METHODS

`.gauge() → float`

Current value on the resonance sweep (**0-100**). Rises each tick while the generator's script runs and wraps at **100**. With no script running, the sweep pauses where it is. Compare it to `next_window_low()` and `next_window_high()`; when the gauge is inside that range, both edges included, call `sync()`.

RETURNS

`float`

0-100.

`.next_window_low() → float`

Lower edge of the current sweep's sync window, included in the window. Use with `next_window_high()` and `gauge()`; call `sync()` when the gauge is inside the range. The value changes when the sweep wraps to the next cycle.

RETURNS

`float`

Lower bound of this cycle's sync window.

`.next_window_high() → float`

Upper edge of the current sweep's sync window, included in the window. Use with `next_window_low()` and `gauge()`; call `sync()` when the gauge is inside the range. The value changes when the sweep wraps to the next cycle.

RETURNS

`float`

Upper bound of this cycle's sync window.

`.sync() → ActionResult` SELF ONLY

Try to sync this sweep. First call per sweep counts; later calls before the gauge wraps do nothing. Hit (gauge in window) -> **+25%** efficiency. Miss (outside window, or no sync before the sweep ends) -> **-10%**.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.efficiency() → float`

Current compression efficiency (**0-100%**). Climbs with hits, drops with misses, floored at **0**. A well-tuned script holds this at or near **100%** by hitting every cycle's window. Use it to detect a script that's out of sync with the shifting window.

RETURNS

`float`

0-100%.

`.output() → float`

Current **kPa/h** production rate at the current `efficiency()`. Proportional to `efficiency()` × the tier multiplier. The resonance gauge advances slowly, so pressure progress is best judged from `efficiency()` and the per-day projection on the sensor display rather than from a single `output()` read.

RETURNS

`float`

Current pressure output rate, in kPa/h. Derived live from `efficiency()` and the tier multiplier.

`.tier() → int`

Permanently installed Mk tier as an integer (**1-4**). Upgrade packs raise this value; temporary Mk III fluid starvation does not. Compare with `effective_tier()` when diagnosing a supplied or degraded generator.

RETURNS

`int`

Permanently installed Mk tier.

`.is_degraded() → bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick. Check after applying a Mk III pack, if `True`, the pack is in fallback; investigate `self.water_in.level()` and the upstream supply.

RETURNS

`bool`

`True` when a Mk III pack is starved of its required fluid input and the machine has fallen back to the previous tier multiplier for this tick.

`.effective_tier() → int`

The tier actually in effect this tick: `tier()` normally, previous tier while `is_degraded()` is `True`. Scripts rebalancing water flow between generators should compare `effective_tier()` with `tier()`.

RETURNS

`int`

The tier actually in effect this tick: `tier()` normally, the previous tier while `is_degraded()` is `True`.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Refiner

Refines raw exotic feedstock into creature-grade gas or liquid, using tar as a reagent. Only uncommon and rare exotics need refining; commons are used directly. Recipes unlock through Biolab orders.

Type	Biosphere
Power in	Variable (draws from grid)
Produces	Depends on the selected recipe
Input buffer	Units: 50
Recipes	4 available

HOW TO OBTAIN

1. Requires the **Exotic Fluids** research (Wildlife 1,000).
2. Buy from the Shop for 80,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.gas_in: FluidPort`

Receives raw exotic gas. Connect a tank holding the raw gas with `self.gas_in.connect("Raw Chlorine Tank")`. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

Raw exotic GAS feedstock in.
Wire from a Gas Tank:
`self.gas_in.connect("Raw Chlorine Tank")`.

`.liquid_in: FluidPort`

Receives raw exotic liquid. Connect a tank holding the raw liquid with `self.liquid_in.connect("Raw Cryofluid Tank")`. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

Raw exotic LIQUID feedstock in. Wire from a Liquid Tank: `self.liquid_in.connect("Raw Cryofluid Tank")`.

`.gas_out: FluidPort`

Sends refined gas such as Sulfur Gas or Chlorine onward. Connect a destination tank with `self.gas_out.connect("Chlorine Tank")`. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

Refined exotic GAS out (Sulfur Gas / Chlorine). Wire to a Gas Tank: `self.gas_out.connect("Chlorine Tank")`.

`.liquid_out: FluidPort`

Sends refined liquid such as Cryofluid or Quicksilver onward. Connect a destination tank with `self.liquid_out.connect("Cryofluid Tank")`. See `FluidPort` for level, capacity, and flow queries.

RETURNS

`FluidPort`

Refined exotic LIQUID out (Cryofluid / Quicksilver). Wire to a Liquid Tank: `self.liquid_out.connect("Cryofluid Tank")`.

`.input: InputSlot`

Loads the tar consumed during refining. Connect a source with `self.input.connect("Tar Bin")`, then pull tar with `self.input.take("tar", 20)`. This slot holds only tar. See `InputSlot`.

RETURNS

`InputSlot`

Load the tar reagent: `self.input.connect("Tar Bin")` then `self.input.take("tar", 20)`. This is a single-material bin; use `eject(...)` to recover staged tar while the machine is idle.

METHODS

`.list_recipes() → list[Recipe]`

Lists the refining recipes unlocked through Bio Lab orders, one `Recipe` per exotic fluid. Locked recipes do not appear. Each recipe includes `.tier`, names its exact raw feedstock in `.input_fluid`, gives port tons consumed per run in `.fluid_inputs`, lists tar in `.inputs`, and identifies the refined product and output port through `.output_fluid` / `.fluid_outputs`. Use `for recipe in self.list_recipes(): print(recipe.tier, recipe.id, recipe.input_fluid, recipe.output_fluid)` to discover what is available.

RETURNS

`list[Recipe]`

One per refinable exotic whose recipe was unlocked by completing a Bio Order at the Bio Exchange. Raw feedstock is in `.input_fluid` / `.fluid_inputs`, the refined product and its port in `.output_fluid` / `.fluid_outputs`.

`.find_recipe(recipe_id: str) → Recipe | None`

Find one unlocked refining recipe by id without looping through `list_recipes()`. Returns its `Recipe` object, or `None` when the id is unknown, locked, or belongs to another machine.

PARAMETERS

Name	Type	Description
`recipe_id`	`str`	Refiner recipe id

RETURNS

`Recipe | None`

`None` if this Refiner cannot currently run that id.

`.set_recipe(recipe_or_id: str | Recipe | IdRecord) → ActionResult` SELF ONLY

Pick which exotic to refine by its recipe id or by passing a `Recipe` from `list_recipes()`, e.g. `self.set_recipe("refine_chlorine")`. Once set, the refiner crafts automatically whenever the raw feedstock + tar are present and the out port has room.

PARAMETERS

Name	Type	Description
`recipe_or_id`	`str` | `Recipe` | `IdRecord`	Refiner recipe id, a `Recipe` object, or a dictionary or class instance with a string `id` field. The recipe must belong to this machine and be unlocked.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_recipe"`	rejection	The supplied recipe is unknown or does not apply to this machine.
`"offline"`	transient	The component is offline.
`"recipe_locked"`	rejection	The selected recipe is locked.
`"busy"`	transient	The component is already performing another operation.
`"output_busy"`	transient	Incompatible fluid remains in the output, preventing the recipe change.

`.clear_recipe() → ActionResult` SELF ONLY

Unset the selected refine recipe and leave the Refiner idle. Tar and raw feedstock inputs are preserved because they are staged supply.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"material_present"`	rejection	Existing material prevents the requested configuration change.

`.purge_input() → ActionResult` SELF ONLY

Vents whatever raw feedstock is sitting in `gas_in` and `liquid_in`, releasing the port so it can accept a different fluid. The feedstock ports take the first fluid that reaches them and then only accept that one, so a port wired to the wrong Cap holds a fluid the recipe cannot use. Purge it, rewire, and carry on: `self.purge_input()` then `self.gas_in.connect("Raw Sulfur Cap")`. The vented fluid is destroyed, and tar in the input bin is untouched.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"empty"`	● rejection	The relevant source or queue is empty.
`"busy"`	● transient	The component is already performing another operation.

`.get_recipe() → str`

Returns the current recipe id, or `""` when none is set (or the set recipe is no longer unlocked). Use to check state before re-setting: `if self.get_recipe() == "": self.set_recipe("refine_sulfur_gas")`.

RETURNS
`str`
Empty when no recipe is set, or when the set one is no longer unlocked.

POSSIBLE VALUES
`""`, `"refine_sulfur_gas"`, `"refine_cryofluid"`, `"refine_chlorine"`, `"refine_quicksilver"`

`.get_recipe_inputs() → dict[str, int]`

A `dict` mapping each input `item_id` → units consumed per craft, for the Refiner this is the **tar** cost, e.g. `{"tar": 5}` for a rare exotic. Empty `dict` if no recipe is set. Read it to keep the tar bin stocked: `for item, qty in self.get_recipe_inputs().items(): self.input.take(item, qty * 5)`. (The raw-feedstock fluid amount is metered on the input ports, not listed here.)

RETURNS

`dict[str, int]`

`{item_id: count}` consumed per craft: for the Refiner this is the tar cost, e.g. `{"tar": 5}`. Empty if no recipe is set. The raw-feedstock fluid cost is on the input ports, not here.

`.is_running() → bool`

`True` while a refine craft is actively advancing this tick (recipe set + unlocked, raw feedstock + tar present, out port has room). `False` when stalled, idle, or powered off.

RETURNS

`bool`

`True` while a refine craft is actively advancing this tick (recipe set and unlocked, raw feedstock and tar present, out port has room).

`.is_stalled() → bool`

`True` when the refiner is powered and a recipe is set but the craft can't advance, for example because raw feedstock is missing (check `self.gas_in.level()` / `self.liquid_in.level()`), tar has run out (refill the input bin), or the refined-fluid out port is full (downstream backpressure, drain the out tank). `False` when unpowered, running, or no recipe is set. Poll to diagnose a stuck line.

RETURNS

`bool`

`True` when the refiner is powered and a recipe is set but the craft can't advance, for example because raw feedstock or tar is missing, or the refined-fluid out port is full (downstream backpressure). `False` when unpowered, running, or no recipe is set.

`.get_rate() → float`

Current refined exotic output rate in t/h, measured this tick. **0** when stalled or idle. Use to confirm throughput while balancing feedstock against demand.

RETURNS

`float`

Refined exotic produced this tick, in t/h. **0** when stalled or idle.

`.get_progress() → float`

Fraction **0-1** through the current refine craft. Resets to 0 each time a craft completes (a batch of refined fluid lands in the out port) and starts again if the feedstock + tar remain.

RETURNS

`float`

Progress through the current refine craft (**0-1**).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Terraforming

Read progress, completion targets, and milestones for the six terraforming pillars through `get_component("terraforming")`. The Terraform Index combines their capped contributions toward a fixed 1,000,000 TP goal.

Access via: `get_component("terraforming")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.pillars() → list[TerraformPillar]`

Read fresh snapshots of the available pillars in order: temperature, oxygen, pressure, then biomass, plants, and wildlife. The three Biosphere pillars appear together after Biosphere research. Call again to read updated progress.

RETURNS

`list[TerraformPillar]`

Fresh `TerraformPillar` snapshots for the unlocked pillars.

`.get_pillar(pillar_id: str) → TerraformPillar | None`

Read a fresh snapshot of one pillar, for example `terraforming.get_pillar("biomass")`. Returns `None` only when that pillar is locked by Biosphere research. Unknown ids raise `ValueError`.

PARAMETERS

Name	Type	Description
`pillar_id`	`str`	One of `"temperature"`, `"oxygen"`, `"pressure"`, `"biomass"`, `"plants"`, or `"wildlife"`.

RETURNS

`TerraformPillar | None`

A fresh `TerraformPillar` snapshot, or `None` for a pillar locked by Biosphere research.

RAISES

Exception	Condition
`ValueError`	The supplied `pillar_id` is not one of the six terraforming pillar ids.

`.index_progress() → float`

Read the overall Terraform Index percentage toward its fixed 1,000,000 TP goal. Atmosphere accounts for 70% and Biosphere for 30%; unlocking Biosphere does not change the goal.

RETURNS

`float`

Terraform Index percentage on a **0-100** scale, with a fixed 1,000,000 TP goal.

`.total_tp() → int`

Read the Terraform Index in whole TP, capped at 1,000,000. Completing one pillar cannot replace progress in another.

RETURNS

`int`

Whole Terraform Index points in the **0-1,000,000** range.

Fabricator

Assembles finished parts from several refined materials at once. A script picks a recipe, gathers each ingredient into its shared stockpile, and drains the finished items out.

Type	Mining
Power in	Variable (draws from grid)
Consumes	Steam, buffer 10 t
Consumes	Water, buffer 10 t
Consumes	Oil, buffer 10 t
Output buffer	Units: 20
Byproduct buffer	Units: 20
Stockpile	200 units (mixed)
Recipes	75 available
Tiers	Mk II and Mk III

HOW TO OBTAIN

1. Requires the **Fabrication** research (Terraform Index 130,000).
2. Buy from the Shop for 1,400 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.output: OutputSlot`

Output I/O port for finished items. Inventory works at home only; remote Fabricators connect a local Storage Bin or Warehouse. Requires **Auto Feeders** research. See `OutputSlot`.

RETURNS

`OutputSlot`

`.byproduct: OutputSlot`

Secondary output port for **byproducts**. Most recipes only emit through `self.output`; oil-refining recipes (lubricant, plastic, rubber) emit `tar` here on every craft. Same API as `self.output`: `self.byproduct.connect("Tar Bin")` then drain with `self.byproduct.send("tar", count)`. Requires **Auto Feeders** research. The byproduct bin must have space for the next craft, otherwise the recipe stalls, you have to keep this drained, not just `self.output`. See `OutputSlot`.

RETURNS

`OutputSlot`

Recipes that emit a byproduct
(e.g. oil refining produces tar)
deposit it here.

`.steam_in: FluidPort`

Internal steam process buffer for recipes that declare `fluid_inputs["steam_in"]`. Call `self.steam_in.connect(...)` with a Thermal Cap or steam Gas Tank's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free gas-pipe route between both locations. Read `self.steam_in.level()` / `.flow_rate()` to detect starvation. **10** ton buffer. See `FluidPort`.

RETURNS

`FluidPort`

Process buffer. Call `connect(...)` with the provider's stable machine id or display name; a local provider transfers directly, while a remote provider uses any completed gas network reaching both outposts. Recipes with `fluid_inputs["steam_in"]` consume from it on craft completion.

`.water_in: FluidPort`

Internal water process buffer for recipes that declare `fluid_inputs["water_in"]`. Call `self.water_in.connect(...)` with a Water Pump, Steam Condenser, water Liquid Tank, or Large Liquid Tank's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free liquid-pipe route between both locations. Read `self.water_in.level()` / `.flow_rate()` to detect starvation. **10** ton buffer. See `FluidPort`.

RETURNS

`FluidPort`

Process buffer. Call `connect(...)` with the provider's stable machine id or display name; a local provider transfers directly, while a remote provider uses any completed liquid network reaching both outposts. Recipes with `fluid_inputs["water_in"]` consume from it on craft completion.

`.oil_in: FluidPort`

Internal oil process buffer for recipes that declare `fluid_inputs["oil_in"]` (oil-refining recipes). Call `self.oil_in.connect(...)` with an Oil Pump, oil Liquid Tank, or Large Liquid Tank's stable machine id or display name. A local source transfers directly; a remote source also needs a completed conflict-free liquid-pipe route between both locations. Read `self.oil_in.level()` / `.flow_rate()` to detect starvation. **10** ton buffer. See `FluidPort`.

RETURNS

`FluidPort`

Process buffer. Call `connect(...)` with the provider's stable machine id or display name. A complete oil provider-consumer connection sets the reachable pipe network's contents to oil even while idle, full, or off; mere machine presence does not. Recipes with `fluid_inputs["oil_in"]` consume from it on craft completion.

METHODS

`.tier() → int`

Installed machinery tier: `1` for Mk I, `2` for Mk II, or `3` for Mk III where supported.

RETURNS

`int`

`.list_recipes() → list[Recipe]`

Every recipe this fabricator has been given a blueprint for. Returns `Recipe` objects with `.tier`, `.id`, `.name`, `.inputs`, `.output_item`, `.output_count`, `.duration_game_hours`, `.power_draw`, `.fluid_inputs` (tons consumed per run), and optional byproduct fields. Locked recipes (no blueprint yet) do not appear, the list reflects what the player can actually craft today. `sorted(self.list_recipes(), key=lambda recipe: recipe.tier)` orders the available queue from foundations upward.

RETURNS

`list[Recipe]`

The recipes this Fabricator has unlocked.

`.find_recipe(recipe_id: str) → Recipe | None`

Find one unlocked recipe by id without looping through `list_recipes()`. Returns its `Recipe` object, or `None` when the id is unknown, locked, or belongs to another machine.

PARAMETERS

Name	Type	Description
`recipe_id`	`str`	Fabricator recipe id

RETURNS

`Recipe | None`

`None` if this Fabricator cannot currently run that id.

`.set_recipe(recipe_or_id: str | Recipe | IdRecord) → ActionResult` SELF ONLY

Select which recipe to assemble: `self.set_recipe("craft_gas_pipe_segment")`, or pass a `Recipe` from `list_recipes()`. Setting a recipe doesn't clear the stockpile, so leftovers from a previous recipe stay until consumed or `self.input.flush()` discards them.

PARAMETERS

Name	Type	Description
`recipe_or_id`	`str` | `Recipe` | `IdRecord`	Recipe id, a `Recipe` object, or a dictionary or class instance with a string `id` field. The recipe must belong to this machine and be unlocked.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_recipe"`	rejection	The supplied recipe is unknown or does not apply to this machine.
`"offline"`	transient	The component is offline.
`"recipe_locked"`	rejection	The selected recipe is locked.
`"busy"`	transient	The component is already performing another operation.
`"material_mismatch"`	rejection	The existing material does not match the requested material.

`.clear_recipe() → ActionResult` SELF ONLY

Unset the selected recipe and leave the Fabricator idle. The input stockpile is preserved because it is general staged material, not the selected recipe.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"material_present"`	rejection	Existing material prevents the requested configuration change.

`.get_recipe() → str`

Current recipe id as a string, or the empty string if no recipe is set. Use to gate other logic or confirm after `set_recipe()`.

RETURNS

`str`

Empty when no recipe is set.

POSSIBLE VALUES

`""`, `"craft_smelter_pack_mk2"`, `"craft_fabricator_pack_mk2"`, `"craft_oil_pump_pack_mk2"`, `"craft_smelter_pack_mk3"`, `"craft_fabricator_pack_mk3"`, `"craft_gas_pipe_segment"`, `"craft_liquid_pipe_segment"`, `"craft_power_line_segment"`, `"craft_gas_pipe_bridge"`, `"craft_liquid_pipe_bridge"`, `"craft_power_line_bridge"`, `"craft_pressure_valve"`, `"craft_machine_frame"`, `"craft_circuit_panel"`, `"craft_control_unit"`, `"craft_battery_cell"`, `"craft_thermal_cap_kit"`, `"craft_turbine_rotor"`, `"craft_tank_lining"`, `"craft_water_pump"`, `"craft_oil_pump"`, `"craft_lubricant"`, `"craft_plastic"`, `"craft_rubber"`, `"craft_tar"`, `"craft_reinforced_biopolymer"`, `"craft_enrichment_compoun`

d",
"craft_drone_station_kit",
"craft_drone_station_kit_medium",
"craft_drone_station_kit_large",
"craft_drone_service_station_kit",
"craft_mining_drill_kit",
"craft_mining_drill_industrial_kit",
"craft_mining_drill_heavy_kit",
"craft_drone_small",
"craft_drone_medium",
"craft_drone_large",
"craft_electric_thruster", "craft_heli_thruster",
"craft_cargo_pod_small",
"craft_cargo_pod_medium",
"craft_cargo_pod_large",
"craft_battery_pack",
"craft_oil_tank_small",
"craft_oil_tank_medium",
"craft_oil_tank_large",
"craft_coolant_loop",
"craft_neutron_capacitor",
"craft_seed_maker_kit",
"craft_plant_terraformer_kit",
"craft_grow_lamp_kit",
"craft_sprinkler_kit",
"craft_dispenser_kit",
"craft_garbage_disposal_kit",
"craft_exotic_gas_cap_kit",
"craft_exotic_spring_tap_kit", "craft_fertilizer",
"craft_fertilizer_mk2",

```
"craft_fertilizer_mk3",
"craft_growth_accelerant",
"craft_yield_amplifier",
"craft_plant_terraformer_pack_mk2",
"craft_grow_lamp_pack_mk2",
"craft_grow_lamp_pack_mk3",
"craft_sprinkler_pack_mk2",
"craft_sprinkler_pack_mk3",
"craft_feed_maker_pack_mk2",
"craft_habitat_pack_mk2",
"craft_lead_plate",
"craft_oxygen_upgrade_pack_mk4",
"craft_heat_upgrade_pack_mk4",
"craft_pressure_upgrade_pack_mk4",
"craft_lead_cask",
"craft_shield_plating",
"craft_lightning_rod_kit"
```

`.get_recipe_inputs() → dict[str, int]`

Input requirements for the current recipe as a `dict` `{item_id: count_per_craft}`. Empty `dict` if no recipe is set. Use with `.keys()` / `.values()` / `.items()` to drive a loop: `for mat, need in self.get_recipe_inputs().items(): self.input.connect(bin_for(mat)); self.input.take(mat, need)`.

RETURNS

`dict[str, int]`

`{item_id: count}` consumed per craft, or empty if no recipe is set.

`.get_stockpile() → dict[str, int]`

Current stockpile contents as a `dict` `{item_id: count_currently_stored}`. Iterate with `.items()` to see every material; index directly with `self.get_stockpile()["iron_ingot"]` to read one. Essential for deciding what else needs pulling in.

RETURNS

`dict[str, int]`

`{item_id: count}` currently stored.

`.get_stockpile_used() → int`

Total units across every material in the stockpile. Compare to `get_stockpile_capacity()` to detect when the pile is full. When it is full, further input is blocked until the running craft consumes some material.

RETURNS

`int`

Total units across every material.

`.get_stockpile_capacity() → int`

Combined unit cap across all materials (fixed for this fabricator). Queryable rather than hardcoded, the cap tunes separately from your script. Use `used / capacity` for a fill-percent gauge.

RETURNS

`int`

Combined unit cap across all materials.

`.is_running() → bool`

`True` while a craft is in progress. Use before `set_recipe()` to avoid `"busy"`, or to show status. Stays `True` across ticks until the craft completes.

RETURNS

`bool`

`.get_progress() → float`

Progress toward the current craft's completion (**0-1**). Resets to **0** when a craft finishes. Use for progress bars and to detect completions by watching the value drop.

RETURNS

`float`

Progress through the current craft (**0-1**).

`.get_output_count() → int`

Completed units waiting in the output buffer. Drain them via `self.output.send(...)` before the buffer fills, processing stalls when the output is full.

RETURNS

`int`

Completed units waiting for pickup.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Fuel Assembler

Presses Raw Uranium and lead plates into Fuel Rods or Nuclear Batteries, working like the Fabricator. It draws heavy recipe power while running, so it is best run in bursts when your lightning banks are full.

Type	Mining
Power in	Variable (draws from grid)
Produces	Depends on the selected recipe
Output buffer	Units: 5
Stockpile	40 units (mixed)
Recipes	2 available

HOW TO OBTAIN

1. Requires the **Fuel Assembler** research (Temperature 10,000).
2. Buy from the Shop for 225,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Receives Raw Uranium only from a Lead Cask and lead plates from an ordinary compatible local source. The port has one source relationship at a time, so reconnect it between materials.

RETURNS

`InputSlot`

Stages Raw Uranium and lead plates. Raw Uranium may enter only from a Lead Cask; reconnect the one-source input to an ordinary local source for lead plates. While idle, use `eject(...)` to recover ordinary items into compatible storage at this outpost and radioactive cargo into a compatible Lead Cask.

`.output: OutputSlot`

Output port. Fuel Rods are radioactive and only a Lead Cask (or their exact Supply Dock order) accepts them. Nuclear Batteries are ordinary fabricated products and may go to compatible local storage or home Inventory.

RETURNS

`OutputSlot`

Route Fuel Rods to a separate Lead Cask (or their exact Supply Dock order). Nuclear Batteries are ordinary fabricated products and may route to ordinary compatible local storage or home Inventory.

METHODS

`.list_recipes() → list[Recipe]`

Unlocked recipes this machine can run, including each recipe's derived `.tier`. The Fuel Rod recipe arrives through Vestibule's queue; the Nuclear Battery recipe arrives through Helios's queue.

RETURNS

`list[Recipe]`

The unlocked recipes this machine can run.

`.find_recipe(recipe_id: str) → Recipe | None`

Find one unlocked fuel recipe by id without looping through `list_recipes()`. Returns its `Recipe` object, or `None` when the id is unknown, locked, or belongs to another machine.

PARAMETERS

Name	Type	Description
`recipe_id`	`str`	Fuel Assembler recipe id

RETURNS

`Recipe | None`

`None` if this Fuel Assembler cannot currently run that id.

`.set_recipe(recipe_or_id: str | Recipe | IdRecord) → ActionResult` SELF ONLY

Select a Fuel Rod or Nuclear Battery recipe by id or by passing a `Recipe` from `list_recipes()`.

PARAMETERS

Name	Type	Description
`recipe_or_id`	`str` | `Recipe` | `IdRecord`	Fuel recipe id, a `Recipe` object, or a dictionary or class instance with a string `id` field. The recipe must belong to this machine and be unlocked.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_recipe"`	rejection	The supplied recipe is unknown or does not apply to this machine.
`"recipe_locked"`	rejection	The selected recipe is locked.
`"offline"`	transient	The component is offline.
`"busy"`	transient	The component is already performing another operation.
`"material_mismatch"`	rejection	The existing material does not match the requested material.

`.clear_recipe() → ActionResult` SELF ONLY

Release the recipe once the current craft is idle and the output buffer is drained.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"material_present"`	rejection	Existing material prevents the requested configuration change.

`.get_recipe() → str`

The committed recipe id, empty when none.

RETURNS
`str`
Empty when no recipe is set.

POSSIBLE VALUES
`""`, `"craft_fuel_rod"`, `"craft_nuclear_battery"`

`.get_recipe_inputs() → dict[str, int]`

Input requirements for the committed recipe as a `dict` `{item_id: count_per_craft}`. Returns an empty `dict` when no recipe is committed.

RETURNS
`dict[str, int]`
`{item_id: count}` consumed per craft, or empty if no recipe is set.

`.is_running() → bool`

`True` while a craft is actually advancing, power, inputs, and output space all present.

RETURNS

`bool`

`True` while a craft is advancing this tick.

`.get_progress() → float`

Current craft progress **0-1**. Progress survives power cuts and resumes.

RETURNS

`float`

Progress through the current craft (**0-1**).

`.get_stockpile() → dict[str, int]`

Staged inputs by item id, `{"raw_uranium": 12, "lead_plate": 4}`-shaped `dict`.

RETURNS

`dict[str, int]`

Staged input materials, by item id.

`.get_output_count() → int`

Finished products for the selected recipe waiting in the small output buffer.

RETURNS

`int`

Finished products for the selected recipe waiting in the output buffer.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Harvester

A slow general-purpose surface vehicle that collects loose items, plants and tends crops, harvests Forage, and deploys fixed field machines. Movement and field work take time and build heat, so long routes need cooling pauses.

Type	Harvesting

Access via: `self`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.move(sector: str) → ActionResult` SELF ONLY

Move one sector up, down, left, or right with `self.move("E14")`; diagonal moves are invalid. Travel takes **0.5 hours** and pauses the script. `self.get_position()` shows the destination immediately, but physical actions stay locked until arrival. Moving into an item sector adds **1** heat; moving into an empty one adds **7**, so scan first.

PARAMETERS

Name	Type	Description
`sector`	`str`	Adjacent grid sector id to move to (one cardinal step).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.
`"too_far"`	rejection	The requested target is outside interaction distance.
`"already_here"`	success	The vehicle is already at the requested destination.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.collect()` → `ScanResult` SELF ONLY

Pick up the item in the current sector into the Harvester's single held slot, not Inventory. Collection takes **0.25 hours** and pauses the script. Empty sectors still take time and add **9** heat.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ScanResult`	`.status`, `.message`	`.id`, `.name`, `.value`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Collected `item_id` into the Harvester's held slot.
`"empty"`	success	The current sector contains no collectible item.
`"holding"`	rejection	The Harvester's held slot is occupied.
`"overheated"`	rejection	The Harvester has reached its heat limit, or the attempted collection would exceed it.
`"moving"`	transient	The Harvester is still completing a physical movement.
`"busy"`	transient	The Harvester is occupied by another field action.
`"collecting"`	transient	A previous collection cycle is still in progress.

`.store()` → `ItemResult` SELF ONLY

Move the Harvester's held item into the first empty inventory slot, freeing the held slot for the next pickup. It works from any sector; the Harvester never has to return to a base sector to unload. A full inventory leaves the item held; drop it or free space by selling through the Shop.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ItemResult`	`.status`, `.message`	`.item_id`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation affected item `.item_id`.
`"empty"`	success	The selected item source was already empty.
`"inventory_full"`	rejection	Inventory has no capacity for the held item.

`.drop() → ActionResult` SELF ONLY

Release the currently held item into the Harvester's current sector, the cell becomes collectable again. Use it to stage items for later pickup or clear the held slot without storing.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"dropped"`	success	The item was removed from its source.
`"empty"`	rejection	The relevant source or queue is empty.
`"occupied"`	rejection	The requested cell, sector, or Habitat is already occupied.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.get_held() → str`

Item id currently held by the Harvester, or empty string if the held slot is empty. Call `self.get_held()` before `collect()` (if non-empty, the slot is busy) or before `self.store()` (if empty, nothing to store). Essential first line of any collect loop.

RETURNS
`str`
The held item id, or empty if holding nothing.

`.get_position() → str`

Current sector id as a string (e.g. `"E14"`). Use to plan the next `move()` target, movement is strictly to adjacent cells, so the script needs to know where it is to compute where it can go.

RETURNS
`str`
The current sector.

`.get_heat() → float`

Exact current heat level (**0-100**), including fractional cooling between whole heat costs. Every `move()` adds heat, and an empty-sector `collect()` attempt also adds heat; passive cooling runs continuously as game time passes, including during movement and collection. Read before moving, if you're close to **100** and your planned route crosses empty cells (+7 heat each), stop and cool. Losing the route to `"overheated"` mid-sweep wastes hours.

RETURNS

`float`

0-100.

`.get_max_heat() → int`

Maximum heat capacity, always **100**. Reaching this stalls all movement and collection until heat drops below the cap. Exposed as a method so scripts can reason about thresholds without hardcoding the number.

RETURNS

`int`

Always 100.

`.is_overheated() → bool`

`True` when heat `>= 100`. Shortcut for `self.get_heat() >= self.get_max_heat()`. Use it as an early-exit guard when you intentionally want the Harvester to cool: `if self.is_overheated(): sleep(1)`. The Harvester cools passively whenever game time advances.

RETURNS

`bool`

`.water_level() → float`

Read the water currently in the Harvester's onboard tank, in tons, including fractional amounts. Each watering uses **1 t**. Compare with `water_capacity()` to plan refills from the home outpost's water tanks.

RETURNS

`float`

Onboard water, in tons, including fractional amounts.

`.water_capacity() → float`

Read the Harvester's maximum onboard water supply in tons, currently **5 t**. Compare with `water_level()` to check how full the tank is.

RETURNS

`float`

Onboard water capacity, in tons.

`.load_seed(seed: str) → ActionResult` SELF ONLY

Transfer one species seed from home Inventory, from anywhere on the grid, into the Harvester's single held slot. Use the same seed id with `plant(seed)` after driving to an empty field cell. Loading fails while another item is held.

PARAMETERS

Name	Type	Description
`seed`	`str`	Seed item id from the Seed Maker. Loading transfers one seed from Inventory into the Harvester's single held slot, from anywhere on the grid.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"invalid_seed"`	rejection	The supplied item is not a plant seed.
`"no_seed"`	rejection	The requested seed is not available.
`"holding"`	rejection	The Harvester's single held slot is already occupied.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.plant(seed: str) → ActionResult` SELF ONLY

Sow the physical seed currently in the Harvester's held slot into this empty cell. The argument must identify that held seed. The base sector is depot ground and refuses sowing. Sowing takes **0.5 hours**; the held seed is consumed and the new crop appears only after the action finishes.

PARAMETERS

Name	Type	Description
`seed`	`str`	Seed item id matching the physical seed in the Harvester's held slot.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"invalid_seed"`	rejection	The supplied item is not a plant seed.
`"no_seed"`	rejection	The requested seed is not available.
`"not_empty"`	rejection	The relevant cell, slot, or component is not empty.
`"base_sector"`	rejection	The base sector is the Harvester's depot pad and cannot be planted.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.deploy(kit: str) → ActionResult` SELF ONLY

Place a supported field-machine kit in the current empty cell, consuming one kit from Inventory. Installation takes **0.25 hours** and pauses the script. Call `deployables()` for the available kit ids. Seed Makers and other outpost buildings deploy through Inventory.

PARAMETERS

Name	Type	Description
`kit`	`str`	Fixed field-machine kit item id unlocked by current research.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_kit"`	rejection	The required deployable kit is not available.
`"not_empty"`	rejection	The relevant cell, slot, or component is not empty.
`"invalid_kit"`	rejection	The supplied deployable kit is invalid for this operation.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.deployables() → list[str]`

List of fixed field-machine kit ids unlocked by your current research. This is a capability list, not a live deployment check: `deploy(...)` still checks Inventory stock, the current cell, movement, heat, and whether the Harvester is busy.

RETURNS

`list[str]`

Fixed field-machine kit ids unlocked by research. It does not check Inventory stock, placement, heat, movement, or busy state. Empty until Grow Lamp research.

`.light() → ActionResult` SELF ONLY

Light the current plantable cell with the Harvester's work lamp, even if it is already lit or covered by a Grow Lamp. Each application resets the treatment to **24 hours** when the work finishes, replacing any remaining time. Lighting takes **0.25 hours** and pauses the script. Use a Grow Lamp later to cover four orthogonally adjacent cells (directly above, below, left, and right) continuously.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"not_plantable"`	rejection	The current cell cannot receive this light, water, or salt treatment.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.water()` → `ActionResult` SELF ONLY

Water the current plantable cell from the Harvester's onboard supply, even if it is already watered or covered by a Sprinkler. Each watering uses **1 t** and resets the treatment to **24 hours** when the work finishes, replacing any remaining time. Watering takes **0.25 hours** and pauses the script. Use `refill_water()` anywhere on the local grid to draw from the home outpost's tanks.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"insufficient_water"`	rejection	The Harvester has less than the 1 t of onboard water required for treatment.
`"not_plantable"`	rejection	The current cell cannot receive this light, water, or salt treatment.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.refill_water()` → `ActionResult` SELF ONLY

Refill the Harvester's onboard water supply from the home outpost's water tanks, from anywhere on the grid. Refilling takes **0.25 hours** and pauses the script.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"moving"`	transient	The vehicle or component is currently moving.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"tank_empty"`	rejection	The source tank is empty.
`"already_full"`	success	The target is already at full capacity.
`"busy"`	transient	The component is already performing another operation.

`.dispense_salt()` → `ActionResult` SELF ONLY

Treat the current plantable cell with one unit of `salt` from Inventory, even if it is already salted or covered by a Dispenser. Each application resets the treatment to **24 hours** when the work finishes, replacing any remaining time. Dispensing takes **0.25 hours** and pauses the script. Water Pumps produce salt as a byproduct.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_salt"`	rejection	No salt is available for the treatment.
`"not_plantable"`	rejection	The current cell cannot receive this light, water, or salt treatment.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.fertilize(item_id: str = "fertilizer") → ActionResult` SELF ONLY

Dose the current cell's growing plant with one Fertilizer Mk I, II, or III. Each unit boosts output for **8 hours**. Additional units of the same tier extend it; wait for the active dose to drain before switching tiers. Dosing occupies the Harvester for **0.25 hours**.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Fertilizer item id picking the tier; omit for Mk I.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_input"`	rejection	The supplied input value is invalid.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_plant"`	rejection	The current cell contains no plant.
`"already_mature"`	rejection	The plant is already mature, so a growth treatment cannot affect it.
`"tier_conflict"`	rejection	The cell already has a different active Fertilizer tier.
`"no_dose"`	rejection	The required treatment item is not available.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.accelerate()` → `ActionResult` SELF ONLY

Dose the current cell's growing plant with one `growth_accelerant` to double its growth rate for **8 hours**. Dosing occupies the Harvester for **0.25 hours**. The dose cannot affect a crop that is already mature.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_plant"`	rejection	The current cell contains no plant.
`"already_mature"`	rejection	The plant is already mature, so a growth treatment cannot affect it.
`"no_dose"`	rejection	The required treatment item is not available.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.amplify() → ActionResult` SELF ONLY

Apply one `yield_amplifier`, the capstone **field-wide** Forage-output boost (no cell needed). Applying it occupies the Harvester for **0.25 hours**. The dose drains over about one game-day, so re-call to sustain.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_dose"`	rejection	The required treatment item is not available.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.amplifier_remaining() → float`

Hours remaining on the field-wide Yield Amplifier effect. Each applied unit adds **24 hours**. Returns **0** when the field is not amplified.

RETURNS

`float`

Hours remaining on the field-wide
Yield Amplifier effect.

`.uproot()` → `ActionResult` SELF ONLY

Remove the plant in the current cell and place one matching seed in Inventory. Uprooting takes **0.5 hours** and pauses the script.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_plant"`	rejection	The current cell contains no plant.
`"inventory_full"`	rejection	Inventory has no capacity for the result.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.undeploy() → ActionResult` SELF ONLY

Remove the fixed field machine in the current cell and return its kit to Inventory. This is the only removal path for Grow Lamps, Sprinklers, Dispensers, and Crop Automators. Its stored items must be empty, the hardware refund must fit, and its attached script must not be running. Authored scripts are stopped and kept in Computer > Scripts > Unassigned. Unfinished machine work and machine-only result history are discarded; queued Crop Automator jobs have not consumed their inputs. Removal takes **0.25 hours** and pauses the Harvester script.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"nothing"`	rejection	There is no applicable target at the requested location.
`"not_empty"`	rejection	The relevant cell, slot, or component is not empty.
`"script_present"`	rejection	An attached script is currently running on the machine.
`"inventory_full"`	rejection	Inventory has no capacity for the result.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.harvest()` → `ActionResult` SELF ONLY

Harvest the ready crop in the current cell. The action takes **0.5 hours** and transfers nothing until it finishes. On completion it moves as much of `cell.forage` as Inventory can hold: a crop larger than the free room leaves its remainder banked and the plant standing, so drain Inventory and harvest again to collect the rest. Only a fully collected crop is removed and frees the cell for another physical seed.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"partial"`	partial	The operation moved part of the requested amount; the remainder is still available at the source.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"no_plant"`	rejection	The current cell contains no plant.
`"not_mature"`	rejection	The plant has not reached maturity.
`"no_forage"`	rejection	The ready crop has no whole forage unit to collect.
`"inventory_full"`	rejection	Inventory has no capacity for the result.
`"overheated"`	transient	The component is too hot for the requested operation or would exceed its heat limit by performing it.
`"moving"`	transient	The vehicle or component is currently moving.
`"busy"`	transient	The component is already performing another operation.

`.cell(sector: str) → Cell | None`

Read one grid sector as a `Cell` snapshot. Unscanned natural ground has `status` `"unknown"` until scanned; the depot and player-created plants or providers stay visible. The snapshot includes plant, growth, conditions, and remaining treatment hours. Returns `None` for an invalid sector.

PARAMETERS

Name	Type	Description
`sector`	`str`	Grid sector id (e.g. `"E13"`).

RETURNS

`Cell | None`

`None` for an invalid or off-grid id. Natural ground is `"unknown"` until scanned; the depot and player-created plants or providers remain visible.

`.cells() → list[Cell]`

Read every harvester-grid sector as a list of `Cell` snapshots. Unscanned natural ground reports status `"unknown"`; scan sectors before planning around occupancy. Use the list for field-wide planting, treatment-route scheduling, uprooting, undeploying, and harvesting policies.

RETURNS

`list[Cell]`

Every cell on the board. Unscanned natural ground has `status` `"unknown"`; scanned cells expose occupancy. Player-created plants and providers, growth, direct conditions, precise remaining treatment times, active fertilizer tier, and accumulated forage stay visible.

`.position() → str`

Current sector id as a string. Same position source as `get_position()`, exposed as a property-style read for grid scripts.

RETURNS

`str`

The sector the harvester currently occupies (e.g. `"E13"`). The Plants verbs all act on this cell; `self.move(...)` to act elsewhere.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Inventory

Inventory is the physical storeroom at **Nocturna Base**. Its page and read-only script methods are visible planet-wide, but it exchanges ordinary freight with machines or vehicles, in either direction, only at the home outpost; remote sites use local storage and vehicles. Purchases land here.

Deploy, undeploy, decommission, upgrade, and hardware mounting controls are explicit commissioning or service orders, not freight routes. Manual Biology uses Inventory at home and a selected same-outpost Warehouse elsewhere; Habitat reagents are staged locally.

Type	Storage

Access via: `get_component("inventory")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.stacks() → list[ItemStack]`

Lists every occupied item stack as an `ItemStack` with `.id`, `.count`, and exact `.properties`. Items with the same id but different properties appear separately. Storage Bins and Warehouses use the same format, so routing scripts can inspect all three in one way and pass exact properties to transfer methods.

RETURNS

`list[ItemStack]`

Occupied, property-distinct stacks. The same storage view Storage Bins and Warehouses expose.

`.get_slots() → list[Slot]`

Lists every current Inventory slot. Inventory starts with **36** slots, and Cargo Expansion can increase it to **60**. Stackable items hold **10** units per slot, or **20** after **Bigger Stacks**. Each `Slot` has a zero-based index from `0` through `get_size() - 1`, plus its item `id`, `name`, `value`, `count`, and `properties`. Properties are an exact identity `dict`, or `None` for ordinary items. Use them to distinguish variants with the same id and to select an exact item during transfers.

RETURNS

`list[Slot]`

Snapshots, each with the stack's exact `.properties` identity.

`.count(item_id: str) → int`

How many units of `item_id` are currently stored across all slots. Returns **0** if no slot holds that item. Storage Bins, Warehouses, and Lead Casks expose the same `count(item_id)` query, so one helper can search every store.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to count

RETURNS

`int`

Units of that item across all slots.

`.has_space(item_id: str | None = None, properties: ItemProperties | None = None) → bool`

Check whether Inventory has room. With no argument, `has_space()` is `True` when any slot is free. Passing an item id and properties checks that exact variant: a partial stack counts only when both match, while an empty slot accepts it. Pass `slot.properties` when checking an item with properties.

PARAMETERS

Name	Type	Description
`item_id`	`str` | `None`	Optional item id, makes the check stack-aware for that exact variant
`properties`	`ItemProperties` | `None`	Exact property `dict` from a `Slot` or `ItemStack`; `None` selects the propertyless variant

RETURNS

`bool`

Uses exact property identity
when an item id is supplied.

`.space_for(item_id: str, properties: ItemProperties | None = None) → int`

How many units of one exact item identity fit **right now**: remaining room in partial stacks with the same id and properties, plus empty slots times the current stack size. **Bigger Stacks** raises that size from **10** to **20** for stackable items. Use `inventory.space_for(slot.id, slot.properties)` for a property-bearing item. Omitting properties checks the ordinary propertyless variant.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to size remaining room for
`properties`	`ItemProperties` | `None`	Exact property `dict` from a `Slot` or `ItemStack`; `None` selects the propertyless variant

RETURNS

`int`

Units of this exact property variant that fit right now.

`.transfer_to(target: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research. Move up to whole-number `count` units of `item_id` from this storage endpoint to another Storage Bin, Warehouse, Large Warehouse, Lead Cask, or Inventory. Pass a storage building's display name or instance id, or `"inventory"`. Inventory participates only at **Nocturna Base**. The call waits for the physical store's feeder cycle to finish, and every participating storage building remains busy during that cycle. Exact item properties are preserved; optional `properties` and `property_match` select a variant. The calling script may run anywhere, but cargo never crosses outpost boundaries through this method.

PARAMETERS

Name	Type	Description
`target`	`str`	Display name or instance id of another storage endpoint at the same outpost
`item_id`	`str`	Item id to move
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"target_missing"`	rejection	The configured target no longer exists.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"same_storage"`	rejection	The source and destination are the same storage endpoint.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

`.drop(slot: int) → ItemResult`

Remove **1** unit from a specific slot by its zero-based index. Valid indexes run from `0` through `get_size() - 1`, including slots added by Cargo Expansion. Dropped items are deleted, not returned to the world; use `shop.sell(item_id)` if you want credits.

PARAMETERS

Name	Type	Description
`slot`	`int`	Whole-number inventory slot index

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ItemResult`	`.status`, `.message`	`.item_id`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation affected item `.item_id`.
`"empty"`	● success	The selected item source was already empty.
`"invalid_slot"`	● rejection	The requested inventory slot does not exist.

`.drop_all(item_id: str) → CountResult`

Remove every unit of `item_id` from inventory. For credits, use `shop.sell_all(item_id)`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to drop every stack of

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Command completed. Affected entries or units: `.count`.
`"no_op"`	● success	The command affected no entries or units.

`.compact() → TransferResult`

Do what the Inventory page's **Compact** button does: merge partial stacks of the same item and pack every stack toward the first slot, so the empty slots come last. Takes effect at once. Items with different properties stay in separate stacks.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`TransferResult`	`.status`, `.message`	`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Repositioned units: `.moved`, merging matching stacks and packing them from the first Inventory slot.
`"already_compact"`	● success	The Inventory is already compact: no matching stacks can merge and no empty slot comes before an occupied one.

`.get_size() → int`

Current number of Inventory slots. Inventory starts with **36**, and Cargo Expansion can increase it one slot at a time to **60**. Use this value instead of hardcoding a slot count.

RETURNS

`int`

Slots available now: 36 at the start, up to 60.

`.get_used() → int`

Number of occupied slots. `get_used() == get_size()` means inventory is full.

RETURNS

`int`

Large Warehouse

High-bay multi-material depot, 15 material-locked slots, 2,000 each (30,000 total). Broad enough to stage a full biological catalogue.

Type	Mining
Storage	15 slots × 2,000 units

HOW TO OBTAIN

1. Requires the **Large Warehouse** research (Biomass 30,000).
2. Buy from the Shop for 60,000 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.count(item_id: str) → int`

Units of `item_id` held across every slot. Returns **0** if no slot holds it. Inventory, Storage Bins, and Lead Casks expose the same query. `wh.count("iron_ore")`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to count

RETURNS

`int`

Units of that item across all slots.

`.total() → int`

Total units across all slots (every material combined). For one material use `count(item_id)`.

RETURNS

`int`

Total units across every slot.

`.capacity() → int`

Total capacity across all physical slots. A Warehouse returns **10,000** (**5** × **2,000**); a Large Warehouse returns **30,000** (**15** × **2,000**). Query this value instead of hardcoding a tier.

RETURNS

`int`

Total capacity across all slots.

`.fill_percent() → float`

Fraction full across the whole warehouse, `total() / capacity()`, in the range **0-1**.

RETURNS

`float`

A fraction, **0-1**.

`.is_empty() → bool`

`True` if every slot is empty.

RETURNS

`bool`

`.materials() → list[str]`

List of item ids currently stored (one entry per material with units in a slot). Iterate it: `for m in wh.materials():`.

RETURNS

`list[str]`

Ids of the items currently stored.

`.stacks() → list[ItemStack]`

Lists the item variants stored across all physical slots as `ItemStack` values. Items with the same id but different properties occupy separate slots. Call it again when you need current contents.

RETURNS

`list[ItemStack]`

Property-distinct stacks across all physical slots.

`.space_for(item_id: str, properties: ItemProperties | None = None) → int`

How many more units of one exact item variant fit **right now**, using room in matching-identity slots plus every empty slot. Omit `properties` for ordinary propertyless items, or pass the full `.properties` `dict` returned by `stacks()`. This never counts room belonging to a different property variant.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to size remaining room for
`properties`	`ItemProperties` | `None`	Full property `dict` for the variant, or `None` for propertyless items

RETURNS

`int`

Units of that exact item variant that fit right now.

`.has_space(item_id: str, amount: int, properties: ItemProperties | None = None) → bool`

`True` if at least whole-number `amount` more units of that exact item variant fit. Omit `properties` for propertyless items or pass the full property `dict`. Use before a transfer to avoid partial moves.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id
`amount`	`int`	Whole-number units required
`properties`	`ItemProperties` | `None`	Full property `dict` for the variant, or `None` for propertyless items

RETURNS

`bool`

`.slots() → list[WarehouseSlot]`

Every physical slot as a `WarehouseSlot` record with `.index`, `.item`, `.count`, `.capacity`, and `.properties`. Property-distinct variants use distinct slots.

RETURNS

`list[WarehouseSlot]`

One record per slot.

`.compact() → TransferResult`

Requires **Auto Feeders** research. Consolidate every exact item variant into the fewest Warehouse slots that can hold it. The smallest redundant stacks move into larger compatible stacks, minimizing physical handling; equal item ids with different properties always remain separate. The call waits for time proportional to the units repositioned and locks this Warehouse as a material endpoint for the cycle. Port transfers and manual Biology actions using this Warehouse wait until that cycle finishes.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`TransferResult`	`.status`, `.message`	`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Compacted units: `.moved` into fewer Warehouse slots.
`"already_compact"`	success	Every exact item variant already occupies the fewest possible Warehouse slots.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.

`.transfer_to(target: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research. Move up to whole-number `count` units of `item_id` from this storage endpoint to another Storage Bin, Warehouse, Large Warehouse, Lead Cask, or Inventory. Pass a storage building's display name or instance id, or `"inventory"`. Inventory participates only at **Nocturna Base**. The call waits for the physical store's feeder cycle to finish, and every participating storage building remains busy during that cycle. Exact item properties are preserved; optional `properties` and `property_match` select a variant. The calling script may run anywhere, but cargo never crosses outpost boundaries through this method.

PARAMETERS

Name	Type	Description
`target`	`str`	Display name or instance id of another storage endpoint at the same outpost
`item_id`	`str`	Item id to move
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"target_missing"`	rejection	The configured target no longer exists.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"same_storage"`	rejection	The source and destination are the same storage endpoint.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

Lead Cask

Shielded stationary storage for radioactive cargo. Drones drop Raw Uranium into it, the Fuel Assembler draws from it and returns finished Fuel Rods, and the Reactor pulls its fuel from it.

Type	Mining
Storage	Units: 100

HOW TO OBTAIN

1. The recipe unlocks when you complete **Helios, Plate Order**.
2. Requires the **Shielded Logistics** research (Oxygen 3,000).
3. Fabricate **Lead Cask** on a **Fabricator**: 3× Lead Plate and 1× Machine Frame.
4. Deploy it from your Inventory.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.count(item_id: str) → int`

Units of `item_id` currently casked. Returns **0** when the cask is empty or latched to the other radioactive item. Inventory, Storage Bins, and Warehouses expose the same `count(item_id)` query.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Radioactive item id to count

RETURNS

`int`

Units of that radioactive item currently casked.

`.fill_percent() → float`

Fill fraction **0-1**, watch your strategic reserve.

RETURNS

`float`

A fraction, **0-1**.

`.capacity() → int`

Maximum hot units (**100**).

RETURNS

`int`

Maximum capacity (**100**).

`.material() → str`

What the cask is latched to: `"raw_uranium"`, `"fuel_rod"`, or `""` when it is empty. One material per cask, like every stock bin.

RETURNS

`str`

Empty when unassigned. Lead Casks accept only radioactive items. Ordinary storage and inventory refuse them; designated shielded receivers accept them.

POSSIBLE VALUES

`""`, `"raw_uranium"`, `"fuel_rod"`

`.transfer_to(target: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research. Move up to whole-number `count` units of `item_id` from this storage endpoint to another Storage Bin, Warehouse, Large Warehouse, Lead Cask, or Inventory. Pass a storage building's display name or instance id, or `"inventory"`. Inventory participates only at **Nocturna Base**. The call waits for the physical store's feeder cycle to finish, and every participating storage building remains busy during that cycle. Exact item properties are preserved; optional `properties` and `property_match` select a variant. The calling script may run anywhere, but cargo never crosses outpost boundaries through this method.

PARAMETERS

Name	Type	Description
`target`	`str`	Display name or instance id of another storage endpoint at the same outpost
`item_id`	`str`	Item id to move
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"target_missing"`	rejection	The configured target no longer exists.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"same_storage"`	rejection	The source and destination are the same storage endpoint.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

Mining Drill

Mk I static drill for deposits up to hardness 1 (Iron, Silicon): 25 t/h at standard purity and 10 W while extracting.

Type	Mining
Built on	Mineral sites
Power in	-10 W (draws from grid)
Stockpile	2,000 units (mixed)

HOW TO OBTAIN

1. The recipe unlocks when you complete **Helios, Cargo Pod Run**.
2. Requires the **Basic Drone Operations** research (Terraform Index 180,000).
3. Fabricate **Mining Drill Kit** on a **Fabricator**: 3× Machine Frame, 1× Control Unit, 1× Circuit Panel, and 2 t Water.
4. Build it on a mineral site with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.output: PickupOutputSlot`

`PickupOutputSlot` exposing the Drill's stockpile through `count()`, `capacity()`, and `stacks()`. A physically present Rover or Pioneer pulls through its own `input`; a drone flies with `go_to_drill()` and loads with `cargo.load()`. The Drill has no direct item-routing methods.

RETURNS

`PickupOutputSlot`

Exposes stockpile reads. A physically present Rover or Pioneer pulls through its `input`; a drone uses `go_to_drill()` and `cargo.load()`.

METHODS

`.site() → MiningSite | None`

The `MiningSite` this drill sits on. Read `.item_id` for the mineral it extracts, and `.purity` and `.hardness` for the deposit under it. `None` when the drill stands on no mineral site.

RETURNS

`MiningSite | None`

A snapshot of the mineral site this drill sits on, or `None` when it stands on none.

`.drill_rate() → float`

Mineral extraction rate in t/h right now: the full rate while drilling, and **0** whenever the drill is powered off, has no deposit under it, cannot cut the deposit's hardness, or its stockpile is full. Adjusted for site purity and drill tier.

RETURNS

`float`

t/h being extracted right now, or **0** when not drilling. Accounts for site purity and drill tier.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Smelter

Refines raw ore into metal stock, one unit at a time, following a recipe you choose. A script sets the recipe, feeds ore in from a bin, and drains the finished metal out to another.

Type	Mining
Power in	Variable (draws from grid)
Produces	Depends on the selected recipe
Input buffer	Units: 50
Output buffer	Units: 50
Recipes	7 available
Tiers	Mk II and Mk III

HOW TO OBTAIN

1. Requires the **Ore Refinement** research (Oxygen 5).
2. Buy from the Shop for 650 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Loads ore into the Smelter. At home it can use Inventory; remote Smelters must connect a local Storage Bin or Warehouse. Pull material with `self.input.take(item_id, count)`. Failed transfers leave the source cargo unchanged. Requires **Auto Feeders** research. See `InputSlot`.

RETURNS

`InputSlot`

`.output: OutputSlot`

Sends refined material out of the Smelter. At home it can use Inventory; remote Smelters must connect a local Storage Bin or Warehouse. Send material with `self.output.send(item_id, count)`. Failed transfers leave the output unchanged. Requires **Auto Feeders** research. See `OutputSlot`.

RETURNS

`OutputSlot`

METHODS

`.tier() → int`

Installed machinery tier: `1` for Mk I, `2` for Mk II, or `3` for Mk III where supported.

RETURNS

`int`

`.list_recipes() → list[Recipe]`

Every recipe this smelter has been given a blueprint for. Returns `Recipe` objects with `.tier`, `.id`, `.name`, `.inputs`, `.output_item`, `.output_count`, `.duration_game_hours`, and `.power_draw`. Locked recipes (no blueprint yet) do not appear, the list reflects what the player can actually run today. Day-1 starts with `"smelt_iron_ingot"` only; more arrive as blueprints unlock.

RETURNS

`list[Recipe]`

The recipes this Smelter has unlocked.

`.find_recipe(recipe_id: str) → Recipe | None`

Find one unlocked recipe by id without looping through `list_recipes()`. Returns its `Recipe` object, or `None` when the id is unknown, locked, or belongs to another machine.

PARAMETERS

Name	Type	Description
`recipe_id`	`str`	Smelter recipe id

RETURNS

`Recipe | None`

`None` if this Smelter cannot currently run that id.

`.set_recipe(recipe_or_id: str | Recipe | IdRecord) → ActionResult` SELF ONLY

Select which recipe the smelter should run. Call `self.set_recipe("smelt_iron_ingot")` or pass a `Recipe` from `list_recipes()`.

PARAMETERS

Name	Type	Description
`recipe_or_id`	`str` | `Recipe` | `IdRecord`	Recipe id, a `Recipe` object, or a dictionary or class instance with a string `id` field. The recipe must belong to this machine and be unlocked.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_recipe"`	rejection	The supplied recipe is unknown or does not apply to this machine.
`"offline"`	transient	The component is offline.
`"recipe_locked"`	rejection	The selected recipe is locked.
`"busy"`	transient	The component is already performing another operation.
`"material_mismatch"`	rejection	The existing material does not match the requested material.

`.clear_recipe() → ActionResult` SELF ONLY

Unset the current recipe and leave the smelter idle. Empty latched buffers clear back to no material.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"busy"`	transient	The component is already performing another operation.
`"material_present"`	rejection	Existing material prevents the requested configuration change.

`.get_recipe() → str`

Current recipe id as a string, or the empty string if no recipe is set. Use after `set_recipe()` to confirm, or to gate other logic (`if self.get_recipe() == "": ...`).

RETURNS
`str`
Empty when no recipe is set.

POSSIBLE VALUES
`""`, `"smelt_iron_ingot"`, `"smelt_glass"`, `"smelt_titanium_ingot"`, `"smelt_cobalt_ingot"`, `"smelt_rare_earth_core"`, `"smelt_neutronium_bar"`, `"smelt_lead_ingot"`

`.get_recipe_inputs() → dict[str, int]`

Input requirements for the current recipe as a `dict` `{item_id: count_per_craft}`. Returns an empty `dict` if no recipe is set.

RETURNS

`dict[str, int]`

`{item_id: count}` consumed per craft, or empty if no recipe is set.

`.is_running() → bool`

Whether the smelter is actively processing. Can be `False` while an unfinished unit is paused, for example when its output is full. Paused progress is retained; use `get_progress()` to check for unfinished work before changing recipes.

RETURNS

`bool`

`.get_progress() → float`

Progress toward the next completed unit (**0-1**). Resets to **0** when a unit completes and a new one starts. Useful for progress bars and scripts that want to detect completions by watching the value drop.

RETURNS

`float`

Progress through the current craft (**0-1**).

`.get_input_count() → int`

Units currently in the input buffer, waiting to be smelted. Check before `self.input.take(...)` to avoid overfilling, or to decide whether to pull more.

RETURNS

`int`

Units in the input buffer.

`.get_output_count() → int`

Units currently in the output buffer, waiting to be drained. Check before `self.output.send(...)`, if high, send this output onward first; if low, let processing catch up.

RETURNS

`int`

Units in the output buffer.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Storage Bin

A passive base container that holds one material at a time. The first deposit sets what it stores, and the lock clears only once it drains empty. Other scripts can read and move its contents.

Type	Mining
Storage	Units: 500

HOW TO OBTAIN

1. Requires the **Storage Bins** research (Temperature 5).
2. Buy from the Shop for 120 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.count(item_id: str) → int`

Units of `item_id` currently stored. Returns **0** when the bin is empty or latched to another item. Inventory, Warehouses, and Lead Casks expose the same `count(item_id)` query.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to count

RETURNS

`int`

Units of that item currently stored.

`.get_capacity() → int`

Maximum units the bin holds, **500** by default. Queryable rather than hardcoded so a retune doesn't break scripts. Use `fill_percent()` when you need the current fill ratio.

RETURNS

`int`

Maximum units.

`.get_material() → str`

Currently latched material id, or the empty string if the bin is empty (and therefore accepts any material on the next deposit). Use to check a bin's material before routing transfers: `if bin.get_material() in ("", "iron_ore"): # safe to deposit iron`.

RETURNS

`str`

The stored item id, or empty if the bin is empty.

`.stacks() → list[ItemStack]`

Lists the item variants stored in this bin as `ItemStack` values. Items with the same id but different properties remain separate. Call it again when you need current contents.

RETURNS

`list[ItemStack]`

Property-distinct stacks currently stored.

`.is_empty() → bool`

`True` if the bin holds nothing. An empty bin has no material lock, any material can take the slot on the next deposit. Different from `has_space(0)` which is always `True`.

RETURNS

`bool`

`.has_space(amount: int) → bool`

`True` if the bin has room for whole-number `amount` more units. Use before a transfer to avoid partial moves.

PARAMETERS

Name	Type	Description
`amount`	`int`	Whole-number units of free capacity required

RETURNS

`bool`

`.space() → int`

Free units of capacity remaining. Sizes a transfer in one call: `n = bin.space()`, then move up to `n`.

RETURNS

`int`

Free units remaining.

`.fill_percent() → float`

Fraction full in the range **0-1**. Common threshold for rebalance scripts: `if bin.fill_percent() < 0.2: # route more here`.

RETURNS

`float`

A fraction, **0-1**.

`.transfer_from_inventory(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research and a Storage Bin at the home outpost. Move up to whole-number `count` units of `item_id` from Inventory into the bin, preserving exact properties. The call waits for the feeder cycle to finish, and the bin cannot start another transfer during that cycle. A property `dict` matches as a subset (`"subset"`) by default. Pass `"exact"` as `property_match` for a full identity, including `None` for propertyless items.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to pull from inventory
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"busy"`	● transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"inventory_not_local"`	● rejection	This endpoint cannot access the home freight Inventory from its current outpost.
`"source_empty"`	● rejection	The source has no matching units available.
`"source_changed"`	● transient	The source changed between transfer planning and commit.

Status	Kind	Meaning
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

`.transfer_to_inventory(count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research and a Storage Bin at the home outpost. Move up to whole-number `count` units back to Inventory, preserving exact properties. The call waits for the feeder cycle to finish, and the bin cannot start another transfer during that cycle. A property `dict` matches as a subset (`"subset"`) by default. Pass `"exact"` as `property_match` for a full identity, including `None` for propertyless items. If the bin drains completely, its item-id latch clears.

PARAMETERS

Name	Type	Description
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"busy"`	● transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"source_empty"`	● rejection	The source has no matching units available.
`"source_changed"`	● transient	The source changed between transfer planning and commit.
`"inventory_not_local"`	● rejection	This endpoint cannot access the home freight Inventory from its current outpost.
`"target_full"`	● rejection	The destination has no capacity for matching units.

Status	Kind	Meaning
`"target_changed"`	● transient	The destination changed between transfer planning and commit.

`.transfer_to(target: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research. Move up to whole-number `count` units of `item_id` from this storage endpoint to another Storage Bin, Warehouse, Large Warehouse, Lead Cask, or Inventory. Pass a storage building's display name or instance id, or `"inventory"`. Inventory participates only at **Nocturna Base**. The call waits for the physical store's feeder cycle to finish, and every participating storage building remains busy during that cycle. Exact item properties are preserved; optional `properties` and `property_match` select a variant. The calling script may run anywhere, but cargo never crosses outpost boundaries through this method.

PARAMETERS

Name	Type	Description
`target`	`str`	Display name or instance id of another storage endpoint at the same outpost
`item_id`	`str`	Item id to move
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"target_missing"`	rejection	The configured target no longer exists.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"same_storage"`	rejection	The source and destination are the same storage endpoint.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

Warehouse

Multi-material bulk depot, 5 material-locked slots, 2,000 each (10,000 total). Drone-scale haulage absorption.

Type	Mining
Storage	5 slots × 2,000 units

HOW TO OBTAIN

1. Requires the **Warehouse** research (Oxygen 150).
2. Buy from the Shop for 3,000 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

METHODS

`.count(item_id: str) → int`

Units of `item_id` held across every slot. Returns **0** if no slot holds it. Inventory, Storage Bins, and Lead Casks expose the same query. `wh.count("iron_ore")`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to count

RETURNS

`int`

Units of that item across all slots.

`.total() → int`

Total units across all slots (every material combined). For one material use `count(item_id)`.

RETURNS

`int`

Total units across every slot.

`.capacity() → int`

Total capacity across all physical slots. A Warehouse returns **10,000** (**5** × **2,000**); a Large Warehouse returns **30,000** (**15** × **2,000**). Query this value instead of hardcoding a tier.

RETURNS

`int`

Total capacity across all slots.

`.fill_percent() → float`

Fraction full across the whole warehouse, `total() / capacity()`, in the range **0-1**.

RETURNS

`float`

A fraction, **0-1**.

`.is_empty() → bool`

`True` if every slot is empty.

RETURNS

`bool`

`.materials() → list[str]`

List of item ids currently stored (one entry per material with units in a slot). Iterate it: `for m in wh.materials():`.

RETURNS

`list[str]`

Ids of the items currently stored.

`.stacks() → list[ItemStack]`

Lists the item variants stored across all physical slots as `ItemStack` values. Items with the same id but different properties occupy separate slots. Call it again when you need current contents.

RETURNS

`list[ItemStack]`

Property-distinct stacks across all physical slots.

`.space_for(item_id: str, properties: ItemProperties | None = None) → int`

How many more units of one exact item variant fit **right now**, using room in matching-identity slots plus every empty slot. Omit `properties` for ordinary propertyless items, or pass the full `.properties` `dict` returned by `stacks()`. This never counts room belonging to a different property variant.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to size remaining room for
`properties`	`ItemProperties` | `None`	Full property `dict` for the variant, or `None` for propertyless items

RETURNS

`int`

Units of that exact item variant that fit right now.

`.has_space(item_id: str, amount: int, properties: ItemProperties | None = None) → bool`

`True` if at least whole-number `amount` more units of that exact item variant fit. Omit `properties` for propertyless items or pass the full property `dict`. Use before a transfer to avoid partial moves.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id
`amount`	`int`	Whole-number units required
`properties`	`ItemProperties` | `None`	Full property `dict` for the variant, or `None` for propertyless items

RETURNS

`bool`

`.slots() → list[WarehouseSlot]`

Every physical slot as a `WarehouseSlot` record with `.index`, `.item`, `.count`, `.capacity`, and `.properties`. Property-distinct variants use distinct slots.

RETURNS

`list[WarehouseSlot]`

One record per slot.

`.compact() → TransferResult`

Requires **Auto Feeders** research. Consolidate every exact item variant into the fewest Warehouse slots that can hold it. The smallest redundant stacks move into larger compatible stacks, minimizing physical handling; equal item ids with different properties always remain separate. The call waits for time proportional to the units repositioned and locks this Warehouse as a material endpoint for the cycle. Port transfers and manual Biology actions using this Warehouse wait until that cycle finishes.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`TransferResult`	`.status`, `.message`	`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Compacted units: `.moved` into fewer Warehouse slots.
`"already_compact"`	success	Every exact item variant already occupies the fewest possible Warehouse slots.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.

`.transfer_to(target: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Requires **Auto Feeders** research. Move up to whole-number `count` units of `item_id` from this storage endpoint to another Storage Bin, Warehouse, Large Warehouse, Lead Cask, or Inventory. Pass a storage building's display name or instance id, or `"inventory"`. Inventory participates only at **Nocturna Base**. The call waits for the physical store's feeder cycle to finish, and every participating storage building remains busy during that cycle. Exact item properties are preserved; optional `properties` and `property_match` select a variant. The calling script may run anywhere, but cargo never crosses outpost boundaries through this method.

PARAMETERS

Name	Type	Description
`target`	`str`	Display name or instance id of another storage endpoint at the same outpost
`item_id`	`str`	Item id to move
`count`	`int`	Whole-number max units to transfer
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"target_missing"`	rejection	The configured target no longer exists.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"same_storage"`	rejection	The source and destination are the same storage endpoint.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

Waste Processor

Permanently destroys one script-selected waste stream: items, liquids, or gases. Use the item `input` for unwanted stock, `liquid_in` for surplus water or other liquids, and `gas_in` for gases. It destroys only while the script that armed it is still running. It has no output and recovers no value.

Type	Mining
Power in	Variable (draws from grid)
Stockpile	200 units (mixed)

HOW TO OBTAIN

1. The recipe unlocks with the **Waste Processing** research (Oxygen 1,400).
2. Fabricate **Waste Processor Kit** on a **Fabricator**: 2× Machine Frame, 4× Iron Ingot, and 1× Circuit Panel.
3. Deploy it from your Inventory.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Stage unwanted items for permanent destruction. Connect a storage bin or machine buffer with `self.input.connect(name)`, then pull items with `self.input.take(item_id, count)`. Item transfers are explicit, so they may be staged while another mode is selected; only enabled `"items"` mode destroys them. See `InputSlot`.

RETURNS

`InputSlot`

Route any item here from a storage bin or machine byproduct bin. The input accepts **any** item in every mode; only enabled `"items"` mode destroys it. Disable destruction before recovering a mistaken load with `eject(...)`.

`.liquid_in: FluidPort`

Generic **120 t** liquid input buffer. Connect a liquid source, select `"liquid"`, and enable the processor. A non-empty buffer accepts only its currently latched liquid until it drains. See `FluidPort`.

RETURNS

`FluidPort`

A **120 t** input for any liquid. It accepts flow only while the processor is enabled in `"liquid"` mode. A non-empty buffer remains latched to one exact liquid until drained.

`.gas_in: FluidPort`

Generic **120 t** gas input buffer. Connect a gas source, select `"gas"`, and enable the processor. A non-empty buffer accepts only its currently latched gas until it drains. See `FluidPort`.

RETURNS

`FluidPort`

A **120 t** input for any gas. It accepts flow only while the processor is enabled in `"gas"` mode. A non-empty buffer remains latched to one exact gas until drained.

METHODS

`.set_enabled(enabled: bool) → ActionResult` SELF ONLY

Arm or pause destruction in the selected mode. The processor destroys only while the script that armed it keeps running: when that script stops, completes, or errors, this setpoint returns to off and `status()` reads `"disabled"`. A Smelter keeps working from its committed recipe, this machine does not. Staged items and fluids remain intact.

PARAMETERS

Name	Type	Description
`enabled`	`bool`	`True` arms destruction in the selected mode. `False` stops destruction and rejects new fluid while preserving every staged buffer.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

Read whether the processor is currently armed. The arm belongs to a running script, so it reads `False` again once that script ends.

RETURNS

`bool`

`.set_mode(mode: str) → ActionResult` SELF ONLY

Select exactly one destruction stream: `"items"`, `"liquid"`, or `"gas"`. Changing modes pauses the other buffers without deleting them. The selected fluid port accepts flow only while enabled.

PARAMETERS

Name	Type	Description
`mode`	`str`	Selected waste stream: `"items"`, `"liquid"`, or `"gas"`. Only that stream is destroyed. The selected fluid port accepts flow only while enabled; explicit item transfers may stage items in any mode. Other buffers pause without losing their contents.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

RAISES

Exception	Condition
`ValueError`	`mode` must be `"items"`, `"liquid"`, or `"gas"`.

`.mode() → str`

Read the selected waste stream.

RETURNS

`str`

POSSIBLE VALUES

`"items"`, `"liquid"`, `"gas"`

`.status() → str`

Read the exact live state: `"disabled"`, `"no_power"`, `"idle"`, or `"processing"`. `"disabled"` means no running script has armed the processor, so staged waste waits untouched.

RETURNS

`str`

POSSIBLE VALUES

`"disabled"`, `"no_power"`, `"idle"`, `"processing"`

`.throughput() → float`

Read the selected mode's current destruction rate. Item mode reports units/h; liquid and gas modes report t/h.

RETURNS

`float`

Current destruction rate for the selected mode. Item mode reports units/h; liquid and gas modes report t/h. **0** when the selected stream is not processing.

`.item_throughput() → float`

Read the current item destruction rate in units/h. At 100% outpost efficiency, the maximum is **60 units/h**.

RETURNS

`float`

Current whole-item destruction rate, in units/h. At 100% outpost efficiency, the maximum is **60 units/h**. **0** unless item mode is powered, enabled, and has staged items.

`.liquid_throughput() → float`

Read the last liquid destruction rate in t/h. At 100% outpost efficiency, the maximum is **120 t/h**.

RETURNS

`float`

Liquid destroyed during the last processing tick, in t/h. At 100% outpost efficiency, the maximum is **120 t/h**.

`.gas_throughput() → float`

Read the last gas destruction rate in t/h. At 100% outpost efficiency, the maximum is **120 t/h**.

RETURNS

`float`

Gas destroyed during the last processing tick, in t/h. At 100% outpost efficiency, the maximum is **120 t/h**.

`.is_running() → bool`

Read whether the selected mode is currently processing staged material.

RETURNS

`bool`

`True` while the selected mode is processing staged material. Pure destruction has no output port and recovers no value.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Constructor Module MODULE

Pioneer-exclusive module for construction and deconstruction blueprints created in Plan Mode or by scripts: gas/liquid pipes, utility bridges, power lines, outposts, pumps, caps, and mining drills. Fits a `universal` slot. Load the required kits, segments, or bridge items into the Pioneer's cargo for build jobs, drive within interaction range of the blueprint position, then call `execute(blueprint_id)`. Deconstruction reclaims the dismantled kit or segment into the Pioneer's cargo. Internal outpost machines, including drone facilities, deploy directly from Inventory and are not Constructor jobs.

Access via: `self.constructor`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.execute(blueprint_id: str) → ActionResult` SELF ONLY

Pick up one Construction job from the shared planning queue and build or deconstruct it. Plan Mode and `get_component("construction_blueprint")` create equivalent jobs. Drive the Pioneer within interaction range of `blueprint.position` first, and use `get_component("construction_blueprint").pending_constructions()` to see what's ready. A Pioneer performs only one field action at a time. Stop, power loss, leaving the site, rescue, or removing the Constructor Module pauses paid work without losing its progress or materials. Resume the same id from `get_component("construction_blueprint").paused_constructions()`.

PARAMETERS

Name	Type	Description
`blueprint_id`	`str`	Blueprint id from `get_component("construction_blueprint").pending_constructions()`, `.active_constructions()`, or `.paused_constructions()`.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_mounted"`	● rejection	The required module is not mounted.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"already_active"`	● transient	The requested operation is already active.
`"busy"`	● transient	The component is already performing another operation.
`"wrong_position"`	● rejection	The worker is not at the operation's required position.
`"insufficient_materials"`	● rejection	The Pioneer does not have enough of the required construction material in cargo.
`"paused_no_power"`	● transient	The operation is paused because power is unavailable.
`"cargo_present"`	● rejection	Existing cargo prevents the requested configuration change.
`"blocked"`	● rejection	The operation is blocked by the current world state.
`"paused"`	● transient	The operation is paused.

Status	Kind	Meaning
`"canceled"`	rejection	The operation was canceled.
`"not_ready"`	rejection	The operation's prerequisites are not currently satisfied.

Drill Module MODULE

Extracts minerals through `self.drill`. Each drill cuts its hardness limit and everything below it: the basic drill reaches hardness **1** (Iron, Silicon) at **1.0×** mining time using **10 W**; Industrial reaches **3**, adding Titanium, Cobalt, Lead and Rare Earth, at **0.75×** using **20 W**; Heavy reaches **4**, adding Neutronium, at **0.6×** using **30 W**. These are per-unit time multipliers, so lower means faster mining. There is no hardness-2 drill: Titanium and Cobalt are cut by the Industrial. Without a mounted Drill Module, the vehicle cannot mine.

Access via: `self.drill`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.mine()` → `ActionResult` SELF ONLY

Extract **1** unit of the current site's mineral into vehicle cargo. Mining takes `mineral_base_minutes × drill.speed_multiplier() / site_purity` game-time, and the script pauses until it finishes. Base minutes: Iron and Silicon **15**, Lead **18**, Titanium and Cobalt **20**, Rare Earth **25**, Neutronium **30**; `site_purity` is **1** `standard`, **2** `rich`, **3** `pure`. The drill draws its power for the whole dig, so one unit costs its watts × the mining time: the basic drill (**10 W**) mining for **15** minutes uses **2.5 Wh**.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.
`"not_at_site"`	rejection	The vehicle is not positioned at a compatible site.
`"not_surveyed"`	rejection	The mineral site has not been surveyed.
`"too_hard"`	rejection	The target exceeds the mounted tool's hardness limit.
`"no_cargo_space"`	rejection	Cargo has no compatible space for the result.
`"no_power"`	transient	The component has no available power.
`"not_enough_power"`	rejection	The available energy is below the operation's requirement.
`"busy"`	transient	The component is already performing another operation.

`.hardness_limit() → int`

Maximum mineral hardness this drill can extract.

RETURNS

`int`

1 basic, **3** Industrial, **4** Heavy. A stale captured module reference raises `ReferenceError`.

RAISES

Exception	Condition
`ReferenceError`	This `DrillModule` reference is stale because its module is no longer mounted. Read `self.drill` again after mounting a drill.

`.speed_multiplier() → float`

Per-unit time multiplier (lower = faster).

RETURNS

`float`

1.0 basic, **0.75** Industrial, **0.6** Heavy. A stale captured module reference raises `ReferenceError`.

RAISES

Exception	Condition
`ReferenceError`	This `DrillModule` reference is stale because its module is no longer mounted. Read `self.drill` again after mounting a drill.

Drone

Small aerial cargo drone, 1 thruster + 2 modules.

Type	Mining

HOW TO OBTAIN

1. The recipe unlocks when you complete **Helios, Rotor Run**.
2. Requires the **Basic Drone Operations** research (Terraform Index 180,000).
3. Fabricate **Drone (Small)** on a **Fabricator**: 1× Rare Earth Core, 1× Titanium Ingot, 1× Control Unit, and 2 t Water.
4. Deploy it from your Inventory.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

`.battery: DroneBattery`

Reads power on electric drones. Use `self.battery.level()` for current charge in **Wh**, `capacity()` for total Battery Pack capacity, and `percent()` for the **0-1** charge fraction. These calls raise `ReferenceError` on a heli drone. In mixed fleets, check `DroneRef.engine` from `fleet.drones()` first.

RETURNS

`DroneBattery`

On **electric drones** only. Its methods raise `ReferenceError` on a drone without an electric powertrain; inspect `fleet.drones()` and branch on `DroneRef.engine` before calling across a mixed fleet.

`.oil_tank: DroneOilTank`

Reads fuel on heli drones. Use `self.oil_tank.level()` for current oil in **tons**, plus `capacity()` and `percent()`. These calls raise `ReferenceError` on an electric drone. In mixed fleets, check `DroneRef.engine` from `fleet.drones()` first.

RETURNS

`DroneOilTank`

On **heli drones** only. Its methods raise `ReferenceError` on a drone without a heli powertrain; inspect `fleet.drones()` and branch on `DroneRef.engine` before calling across a mixed fleet.

`.cargo: DroneCargo`

Each Cargo Pod holds one item type; the Bio Extractor chamber holds one life-form type. Containers unlatch when empty. Loads top up matching containers before using empty ones, with life-form chambers before pods within each group. Plan with `self.cargo.count()`, `contents()`, `capacity()`, and `space_for(item_id)`. `load()` and `unload()` work while docked at a Drone Depot; `load()` also works at a field Mining Drill or Lead Cask. Item properties are preserved. See `DroneCargo`.

RETURNS

`DroneCargo`

Mounted Cargo Pods plus the Bio Extractor's typed 25 t chamber. Use `space_for(item_id)` because biological room is item-specific.

`.bio_scanner: PortableBioScanner`

Portable Bio Scanner

RETURNS

`PortableBioScanner`

`scan()` requires the module, a completed route, and working electronics and exposes the completed biological reading through `BioScanResult.scan`.

`.bio_extractor: PortableBioExtractor`

Portable Bio Extractor

RETURNS

`PortableBioExtractor`

`extract()` requires a hovering, non-scrambled drone and uses its integrated 25 t one-life-form chamber plus any Cargo Pods.

METHODS

`.go_to_station(name: str) → ActionResult` SELF ONLY

Queue a route to the named Drone Depot or Drone Service Station; the call returns at once, without waiting for docking. Once a powered route is accepted, `status()` reports `"traveling"`; position and docking advance with the simulation. Stop, completion, or error cancels the flight and clears the route. Compare `current_station()` with the destination's stable id to confirm arrival. Moving between drone buildings inside the same outpost is a local transfer and costs no flight fuel. Drone Service Stations accept parked arrivals even while unpowered. A full Drone Depot keeps the drone undocked in `"waiting_bay"` until a physical bay opens.

PARAMETERS

Name	Type	Description
`name`	`str`	Display name or id of the destination Drone Depot or Drone Service Station.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"station_not_found"`	rejection	The requested station does not exist.
`"out_of_range"`	rejection	The requested target lies outside the operation's range.
`"busy"`	transient	The component is already performing another operation.
`"scrambled"`	transient	Radiation scrambling prevents the operation.

`.undock()` → `ActionResult` SELF ONLY

Release the station berth without flying anywhere. The drone keeps its exact world position, cargo, modules, fuel, and exposure, clears any dormant route, resets throttle to **0**, and becomes `idle`. An active rescue, or an active or queued Drone Service Station charge/refuel job, retains control until that station-owned work ends. Use `go_to_station(...)` when the drone should claim a berth again. Self-only.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_docked"`	rejection	The requested vehicle is not docked at this station.
`"busy"`	transient	The component is already performing another operation.

`.go_to_drill(name: str) → ActionResult` SELF ONLY

Fly to a named field Mining Drill for ore pickup. Hauling needs no field module. Once a powered route is accepted, `status()` reports `"traveling"` immediately, then the drone flies in a straight line and hovers on arrival. Compare `current_drill()` with the destination's stable id to confirm that cargo loading is available. Stopping or completing the script, hitting an error, or calling `go_to_station()` cancels this route.

PARAMETERS

Name	Type	Description
`name`	`str`	Display name or id of a deployed Mining Drill (any tier).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"drill_not_found"`	rejection	No completed mining drill matching the request is available within the planet's bounds.
`"out_of_range"`	rejection	The requested target lies outside the operation's range.
`"busy"`	transient	The component is already performing another operation.
`"scrambled"`	transient	Radiation scrambling prevents the operation.

`.current_station() → str`

Stable station id where the drone is physically docked. Returns an empty string while flying to coordinates, traveling between stations, or waiting outside a full Drone Depot. Use equality with the destination id as the authoritative station-arrival check, even when `go_to_station()` was called with a display name.

RETURNS

`str`

The station id, or empty if in transit.

`.current_drill() → str`

Stable Mining Drill id where the drone can currently load cargo, or an empty string when no Drill is available. The drone must be within the Drill's loading area with no active route; merely passing over the Drill or holding a zero-throttle route does not count. Compare this value with the destination id as the authoritative Drill-arrival check, even when `go_to_drill()` was called with a display name.

RETURNS

`str`

The Mining Drill id available for cargo loading, or empty otherwise.

`.position() → Position`

World coordinates `(.x, .y)`, lerped each tick by DroneSystem during transit, snapped to station coords on dock.

RETURNS

`Position`

Current world coordinates.

`.get_distance_to(x: float, y: float) → float`

Straight-line distance in meters from the drone's current position to the given world coordinate. Use it to compare possible destinations, check remaining route distance, or pair it with `range_remaining()` before dispatch. It measures geometry only and does not select a destination or account for available fuel.

PARAMETERS

Name	Type	Description
`x`	`float`	Target X coordinate in meters
`y`	`float`	Target Y coordinate in meters

RETURNS

`float`

Straight-line distance in meters from the drone's current position to the given point.

`.go_to(x: float, y: float) → ActionResult` SELF ONLY

Fly to any world coordinate as a base drone capability; no field module is required. Once a powered route is accepted, `status()` reports `"traveling"` immediately, then the drone flies in a straight line and hovers on arrival. Stopping or completing the script, hitting an error, or calling `go_to_station()` cancels this route. For Weather, pass the exact `x` and `y` assembled from checksum-valid storm packets.

PARAMETERS

Name	Type	Description
`x`	`float`	World x, m.
`y`	`float`	World y, m.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_target"`	rejection	The supplied target is not valid for this operation.
`"out_of_bounds"`	rejection	The requested position lies outside the valid world bounds.
`"out_of_range"`	rejection	The requested target lies outside the operation's range.
`"busy"`	transient	The component is already performing another operation.
`"scrambled"`	transient	Radiation scrambling prevents the operation.

`.collect() → CollectResult` SELF ONLY

Collect one weather aftermath batch at the drone's exact current coordinate. A successful batch transfers at most **5** Storm Glass or Raw Uranium. Raw Uranium collection adds **40** exposure without Shield Plating and **0** with it; the batch is kept even if the drone's exposure reaches capacity and the drone is scrambled.

RETURNS
`CollectResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
`.item_id`, `.collected`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Drone cargo received: `.item_id` × `.collected`.
`"moving"`	transient	The drone is still moving.
`"busy"`	transient	The drone is occupied by a biological extraction.
`"nothing_here"`	rejection	No live weather aftermath exists at the drone's current coordinate.
`"no_cargo_space"`	rejection	The drone has no cargo capacity for the material at this site.
`"scrambled"`	transient	The drone's electronics are radiation-scrambled and cannot collect.

`.exposure() → float`

Current extraction exposure, from **0** to `exposure_capacity()`. It changes only when collecting Raw Uranium or receiving Service Station care; flying over a hidden aftermath deposit adds no exposure. Any docked drone that is not scrambled clears **10 per hour** at a powered Drone Service Station. At capacity the drone is scrambled and requires Service Station rescue.

RETURNS
`float`

Radiation exposure from **0** to capacity. An unplated uranium collection adds **40**; Shield Plating adds **0**. A powered Drone Service Station clears non-terminal exposure at **10/h**. At capacity the drone scrambles and requires rescue.

`.exposure_capacity() → float`

The **100** exposure scramble threshold. An unplated drone can collect two 5-unit batches safely; the third batch is retained and then scrambles it.

RETURNS

`float`

The scramble threshold.

`.is_plated() → bool`

`True` with Shield Plating mounted. Plating reduces Raw Uranium extraction exposure to zero, halves each Cargo Pod's capacity, and raises fuel burn **1.5×** because lead is heavy.

RETURNS

`bool`

`True` when Shield Plating is mounted: zero Raw Uranium extraction exposure and required for radioactive-cargo loads from Lead Casks; halves Cargo Pod capacity and multiplies energy use by **1.5**.

`.range_remaining() → float`

Estimated flight distance in meters at the current energy and throttle. Electric burn is **5 Wh/h** at full throttle; Heli burn is **5 t/h Oil**. Both scale with throttle squared, so slower routes stretch range.

RETURNS

`float`

Estimated meters of flight from the current battery charge or oil, at the current throttle. **0** if throttle is 0 or battery or oil is empty.

`.throttle() → float`

Current throttle (**0-1**). Full-throttle burn is **5 Wh/h** for electric propulsion or **5 t/h Oil** for Heli; both scale with throttle squared. Reads **0** after Stop, completion, or error.

RETURNS

`float`

Current throttle setting (**0-1**).

`.set_throttle(rate: float) → ActionResult` SELF ONLY

Set throttle (**0-1**). At full throttle, electric drones fly **300 m/h** using **5 Wh/h**; Heli drones fly **900 m/h** using **5 t/h Oil**. Lower throttle reduces burn quadratically. Stop, completion, or error resets throttle to **0**. Self-only.

PARAMETERS

Name	Type	Description
`rate`	`float`	Throttle setting (**0-1**).

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.modules() → list[MountSlot]`

Inspect what's coupled. Returns a list of `MountSlot`, one per slot on the chassis. Each has `.index` (pass to `couple` / `uncouple`), `.type` (`"thruster"` for slot 0, `"drone_module"` for the rest), `.module_id` (the coupled module, or `None` for an empty slot). Works on any drone, not only `self`. See `MountSlot`.

RETURNS

`list[MountSlot]`

One per slot on the chassis.

`.couple(slot_index: int, module_id: str) → ActionResult` SELF ONLY

Request a hardware service order from Inventory into an explicit whole-number slot: `self.couple(0, "electric_thruster")` for the thruster slot, or `self.couple(1, "battery_pack")` for a module slot. The drone must be docked at an operational Drone Depot. This dedicated-hardware exception does not expose ordinary Inventory freight at that outpost. Self-only.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number drone slot index, starting at 0.
`module_id`	`str`	Shop id of the drone module

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_at_station"`	rejection	The vehicle is not docked at a compatible station.
`"item_not_in_inventory"`	rejection	Inventory does not contain the requested item.
`"unknown_module"`	rejection	The supplied module identifier does not exist.
`"slot_not_compatible"`	rejection	The selected slot is not compatible with the module.
`"slot_occupied"`	rejection	The selected slot is occupied.
`"invalid_slot"`	rejection	The supplied slot does not exist.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"wrong_engine_for_module"`	rejection	The module is incompatible with the vehicle's engine type.
`"cargo_capacity_exceeded"`	rejection	Current cargo exceeds the requested rig's capacity.

`.uncouple(slot_index: int) → ActionResult` SELF ONLY

Request a hardware service order that returns the module in an explicit whole-number slot to Inventory: `self.uncouple(1)`. The drone must be docked at an operational Drone Depot. Cargo Pods must be empty before removal, and Shield Plating cannot be removed while Raw Uranium or Fuel Rod cargo remains aboard. Fuel-bearing modules preserve their contents. Self-only.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number drone slot index, starting at 0.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_slot"`	rejection	The supplied slot does not exist.
`"module_not_mounted"`	rejection	The selected module slot has no mounted module.
`"not_at_station"`	rejection	The vehicle is not docked at a compatible station.
`"cargo_capacity_exceeded"`	rejection	Current cargo exceeds the requested rig's capacity.
`"container_not_empty"`	rejection	The module's cargo container is not empty.
`"hot_cargo_requires_plating"`	rejection	Raw Uranium or Fuel Rod cargo remains aboard and requires Shield Plating.
`"inventory_full"`	rejection	Inventory has no capacity for the result.

`.status() → str`

Current operational activity for progress and blocker handling, not an arrival test. `"idle"` can mean docked, hovering at a field coordinate, or holding a queued route at zero throttle; `charging` or `refueling` can begin immediately after docking. `"waiting_bay"` means the drone reached a full Depot but is not docked, `"holding_weather"` is a temporary heli hold, and `stalled` or `scrambled` states need intervention. Use `current_station()` or `current_drill()` to confirm arrival at an interaction endpoint.

RETURNS

`str`

Current operational activity. Use `current_station()` for the separate physical-arrival check.

POSSIBLE VALUES

`"idle"`, `"traveling"`, `"charging"`, `"refueling"`, `"waiting_service"`, `"waiting_oil"`, `"waiting_bay"`, `"being_rescued"`, `"holding_weather"`, `"scrambled"`, `"stalled_no_battery"`, `"stalled_no_oil"`, `"stalled_no_route"`

`.is_being_rescued() → bool`

`True` while a Drone Service Station's recovery vehicle is outbound to, servicing, or carrying this drone. Use this to pause route scripts while the recovery vehicle has control.

RETURNS

`bool`

`.rescue_status() → str`

Current rescue mission phase for this drone: `"none"`, `"outbound"`, `"charging"`, `"carrying"`, or `"returning"`. `"returning"` means the recovery vehicle is heading home and the drone is no longer under rescue control.

RETURNS

`str`

POSSIBLE VALUES

`"none"`, `"outbound"`, `"charging"`, `"carrying"`, `"returning"`

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Fleet

Read-only index of every owned mobile unit: ground vehicles and drones. Use it for dashboards, charging scripts, rescue thresholds, and dispatch decisions without hardcoding names. Fleet refs are snapshots; control still goes through the unit's own script or the relevant station API.

Access via: `get_component("fleet")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.vehicles() → list[VehicleRef]`

All owned ground vehicles as `VehicleRef` snapshots. Each ref includes `.id`, `.name`, `.kind`, `.x`, `.y`, `.battery_level`, `.is_docked`, `.current_station`, `.is_being_rescued`, and `.rescue_status`.

RETURNS

`list[VehicleRef]`

Snapshots. Re-query for fresh positions and status, or use `get_component(ref.id)` for the live vehicle API.

`.drones() → list[DroneRef]`

All owned drones as `DroneRef` snapshots. Each ref includes `.id`, `.name`, `.kind`, `.engine`, `.current_station`, `.battery_level` or `.oil_level`, `.is_docked`, `.is_being_rescued`, and `.rescue_status`.

RETURNS

`list[DroneRef]`

Snapshots. Re-query for fresh positions and status, or use `get_component(ref.id)` for the live drone API.

`.mobile_units() → list[MobileUnitRef]`

All owned ground vehicles and drones in one list. Use `.category` to branch between `"vehicle"` and `"drone"`.

RETURNS

`list[MobileUnitRef]`

Snapshots. Re-query for fresh positions and status.

Nav Module MODULE

Lets a vehicle drive through `self.nav`. A basic module provides **1.0×** top speed. One Sport Nav on the same rig provides **2×** top speed with **2.6×** movement power draw, about **1.3×** battery use per meter at full throttle. Further Sport Navs add **1.0×** base top speed each and raise draw faster. It fits a `nav` or `universal` slot. Keep the script running until arrival. The vehicle stops and clears its route if the script stops, ends, or errors.

Access via: `self.nav`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

METHODS

`.set_target(x: float, y: float) → ActionResult` SELF ONLY

Set target coordinates without changing the throttle. **Returns immediately**. Call `self.nav.set_throttle(0.5)` to drive asynchronously while this script remains active. Poll `get_distance_to(x, y)` or `get_position()` in a wait loop. Use a distance tolerance, such as `while self.nav.get_distance_to(x, y) > 2:`, rather than exact zero, then call `self.nav.brake()` before `drill.mine()` or another stationary action. For an at-building action, target the building's `BuildingRef.position`, not its outpost footprint anchor. Stopping, ending, or failing the script stops the vehicle and clears its route.

PARAMETERS

Name	Type	Description
`x`	`float`	Target X coordinate in meters
`y`	`float`	Target Y coordinate in meters

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_mounted"`	● rejection	The required module is not mounted.
`"invalid"`	● rejection	One or more supplied arguments are outside the accepted domain.
`"out_of_bounds"`	● rejection	The requested position lies outside the valid world bounds.

`.set_throttle(power: float) → ActionResult` SELF ONLY

Set throttle (**0.0-1.0**, clamped). `self.nav.set_throttle(0.5)` cruises; `1.0` sprints but burns more battery per meter. Use to trade speed for range on long runs. Stop, completion, error, and `brake()` reset throttle to **0**.

PARAMETERS

Name	Type	Description
`power`	`float`	Throttle fraction in the **0-1** range

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.

`.throttle() → float`

Current throttle setpoint (**0.0-1.0**). Returns the value the script last wrote via `self.nav.set_throttle(...)`, or **0** after Stop, completion, error, or `brake()`. Distinct from `get_speed()`, `throttle()` is your intent, `get_speed()` is the vehicle's speed in meters per hour on its last drive tick.

RETURNS

`float`

Current throttle setpoint (**0.0-1.0**).

`.brake() → ActionResult` SELF ONLY

Stop the vehicle immediately, throttle and speed set to **0**, and the current target is cleared to the vehicle's current position. Use when a script needs to abort a drive mid-route (e.g. re-pathing toward a closer site). Cheaper battery-wise than driving to destination.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.

`.get_position() → Position`

Current position as a `Position` object with `.x` and `.y` in meters from base. Read each iteration of a drive loop to detect arrival, plan next hop, or log the path. See `Position`.

RETURNS

`Position`

In meters from base.

`.get_speed() → float`

Speed in meters per hour as recorded on the last drive tick. **0** while idle or braked; up to the Nav's top speed at throttle **1.0**. Use to confirm the vehicle is actually moving (if `0` when you expected drive, there's a power or target issue). A fresh `set_throttle(...)` won't show up here until the next drive tick.

RETURNS

`float`

In m/h.

`.get_distance_to(x: float, y: float) → float`

Euclidean distance in meters from the vehicle's current position to the given point. Use it inside an intentional wait loop to detect proximity. The vehicle has arrived once the distance is 2 or less (never wait for exact zero), so the loop keeps going while it is `> 2`. Arriving does not stop the vehicle; call `brake()` before a stationary at-site action. For an at-building action, route to that building's `BuildingRef.position`. `set_target()` does not block while the vehicle drives, and the drive is canceled if the script stops, completes, or errors. This is pure straight-line distance and does not account for obstacles.

PARAMETERS

Name	Type	Description
`x`	`float`	Target X coordinate in meters
`y`	`float`	Target Y coordinate in meters

RETURNS

`float`

In meters.

`.speed_multiplier() → float`

Current top-speed multiplier: **1.0** with Basic Nav/no Sport Nav, or **1 + mounted Sport Nav count** on Pioneer. Use it to plan trip times and scout builds. This is speed only; range still depends on battery, throttle, cargo load, and movement draw.

RETURNS

`float`

1.0 basic, or **1 + Sport Nav count** on Pioneer. Speed only; range depends on battery, throttle, cargo load, and movement draw.

Pioneer

A modular long-range vehicle for driving, scanning, mining, and building in the field. The bare chassis does nothing; everything comes from the modules, batteries, and cargo you mount in its eight slots.

Type	Mining
Stockpile	0 units (mixed)

HOW TO OBTAIN

1. Requires the **Pioneer Chassis** research (Terraform Index 100,000).
2. Buy from the Shop for 5,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.battery: Battery`

Aggregated battery pool across every Portable Battery in every mounted Battery Holder. `self.battery.level()` returns a **0-1** fraction; `self.battery.wh()` raw watt-hours; `self.battery.capacity()` the summed max. For per-holder detail, `self.battery.holders()` returns each `Holder` with its `.batteries` list. The aggregate view covers most scripts, introspection is for advanced rebalance logic. If zero Portable Batteries are installed, the Pioneer can't move. See `Battery`.

RETURNS

`Battery`

Aggregated across every battery
in every mounted Battery Holder.

`.cargo: Cargo`

Combines the Portable Bins in the Pioneer's Cargo Racks. Use `count()`, `capacity()`, and `full()` for totals, or `racks()` to inspect the physical layout. Each bin holds one item type. `take()` and `send()` handle bins independently. `compact()` consolidates matching cargo into fewer bins without changing `send()` order. `discard(rack_index)` permanently empties one rack and takes **1 hour** when cargo is present; an empty rack finishes immediately. See `Cargo`.

RETURNS

`Cargo`

Aggregated across every bin in
every mounted Cargo Rack.

`.nav: NavModule`

Set a destination with `self.nav.set_target(x, y)` and open the throttle with `self.nav.set_throttle(0.5)` to drive the Pioneer. Both calls return immediately; setting a destination leaves the throttle unchanged. Distance tolerance means close enough, not stopped: call `self.nav.brake()` before mining, construction, or cargo transfer. Stopping, ending, or failing the script stops the Pioneer and clears its route. Requires a Nav Module in a Universal slot.

RETURNS

`NavModule`

When a Nav Module is mounted.

`.sonar: SonarModule`

Finds useful sites near the Pioneer. Use `self.sonar.scan()` to find nearby sites, then `self.sonar.survey(site)` to reveal details such as mineral hardness, purity, or vent output. Scanning and new surveys take time and use battery, so the script pauses while the sonar works. Requires a mounted Sonar Module. See `SonarModule` for its range, capabilities, and battery use.

RETURNS

`SonarModule`

When a Sonar Module is mounted.

`.drill: DrillModule`

Extracts one mineral unit from the surveyed site under the Pioneer with `self.drill.mine()`. The vehicle must be stationary and have enough cargo space and power. Requires a mounted Drill Module. See `DrillModule` for speed and hardness limits.

RETURNS

`DrillModule`

When a Drill Module is mounted.

`.constructor: ConstructorModule`

Builds or removes Planet Map blueprints from Plan Mode or scripts. Read pending work from `get_component("construction_blueprint")`, drive near `construction.position`, then call `self.constructor.execute(construction.id)`. Building consumes the required kits or segments from cargo; removal returns reclaimed parts to cargo, so leave room. Requires a mounted Constructor Module. See `ConstructorModule`.

RETURNS

`ConstructorModule`

When a Constructor Module is mounted.

`.input: VehicleInputSlot`

Loads cargo from Inventory at home, local storage at outposts, field-extractor stockpiles, or a nearby stopped cargo vehicle. Loading from a field stockpile or from another vehicle requires service range; a vehicle-to-vehicle load also needs both vehicles stopped. It never destroys cargo; use `self.cargo.discard(rack_index)` to jettison one rack. Requires **Auto Feeders**. See `VehicleInputSlot`.

RETURNS

`VehicleInputSlot`

`.output: OutputSlot`

Unloads cargo to Inventory, a Storage Bin, a Warehouse, or a nearby stopped cargo vehicle. Unloading into Inventory works only while the Pioneer is parked at home. Remote outposts use their local stores. Vehicle-to-vehicle handoffs require both vehicles stopped within **~2 m**. Requires **Auto Feeders** research. See `OutputSlot`.

RETURNS

`OutputSlot`

METHODS

`.swap_batteries(charger: str, slots: list[tuple[int, int]] | None = None, min_level: float = 1.0) → BatterySwapResult` SELF ONLY

Atomically exchange selected installed batteries for unqueued cells of the same type at or above `min_level`. The Pioneer must be parked in the charger's service area. All selected cells must have replacements; failure changes nothing. Drained cells occupy the vacated sockets, even when full. The call yields during normal handling and locks both endpoints. Charged stock can be used during an outage. No Inventory items are read or written.

PARAMETERS

Name	Type	Description
`charger`	`str`	Local Battery Charger instance ID or unique display name.
`slots`	`list[tuple[int, int]]` | `None`	A list of (holder slot, internal slot) pairs, or `None` for every installed portable battery. Empty list exchanges nothing.
`min_level`	`float`	Required charge fraction greater than 0 and at most 1; defaults to 1.0.

RETURNS

`BatterySwapResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.swapped_count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed.
`"no_op"`	● success	No changes were needed.
`"not_found"`	● rejection	No Battery Charger matches that reference.
`"under_construction"`	● rejection	Construction is not complete.
`"busy"`	● transient	An endpoint is still handling batteries or cargo.
`"not_at_service_point"`	● rejection	The Pioneer is not parked in this charger's service area.
`"invalid_slots"`	● rejection	A selected rig socket does not contain a supported battery.
`"no_compatible_battery"`	● rejection	There are not enough unqueued replacements of the required battery type.
`"insufficient_charged_batteries"`	● rejection	Available replacements do not meet the requested minimum charge.

`.status() → str`

Read the Pioneer's current physical activity. Each call reads fresh state, including through `get_component(...)`. An idle Pioneer may still have a script running or a job assigned.

RETURNS

`str`

Current vehicle activity, evaluated when called.

POSSIBLE VALUES

`"idle"`, `"moving"`, `"stranded"`, `"scanning"`, `"surveying"`, `"drilling"`, `"discarding"`, `"constructing"`, `"transferring"`, `"charging"`, `"queued"`, `"being_rescued"`

`.is_being_rescued() → bool`

`True` while a Vehicle Charging Station rescue drone is charging this Pioneer where it stands. Use this to pause movement, mining, or construction scripts even if the battery has started rising above zero.

RETURNS

`bool`

`.rescue_status() → str`

Current rescue mission phase for this Pioneer: `"none"`, `"outbound"`, `"charging"`, or `"returning"`. `"returning"` means the rescue drone is going home and the Pioneer is free again.

RETURNS

`str`

POSSIBLE VALUES

`"none"`, `"outbound"`, `"charging"`, `"returning"`

`.dock(station: str) → ActionResult` SELF ONLY

Pick which Vehicle Charging Station this parked Pioneer docks at: `self.dock("charging_station_2")`. Every station inside an outpost covers the whole outpost, so this chooses between them. Without a choice, the Pioneer docks at the eligible station closest to the vehicle, measured to the station or its outpost's center, whichever is closer; equal-distance ties are broken alphabetically by station id. The choice lasts until the Pioneer drives away or the station is removed. Moving to another station ends any charging job at the old one, and the new station charges the Pioneer once its own script calls `charge()`.

PARAMETERS

Name	Type	Description
`station`	`str`	Id or display name of a Vehicle Charging Station whose service area contains the parked Pioneer.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"station_not_found"`	rejection	The requested station does not exist.
`"under_construction"`	transient	The target is still under construction.
`"not_at_station"`	rejection	The vehicle is not parked inside the requested station's service area.

`.current_station() → str`

Stable id of the Vehicle Charging Station this Pioneer is docked at, whether it picked it with `dock()` or docked there by default. Empty string while the Pioneer is moving or parked away from every station.

RETURNS

`str`

The station id, or empty if the Pioneer is not docked at a station.

`.modules() → list[MountSlot]`

Inspect every slot on the chassis. Returns a list of `MountSlot`, eight entries for the Pioneer. Each has `.index` (pass to `mount` / `unmount`), `.type` (always `"universal"` on Pioneer), `.module_id` (what's mounted, or `None` for empty), `.internal_count` (non-zero for Battery Holders / Cargo Racks), `.internal_items` (list of installed portable item ids). Call before `self.mount(...)` or `self.install(...)` to find an empty target. See `MountSlot`.

RETURNS

`list[MountSlot]`

One per mount point on the chassis.

`.mount(slot_index: int, item_id: str) → ActionResult` SELF ONLY

Request a hardware service order from Inventory into whole-number `slot_index`: `self.mount(0, "nav_module")`. Biological field modules are drone-only. The Pioneer must be parked inside the service area of the home base or an operational founded outpost. This dedicated-hardware service does not make Inventory a freight endpoint at remote outposts.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number zero-based slot position on the chassis.
`item_id`	`str`	Shop id of the module to mount.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"item_not_in_inventory"`	● rejection	Inventory does not contain the requested item.
`"unknown_module"`	● rejection	The supplied module identifier does not exist.
`"slot_not_compatible"`	● rejection	The selected slot is not compatible with the module.
`"slot_occupied"`	● rejection	The selected slot is occupied.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"capability_already_mounted"`	● rejection	The vehicle already has a module providing this capability.

`.unmount(slot_index: int) → ActionResult` SELF ONLY

Request a hardware service order that returns the module at whole-number `slot_index` to Inventory: `self.unmount(0)`. The Pioneer must be parked inside the service area of the home base or an operational founded outpost. Empty every internal bay before removing a holder or rack.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number zero-based slot position on the chassis.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"slot_empty"`	● rejection	The selected slot is empty.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"holder_not_empty"`	● rejection	The equipment holder still contains an item.
`"inventory_full"`	● rejection	Inventory has no capacity for the result.

`.install(slot_index: int, internal_index: int, item_id: str) → ActionResult`

SELF ONLY

Request a service order that installs a Portable Battery or empty Portable Storage Bin from Inventory into a container's whole-number internal slot. `self.install(0, 1, "portable_battery")` uses bay **1** of the Battery Holder at chassis slot **0**. `self.install(4, 0, "portable_bin")` uses bay **0** of the Cargo Rack at chassis slot **4**. The Pioneer must be parked inside the service area of the home base or an operational founded outpost.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number slot where the container (Battery Holder / Cargo Rack) is mounted.
`internal_index`	`int`	Whole-number zero-based index within the container's internal slots.
`item_id`	`str`	Shop id of the portable item to install.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"invalid_internal_slot"`	● rejection	The supplied internal slot does not exist.
`"slot_empty"`	● rejection	The selected slot is empty.
`"not_container"`	● rejection	The supplied target is not a compatible container.
`"item_not_accepted"`	● rejection	The destination does not accept the supplied item.
`"internal_slot_occupied"`	● rejection	The selected internal slot is occupied.
`"item_not_in_inventory"`	● rejection	Inventory does not contain the requested item.

`.uninstall(slot_index: int, internal_index: int) → ActionResult` SELF ONLY

Request a service order that returns the portable item in a container's whole-number internal slot to Inventory: `self.uninstall(0, 1)`. The Pioneer must be parked inside the service area of the home base or an operational founded outpost, and Portable Storage Bins must be empty.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number slot where the container is mounted.
`internal_index`	`int`	Whole-number zero-based index within the container's internal slots.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_at_service_point"`	rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"invalid_slot"`	rejection	The supplied slot does not exist.
`"invalid_internal_slot"`	rejection	The supplied internal slot does not exist.
`"slot_empty"`	rejection	The selected slot is empty.
`"not_container"`	rejection	The supplied target is not a compatible container.
`"internal_slot_empty"`	rejection	The selected internal slot is empty.
`"inventory_full"`	rejection	Inventory has no capacity for the result.
`"container_not_empty"`	rejection	The module's cargo container is not empty.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Rover

Your starter expedition vehicle for driving, scanning, and mining. The bare chassis does nothing on its own; every ability comes from the modules mounted in its three fixed slots.

Type	Mining
Energy	100 Wh
Stockpile	10 units (mixed)

HOW TO OBTAIN

1. Requires the **Rover Chassis** research (Pressure 0.11).
2. Buy from the Shop for 2,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.battery: Battery`

How much charge the Rover has left. `self.battery.level()` returns a **0-1** fraction, `self.battery.wh()` raw watt-hours, `self.battery.capacity()` the **100 Wh** max. Below **0.1** you're close to getting stranded, `charging_station.dispatch_rescue()` can fetch you, but it's slow. `.holders()` returns an empty list (sealed rig). See `Battery`.

RETURNS

`Battery`

`.cargo: Cargo`

Everything carried in the Rover's integrated hold. `self.cargo.count()` returns total units across every item and property variant; `self.cargo.capacity()` is **10**; `self.cargo.full()` is `True` at capacity, check it before loading or drilling. `.racks()` is empty because the hold is integrated. `self.cargo.discard(0)` permanently jettisons the whole hold and takes **1 hour** when cargo is present; an empty hold finishes immediately. See `Cargo`.

RETURNS

`Cargo`

`.nav: NavModule`

Set a destination with `self.nav.set_target(x, y)` and open the throttle with `self.nav.set_throttle(0.5)` to drive the Rover. Both calls return immediately; setting a destination leaves the throttle unchanged. Distance tolerance means close enough, not stopped: call `self.nav.brake()` before mining, scanning, surveying, or cargo transfer. Stopping, ending, or failing the script stops the Rover and clears its route. Requires a mounted Nav Module.

RETURNS

`NavModule`

When a Nav Module is mounted,
else unavailable.

`.sonar: SonarModule`

Finds useful sites near the Rover. Use `self.sonar.scan()` to find nearby sites, then `self.sonar.survey(site)` to reveal details such as mineral hardness, purity, or vent output. Scanning and new surveys take time, so the script pauses while the sonar works. Requires a mounted Sonar Module. See `SonarModule` for its range, capabilities, and battery use.

RETURNS

`SonarModule`

When a Sonar Module is mounted, else unavailable.

`.drill: DrillModule`

Extracts one mineral unit from the surveyed site under the Rover with `self.drill.mine()`. The vehicle must be stationary and have enough cargo space and power. Requires a mounted Drill Module. See `DrillModule` for speed and hardness limits.

RETURNS

`DrillModule`

When a Drill Module is mounted, else unavailable.

`.input: VehicleInputSlot`

Loads cargo from Inventory at home, local storage at outposts, field-extractor stockpiles, or a nearby stopped cargo vehicle. Loading from a field stockpile or from another vehicle requires service range; a vehicle-to-vehicle load also needs both vehicles stopped. It never destroys cargo; use `self.cargo.discard(0)` to jettison the hold. Requires **Auto Feeders**. See `VehicleInputSlot`.

RETURNS

`VehicleInputSlot`

`.output: OutputSlot`

Unloads cargo to Inventory, a Storage Bin, a Warehouse, or a nearby stopped cargo vehicle. Unloading into Inventory works only while the Rover is parked at home. Remote outposts use their local stores. Vehicle-to-vehicle handoffs require both vehicles stopped within **~2 m**. Requires **Auto Feeders** research. See `OutputSlot`.

RETURNS

`OutputSlot`

METHODS

`.status() → str`

Read the Rover's current physical activity. Each call reads fresh state, including through `get_component(...)`. An `idle` Rover may still have a script running or a job assigned.

RETURNS

`str`

Current vehicle activity, evaluated when called.

POSSIBLE VALUES

`"idle"`, `"moving"`, `"stranded"`, `"scanning"`, `"surveying"`, `"drilling"`, `"discarding"`, `"constructing"`, `"transferring"`, `"charging"`, `"queued"`, `"being_rescued"`

`.is_being_rescued() → bool`

`True` while a Vehicle Charging Station rescue drone is charging this Rover where it stands. Use this to pause movement or mining scripts even if the battery has started rising above zero.

RETURNS

`bool`

`.rescue_status() → str`

Current rescue mission phase for this Rover: `"none"`, `"outbound"`, `"charging"`, or `"returning"`. `"returning"` means the rescue drone is going home and the Rover is free again.

RETURNS
`str`

POSSIBLE VALUES
`"none"`, `"outbound"`, `"charging"`, `"returning"`

`.dock(station: str) → ActionResult` SELF ONLY

Pick which Vehicle Charging Station this parked Rover docks at: `self.dock("charging_station_2")`. Every station inside an outpost covers the whole outpost, so this chooses between them. Without a choice, the Rover docks at the eligible station closest to the vehicle, measured to the station or its outpost's center, whichever is closer; equal-distance ties are broken alphabetically by station id. The choice lasts until the Rover drives away or the station is removed. Moving to another station ends any charging job at the old one, and the new station charges the Rover once its own script calls `charge()`.

PARAMETERS

Name	Type	Description
`station`	`str`	Id or display name of a Vehicle Charging Station whose service area contains the parked Rover.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"station_not_found"`	rejection	The requested station does not exist.
`"under_construction"`	transient	The target is still under construction.
`"not_at_station"`	rejection	The vehicle is not parked inside the requested station's service area.

`.current_station() → str`

Stable id of the Vehicle Charging Station this Rover is docked at, whether it picked it with `dock()` or docked there by default. Empty string while the Rover is moving or parked away from every station.

RETURNS

`str`

The station id, or empty if the Rover is not docked at a station.

`.modules() → list[MountSlot]`

Inspect what's mounted. Returns a list of `MountSlot`, one per chassis slot. Each has `.index` (pass to `mount` / `unmount`), `.type` (which modules fit), `.module_id` (mounted id or `None`), `.internal_items` (empty on the Rover's function slots). Call this before `self.mount(...)` to find an empty slot and verify the slot type accepts the module you want. See `MountSlot`.

RETURNS

`list[MountSlot]`

One per mount point on the chassis.

`.mount(slot_index: int, item_id: str) → ActionResult` SELF ONLY

Request a hardware service order from Inventory into whole-number `slot_index`: `self.mount(0, "nav_module")`. Biological field modules are drone-only. The Rover must be parked inside the service area of the home base or an operational founded outpost. This dedicated-hardware service does not make Inventory a freight endpoint at remote outposts.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number zero-based slot position on the chassis.
`item_id`	`str`	Shop id of the module to mount.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"item_not_in_inventory"`	● rejection	Inventory does not contain the requested item.
`"unknown_module"`	● rejection	The supplied module identifier does not exist.
`"slot_not_compatible"`	● rejection	The selected slot is not compatible with the module.
`"slot_occupied"`	● rejection	The selected slot is occupied.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"capability_already_mounted"`	● rejection	The vehicle already has a module providing this capability.

`.unmount(slot_index: int) → ActionResult` SELF ONLY

Request a hardware service order that returns the module at whole-number `slot_index` to Inventory: `self.unmount(0)`. The Rover must be parked inside the service area of the home base or an operational founded outpost, and container modules must be empty.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number zero-based slot position on the chassis.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"slot_empty"`	● rejection	The selected slot is empty.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"holder_not_empty"`	● rejection	The equipment holder still contains an item.
`"inventory_full"`	● rejection	Inventory has no capacity for the result.

`.install(slot_index: int, internal_index: int, item_id: str) → ActionResult`

SELF ONLY

Install a portable item (Portable Battery / Portable Storage Bin) into a container module's internal slot. **Not used on the Rover**, its fixed slots only accept Nav, Sonar, and Drill function modules, never containers. See Pioneer for the modular version.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number slot where the container (Battery Holder / Cargo Rack) is mounted.
`internal_index`	`int`	Whole-number zero-based index within the container's internal slots.
`item_id`	`str`	Shop id of the portable item to install.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_at_service_point"`	● rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"invalid_slot"`	● rejection	The supplied slot does not exist.
`"invalid_internal_slot"`	● rejection	The supplied internal slot does not exist.
`"slot_empty"`	● rejection	The selected slot is empty.
`"not_container"`	● rejection	The supplied target is not a compatible container.
`"item_not_accepted"`	● rejection	The destination does not accept the supplied item.
`"internal_slot_occupied"`	● rejection	The selected internal slot is occupied.
`"item_not_in_inventory"`	● rejection	Inventory does not contain the requested item.

`.uninstall(slot_index: int, internal_index: int) → ActionResult` SELF ONLY

Uninstall a portable item from a container module's internal slot. **Not used on the Rover**, same reason as `install`. See Pioneer for the modular version.

PARAMETERS

Name	Type	Description
`slot_index`	`int`	Whole-number slot where the container is mounted.
`internal_index`	`int`	Whole-number zero-based index within the container's internal slots.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_at_service_point"`	rejection	The vehicle is not parked inside base's or an operational outpost's service area. Parked means not moving and no active throttle; a vehicle with no drive energy counts as parked whatever its throttle says.
`"invalid_slot"`	rejection	The supplied slot does not exist.
`"invalid_internal_slot"`	rejection	The supplied internal slot does not exist.
`"slot_empty"`	rejection	The selected slot is empty.
`"not_container"`	rejection	The supplied target is not a compatible container.
`"internal_slot_empty"`	rejection	The selected internal slot is empty.
`"inventory_full"`	rejection	Inventory has no capacity for the result.
`"container_not_empty"`	rejection	The module's cargo container is not empty.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Sonar Module MODULE

Find and survey world sites through `self.sonar`. A scan checks the vehicle's surroundings; driving alone does not scan. Use `get_component("nocturna").points_of_interest()` to find unscanned contacts, travel near one, then call `scan()` and `survey(site)`. Basic sonar reaches **50 m** and hardness **1**; Extended **180 m** and **3**; Deep or Seismic **280 m** and **4**. Seismic also identifies deep oil potential after Advanced Oil Extraction. Resource research gates still apply. Results are saved in the Journal. See sonar guide for the sonar comparison, research requirements and scan-to-survey workflow.

Access via: `self.sonar`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.scan()` → `SonarScanResult` SELF ONLY

Sweep for compatible `Site`s within range of the vehicle's current position. Contact types include `MiningSite`, `ThermalVent`, `WaterWell`, `OilWell`, `ExoticDeposit`, and `GeologicalAnomaly`. A completed sweep updates the Journal with newly classified contacts.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`SonarScanResult`	`.status`, `.message`	`.sites`, `.blocked`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The sonar sweep completed. `result.sites` contains every compatible contact found; an empty list means there were no compatible contacts in range.
`"too_hard"`	● partial	The completed sweep detected a nearby unresolved contact above the mounted sonar's hardness limit.
`"tier_too_low"`	● partial	The completed sweep detected a nearby unresolved contact that the mounted sonar tier cannot classify.
`"research_required"`	● partial	The completed sweep detected a nearby unresolved contact whose classification research is not unlocked.
`"wrong_scanner"`	● partial	The completed sweep detected a nearby contact that vehicle sonar cannot classify.
`"busy"`	● transient	Another field action currently occupies the vehicle.
`"no_power"`	● rejection	The vehicle battery does not contain enough energy for a sonar sweep.

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.survey(site: str | Site | IdRecord) → SurveyResult` SELF ONLY

Reveal the details available for a productive `Site`. Pass either its string id or a `Site` from `scan()`. A repeat survey is free and instant unless a better sonar tier can reveal more. Ordinary scanning resolves inert `GeologicalAnomaly` contacts. Seismic Sonar and Advanced Oil Extraction allow a paid deep-oil survey of those formations, including ones already surveyed. Confirmed reservoirs become Oil Wells at the same coordinates; known dry formations are free to revisit.

PARAMETERS

Name	Type	Description
`site`	`str` | `Site` | `IdRecord`	A site id (string), a `Site` from `scan().sites`, or a dictionary or class instance with its string `id` field. The site must still be discovered, in range, and within this sonar's capability.

RETURNS

`SurveyResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.site`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Surveyed `site_id` successfully.
`"busy"`	transient	Another field action currently occupies the vehicle.
`"not_discovered"`	rejection	Site `site_id` is not in this planet's discovered-site journal.
`"research_required"`	rejection	Site `site_id` requires research that is not currently unlocked.
`"tier_too_low"`	rejection	The mounted sonar tier cannot survey site `site_id`.
`"out_of_range"`	rejection	Site `site_id` is outside the mounted sonar's current range.
`"too_hard"`	rejection	Site `site_id` exceeds the mounted sonar's mineral hardness limit.
`"no_power"`	rejection	The vehicle battery does not contain enough energy to survey site `site_id`.

RAISES

Exception	Condition
`ValueError`	`survey()` requires a non-empty site id or a `Site` object.
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.range() → float`

Current sonar range in meters.

RETURNS

`float`

In meters: **50** basic, **180** Extended, **280** Deep or Seismic. A stale captured module reference raises `ReferenceError`.

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.hardness_limit() → int`

Maximum mineral hardness this sonar can identify.

RETURNS

`int`

1 basic, **3** Extended, **4** Deep or Seismic. A stale captured module reference raises `ReferenceError`.

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.tier() → str`

Survey-depth tier granted by this sonar: `"basic"` / `"wide"` / `"deep"` / `"seismic"`. Controls how much of a thermal vent or exotic deposit is revealed by `survey()`; Deep or Seismic Sonar is also required for oil-well discovery.

RETURNS

`str`

The survey depth this sonar grants. It controls thermal and exotic detail; Deep or Seismic Sonar is required for oil-well discovery. A stale captured module reference raises `ReferenceError`.

POSSIBLE VALUES

`"basic"`, `"wide"`, `"deep"`, `"seismic"`

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

Data Archive

Stores JSON-safe data that survives script restarts and save/load. Access the Data Archive with `get_component("notebook")` after its research unlocks. Use Libraries to share code and the Signal Bus to share temporary live state.

Access via: `get_component("notebook")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.set(key: str, value: JsonValue) → ActionResult`

Store a JSON-safe value under a named key. Dictionaries inside the value must use string keys. The archive holds up to 2,048 entries; each value supports 8 nested levels, 16,384 total nodes counting values and containers, and 4,096 characters per string or dictionary key.

PARAMETERS

Name	Type	Description
`key`	`str`	Archive key, 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`
`value`	`JsonValue`	JSON-safe value, up to 8 nested levels, 16,384 total nodes counting values and containers, and 4,096 characters per string or dictionary key

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"invalid_key"`	● rejection	The supplied key is invalid.
`"entry_limit"`	● rejection	The archive has reached its entry limit.
`"invalid_value"`	● rejection	The supplied value is invalid.

`.transaction(key: str, default: JsonValue, updater: Callable) → ActionResult`

Atomically transform one stored value within the same archive value limits. The `updater` may be any pure callable; it receives the latest value or supplied `default` and cannot `sleep()`, yield, or mutate the world.

PARAMETERS

Name	Type	Description
`key`	`str`	Archive key, 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`
`default`	`JsonValue`	JSON-safe value within the archive value limits, used when the key is missing
`updater`	`Callable`	Pure callable that receives the current value and returns the next value within the archive value limits

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_key"`	rejection	The supplied key is invalid.
`"entry_limit"`	rejection	The archive has reached its entry limit.
`"invalid_value"`	rejection	The supplied value is invalid.
`"busy"`	transient	The component is already performing another operation.

`.get(key: str, default: JsonValue = None) → JsonValue`

Read a stored value by key. If the key is missing, returns the optional `default` argument; if no `default` is provided, returns `None`. Reading does not consume or modify the entry.

PARAMETERS

Name	Type	Description
`key`	`str`	Archive key, 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`
`default`	`JsonValue`	Returned when the key is missing

RETURNS

`JsonValue`

Stored value, the optional default, or `None`.

RAISES

Exception	Condition
`ValueError`	The archive key must be 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.has(key: str) → bool`

Return `True` when the archive contains the key, otherwise `False`.

PARAMETERS

Name	Type	Description
`key`	`str`	Archive key, 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`bool`

`True` when the archive contains the key.

RAISES

Exception	Condition
`ValueError`	The archive key must be 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.delete(key: str) → ActionResult`

Remove one key.

PARAMETERS

Name	Type	Description
`key`	`str`	Archive key, 1-96 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"invalid_key"`	● rejection	The supplied key is invalid.

`.keys(prefix: str = "") → list[str]`

Return archive keys as a sorted list. Pass a prefix such as `"rover."` to list only matching keys.

PARAMETERS

Name	Type	Description
`prefix`	`str`	Optional key prefix using the archive key character set

RETURNS

`list[str]`

Archive keys, sorted, optionally filtered by prefix.

RAISES

Exception	Condition
`ValueError`	The archive prefix must use letters, numbers, `_`, `.`, `:`, or `-` and be at most 96 characters.

`.clear(prefix: str = "") → CountResult`

Remove archived entries. With no prefix it clears the whole archive; with a prefix it clears matching keys.

PARAMETERS

Name	Type	Description
`prefix`	`str`	Optional key prefix using the archive key character set

RETURNS

`CountResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Command completed. Affected entries or units: `.count`.
`"no_op"`	● success	The command affected no entries or units.
`"invalid_key"`	● rejection	The supplied key is invalid.

Drone Depot

1-bay logistics endpoint at an outpost. Cargo I/O.

Type	Mining
Power in	-2 W (draws from grid)
Stockpile	50 units (mixed)

HOW TO OBTAIN

1. The recipe unlocks when you complete **Helios, Frame Order**.
2. Requires the **Basic Drone Operations** research (Terraform Index 180,000).
3. Fabricate **Drone Depot Kit** on a **Fabricator**: 2× Machine Frame, 1× Control Unit, 2× Gas Pipe Segment, 2× Liquid Pipe Segment, and 3 t Water.
4. Deploy it from your Inventory.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

`InputSlot` for this station's local stockpile. Connect `"inventory"` or a machine or storage source at the same outpost, then call `self.input.take(item_id, count)`. `self.input.flush()` discards the station stockpile. Stock at another Drone Depot is not visible here.

RETURNS

`InputSlot`

`.output: OutputSlot`

`OutputSlot` for this station's local stockpile. Connect `"inventory"` or a machine or storage target at the same outpost, then call `self.output.send(item_id, count)`. Move cargo between outposts with a drone; stations do not share stock.

RETURNS

`OutputSlot`

METHODS

`.get_docked() → list[str]`

List of drone ids currently docked at this Depot, in stable id order. Read each drone's state with `get_component(id)`.

RETURNS

`list[str]`

Instance ids of the drones currently docked at this Depot, in stable id order.

`.bay_count() → int`

Total bays at this station: **1** (basic) / **2** (medium) / **4** (large). Upgrading the Depot in place with a larger kit raises it at once.

RETURNS

`int`

Total bays at this station (**1** / **2** / **4** depending on tier).

`.bays_occupied() → int`

Bays currently occupied by docked drones. When equal to `bay_count`, arriving drones queue in airspace.

RETURNS

`int`

Bays currently occupied by docked drones.

`.slots_used() → int`

How many distinct materials the stockpile currently holds. One material is one slot no matter how many units of it are stored, so **50** units of one material fills a single slot.

RETURNS

`int`

Distinct materials currently held. Each material occupies one slot regardless of how many units of it are stored.

`.slot_capacity() → int`

How many distinct materials this depot can hold at once. A depot is a transfer proxy, not a warehouse: when every slot is taken, a `cargo.unload()` of a new material moves **0** units and reports that no slot is free, even while units remain free. Drain a material out to release its slot.

RETURNS

`int`

How many distinct materials this depot can hold at once (**3** / **4** / **6** depending on tier).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Drone Service Station

Charges electric drones in the field and recovers heli drones for queued refueling. Grid-tied.

Type	Mining
Power in	Variable (draws from grid)
Consumes	Oil, buffer 100 t

HOW TO OBTAIN

1. The recipe unlocks when you complete **Spire, Drone Power Trial**.
2. Requires the **Basic Drone Operations** research (Terraform Index 180,000).
3. Fabricate **Drone Service Station Kit** on a **Fabricator**: 2× Machine Frame, 1× Control Unit, 2× Circuit Panel, 1× Battery Cell, and 1× Liquid Pipe Segment.
4. Deploy it from your Inventory.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.oil_in: FluidPort`

Oil `FluidPort` for heli refueling. Call `self.oil_in.connect(...)` with an Oil Pump or oil tank's stable machine id or display name. A remote source also needs a completed conflict-free Liquid Pipe route between both locations.

RETURNS

`FluidPort`

Heli refueling supply. Call `connect(...)` with the provider's stable machine id or display name; local providers transfer directly and remote providers use completed liquid-pipe networks.

METHODS

`.get_docked() → list[str]`

List of all drone ids (electric and heli) currently parked at this station. Read each drone's state via `get_component(id)`.

RETURNS

`list[str]`

Ids of every drone docked at this station, electric and heli. Read each drone's state via `get_component(id)`.

`.charge(drone_id: str, target_level: float = 1.0) → ActionResult` SELF ONLY

Queue a parked electric drone to charge until its battery reaches `target_level` (greater than **0** and at most **1**, defaults to **1.0**). Electric charge and heli refuel jobs share one FIFO and the same service bays. An oil-blocked heli keeps its queue position but does not occupy a bay, so ready electric work may bypass it. Example: `self.charge("drone_small_1", 0.8)`. Self-only.

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a docked electric drone
`target_level`	`float`	Optional battery target fraction greater than **0** and at most **1**. Defaults to **1.0**.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"charging"`	success	A charging job is active.
`"queued"`	success	The operation was added to the queue.
`"target_reached"`	success	The vehicle is already at or above the requested charge or fuel level.
`"not_docked"`	rejection	The requested vehicle is not docked at this station.
`"station_offline"`	transient	The station is offline.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.refuel(drone_id: str, target_level: float = 1.0) → ActionResult` SELF ONLY

Queue a parked heli drone to refuel until its oil tank reaches `target_level` (greater than **0** and at most **1**, defaults to **1.0**). Electric charge and heli refuel jobs share one FIFO and the same service bays. Draws oil from `self.oil_in`. Self-only.

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a docked heli drone
`target_level`	`float`	Optional oil target fraction greater than **0** and at most **1**. Defaults to **1.0**.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"refueling"`	success	A refueling job is active.
`"queued"`	success	The operation was added to the queue.
`"target_reached"`	success	The vehicle is already at or above the requested charge or fuel level.
`"not_docked"`	rejection	The requested vehicle is not docked at this station.
`"station_offline"`	transient	The station is offline.
`"no_oil"`	rejection	No oil is available for the operation.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.dispatch_rescue(drone_name: str, target_level: float = 1.0) → ActionResult`

SELF ONLY

Send the recovery vehicle to a field drone chosen by your script; there is no hidden fuel threshold. Electric drones are charged in the field to `target_level` and resume their route. Heli drones are carried home, then join the normal refueling queue. Scrambled drones are also carried home so docking can reset their electronics. `target_level` must be above **0** and at most **1**, and defaults to **1.0**. Launch requires station power, but the mission can finish through a later outage. Self-only.

PARAMETERS

Name	Type	Description
`drone_name`	`str`	Display name or id of the stranded drone to service.
`target_level`	`float`	Optional battery/oil target greater than **0** and at most **1**. Electric drones are charged up to it in the field; heli drones are carried back and join the station's refueling queue with it as their target. Defaults to **1.0**.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"already_dispatched"`	rejection	This station already has an active rescue, or the requested target is already assigned to a rescue.
`"station_offline"`	transient	The station is offline.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"not_stranded"`	rejection	The target is not considered stranded.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.cancel_rescue() → ActionResult` SELF ONLY

Abort the in-flight rescue. The drone is released at its current position (it keeps any charge already delivered) and the recovery vehicle flies home. Self-only.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_rescue"`	rejection	There is no active rescue mission.

`.is_rescuing() → bool`

`True` while the recovery vehicle is on a rescue mission.

RETURNS
`bool`
`True` while the service vehicle is mid-mission.

`.get_rescue_target() → str`

Mission target's display name while the recovery vehicle is outbound, servicing, carrying, or returning; empty string when idle.

RETURNS
`str`
The mission target's display name while the service vehicle is outbound, servicing, carrying, or returning; empty string when idle.

`.stop(drone_id: str) → ActionResult` SELF ONLY

Cancel one active or queued job on this station, whether it is a charge or a refuel. The drone keeps any energy or oil already delivered.

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a queued drone

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.

`.clear_queue() → CountResult` SELF ONLY

Clear every active or queued charge and refuel job on this station. Docked drones stay parked and keep their current battery and oil.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Command completed. Affected entries or units: `.count`.
`"no_op"`	success	The command affected no entries or units.

`.get_active() → list[str]`

List of drone ids currently occupying active service bays (charging or refueling). An oil-blocked heli is waiting, not active.

RETURNS

`list[str]`

Ids of the drones occupying active service bays (charging or refueling). Heli drone refueling jobs held up because the station has no oil are waiting, not active.

`.get_queue() → list[str]`

One FIFO list of drone ids across electric charging and heli refueling. Oil-blocked helis stay in order while ready later jobs may use otherwise-idle bays.

RETURNS

`list[str]`

Queued drone ids, first in first out, across electric charging and heli refueling. Oil-blocked helis stay in this order while ready later jobs may use otherwise-idle bays.

`.status(drone_id: str) → dict[str, JsonValue]`

Detailed status for one drone's service job. Electric drones return a `dict` with `state`, `target_level`, `battery_wh`, `capacity_wh`, `rate_w`, `bay_index`, `queue_index`. Heli drones return `state`, `target_level`, `oil_tons`, `capacity_tons`, `rate_tons_per_hour`, `bay_index`, `queue_index`.

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a drone

RETURNS

`dict[str, JsonValue]`

The drone's active or queued service job. Electric: `state`, `target_level`, `battery_wh`, `capacity_wh`, `rate_w`, `bay_index`, `queue_index`. Heli: `state`, `target_level`, `oil_tons`, `capacity_tons`, `rate_tons_per_hour`, `bay_index`, `queue_index`.

`.get_bay_count() → int`

Number of simultaneous service bays.

RETURNS

`int`

How many drones can be serviced at once.

`.get_charge_rate(drone_id: str) → float`

Returns the Wh/h currently being pushed into the named electric drone (**0** if it is not in an active bay).

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a docked electric drone

RETURNS

`float`

Wh/h currently being pushed into that electric drone, **0** if it is not being charged.

`.get_refuel_rate(drone_id: str) → float`

Returns the oil t/h currently being pushed into the named heli drone (**0** if it is not in an active bay or the station has no oil).

PARAMETERS

Name	Type	Description
`drone_id`	`str`	Display name or id of a docked heli drone

RETURNS

`float`

t/h currently being pushed into that heli drone, **0** if it is not being refueled.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Earth Orders

Read Earth's current orders, the requirements and rewards of upcoming campaign orders, and completed campaign orders through `get_component("orders")`. Supply Dock scripts can use campaign and Weekly Orders to decide what to ship, while dashboards can show progress. Scripts cannot create or cancel Earth Orders. Bio Orders come from a Bio Exchange instead.

Access via: `get_component("orders")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.list_orders() → list[Order]`

Current contractor campaign Orders as a stable list of `Order` objects. These orders never expire and disappear from this list when fully shipped. Empty means no contractor shipment is currently available. See `Order`.

RETURNS

`list[Order]`

Current contractor campaign orders.

`.list_upcoming_orders() → list[Order]`

Plan future production and reward paths with upcoming contractor campaign Orders, in campaign declaration order, preserving each contractor's queue sequence. Excludes current orders, completed orders, and Weekly Earth Orders. Upcoming orders cannot be assigned to Supply Docks until they become current. See `Order`.

RETURNS

`list[Order]`

Future campaign orders (`status` is `"upcoming"`), in campaign declaration order. Empty when no upcoming campaign orders remain.

`.list_weekly_orders() → list[Order]`

The current five Weekly Earth Orders, including offers already fulfilled during this cycle. Returns an empty list until an eligible production chain is available. Weekly objects have `.kind == "weekly"`, `.expires_day`, no contractor, credits-only rewards, and `.status` of `"active"` or `"completed"`. The whole list is replaced every seven days.

RETURNS

`list[Order]`

The current five Weekly Earth Orders, including offers already fulfilled this cycle. Empty until an eligible production chain is available.

`.get_order(order_id: str) → Order | None`

Look up a specific Earth Order by id, including upcoming campaign orders for planning. Upcoming orders cannot be assigned to Supply Docks until they become current. Unknown or expired weekly ids return `None`; weekly completions remain visible only until their board refreshes. See `Order`.

PARAMETERS

Name	Type	Description
`order_id`	`str`	Earth Order id from `list_orders()`, `list_upcoming_orders()`, `list_weekly_orders()`, or `completed_orders()`

RETURNS

`Order | None`

A campaign order whose `status` is `"upcoming"`, `"active"`, or `"completed"`, or a Weekly Earth Order from the current board. `None` for unknown or expired weekly ids. Bio Order ids are read from a Bio Exchange.

`.completed_orders() → list[Order]`

Permanent contractor campaign history, ordered by completion time (oldest first). Weekly completions stay on the current Weekly board and are intentionally excluded from this ledger.

RETURNS

`list[Order]`

Completed contractor campaign orders, in order of completion. Excludes Weekly Earth Orders.

Signal Bus

Coordinates scripts through shared JSON-safe values. Access the Signal Bus with `get_component("comms")` after its research unlocks. Use `send()` and `receive()` for work that should be handled once; use `broadcast()` and `latest()` for the newest shared value.

Access via: `get_component("comms")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.send(channel: str, value: JsonValue) → SendResult`

Add a JSON-safe value to a named channel queue. Keep the send receipt to identify or cancel that exact request later, even when several requests contain identical values. Channel ids may contain letters, numbers, `_`, `.`, `:`, and `-`. Use queues for work items that should be handled once.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`
`value`	`JsonValue`	JSON-safe value, up to 8 nested levels, 1,024 total values, and 4,096 characters per string

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`SendResult`	`.status`, `.message`	`.message_id`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	One message was appended to the channel's queue, and its id is available in `message_id`.
`"invalid_channel"`	● rejection	The supplied channel id is outside the Signal Bus naming rules. No message was queued.
`"channel_limit"`	● rejection	The channel does not exist and the Signal Bus has no capacity for another channel. No message was queued.
`"queue_full"`	● rejection	The channel's queue has reached its message limit. No message was queued.
`"id_exhausted"`	● rejection	No additional stable message ids are available. No message was queued.
`"invalid_value"`	● rejection	The supplied value violates the Signal Bus value rules. No message was queued.

`.receive(channel: str, message_id: int | None = None) → ReceiveResult`

Take one queued message. Omit `message_id` or pass `None` to take the oldest, or supply a message id to take exactly that job. Selection and removal happen together, so only one competing receiver can take it. Other queued messages keep their order, and broadcasts are preserved. Use `pending()` to choose work by priority, location, or capability before receiving it.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`
`message_id`	`int` | `None`	Positive whole-number message id from `pending()` or a successful `send()` receipt. Omit or pass `None` to receive the oldest message. An id selects that exact message, never its position in the queue.

RETURNS

`ReceiveResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.packet`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	One message was removed from the queue and returned. Without a message id it was the oldest; with an id it was exactly the requested message.
`"empty"`	● success	No message id was requested and the channel has no queued messages. Nothing was consumed.
`"not_found"`	● rejection	The supplied message id is not waiting on this channel. No other message was consumed.
`"invalid_channel"`	● rejection	The supplied channel id is outside the Signal Bus naming rules.

RAISES

Exception	Condition
`TypeError`	`message_id` must be a finite whole number within the supported integer range.
`ValueError`	`message_id` must be greater than zero.
`OverflowError`	`message_id` must fit within the supported integer range.

`.wait(channel: str) → ReceiveResult`

Wait for and take the oldest queued message. If the queue is empty, only this script pauses until work is available; the game and other scripts keep running. Messages already queued are taken immediately. Broadcasts do not satisfy the wait. Pausing preserves the wait, and stopping the script abandons it without consuming a message.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`. A valid missing channel waits for its first queued message.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ReceiveResult`	`.status`, `.message`	`.packet`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	One message was removed from the queue and returned. Without a message id it was the oldest; with an id it was exactly the requested message.
`"invalid_channel"`	rejection	The supplied channel id is outside the Signal Bus naming rules.

`.wait_any(channels: list[str]) → WaitAnyResult`

Wait for and take one queued message from any listed channel. Channels listed first have priority whenever work is selected; each channel keeps its oldest-first order. If every queue is empty, only this script waits. Broadcasts do not satisfy the wait. The channel list is copied when called, and repeated names are considered once at their first position. Pausing preserves the wait; stopping abandons it without taking work.

PARAMETERS

Name	Type	Description
`channels`	`list[str]`	List of 1-128 channel-id strings, in priority order. Each id has 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`. Missing channels may receive work later. Repeated names keep their first position.

RETURNS

`WaitAnyResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.channel`, `.packet`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	One queued message was consumed from the first listed channel with waiting work. The result includes that channel and its copied message.
`"invalid_channel"`	● rejection	At least one listed channel id is invalid. No message was consumed; `channel` and `packet` are `None`.

RAISES

Exception	Condition
`TypeError`	`channels` must be a list containing only channel-id strings.
`ValueError`	`channels` must contain 1-128 entries.

`.wait_broadcast(channel: str) → WaitBroadcastResult`

Wait for the next broadcast on a channel. Every script already waiting captures that publication, including a repeated value or `None`. Existing broadcasts do not satisfy a new wait. Only this script pauses; queued messages remain untouched. The first publication is retained even if another broadcast follows or the channel is cleared. Pausing retains that signal for resume; stopping abandons the wait.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`. A valid missing channel can be watched without creating it. Only successful broadcasts on this channel satisfy the wait.

RETURNS

`WaitBroadcastResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.broadcast`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The first successful broadcast published after this call was captured without consuming shared state. Every script already waiting receives its own copy.
`"invalid_channel"`	● rejection	The supplied channel id is outside the Signal Bus naming rules.

`.pending(channel: str) → list[CommsMessage]`

Inspect all waiting messages on a channel in receive order without consuming them. Use the snapshot to display pending work in a Control Room card or total outstanding requests. Each message and its nested value are copied; editing the returned list or messages does not change the Signal Bus. Broadcasts and messages already received are excluded. Read again to refresh the snapshot.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`list[CommsMessage]`

Copied in receive order, oldest first. Empty when no messages are waiting, including a missing channel or one with only a broadcast.

RAISES

Exception	Condition
`ValueError`	The channel id must be 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.cancel(channel: str, message_id: int) → ActionResult`

Cancel one waiting message using its send receipt's `.message_id` or its `.id` from `pending(channel)`. Use it to remove an obsolete request or add a Cancel button to a Control Room job board. Other messages keep their ids, values, and receive order, including messages sent after the snapshot. The latest broadcast is preserved. Cancellation only removes queued work; it cannot stop a worker that has already received the message.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`
`message_id`	`int`	Positive whole-number message id from a successful `send()` receipt or `pending(channel)`, not a position in the queue.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The specified message was removed from this channel's queue. Other messages and the latest broadcast were preserved.
`"not_found"`	rejection	No queued message on this channel has the supplied id. Nothing was removed.
`"invalid_channel"`	rejection	The supplied channel id is outside the Signal Bus naming rules.

RAISES

Exception	Condition
`TypeError`	`message_id` must be a finite whole number within the supported integer range.
`ValueError`	`message_id` must be greater than zero.
`OverflowError`	`message_id` must fit within the supported integer range.

`.update(channel: str, message_id: int, value: JsonValue) → ActionResult`

Replace the complete value of one waiting message. Use its send receipt or an id from `pending()` to edit a delivery, priority, or destination from a script or Control Room card. The id, queue position, original sender, and send time stay unchanged. Replacement happens in one operation and works even when the queue is full. Messages already received cannot be edited. Read `pending()` again to refresh an earlier snapshot.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`
`message_id`	`int`	Positive whole-number id of a waiting message, from a successful `send()` receipt or `pending(channel)`. This identifies a message, not its position in the queue.
`value`	`JsonValue`	Complete replacement JSON-safe value, up to 8 nested levels, 1,024 total values, and 4,096 characters per string or dictionary key. Dictionary fields are replaced, not merged.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The queued message now contains the replacement value. Its id, queue position, original sender, and send time were preserved.
`"not_found"`	● rejection	No queued message on this channel has the supplied id. No value was changed.
`"invalid_channel"`	● rejection	The supplied channel id is outside the Signal Bus naming rules.
`"invalid_value"`	● rejection	The replacement violates the Signal Bus value rules. The queued message was not changed.

RAISES

Exception	Condition
`TypeError`	`message_id` must be a finite whole number within the supported integer range.
`ValueError`	`message_id` must be greater than zero.
`OverflowError`	`message_id` must fit within the supported integer range.

`.broadcast(channel: str, value: JsonValue) → ActionResult`

Store a channel's latest JSON-safe value without consuming queue slots. Dictionaries must use string keys. Use broadcasts for shared telemetry like fleet mode, target sector, or current priority.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`
`value`	`JsonValue`	JSON-safe value, up to 8 nested levels, 1,024 total values, and 4,096 characters per string

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"invalid_channel"`	rejection	The supplied channel identifier is invalid.
`"channel_limit"`	rejection	The maximum number of communication channels has been reached.
`"invalid_value"`	rejection	The supplied value is invalid.

`.latest(channel: str) → JsonValue`

Return the most recent value broadcast on a channel, or `None` if the channel has no latest value. Reading `latest()` does not consume it.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`JsonValue`

Latest broadcast value on the channel, or `None` if nothing has been broadcast.

RAISES

Exception	Condition
`ValueError`	The channel id must be 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.latest_info(channel: str) → BroadcastInfo | None`

Inspect the latest broadcast, who published it, and how long ago it was updated. Use the snapshot to detect outdated worker reports or show freshness in a Control Room card. Reading does not consume messages or change the channel. Read again to refresh the value and age.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`BroadcastInfo | None`

`None` when the channel has no broadcast. Missing saved sender or timestamp information is `None` in the corresponding field.

RAISES

Exception	Condition
`ValueError`	The channel id must be 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.queue_size(channel: str) → int`

Number of queued messages waiting on the channel.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS

`int`

Messages queued on the channel.

RAISES

Exception	Condition
`ValueError`	The channel id must be 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`, and cannot be a reserved object-field name.

`.channels() → list[str]`

All channel ids that currently have queued messages or a latest broadcast value.

RETURNS

`list[str]`

Ids of the channels with queued
or broadcast state.

`.clear(channel: str) → CountResult`

Remove a channel's queued messages and latest broadcast.

PARAMETERS

Name	Type	Description
`channel`	`str`	Channel id, 1-64 characters using letters, numbers, `_`, `.`, `:`, or `-`

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Command completed. Affected entries or units: `.count`.
`"no_op"`	success	The command affected no entries or units.
`"invalid_channel"`	rejection	The supplied channel id is outside the Signal Bus naming rules.

Supply Dock

Ships finished goods to Earth at **25 units/h** before throughput research. A script assigns a contractor or Weekly Earth Order, loads what it needs, and enables dispatch; completion or expiry stops the dock until reassigned.

Type	Logistics
Power in	-15 W (draws from grid)

HOW TO OBTAIN

1. Requires the **Supply Logistics** research (Terraform Index 110,000).
2. Buy from the Shop. Price starts at 5,000 cr and rises as you own more.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS
`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS
`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Load only what the active Order still needs with `connect(...)` and `take(...)`, or push from a parked cargo vehicle. Excess stays at the source, and active-order cargo stays reserved. After `clear_order()` or completion, use `eject(destination, item_id, count)` to recover leftovers to Inventory at Nocturna Base, or into compatible storage at the dock's own outpost elsewhere. `flush()` destroys loaded cargo. Requires **Auto Feeders** research. See `InputSlot`.

RETURNS

`InputSlot`

Order-aware: accepts only the active Order's remaining need. After the Order clears or completes, `eject(...)` recovers leftover cargo into a compatible destination you name at this outpost; `flush()` destroys it.

METHODS

`.capacity() → int`

Total units still owed across every item of the active Order (the dock's remaining demand). Returns **0** when no Order is assigned. Use as the upper bound for how much you still need to load + ship.

RETURNS

`int`

Total units still owed across the active Order; **0** when idle.

`.total() → int`

Sum of units currently loaded across every slot. Compare to `capacity()` to see how much more the dock still needs to ingest; `total() == 0` means every slot is empty.

RETURNS

`int`

Current units summed across every slot.

`.count(item_id: str) → int`

Units of `item_id` currently held across the dock's slots. Returns **0** if the dock holds none of that item. Use before loading more to avoid redundant `take()` calls: `if self.count("iron_ore") < 20: self.input.take("iron_ore", 20)`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to count

RETURNS

`int`

Units of `item_id` currently in any slot.

`.slots() → list[DockSlot]`

The dock's physical slots as a list of `DockSlot` objects (`.index` , `.item_id` , `.count`). Always **5** entries, indexed **0-4**; slots not opened by the current Order have `.item_id == None` and `.count == 0` . See `DockSlot` .

RETURNS

`list[DockSlot]`

One per physical slot (indexes **0-4**).

`.current_order() → Order | None`

Returns this dock's active Earth `Order` , or `None` if no Earth Order is assigned. Flips to `None` automatically when the Earth Order completes. Use it to read `.requires` and `.shipped` before deciding what to load.

RETURNS

`Order | None`

The order currently assigned to this dock, or `None` .

`.set_order(order_id: str) → ActionResult` SELF ONLY

Assign an Earth Order to this dock. Discover ids with `orders.list_orders()` or `orders.list_weekly_orders()`, then pass one to `self.set_order(id)`. Several docks may serve the **same** order and share shipped progress. Cargo is physical: drain this dock through a local machine or vehicle before switching orders.

PARAMETERS

Name	Type	Description
`order_id`	`str`	Earth Order id from `orders.list_orders()` or `orders.list_weekly_orders()`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"unknown_order"`	rejection	The supplied order is unknown or currently unavailable for assignment.
`"completed"`	success	The requested order is already completed and cannot be assigned again.
`"cargo_present"`	rejection	Existing cargo prevents the requested configuration change.

`.clear_order() → ActionResult` SELF ONLY

Release this dock's assignment and stop dispatch. Loaded cargo stays inside the dock. Recover it directly with `self.input.eject(destination, item_id, count)`, or by connecting a local machine or vehicle input to this Supply Dock.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.

`.set_enabled(on: bool) → ActionResult` SELF ONLY

Toggle the continuous dispatcher. `True` resumes shipping; `False` pauses it. Loading is unaffected either way, the input port still accepts material. **Auto-flips off** when the assigned order completes; the script must re-enable after the next `set_order` call.

PARAMETERS

Name	Type	Description
`on`	`bool`	`True` to resume dispatch, `False` to pause

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

`True` when the dispatcher is enabled, even if it is idle. `set_enabled(False)`, order completion, and weekly expiry disable it. A new dock starts enabled. Assign an `Order` while the dock is empty, then load it; an enabled, powered dock ships cargo its `Order` still needs.

RETURNS

`bool`

`True` when the dispatcher is active, `False` after `set_enabled(False)`. Newly deployed docks default to enabled.

`.dispatch_rate() → float`

The dispatcher's current effective throughput in **units/h**, already including throughput research and any outpost overcrowding penalty. The base rate is **25** (one unit every **2.4** minutes); **Bulk Logistics II** multiplies it by **4**, and **Bulk Logistics III** by **16**. Multiply this returned value by hours elapsed to predict how much the dock will ship.

RETURNS

`float`

Current dispatcher throughput, in **units/h**.

`.current_dispatch() → str | None`

The `item_id` the dispatcher is currently emitting, or `None` when idle (no power, no Order, dispatcher paused via `set_enabled(False)`, or no shippable unit loaded). Useful for scripts that want to know which material is flowing right now.

RETURNS

`str | None`

The item id currently being dispatched, or `None` when the dock is idle (no order, dispatcher paused, or no shippable unit loaded).

`.dispatch_progress() → float`

Normalized **0-1** progress toward the next dispatch pulse or launch cycle; a cycle can emit multiple cargo units according to the dispatch upgrade. Holds at **0** while the dock has nothing shippable loaded, the charge starts when a shippable unit lands. Drives the perimeter-clock animation on the dock card; scripts can use it to estimate "next launch in X hours."

RETURNS

`float`

Normalized progress toward the next dispatch pulse (**0-1**).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Construction Blueprint

Manages planned construction and removal work. Plan Mode and scripts share the same queue. Scripts can place structures, pipes, power lines, and bridges, or mark existing structures for removal. Planning creates the map marker immediately without needing a vehicle at the site. A Pioneer with a Constructor Module must still travel to each job and call `self.constructor.execute(construction.id)` to perform the work.

Access via: `get_component("construction_blueprint")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.plan_upgrade(item_id: str, machine: str | Component) → BlueprintPlanResult`

Queue an Oil Pump pack installation. The pump keeps operating until a parked Pioneer with the pack in cargo completes the construction job at its service area. Each pump can have one pending upgrade or dismantling job.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Oil Pump upgrade pack item id.
`machine`	`str` | `Component`	Oil Pump name, id, or component to upgrade in place.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`BlueprintPlanResult`	`.status`, `.message`	`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Construction blueprints created: `count`.
`"locked"`	● rejection	The required construction technology or kit recipe has not been unlocked.
`"not_found"`	● rejection	The target machine no longer exists.
`"not_upgrade_item"`	● rejection	The supplied item is not an upgrade pack.
`"wrong_machine_type"`	● rejection	The pack or target is not compatible with field Oil Pump installation.
`"under_construction"`	● rejection	The target machine is still under construction.
`"tier_not_ready"`	● rejection	The target machine is not at the tier required by this pack.
`"target_claimed"`	● rejection	The required map feature is claimed, or the target machine already has a planned upgrade or dismantling job.
`"blocked"`	● rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.

`.plan_structure(kind: str, x: float, y: float, rotation: int = 0) → BlueprintPlanResult`

Create one point-structure construction ghost from script coordinates. Supported kinds are `"outpost"`, `"thermal_cap"`, `"water_pump"`, `"oil_pump"`, `"exotic_gas_cap"`, `"exotic_spring_tap"`, `"mining_drill"`, `"mining_drill_industrial"`, and `"mining_drill_heavy"`. Drill kinds require their matching Earth Order kit recipe. Coordinates snap to the map grid. Extraction structures snap to the exact matching surveyed feature, while Outposts use the snapped footprint anchor. The optional clockwise rotation is `0`, `90`, `180`, or `270`. The ghost enters the shared queue immediately; a Pioneer still constructs it later.

PARAMETERS

Name	Type	Description
`kind`	`str`	Point blueprint kind.
`x`	`float`	World x coordinate.
`y`	`float`	World y coordinate.
`rotation`	`int`	Optional clockwise rotation: 0, 90, 180, or 270 degrees.

RETURNS

`BlueprintPlanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Construction blueprints created: `count`.
`"locked"`	● rejection	The required construction technology or kit recipe has not been unlocked.
`"invalid_kind"`	● rejection	The requested map structure kind is not supported.
`"invalid_rotation"`	● rejection	A structure rotation must be 0, 90, 180, or 270 degrees.
`"out_of_bounds"`	● rejection	At least one requested construction tile lies outside the planet bounds.
`"wrong_target"`	● rejection	The requested position does not contain the map feature required by this structure.
`"unsurveyed_target"`	● rejection	The required map feature has not been surveyed.
`"too_hard"`	● rejection	The selected Mining Drill cannot cut this deposit's mineral hardness.
`"target_claimed"`	● rejection	The required map feature is claimed, or the target machine already has a planned upgrade or dismantling job.

Status	Kind	Meaning
`"occupied"`	rejection	The requested structure footprint overlaps existing or planned construction, or a physical anomaly.
`"clearance"`	rejection	The requested structure footprint crosses an Outpost or physical-anomaly clearance boundary.
`"blocked"`	rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.

`.plan_pipe(medium: str, x1: float, y1: float, x2: float, y2: float) → BlueprintPlanResult`

Create pipe construction jobs from script coordinates. Requires Constructor Module research. `medium` is `"gas"`, `"liquid"`, or a registered fluid id such as `"steam"`, `"water"`, or `"oil"`. Coordinates snap to tile-center lanes. A field structure uses its site coordinates. An internal outpost machine uses its owning outpost as the utility anchor, so route to a point in `building.outpost`'s footprint, such as `[building.outpost.x, building.outpost.y]`, not to the vehicle docking point in `building.position`. Existing matching pieces are reused automatically.

PARAMETERS

Name	Type	Description
`medium`	`str`	`"gas"` / `"liquid"`, or any registered fluid id.
`x1`	`float`	World x coordinate of the route start. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.x`.
`y1`	`float`	World y coordinate of the route start. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.y`.
`x2`	`float`	World x coordinate of the route end. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.x`.
`y2`	`float`	World y coordinate of the route end. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.y`.

RETURNS

`BlueprintPlanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Construction blueprints created: `count`.
`"locked"`	● rejection	The required construction technology or kit recipe has not been unlocked.
`"invalid_medium"`	● rejection	The requested infrastructure medium is not supported.
`"out_of_bounds"`	● rejection	At least one requested construction tile lies outside the planet bounds.
`"invalid_route"`	● rejection	The requested endpoints do not define a usable construction route.
`"blocked"`	● rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.
`"already_exists"`	● success	The matching infrastructure route already exists, so no blueprint was created.

`.plan_power_line(x1: float, y1: float, x2: float, y2: float) → BlueprintPlanResult`

Create power-line construction jobs from script coordinates. Requires Constructor Module research. Coordinates snap to tile-center lanes; off-axis paths choose the valid L-shaped elbow with the least new construction. A field structure uses its site coordinates. An internal outpost machine uses its owning outpost as the utility anchor, so route to a point in `building.outpost`'s footprint, such as `[building.outpost.x, building.outpost.y]`, not to the vehicle docking point in `building.position`. Existing matching pieces are reused automatically.

PARAMETERS

Name	Type	Description
`x1`	`float`	World x coordinate of the line start. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.x`.
`y1`	`float`	World y coordinate of the line start. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.y`.
`x2`	`float`	World x coordinate of the line end. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.x`.
`y2`	`float`	World y coordinate of the line end. For an internal outpost machine, use a point in its owning outpost footprint, such as `building.outpost.y`.

RETURNS

`BlueprintPlanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Construction blueprints created: `count`.
`"locked"`	● rejection	The required construction technology or kit recipe has not been unlocked.
`"out_of_bounds"`	● rejection	At least one requested construction tile lies outside the planet bounds.
`"invalid_route"`	● rejection	The requested endpoints do not define a usable construction route.
`"blocked"`	● rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.
`"already_exists"`	● success	The matching infrastructure route already exists, so no blueprint was created.

`.plan_bridge(medium: str, x: float, y: float, axis: str) → BlueprintPlanResult`

Create one utility bridge job from script coordinates. Requires Constructor Module research. `medium` is `"gas"`, `"liquid"`, `"power"`, or a registered fluid id. `x` / `y` are the bridge center tile and `axis` is `"horizontal"` or `"vertical"`.

PARAMETERS

Name	Type	Description
`medium`	`str`	`"gas"`, `"liquid"`, `"power"`, or any registered fluid id.
`x`	`float`	World x coordinate of the bridge tile.
`y`	`float`	World y coordinate of the bridge tile.
`axis`	`str`	`"horizontal"` or `"vertical"`.

RETURNS

`BlueprintPlanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Construction blueprints created: `count`.
`"locked"`	● rejection	The required construction technology or kit recipe has not been unlocked.
`"invalid_medium"`	● rejection	The requested infrastructure medium is not supported.
`"invalid_axis"`	● rejection	A bridge axis must be `"horizontal"` or `"vertical"`.
`"out_of_bounds"`	● rejection	At least one requested construction tile lies outside the planet bounds.
`"blocked"`	● rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.
`"already_exists"`	● success	The matching infrastructure route already exists, so no blueprint was created.

`.mark_deconstruct(x: float, y: float, layer: str = "auto", target_id: str = "") → BlueprintPlanResult`

Mark built infrastructure or a normal map building at the coordinate for deconstruction. Requires Constructor Module research. Base and Outposts are protected. When several independent map layers overlap, choose the `"building"`, `"gas"`, `"liquid"`, or `"power"` layer; the default `"auto"` requires an unambiguous target. At a same-layer junction, provide the optional exact target id. A Pioneer still executes the job with `self.constructor.execute(id)` at the job's reported `position`.

PARAMETERS

Name	Type	Description
`x`	`float`	World x coordinate of the target tile.
`y`	`float`	World y coordinate of the target tile.
`layer`	`str`	Optional target layer. Use `"building"`, `"gas"`, `"liquid"`, or `"power"` when several layers overlap; `"auto"` selects the first target on an unambiguous tile.
`target_id`	`str`	Optional exact machine or infrastructure id used to resolve same-layer junctions.

RETURNS

`BlueprintPlanResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.blueprint_ids`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Construction blueprints created: `count`.
`"locked"`	rejection	The required construction technology or kit recipe has not been unlocked.
`"nothing_here"`	rejection	There is no completed deconstructable target at that position.
`"already_queued"`	success	A matching deconstruction blueprint is already queued.
`"ambiguous_target"`	rejection	More than one deconstructable target matches that position and layer. Choose a more specific layer or another tile.
`"out_of_bounds"`	rejection	At least one requested construction tile lies outside the planet bounds.
`"invalid_layer"`	rejection	A deconstruction layer must be `"auto"`, `"building"`, `"gas"`, `"liquid"`, or `"power"`.

Status	Kind	Meaning
`"blocked"`	● rejection	The requested construction or deconstruction is blocked by existing or planned infrastructure.

`.cancel(blueprint_id: str) → ActionResult`

Cancel a queued, active, or paused construction blueprint by id. Unpaid jobs cancel immediately. A paid job keeps its material at the build site: park a Pioneer there to recover it into cargo. A failed recovery leaves the job and material intact.

PARAMETERS

Name	Type	Description
`blueprint_id`	`str`	Blueprint id from `.pending_constructions()`, `.active_constructions()`, or `.paused_constructions()`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"worker_not_present"`	● rejection	No Pioneer is present at the construction site to receive the refund.
`"no_cargo_space"`	● rejection	Cargo has no compatible space for the result.
`"construction_dependency"`	● rejection	Another construction job depends on this job.

`.pending_constructions() → list[Construction]`

Returns blueprints awaiting a worker as `list[Construction]`. Drawn pipe/power paths and marked deconstruction targets are split into independent jobs, usually in placement order. Pass each `c.id` to `self.constructor.execute(c.id)` to build or deconstruct it. Read `c.required_item` and `c.required_count` to load the exact cargo before executing; never infer material from `c.kind`. Filter by `c.kind` to specialize a Pioneer's role, then use `c.medium` to distinguish `"gas"`, `"liquid"`, and `"power"` utility jobs. Point structures have `c.medium == None`; see `Construction.kind` for the full kind list.

RETURNS

`list[Construction]`

Snapshots of constructions awaiting a worker. Pass each `c.id` to `self.constructor.execute(c.id)` to build or deconstruct it.

`.active_constructions() → list[Construction]`

Returns blueprints currently being built (a Pioneer is working). Useful for monitor scripts, read `c.progress` to see how far along.

RETURNS

`list[Construction]`

Snapshots of constructions being built or removed right now (a Pioneer is working). Re-query for fresh `.progress`.

`.paused_constructions() → list[Construction]`

Returns blueprints started then abandoned (worker died, ran out of fuel, or script stopped). Any Pioneer can resume by navigating to `c.position` and calling `self.constructor.execute(c.id)`.

RETURNS

`list[Construction]`

Snapshots of constructions started then abandoned (the worker died or left). Resumable by any Pioneer via `self.constructor.execute(c.id)`. Re-query for fresh `.progress`.

Exotic Gas Cap

Captures gas from a cyclic exotic deposit during its active phase. Connect `self.gas_out` to a consumer, build a completed Gas Pipe route from the field Cap to that destination, then set a **0-1** release rate with `self.set_throttle(value)`. A full buffer pauses collection without losing gas.

Type	Fluids
Built on	Exotic deposits
Power in	-30 W (draws from grid)

HOW TO OBTAIN

1. The recipe unlocks with the **Exotic Fluids** research (Wildlife 1,000).
2. Fabricate **Exotic Gas Cap Kit** on a **Fabricator**: 2× Titanium Ingot and 2× Gas Pipe Segment.
3. Build it on a surveyed exotic deposit with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.gas_out: FluidPort`

Fluid output for the buffered gas. This port may declare one destination with `self.gas_out.connect("Chlorine Tank")`; additional consumers may connect their own compatible input ports to this Cap. Because the Cap is a field structure, every destination needs a compatible completed Gas Pipe component reaching the Cap and the destination. `connected_to()` reports only this port's own declaration; `flow_rate()` reports total live release. See `FluidPort`.

RETURNS

`FluidPort`

The deposit's gas. This port may declare one destination with `self.gas_out.connect("Chlorine Tank")`; additional consumers may connect their own compatible inputs to the Cap. Completed gas-pipe networks carry gas between outposts.

METHODS

`.deposit() → ExoticDeposit | None`

The `ExoticDeposit` this cap is bolted to, `.id`, `position()`, `fluid()`, `current_phase()`, cycle timing. Field availability is gated by the sonar tier that last surveyed the deposit: `"basic"` reveals phase only, `"wide"` adds rates, `"deep"` adds cycle timing. `None` if the cap isn't on a deposit. Use `deposit.current_phase()` to check whether the source is `active`. See `ExoticDeposit`.

RETURNS

`ExoticDeposit | None`

The deposit attached to this cap, or `None`. Its cycle, survey and collector methods read live state on each call: keep the object and call its methods for fresh readings. The `.surveyed` property remains a creation-time snapshot.

`.capture_rate() → float`

Exotic gas captured from the deposit on the last flow tick in t/h. **0** during the deposit's dormant phase, or when the buffer is full and holding (see `is_venting()`). Already factors in current phase and buffer headroom, read it instead of computing from the deposit's rate. Updates once per flow tick.

RETURNS

`float`

Exotic gas captured from the deposit this tick, in t/h. **0** during the deposit's dormant phase, or when the buffer is full and holding (see `is_venting()`).

`.is_venting() → bool`

`True` if active gas production exceeded the capture buffer's available space on the last flow tick. The excess is held upstream without losing gas. Dormancy returns `False` even with a full buffer, as does loss of power. This is a capture-space limit, unlike `is_stalled()`, which reports a blocked release. Open `self.set_throttle(...)` toward a tank with room.

RETURNS

`bool`

`True` if active gas production exceeded the capture buffer's available space on the last flow tick. The excess is held upstream, not lost. `False` during dormancy even if the buffer is full, or when unpowered. Open `set_throttle(...)` to a tank with room.

`.is_stalled() → bool`

`True` if, on the last flow tick, the powered cap had an open throttle and buffered gas available for release but could transfer none across its connected routes. An empty buffer, closed throttle, or lack of power does not report a stall.

RETURNS

`bool`

`True` if, on the last flow tick, the powered cap had `throttle > 0` and buffered gas available for release but could transfer none across its connected routes. `False` when unpowered, the throttle is closed, or the buffer has no gas to release.

`.throttle() → float`

Current release-valve setting, `0.0` (holding) to `1.0` (wide open). Read it back after `set_throttle(...)`.

RETURNS

`float`

Current release-valve setting (**0-1**).

`.set_throttle(t: float) → ActionResult` SELF ONLY

Open the cap's release valve from `0.0` to `1.0` (clamped). `0` holds the buffer; `1.0` releases gas across reachable connected destinations as fast as buffer supply, headroom, and throughput allow. This script-owned setpoint resets to `0` when the script stops, ends, or errors. SELF ONLY

PARAMETERS

Name	Type	Description
`t`	`float`	Release-valve fraction in the **0-1** range

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Exotic Spring Tap

Captures liquid from a cyclic exotic spring during its active phase. Connect `self.liquid_out` to a consumer, build a completed Liquid Pipe route from the field Tap to that destination, then set a **0-1** release rate with `self.set_throttle(value)`. A full buffer pauses collection without losing liquid.

Type	Fluids
Built on	Exotic deposits
Power in	-35 W (draws from grid)

HOW TO OBTAIN

1. The recipe unlocks with the **Exotic Fluids** research (Wildlife 1,000).
2. Fabricate **Exotic Spring Tap Kit** on a **Fabricator**: 2× Titanium Ingot, 2× Liquid Pipe Segment, and 1× Pressure Valve.
3. Build it on a surveyed exotic deposit with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.liquid_out: FluidPort`

Fluid output for the buffered liquid. This port may declare one destination with `self.liquid_out.connect("Cryofluid Tank")`; additional consumers may connect their own compatible input ports to this Tap. Because the Tap is a field structure, every destination needs a compatible completed Liquid Pipe network connecting the Tap and the destination. `connected_to()` reports only this port's own declaration; `flow_rate()` reports total live release. See `FluidPort`.

RETURNS

`FluidPort`

The deposit's liquid. This port may declare one destination with `self.liquid_out.connect("Cryofluid Tank")`; additional consumers may connect their own compatible inputs to the Tap. Completed liquid-pipe networks carry liquid between outposts.

METHODS

`.deposit() → ExoticDeposit | None`

The `ExoticDeposit` this tap is bolted to, `.id`, `position()`, `fluid()`, `current_phase()`, cycle timing. Field availability is gated by the sonar tier that last surveyed the deposit (`basic` / `wide` / `deep`). `None` if the tap isn't on a deposit. Use `deposit.current_phase()` to check whether the source is `active`. See `ExoticDeposit`.

RETURNS

`ExoticDeposit | None`

The deposit attached to this tap, or `None`. Its cycle, survey and collector methods read live state on each call: keep the object and call its methods for fresh readings. The `.surveyed` property remains a creation-time snapshot.

`.capture_rate() → float`

Exotic liquid captured from the spring on the last flow tick in t/h. **0** during the deposit's dormant phase, or when the buffer is full and holding (see `is_venting()`). Already factors in current phase and buffer headroom. Updates once per flow tick.

RETURNS

`float`

Exotic liquid captured from the spring this tick, in t/h. **0** during the deposit's dormant phase, or when the buffer is full and holding (see `is_venting()`).

`.is_venting() → bool`

`True` if active liquid production exceeded the capture buffer's available space on the last flow tick. The excess is held upstream without losing liquid. Dormancy returns `False` even with a full buffer, as does loss of power. This is a capture-space limit, unlike `is_stalled()`, which reports a blocked release. Open `self.set_throttle(...)` toward a tank with room.

RETURNS

`bool`

`True` if active liquid production exceeded the capture buffer's available space on the last flow tick. The excess is held upstream, not lost. `False` during dormancy even if the buffer is full, or when unpowered. Open `set_throttle(...)` to a tank with room.

`.is_stalled() → bool`

`True` if, on the last flow tick, the powered tap had an open throttle and buffered liquid available for release but could transfer none across its connected routes. An empty buffer, closed throttle, or lack of power does not report a stall.

RETURNS

`bool`

`True` if, on the last flow tick, the powered tap had `throttle > 0` and buffered liquid available for release but could transfer none across its connected routes. `False` when unpowered, the throttle is closed, or the buffer has no liquid to release.

`.throttle() → float`

Current release-valve setting, `0.0` (holding) to `1.0` (wide open). Read it back after `set_throttle(...)`.

RETURNS

`float`

Current release-valve setting (**0-1**).

`.set_throttle(t: float) → ActionResult` SELF ONLY

Open the tap's release valve from `0.0` to `1.0` (clamped). `0` holds the buffer; `1.0` releases liquid across reachable connected destinations as fast as buffer supply, headroom, and throughput allow. This script-owned setpoint resets to `0` when the script stops, ends, or errors. SELF ONLY

PARAMETERS

Name	Type	Description
`t`	`float`	Release-valve fraction in the **0-1** range

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Gas Tank

A passive buffer that latches onto the first gas piped in (steam, ammonia, swamp gas) and holds only that until it drains. Sitting between a source and its consumer, it smooths out the gaps in supply.

Type	Fluids
Storage	Any gas, 5,000 t

HOW TO OBTAIN

1. Requires the **Gas Tank** research (Pressure 2.75).
2. Buy from the Shop for 1,200 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.gas_in: FluidPort`

Connect a gas output to this input port to store its gas.

RETURNS

`FluidPort`

Accepts any gas while empty. Call `connect(...)` with the provider's stable machine id or display name; after that, the tank accepts only the first gas delivered until it is empty.

`.gas_out: FluidPort`

Connect this output port to a gas input to supply the stored gas.

RETURNS

`FluidPort`

Neutral while empty; provides the tank's latched gas to connected consumers.

`.steam_in: FluidPort`

Connect a gas output to this input port to store its gas.

RETURNS

`FluidPort`

`FluidPort` input for the tank's exact latched gas. This property exists only while `fluid()` is `"steam"`.

`.steam_out: FluidPort`

Connect this output port to a gas input to supply the stored gas.

RETURNS

`FluidPort`

`FluidPort` output for the tank's exact latched gas. This property exists only while `fluid()` is `"steam"`.

METHODS

`.fluid() → str`

The latched gas id (e.g. `"steam"`, `"ammonia"`), or `""` while empty. The tank commits to the first gas it receives and holds only that until it drains to **0**, then re-latches.

RETURNS

`str`

The latched gas id, or `""` while empty or unlatched.

POSSIBLE VALUES

`""`, `"steam"`, `"ammonia"`, `"swamp_gas"`, `"raw_sulfur_gas"`, `"sulfur_gas"`, `"raw_chlorine"`, `"chlorine"`

`.level() → float`

Current gas stored in tons, from **0** to `capacity()`. Read each iteration to gauge the buffer, near **0** means downstream is about to starve; near `capacity()` means upstream is backpressured. At **0** the tank unlatches and can accept a different gas next.

RETURNS

`float`

Gas stored now, in tons.

`.capacity() → float`

Maximum tons of gas this tank holds, queryable rather than hardcoded so tank tuning doesn't break scripts. Use with `level()` for a fill-percent indicator, or directly via `fill_pct()`.

RETURNS

`float`

Maximum gas capacity, in tons.

`.fill_pct() → float`

Fill fraction (**0.0-1.0**), shortcut for `level() / capacity()`. Use for threshold checks: `if self.fill_pct() < 0.2: # boost throttle upstream`.

RETURNS

`float`

A fraction, **0-1**.

`.inflow_rate() → float`

Gas arriving in t/h. **0** means no upstream flow, for example a dormant source, unavailable relationship, incomplete remote route, or full tank. Compare to `outflow_rate()` to see if the tank is filling or draining.

RETURNS

`float`

Gas arriving, in t/h.

`.outflow_rate() → float`

Gas leaving in t/h. **0** means no gas was delivered, for example because the tank is empty, consumers are full or not drawing, or no valid route is connected.

RETURNS

`float`

Gas leaving, in t/h.

`.is_full() → bool`

`True` when `level() == capacity()`, upstream backpressure is kicking in, and a Cap may start venting to atmosphere. Check it to detect when connected destinations cannot absorb current production.

RETURNS

`bool`

`.is_empty() → bool`

`True` when `level() == 0`; the tank is unlatched and nothing can be sent downstream. If the source is still active while the tank remains empty, inspect the source and its pipe connection.

RETURNS

`bool`

Large Liquid Tank

A big passive tank holding 1,000 t of one liquid. Like a Liquid Tank it sticks to the first fluid piped in, and only lets go once it has drained completely.

Type	Fluids
Storage	Any liquid, 1,000 t

HOW TO OBTAIN

1. Requires the **Large Liquid Tank** research (Plants 900,000).
2. Buy from the Shop for 15,000 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.liquid_in: FluidPort`

Connect a liquid output to this input port to store its liquid.

RETURNS

`FluidPort`

Accepts any liquid while empty. Call `connect(...)` with the provider's stable machine id or display name; after that, the reservoir accepts only the first liquid delivered until it is empty.

`.liquid_out: FluidPort`

Connect this output port to a liquid input to supply the stored liquid.

RETURNS

`FluidPort`

Carries the reservoir's latched liquid.

`.water_in: FluidPort`

Connect a liquid output to this input port to store its liquid.

RETURNS

`FluidPort`

`FluidPort` input for the tank's exact latched liquid. This property exists only while `fluid()` is `"water"`.

`.water_out: FluidPort`

Connect this output port to a liquid input to supply the stored liquid.

RETURNS

`FluidPort`

`FluidPort` output for the tank's exact latched liquid. This property exists only while `fluid()` is `"water"`.

METHODS

`.fluid() → str`

The latched liquid id (e.g. `"water"`, `"oil"`, `"frozen_essence"`), or `""` while empty. The tank commits to the first liquid it receives and holds only that until it drains to **0**, then re-latches.

RETURNS

`str`

The latched liquid id, or `""` while empty or unlatched.

POSSIBLE VALUES

`""`, `"water"`, `"oil"`, `"frozen_essence"`, `"coastal_essence"`, `"geothermal_essence"`, `"volcanic_essence"`, `"deep_essence"`, `"brine"`, `"raw_cryofluid"`, `"cryofluid"`, `"raw_quicksilver"`, `"quicksilver"`

`.level() → float`

Current liquid stored in tons, from **0** to `capacity()`. At **0** the tank unlatches and can accept a different liquid next.

RETURNS

`float`

Liquid stored now, in tons.

`.capacity() → float`

Maximum tons this tank holds. Queryable rather than hardcoded. Use with `level()` or `fill_pct()` for threshold checks.

RETURNS

`float`

Maximum capacity, in tons.

`.fill_pct() → float`

Fill fraction (**0.0-1.0**), shortcut for `level() / capacity()`. Common threshold in supply-control scripts.

RETURNS

`float`

A fraction, **0-1**.

`.inflow_rate() → float`

Liquid arriving in t/h. **0** = no upstream flow.

RETURNS

`float`

Liquid arriving, in t/h.

`.outflow_rate() → float`

Liquid leaving in t/h. **0** means no liquid was delivered, for example because the tank is empty, consumers are full or not drawing, or no valid route is connected.

RETURNS

`float`

Liquid leaving, in t/h.

`.is_full() → bool`

`True` when `level() == capacity()`; upstream source is backpressured.

RETURNS

`bool`

`.is_empty() → bool`

`True` when `level() == 0`; the tank is no longer bound to a liquid, so the next liquid it receives sets its contents, and downstream consumers get nothing.

RETURNS

`bool`

Liquid Tank

Passive buffer that holds any one liquid, water, oil, or a biome essence. It commits to the first liquid piped in.

Type	Fluids
Storage	Any liquid, 100 t

HOW TO OBTAIN

1. Requires the **Liquid Tank** research (Oxygen 400).
2. Buy from the Shop for 900 cr.

Access via: `get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.liquid_in: FluidPort`

Connect a liquid output to this input port to store its liquid.

RETURNS

`FluidPort`

Accepts any liquid while empty. Call `connect(...)` with the provider's stable machine id or display name; after that, the tank accepts only the first liquid delivered until it is empty.

`.liquid_out: FluidPort`

Connect this output port to a liquid input to supply the stored liquid.

RETURNS

`FluidPort`

Neutral while empty; provides the tank's latched liquid to connected consumers.

`.water_in: FluidPort`

Connect a liquid output to this input port to store its liquid.

RETURNS

`FluidPort`

`FluidPort` input for the tank's exact latched liquid. This property exists only while `fluid()` is `"water"`.

`.water_out: FluidPort`

Connect this output port to a liquid input to supply the stored liquid.

RETURNS

`FluidPort`

`FluidPort` output for the tank's exact latched liquid. This property exists only while `fluid()` is `"water"`.

METHODS

`.fluid() → str`

The latched liquid id (e.g. `"water"`, `"oil"`, `"frozen_essence"`), or `""` while empty. The tank commits to the first liquid it receives and holds only that until it drains to **0**, then re-latches.

RETURNS

`str`

The latched liquid id (e.g. `"water"`, `"oil"`, `"frozen_essence"`), or `""` while empty or unlatched.

POSSIBLE VALUES

`""`, `"water"`, `"oil"`, `"frozen_essence"`, `"coastal_essence"`, `"geothermal_essence"`, `"volcanic_essence"`, `"deep_essence"`, `"brine"`, `"raw_cryofluid"`, `"cryofluid"`, `"raw_quicksilver"`, `"quicksilver"`

`.level() → float`

Current liquid stored in tons, from **0** to `capacity()`. At **0** the tank unlatches and can accept a different liquid next.

RETURNS

`float`

Liquid stored now, in tons.

`.capacity() → float`

Maximum tons this tank holds. Queryable rather than hardcoded. Use with `level()` or `fill_pct()` for threshold checks.

RETURNS

`float`

Maximum capacity, in tons.

`.fill_pct() → float`

Fill fraction (**0.0-1.0**), shortcut for `level() / capacity()`. Common threshold in supply-control scripts.

RETURNS

`float`

A fraction, **0-1**.

`.inflow_rate() → float`

Liquid arriving in t/h. **0** = no upstream flow.

RETURNS

`float`

Liquid arriving, in t/h.

`.outflow_rate() → float`

Liquid leaving in t/h. **0** means no liquid was delivered, for example because the tank is empty, consumers are full or not drawing, or no valid route is connected.

RETURNS

`float`

Liquid leaving, in t/h.

`.is_full() → bool`

`True` when `level() == capacity()`; upstream source is backpressured.

RETURNS

`bool`

`.is_empty() → bool`

`True` when `level() == 0`; the tank is no longer bound to a liquid, so the next liquid it receives sets its contents, and downstream consumers get nothing.

RETURNS

`bool`

Oil Pump

Extracts oil from a surveyed well at a throttle your script sets. Oil wells run in active and dormant phases, so buffer the output through a Liquid Tank to ride out the dry spells.

MK I

Type	Fluids
Built on	Oil wells
Power in	Up to -5 W at full throttle (draws from grid)
Produces	Oil, buffer 10 t
Tiers	Mk II

MK II

Power in	Up to -100 W at full throttle (draws from grid)

HOW TO OBTAIN

1. The recipe unlocks with the **Petroleum Survey** research (Oxygen 1,500).
2. Fabricate **Oil Pump** on a **Fabricator**: 2× Iron Ingot, 1× Titanium Ingot, 1× Pressure Valve, and 1× Circuit Panel.
3. Build it on a surveyed oil well with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.oil_out: FluidPort`

Fluid output for oil. This port may declare one destination with `self.oil_out.connect("Oil Reserve")`; additional consumers may connect their own `oil_in` ports to this Pump. Because the Pump is a field structure, every destination needs a compatible completed Liquid Pipe network connecting the Pump and the destination. `connected_to()` reports only this output port's declaration; `flow_rate()` reports total live delivery. The Pump stores nothing, so `level()` reads **0**. See `FluidPort`.

RETURNS

`FluidPort`

Routes oil to a connected target. Call `connect(...)` with the target's stable machine id or display name, then call `set_throttle(...)` with a value above **0**. Completed liquid-pipe networks carry oil between outposts.

METHODS

`.tier() → int`

Installed machinery tier: `1` for Mk I, `2` for Mk II, or `3` for Mk III where supported.

RETURNS

`int`

`.well() → OilWell`

The `OilWell` this pump is bolted to. Same shape as `WaterWell`, yield tier (1×/2×/3×) and base flow rate.

RETURNS

`OilWell`

A snapshot of the well this pump is on.

`.pump_rate() → float`

Current total oil delivery rate across every reachable connected destination, in t/h, measured this tick. **0** when throttle is **0**, the well is dormant, or no destination can accept flow. See `is_stalled()` to tell a routing block from a dormant well.

RETURNS

`float`

Current total oil delivery rate across every reachable connected destination, in t/h, measured this tick. **0** when throttle is **0**, the well is dormant, or no destination can accept flow.

`.well_active() → bool`

Reads the well's pulse. `True`, the well below is in its active phase and delivers at full rate. `False`, dormant: no oil at any throttle, typically for several hours. Bank oil in a downstream Liquid Tank and throttle down during the gap to save watts.

RETURNS

`bool`

`True` while the well below is in its active phase. Dormant wells deliver nothing at any throttle: bank oil in a downstream Liquid Tank to ride the gap.

`.is_stalled() → bool`

`True` if, on the last flow tick, the powered pump had an open throttle and oil available from its active well but could transfer none across its connected routes. Dormancy, a closed throttle, or lack of power does not report a stall. Declare a destination with `self.oil_out.connect(...)`, or let consumers connect their own `oil_in` ports to this Pump.

RETURNS

`bool`

`True` if, on the last flow tick, the powered pump had `throttle > 0` and oil available from its active well but could transfer none across its connected routes. `False` when unpowered, the throttle is closed, or the well supplies no oil, including during dormancy.

`.throttle() → float`

Current throttle setting (**0-1**). **0** by default.

RETURNS

`float`

The throttle setting (**0-1**).

`.set_throttle(rate: float) → ActionResult` SELF ONLY

Set the pump's total output rate (**0-1**) across reachable connected destinations. `0` idles the pump; `1` allows full active-well output subject to headroom and throughput. This script-owned setpoint resets to `0` when the script stops, ends, or errors, so keep the control loop running while the Pump should operate.

PARAMETERS

Name	Type	Description
`rate`	`float`	Throttle setting (**0-1**).

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Outpost

Represents a home or player-founded outpost. Look one up by stable id with `get_component("outpost_1")` or by display name with `get_component_by_name("Mining Camp")`. Players can rename outposts from the Computer System tab, so use the id for scripts that must survive renames.

Access via: `get_component("outpost")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Immutable instance id (e.g. `"outpost_home"`, `"outpost_1"`). Use this in scripts that need to outlive renames.

RETURNS

`str`

Immutable instance id, e.g. `"outpost_home"` or `"outpost_1"`.

METHODS

`.name() → str`

Display name. The home outpost starts as `"Nocturna Base"`; player-founded outposts start as `"Outpost N"`. Freely renameable from the Computer System tab. Mutable, prefer `id` for stable references.

RETURNS

`str`

The outpost's display name, freely renameable.

`.coords() → list[int]`

World coordinates as `[x, y]`. Home outpost sits at `[0, 0]`.

RETURNS

`list[int]`

`[x, y]`: world coordinates of the outpost anchor.

`.buildings_used() → int`

Number of buildings deployed at this outpost. Sensors, mobile units, structural hubs, and POI extraction don't count.

RETURNS

`int`

Buildings deployed at this outpost.

`.buildings_capacity() → int`

Soft building threshold at this outpost. Each counted building above it reduces productive and service throughput. Nocturna Base has a few extra starter slots; player-founded outposts use the standard threshold.

RETURNS

`int`

Soft building threshold at this outpost.

`.is_full() → bool`

Returns `True` when this outpost has reached or exceeded its soft building threshold. The threshold itself does not block ordinary deployment.

RETURNS

`bool`

`True` if the soft building threshold is reached or exceeded.

`.is_home() → bool`

Returns `True` when this is the home outpost.

RETURNS

`bool`

`True` if this is the home outpost.

`.buildings(type_id?: str) → list[BuildingRef]`

Lists buildings deployed at this outpost as `BuildingRef` snapshots. Call `self.outpost.buildings()` for all of them or pass a building `type_id` to filter. Each ref has `.id`, `.name`, `.type_id`, `.outpost`, `.powered`, and `.position`; use `get_component(ref.id)` for type-specific live reads. The list covers machines that count toward building capacity, not sensors, mobile units, POI extraction, or fixed Harvesting-field machines. Use `harvesting_machines()` for those field machines.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional building type id used to filter the returned list.

RETURNS

`list[BuildingRef]`

One per building deployed here. Pass `type_id` (e.g. `"storage_bin"`) to filter, or omit it for every building. Empty if no buildings match.

`.harvesting_machines(type_id?: str) → list[HarvestingMachineRef]`

Lists fixed machines deployed on this outpost's Harvesting field as `HarvestingMachineRef` snapshots. Call without an argument for every Grow Lamp, Sprinkler, Dispenser, and Crop Automator there, or pass one of those `type_id` values to filter. They occupy field cells, do not use building capacity, and do not appear in `buildings()`. This excludes the mobile Harvester, crops, loose items, Water Wells, and other POI extractors. Only Nocturna Base currently has a Harvesting field, so founded outposts return an empty list. Use `get_component(ref.id)` for type-specific live reads.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional fixed Harvesting-field machine type id used to filter the returned list.

RETURNS

`list[HarvestingMachineRef]`

Snapshots of the fixed machines on this outpost's Harvesting field. Pass `type_id` to filter, or omit it for Grow Lamps, Sprinklers, Dispensers, and Crop Automators. Only Nocturna Base has this field; founded outposts return an empty list.

Outpost Network

Read-only index of every owned outpost, including home. Use it for routing, deployment planning, capacity dashboards, and nearest-service decisions without hardcoding `outpost_1`, `outpost_2`, etc. An outpost's `.x` and `.y` identify its footprint anchor. For an at-building action, select that outpost's `BuildingRef` and route to `.position`. Construction planning belongs to Plan Mode and the shared `construction_blueprint` component; physical work belongs to Constructor scripts.

Access via: `get_component("outpost_network")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.outposts() → list[OutpostRef]`

All owned outposts as `OutpostRef` snapshots. Each ref includes `.id`, `.name`, `.x`, `.y`, `.is_home`, `.buildings_used`, `.buildings_capacity`, and `.is_full`. The coordinates are the top-left footprint anchor, not a particular building's docking point.

RETURNS

`list[OutpostRef]`

Snapshots. Re-query for fresh names and building counts.

`.home() → OutpostRef`

The home outpost as an `OutpostRef`.

RETURNS

`OutpostRef`

A snapshot.

`.nearest(x: float, y: float) → OutpostRef`

Nearest owned outpost to the given world coordinate. Useful before routing a vehicle to the nearest outpost to recharge or choosing where a constructor should stage.

PARAMETERS

Name	Type	Description
`x`	`float`	World X coordinate
`y`	`float`	World Y coordinate

RETURNS

`OutpostRef`

A snapshot of the nearest outpost.

Power Control

Discover every independent power grid, inspect connected outposts, buildings, and field power structures, read generation, consumption, battery charge, and Lightning reserve, or operate machine breakers from one shared controller. Grid objects are snapshots of the latest completed power allocation. After changing a breaker or rewiring infrastructure, re-query on the next loop iteration for refreshed grid totals and membership.

Access via: `get_component("power_control")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.grids() → list[PowerGrid]`

Returns every independent power grid on the planet as a fresh list of `PowerGrid` snapshots. Isolated completed outposts and field structures appear as their own grids, so scripts do not need to guess or hardcode grid ids.

RETURNS

`list[PowerGrid]`

A fresh snapshot of every independent grid on the planet.

`.grid(target_id: str) → PowerGrid | None`

Finds the grid containing `target_id`, which may be an outpost, building, or field power-structure id. Pass a returned grid's `.anchor_id` to look it up again. Returns `None` for an unknown, mobile, under-construction, non-grid, or currently unmapped target.

PARAMETERS

Name	Type	Description
`target_id`	`str`	Outpost, building, or field power-structure instance id

RETURNS

`PowerGrid | None`

`NONE` MEANS

`None` means the target is unknown, mobile, under construction, not a building or field power structure, or not part of a completed power grid.

OUTCOMES

Status	Meaning
`None`	`None` means the target is unknown, mobile, under construction, not a building or field power structure, or not part of a completed power grid.

`.total() → PowerSummary`

Returns a planet-wide `PowerSummary` across every independent grid. Conventional battery storage and Lightning reserve remain separate so automation can decide which supply it is relying on.

RETURNS

`PowerSummary`

A planet-wide snapshot across every independent grid.

`.is_powered(machine_id: str) → bool`

Returns `True` when the named machine is currently switched on. Unknown machine ids return `False`, so this is safe to call before deciding whether to send a breaker command.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`bool`

`True` when the named machine is powered on.

`.can_power_off(machine_id: str) → bool`

Returns `True` when the named machine exists and has a visible breaker toggle. Terraforming machines and many production machines can usually be switched; batteries, passive tanks, mobile units, and ship equipment usually cannot.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`bool`

`True` when the named machine exposes a breaker/power toggle.

`.set_powered(machine_id: str, on: bool) → ActionResult`

Send the same breaker command as clicking the machine card toggle. `power.set_powered("o2gen_1", False)` switches a machine off; `True` switches it back on. Switching off pauses scripts attached to that machine and preserves their setpoints; switching back on resumes only scripts that were paused by the power-off.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id
`on`	`bool`	`True` powers on; `False` powers off

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"not_toggleable"`	rejection	The target does not expose a script power toggle.
`"under_construction"`	transient	The target is still under construction.
`"not_connected"`	rejection	The component has no active connection.
`"not_enough_power"`	rejection	The available energy is below the operation's requirement.

```
power = get_component("power_control")
for grid in power.grids():
    print(grid.anchor_id, grid.generated, grid.consumed, grid.net)
    for member in grid.members:
        print("  ", member.id, member.roles, member.generated, member.consumed)
```

Run Control

Inspect machine scripts, discover and apply saved variants, and control execution remotely. Shared start/stop controller for machine scripts, the remote equivalent of a machine card's Run / Stop buttons and the script window's Pause / Resume. Use it to build a supervisor: one script that watches the base and shuts down another machine when it detects a fault, without parking that machine in a permanent `sleep` loop. This is the **run/stop axis**, separate from `power_control` (the breaker): `stop` ends a script and latches it off, `pause` holds it at its current line until `resume`, while a power toggle only pauses and auto-resumes.

Access via: `get_component("run_control")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.variants(machine_id: str, slot: int = 0) → list[ScriptVariantRef]`

Lists the saved variants available to one machine script, including its private `Main` and compatible shared variants. The optional `slot` is a zero-based script slot and defaults to 0. Results are snapshots; applying an id loads the latest saved code under that id.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id
`slot`	`int`	Zero-based script slot. Defaults to 0. Must be a nonnegative whole number.

RETURNS

`list[ScriptVariantRef]`

RAISES

Exception	Condition
`ReferenceError`	The machine or an executable script in the requested slot does not exist.
`TypeError`	`slot` must be a whole number.
`ValueError`	`slot` must be zero or greater.
`OverflowError`	`slot` is outside the supported integer range.

`.status(machine_id: str, slot: int = 0) → RunControlStatus`

Inspect a machine script's execution state and assigned variant. The optional `slot` is a zero-based script slot and defaults to 0. Results are snapshots; call again for current information.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id
`slot`	`int`	Zero-based script slot. Defaults to 0. Must be a nonnegative whole number.

RETURNS

`RunControlStatus`

RAISES

Exception	Condition
`ReferenceError`	The machine or an executable script in the requested slot does not exist.
`TypeError`	`slot` must be a whole number.
`ValueError`	`slot` must be zero or greater.
`OverflowError`	`slot` is outside the supported integer range.

`.apply_variant(machine_id: str, variant_id: str, slot: int = 0) → ActionResult`

Apply a saved variant to one machine script without starting it. The target must be fully built and its script must be stopped, completed, or errored. Running and paused scripts must be stopped first. Pending editor changes and unresolved conflicts block replacement. `Main` is preserved when switching away from it. Shared variants copy their current code into the target; later edits elsewhere do not automatically update it. The optional `slot` defaults to 0.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id
`variant_id`	`str`	Exact id returned by `run.variants()` for a compatible target.
`slot`	`int`	Zero-based script slot. Defaults to 0. Must be a nonnegative whole number.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The variant was applied. The script remains stopped.
`"not_found"`	● rejection	The target machine does not exist.
`"no_script"`	● rejection	The target slot has no executable machine script.
`"under_construction"`	● rejection	The target machine is still under construction.
`"script_running"`	● rejection	The target script is running.
`"script_paused"`	● rejection	The target script is paused.
`"editor_busy"`	● transient	An editor has pending changes or a conflicting source version is unresolved.
`"variant_not_found"`	● rejection	The variant is no longer available under that id.
`"incompatible_variant"`	● rejection	The variant belongs to a different script or an incompatible machine catalog.
`"source_too_large"`	● rejection	The variant source exceeds the supported script size.

RAISES

Exception	Condition
`TypeError`	`slot` must be a whole number.

Exception	Condition
`ValueError`	`slot` must be zero or greater.
`OverflowError`	`slot` is outside the supported integer range.

`.is_running(machine_id: str) → bool`

Returns `True` when the named machine has a script actively scheduled, including while it sits mid-`sleep` or mid-action. A paused, stopped, completed, or errored script reads `False`, as does an unknown machine id. Call it before `start` / `stop` to avoid redundant commands.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`bool`

`True` when the named machine has a script actively scheduled (running, including mid-`sleep`/mid-action). A paused, stopped, completed, or errored script reads `False`.

`.stop(machine_id: str) → ActionResult`

Stops the named machine's script the same way the card's Stop button does: `run.stop("o2gen_1")` ends the script, resets its setpoints to idle, and zeroes its live readouts. Structural state (recipes, in-flight progress, loaded materials) is preserved. The stop is **latched**, unlike a power-off, the script does not auto-resume; restart it with `start`.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"no_script"`	● rejection	No attached script could be started or stopped by this call.

`.start(machine_id: str) → ActionResult`

Runs the named machine's script from the top, the same way the card's Run button does: `run.start("o2gen_1")`. A fresh run restarts from the first line; it does not resume mid-script. The machine must be powered and fully built.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"no_script"`	rejection	No attached script could be started or stopped by this call.
`"already_running"`	transient	The operation is already running.
`"not_powered"`	rejection	The component is not powered.
`"under_construction"`	transient	The target is still under construction.

`.pause(machine_id: str) → ActionResult`

Pauses the named machine's running script where it is: `run.pause("rover_1")`. A paused script keeps its place and its setpoints, so a vehicle already driving finishes the move it was making. It stays paused through power cuts until `resume`, and pausing a machine whose power is off keeps its script paused when power returns. A script can pause its own machine.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The machine's script is paused.
`"already_paused"`	● success	The machine's script was already paused.
`"not_found"`	● rejection	The target machine does not exist.
`"no_script"`	● rejection	The machine has no running or paused script.

`.resume(machine_id: str) → ActionResult`

Continues the named machine's paused script from the line where it paused: `run.resume("rover_1")`, whether `pause` or the script window paused it. The machine must be powered and fully built. A save does not keep a script's place: after loading, a script that was paused when the game was saved stays paused, and resuming it starts again from its first line.

PARAMETERS

Name	Type	Description
`machine_id`	`str`	Machine instance id

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The machine's script is running again.
`"already_running"`	success	The machine's script was already running.
`"not_found"`	rejection	The target machine does not exist.
`"no_script"`	rejection	The machine has no paused or running script.
`"not_powered"`	rejection	The machine is powered off.
`"under_construction"`	rejection	The target machine is still under construction.

```
run = get_component("run_control")

if run.is_running("o2gen_1"):
    run.stop("o2gen_1")    # latched off , won't auto-resume on power
else:
    run.start("o2gen_1")   # re-run the machine's script from the top

# Select saved code, then explicitly start it.
for variant in run.variants("o2gen_1"):
    if variant.name == "Production":
        run.stop("o2gen_1")
        result = run.apply_variant("o2gen_1", variant.id)
        if result.status == "ok":
            run.start("o2gen_1")
        break
```

Ship Computer

Manages the hardware roster: deploy a machine, vehicle, or drone from Inventory into an outpost, upgrade a deployed machine, remove one back to Inventory, decommission an emptied outpost, and rename anything you own. These are the Inventory page's Deploy and Upgrade buttons and the Computer's System tab, reached from a script. Every call needs Ship Computer research; the buttons themselves keep working before it.

Access via: `get_component("computer")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.deploy(item_id: str, outpost: str | Outpost | None = None) → ComputerDeployResult`

Deploy one unit of an inventory item into an outpost, defaulting to home. The machine lands with no script and does nothing until you attach one, exactly as a hand-placed machine does. A machine that the outpost's subnet cannot yet carry lands powered off.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Inventory item id of the machine, vehicle, or drone kit to deploy.
`outpost`	`str` | `Outpost` | `None`	Target outpost id, name, or `Outpost`. Omit for the home outpost.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ComputerDeployResult`	`.status`, `.message`	`.machine_id`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Deployed `name` at `location`.
`"no_kit"`	● rejection	Inventory holds no `item_id` to deploy.
`"locked"`	● rejection	Deploying from a script needs Ship Computer research, or the machine's own research is not yet complete.
`"not_deployable"`	● rejection	That item is not a deployable machine.
`"deploy_limit"`	● rejection	The deployed count for this machine type is already at its maximum.
`"location_not_found"`	● rejection	The requested outpost does not exist or is not operational.
`"wrong_biome_for_machine"`	● rejection	This machine only operates in another biome.
`"duplicate_outpost_machine"`	● rejection	This outpost already has a machine of this type, and only one is allowed.
`"missing_drone_station"`	● rejection	The target outpost has no Drone Depot to assemble a drone at.
`"drone_station_full"`	● rejection	Every Drone Depot bay at the target outpost is occupied.

`.upgrade(item_id: str, machine: str | Component) → ActionResult`

Upgrade a deployed machine with one item from Inventory. An upgrade pack raises the machine to the pack's tier. A larger deploy kit upgrades a smaller machine of the same family in place: the machine keeps its id, name, cargo, scripts, and connections, and its old kit returns to Inventory. The machine must be finished and part of an operational outpost, and a running machine's subnet must carry the larger kit's extra draw. These are the Inventory page's UPGRADE and UPGRADE EXISTING buttons, reached from a script.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Inventory item id of the upgrade pack, or of the larger kit that replaces a smaller machine in place.
`machine`	`str` | `Component`	Machine id, name, or component to upgrade.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"not_upgrade_item"`	rejection	The item is not an upgrade pack or a kit that upgrades a machine in place.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"under_construction"`	transient	The target is still under construction.
`"not_at_outpost"`	rejection	The machine is not part of an operational outpost.
`"item_not_in_inventory"`	rejection	Inventory does not contain the requested item.
`"wrong_machine_type"`	rejection	The upgrade does not fit the machine's type.
`"tier_too_high"`	rejection	The machine is already at or above the upgrade's tier.
`"tier_not_ready"`	rejection	The machine is below the tier the upgrade applies to.
`"not_enough_power"`	rejection	The available energy is below the operation's requirement.
`"inventory_full"`	rejection	Inventory has no capacity for the result.

`.undeploy(machine: str | Component) → ActionResult`

Remove a deployed machine, vehicle, or drone and return its kit, mounted modules, contained items, and tier upgrade packs to Inventory. Stored cargo blocks removal, so empty it first. Authored scripts survive as detached records. Field equipment is recovered by its Harvester and map structures by a Pioneer, not here.

PARAMETERS

Name	Type	Description
`machine`	`str` | `Component`	Machine id, name, or component to remove. Its hardware returns to Inventory.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"not_undeployable"`	● rejection	This machine type is not removed through the Computer. Field equipment is recovered by the Harvester and map structures by a Pioneer.
`"self_target"`	● rejection	A machine's own script cannot remove the machine it runs on.
`"cargo_present"`	● rejection	Existing cargo prevents the requested configuration change.
`"docked_drone"`	● rejection	A drone is still assigned to this station.
`"construction_dependency"`	● rejection	Another construction job depends on this job.
`"inventory_full"`	● rejection	Inventory has no capacity for the result.

`.decommission(outpost: str | Outpost) → ActionResult`

Remove a founded outpost and return its Outpost Kit to Inventory. The outpost must hold no machines; nothing is cascade-destroyed. Drones and vehicles don't belong to an outpost, so they never block it. Pipes, power lines, and bridges that touched it stay on the map for a Pioneer to reclaim. The home outpost is permanent.

PARAMETERS

Name	Type	Description
`outpost`	`str` | `Outpost`	Outpost id, name, or `Outpost` to remove. It must hold no machines.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"locked"`	● rejection	The required feature, recipe, or operation is locked.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"is_home"`	● rejection	The home outpost is permanent.
`"not_empty"`	● rejection	The relevant cell, slot, or component is not empty.
`"construction_dependency"`	● rejection	Another construction job depends on this job.
`"inventory_full"`	● rejection	Inventory has no capacity for the result.

`.rename(target: str | Component | Outpost, name: str) → ActionResult`

Set the display name of a machine or outpost. Names are unique across every machine, outpost, and panel. Ids never change, so saved references keep working.

PARAMETERS

Name	Type	Description
`target`	`str` | `Component` | `Outpost`	Machine or outpost id, name, or component to rename.
`name`	`str`	New display name. Must be unique across machines, outposts, and panels.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"name_empty"`	rejection	A display name cannot be blank.
`"name_too_long"`	rejection	The display name exceeds the maximum length.
`"name_taken"`	rejection	Another machine, outpost, or panel already uses that name.

Steam Condenser

Converts incoming steam into clean water at a 1:1 mass ratio. Scripted throttle controls its 250 t/h peak and 150 W draw.

Type	Fluids
Power in	Variable (draws from grid)
Consumes	Steam, up to 250 t per hour, buffer 250 t
Produces	Water, up to 250 t per hour, buffer 250 t

HOW TO OBTAIN

1. Requires the **Steam Condensation** research (Plants 1,000,000).
2. Buy from the Shop for 50,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.steam_in: FluidPort`

Steam input port. Connect a Thermal Cap or a steam-latched Gas Tank. Its 250 t internal buffer is consumed by condensation after the grid powers the machine.

RETURNS

`FluidPort`

Accepts steam from a Thermal Cap or Gas Tank.

`.water_out: FluidPort`

Clean-water output port. Connect a Liquid Tank, Large Liquid Tank, Plant Terraformer, Sprinkler, or other water consumer. Its 250 t internal buffer backpressures condensation when full.

RETURNS

`FluidPort`

Supplies the condensed clean water.

METHODS

`.condensation_rate() → float`

Clean water produced on the last simulation tick in t/h. Full throttle reaches **250 t/h** when the steam input has supply, the water output has room, and the host outpost is not overcrowded.

RETURNS

`float`

Clean water produced on the last tick, in t/h.

`.efficiency() → float`

Fraction of the throttle's requested condensation completed on the last tick (**0.0-1.0**). Low values mean the steam input ran short or the water output filled before the tick completed.

RETURNS

`float`

Fraction of requested condensation completed (**0-1**).

`.is_stalled() → bool`

`True` when throttle is above zero and the current fluid state blocks condensation because `steam_in` is empty or `water_out` is full. This is derived immediately from both ports; use `status()` to distinguish the blockers.

RETURNS

`bool`

`True` when the condenser is fully built, powered, and has an open throttle, but steam is empty or the water buffer is full. `False` when the throttle is closed, power is unavailable, or construction is unfinished.

`.status() → str`

Current actionable state: `"idle"`, `"no_power"`, `"no_steam"`, `"output_full"`, or `"running"`. This is derived live from throttle, power, and both fluid buffers. Once throttle is **0**, it reports `"idle"`; inspect port levels to decide when to reopen it.

RETURNS

`str`

POSSIBLE VALUES

`"idle"`, `"no_power"`, `"no_steam"`, `"output_full"`, `"running"`

`.throttle() → float`

Current condensation setpoint (**0.0-1.0**). It scales steam use, water output, and power draw linearly.

RETURNS

`float`

Current condensation setpoint (**0-1**).

`.set_throttle(t: float) → ActionResult` SELF ONLY

Set condensation from **0.0-1.0** (clamped). **0** idles with no conversion or variable draw. **1.0** requests **250 t/h** and **150 W**. Draw follows the throttle even when steam is empty or the output is full, so set **0** to save power while blocked. This script-owned setpoint resets to **0** when the script stops, ends, or errors. Call from this Condenser's own script.

PARAMETERS

Name	Type	Description
`t`	`float`	Condensation rate fraction in the **0-1** range.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Thermal Cap

Captures Steam from a thermal vent. If its chamber reaches 100%, every stored ton blows into the atmosphere; a script must release, route, or relieve pressure.

Type	Fluids
Built on	Thermal vents
Power in	-3 W (draws from grid)
Produces	Steam, buffer 1,000 t

HOW TO OBTAIN

1. The recipe unlocks with the **Thermal Cap** research (Pressure 2.5).
2. Fabricate **Thermal Cap Kit** on a **Fabricator**: 2× Titanium Ingot and 2× Gas Pipe Segment.
3. Build it on a surveyed thermal vent with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.steam_out: FluidPort`

Fluid output for the chamber's steam. This port may declare one destination with `self.steam_out.connect("bio_caster_1")`; additional consumers may connect their own `steam_in` ports to this Cap. Because the Cap is a field structure, every destination needs a compatible completed Gas Pipe component reaching the Cap and the destination. `connected_to()` reports only this output port's own declaration; `flow_rate()` reports total live release. See `FluidPort`.

RETURNS

`FluidPort`

Chamber steam. This port may declare one destination with `connect("bio_caster_1")`; additional consumers may connect their own `steam_in` ports to the Cap. Completed gas-pipe networks carry steam between outposts.

METHODS

`.vent() → ThermalVent`

The `ThermalVent` this cap sits on. Field availability follows the sonar tier that last surveyed the vent: `"basic"` reveals phase, `"wide"` adds steam rates, `"deep"` adds cycle timing. Use `self.vent().current_phase()` (or the cap's own `phase()`) to know when steam is coming. See `ThermalVent`.

RETURNS

`ThermalVent`

The vent this cap sits on. Read `current_phase()` / `next_phase_in()` / rates off it, or use the cap's own `phase()` / `next_phase_in()`.

`.phase() → str | None`

`"active"` while the vent produces steam (your chamber fills) or `"dormant"` while it rests (the chamber only drains). `None` until the vent is surveyed. Drive your release loop off this: open the throttle when active, ease it when dormant so you do not run downstream dry.

RETURNS

`str | None`

`"active"` while the vent produces steam (chamber fills), `"dormant"` while it rests. `None` until the vent is surveyed. Ease the throttle in dormancy so you do not run downstream dry.

POSSIBLE VALUES

`"active"`, `"dormant"`

`.next_phase_in() → float | None`

Game-minutes until the vent flips between active and dormant, so you can open up before a surge or ease off before a dry spell. Returns `None` unless the vent was **Deep**-surveyed, so a predictive loop is the payoff for deep sonar.

RETURNS

`float | None`

Game-minutes until the vent flips phase, to pre-empt a surge or a dry spell. `None` unless the vent was Deep-surveyed.

`.pressure() → float`

Chamber fill from `0.0` to `1.0`. It climbs while the cap captures steam from the vent and drops as you release through `steam_out`. Hit `1.0` and the cap **overpressurizes**: the whole chamber blows off to atmosphere and rebuilds from empty. The job is keeping this off the ceiling, so poll it every tick and open the throttle as it rises.

RETURNS

`float`

Chamber fill from `0.0` to `1.0`. Climbs as the vent captures, drops as you release. At `1.0` it overpressurizes: the whole chamber blows off to atmosphere and refills from empty. Keep it off the ceiling.

`.capture_rate() → float`

Steam captured from the vent on the last tick, in t/h. **0** during the dormant phase, up to the vent's current output while active. Already factors in the vent's phase, so read it instead of computing from the vent rate. Updates once per flow tick.

RETURNS

`float`

Steam captured from the vent this tick, t/h. `0` while the vent is dormant.

`.is_overpressured() → bool`

`True` the tick the chamber tops out and blows its whole contents to atmosphere. After that the chamber is empty and must refill from the vent before you get steam again, so everything you had banked is gone. If you see this, you released too slowly, open the throttle sooner.

RETURNS

`bool`

`True` the tick the chamber tops out and blows off to atmosphere. You then rebuild from empty, losing everything banked. Release sooner next time.

`.is_stalled() → bool`

`True` if, on the last flow tick, the powered cap had an open throttle and chamber steam available for release but could transfer none across its connected routes. An empty chamber, closed throttle, or lack of power does not report a stall. Declare a destination with `self.steam_out.connect(...)`, or let consumers connect their own `steam_in` ports to this Cap.

RETURNS

`bool`

`True` if, on the last flow tick, the powered cap had `throttle > 0` and chamber steam available for release but could transfer none across its connected routes. `False` when unpowered, the throttle is closed, or the chamber has no steam to release.

`.throttle() → float`

The current release-valve setting, `0.0` (sealed) to `1.0` (wide open). Read it back after `set_throttle(...)`.

RETURNS

`float`

Current release-valve setting from `0.0` to `1.0`.

`.set_throttle(t: float) → ActionResult` SELF ONLY

Open the cap's release valve from `0.0` to `1.0` (clamped). `0` seals the chamber so it fills; `1.0` releases steam across all reachable connected destinations as fast as chamber supply, destination headroom, and throughput allow. Your primary knob against overpressure, so call it every tick against `pressure()`. This script-owned setpoint resets to `0` when the script stops, ends, or errors. If no destination can accept enough and `pressure()` still climbs, use `set_relief(...)` to shed the surplus. SELF ONLY

PARAMETERS

Name	Type	Description
`t`	`float`	Release-valve fraction in the **0-1** range

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.relief() → float`

The current relief-valve setting, `0.0` (shut) to `1.0` (wide open). Read it back after `set_relief(...)`.

RETURNS

`float`

Current relief-valve setting from `0.0` to `1.0`: how wide the atmosphere overflow is open.

`.relief_rate() → float`

Steam wasted to atmosphere through the relief valve this tick, in t/h. `0` when the relief valve is shut. Watch it to see how much surplus you're dumping.

RETURNS

`float`

Steam wasted to atmosphere through the relief valve this tick, t/h. `0` when the valve is shut.

`.set_relief(t: float) → ActionResult` SELF ONLY

Open the relief valve from **0-1** to dump excess chamber steam into the atmosphere. Use it when connected consumers cannot keep up and `pressure()` is still climbing. `0` keeps all steam available for consumers. Values outside the range are clamped. It returns to **0** when the script stops, ends, or errors. Call this only from the Thermal Cap's own script.

PARAMETERS

Name	Type	Description
`t`	`float`	Relief-valve fraction in the **0-1** range

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Water Pump

Extracts water from a surveyed well at a throttle your script sets. Its Planet Map blueprint can be placed in Plan Mode or by script; a Pioneer must still build it on the well.

Type	Fluids
Built on	Water wells
Power in	Up to -4 W at full throttle (draws from grid)
Produces	Water, buffer 10 t
Output buffer	Units: 20

HOW TO OBTAIN

1. The recipe unlocks with the **Hydrology Survey** research (Pressure 18).
2. Fabricate **Water Pump** on a **Fabricator**: 2× Iron Ingot, 2× Glass, and 4× Liquid Pipe Segment.
3. Build it on a surveyed water well with a Pioneer's Constructor.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.output: PickupOutputSlot`

`PickupOutputSlot` holding salt created as a water-pumping byproduct. This is separate from `water_out`, which carries water through Liquid Pipes. Read `count()`, `capacity()`, or `stacks()` to inspect it. A physically present Rover or Pioneer connects its own `input` and pulls salt; the Pump has no direct item-routing methods.

RETURNS

`PickupOutputSlot`

Holds the discrete salt byproduct for a physically present Rover or Pioneer.

`.water_out: FluidPort`

Fluid output for water. This port may declare one destination with `self.water_out.connect("bio_caster_1")`; additional consumers may connect their own `water_in` ports to this Pump. Because the Pump is a field structure, every destination needs a completed Liquid Pipe route reaching the Pump and destination. `connected_to()` reports only this output port's declaration; `flow_rate()` reports total live delivery. The Pump stores nothing, so `level()` reads **0**. See `FluidPort`.

RETURNS

`FluidPort`

Routes water to a connected target. Call `connect(...)` with the target's stable machine id or display name, then call `set_throttle(...)` with a value above **0**. Completed liquid-pipe networks carry water between outposts.

METHODS

`.well() → WaterWell`

The `WaterWell` this pump is bolted to. Read `.yield_tier()` to see whether the well is `"standard"` / `"rich"` / `"pure"` (1×/2×/3× multiplier) and `.flow_rate()` for the well's per-hour output. Useful for prioritization scripts that compare yields across the fleet.

RETURNS

`WaterWell`

A snapshot of the well this pump is on.

`.pump_rate() → float`

Total water delivered to connected destinations this tick, in t/h. Reads **0** when throttle is **0** or no destination can accept flow. If a productive well still reports 0, check the connections, completed pipe routes, conflicts, power, and destination capacity.

RETURNS

`float`

Total water delivered across every reachable connected destination this tick, in t/h. **0** when throttle is **0** or no destination can accept flow.

`.is_stalled() → bool`

`True` if, on the last flow tick, the powered pump had an open throttle and water available from its well but could transfer none across its connected routes. No available water, a closed throttle, or lack of power does not report a stall. Declare a destination with `self.water_out.connect(...)`, or let consumers connect their own `water_in` ports to this Pump.

RETURNS

`bool`

`True` if, on the last flow tick, the powered pump had `throttle > 0` and water available from its well but could transfer none across its connected routes. `False` when unpowered, the throttle is closed, or no water is available to pump.

`.throttle() → float`

Current throttle setting (**0-1**). **0** by default, the pump idles until a script calls `set_throttle()`.

RETURNS

`float`

Current throttle setting (**0-1**).

`.set_throttle(rate: float) → ActionResult` SELF ONLY

Set the pump's total output rate (**0-1**) across reachable connected destinations. **0** idles the pump (no extraction, no draw); **1** allows full well output subject to headroom and throughput. This script-owned setpoint resets to **0** when the script stops, ends, or errors, so keep the control loop running while the Pump should operate.

PARAMETERS

Name	Type	Description
`rate`	`float`	Throttle setting (**0-1**). Default **0**: pump idles until scripted.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Biomass Mixer

Blends biome essences into biomass. The global Biomass phase sets the minimum diversity, while every additional well-balanced essence can raise output. Mk II raises output 4.5× while essence demand rises only 2.4×, at five times the power draw.

Type	Biosphere
Power in	Variable (draws from grid)
Consumes	Frozen Essence, buffer 30 t
Consumes	Coastal Essence, buffer 30 t
Consumes	Geothermal Essence, buffer 30 t
Consumes	Volcanic Essence, buffer 30 t
Consumes	Deep Essence, buffer 30 t
Tiers	Mk II

HOW TO OBTAIN

1. Requires the **Biosphere** research (Terraform Index 210,000).
2. Buy from the Shop for 60,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.frozen_essence_in: FluidPort`

Input port for Frozen Essence. Wire with `self.frozen_essence_in.connect(...)`. See `FluidPort`.

RETURNS

`FluidPort`

Frozen Essence input.

`.coastal_essence_in: FluidPort`

Input port for Coastal Essence. Wire with `self.coastal_essence_in.connect(...)`. See `FluidPort`.

RETURNS

`FluidPort`

Coastal Essence input.

`.geothermal_essence_in: FluidPort`

Input port for Geothermal Essence. Wire with `self.geothermal_essence_in.connect(...)`. See `FluidPort`.

RETURNS

`FluidPort`

Geothermal Essence input.

`.volcanic_essence_in: FluidPort`

Input port for Volcanic Essence. Wire with `self.volcanic_essence_in.connect(...)`. See `FluidPort`.

RETURNS

`FluidPort`

Volcanic Essence input.

`.deep_essence_in: FluidPort`

Input port for Deep Essence. Wire with `self.deep_essence_in.connect(...)`. See `FluidPort`.

RETURNS

`FluidPort`

Deep Essence input.

METHODS

`.biomass_rate() → float`

Biomass tons produced last tick, in **t/h**. Sum across every Biomass Mixer on the planet = total biomass production rate.

RETURNS

`float`

Biomass produced last tick, in t/h.

`.active_essences() → int`

Number of essence input buffers currently carrying supply (**0-5**). The current phase decides the minimum required by `required_essences()`. Extra balanced essence types can increase output.

RETURNS

`int`

Count of essence input buffers
that currently have supply (0-5).

`.mixing_essences() → int`

Number of supplied essence types selected for the strongest balanced mix on the last tick (**0-5**). The Mixer evaluates every diversity level from the phase minimum upward, so a weak extra feed can never reduce output.

RETURNS

`int`

The count of supplied essence
types selected for the strongest
balanced mix last tick (**0-5**).

`.tier() → int`

Installed Mixer tier: **1** for Mk I or **2** for Mk II. Mk II produces **4.5×** biomass while consuming only **2.4×** as much of every selected essence, with **5×** power draw, so it yields **87.5%** more biomass per ton of essence.

RETURNS

`int`

Installed Mixer tier: **1** (Mk I) or **2** (Mk II).

`.is_stalled() → bool`

`True` if the mixer is powered and fewer input buffers contain usable essence than the current Biomass phase requires. Buffered essence counts even without new inflow. `False` when unpowered or enough essence types are available. Connect each missing essence input to a compatible source and complete a remote Liquid Pipe route where needed; Biomass phases only advance and never drop.

RETURNS

`bool`

`True` if the mixer is powered and fewer input buffers contain usable essence than the current Biomass phase requires. Buffered essence counts even without new inflow. `False` when unpowered or enough essence types are available.

`.phase() → int`

The global Biomass phase (**1-6**), derived from cumulative biomass tons, using the same thresholds as the Sensors phase badge. The phase sets the minimum essence diversity but does not directly multiply output.

RETURNS

`int`

The global Biomass phase (**1-6**), derived from cumulative biomass tons. It sets the minimum required essence diversity.

`.required_essences() → int`

How many distinct biome essences must be supplied this phase (**1-5**, equal to the phase, capped at 5). Below this the Mixer stalls. Supplying more can raise output when the larger mix is balanced enough.

RETURNS

`int`

The minimum distinct biome essences needed this phase (**1-5**). Extra balanced essences can raise output.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Crop Automator

Queues harvest, plant, and treatment jobs across up to 24 other cells in a centered 5 by 5 service area, then executes one job at a time with a short pause between them. Scripts find it with `outpost.harvesting_machines()`.

Type	Biosphere
Power in	-60 W (draws from grid)
Output buffer	Units: 50,000
Stockpile	400 units (mixed)

HOW TO OBTAIN

1. Requires the **Field Automation** research (Plants 620,000).
2. Buy from the Shop for 30,000 cr.
3. Deploy the kit on an empty field cell with a Harvester's `deploy()`.

Access via: `self`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.input: InputSlot`

Standard multi-material `InputSlot` accepting species seeds, Fertilizer Mk I/II/III, and Growth Accelerant.

RETURNS

`InputSlot`

`.output: OutputSlot`

Standard `OutputSlot` containing collected Forage. Send to a local destination through this slot, or let a local Plant Terraformer pull from it through its standard input. Every route uses normal timed transfers.

RETURNS

`OutputSlot`

METHODS

`.harvest(sector: str) → JobReceipt` SELF ONLY

Submit one harvest job for a covered sector. Submission is immediate; valid field work later takes **0.1 hours**. Missing output space pauses this FIFO head without bypassing it. A target mismatch is terminal, takes no work time, and advances the queue.

PARAMETERS

Name	Type	Description
`sector`	`str`	One of the cells in the automator's service area.

RETURNS

`JobReceipt`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.job_id`, `.queue_position`

OUTCOMES

Status	Kind	Meaning
`"queued"`	● success	Queued job `.job_id` at execution position `.queue_position`.
`"queue_full"`	● transient	The Crop Automator's 50-job queue is full.
`"not_placed"`	● rejection	The Crop Automator is not deployed on the field.
`"out_of_range"`	● rejection	The requested sector is outside the Crop Automator's service area.

RAISES

Exception	Condition
`ValueError`	The `sector` argument is not a valid field sector id.

`.plant(sector: str, seed_id: str) → JobReceipt` SELF ONLY

Submit one planting job with a specific species seed. Submission is immediate and does not require the seed to be loaded yet. The serial executor pauses at this FIFO head until the seed is present, then spends **0.1 hours** and rechecks the target before committing.

PARAMETERS

Name	Type	Description
`sector`	`str`	One of the cells in the automator's service area.
`seed_id`	`str`	A species-specific seed item id.

RETURNS

`JobReceipt`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.job_id`, `.queue_position`

OUTCOMES

Status	Kind	Meaning
`"queued"`	● success	Queued job `.job_id` at execution position `.queue_position`.
`"queue_full"`	● transient	The Crop Automator's 50-job queue is full.
`"not_placed"`	● rejection	The Crop Automator is not deployed on the field.
`"out_of_range"`	● rejection	The requested sector is outside the Crop Automator's service area.
`"invalid_seed"`	● rejection	The requested item is not a species seed.

RAISES

Exception	Condition
`ValueError`	The `sector` argument is not a valid field sector id.

`.apply(sector: str, item_id: str) → JobReceipt` SELF ONLY

Submit one Fertilizer Mk I/II/III or Growth Accelerant job. Submission is immediate and does not require the material to be loaded yet. The serial executor pauses at this FIFO head until the material is present, then spends **0.1 hours** and rechecks the target before committing.

PARAMETERS

Name	Type	Description
`sector`	`str`	One of the cells in the automator's service area.
`item_id`	`str`	Fertilizer Mk I/II/III or Growth Accelerant.

RETURNS

`JobReceipt`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.job_id`, `.queue_position`

OUTCOMES

Status	Kind	Meaning
`"queued"`	● success	Queued job `.job_id` at execution position `.queue_position`.
`"queue_full"`	● transient	The Crop Automator's 50-job queue is full.
`"not_placed"`	● rejection	The Crop Automator is not deployed on the field.
`"out_of_range"`	● rejection	The requested sector is outside the Crop Automator's service area.
`"invalid_material"`	● rejection	The requested item is not a supported crop treatment.

RAISES

Exception	Condition
`ValueError`	The `sector` argument is not a valid field sector id.

`.position() → str`

Grid sector occupied by this automator. Jobs can target up to 24 other cells in its centered 5 by 5 service area.

RETURNS

`str`

The automator's own field sector. Jobs can target up to 24 other cells in its centered 5 by 5 service area.

`.cell(sector: str) → Cell | None`

Read one sector inside this automator's service area as a `Cell` snapshot, covering plant, status, growth, conditions, and remaining treatment hours. The addressable set is exactly the set `harvest()`, `plant()`, and `apply()` accept, so a sector this returns `None` for is one no job can target either.

PARAMETERS

Name	Type	Description
`sector`	`str`	One of the cells in the automator's service area.

RETURNS

`Cell | None`

`NONE` MEANS

`None` means this automator cannot address that sector: it is outside the service area, or not a valid field sector.

OUTCOMES

Status	Meaning
`None`	`None` means this automator cannot address that sector: it is outside the service area, or not a valid field sector.

`.cells() → list[Cell]`

Read every sector this automator serves as a list of `Cell` snapshots, for sweeping the whole service area in one pass. Unscanned natural ground reports status `"unknown"`.

RETURNS

`list[Cell]`

One snapshot per sector this automator can address. Empty while the automator is not placed on the field.

`.status() → str`

Exact executor state: `"not_placed"`, `"no_power"`, `"working"`, `"no_seed"`, `"no_material"`, `"output_full"`, `"results_full"`, or `"idle"`.

RETURNS

`str`

POSSIBLE VALUES

`"not_placed"`, `"no_power"`, `"working"`, `"no_seed"`, `"no_material"`, `"output_full"`, `"results_full"`, `"idle"`

`.current_job() → CropJob | None`

Active FIFO head as a `CropJob`, including action, target, progress, and blocker. Returns `None` while idle.

RETURNS

`CropJob | None`

NONE MEANS

`None` means the Crop Automator has no active job.

OUTCOMES

Status	Meaning
`None`	`None` means the Crop Automator has no active job.

`.get_queue() → list[CropJob]`

Snapshot of pending `CropJob` values in exact FIFO order (first in, first out). The active job is reported separately by `current_job()`.

RETURNS

`list[CropJob]`

`.queue_count() → int`

Total unfinished jobs, counting the active job and every pending job. Maximum **50**.

RETURNS

`int`

`.result_count() → int`

Completed terminal results waiting in the result inbox. At **50**, execution pauses until results are consumed.

RETURNS

`int`

`.next_result() → CropJobResult` SELF ONLY

Consume the oldest terminal result. An empty inbox is reported without changing machine state.

RETURNS

`CropJobResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.job_id`, `.action`, `.sector`, `.item_id`, `.collected`, `.discarded`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Job `.job_id` completed at `.sector`; the world mutation committed successfully.
`"partial"`	● partial	Job `.job_id` harvested `.sector`, but the output bin could not hold all of it. Forage kept: `.collected`. Forage discarded: `.discarded`. The cell was cleared.
`"empty"`	● success	No completed Crop Automator result is waiting.
`"out_of_range"`	● rejection	The target left this Crop Automator's service area before execution.
`"no_plant"`	● rejection	The target sector held no plant when the job reached the executor.
`"not_mature"`	● rejection	The target plant was not mature when the harvest job reached the executor.
`"no_forage"`	● rejection	The ready crop held no whole Forage when the harvest job reached the executor.
`"not_empty"`	● rejection	The target sector was not empty when the planting job reached the executor.
`"base_sector"`	● rejection	The Harvester base sector cannot be planted.
`"already_mature"`	● rejection	The target plant was already mature when the treatment job reached the executor.
`"tier_conflict"`	● rejection	The requested Fertilizer tier conflicted with the active dose on the target plant.
`"invalid_seed"`	● rejection	The planting job referred to an invalid species seed.
`"invalid_material"`	● rejection	The treatment job referred to an unsupported material.

`.cancel_job(job_id: int) → ActionResult` SELF ONLY

Cancel one active or pending job by id. Canceling active work discards only its progress; inputs and field state remain unchanged.

PARAMETERS

Name	Type	Description
`job_id`	`int`	Job id returned by `harvest()`, `plant()`, or `apply()`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.

`.move_job(job_id: int, position: int) → ActionResult` SELF ONLY

Move one unfinished job to a one-based execution position, counting the active job first and then pending jobs. Reordering only pending work preserves active progress. While a job is active, changing which job is first preempts the arm: the displaced job keeps its id and request, loses its progress, and creates no terminal result. Position **1** is first; `queue_count()` is the last position.

PARAMETERS

Name	Type	Description
`job_id`	`int`	Stable id of an active or pending job.
`position`	`int`	One-based execution position across the active and pending jobs.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The unfinished job now occupies the requested execution position.
`"not_found"`	● rejection	No active or pending job has the supplied id.
`"invalid"`	● rejection	The supplied position is outside the one-based range from **1** through `queue_count()`.

`.clear_queue() → CountResult` SELF ONLY

Cancel the active job and every pending job. Completed results remain available through `next_result()`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`CountResult`	`.status`, `.message`	`.count`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Command completed. Affected entries or units: `.count`.
`"no_op"`	● success	The command affected no entries or units.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Dispenser

Salts the four orthogonally adjacent field cells (directly above, below, left, and right) while powered, supplied, and enabled. Scripts find it with `outpost.harvesting_machines()`.

Type	Biosphere
Power in	Variable (draws from grid)
Input buffer	Units: 50

HOW TO OBTAIN

1. The recipe unlocks with the **Dispenser** research (Plants 300,000).
2. Fabricate **Dispenser Kit** on a **Fabricator**: 1× Machine Frame, 1× Control Unit, and 1× Circuit Panel.
3. Deploy the kit on an empty field cell with a Harvester's `deploy()`.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.input: InputSlot`

Loads salt from a storage bin or other source. Connect a source with `self.input.connect(name)`, then pull salt with `self.input.take("salt", count)`. See `InputSlot`.

RETURNS

`InputSlot`

Feed salt in from a storage bin: `self.input.connect("Salt Bin")` then `self.input.take("salt", 50)`. Salt is a Water Pump byproduct.

METHODS

`.set_enabled(enabled: bool) → ActionResult` SELF ONLY

Command salting on or off. Power loss pauses the script but preserves this setpoint; stopping the machine script resets it to `False`.

PARAMETERS

Name	Type	Description
`enabled`	`bool`	Whether this script commands salting.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

`True` when the running script has commanded salting on.

RETURNS

`bool`

`.is_active() → bool`

`True` when commanded on with power and salt available.

RETURNS

`bool`

`.is_supplied() → bool`

`True` when the dispenser is commanded on, powered, and has salt in its input buffer. If disabled, unpowered, or empty, covered cells lose `salted`.

RETURNS

`bool`

`True` when the dispenser is placed, commanded on, powered, and has salt in its input buffer. `False` when unplaced, disabled, unpowered, or empty; covered cells then lose `salted` and their plants pause.

`.status() → str`

Exact operating state: `"not_placed"`, `"disabled"`, `"no_power"`, `"no_salt"`, or `"active"`.

RETURNS

`str`

POSSIBLE VALUES

`"not_placed"`, `"disabled"`, `"no_power"`, `"no_salt"`, `"active"`

`.buffer() → float`

Fraction of the onboard salt buffer currently filled (**0-1**). It drops while dosing cells and refills through `self.input`.

RETURNS

`float`

Fraction of the onboard salt buffer currently filled (**0-1**). Drops as the dispenser doses cells; refilled by feeding salt into `self.input`. **0** means empty (covered cells lose `salted`).

`.tier() → int`

Always **1**. The Dispenser ships at Mk I and has no upgrade pack; salt providers don't tier, so every deployed Dispenser reads **1**.

RETURNS

`int`

Always **1**. The Dispenser ships at Mk I and has no upgrade pack; salt providers don't tier.

`.position() → str`

Grid sector occupied by this dispenser, such as `"E14"`.

RETURNS

`str`

The grid sector this dispenser occupies (e.g. `"E14"`).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Essence Liquifier

Renders native life-form samples down into their biome's essence fluid. It only accepts life forms from its own outpost's biome, and produces that biome's essence.

Type	Biosphere
Power in	Variable (draws from grid)
Produces	One biome-matched port: Frozen Essence, Coastal Essence, Geothermal Essence, Volcanic Essence, and Deep Essence, buffer 50 t
Input buffer	Units: 50

HOW TO OBTAIN

1. Requires the **Biosphere** research (Terraform Index 210,000).
2. Buy from the Shop for 35,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

The input slot for life-form samples native to this outpost's biome. Machine and storage sources must share the outpost. Inventory works only at home; remote Liquifiers receive samples from local storage. A wrong-biome sample is rejected without being moved.

RETURNS

`InputSlot`

For life-form samples native to this outpost's biome. Wrong-biome samples are rejected without being moved; `eject(...)` recovers staged samples to an explicit local destination.

`.frozen_essence_out: FluidPort`

This machine has exactly one essence output, named for its outpost biome: `frozen_essence_out`, `coastal_essence_out`, `geothermal_essence_out`, `volcanic_essence_out`, or `deep_essence_out`. It is a `FluidPort`. Connect it to a local Liquid Tank or matching Mixer input. A remote peer relationship also needs a completed conflict-free Liquid Pipe route between both locations. Example: `self.frozen_essence_out.connect("liquid_tank_1")`.

RETURNS

`FluidPort`

Frozen essence output. A Liquifier exposes only the port matching its outpost biome. Call `self.biome()` to confirm the port, then connect it like any fluid source: `self.frozen_essence_out.connect("liquid_tank_1")`.

METHODS

`.essence_rate() → float`

Current biome essence output, in **t/h**. At 100% outpost efficiency, the Essence Liquifier has a base **1 t/h** life-form intake. This readout applies both outpost efficiency and the loaded life form's rarity yield (common **×5**, uncommon **×10**, rare **×25**), and reads **0** when the machine cannot run or buffer its next whole output.

RETURNS

`float`

Current biome essence output, in t/h.

`.yield_multiplier() → float`

Unscaled essence tons produced per ton of the life form currently in the input bin: **5** for a common life form, **10** uncommon, **25** rare. Returns **0** when the input is empty. At 100% outpost efficiency, `essence_rate()` equals this multiplier while the machine is powered, fed, and able to buffer its next whole output; overcrowding can reduce the actual output rate.

RETURNS

`float`

Essence tons produced per ton of the life form currently loaded (**5** common / **10** uncommon / **25** rare), or **0** when the input is empty.

`.is_stalled() → bool`

`True` whenever `stall_reason()` is not `"ok"`: no valid host biome, no input, or insufficient output room for the next whole rarity-scaled yield. Use `stall_reason()` for the specific cause.

RETURNS

`bool`

`True` whenever `stall_reason()` is not `"ok"`.

`.stall_reason() → str`

Why the Liquifier is idle, as a string you can branch on: `"no_biome"` (the machine has no valid host outpost), `"no_input"` (input bin empty, feed it native-biome life forms), `"output_full"` (the next whole rarity-scaled yield cannot fit and a configured output cannot drain right now), `"unconnected"` (the next whole yield cannot fit and the essence output has no effective peer relationship), or `"ok"` (running, ready, or able to buffer the next whole yield). More specific than `is_stalled()`.

RETURNS

`str`

Why the Liquifier is idle, or `"ok"` when it is not.

POSSIBLE VALUES

`"ok"`, `"no_biome"`, `"no_input"`, `"output_full"`, `"unconnected"`

`.biome() → str | None`

Returns the biome the host outpost sits in: `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, or `"deep"`. Returns `None` when the machine has no valid outpost. This determines which life-form items the Essence Liquifier will accept; wrong-biome items are rejected by the input port.

RETURNS

`str | None`

The biome the outpost sits in, or `None` if the machine has no valid outpost. It determines which life-form items the Essence Liquifier accepts.

POSSIBLE VALUES

`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Grow Lamp

Lights the four orthogonally adjacent field cells (directly above, below, left, and right) while powered and enabled. Scripts find it with `outpost.harvesting_machines()`.

Type	Biosphere
Power in	Variable (draws from grid)
Tiers	Mk II, Mk III, and Mk IV

HOW TO OBTAIN

1. The recipe unlocks with the **Grow Lamp** research (Plants 500,000).
2. Fabricate **Grow Lamp Kit** on a **Fabricator**: 1× Machine Frame, 2× Circuit Panel, and 2× Glass.
3. Deploy the kit on an empty field cell with a Harvester's `deploy()`.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.set_enabled(enabled: bool) → ActionResult` SELF ONLY

Command the lamp on or off. Power loss pauses the script but preserves this setpoint; stopping the machine script resets it to `False`.

PARAMETERS

Name	Type	Description
`enabled`	`bool`	Whether this script commands the lamp to light its cells.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

`True` when the running script has commanded this lamp on.

RETURNS

`bool`

`.is_active() → bool`

`True` when the lamp is commanded on and has power.

RETURNS

`bool`

`.is_supplied() → bool`

`True` when the lamp is commanded on, powered, and actively lighting its covered cells. If disabled or unpowered, covered cells lose `lit` and their plants pause.

RETURNS

`bool`

`True` when the lamp is placed, commanded on, powered, and actively lighting its covered cells. `False` when unplaced, disabled, or unpowered; covered cells then lose `lit` and their plants pause.

`.status() → str`

Exact operating state: `"not_placed"`, `"disabled"`, `"no_power"`, or `"active"`.

RETURNS

`str`

`"not_placed"` when a repaired save cleared an invalid placement but kept the lamp, `"disabled"` when the script command is off, `"no_power"` when commanded on but unpowered, or `"active"` while lighting.

POSSIBLE VALUES

`"not_placed"`, `"disabled"`, `"no_power"`, `"active"`

`.tier() → int`

Deployed tier (**1-4**). Mk I/II/III/IV multiply the Forage output of covered plants that need light by **1×/2×/4×/8×** and draw **5/25/100/500 W** while active.

RETURNS

`int`

The deployed tier (**1-4**). Higher tiers boost the output of plants they cover and draw far more power; tier up by fabricating and applying a Grow Lamp upgrade pack.

`.position() → str`

Grid sector occupied by this lamp, such as `"E14"`.

RETURNS

`str`

The grid sector this lamp occupies (e.g. `"E14"`).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Plant Terraformer

The sole converter from harvested physical Forage to permanent Plants km². Load its input through ordinary timed item transfers; its high-capacity feeder handles 16 items per step at Mk I and 80 at Mk II. Each later phase adds one more input: Water, then Salt, then Fertilizer, then Growth Accelerant. Mk I stops at the Fields threshold; Mk II carries the final two phases.

MK I

Type	Biosphere
Power in	Variable (draws from grid)
Consumes	Water, buffer 60 t
Stockpile	1,243 units (mixed)
Tiers	Mk II

MK II

Consumes	Water, buffer 330 t
Stockpile	6,654 units (mixed)

HOW TO OBTAIN

1. The recipe unlocks with the **Plant Terraformer** research (Biomass 2,000).
2. Fabricate **Plant Terraformer Kit** on a **Fabricator**: 5× Machine Frame, 3× Control Unit, 5× Circuit Panel, 4× Liquid Pipe Segment, and 10 t Water.
3. Deploy it from your Inventory.

Access via: `self`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

Standard timed `InputSlot` for Forage, Salt, Fertilizer, and Growth Accelerant. Connect Inventory, a local Storage Bin or Warehouse, or a local machine output such as a Crop Automator after Auto Feeders research. The destination feeder handles **16 items per step** at Mk I and **80** at Mk II, in either transfer direction. Its holders fit one full Forage batch plus Salt and up to **10 of each** support item type.

RETURNS

`InputSlot`

Timed item transfers into the Terraformer's one-cycle material holders. Mk I handles 16 items per ordinary feeder quantum; Mk II handles 80.

`.water_in: FluidPort`

Water `FluidPort` sized for one full cycle: **60 t** at Mk I or **330 t** at Mk II. Connect it to a water source; a remote source also needs a completed conflict-free Liquid Pipe route between both locations. Water is required from the Seedlings phase onward.

RETURNS

`FluidPort`

Water input, required from the Seedlings phase onward.

METHODS

`.set_enabled(enabled: bool) → ActionResult` SELF ONLY

Enable or pause conversion. `True` starts a cycle whenever the onboard item holders and Water can supply at least one proportional Forage unit. A cycle runs for **3 hours** at full outpost efficiency. Stopping this machine's script resets the setpoint to `False`; an in-flight batch remains loaded.

PARAMETERS

Name	Type	Description
`enabled`	`bool`	`True` to convert from materials already loaded through `input`, `False` to pause the machine.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

`True` when the current machine script has commanded conversion on.

RETURNS

`bool`

`.tier() → int`

Permanently installed Plant Terraformer tier as an integer (**1-2**). Mk II raises batch throughput and enables the late Plants recipes.

RETURNS

`int`

Permanently installed Plant Terraformer tier (**1-2**).

`.status() → str`

Exact live state: `"complete"`, `"disabled"`, `"no_power"`, `"needs_mk2"`, `"no_forage"`, `"no_water"`, `"no_salt"`, `"no_fertilizer"`, `"no_accelerant"`, or `"running"`.

RETURNS

`str`

POSSIBLE VALUES

`"complete"`, `"disabled"`, `"no_power"`, `"needs_mk2"`, `"no_forage"`, `"no_water"`, `"no_salt"`, `"no_fertilizer"`, `"no_accelerant"`, `"running"`

`.is_running() → bool`

`True` while a conversion cycle is running. An unpowered or disabled machine keeps its in-flight batch but reads `False` until it resumes.

RETURNS

`bool`

`.get_progress() → float`

Completion of the current conversion cycle, **0-1**. A full cycle requires **3 hours** of work at 100% outpost efficiency; overcrowding slows its progress proportionally. Reads **0** whenever no batch is loaded.

RETURNS

`float`

Progress through the current cycle (**0-1**).

`.batch_size() → int`

Whole Forage items in the running cycle, or the batch that can load now. A full batch is **1,200** at Mk I and **6,600** at Mk II. Limited materials or a nearby phase boundary load less.

RETURNS

`int`

Whole Forage items committed to the running cycle, or the batch the next cycle would load. Returns 0 while stalled, disabled, or complete.

`.km2_rate() → float`

This machine's current permanent Plants output in km²/h. It combines the loaded batch, its nominal **3 hour** work cycle, the pinned phase exchange rate from **20 km² per Forage** early to **1 km² per 3 Forage** late, and this outpost's overcrowding efficiency. Returns **0** while blocked, disabled, or complete.

RETURNS

`float`

Current conversion rate in km²/h, the running batch spread across its cycle. Returns 0 while stalled, disabled, or complete.

`.phase() → int`

Current global Plants phase number, **1-6**.

RETURNS

`int`

Current global Plants phase index from 1 to 6.

`.recipe_tier() → int | None`

Derived production tier of the current cumulative Plants conversion recipe. Returns `None` after Continental completion.

RETURNS

`int | None`

Derived production tier for the current Plants conversion, or `None` after completion.

`.next_threshold() → float`

Permanent Plants km² required for the next phase. At completion, returns the **5,000,000 km²** ceiling.

RETURNS

`float`

Plants km² required for the next phase, or the completed 5,000,000 km² ceiling.

`.remaining() → float`

Permanent Plants km² still needed for the next phase. Returns **0** when Continental is complete.

RETURNS

`float`

Plants km² remaining to the next phase. Returns 0 at completion.

`.required_inputs() → list[str]`

Current cumulative material ids. Starts with `forage`, then adds `water`, `salt`, the `fertilizer` category, and `growth_accelerant` across the five conversions.

RETURNS

`list[str]`

Material ids the current cumulative phase recipe requires.

`.batch_requirements() → dict[str, int]`

Exact amounts for the largest next batch allowed by this tier and phase. The `dict` uses `forage`, `water`, `salt`, `fertilizer_potency`, and `growth_accelerant` as needed. Salt and Growth Accelerant are whole-item counts, rounded up per batch. Fertilizer potency is a whole number. The whole `dict` stays at the full next batch even when onboard stock is short; `batch_size()` reports what can load now.

RETURNS

`dict[str, int]`

Material amounts for the largest next batch allowed by this tier and phase. Salt and Growth Accelerant are whole-item counts, rounded up per batch. Fertilizer is reported as whole `fertilizer_potency`.

`.fertilizer_potency(item_id: str) → int`

Return one Fertilizer item's whole potency: **10** for Mk I, **30** for Mk II, or **50** for Mk III. Any other item id raises `ValueError`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	A Fertilizer Mk I, Mk II, or Mk III item id.

RETURNS

`int`

Potency of one Fertilizer item: 10 for Mk I, 30 for Mk II, or 50 for Mk III.

RAISES

Exception	Condition
`ValueError`	The item id must identify Fertilizer Mk I, Mk II, or Mk III.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()` SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Seed Maker

An outpost processing building that combines three life-form samples into a viable seed. Most blends fail; the working recipes are unique to this planet and found by trial, and a discovered one can be re-run for more.

Type	Biosphere
Power in	Variable (draws from grid)
Output buffer	Units: 1
Stockpile	3 units (mixed)

HOW TO OBTAIN

1. The recipe unlocks with the **Seed Maker** research (Biomass 500).
2. Fabricate **Seed Maker Kit** on a **Fabricator**: 2× Machine Frame, 1× Control Unit, 2× Circuit Panel, and 3 t Water.
3. Deploy it from your Inventory.

Access via: `self`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.input: InputSlot`

`InputSlot` for loading the Seed Maker's three-sample reaction chamber. The natural source is a local Drone Depot where biological drones unload; a local Warehouse, Storage Bin, or home Inventory is also valid. The chamber accepts exactly **1 t each of three distinct life forms** and cannot stockpile surplus material.

RETURNS

`InputSlot`

The **3 t** reaction chamber. It accepts one unit each of three distinct life forms and rejects duplicates or a fourth sample. `combine()` reserves the trio while running. When idle, use `eject(...)` to recover a wrongly loaded sample, or `flush()` to destroy everything in the chamber.

`.output: OutputSlot`

`OutputSlot` containing the one physical seed produced by a successful trial. Send it to home Inventory, then call the Harvester's `load_seed()` anywhere on the local grid, or route it to a Crop Automator. The Seed Maker cannot start another trial until this result bay is empty. Sludge produces no item.

RETURNS

`OutputSlot`

Holds the single physical result seed. Send it to home Inventory, then transfer it into the Harvester with `load_seed()`; another local store is also valid. A successful trial leaves exactly **1** species-specific seed here, and no further trial can start until `send(...)` removes it. A sludge result produces no output item.

METHODS

`.combine(blend: list[str]) → SeedResult` SELF ONLY

Run the exact three different life-form ids loaded in this machine's reaction chamber. Every accepted trial consumes the chamber's **1 t of each**. The one-seed result bay must be empty before any trial can start.

PARAMETERS

Name	Type	Description
`blend`	`list[str]`	A list of exactly 3 distinct life-form item ids, e.g. `["ice_algae", "sea_algae", "vent_algae"]`. Order does not matter; duplicates and any other length raise `ValueError`.

RETURNS

`SeedResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.seed_id`, `.species`

OUTCOMES

Status	Kind	Meaning
`"seed_found"`	● success	Discovered `.species` and deposited `.seed_id` into the Seed Maker output.
`"sludge"`	● success	The accepted three-life-form blend produced no seed.
`"locked"`	● rejection	Seed Maker research is not unlocked.
`"busy"`	● transient	A Seed Maker combine trial is already in progress.
`"missing_life_forms"`	● rejection	The Seed Maker chamber does not contain exactly the requested three 1 t life-form samples.
`"output_full"`	● rejection	The Seed Maker result bay already contains its one physical seed.

RAISES

Exception	Condition
`ValueError`	The blend must contain exactly three distinct, known life-form ids.

`.life_forms() → list[str]`

List of the **30** accepted life-form item ids. For each `blend` from `combinations(self.life_forms(), 3)`, load its three items with `self.input.take(item_id, 1)`, then call `self.combine(blend)`. Enumeration does not move materials. Send any resulting physical seed from `self.output` before continuing.

RETURNS

`list[str]`

The **30** accepted life-form item ids. For each `blend` from `combinations(self.life_forms(), 3)`, load its three items with `self.input.take(item_id, 1)`, then call `self.combine(blend)`. Enumeration does not move materials. Send any resulting physical seed with `self.output.send(...)` before the next trial.

`.is_running() → bool`

`True` while a combine trial is in flight.

RETURNS

`bool`

`True` while a combine trial is in flight.

`.get_progress() → float`

Progress of the current combine trial as **0-1**; returns **0** when idle.

RETURNS

`float`

Progress of the current trial (**0-1**);
0 when idle.

`.get_output_count() → int`

Number of physical seeds waiting in the single-result bay: **0** or **1**.

RETURNS

`int`

1 when a physical seed is waiting
in the result bay, else **0**.

`.recipes() → list[SeedRecipe]`

List of `SeedRecipe` for every blend discovered so far, the same discover-once-kept-forever journal the Flora / Seed Recipes tab shows. Each carries `.tier`, the physical `.seed_id`, bare `.species`, `.blend`, `.requirements`, `.requirement`, and `.growth_time`. `.requirements` is the programmable form: every `PlantRequirement` has `.kind` and optional `.species`, so `companion` and `antagonist` entries identify the exact related plant. `.requirement` remains a compact string summary. Empty until your first hit; re-run a known `.blend` with `self.combine(...)` to reproduce that seed without re-sweeping.

RETURNS

`list[SeedRecipe]`

Every discovered recipe; empty
before the first discovery. Use
`self.combine(recipe.blend)`
to reproduce its seed.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Sprinkler

Waters the four orthogonally adjacent field cells (directly above, below, left, and right) while powered, supplied, and enabled. Scripts find it with `outpost.harvesting_machines()`.

Type	Biosphere
Power in	Variable (draws from grid)
Consumes	Water, buffer 10 t
Tiers	Mk II, Mk III, and Mk IV

HOW TO OBTAIN

1. The recipe unlocks with the **Sprinkler** research (Plants 100,000).
2. Fabricate **Sprinkler Kit** on a **Fabricator**: 1× Machine Frame, 2× Liquid Pipe Segment, 1× Pressure Valve, and 2 t Water.
3. Deploy the kit on an empty field cell with a Harvester's `deploy()`.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.water_in: FluidPort`

Supplies water from a connected source. Call `self.water_in.connect(...)` with the source's stable machine id or display name. A remote source also needs a completed conflict-free Liquid Pipe route between both locations. See `FluidPort` for level, capacity, flow, and connection queries.

RETURNS

`FluidPort`

Water buffer. Call `connect(...)` with the provider's stable machine id or display name; completed liquid-pipe networks carry water between outposts. Sharing an outpost with a pipe or tank does not connect it automatically.

METHODS

`.set_enabled(enabled: bool) → ActionResult` SELF ONLY

Command watering on or off. Power loss pauses the script but preserves this setpoint; stopping the machine script resets it to `False`.

PARAMETERS

Name	Type	Description
`enabled`	`bool`	Whether this script commands watering.

RETURNS
`ActionResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.is_enabled() → bool`

`True` when the running script has commanded watering on.

RETURNS
`bool`

`.is_active() → bool`

`True` when commanded on with power and water available.

RETURNS
`bool`

`.is_supplied() → bool`

`True` when the sprinkler is commanded on, powered, and has water in its `water_in` buffer. If disabled, unpowered, or dry, covered cells lose `watered`.

RETURNS

`bool`

`True` when the sprinkler is placed, commanded on, powered, and has water in its `water_in` buffer. `False` when unplaced, disabled, unpowered, or dry; covered cells then lose `watered` and their plants pause.

`.status() → str`

Exact operating state: `"not_placed"`, `"disabled"`, `"no_power"`, `"no_water"`, or `"active"`.

RETURNS

`str`

POSSIBLE VALUES

`"not_placed"`, `"disabled"`, `"no_power"`, `"no_water"`, `"active"`

`.buffer() → float`

Fraction of the onboard water buffer currently filled (**0-1**). It drops while watering and refills from the connected `water_in` source.

RETURNS

`float`

Fraction of the onboard water buffer currently filled (**0-1**). Drops as the sprinkler waters; refilled by the connected `water_in` flow source. **0** means dry (covered cells lose `watered`).

`.tier() → int`

Deployed tier (**1-4**). Mk I/II/III/IV multiply the Forage output of covered plants that need water by **1×/2×/4×/8×**, draw **5/25/100/500 W**, and consume **2/4/8/16 t/h Water** while active.

RETURNS

`int`

The deployed tier (**1-4**). Higher tiers boost the output of plants they cover and drink more water and power; tier up by fabricating and applying a Sprinkler upgrade pack.

`.position() → str`

Grid sector occupied by this sprinkler, such as `"E14"`.

RETURNS

`str`

The grid sector this sprinkler occupies (e.g. `"E14"`).

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Item Catalog

Looks up static identity metadata for any known item id. Use `get_component("item_catalog")` when a script needs to classify an item without maintaining its own data archive.

Access via: `get_component("item_catalog")`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

METHODS

`.lookup(item_id: str) → ItemInfo | None`

Return an `ItemInfo` with `.id`, `.name`, `.category`, `.stackable`, `.biome`, `.rarity`, and `.production_tier`. Categories distinguish `"mineral"`, `"refined"`, `"crafted"`, `"agriculture"`, `"life_form"`, `"field_resource"`, `"biology_sample"`, `"reagent"`, `"equipment"`, `"module"`, `"portable"`, `"upgrade_pack"`, and `"construction_kit"`. Production tier is `None` for source items and biome and rarity are `None` when they do not apply. An unknown item id returns `None`.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to identify

RETURNS

`ItemInfo | None`

`None` for an unknown item id.

```
catalog = get_component("item_catalog")
item = catalog.lookup("water_pump")
if item is not None:
    print(item.category, item.production_tier)
```

Weather Station

A programmable local storm station and live receiver. It measures immutable local reports, hears event transmissions on `broadcast` and biome channels, and may publish to the Signal Board.

Type	Sensors
Power in	-50 W (draws from grid)

HOW TO OBTAIN

1. Requires the **Weather Program** research (Terraform Index 330,000).
2. Buy from the Shop for 60,000 cr.

Access via: `self / get_component(id)`

Every component has a stable `.id`. For a deployed machine, open the ⓘ on its card to find the exact ID, then pass that value to `get_component(id)`. IDs are case-sensitive.

PROPERTIES

`.id: str`

Stable programmatic identifier for this component. Use it with `get_component(id)` and APIs that ask for component, planet, vehicle, station, or order ids.

RETURNS

`str`

`.name: str`

Human-readable display name. Prefer `.id` for scripts that need to survive renames.

RETURNS

`str`

`.outpost: OutpostRef`

The outpost where this building is deployed. The returned `OutpostRef` includes its stable id, display name, biome, position, capacity, and `buildings()` query. Read the property again when you need current values.

RETURNS

`OutpostRef`

The outpost where this building is deployed.

`.signal_board: WeatherSignalBoard`

This station's programmable Weather display. `reveal()` publishes a transmission into a numbered slot, `reject()` counts a refused copy, and `resolve()` publishes labelled values. The board displays what your script supplies without interpreting it.

RETURNS

`WeatherSignalBoard`

This station's optional player-authored board. It displays supplied transmissions and labelled values without interpreting them.

`.signal_receiver: SignalReceiver`

Live receiver. `transmissions()` returns only the raw copies audible to this powered station now and stores no history. Dust reception uses local biome channels; thunder reception uses the `broadcast` channel.

RETURNS

`SignalReceiver`

This station's live receiver. `transmissions()` exposes only what is audible now and stores no raw history. Dust-event channels are biome-specific; all thunder packets use the `broadcast` channel.

METHODS

`.observe() → WeatherReport` SELF ONLY

Measure one immutable local report. The station refreshes at most once per world-clock hour; faster calls return the same report id. The report includes local coverage, active storm snapshots, and a local forecast reaching 8 world-clock hours ahead, or 24 once Weather Forecasting is researched. It never reveals where a storm's hidden aftermath deposit lies.

RETURNS

`WeatherReport`

An immutable report from this station's local coverage. Repeated calls inside one observation hour return the same report. Raises `RuntimeError` when the station is unfinished or unpowered.

RAISES

Exception	Condition
`RuntimeError`	The Weather Station is unfinished or unpowered, so no observation can be recorded.

`.last_report() → WeatherReport | None`

Read the last measured report without taking a new observation. This is useful for startup recovery and stale-data handling; `None` means this station has never completed `observe()`.

RETURNS

`WeatherReport | None`

The last local report, or `None` before the first successful observation. The snapshot remains readable while stale or unpowered.

`.strikes() → list[WeatherStrike]`

Return this station's bounded strike history. Each `WeatherStrike` includes its event id, observation time, energy, and whether a Lightning Rod banked it. Exact strike positions and Storm Glass eligibility are not included. Only strikes this station physically observed are returned.

RETURNS

`list[WeatherStrike]`

Lightning strikes observed by this station, oldest first.

`.set_status(message: str, level: str = "info") → None` SELF ONLY

Show a status message for this machine's current script run. Use `self.set_status(message, "info")`. The same reporting capability is available as `set_status()` in every script. Messages follow the current execution, independently of machine state and game warnings.

PARAMETERS

Name	Type	Description
`message`	`str`	Non-empty plain text, at most 240 characters. Null characters are not accepted.
`level`	`str`	Presentation severity: `info`, `warn`, or `error`. Defaults to `info`.

RETURNS

`None`

`None`.

`.clear_status() → None` SELF ONLY

Clear the current script run's status message. Clearing an absent message has no effect. Does not wait or change machine behaviour.

RETURNS

`None`

`None`.

`.get_status_report() → ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

RETURNS

`ScriptStatusReport | None`

Read this machine's script status report from any script. Returns `None` when it has no report. The returned snapshot includes `message`, `level`, `active`, and `run_id`.

`.peek_command() · .next_command() · .command_count() · .clear_commands()`

SELF ONLY

Read commands you send from the editor's **Commands** tab while this script runs. Identical on every scriptable machine, see the guide: Script Commands

Production Tiers

Every manufacturing and transformation recipe arranged by dependency depth. Smelter foundations begin at Tier 0. Every other recipe is at least Tier 1 and sits one tier above its deepest manufactured input; recipes that consume only source materials therefore enter at Tier 1. Discovered Seed Maker and Bio Lab recipes join the graph without exposing undiscovered combinations or specimens.

Fabricator Recipes

The Fabricator assembles components from refined stock, with several inputs per blueprint and some recipes producing byproducts. Select with `fabricator.set_recipe`. Most blueprints arrive as contractor order rewards, each card below names its order.

Fabricate Smelter Mk II Upgrade Pack `craft_smelter_pack_mk2` NOT YET UNLOCKED

2× Machine Frame, 2× Control Unit, 2× Pressure Valve, and 2 t Water → 1× Smelter Mk II Upgrade Pack
Blueprint unlocks with the **Industrial Machinery Mk II** research.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 60 W, Mk II: 90 W, Mk III: 180 W
Production tier	T3

Fabricate Fabricator Mk II Upgrade Pack `craft_fabricator_pack_mk2`

NOT YET UNLOCKED

2× Machine Frame, 2× Control Unit, 4× Circuit Panel, and 2 t Water → 1× Fabricator Mk II Upgrade Pack
Blueprint unlocks with the **Industrial Machinery Mk II** research.

Craft time	Mk I: 1.1 h, Mk II: 0.55 h, Mk III: 0.28 h
Power	Mk I: 65 W, Mk II: 97.5 W, Mk III: 195 W
Production tier	T3

Fabricate Oil Pump Mk II Upgrade Pack `craft_oil_pump_pack_mk2` NOT YET UNLOCKED

2× Machine Frame, 1× Control Unit, 3× Pressure Valve, 2× Cobalt Ingot, and 2 t Water → 1× Oil Pump Mk II Upgrade Pack
Blueprint unlocks with the **Advanced Oil Extraction** research.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 60 W, Mk II: 90 W, Mk III: 180 W
Production tier	T3

Fabricate Smelter Mk III Upgrade Pack `craft_smelter_pack_mk3` NOT YET UNLOCKED

6× Machine Frame, 4× Control Unit, 2× Rare Earth Core, 4× Pressure Valve, and 4 t Water → 1× Smelter Mk III Upgrade Pack
Blueprint unlocks with the **Industrial Machinery Mk III** research.

Craft time	Mk I: 1.5 h, Mk II: 0.75 h, Mk III: 0.38 h
Power	Mk I: 90 W, Mk II: 135 W, Mk III: 270 W
Production tier	T3

Fabricate Fabricator Mk III Upgrade Pack `craft_fabricator_pack_mk3` NOT YET UNLOCKED

4× Machine Frame, 4× Control Unit, 6× Circuit Panel, 3× Rare Earth Core, and 4 t Water → 1× Fabricator Mk III Upgrade Pack
Blueprint unlocks with the **Industrial Machinery Mk III** research.

Craft time	Mk I: 1.6 h, Mk II: 0.8 h, Mk III: 0.4 h
Power	Mk I: 95 W, Mk II: 142.5 W, Mk III: 285 W
Production tier	T3

Iron → Gas Pipe Segment `craft_gas_pipe_segment` NOT YET UNLOCKED

2× Iron Ingot → 1× Gas Pipe Segment
Blueprint unlocks by completing the order **Helios, Iron Production**.

Craft time	Mk I: 0.1 h, Mk II: 0.05 h, Mk III: 0.03 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T1

Iron + Silicon → Liquid Pipe Segment `craft_liquid_pipe_segment` NOT YET UNLOCKED

1× Iron Ingot and 1× Silicon → 1× Liquid Pipe Segment
Blueprint unlocks by completing the order **Spire, Silicon Stock**.

Craft time	Mk I: 0.12 h, Mk II: 0.06 h, Mk III: 0.03 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T1

Iron + Titanium → Power Line Segment `craft_power_line_segment` NOT YET UNLOCKED

1× Iron Ingot and 1× Titanium Ingot → 1× Power Line Segment
Blueprint unlocks by completing the order **Vestibule, Utility Conduit Stock**.

Craft time	Mk I: 0.14 h, Mk II: 0.07 h, Mk III: 0.04 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T1

Gas Segments + Valve → Gas Bridge `craft_gas_pipe_bridge` NOT YET UNLOCKED

2× Gas Pipe Segment and 1× Pressure Valve → 1× Gas Pipe Bridge
Blueprint unlocks by completing the order **Helios, Iron Production**.

Craft time	Mk I: 0.18 h, Mk II: 0.09 h, Mk III: 0.05 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T2

Liquid Segments + Valve → Liquid Bridge `craft_liquid_pipe_bridge`
NOT YET UNLOCKED

2× Liquid Pipe Segment and 1× Pressure Valve → 1× Liquid Pipe Bridge
Blueprint unlocks by completing the order **Spire, Silicon Stock**.

Craft time	Mk I: 0.2 h, Mk II: 0.1 h, Mk III: 0.05 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T2

Power Segments + Circuit → Power Bridge `craft_power_line_bridge` NOT YET UNLOCKED

2× Power Line Segment and 1× Circuit Panel → 1× Power Line Bridge
Blueprint unlocks by completing the order **Vestibule, Utility Conduit Stock**.

Craft time	Mk I: 0.22 h, Mk II: 0.11 h, Mk III: 0.06 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T2

Iron + Glass → Pressure Valve `craft_pressure_valve` NOT YET UNLOCKED

1× Iron Ingot and 1× Glass → 1× Pressure Valve
Blueprint unlocks by completing the order **Spire, Cobalt Run**.

Craft time	Mk I: 0.12 h, Mk II: 0.06 h, Mk III: 0.03 h
Power	Mk I: 25 W, Mk II: 37.5 W, Mk III: 75 W
Production tier	T1

Iron Ingot + Titanium Ingot → Machine Frame `craft_machine_frame` NOT YET UNLOCKED

3× Iron Ingot, 1× Titanium Ingot, and 2 t Water → 1× Machine Frame
Blueprint unlocks by completing the order **Helios, Battery Order**.

Craft time	Mk I: 0.22 h, Mk II: 0.11 h, Mk III: 0.06 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T1

Iron + Glass → Circuit Panel `craft_circuit_panel` NOT YET UNLOCKED

1× Iron Ingot, 2× Glass, and 1 t Water → 1× Circuit Panel
Blueprint unlocks by completing the order **Spire, Optical Glass**.

Craft time	Mk I: 0.18 h, Mk II: 0.09 h, Mk III: 0.05 h
Power	Mk I: 30 W, Mk II: 45 W, Mk III: 90 W
Production tier	T1

Circuit Panel + Titanium Ingot + Glass → Control Unit `craft_control_unit`

NOT YET UNLOCKED

1× Circuit Panel, 1× Titanium Ingot, 1× Glass, and 2 t Water → 1× Control Unit
Blueprint unlocks by completing the order **Spire, Circuit Order**.

Craft time	Mk I: 0.3 h, Mk II: 0.15 h, Mk III: 0.08 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Cobalt Ingot + Iron Ingot + Glass → Battery Cell `craft_battery_cell`

NOT YET UNLOCKED

1× Cobalt Ingot, 1× Iron Ingot, 1× Glass, and 1 t Water → 1× Battery Cell
Blueprint unlocks by completing the order **Helios, Titanium Run**.

Craft time	Mk I: 0.22 h, Mk II: 0.11 h, Mk III: 0.06 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T1

Titanium Ingot + Gas Pipe Segments → Thermal Cap Kit `craft_thermal_cap_kit`

UNLOCKED

2× Titanium Ingot and 2× Gas Pipe Segment → 1× Thermal Cap Kit
Blueprint unlocks with the **Thermal Cap** research.

Craft time	Mk I: 0.28 h, Mk II: 0.14 h, Mk III: 0.07 h
Power	Mk I: 38 W, Mk II: 57 W, Mk III: 114 W
Production tier	T2

Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor

`craft_turbine_rotor` NOT YET UNLOCKED

2× Titanium Ingot, 1× Cobalt Ingot, 1× Rare Earth Core, and 4 t Steam → 1× Turbine Rotor
Blueprint unlocks by completing the order **Helios, Thermal Cap Kit Order**.

Craft time	Mk I: 0.34 h, Mk II: 0.17 h, Mk III: 0.09 h
Power	Mk I: 42 W, Mk II: 63 W, Mk III: 126 W
Production tier	T1

Iron + Titanium + Glass → Tank Lining `craft_tank_lining` NOT YET UNLOCKED

2× Iron Ingot, 1× Titanium Ingot, 1× Glass, and 2 t Water → 1× Tank Lining
Blueprint unlocks by completing the order **Helios, Bulk Iron Run**.

Craft time	Mk I: 0.22 h, Mk II: 0.11 h, Mk III: 0.06 h
Power	Mk I: 38 W, Mk II: 57 W, Mk III: 114 W
Production tier	T1

Iron + Glass + Liquid Pipe Segment → Water Pump `craft_water_pump` UNLOCKED

2× Iron Ingot, 2× Glass, and 4× Liquid Pipe Segment → 1× Water Pump
Blueprint unlocks with the **Hydrology Survey** research.

Craft time	Mk I: 0.3 h, Mk II: 0.15 h, Mk III: 0.08 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T2

Iron + Titanium + Valve + Panel → Oil Pump `craft_oil_pump` NOT YET UNLOCKED

2× Iron Ingot, 1× Titanium Ingot, 1× Pressure Valve, and 1× Circuit Panel → 1× Oil Pump
Blueprint unlocks with the **Petroleum Survey** research.

Craft time	Mk I: 0.38 h, Mk II: 0.19 h, Mk III: 0.1 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Iron + Oil → Lubricant + Tar `craft_lubricant` NOT YET UNLOCKED

1× Iron Ingot and 3 t Oil → 1× Lubricant and + byproduct 1× Tar
Blueprint unlocks by completing the order **Vestibule, Liquid Pipe Order**.

Craft time	Mk I: 0.12 h, Mk II: 0.06 h, Mk III: 0.03 h
Power	Mk I: 30 W, Mk II: 45 W, Mk III: 90 W
Production tier	T1

Glass + Oil → Plastic + Tar `craft_plastic` NOT YET UNLOCKED

1× Glass and 4 t Oil → 1× Plastic and + byproduct 1× Tar
Blueprint unlocks by completing the order **Spire, Magnetic Stator Build**.

Craft time	Mk I: 0.14 h, Mk II: 0.07 h, Mk III: 0.04 h
Power	Mk I: 32 W, Mk II: 48 W, Mk III: 96 W
Production tier	T1

Cobalt + Oil → Rubber + Tar `craft_rubber` NOT YET UNLOCKED

1× Cobalt Ingot and 2 t Oil → 1× Rubber and + byproduct 1× Tar
Blueprint unlocks by completing the order **Vestibule, Pipe Network Run**.

Craft time	Mk I: 0.12 h, Mk II: 0.06 h, Mk III: 0.03 h
Power	Mk I: 28 W, Mk II: 42 W, Mk III: 84 W
Production tier	T1

Heavy Oil Cracking: Oil → Tar `craft_tar` NOT YET UNLOCKED

5 t Oil → 2× Tar
Blueprint unlocks by completing the order **Vestibule, Polymer Stockpile**.

Craft time	Mk I: 0.15 h, Mk II: 0.08 h, Mk III: 0.04 h
Power	Mk I: 32 W, Mk II: 48 W, Mk III: 96 W
Production tier	T1

Plastic + Forage + Water → Reinforced Biopolymer `craft_reinforced_biopolymer` NOT YET UNLOCKED

2× Plastic, 6× Plant Forage, and 2 t Water → 4× Reinforced Biopolymer
Blueprint unlocks with the **Advanced Biopolymers** research.

Craft time	Mk I: 0.28 h, Mk II: 0.14 h, Mk III: 0.07 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Reinforced Biopolymer + Forage + Water → Bio Enrichment Compound

`craft_enrichment_compound` NOT YET UNLOCKED

1× Reinforced Biopolymer, 6× Plant Forage, and 2 t Water → 4× Bio Enrichment Compound
Blueprint unlocks with the **Enrichment Chemistry** research.

Craft time	Mk I: 0.32 h, Mk II: 0.16 h, Mk III: 0.08 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit

`craft_drone_station_kit` NOT YET UNLOCKED

2× Machine Frame, 1× Control Unit, 2× Gas Pipe Segment, 2× Liquid Pipe Segment, and 3 t Water → 1× Drone Depot Kit
Blueprint unlocks by completing the order **Helios, Frame Order**.

Craft time	Mk I: 0.5 h, Mk II: 0.25 h, Mk III: 0.13 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Frame + Control + Panel → Drone Depot Kit (Medium) `craft_drone_station_kit_medium`

NOT YET UNLOCKED

4× Machine Frame, 2× Control Unit, 2× Circuit Panel, and 4 t Water → 1× Drone Depot Kit (Medium)
Blueprint unlocks by completing the order **Helios, Mid-Cargo Order**.

Craft time	Mk I: 0.65 h, Mk II: 0.33 h, Mk III: 0.16 h
Power	Mk I: 50 W, Mk II: 75 W, Mk III: 150 W
Production tier	T3

Frame + Control + Panel → Drone Depot Kit (Large) `craft_drone_station_kit_large`

NOT YET UNLOCKED

8× Machine Frame, 3× Control Unit, 4× Circuit Panel, and 6 t Water → 1× Drone Depot Kit (Large)
Blueprint unlocks by completing the order **Helios, Drone Hub Stockpile**.

Craft time	Mk I: 0.85 h, Mk II: 0.43 h, Mk III: 0.21 h
Power	Mk I: 55 W, Mk II: 82.5 W, Mk III: 165 W
Production tier	T3

Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit `craft_drone_service_station_kit`

NOT YET UNLOCKED

2× Machine Frame, 1× Control Unit, 2× Circuit Panel, 1× Battery Cell, and 1× Liquid Pipe Segment → 1× Drone Service Station Kit
Blueprint unlocks by completing the order **Spire, Drone Power Trial**.

Craft time	Mk I: 0.5 h, Mk II: 0.25 h, Mk III: 0.13 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Frame + Control + Panel → Mining Drill Kit `craft_mining_drill_kit`

NOT YET UNLOCKED

3× Machine Frame, 1× Control Unit, 1× Circuit Panel, and 2 t Water → 1× Mining Drill Kit
Blueprint unlocks by completing the order **Helios, Cargo Pod Run**.

Craft time	Mk I: 0.45 h, Mk II: 0.23 h, Mk III: 0.11 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Frame + Control + Panel + Rotor → Industrial Mining Drill Kit

`craft_mining_drill_industrial_kit` NOT YET UNLOCKED

5× Machine Frame, 2× Control Unit, 2× Circuit Panel, 1× Turbine Rotor, and 3 t Water → 1× Industrial Mining Drill Kit

Blueprint unlocks by completing the order **Helios, Mining Drill Kit Build**.

Craft time	Mk I: 0.65 h, Mk II: 0.33 h, Mk III: 0.16 h
Power	Mk I: 52 W, Mk II: 78 W, Mk III: 156 W
Production tier	T3

Frame + Control + Panel + Rotors → Heavy Mining Drill Kit

`craft_mining_drill_heavy_kit` NOT YET UNLOCKED

8× Machine Frame, 3× Control Unit, 4× Circuit Panel, 2× Turbine Rotor, and 4 t Water → 1× Heavy Mining Drill Kit

Blueprint unlocks by completing the order **Helios, Tether Project**.

Craft time	Mk I: 0.85 h, Mk II: 0.43 h, Mk III: 0.21 h
Power	Mk I: 58 W, Mk II: 87 W, Mk III: 174 W
Production tier	T3

Rare Earth Core + Titanium Ingot + Control → Drone (Small) `craft_drone_small`

NOT YET UNLOCKED

1× Rare Earth Core, 1× Titanium Ingot, 1× Control Unit, and 2 t Water → 1× Drone (Small)

Blueprint unlocks by completing the order **Helios, Rotor Run**.

Craft time	Mk I: 0.55 h, Mk II: 0.28 h, Mk III: 0.14 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Rare Earth Core + Titanium Ingot + Control → Drone (Medium) `craft_drone_medium`

NOT YET UNLOCKED

2× Rare Earth Core, 2× Titanium Ingot, 1× Control Unit, and 3 t Water → 1× Drone (Medium)
Blueprint unlocks by completing the order **Helios, Mid-Drone Build**.

Craft time	Mk I: 0.75 h, Mk II: 0.38 h, Mk III: 0.19 h
Power	Mk I: 50 W, Mk II: 75 W, Mk III: 150 W
Production tier	T3

Rare Earth Core + Titanium Ingot + Control → Drone (Large) `craft_drone_large`

NOT YET UNLOCKED

3× Rare Earth Core, 3× Titanium Ingot, 2× Control Unit, and 4 t Water → 1× Drone (Large)
Blueprint unlocks by completing the order **Helios, Heavy Drone Build**.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 55 W, Mk II: 82.5 W, Mk III: 165 W
Production tier	T3

Rare Earth Core + Rotor → Electric Thruster `craft_electric_thruster`

NOT YET UNLOCKED

2× Rare Earth Core, 1× Turbine Rotor, and 2 t Steam → 1× Electric Thruster
Blueprint unlocks by completing the order **Spire, Rare Earth Order**.

Craft time	Mk I: 0.55 h, Mk II: 0.28 h, Mk III: 0.14 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T2

Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster

`craft_heli_thruster` NOT YET UNLOCKED

2× Rare Earth Core, 1× Control Unit, 2× Lubricant, 1× Rubber, and 2 t Water → 1× Heli Thruster
Blueprint unlocks by completing the order **Vestibule, Refueling Pad Order**.

Craft time	Mk I: 0.65 h, Mk II: 0.33 h, Mk III: 0.16 h
Power	Mk I: 50 W, Mk II: 75 W, Mk III: 150 W
Production tier	T3

Titanium + Glass → Cargo Pod (Small) `craft_cargo_pod_small` NOT YET UNLOCKED

1× Titanium Ingot, 1× Glass, and 1 t Steam → 1× Cargo Pod (Small)
Blueprint unlocks by completing the order **Helios, Pilot Drone Fleet Order**.

Craft time	Mk I: 0.2 h, Mk II: 0.1 h, Mk III: 0.05 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T1

Titanium + Glass + Panel → Cargo Pod (Medium) `craft_cargo_pod_medium`

NOT YET UNLOCKED

2× Titanium Ingot, 1× Glass, 1× Circuit Panel, and 2 t Steam → 1× Cargo Pod (Medium)
Blueprint unlocks by completing the order **Helios, Mid-Cargo Build**.

Craft time	Mk I: 0.3 h, Mk II: 0.15 h, Mk III: 0.08 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Titanium + Glass + Panel → Cargo Pod (Large) `craft_cargo_pod_large`

NOT YET UNLOCKED

3× Titanium Ingot, 2× Glass, 1× Circuit Panel, and 3 t Steam → 1× Cargo Pod (Large)
Blueprint unlocks by completing the order **Helios, Cargo Megaorder**.

Craft time	Mk I: 0.42 h, Mk II: 0.21 h, Mk III: 0.11 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T2

Titanium Ingot + Glass + Cell → Battery Pack `craft_battery_pack`

NOT YET UNLOCKED

1× Titanium Ingot, 1× Glass, and 1× Battery Cell → 1× Battery Pack
Blueprint unlocks by completing the order **Spire, Pressure Hardware**.

Craft time	Mk I: 0.18 h, Mk II: 0.09 h, Mk III: 0.05 h
Power	Mk I: 32 W, Mk II: 48 W, Mk III: 96 W
Production tier	T2

Titanium Ingot + Lubricant + Rubber → Oil Tank (Small) `craft_oil_tank_small`

NOT YET UNLOCKED

1× Titanium Ingot, 1× Lubricant, 1× Rubber, and 1 t Water → 1× Oil Tank (Small)
Blueprint unlocks by completing the order **Vestibule, Heli Thruster Order**.

Craft time	Mk I: 0.22 h, Mk II: 0.11 h, Mk III: 0.06 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T2

Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium)

`craft_oil_tank_medium` NOT YET UNLOCKED

2× Titanium Ingot, 1× Lubricant, 1× Rubber, 1× Tank Lining, and 2 t Water → 1× Oil Tank (Medium)
Blueprint unlocks by completing the order **Vestibule, Oil Tank Order**.

Craft time	Mk I: 0.32 h, Mk II: 0.16 h, Mk III: 0.08 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large)

`craft_oil_tank_large` NOT YET UNLOCKED

3× Titanium Ingot, 2× Lubricant, 1× Rubber, 2× Tank Lining, and 3 t Water → 1× Oil Tank (Large)
Blueprint unlocks by completing the order **Vestibule, Mid Tank Order**.

Craft time	Mk I: 0.45 h, Mk II: 0.23 h, Mk III: 0.11 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T2

Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop

`craft_coolant_loop` NOT YET UNLOCKED

4× Liquid Pipe Segment, 2× Pressure Valve, 1× Rare Earth Core, 2× Titanium Ingot, 2× Tar, and 4 t Water → 1× Coolant Loop
Blueprint unlocks by completing the order **Spire, Neutronium Order**.

Craft time	Mk I: 0.75 h, Mk II: 0.38 h, Mk III: 0.19 h
Power	Mk I: 50 W, Mk II: 75 W, Mk III: 150 W
Production tier	T2

Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor

`craft_neutron_capacitor` NOT YET UNLOCKED

1× Neutronium Bar, 2× Battery Cell, 2× Control Unit, 1× Rare Earth Core, 1× Tar, 4 t Water, and 4 t Steam → 1× Neutron Capacitor
Blueprint unlocks by completing the order **Spire, Cobalt Stockpile**.

Craft time	Mk I: 1.2 h, Mk II: 0.6 h, Mk III: 0.3 h
Power	Mk I: 70 W, Mk II: 105 W, Mk III: 210 W
Production tier	T3

Frame + Control + Panel → Seed Maker Kit `craft_seed_maker_kit` NOT YET UNLOCKED

2× Machine Frame, 1× Control Unit, 2× Circuit Panel, and 3 t Water → 1× Seed Maker Kit
Blueprint unlocks with the **Seed Maker** research.

Craft time	Mk I: 0.5 h, Mk II: 0.25 h, Mk III: 0.13 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T3

Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit

`craft_plant_terraformer_kit` NOT YET UNLOCKED

5× Machine Frame, 3× Control Unit, 5× Circuit Panel, 4× Liquid Pipe Segment, and 10 t Water → 1× Plant Terraformer Kit
Blueprint unlocks with the **Plant Terraformer** research.

Craft time	Mk I: 1.25 h, Mk II: 0.63 h, Mk III: 0.31 h
Power	Mk I: 80 W, Mk II: 120 W, Mk III: 240 W
Production tier	T3

Frame + Panel + Glass → Grow Lamp Kit `craft_grow_lamp_kit` NOT YET UNLOCKED

1× Machine Frame, 2× Circuit Panel, and 2× Glass → 1× Grow Lamp Kit
Blueprint unlocks with the **Grow Lamp** research.

Craft time	Mk I: 0.35 h, Mk II: 0.18 h, Mk III: 0.09 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T2

Frame + Pipes + Valve → Sprinkler Kit `craft_sprinkler_kit` NOT YET UNLOCKED

1× Machine Frame, 2× Liquid Pipe Segment, 1× Pressure Valve, and 2 t Water → 1× Sprinkler Kit
Blueprint unlocks with the **Sprinkler** research.

Craft time	Mk I: 0.38 h, Mk II: 0.19 h, Mk III: 0.1 h
Power	Mk I: 38 W, Mk II: 57 W, Mk III: 114 W
Production tier	T2

Frame + Control + Panel → Dispenser Kit `craft_dispenser_kit` NOT YET UNLOCKED

1× Machine Frame, 1× Control Unit, and 1× Circuit Panel → 1× Dispenser Kit
Blueprint unlocks with the **Dispenser** research.

Craft time	Mk I: 0.4 h, Mk II: 0.2 h, Mk III: 0.1 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T3

Frame + Iron + Panel → Waste Processor Kit `craft_garbage_disposal_kit`

NOT YET UNLOCKED

2× Machine Frame, 4× Iron Ingot, and 1× Circuit Panel → 1× Waste Processor Kit
Blueprint unlocks with the **Waste Processing** research.

Craft time	Mk I: 0.4 h, Mk II: 0.2 h, Mk III: 0.1 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Titanium + Gas Pipe Segments → Exotic Gas Cap Kit `craft_exotic_gas_cap_kit`

NOT YET UNLOCKED

2× Titanium Ingot and 2× Gas Pipe Segment → 1× Exotic Gas Cap Kit
Blueprint unlocks with the **Exotic Fluids** research.

Craft time	Mk I: 0.32 h, Mk II: 0.16 h, Mk III: 0.08 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Titanium + Liquid Pipe Segments + Valve → Exotic Spring Tap Kit `craft_exotic_spring_tap_kit`

NOT YET UNLOCKED

2× Titanium Ingot, 2× Liquid Pipe Segment, and 1× Pressure Valve → 1× Exotic Spring Tap Kit
Blueprint unlocks with the **Exotic Fluids** research.

Craft time	Mk I: 0.34 h, Mk II: 0.17 h, Mk III: 0.09 h
Power	Mk I: 42 W, Mk II: 63 W, Mk III: 126 W
Production tier	T2

Tar + Glass → Fertilizer `craft_fertilizer` UNLOCKED

1× Tar, 1× Glass, and 2 t Water → 2× Fertilizer
Blueprint unlocks by completing the Bio Order **Soil Enrichment Assay**.

Craft time	Mk I: 0.18 h, Mk II: 0.09 h, Mk III: 0.05 h
Power	Mk I: 32 W, Mk II: 48 W, Mk III: 96 W
Production tier	T2

Tar + Glass + Panel → Fertilizer Mk II `craft_fertilizer_mk2` NOT YET UNLOCKED

2× Tar, 2× Glass, 1× Circuit Panel, and 3 t Water → 2× Fertilizer Mk II
Blueprint unlocks by completing the Bio Order **Hydrothermal Fragment Manifest**.

Craft time	Mk I: 0.3 h, Mk II: 0.15 h, Mk III: 0.08 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III `craft_fertilizer_mk3` NOT YET UNLOCKED

3× Tar, 3× Glass, 1× Neutron Capacitor, 2× Rare Earth Core, and 4 t Water → 2× Fertilizer Mk III
Blueprint unlocks by completing the Bio Order **Deep-Trench Capstone**.

Craft time	Mk I: 0.55 h, Mk II: 0.28 h, Mk III: 0.14 h
Power	Mk I: 52 W, Mk II: 78 W, Mk III: 156 W
Production tier	T4

Plastic + Rare Earth Core → Growth Accelerant `craft_growth_accelerant`

NOT YET UNLOCKED

1× Plastic, 1× Rare Earth Core, and 2 t Water → 2× Growth Accelerant
Blueprint unlocks by completing the Bio Order **Volcanic Grand Compendium**.

Craft time	Mk I: 0.28 h, Mk II: 0.14 h, Mk III: 0.07 h
Power	Mk I: 38 W, Mk II: 57 W, Mk III: 114 W
Production tier	T2

Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier `craft_yield_amplifier`

NOT YET UNLOCKED

1× Neutron Capacitor, 2× Control Unit, 2× Rare Earth Core, 1× Coolant Loop, and 4 t Water → 1× Yield Amplifier
Blueprint unlocks by completing the order **Spire, Neutron Capacitor Order**.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 65 W, Mk II: 97.5 W, Mk III: 195 W
Production tier	T4

Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II `craft_plant_terraformer_pack_mk2`

NOT YET UNLOCKED

4× Control Unit, 6× Circuit Panel, 3× Rare Earth Core, 4× Pressure Valve, and 6 t Water → 1× Plant Terraformer Mk II Upgrade Pack
Blueprint unlocks with the **Plant Terraformer Mk II** research.

Craft time	Mk I: 1.5 h, Mk II: 0.75 h, Mk III: 0.38 h
Power	Mk I: 90 W, Mk II: 135 W, Mk III: 270 W
Production tier	T3

Panel + Glass + Rare Earth Core → Grow Lamp Pack Mk II `craft_grow_lamp_pack_mk2`

NOT YET UNLOCKED

3× Circuit Panel, 4× Glass, and 1× Rare Earth Core → 1× Grow Lamp Mk II Upgrade Pack
Blueprint unlocks by completing the order **Spire, Optics Stockpile**.

Craft time	Mk I: 0.45 h, Mk II: 0.23 h, Mk III: 0.11 h
Power	Mk I: 42 W, Mk II: 63 W, Mk III: 126 W
Production tier	T2

Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III `craft_grow_lamp_pack_mk3`

NOT YET UNLOCKED

1× Neutron Capacitor, 3× Rare Earth Core, 2× Control Unit, and 3 t Water → 1× Grow Lamp Mk III Upgrade Pack
Blueprint unlocks by completing the order **Spire, Capacitor Bulk Order**.

Craft time	Mk I: 0.75 h, Mk II: 0.38 h, Mk III: 0.19 h
Power	Mk I: 55 W, Mk II: 82.5 W, Mk III: 165 W
Production tier	T4

Plastic + Coolant Loop + Valves → Sprinkler Pack Mk II `craft_sprinkler_pack_mk2`

NOT YET UNLOCKED

4× Plastic, 1× Coolant Loop, and 2× Pressure Valve → 1× Sprinkler Mk II Upgrade Pack
Blueprint unlocks by completing the order **Spire, Polymer Megastock**.

Craft time	Mk I: 0.5 h, Mk II: 0.25 h, Mk III: 0.13 h
Power	Mk I: 44 W, Mk II: 66 W, Mk III: 132 W
Production tier	T3

Controls + Panels + Coolant Loop → Sprinkler Pack Mk III

`craft_sprinkler_pack_mk3` NOT YET UNLOCKED

2× Control Unit, 4× Circuit Panel, 2× Coolant Loop, and 4 t Water → 1× Sprinkler Mk III Upgrade Pack
Blueprint unlocks by completing the order **Spire, Avionics Megastock**.

Craft time	Mk I: 0.85 h, Mk II: 0.43 h, Mk III: 0.21 h
Power	Mk I: 58 W, Mk II: 87 W, Mk III: 174 W
Production tier	T3

Frames + Controls + Panels + Valves → Feed Maker Pack Mk II

`craft_feed_maker_pack_mk2` NOT YET UNLOCKED

2× Machine Frame, 2× Control Unit, 4× Circuit Panel, 2× Pressure Valve, and 4 t Water → 1× Feed Maker Mk II Upgrade Pack
Blueprint unlocks with the **Feed Maker Mk II** research.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 60 W, Mk II: 90 W, Mk III: 180 W
Production tier	T3

Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II

`craft_habitat_pack_mk2` NOT YET UNLOCKED

2× Neutron Capacitor, 3× Rare Earth Core, 4× Control Unit, 3× Coolant Loop, and 6 t Water → 1× Habitat Mk II Upgrade Pack
Blueprint unlocks with the **Habitat Engineering Mk II** research.

Craft time	Mk I: 1.4 h, Mk II: 0.7 h, Mk III: 0.35 h
Power	Mk I: 110 W, Mk II: 165 W, Mk III: 330 W
Production tier	T4

Lead Plate `craft_lead_plate` NOT YET UNLOCKED

2× Lead Ingot → 1× Lead Plate
Blueprint unlocks by completing the order **Helios, Lead Consignment**.

Craft time	Mk I: 0.2 h, Mk II: 0.1 h, Mk III: 0.05 h
Power	Mk I: 30 W, Mk II: 45 W, Mk III: 90 W
Production tier	T1

Oxygen Upgrade Pack Mk IV `craft_oxygen_upgrade_pack_mk4` NOT YET UNLOCKED

4× Lead Plate, 2× Machine Frame, 3× Circuit Panel, and 2× Cobalt Ingot → 1× Oxygen Upgrade Pack Mk IV
Blueprint unlocks with the **Oxygen Generator Mk IV** research.

Craft time	Mk I: 2 h, Mk II: 1 h, Mk III: 0.5 h
Power	Mk I: 220 W, Mk II: 330 W, Mk III: 660 W
Production tier	T2

Heat Upgrade Pack Mk IV `craft_heat_upgrade_pack_mk4` NOT YET UNLOCKED

4× Lead Plate, 2× Machine Frame, 3× Circuit Panel, and 2× Cobalt Ingot → 1× Heat Upgrade Pack Mk IV
Blueprint unlocks with the **Heat Generator Mk IV** research.

Craft time	Mk I: 2 h, Mk II: 1 h, Mk III: 0.5 h
Power	Mk I: 220 W, Mk II: 330 W, Mk III: 660 W
Production tier	T2

Pressure Upgrade Pack Mk IV `craft_pressure_upgrade_pack_mk4` NOT YET UNLOCKED

4× Lead Plate, 2× Machine Frame, 3× Circuit Panel, and 2× Cobalt Ingot → 1× Pressure Upgrade Pack Mk IV
Blueprint unlocks with the **Pressure Generator Mk IV** research.

Craft time	Mk I: 2 h, Mk II: 1 h, Mk III: 0.5 h
Power	Mk I: 220 W, Mk II: 330 W, Mk III: 660 W
Production tier	T2

Lead Cask `craft_lead_cask` NOT YET UNLOCKED

3× Lead Plate and 1× Machine Frame → 1× Lead Cask
Blueprint unlocks by completing the order **Helios, Plate Order**.

Craft time	Mk I: 0.6 h, Mk II: 0.3 h, Mk III: 0.15 h
Power	Mk I: 36 W, Mk II: 54 W, Mk III: 108 W
Production tier	T2

Shield Plating `craft_shield_plating` NOT YET UNLOCKED

4× Lead Plate and 1× Machine Frame → 1× Shield Plating
Blueprint unlocks by completing the order **Vestibule, Shielded Transport Trial**.

Craft time	Mk I: 0.8 h, Mk II: 0.4 h, Mk III: 0.2 h
Power	Mk I: 36 W, Mk II: 54 W, Mk III: 108 W
Production tier	T2

Lightning Rod `craft_lightning_rod_kit` NOT YET UNLOCKED

1× Machine Frame, 4× Battery Cell, and 2× Circuit Panel → 1× Lightning Rod
Blueprint unlocks with the **Lightning Rods** research.

Craft time	Mk I: 1 h, Mk II: 0.5 h, Mk III: 0.25 h
Power	Mk I: 40 W, Mk II: 60 W, Mk III: 120 W
Production tier	T2

Smelter Recipes

The Smelter refines raw ore into stock: one ore in, one refined unit out. Select a blueprint with `smelter.set_recipe` and keep the input buffer fed. Every blueprint is listed here from day one, unlocked or not.

Iron Ore → Iron Ingot `smelt_iron_ingot` UNLOCKED

1× Iron Ore → 1× Iron Ingot
Available from the start.

Craft time	Mk I: 0.08 h, Mk II: 0.04 h, Mk III: 0.02 h
Power	Mk I: 20 W, Mk II: 30 W, Mk III: 60 W
Production tier	T0

Silicon → Glass `smelt_glass` NOT YET UNLOCKED

1× Silicon → 1× Glass
Blueprint unlocks by completing the order **Spire, Silicon Bootstrap**.

Craft time	Mk I: 0.08 h, Mk II: 0.04 h, Mk III: 0.02 h
Power	Mk I: 20 W, Mk II: 30 W, Mk III: 60 W
Production tier	T0

Titanium → Titanium Ingot `smelt_titanium_ingot` NOT YET UNLOCKED

1× Titanium → 1× Titanium Ingot
Blueprint unlocks by completing the order **Helios, Iron Bootstrap**.

Craft time	Mk I: 0.1 h, Mk II: 0.05 h, Mk III: 0.03 h
Power	Mk I: 28 W, Mk II: 42 W, Mk III: 84 W
Production tier	T0

Cobalt → Cobalt Ingot `smelt_cobalt_ingot` NOT YET UNLOCKED

1× Cobalt → 1× Cobalt Ingot
Blueprint unlocks by completing the order **Spire, Avionics Run**.

Craft time	Mk I: 0.1 h, Mk II: 0.05 h, Mk III: 0.03 h
Power	Mk I: 28 W, Mk II: 42 W, Mk III: 84 W
Production tier	T0

Rare Earth → Rare Earth Core `smelt_rare_earth_core` NOT YET UNLOCKED

1× Rare Earth → 1× Rare Earth Core
Blueprint unlocks by completing the order **Spire, Control Run**.

Craft time	Mk I: 0.14 h, Mk II: 0.07 h, Mk III: 0.04 h
Power	Mk I: 35 W, Mk II: 52.5 W, Mk III: 105 W
Production tier	T0

Neutronium → Neutronium Bar `smelt_neutronium_bar` NOT YET UNLOCKED

1× Neutronium → 1× Neutronium Bar
Blueprint unlocks by completing the order **Spire, Polymer Optics Run**.

Craft time	Mk I: 0.2 h, Mk II: 0.1 h, Mk III: 0.05 h
Power	Mk I: 45 W, Mk II: 67.5 W, Mk III: 135 W
Production tier	T0

Lead Ore → Lead Ingot `smelt_lead_ingot` NOT YET UNLOCKED

1× Lead Ore → 1× Lead Ingot

Blueprint unlocks by completing the order **Helios, Lead Survey Stock**.

Craft time	Mk I: 0.1 h, Mk II: 0.05 h, Mk III: 0.03 h
Power	Mk I: 22 W, Mk II: 33 W, Mk III: 66 W
Production tier	T0

Feed Maker Recipes

The Feed Maker presses creature feeds from `forage` and life forms. Each feed's blueprint unlocks by completing its creature's Bio Order at the Bio Exchange.

Forage + Sea Algae + Snow Moss → Salt Tortoise Feed `craft_feed_salt_tortoise`

NOT YET UNLOCKED

100× Plant Forage, 1× Sea Algae, and 1× Snow Moss → 20× Salt Tortoise Feed
Blueprint unlocks by completing the Bio Order **Reef Fragment Survey**.

Craft time	Mk I: 0.3 h, Mk II: 0.2 h
Power	Mk I: 30 W, Mk II: 60 W
Production tier	T1

Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed

`craft_feed_magmatic_annelid` NOT YET UNLOCKED

100× Plant Forage, 1× Lava Algae, and 1× Cave Moss → 20× Magmatic Annelid Feed
Blueprint unlocks by completing the Bio Order **Basalt Tissue Sample**.

Craft time	Mk I: 0.3 h, Mk II: 0.2 h
Power	Mk I: 30 W, Mk II: 60 W
Production tier	T1

Forage + Cave Fungus + Frost Fungus + Coral Fungus → Mycelial Husk Feed

`craft_feed_mycelial_husk` NOT YET UNLOCKED

100× Plant Forage, 1× Cave Fungus, 1× Frost Fungus, and 1× Coral Fungus → 20× Mycelial Husk Feed
Blueprint unlocks by completing the Bio Order **Trench Specimen Panel**.

Craft time	Mk I: 0.34 h, Mk II: 0.23 h
Power	Mk I: 32 W, Mk II: 64 W
Production tier	T1

Forage + Magma Crust + Stone Mat → Mantle Strider Feed `craft_feed_mantle_strider`

NOT YET UNLOCKED

100× Plant Forage, 1× Magma Crust, and 1× Stone Mat → 20× Mantle Strider Feed
Blueprint unlocks by completing the Bio Order **Obsidian Tissue Lot**.

Craft time	Mk I: 0.34 h, Mk II: 0.23 h
Power	Mk I: 32 W, Mk II: 64 W
Production tier	T1

Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed

`craft_feed_glasswing_mantis` NOT YET UNLOCKED

100× Plant Forage, 1× Heat Lichen, and 1× Frost Lichen → 20× Glasswing Mantis Feed
Blueprint unlocks by completing the Bio Order **Thermal Bio Compendium**.

Craft time	Mk I: 0.34 h, Mk II: 0.23 h
Power	Mk I: 32 W, Mk II: 64 W
Production tier	T1

Forage + Cold Spores + Hot Spores → Veil Mantle Feed `craft_feed_veil_mantle`

UNLOCKED

100× Plant Forage, 1× Cold Spores, and 1× Hot Spores → 20× Veil Mantle Feed
Blueprint unlocks by completing the Bio Order **Veil Mantle Diet Survey**.

Craft time	Mk I: 0.34 h, Mk II: 0.23 h
Power	Mk I: 32 W, Mk II: 64 W
Production tier	T1

Forage + Stone Lichen + Salt Crust → Vault Crab Feed `craft_feed_vault_crab`

NOT YET UNLOCKED

100× Plant Forage, 1× Stone Lichen, and 1× Salt Crust → 20× Vault Crab Feed
Blueprint unlocks by completing the Bio Order **Cold-Seep Assay**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed `craft_feed_tidal_cephalopod`

NOT YET UNLOCKED

100× Plant Forage, 1× Brine Plankton, 1× Deep Algae, and 1× Tide Moss → 20× Tidal Cephalopod Feed
Blueprint unlocks by completing the Bio Order **Lagoon Specimen Study**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Ice Crust + Ash Spores → Bone Walker Feed `craft_feed_bone_walker`

UNLOCKED

100× Plant Forage, 1× Ice Crust, and 1× Ash Spores → 20× Bone Walker Feed
Blueprint unlocks by completing the Bio Order **Bone Walker Forage Index**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed

`craft_feed_vent_drifter` NOT YET UNLOCKED

100× Plant Forage, 1× Vent Algae, 1× Steam Moss, and 1× Sea Algae → 20× Vent Drifter Feed
Blueprint unlocks by completing the Bio Order **Caldera-Rim Catalog**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed

`craft_feed_hive_sentinel` NOT YET UNLOCKED

100× Plant Forage, 1× Hot Spores, and 1× Crystal Spores → 20× Hive Sentinel Feed
Blueprint unlocks by completing the Bio Order **Mud-Pot Fragment Batch**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Cave Moss + Heat Crust → Hollow Choir Feed `craft_feed_hollow_choir`

NOT YET UNLOCKED

100× Plant Forage, 1× Cave Moss, and 1× Heat Crust → 20× Hollow Choir Feed
Blueprint unlocks by completing the Bio Order **Subduction Specimen Cache**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed `craft_feed_ferric_sea_lily`

NOT YET UNLOCKED

100× Plant Forage, 1× Shore Lichen, 1× Crystal Spores, and 1× Vent Fungus → 20× Ferric Sea-Lily Feed
Blueprint unlocks by completing the Bio Order **Shoal Specimen Cache**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed `craft_feed_crustal_echo`

NOT YET UNLOCKED

100× Plant Forage, 1× Stone Mat, and 1× Cinder Lichen → 20× Crustal Echo Feed
Blueprint unlocks by completing the Bio Order **Pelagic Bio Survey**.

Craft time	Mk I: 0.4 h, Mk II: 0.27 h
Power	Mk I: 35 W, Mk II: 70 W
Production tier	T1

Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed `craft_feed_glacial_wyrm` UNLOCKED

100× Plant Forage, 1× Ice Algae, 1× Frost Lichen, 1× Vent Algae, and 1× Black Fungus → 20× Glacial Wyrm Feed
Blueprint unlocks by completing the Bio Order **Glacial Wyrm Husbandry Dossier**.

Craft time	Mk I: 0.5 h, Mk II: 0.33 h
Power	Mk I: 42 W, Mk II: 84 W
Production tier	T1

Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed `craft_feed_spire_drake` NOT YET UNLOCKED

100× Plant Forage, 1× Sulfur Moss, 1× Cinder Lichen, 1× Cold Spores, and 1× Stone Lichen → 20× Spire Drake Feed
Blueprint unlocks by completing the Bio Order **Volcanic Fragment Manifest**.

Craft time	Mk I: 0.5 h, Mk II: 0.33 h
Power	Mk I: 42 W, Mk II: 84 W
Production tier	T1

Refiner Recipes

The Refiner purifies raw exotic fluids into creature-grade form, burning `tar` as the refining reagent: raw fluid in, refined fluid out. Blueprints unlock through Bio Orders.

Raw Sulfur Gas + Tar → Sulfur Gas `refine_sulfur_gas` NOT YET UNLOCKED

2× Tar and 4 t Raw Sulfur Gas → 4 t Sulfur Gas
Blueprint unlocks by completing the Bio Order **Eruption-Zone Capstone**.

Craft time	0.3 h
Power	30 W
Production tier	T2

Raw Cryofluid + Tar → Cryofluid `refine_cryofluid` UNLOCKED

2× Tar and 4 t Raw Cryofluid → 4 t Cryofluid
Blueprint unlocks by completing the Bio Order **Cryofluid Refinement Protocol**.

Craft time	0.3 h
Power	30 W
Production tier	T2

Raw Chlorine + Tar → Chlorine `refine_chlorine` NOT YET UNLOCKED

5× Tar and 4 t Raw Chlorine → 4 t Chlorine
Blueprint unlocks by completing the Bio Order **Geothermal Grand Compendium**.

Craft time	0.4 h
Power	48 W
Production tier	T2

Raw Quicksilver + Tar → Quicksilver `refine_quicksilver` NOT YET UNLOCKED

5× Tar and 4 t Raw Quicksilver → 4 t Quicksilver
Blueprint unlocks by completing the Bio Order **Abyssal Grand Compendium**.

Craft time	0.4 h
Power	48 W
Production tier	T2

Fuel Assembler Recipes

The Fuel Assembler makes Fuel Rods and Nuclear Batteries from Raw Uranium and lead plates. Uranium enters from a Lead Cask; Fuel Rods leave through the shielded lane, while Nuclear Batteries use ordinary product storage.

Fuel Rod `craft_fuel_rod` NOT YET UNLOCKED

4× Raw Uranium and 2× Lead Plate → 1× Fuel Rod
Blueprint unlocks by completing the order **Vestibule, Hot Freight Proof**.

Craft time	6 h
Power	1,800 W
Production tier	T2

Nuclear Battery `craft_nuclear_battery` NOT YET UNLOCKED

4× Raw Uranium and 3× Lead Plate → 1× Nuclear Battery
Blueprint unlocks by completing the order **Helios, Reactor Vessel Chain**.

Craft time	4 h
Power	1,200 W
Production tier	T2

Power Equipment

Power generation, storage, charging, and grid hardware from the Shop. Research-gated equipment is documented here before it appears in the Shop.

Solar Generator `solar_generator`

Script-tracked photovoltaic panel producing up to **50 W** in sunlight and **0 W** at night. Poor tilt reduces daytime output.

Sells for	500 cr
Shop price	500 cr

Small Battery `battery`

Base-station energy storage.

Sells for	300 cr
Shop price	300 cr

Large Battery `battery_large`

High-capacity base-station energy storage.

Sells for	15,000 cr
Shop price	15,000 cr
Component docs	Battery

Oil Generator `oil_generator`

Burns oil to generate power.

Sells for	2,000 cr
Shop price	2,000 cr
Component docs	Oil Generator

Steam Turbine `steam_turbine`

Produces up to **108 W** from **90 t/h Steam** at full throttle. A script controls its throttle.

Sells for	7,500 cr
Shop price	7,500 cr
Component docs	Steam Turbine

Reactor `reactor`

Fission plant producing up to **5,000 W** from Fuel Rods and cooling water. One rod lasts **72 hours** at heat **1.0**; fuel use follows commanded heat even before the core reaches efficient temperature.

Sells for	750,000 cr
Shop price	750,000 cr

Nuclear Battery `nuclear_battery`

Uranium-core storage cell holding thirty times a base battery's charge.

Produced by	Nuclear Battery
Requested by	Helios, Battery Contract (Crown)

Lightning Rod `lightning_rod`

Deploys a **4,000 Wh**, **600 m** lightning reserve. Condition falls **0.05 per day**, reducing capture to zero unless a script repairs it with Storm Glass.

Produced by	Lightning Rod

Atmosphere Equipment

Atmosphere generators and their upgrade packs. Product names match the Shop and Inventory so exact searches lead back to the same hardware.

Oxygen Generator `oxygen_generator`

Converts atmospheric CO_2 into breathable oxygen.

Sells for	1,000 cr
Shop price	1,000 cr

Heat Generator `temp_heater`

Radiates heat to warm the planet surface.

Sells for	800 cr
Shop price	800 cr

Pressure Generator `pressure_generator`

Compresses atmospheric gas to raise surface pressure.

Sells for	900 cr
Shop price	900 cr

Oxygen Generator Mk II Upgrade Pack `oxygen_upgrade_pack_mk2`

Upgrades a deployed Oxygen Generator to Mk II: 5× output and 5× power draw.

Sells for	12,000 cr
Shop price	12,000 cr
Component docs	Oxygen Generator

Heat Generator Mk II Upgrade Pack `heat_upgrade_pack_mk2`

Upgrades a deployed Heat Generator to Mk II: 4.7× output and 5× power draw.

Sells for	12,000 cr
Shop price	12,000 cr
Component docs	Heat Generator

Pressure Generator Mk II Upgrade Pack `pressure_upgrade_pack_mk2`

Upgrades a deployed Pressure Generator to Mk II: 25× output and 5× power draw.

Sells for	12,000 cr
Shop price	12,000 cr
Component docs	Pressure Generator

Oxygen Generator Mk III Upgrade Pack `oxygen_upgrade_pack_mk3`

Upgrades an Oxygen Generator to Mk III: **200×** Mk I output, **20×** Mk I power, and **8 t/h Water** use. Without Water it falls back to Mk II output (**5×**).

Sells for	90,000 cr
Shop price	90,000 cr
Component docs	Oxygen Generator

Heat Generator Mk III Upgrade Pack `heat_upgrade_pack_mk3`

Upgrades a Heat Generator to Mk III: **208×** Mk I output, **20×** Mk I power, and **12 t/h Steam** use. Without Steam it falls back to Mk II output (**4.7×**).

Sells for	90,000 cr
Shop price	90,000 cr
Component docs	Heat Generator

Pressure Generator Mk III Upgrade Pack `pressure_upgrade_pack_mk3`

Upgrades a Pressure Generator to Mk III: **200×** Mk I output, **20×** Mk I power, and **5 t/h Water** use. Without Water it falls back to Mk II output (**25×**).

Sells for	90,000 cr
Shop price	90,000 cr
Component docs	Pressure Generator

Oxygen Upgrade Pack Mk IV `oxygen_upgrade_pack_mk4`

Upgrades one Oxygen Generator to Mk IV: **500× Mk I output** and **100× Mk I power draw** under the same operating conditions. Replaces Water with a magazine holding **4 Fuel Rods**, supplied from a connected Lead Cask. Each rod lasts about **240 hours** of operation. Production stops when the loaded rod is spent and the magazine is empty.

Produced by	Oxygen Upgrade Pack Mk IV
Component docs	Oxygen Generator

Heat Upgrade Pack Mk IV `heat_upgrade_pack_mk4`

Upgrades one Heat Generator to Mk IV: **515× Mk I output** and **100× Mk I power draw** under the same operating conditions. Replaces Steam with a magazine holding **4 Fuel Rods**, supplied from a connected Lead Cask. Each rod lasts about **240 hours** of operation. Production stops when the loaded rod is spent and the magazine is empty.

Produced by	Heat Upgrade Pack Mk IV
Component docs	Heat Generator

Pressure Upgrade Pack Mk IV `pressure_upgrade_pack_mk4`

Upgrades one Pressure Generator to Mk IV: **500× Mk I output** and **100× Mk I power draw** under the same operating conditions. Replaces Water with a magazine holding **4 Fuel Rods**, supplied from a connected Lead Cask. Each rod lasts about **240 hours** of operation. Production stops when the loaded rod is spent and the magazine is empty.

Produced by	Pressure Upgrade Pack Mk IV
Component docs	Pressure Generator

Weather Station `weather_station`

Programmable 50 W local weather station with live reception and an optional Signal Board.

Sells for	60,000 cr
Shop price	60,000 cr

Biology Equipment

Laboratory machines, collectors, portable biology tools, and Bio Order infrastructure.

Bio Collector `bio_collector`

Automated collector that gathers biology specimens from the field.

Sells for	3,000 cr
Shop price	3,000 cr
Component docs	Bio Collector

Bio Lab `bio_lab`

Analyzes collected specimens and extracts biology samples.

Sells for	5,000 cr
Shop price	5,000 cr
Component docs	Bio Lab

Bio Luminizer `bio_luminizer`

Tunes coastal fragments to an order's exact glow color for delivery.

Sells for	60,000 cr
Shop price	60,000 cr
Component docs	Bio Luminizer

DNA Sequencer `dna_sequencer`

Splices genes into geothermal fragments so they carry the genes an order needs.

Sells for	100,000 cr
Shop price	100,000 cr
Component docs	DNA Sequencer

Bio Caster `bio_caster`

Heat-casts volcanic fragments for Bio Orders using fabricated materials, steam, and water.

Sells for	150,000 cr
Shop price	150,000 cr
Component docs	Bio Caster

Bio Conditioner `bio_conditioner`

Conditions deep fragments for Bio Orders through a five-stage quality inspection.

Sells for	225,000 cr
Shop price	225,000 cr
Component docs	Bio Conditioner

Bio Exchange `bio_exchange`

Fills Bio Orders with collected fragments for credits.

Sells for	4,000 cr
Shop price	4,000 cr
Component docs	Bio Exchange

Biosphere Equipment

Plant, habitat, husbandry, and biome-processing hardware, including deploy kits and upgrade packs.

Plant Terraformer Kit `plant_terraformer_kit`

Fabricated deployment bundle for one `plant_terraformer`. Deploy any number from Inventory to operational outposts; every supplied machine adds its tier conversion rate multiplied by that outpost's efficiency to the shared Plants pillar.

Produced by	Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit
Deploys	Plant Terraformer

Crop Automator Kit `crop_automator_kit`

Deploys a Crop Automator that harvests, plants, and treats crops across up to 24 other cells in a centered 5 by 5 area.

Shop price	30,000 cr
Deploys	Crop Automator

Portable Bio Scanner `portable_bio_scanner`

Drone module that scans the current surface tile for life forms and records results in the Journal.

Sells for	10,000 cr
Shop price	10,000 cr

Portable Bio Extractor `portable_bio_extractor`

Drone module that harvests scanned life forms. Includes a 25 t sealed chamber for one life-form type; Cargo Pods add general capacity.

Sells for	14,000 cr
Shop price	14,000 cr

Essence Liquifier `essence_liquifier`

Turns life-form samples into biome essence. Deploys to a chosen outpost.

Sells for	35,000 cr
Shop price	35,000 cr

Biomass Mixer `biomass_mixer`

Combines biome essences into biomass, with higher output from more balanced mixtures. Deploys to a chosen outpost.

Sells for	60,000 cr
Shop price	60,000 cr

Biomass Mixer Mk II Upgrade Pack `biomass_mixer_upgrade_pack_mk2`

Upgrades a deployed Biomass Mixer to Mk II: 4.5× output, 2.4× essence consumption, and 5× power draw.

Sells for	90,000 cr
Shop price	90,000 cr
Component docs	Biomass Mixer

Habitat `habitat`

Revives and houses one creature species at a time. Established colonies can move without losing progress when the destination has enough capacity.

Sells for	100,000 cr
Shop price	100,000 cr
Component docs	Habitat

Feed Maker `feed_maker`

Crafts creature feeds from plant forage and harvested life forms. Deploys to a chosen outpost.

Sells for	50,000 cr
Shop price	50,000 cr
Component docs	Feed Maker

Refiner `refiner`

Turns raw exotic feedstock and tar into creature-grade gas or liquid. Deploys to a chosen outpost.

Sells for	80,000 cr
Shop price	80,000 cr
Component docs	Refiner

Feed Maker Mk II Upgrade Pack `feed_maker_pack_mk2`

Crafted at the Fabricator. Use Upgrade in Inventory to upgrade one deployed Feed Maker: **1.5× crafting speed and 1.5× Auto Feeder speed into its input**, with **2× operating power**.

Produced by	Frames + Controls + Panels + Valves → Feed Maker Pack Mk II
Component docs	Feed Maker

Habitat Mk II Upgrade Pack `habitat_upgrade_pack_mk2`

Upgrades a deployed Habitat to Mk II: **2x carrying capacity** and **144-168 W** draw. Breeding speed and biological costs still come from creature adaptations. Each Habitat needs its own pack.

Produced by	Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Component docs	Habitat

Plant Terraformer Mk II Upgrade Pack `plant_terraformer_upgrade_pack_mk2`

Upgrades a deployed Plant Terraformer to Mk II: **2,200 Forage/h** conversion throughput, an input feeder that handles **80 items per step** instead of **16**, **900 W** draw while enabled, and Fertilizer and Growth Accelerant injectors.

Produced by	Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II
Component docs	Plant Terraformer

Grow Lamp Upgrade Pack Mk IV `grow_lamp_upgrade_pack_mk4`

Upgrades one Grow Lamp to Mk IV: **8×** base Forage yield on its own (**+700%**) and **500 W** draw while enabled. The bonus applies during growth to crops that require light in the four orthogonally adjacent cells, while the lamp is powered and enabled. Coverage and growth speed stay the same. Only the strongest covering lamp counts; its bonus adds to other yield bonuses.

Sells for	400,000 cr
Shop price	400,000 cr
Component docs	Grow Lamp

Sprinkler Upgrade Pack Mk IV `sprinkler_upgrade_pack_mk4`

Upgrades one Sprinkler to Mk IV: **8×** base Forage yield on its own (**+700%**), **500 W** draw while enabled, and **16 t/h Water** use while supplied. The bonus applies during growth to crops that require water in the four orthogonally adjacent cells, while the sprinkler is powered, enabled, and supplied. Coverage and growth speed stay the same. Only the strongest covering sprinkler counts; its bonus adds to other yield bonuses.

Sells for	400,000 cr
Shop price	400,000 cr
Component docs	Sprinkler

Mining Equipment

Exploration vehicles, extraction hardware, site equipment, and the products needed to establish remote mining operations.

Vehicle Charging Station Mk II Upgrade Pack `charging_station_upgrade_pack_mk2`

Upgrades a deployed Vehicle Charging Station to Mk II: **2 bays** and a **120 W** total charging budget. One vehicle pools both idle bays.

Sells for	8,000 cr
Shop price	8,000 cr
Component docs	Vehicle Charging Station

Vehicle Charging Station Mk III Upgrade Pack `charging_station_upgrade_pack_mk3`

Upgrades a Mk II Vehicle Charging Station to Mk III: **4 bays** and a **240 W** total charging budget. One vehicle pools every idle bay.

Sells for	45,000 cr
Shop price	45,000 cr
Component docs	Vehicle Charging Station

Mining Rover `rover`

Starter expedition chassis with an integrated **100 Wh** battery and **10-unit** cargo hold. Mount Nav, Sonar, and Drill modules to configure its role.

Sells for	2,000 cr
Shop price	2,000 cr

Battery Charger `battery_charger`

Charges loose **Portable Batteries (50 Wh)** and **Heavy Portable Batteries (100 Wh)** from the local grid. Mk I has **4 shared slots**, **1 charging bay**, and a **30 W** budget. Place at Base or a founded outpost. Park a Pioneer in its service area for a timed battery exchange. Charging is script-controlled; idle draw is 0 W.

Sells for	1,500 cr
Shop price	1,500 cr

Battery Charger Mk II Upgrade Pack `battery_charger_upgrade_pack_mk2`

Expands one Battery Charger to **8 shared slots**, **2 charging bays**, and **60 W** total. One active battery receives 60 W; two share 30 W each before overcrowding. Preserves the building, stored cells, jobs, connections, and scripts.

Sells for	2,500 cr
Shop price	2,500 cr
Component docs	Battery Charger

Vehicle Charging Station `charging_station`

Grid-powered vehicle charger with **1 bay** and a **30 W** charging budget. Upgrade packs add bays and increase pooled charging speed.

Sells for	1,200 cr
Shop price	1,200 cr

Pioneer `pioneer`

Eight-slot modular expedition chassis with no built-in battery, cargo, or Nav. The bare Pioneer cannot move until separately purchased hardware is installed.

Sells for	5,000 cr
Shop price	5,000 cr

Outpost Kit `outpost_kit`

Founds a new outpost that hosts machines and links utility networks. Additional kits cost more; the Shop and Inventory show the live price.

Starting sale value	15,000 cr
Starting shop price	15,000 cr

Drone Depot Kit `drone_station_kit`

Deploys a one-bay Drone Depot at an outpost, with a **50-unit** transfer stockpile.

Produced by	Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit
Requested by	Helios, Outpost Order
Deploys	Drone Depot

Drone Depot Kit (Medium) `drone_station_kit_medium`

Deploys a two-bay Drone Depot at an outpost, with a **100-unit** transfer stockpile.

Produced by	Frame + Control + Panel → Drone Depot Kit (Medium)
Requested by	Helios, Drone Hub Stockpile
Deploys	Drone Depot
Component docs	Drone Depot

Drone Depot Kit (Large) `drone_station_kit_large`

Deploys a four-bay Drone Depot at an outpost, with a **200-unit** transfer stockpile.

Produced by	Frame + Control + Panel → Drone Depot Kit (Large)
Deploys	Drone Depot
Component docs	Drone Depot

Drone Service Station Kit `drone_service_station_kit`

Deploys a station that charges electric drones, refuels heli drones, and rescues stranded ones.

Produced by	Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit
Requested by	Vestibule, Refueling Pad Order, Vestibule, Refueling Network Expansion, and Vestibule, Continental Pipe Build
Deploys	Drone Service Station

Mining Drill Kit `mining_drill_kit`

Deploys a Mining Drill that cuts deposits up to hardness 1, producing **25 t/h** at standard purity and drawing **10 W** while extracting.

Produced by	Frame + Control + Panel → Mining Drill Kit
Requested by	Helios, Drill Deployment Order and Helios, Mining Drill Kit Build
Deploys	Mining Drill

Industrial Mining Drill Kit `mining_drill_industrial_kit`

Deploys an Industrial Mining Drill that cuts deposits up to hardness 3, producing **75 t/h** at standard purity and drawing **35 W** while extracting.

Produced by	Frame + Control + Panel + Rotor → Industrial Mining Drill Kit
Requested by	Helios, Tether Project
Deploys	Mining Drill
Component docs	Mining Drill

Heavy Mining Drill Kit `mining_drill_heavy_kit`

Deploys a Heavy Mining Drill that cuts deposits up to hardness 4, every mineral on the planet, producing **200 t/h** at standard purity and drawing **100 W** while extracting.

Produced by	Frame + Control + Panel + Rotors → Heavy Mining Drill Kit
Deploys	Mining Drill
Component docs	Mining Drill

Drone (Small) `drone_small`

Small aerial cargo-drone chassis.

Produced by	Rare Earth Core + Titanium Ingot + Control → Drone (Small)
Requested by	Helios, Pilot Drone Fleet Order, Helios, Mid-Drone Build, and Helios, Continental Drone Fleet Order
Deploys	Drone

Drone (Medium) `drone_medium`

Mid-size aerial cargo-drone chassis.

Produced by	Rare Earth Core + Titanium Ingot + Control → Drone (Medium)
Requested by	Helios, Heavy Drone Build and Helios, Continental Drone Fleet Order
Deploys	Drone

Drone (Large) `drone_large`

Heavy industrial cargo-drone chassis.

Produced by	Rare Earth Core + Titanium Ingot + Control → Drone (Large)
Deploys	Drone

Lead Cask `lead_cask`

Shielded storage cask for Raw Uranium and Fuel Rods.

Produced by	Lead Cask

Logistics Equipment

Order fulfillment and dispatch infrastructure from the Shop.

Supply Dock `supply_dock`

Ships Earth Orders at a base rate of **25 units/h**. Additional docks cost more; the Shop and Inventory show the live price.

Starting sale value	5,000 cr
Starting shop price	5,000 cr

Production Equipment

Storage, fabrication, logistics, and nuclear products that build and move the production chain.

Storage Bin `storage_bin`

Outpost storage container for a single material.

Sells for	120 cr
Shop price	120 cr

Smelter `smelter`

Industrial furnace. Refines raw ore into metal stock.

Sells for	650 cr
Shop price	650 cr

Fabricator `fabricator`

Manufacturing press. Assembles equipment from refined stock.

Sells for	1,400 cr
Shop price	1,400 cr

Large Warehouse `large_warehouse`

High-bay depot with 15 material-locked slots and 30,000 units of total storage.

Sells for	60,000 cr
Shop price	60,000 cr
Component docs	Large Warehouse

Smelter Mk II Upgrade Pack `smelter_upgrade_pack_mk2`

Manufactured replacement assemblies for one Smelter Mk II.

Produced by	Fabricate Smelter Mk II Upgrade Pack
Component docs	Smelter

Fabricator Mk II Upgrade Pack `fabricator_upgrade_pack_mk2`

Manufactured replacement assemblies for one Fabricator Mk II.

Produced by	Fabricate Fabricator Mk II Upgrade Pack
Component docs	Fabricator

Oil Pump Mk II Upgrade Pack `oil_pump_upgrade_pack_mk2`

Manufactured replacement assemblies for one Oil Pump Mk II.

Produced by	Fabricate Oil Pump Mk II Upgrade Pack
Component docs	Oil Pump

Smelter Mk III Upgrade Pack `smelter_upgrade_pack_mk3`

Manufactured replacement assemblies for one Smelter Mk III.

Produced by	Fabricate Smelter Mk III Upgrade Pack
Component docs	Smelter

Fabricator Mk III Upgrade Pack `fabricator_upgrade_pack_mk3`

Manufactured replacement assemblies for one Fabricator Mk III.

Produced by	Fabricate Fabricator Mk III Upgrade Pack
Component docs	Fabricator

Fuel Assembler `fuel_assembler`

Nuclear workbench that presses Raw Uranium and lead plates into Fuel Rods or Nuclear Batteries.

Sells for	225,000 cr
Shop price	225,000 cr

Modules & Vehicle Gear

Mountable modules, holders, racks, thrusters, cargo pods, batteries, tanks, and specialist vehicle equipment.

Nav Module `nav_module`

Basic mobility drivetrain for surface vehicles.

Sells for	500 cr
Shop price	500 cr

Sonar Module `sonar_module`

Basic sonar: 50 m point sweeps, minerals up to hardness 1, basic survey detail.

Sells for	800 cr
Shop price	800 cr

Drill Module `drill_module`

Basic drill.

Sells for	1,000 cr
Shop price	1,000 cr

Sport Nav `nav_module_sport`

High-performance drivetrain for a Pioneer's universal slot. One Sport Nav gives 2× top speed with 2.6× movement power draw compared with Basic Nav in the same rig, about 1.3× battery use per meter at full throttle.

Sells for	1,200 cr
Shop price	1,200 cr
Component docs	Nav Module

Extended Sonar `sonar_module_wide`

Pioneer universal-slot sonar: 180 m point sweeps, minerals up to hardness 3, wider survey detail.

Sells for	2,000 cr
Shop price	2,000 cr
Component docs	Sonar Module

Deep Sonar `sonar_module_deep`

Pioneer universal-slot sonar: 280 m point sweeps, minerals up to hardness 4, deepest survey detail.

Sells for	4,500 cr
Shop price	4,500 cr
Component docs	Sonar Module

Industrial Drill `drill_module_industrial`

Heavy-duty drill for a Pioneer's universal slot.

Sells for	2,500 cr
Shop price	2,500 cr
Component docs	Drill Module

Heavy Drill `drill_module_heavy`

Top-tier drill for a Pioneer's universal slot and the densest minerals.

Sells for	7,000 cr
Shop price	7,000 cr
Component docs	Drill Module

Small Battery Holder `battery_holder_small`

Empty **1-bay** battery housing for a Pioneer. Sold without a Portable Battery, which must be purchased and installed separately.

Sells for	500 cr
Shop price	500 cr

Medium Battery Holder `battery_holder_medium`

Empty **2-bay** battery housing for a Pioneer. Portable Batteries are purchased and installed separately.

Sells for	1,000 cr
Shop price	1,000 cr

Large Battery Holder `battery_holder_large`

Empty **3-bay** battery housing for a Pioneer. Portable Batteries are purchased and installed separately.

Sells for	2,000 cr
Shop price	2,000 cr

Small Cargo Rack `cargo_rack_small`

Empty **1-bin** Cargo Rack for a Pioneer. Sold without a Portable Storage Bin, which must be purchased and installed separately.

Sells for	500 cr
Shop price	500 cr

Medium Cargo Rack `cargo_rack_medium`

Empty **2-bin** Cargo Rack for a Pioneer. Portable Storage Bins are purchased and installed separately.

Sells for	1,000 cr
Shop price	1,000 cr

Large Cargo Rack `cargo_rack_large`

Empty **3-bin** Cargo Rack for a Pioneer. Portable Storage Bins are purchased and installed separately.

Sells for	2,000 cr
Shop price	2,000 cr

Portable Battery `portable_battery`

50 Wh rechargeable cell for any Battery Holder, delivered fully charged.

Sells for	150 cr
Shop price	150 cr

Heavy Portable Battery `heavy_portable_battery`

100 Wh high-density cell for any Battery Holder, delivered fully charged.

Sells for	350 cr
Shop price	350 cr

Portable Storage Bin `portable_bin`

Portable single-mineral container holding **25 units**. Latches to the first mineral loaded into it.

Sells for	150 cr
Shop price	150 cr

Heavy Portable Storage Bin `heavy_portable_bin`

Reinforced single-mineral container holding **50 units**, double the standard bin.

Sells for	350 cr
Shop price	350 cr

Constructor Module `constructor_module`

Pioneer module that builds planned surface structures and utility networks.

Sells for	2,500 cr
Shop price	2,500 cr
Component docs	Constructor Module

Electric Thruster `electric_thruster`

Electric drone propulsion: **300 m/h** and **5 Wh/h** at full throttle. Burn rises with throttle squared, so slower flight stretches range.

Produced by	Rare Earth Core + Rotor → Electric Thruster
Requested by	Spire, Thruster Order and Spire, Propulsion Control Run

Heli Thruster `heli_thruster`

Oil-fueled drone propulsion: **900 m/h** and **5 t/h Oil** at full throttle. Burn rises with throttle squared, so slower flight stretches range.

Produced by	Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster
Requested by	Vestibule, Heli Thruster Order, Vestibule, Heavy Tank Order, Vestibule, Heli Fleet Order, and Vestibule, Heli Thruster Megastock

Cargo Pod (Small) `cargo_pod_small`

Drone cargo hold carrying **100 units**. The light freight option for a Small chassis.

Produced by	Titanium + Glass → Cargo Pod (Small)
Requested by	Helios, Cargo Pod Run

Cargo Pod (Medium) `cargo_pod_medium`

Drone cargo hold carrying **250 units**, for sustained outpost-to-outpost freight.

Produced by	Titanium + Glass + Panel → Cargo Pod (Medium)
Requested by	Helios, Mid-Cargo Order, Helios, Mid-Drone Build, and Helios, Cargo Megaorder

Cargo Pod (Large) `cargo_pod_large`

Drone cargo hold carrying **500 units**. Heavy haulage, best on a chassis with slots to spare.

Produced by	Titanium + Glass + Panel → Cargo Pod (Large)
Requested by	Helios, Heavy Drone Build

Battery Pack `battery_pack`

150 Wh rechargeable power cell for long-range electric drones. New packs begin fully charged.

Produced by	Titanium Ingot + Glass + Cell → Battery Pack
Requested by	Spire, Drone Power Trial and Vestibule, Power Reserve Order

Oil Tank (Small) `oil_tank_small`

Heli drone fuel tank holding **30 t** of oil. Short-range work close to a refuelling station.

Produced by	Titanium Ingot + Lubricant + Rubber → Oil Tank (Small)
Requested by	Vestibule, Oil Tank Order

Oil Tank (Medium) `oil_tank_medium`

Heli drone fuel tank holding **75 t** of oil, for routes that leave the outpost cluster.

Produced by	Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium)
Requested by	Vestibule, Mid Tank Order

Oil Tank (Large) `oil_tank_large`

Heli drone fuel tank holding **150 t** of oil. Long-haul range at the cost of a module slot.

Produced by	Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large)
Requested by	Vestibule, Heavy Tank Order, Vestibule, Refueling Network Expansion, Vestibule, Heli Fleet Order, Vestibule, Tether Tank Run, Vestibule, Power Reserve Order, and Vestibule, Tether Megahaul

Seismic Sonar `sonar_module_seismic`

Pioneer universal-slot sonar for ordinary Deep Sonar surveys and deep oil prospecting beneath inert formations. Scan formations to identify potential deep oil, then survey those contacts to confirm Oil Wells. Previously surveyed inert formations can still conceal reservoirs.

Sells for	7,500 cr
Shop price	7,500 cr
Component docs	Sonar Module

Shield Plating `shield_plating`

Lead drone armor for exposure-free Raw Uranium collection and radioactive-cargo loading from Lead Casks. Heavy: **halves every Cargo Pod's capacity** and multiplies fuel burn by **1.5**.

Produced by	Shield Plating

Fluid Infrastructure

Pumps, tanks, turbines, pipe hardware, and other products used to capture, buffer, route, or consume fluids.

Water Pump `water_pump`

Extracts water from a surveyed water well.

Produced by	Iron + Glass + Liquid Pipe Segment → Water Pump
Deploys	Water Pump

Oil Pump `oil_pump`

Extracts oil from a surveyed oil well.

Produced by	Iron + Titanium + Valve + Panel → Oil Pump
Deploys	Oil Pump

Steam Condenser `steam_condenser`

Condenses steam into clean water at up to 250 t/h.

Sells for	50,000 cr
Shop price	50,000 cr
Component docs	Steam Condenser

Gas Tank `gas_tank`

Buffer tank that stores any one gas: steam, ammonia, swamp gas, or any gas it receives first.

Sells for	1,200 cr
Shop price	1,200 cr
Component docs	Gas Tank

Liquid Tank `liquid_tank`

Buffer tank that stores any one liquid: water, oil, or essence.

Sells for	900 cr
Shop price	900 cr
Component docs	Liquid Tank

Large Liquid Tank `bulk_liquid_reservoir`

A big passive tank that holds 1,000 t of one liquid and sticks to the first liquid it receives.

Sells for	15,000 cr
Shop price	15,000 cr
Component docs	Large Liquid Tank

Exotic Gas Cap Kit `exotic_gas_cap_kit`

Deploys an Exotic Gas Cap that collects gas from an exotic vent.

Produced by	Titanium + Gas Pipe Segments → Exotic Gas Cap Kit
Deploys	Exotic Gas Cap

Exotic Spring Tap Kit `exotic_spring_tap_kit`

Deploys an Exotic Spring Tap that collects liquid from an exotic spring.

Produced by	Titanium + Liquid Pipe Segments + Valve → Exotic Spring Tap Kit
Deploys	Exotic Spring Tap

Minerals

Raw ore mined from surveyed mineral sites. Hardness (**H1** to **H4**) gates which sonar detects a site and which drill can work it; each ore smelts 1:1 into its refined form at the `smelter`.

Iron Ore `iron_ore`

Common structural ore from the inner mineral sites. The backbone of early industry: nearly every frame, segment, and machine part starts as iron.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H1
Dig time	Drill Module: 15 min per unit, Industrial Drill: 11.25 min per unit, Heavy Drill: 9 min per unit
Used in	Iron Ore → Iron Ingot
Requested by	Spire, Raw Iron Intake

Silicon `silicon`

Glassmaking ore abundant across the inner rings. Smelts into `glass` for panels, housings, and electronics.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H1
Dig time	Drill Module: 15 min per unit, Industrial Drill: 11.25 min per unit, Heavy Drill: 9 min per unit
Used in	Silicon → Glass and Iron + Silicon → Liquid Pipe Segment
Requested by	Spire, Silicon Stock, Spire, Silicon Bootstrap, and Spire, Silicon Megahaul

Titanium `titanium`

Light, strong mid-ring ore. Smelts into `titanium_ingot` for rotors, thrusters, and high-stress parts.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H2
Dig time	Industrial Drill: 15 min per unit, Heavy Drill: 12 min per unit
Used in	Titanium → Titanium Ingot
Requested by	Spire, Titanium Consignment

Cobalt `cobalt`

Battery-chemistry ore from the mid rings. Smelts into `cobalt_ingot`, the heart of every battery cell.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H2
Dig time	Industrial Drill: 15 min per unit, Heavy Drill: 12 min per unit
Used in	Cobalt → Cobalt Ingot

Rare Earth `rare_earth`

Scarce magnetic ore from the outer rings. Smelts into `rare_earth_core` for control electronics and precision drives.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H3
Dig time	Industrial Drill: 18.75 min per unit, Heavy Drill: 15 min per unit
Used in	Rare Earth → Rare Earth Core

Neutronium `neutronium`

The hardest known ore on the planet, found far from base. Smelts into `neutronium_bar` for capacitors and deep-tier hardware.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H4
Dig time	Heavy Drill: 18 min per unit
Used in	Neutronium → Neutronium Bar

Lead Ore `lead_ore`

Dense, soft ore from mineral sites. Smelts into `lead_ingot`, the base of all radiation shielding.
Time per unit with each drill module that can cut it, on a standard site. Rich sites dig 2× faster and pure sites 3× faster. Field Mining Drills don't use these times.

Hardness	H2
Dig time	Industrial Drill: 13.5 min per unit, Heavy Drill: 10.8 min per unit
Used in	Lead Ore → Lead Ingot

Refined Materials

Smelter output: the standard stock every Fabricator blueprint builds on.

Iron Ingot `iron_ingot`

Smelted iron stock. The universal structural input: frames, segments, valves, and half the Fabricator catalogue start here.

Produced by	Iron Ore → Iron Ingot
Used in	Iron → Gas Pipe Segment, Iron + Silicon → Liquid Pipe Segment, Iron + Titanium → Power Line Segment, Iron + Glass → Pressure Valve, Iron Ingot + Titanium Ingot → Machine Frame, Iron + Glass → Circuit Panel, Cobalt Ingot + Iron Ingot + Glass → Battery Cell, Iron + Titanium + Glass → Tank Lining, Iron + Glass + Liquid Pipe Segment → Water Pump, Iron + Titanium + Valve + Panel → Oil Pump, Iron + Oil → Lubricant + Tar, and Frame + Iron + Panel → Waste Processor Kit
Requested by	Helios, Iron Bootstrap, Helios, Iron Production, Helios, Mixed Alloy Order, Helios, Battery Order, Helios, Bulk Iron Run, Helios, Bulk Frame Run, Vestibule, Utility Conduit Stock, and Helios, Lead Survey Stock

Glass `glass`

Smelted silicon sheet stock for panels, housings, and optics.

Produced by	Silicon → Glass
Used in	Iron + Glass → Pressure Valve, Iron + Glass → Circuit Panel, Circuit Panel + Titanium Ingot + Glass → Control Unit, Cobalt Ingot + Iron Ingot + Glass → Battery Cell, Iron + Titanium + Glass → Tank Lining, Iron + Glass + Liquid Pipe Segment → Water Pump, Glass + Oil → Plastic + Tar, Titanium + Glass → Cargo Pod (Small), Titanium + Glass + Panel → Cargo Pod (Medium), Titanium + Glass + Panel → Cargo Pod (Large), Titanium Ingot + Glass + Cell → Battery Pack, Frame + Panel + Glass → Grow Lamp Kit, Tar + Glass → Fertilizer, Tar + Glass + Panel → Fertilizer Mk II, Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III, and Panel + Glass + Rare Earth Core → Grow Lamp Pack Mk II
Requested by	Spire, Optical Glass, Spire, Pressure Hardware, Spire, Optics Stockpile, and Spire, Silicon Megahaul

Titanium Ingot `titanium_ingot`

High-strength, lightweight stock for rotors, thrusters, and pressure hardware.

Produced by	Titanium → Titanium Ingot
Used in	Iron + Titanium → Power Line Segment, Iron Ingot + Titanium Ingot → Machine Frame, Circuit Panel + Titanium Ingot + Glass → Control Unit, Titanium Ingot + Gas Pipe Segments → Thermal Cap Kit, Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor, Iron + Titanium + Glass → Tank Lining, Iron + Titanium + Valve + Panel → Oil Pump, Rare Earth Core + Titanium Ingot + Control → Drone (Small), Rare Earth Core + Titanium Ingot + Control → Drone (Medium), Rare Earth Core + Titanium Ingot + Control → Drone (Large), Titanium + Glass → Cargo Pod (Small), Titanium + Glass + Panel → Cargo Pod (Medium), Titanium + Glass + Panel → Cargo Pod (Large), Titanium Ingot + Glass + Cell → Battery Pack, Titanium Ingot + Lubricant + Rubber → Oil Tank (Small), Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium), Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large), Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Titanium + Gas Pipe Segments → Exotic Gas Cap Kit, and Titanium + Liquid Pipe Segments + Valve → Exotic Spring Tap Kit
Requested by	Helios, Titanium Run, Helios, Mixed Alloy Order, and Helios, Lead Survey Stock

Cobalt Ingot `cobalt_ingot`

Battery-grade stock. Every `battery_cell` is built around it.

Produced by	Cobalt → Cobalt Ingot
Used in	Fabricate Oil Pump Mk II Upgrade Pack, Cobalt Ingot + Iron Ingot + Glass → Battery Cell, Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor, Cobalt + Oil → Rubber + Tar, Oxygen Upgrade Pack Mk IV, Heat Upgrade Pack Mk IV, and Pressure Upgrade Pack Mk IV
Requested by	Spire, Cobalt Run, Spire, Magnetic Stator Build, and Spire, Cobalt Stockpile

Rare Earth Core `rare_earth_core`

Precision magnetic core stock for control units and advanced electronics.

Produced by	Rare Earth → Rare Earth Core
Used in	Fabricate Smelter Mk III Upgrade Pack, Fabricate Fabricator Mk III Upgrade Pack, Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor, Rare Earth Core + Titanium Ingot + Control → Drone (Small), Rare Earth Core + Titanium Ingot + Control → Drone (Medium), Rare Earth Core + Titanium Ingot + Control → Drone (Large), Rare Earth Core + Rotor → Electric Thruster, Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster, Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor, Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III, Plastic + Rare Earth Core → Growth Accelerant, Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier, Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II, Panel + Glass + Rare Earth Core → Grow Lamp Pack Mk II, Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III, and Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Requested by	Spire, Rare Earth Order, Spire, Drone Power Trial, and Spire, Capacitor Bulk Order

Neutronium Bar `neutronium_bar`

Ultra-dense bar stock for neutron capacitors and advanced hardware.

Produced by	Neutronium → Neutronium Bar
Used in	Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor
Requested by	Spire, Neutronium Order and Spire, Neutronium Megastock

Lead Ingot `lead_ingot`

Soft, dense shielding stock. Rolls into `lead_plate` for casks and reactor hardware.

Produced by	Lead Ore → Lead Ingot
Used in	Lead Plate
Requested by	Helios, Lead Consignment

Fabricated Components

Fabricator output: machined parts, infrastructure segments, deploy kits, and oil-derived polymers. Earth contractors order many of these, check each card's Requested By row.

Gas Pipe Segment `gas_pipe_segment`

A 10 m run of gas pipe. A Pioneer's Constructor consumes one per planned segment when laying gas lines.

Produced by	Iron → Gas Pipe Segment
Used in	Gas Segments + Valve → Gas Bridge, Titanium Ingot + Gas Pipe Segments → Thermal Cap Kit, Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit, and Titanium + Gas Pipe Segments → Exotic Gas Cap Kit
Requested by	Vestibule, Gas Pipe Order, Vestibule, Pipe Network Run, and Vestibule, Pipe Order

Liquid Pipe Segment `liquid_pipe_segment`

A 10 m run of liquid pipe. A Pioneer's Constructor consumes one per planned segment when laying liquid lines.

Produced by	Iron + Silicon → Liquid Pipe Segment
Used in	Liquid Segments + Valve → Liquid Bridge, Iron + Glass + Liquid Pipe Segment → Water Pump, Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit, Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit, Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit, Frame + Pipes + Valve → Sprinkler Kit, and Titanium + Liquid Pipe Segments + Valve → Exotic Spring Tap Kit
Requested by	Vestibule, Liquid Pipe Order, Vestibule, Pressure Network Trial, Vestibule, Pipe Network Run, Vestibule, Pipe Order, Vestibule, Pipe & Lubricant Order, Vestibule, Refueling Hardware Build, and Vestibule, Continental Pipe Build

Power Line Segment `power_line_segment`

A 10 m run of power line. A Pioneer's Constructor consumes one per planned segment when wiring outposts and machines.

Produced by	Iron + Titanium → Power Line Segment
Used in	Power Segments + Circuit → Power Bridge

Gas Pipe Bridge `gas_pipe_bridge`

A 3-tile gas overpass that lets one line cross another without joining networks.

Produced by	Gas Segments + Valve → Gas Bridge

Liquid Pipe Bridge `liquid_pipe_bridge`

A 3-tile liquid overpass that lets one line cross another without joining networks.

Produced by	Liquid Segments + Valve → Liquid Bridge

Power Line Bridge `power_line_bridge`

A 3-tile power overpass that lets one line cross another without joining networks.

Produced by	Power Segments + Circuit → Power Bridge

Pressure Valve `pressure_valve`

Machined flow-control valve used across pumps, tanks, and atmospheric hardware.

Produced by	Iron + Glass → Pressure Valve
Used in	Fabricate Smelter Mk II Upgrade Pack, Fabricate Oil Pump Mk II Upgrade Pack, Fabricate Smelter Mk III Upgrade Pack, Gas Segments + Valve → Gas Bridge, Liquid Segments + Valve → Liquid Bridge, Iron + Titanium + Valve + Panel → Oil Pump, Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Frame + Pipes + Valve → Sprinkler Kit, Titanium + Liquid Pipe Segments + Valve → Exotic Spring Tap Kit, Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II, Plastic + Coolant Loop + Valves → Sprinkler Pack Mk II, and Frames + Controls + Panels + Valves → Feed Maker Pack Mk II
Requested by	Spire, Pressure Hardware and Vestibule, Pressure Network Trial

Machine Frame `machine_frame`

Standard structural chassis most mid-tier machines are assembled on.

Produced by	Iron Ingot + Titanium Ingot → Machine Frame
Used in	Fabricate Smelter Mk II Upgrade Pack, Fabricate Fabricator Mk II Upgrade Pack, Fabricate Oil Pump Mk II Upgrade Pack, Fabricate Smelter Mk III Upgrade Pack, Fabricate Fabricator Mk III Upgrade Pack, Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit, Frame + Control + Panel → Drone Depot Kit (Medium), Frame + Control + Panel → Drone Depot Kit (Large), Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit, Frame + Control + Panel → Mining Drill Kit, Frame + Control + Panel + Rotor → Industrial Mining Drill Kit, Frame + Control + Panel + Rotors → Heavy Mining Drill Kit, Frame + Control + Panel → Seed Maker Kit, Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit, Frame + Panel + Glass → Grow Lamp Kit, Frame + Pipes + Valve → Sprinkler Kit, Frame + Control + Panel → Dispenser Kit, Frame + Iron + Panel → Waste Processor Kit, Frames + Controls + Panels + Valves → Feed Maker Pack Mk II, Oxygen Upgrade Pack Mk IV, Heat Upgrade Pack Mk IV, Pressure Upgrade Pack Mk IV, Lead Cask, Shield Plating, and Lightning Rod
Requested by	Helios, Frame Order, Helios, Bulk Iron Run, Helios, Reservoir Build, Helios, Cargo Pod Run, Helios, Bulk Frame Run, and Helios, Reactor Vessel Chain

Circuit Panel `circuit_panel`

Printed control circuitry for machine logic and instrumentation.

Produced by	Iron + Glass → Circuit Panel
Used in	Fabricate Fabricator Mk II Upgrade Pack, Fabricate Fabricator Mk III Upgrade Pack, Power Segments + Circuit → Power Bridge, Circuit Panel + Titanium Ingot + Glass → Control Unit, Iron + Titanium + Valve + Panel → Oil Pump, Frame + Control + Panel → Drone Depot Kit (Medium), Frame + Control + Panel → Drone Depot Kit (Large), Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit, Frame + Control + Panel → Mining Drill Kit, Frame + Control + Panel + Rotor → Industrial Mining Drill Kit, Frame + Control + Panel + Rotors → Heavy Mining Drill Kit, Titanium + Glass + Panel → Cargo Pod (Medium), Titanium + Glass + Panel → Cargo Pod (Large), Frame + Control + Panel → Seed Maker Kit, Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit, Frame + Panel + Glass → Grow Lamp Kit, Frame + Control + Panel → Dispenser Kit, Frame + Iron + Panel → Waste Processor Kit, Tar + Glass + Panel → Fertilizer Mk II, Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II, Panel + Glass + Rare Earth Core → Grow Lamp Pack Mk II, Controls + Panels + Coolant Loop → Sprinkler Pack Mk III, Frames + Controls + Panels + Valves → Feed Maker Pack Mk II, Oxygen Upgrade Pack Mk IV, Heat Upgrade Pack Mk IV, Pressure Upgrade Pack Mk IV, and Lightning Rod
Requested by	Spire, Circuit Order, Spire, Avionics Run, Spire, Avionics Megabuild, Spire, Polymer Optics Run, Spire, Optics Stockpile, and Spire, Avionics Megastock

Control Unit `control_unit`

Sealed processing module that drives advanced machines and vehicles.

Produced by	Circuit Panel + Titanium Ingot + Glass → Control Unit
Used in	Fabricate Smelter Mk II Upgrade Pack, Fabricate Fabricator Mk II Upgrade Pack, Fabricate Oil Pump Mk II Upgrade Pack, Fabricate Smelter Mk III Upgrade Pack, Fabricate Fabricator Mk III Upgrade Pack, Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit, Frame + Control + Panel → Drone Depot Kit (Medium), Frame + Control + Panel → Drone Depot Kit (Large), Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit, Frame + Control + Panel → Mining Drill Kit, Frame + Control + Panel + Rotor → Industrial Mining Drill Kit, Frame + Control + Panel + Rotors → Heavy Mining Drill Kit, Rare Earth Core + Titanium Ingot + Control → Drone (Small), Rare Earth Core + Titanium Ingot + Control → Drone (Medium), Rare Earth Core + Titanium Ingot + Control → Drone (Large), Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster, Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor, Frame + Control + Panel → Seed Maker Kit, Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit, Frame + Control + Panel → Dispenser Kit, Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier, Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II, Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III, Controls + Panels + Coolant Loop → Sprinkler Pack Mk III, Frames + Controls + Panels + Valves → Feed Maker Pack Mk II, and Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Requested by	Spire, Avionics Run, Spire, Control Run, Spire, Avionics Megabuild, Spire, Magnetic Stator Build, Spire, Propulsion Control Run, Spire, Neutron Capacitor Order, and Spire, Avionics Megastock

Battery Cell `battery_cell`

Cobalt-chemistry storage cell, the building block of battery packs and grid storage.

Produced by	Cobalt Ingot + Iron Ingot + Glass → Battery Cell
Used in	Frame + Control + Panel + Cell + Liquid Pipe Segment → Drone Service Station Kit, Titanium Ingot + Glass + Cell → Battery Pack, Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor, and Lightning Rod
Requested by	Helios, Battery Order, Helios, Drone Chassis Build, and Helios, Mid-Cargo Build

Thermal Cap Kit `thermal_cap_kit`

Constructor-deployed kit that caps a surveyed thermal vent with a `thermal_cap` and captures Steam. If its chamber reaches **100%**, it blows all stored Steam into the atmosphere; a script must release, route, or relieve the pressure.

Produced by	Titanium Ingot + Gas Pipe Segments → Thermal Cap Kit
Requested by	Helios, Thermal Cap Kit Order
Deploys	Thermal Cap

Turbine Rotor `turbine_rotor`

Precision-balanced titanium rotor for steam turbines.

Produced by	Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor
Used in	Frame + Control + Panel + Rotor → Industrial Mining Drill Kit, Frame + Control + Panel + Rotors → Heavy Mining Drill Kit, and Rare Earth Core + Rotor → Electric Thruster
Requested by	Helios, Rotor Run

Tank Lining `tank_lining`

Corrosion-proof lining for gas and liquid tanks.

Produced by	Iron + Titanium + Glass → Tank Lining
Used in	Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium) and Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large)
Requested by	Helios, Drone Chassis Build, Helios, Reservoir Build, and Helios, Mid-Cargo Build

Lubricant `lubricant`

Refined oil derivative that keeps drives and rotors running smoothly.

Produced by	Iron + Oil → Lubricant + Tar
Used in	Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster, Titanium Ingot + Lubricant + Rubber → Oil Tank (Small), Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium), and Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large)
Requested by	Vestibule, Polymer Stockpile, Vestibule, Bulk Polymer Run, Vestibule, Polymer Run, Vestibule, Pipe & Lubricant Order, Vestibule, Refueling Hardware Build, Vestibule, Polymer Bulk Order, Vestibule, Polymer Megaorder, Vestibule, Tether Megahaul, and Vestibule, Shielded Transport Trial

Plastic `plastic`

Oil-derived polymer stock for housings and insulation.

Produced by	Glass + Oil → Plastic + Tar
Used in	Plastic + Forage + Water → Reinforced Biopolymer, Plastic + Rare Earth Core → Growth Accelerant, and Plastic + Coolant Loop + Valves → Sprinkler Pack Mk II
Requested by	Spire, Polymer Optics Run, Spire, Polymer Megastock, and Spire, Polymer Megaorder

Rubber `rubber`

Flexible oil-derived stock for gaskets and seals.

Produced by	Cobalt + Oil → Rubber + Tar
Used in	Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster, Titanium Ingot + Lubricant + Rubber → Oil Tank (Small), Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium), and Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large)
Requested by	Vestibule, Polymer Stockpile, Vestibule, Bulk Polymer Run, Vestibule, Polymer Run, Vestibule, Refueling Hardware Build, Vestibule, Polymer Bulk Order, Vestibule, Polymer Megaorder, and Vestibule, Flexible Habitat Liners

Tar `tar`

Heavy oil residue used in fertilizer, advanced components, and the `refiner`. Oil-product recipes create it as a byproduct; Heavy Oil Cracking produces it directly.

Produced by	Heavy Oil Cracking: Oil → Tar
Byproduct of	Iron + Oil → Lubricant + Tar, Glass + Oil → Plastic + Tar, and Cobalt + Oil → Rubber + Tar
Used in	Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor, Tar + Glass → Fertilizer, Tar + Glass + Panel → Fertilizer Mk II, Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III, Raw Sulfur Gas + Tar → Sulfur Gas, Raw Cryofluid + Tar → Cryofluid, Raw Chlorine + Tar → Chlorine, and Raw Quicksilver + Tar → Quicksilver

Reinforced Biopolymer `reinforced_biopolymer`

Structural biological composite made from plastic, plant forage, and water. Used in late Earth exports and advanced biological chemistry.

Produced by	Plastic + Forage + Water → Reinforced Biopolymer
Used in	Reinforced Biopolymer + Forage + Water → Bio Enrichment Compound
Requested by	Spire, Biopolymer Thermal Stock, Vestibule, Flexible Habitat Liners, Helios, Enriched Habitat Trial, and Spire, Biosphere Export Program

Bio Enrichment Compound `enrichment_compound`

Concentrated biological material made from Reinforced Biopolymer, forage, and water for late Earth exports.

Produced by	Reinforced Biopolymer + Forage + Water → Bio Enrichment Compound
Requested by	Helios, Enriched Habitat Trial and Spire, Biosphere Export Program

Coolant Loop `coolant_loop`

Closed-circuit coolant assembly for reactor-grade heat loads.

Produced by	Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop
Used in	Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier, Plastic + Coolant Loop + Valves → Sprinkler Pack Mk II, Controls + Panels + Coolant Loop → Sprinkler Pack Mk III, and Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Requested by	Spire, Polymer Megastock, Spire, T4 Reactor Build, and Spire, Biopolymer Thermal Stock

Neutron Capacitor `neutron_capacitor`

Neutronium storage device for extreme energy applications.

Produced by	Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor
Used in	Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III, Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier, Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III, and Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Requested by	Spire, Neutron Capacitor Order, Spire, Capacitor Bulk Order, and Spire, T4 Reactor Build

Fertilizer `fertilizer`

Basic soil nutrient blend. One field dose gives a growing crop **2×** base Forage yield on its own (**+100%**) for **8 hours**. Same-tier doses extend duration; switch tiers after the active dose expires. Its yield bonus adds to provider and Yield Amplifier bonuses. A Plant Terraformer consumes each item for **10 potency** toward its recipe. Unused potency stays in the machine for later batches.

Produced by	Tar + Glass → Fertilizer

Fertilizer Mk II `fertilizer_mk2`

Concentrated nutrient blend. One field dose gives a growing crop **3×** base Forage yield on its own (**+200%**) for **8 hours**. Same-tier doses extend duration; switch tiers after the active dose expires. Its yield bonus adds to provider and Yield Amplifier bonuses. A Plant Terraformer consumes each item for **30 potency** toward its recipe. Unused potency stays in the machine for later batches.

Produced by	Tar + Glass + Panel → Fertilizer Mk II

Fertilizer Mk III `fertilizer_mk3`

Top-grade nutrient blend. One field dose gives a growing crop **5×** base Forage yield on its own (**+400%**) for **8 hours**. Same-tier doses extend duration; switch tiers after the active dose expires. Its yield bonus adds to provider and Yield Amplifier bonuses. A Plant Terraformer consumes each item for **50 potency** toward its recipe. Unused potency stays in the machine for later batches.

Produced by	Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III

Growth Accelerant `growth_accelerant`

One field dose doubles a growing crop's growth speed for **8 hours**, reaching the same yield sooner while its requirements are met. Additional doses extend duration. Also consumed as a whole-item input for the Plant Terraformer's final conversion recipe; it has no potency tiers.

Produced by	Plastic + Rare Earth Core → Growth Accelerant

Yield Amplifier `yield_amplifier`

One dose adds **+200% base Forage yield** (**3×** on its own) to growing crops across the whole field for **24 hours**. Its bonus adds to provider and Fertilizer bonuses. Additional doses extend duration. Mature crops keep their already banked Forage.

Produced by	Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier

Grow Lamp Mk II Upgrade Pack `grow_lamp_upgrade_pack_mk2`

Upgrades one Grow Lamp to Mk II: **2×** base Forage yield on its own (**+100%**) and **25 W** draw while enabled. The bonus applies during growth to crops that require light in the four orthogonally adjacent cells, while the lamp is powered and enabled. Coverage and growth speed stay the same. Only the strongest covering lamp counts; its bonus adds to other yield bonuses.

Produced by	Panel + Glass + Rare Earth Core → Grow Lamp Pack Mk II
Component docs	Grow Lamp

Grow Lamp Mk III Upgrade Pack `grow_lamp_upgrade_pack_mk3`

Upgrades one Grow Lamp to Mk III: **4×** base Forage yield on its own (**+300%**) and **100 W** draw while enabled. The bonus applies during growth to crops that require light in the four orthogonally adjacent cells, while the lamp is powered and enabled. Coverage and growth speed stay the same. Only the strongest covering lamp counts; its bonus adds to other yield bonuses.

Produced by	Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III
Component docs	Grow Lamp

Sprinkler Mk II Upgrade Pack `sprinkler_upgrade_pack_mk2`

Upgrades one Sprinkler to Mk II: **2×** base Forage yield on its own (**+100%**), **25 W** draw while enabled, and **4 t/h Water** use while supplied. The bonus applies during growth to crops that require water in the four orthogonally adjacent cells, while the sprinkler is powered, enabled, and supplied. Coverage and growth speed stay the same. Only the strongest covering sprinkler counts; its bonus adds to other yield bonuses.

Produced by	Plastic + Coolant Loop + Valves → Sprinkler Pack Mk II
Component docs	Sprinkler

Sprinkler Mk III Upgrade Pack `sprinkler_upgrade_pack_mk3`

Upgrades one Sprinkler to Mk III: **4×** base Forage yield on its own (**+300%**), **100 W** draw while enabled, and **8 t/h Water** use while supplied. The bonus applies during growth to crops that require water in the four orthogonally adjacent cells, while the sprinkler is powered, enabled, and supplied. Coverage and growth speed stay the same. Only the strongest covering sprinkler counts; its bonus adds to other yield bonuses.

Produced by	Controls + Panels + Coolant Loop → Sprinkler Pack Mk III
Component docs	Sprinkler

Lead Plate `lead_plate`

Rolled lead shielding plate for casks, fuel rods, and reactor internals.

Produced by	Lead Plate
Used in	Oxygen Upgrade Pack Mk IV, Heat Upgrade Pack Mk IV, Pressure Upgrade Pack Mk IV, Lead Cask, Shield Plating, Fuel Rod, and Nuclear Battery
Requested by	Helios, Plate Order, Vestibule, Shielded Transport Trial, and Helios, Reactor Vessel Chain

Raw Uranium `raw_uranium`

Storm-dealt fissile ore. Radioactive cargo: use Lead Casks for ordinary storage; deliver it to Fuel Assemblers or Supply Docks. Any drone can risk extraction, but each unplated batch adds 40 exposure; Shield Plating makes that gain zero.

Used in	Fuel Rod and Nuclear Battery
Requested by	Vestibule, Hot Freight Proof

Fuel Rod `fuel_rod`

Pressed reactor fuel, enriched uranium in a lead jacket. Radioactive cargo: use Lead Casks for ordinary storage; deliver it to machines that use fuel rods, such as Reactors, or to Supply Docks.

Produced by	Fuel Rod
Requested by	Vestibule, Fuel Rod Contract (Crown)

Agriculture & Feeds

Goods of the Plants and Wildlife chains: `salt`, manufactured seeds, `forage`, creature feeds, and field equipment kits.

Salt `salt`

Mineral byproduct of water pumping. Wells produce it alongside fresh water.

Seed Maker Kit `seed_maker_kit`

Fabricated deployment bundle for a `seed_maker`. Deploy it from Inventory to an operational outpost, then manage the building under Biosphere, Plants.

Produced by	Frame + Control + Panel → Seed Maker Kit
Deploys	Seed Maker

Grow Lamp Kit `grow_lamp_kit`

Field kit for a `grow_lamp`. Carry it in Inventory and deploy it from the mobile Harvester; the deployed light provider services surrounding harvester-grid cells.

Produced by	Frame + Panel + Glass → Grow Lamp Kit
Deploys	Grow Lamp

Sprinkler Kit `sprinkler_kit`

Field kit for a `sprinkler`. Carry it in Inventory and deploy it from the mobile Harvester; the deployed water provider services surrounding harvester-grid cells.

Produced by	Frame + Pipes + Valve → Sprinkler Kit
Deploys	Sprinkler

Dispenser Kit `dispenser_kit`

Field kit for a `dispenser`. Carry it in Inventory and deploy it from the mobile Harvester; the deployed salt provider services surrounding harvester-grid cells.

Produced by	Frame + Control + Panel → Dispenser Kit
Deploys	Dispenser

Waste Processor Kit `garbage_disposal_kit`

Fabricated deployment bundle for a `garbage_disposal`, displayed as a Waste Processor. Deploy it from Inventory to an operational outpost. Its script can permanently destroy items, liquids, or gases, one selected stream at a time.

Produced by	Frame + Iron + Panel → Waste Processor Kit
Deploys	Waste Processor

Sunpetal Seed `seed_sunpetal`

Banked Sunpetal seed line from the `seed_maker`. Needs **Light**, matures after **12 h** in total with its needs met, and has a base yield of **10 Forage**.

Shadeleaf Seed `seed_shadeleaf`

Banked Shadeleaf seed line from the `seed_maker`. Needs **Shade**, matures after **12 h** in total with its needs met, and has a base yield of **10 Forage**.

Dewmoss Seed `seed_dewmoss`

Banked Dewmoss seed line from the `seed_maker`. Needs **Water**, matures after **12 h** in total with its needs met, and has a base yield of **10 Forage**.

Lonethorn Seed `seed_lonethorn`

Banked Lonethorn seed line from the `seed_maker`. Needs no orthogonally adjacent plant (directly above, below, left, or right; machines may stand there), matures after **24 h** in total with its needs met, and has a base yield of **25 Forage**.

Packfern Seed `seed_packfern`

Banked Packfern seed line from the `seed_maker`. Needs at least two orthogonally adjacent Packferns (directly above, below, left, or right), matures after **24 h** in total with its needs met, and has a base yield of **25 Forage**.

Twinvine Seed `seed_twinvine`

Banked Twinvine seed line from the `seed_maker`. Needs an orthogonally adjacent Dewmoss (directly above, below, left, or right), matures after **24 h** in total with its needs met, and has a base yield of **25 Forage**.

Spitebud Seed `seed_spitebud`

Banked Spitebud seed line from the `seed_maker`. Needs no orthogonally adjacent Packfern (directly above, below, left, or right), matures after **24 h** in total with its needs met, and has a base yield of **25 Forage**.

Sunspur Seed `seed_sunspur`

Banked Sunspur seed line from the `seed_maker`. Needs **Light** and no orthogonally adjacent plant (directly above, below, left, or right; machines may stand there), matures after **24 h** in total with its needs met, and has a base yield of **25 Forage**.

Glowvine Seed `seed_glowvine`

Banked Glowvine seed line from the `seed_maker`. Needs **Light** and **Water**, matures after **48 h** in total with its needs met, and has a base yield of **60 Forage**.

Crowncap Seed `seed_crowncap`

Banked Crowncap seed line from the `seed_maker`. Needs **Shade** and at least two orthogonally adjacent Crowncaps (directly above, below, left, or right), matures after **48 h** in total with its needs met, and has a base yield of **60 Forage**.

Pondmoss Seed `seed_pondmoss`

Banked Pondmoss seed line from the `seed_maker`. Needs **Water** and at least two orthogonally adjacent Pondmoss plants (directly above, below, left, or right), matures after **48 h** in total with its needs met, and has a base yield of **60 Forage**.

Saltbloom Seed `seed_saltbloom`

Banked Saltbloom seed line from the `seed_maker`. Needs **Salt**, matures after **72 h** in total with its needs met, and has a base yield of **120 Forage**.

Brinethorn Seed `seed_brinethorn`

Banked Brinethorn seed line from the `seed_maker`. Needs **Salt** and no orthogonally adjacent plant (directly above, below, left, or right; machines may stand there), matures after **72 h** in total with its needs met, and has a base yield of **120 Forage**.

Saltmate Seed `seed_saltmate`

Banked Saltmate seed line from the `seed_maker`. Needs **Salt** and an orthogonally adjacent Saltbloom (directly above, below, left, or right), matures after **72 h** in total with its needs met, and has a base yield of **120 Forage**.

Grandbloom Seed `seed_grandbloom`

Banked Grandbloom seed line from the `seed_maker`. Needs **Light**, **Water**, and no orthogonally adjacent plant (directly above, below, left, or right; machines may stand there), matures after **72 h** in total with its needs met, and has a base yield of **150 Forage**.

Plant Forage `forage`

Physical crop yield collected when a ready plant is harvested and removed. The calorie base of every creature feed and the bulk input for Plant Terraformer batches.

Used in	Plastic + Forage + Water → Reinforced Biopolymer, Reinforced Biopolymer + Forage + Water → Bio Enrichment Compound, Forage + Sea Algae + Snow Moss → Salt Tortoise Feed, Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed, Forage + Cave Fungus + Frost Fungus + Coral Fungus → Mycelial Husk Feed, Forage + Magma Crust + Stone Mat → Mantle Strider Feed, Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed, Forage + Cold Spores + Hot Spores → Veil Mantle Feed, Forage + Stone Lichen + Salt Crust → Vault Crab Feed, Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed, Forage + Ice Crust + Ash Spores → Bone Walker Feed, Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed, Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed, Forage + Cave Moss + Heat Crust → Hollow Choir Feed, Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed, Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed, Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed, and Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

Salt Tortoise Feed `feed_salt_tortoise`

Feed pressed for the Salt Tortoise, a coastal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Sea Algae + Snow Moss → Salt Tortoise Feed

Mycelial Husk Feed `feed_mycelial_husk`

Feed pressed for the Mycelial Husk, a deep-cavern species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Cave Fungus + Frost Fungus + Coral Fungus → Mycelial Husk Feed

Mantle Strider Feed `feed_mantle_strider`

Feed pressed for the Mantle Strider, a volcanic species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Magma Crust + Stone Mat → Mantle Strider Feed

Glasswing Mantis Feed `feed_glasswing_mantis`

Feed pressed for the Glasswing Mantis, a geothermal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed

Veil Mantle Feed `feed_veil_mantle`

Feed pressed for the Veil Mantle, a frozen-plains species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Cold Spores + Hot Spores → Veil Mantle Feed

Vault Crab Feed `feed_vault_crab`

Feed pressed for the Vault Crab, a deep-cavern species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Stone Lichen + Salt Crust → Vault Crab Feed

Tidal Cephalopod Feed `feed_tidal_cephalopod`

Feed pressed for the Tidal Cephalopod, a coastal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed

Bone Walker Feed `feed_bone_walker`

Feed pressed for the Bone Walker, a frozen-plains species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Ice Crust + Ash Spores → Bone Walker Feed

Vent Drifter Feed `feed_vent_drifter`

Feed pressed for the Vent Drifter, a geothermal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed

Hive Sentinel Feed `feed_hive_sentinel`

Feed pressed for the Hive Sentinel, a geothermal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed

Hollow Choir Feed `feed_hollow_choir`

Feed pressed for the Hollow Choir, a deep-cavern species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Cave Moss + Heat Crust → Hollow Choir Feed

Ferric Sea-Lily Feed `feed_ferric_sea_lily`

Feed pressed for the Ferric Sea Lily, a coastal species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed

Crustal Echo Feed `feed_crustal_echo`

Feed pressed for the Crustal Echo, a deep-cavern species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed

Glacial Wyrm Feed `feed_glacial_wyrm`

Feed pressed for the Glacial Wyrm, a frozen-plains species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed

Spire Drake Feed `feed_spire_drake`

Feed pressed for the Spire Drake, a volcanic species. A Habitat colony accepts only its own formulation.

Produced by	Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

Life Forms

Native life forms gathered across the five biomes. They liquify into biome essences at the `essence_liquifier` and feature in creature feed recipes.

Ice Algae `ice_algae`

Pale algal ribbons frozen into meltwater films, faintly luminous beneath the ice. The most abundant catch on the frozen plains.

Biome	Frozen
Rarity	Common
Used in	Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed

Snow Moss `snow_moss`

Low moss matting the windward faces of drifts, rippling with every gust. Grows thick wherever snow settles and stays.

Biome	Frozen
Rarity	Common
Used in	Forage + Sea Algae + Snow Moss → Salt Tortoise Feed

Frost Lichen `frost_lichen`

Crusted lichen that pales in daylight and darkens through the long night. Clings to any exposed frozen rock.

Biome	Frozen
Rarity	Common
Used in	Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed and Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed

Cold Spores `cold_spores`

Dormant spore clusters that survive deep cold by shutting down entirely. Far denser in essence than the surface mats.

Biome	Frozen
Rarity	Uncommon
Used in	Forage + Cold Spores + Hot Spores → Veil Mantle Feed and Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

Ice Crust `ice_crust`

A living crust fused into the ice itself, prised away in brittle sheets. Slow-growing, and richer for it.

Biome	Frozen
Rarity	Uncommon
Used in	Forage + Ice Crust + Ash Spores → Bone Walker Feed

Frost Fungus `frost_fungus`

Rare fruiting bodies that push up through the ice only in the deepest cold. The frozen biome's densest essence source.

Biome	Frozen
Rarity	Rare
Used in	Forage + Cave Fungus + Frost Fungus + Coral Fungus → Mycelial Husk Feed

Sea Algae `sea_algae`

Long algal ribbons drifting in the shallows. The staple coastal harvest, abundant along every tide line.

Biome	Coastal
Rarity	Common
Used in	Forage + Sea Algae + Snow Moss → Salt Tortoise Feed and Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed

Tide Moss `tide_moss`

Springy moss carpeting the tidal flats, combed flat by the wind between tides.

Biome	Coastal
Rarity	Common
Used in	Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed

Shore Lichen `shore_lichen`

Salt-tolerant lichen banding the splash zone, bleaching brighter under full light.

Biome	Coastal
Rarity	Common
Used in	Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed

Brine Plankton `brine_plankton`

Luminous plankton blooming in dense brine. The water glows green where a bloom drifts through.

Biome	Coastal
Rarity	Uncommon
Used in	Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed

Salt Crust `salt_crust`

Living crust growing into evaporated salt pans, lifted away in pale plates.

Biome	Coastal
Rarity	Uncommon
Used in	Forage + Stone Lichen + Salt Crust → Vault Crab Feed

Coral Fungus `coral_fungus`

Branching fungal growth that mimics coral along sheltered shores. The coastal biome's richest catch.

Biome	Coastal
Rarity	Rare
Used in	Forage + Cave Fungus + Frost Fungus + Coral Fungus → Mycelial Husk Feed

Vent Algae `vent_algae`

Heat-loving algae streaming in the warm outflow around vents. Thrives wherever the ground runs hot.

Biome	Geothermal
Rarity	Common
Used in	Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed and Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed

Heat Lichen `heat_lichen`

Lichen crusting the warm rock of vent fields, brightening under direct light.

Biome	Geothermal
Rarity	Common
Used in	Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed

Hot Spores `hot_spores`

Spore clusters that need vent heat to open. Concentrated essence, packed and waiting for the next thermal pulse.

Biome	Geothermal
Rarity	Uncommon
Used in	Forage + Cold Spores + Hot Spores → Veil Mantle Feed and Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed

Heat Crust `heat_crust`

Mineral-laced crust grown onto vent rock, chipped free in warm plates.

Biome	Geothermal
Rarity	Uncommon
Used in	Forage + Cave Moss + Heat Crust → Hollow Choir Feed

Sulfur Moss `sulfur_moss`

Yellow-stained moss matting cooled flows, shifting constantly in the ash wind.

Biome	Volcanic
Rarity	Common
Used in	Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

Cinder Lichen `cinder_lichen`

Hardy lichen colonizing fresh cinder fields, palest where the light strikes hardest.

Biome	Volcanic
Rarity	Common
Used in	Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed and Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

Ash Spores `ash_spores`

Spore clusters carried on ashfall and settling wherever it lands. The volcanic staple.

Biome	Volcanic
Rarity	Common
Used in	Forage + Ice Crust + Ash Spores → Bone Walker Feed

Lava Algae `lava_algae`

Algal ribbons living at the very edge of cooling flows, glowing faintly against black rock.

Biome	Volcanic
Rarity	Uncommon
Used in	Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed

Magma Crust `magma_crust`

Living crust bonded to cooled magma, broken free in heavy plates.

Biome	Volcanic
Rarity	Uncommon
Used in	Forage + Magma Crust + Stone Mat → Mantle Strider Feed

Black Fungus `black_fungus`

Rare black fruiting bodies rooted in volcanic glass. The volcanic biome's richest essence.

Biome	Volcanic
Rarity	Rare
Used in	Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrm Feed

Cave Moss `cave_moss`

Pale moss carpeting deep cave floors, stirred by the slow draught through the passages.

Biome	Deep
Rarity	Common
Used in	Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed and Forage + Cave Moss + Heat Crust → Hollow Choir Feed

Crystal Spores `crystal_spores`

Glittering spore clusters that catch and re-emit light in the dark. The deep biome's common catch.

Biome	Deep
Rarity	Common
Used in	Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed and Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed

Deep Algae `deep_algae`

Luminous algal ribbons in still subterranean water, lighting the pools they grow in.

Biome	Deep
Rarity	Uncommon
Used in	Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed

Stone Mat `stone_mat`

A dense living mat fused to deep rock, cut away in slabs.

Biome	Deep
Rarity	Uncommon
Used in	Forage + Magma Crust + Stone Mat → Mantle Strider Feed and Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed

Field Resources

Loose finds harvesters collect in the field. Value climbs with distance from base.

Soil Sample `soil_sample`

A scoop of native regolith, the most common thing a Harvester picks up.

Sells for	50 cr

Basic Plant `basic_plant`

Hardy native flora specimen, common across the surface.

Sells for	125 cr

Organic Matter `organic_matter`

Decayed biological material; richer finds sit a little farther out.

Sells for	250 cr

Mineral Fragment `mineral_fragment`

Crystallized mineral chunk with respectable sale value.

Sells for	425 cr

Rare Fungi `rare_fungi`

Uncommon fungal growth found in the mid rings.

Sells for	850 cr

Crystal Shard `crystal_shard`

Prized crystalline formation from the outer rings.

Sells for	1,700 cr

Alien Fossil `alien_fossil`

Preserved trace of ancient planetary life, a collector's item back on Earth.

Sells for	2,500 cr

Exotic Compound `exotic_compound`

The rarest field find: an unclassified compound Earth's labs pay a premium for.

Sells for	5,000 cr

Storm Glass `storm_glass`

Glass fused where lightning struck open ground. A rare trade good Earth pays well for.

Requested by	Spire, Storm Glass Acquisition and Spire, Storm Glass Contract (Crown)

Biology Samples

Creature fragments recovered from the field and analyzed at the `bio_lab`. Bio Orders request them by name.

Cranial Plate `gw_cranial_plate`

Skull frontal plate preserved in cryogenic stasis. Two orbits flank a central frontal ridge; brow-crest spikes intact.

Rarity	os frontale glaciatum, Legendary
Biome	Frozen
Requested by	Apex Specimen Dossier, Nocturna Frozen Compendium, and Glacial Wyrm Husbandry Dossier
Analyzed at	Bio Lab

Caudal Fin `gw_caudal_fin`

Tail-fin membrane recovered from a brackish wash. Salt-pitted, barnacle colonies clinging to the attachment node.

Rarity	pinna caudalis pelagica, Legendary
Biome	Coastal
Requested by	Mangrove Tissue Consignment, Continental-Shelf Capstone, and Coastal Grand Compendium
Analyzed at	Bio Lab

Cardiac Node `gw_cardiac_node`

Heart muscle infused with mineral salts. Coronary vessels run with calcified gold; the central node still holds residual thermal charge.

Rarity	cor mineralizatum, Legendary
Biome	Geothermal
Requested by	Sinter Tissue Consignment, Geothermal-Field Capstone, and Geothermal Grand Compendium
Analyzed at	Bio Lab

Jaw Fang `gw_jaw_fang`

Predator fang recovered from a cooled lava flow. Obsidian-coated on one side; an interior fracture still vents ember-heat.

Rarity	dens praedatorius, Legendary
Biome	Volcanic
Requested by	Igneous Tissue Consignment, Eruption-Zone Capstone, and Volcanic Grand Compendium
Analyzed at	Bio Lab

Spinal Vertebra `gw_spinal_vertebra`

Pressure-fused vertebra from the abyssal plain. Bioluminescent residue still pulses through the neural canal, DNA viability untested.

Rarity	vertebra abyssalis, Legendary
Biome	Deep
Requested by	Aphotic Tissue Consignment, Deep-Trench Capstone, and Abyssal Grand Compendium
Analyzed at	Bio Lab

Dorsal Carapace `vc_dorsal_carapace`

Segmented dorsal shell preserved in glacial ice. Central ridge and lateral spines intact; intersegmental membrane mineralized.

Rarity	scutum dorsale glaciatum, Rare
Biome	Frozen
Requested by	Frozen Reef Census and Glaciation Index
Analyzed at	Bio Lab

Mandible Claw `vc_mandible_claw`

Pincer chela recovered from a brackish wash. Serrated occlusal edge intact; barnacle colonies fused to the wrist hinge.

Rarity	chela cancrida, Rare
Biome	Coastal
Requested by	Estuary Assay and Breaker Bio Survey
Analyzed at	Bio Lab

Antenna Cluster `vc_antenna_cluster`

Sensory antenna fascicle from a single carapace mount. Five flagella, mineral-fused at the joints; chemoreceptors calcified.

Rarity	flagella sensoria, Rare
Biome	Geothermal
Requested by	Mineral-Crust Assay and Solfatara Bio Survey
Analyzed at	Bio Lab

Walking Leg Joint `vc_walking_leg`

Articulated walking leg recovered from a lava flow. Coxa intact; tibia-dactyl junction obsidian-coated. Ember-heat still bleeds along the fracture.

Rarity	pereiopodium articulatum, Rare
Biome	Volcanic
Requested by	Lava-Tube Assay and Lahar Bio Survey
Analyzed at	Bio Lab

Eye-Stalk `vc_eye_stalk`

Bioluminescent compound eye on a segmented stalk. Recovered from the abyssal plain, facets preserved, neural core still emits a faint pulse.

Rarity	ommatophorus profundus, Rare
Biome	Deep
Requested by	Cold-Seep Assay and Pelagic Bio Survey
Analyzed at	Bio Lab

Cranium `oc_cranium`

Bulbous mantle skull recovered from glacial ice. Cartilaginous frame preserved; the central sinus still holds residual ink.

Rarity	cranium cephalopodi, Rare
Biome	Frozen
Requested by	Cephalopod Tissue Request
Analyzed at	Bio Lab

Tentacle Arm `oc_tentacle_arm`

Articulated tentacle with two rows of suckers along its length. Coastal salt-pitting visible along the proximal margin.

Rarity	brachium tentaculi, Rare
Biome	Coastal
Requested by	Lagoon Specimen Study and Mangrove Tissue Consignment
Analyzed at	Bio Lab

Chitin Beak `oc_chitin_beak`

Opposing parrot-style chitin mandibles articulated at a single hinge. Cutting edges intact; mineral crust along the rim.

Rarity	rostrum chitinosum, Rare
Biome	Geothermal
Requested by	Steam-Field Specimen Study and Sinter Tissue Consignment
Analyzed at	Bio Lab

Ink Sac `oc_ink_sac`

Mantle ink reservoir with the ejection duct preserved. Ember-burned along one side from volcanic exposure.

Rarity	saccus atramenti, Rare
Biome	Volcanic
Requested by	Cinder Specimen Study and Igneous Tissue Consignment
Analyzed at	Bio Lab

Camera Lens Eye `oc_lens_eye`

Hemispherical lens eye from the deep populations. Retinal layer still emits a faint bioluminescent pulse.

Rarity	oculus camera, Rare
Biome	Deep
Requested by	Brine-Pool Specimen Study and Aphotic Tissue Consignment
Analyzed at	Bio Lab

Skull `bw_skull`

Elongated bipedal skull with twin orbital sockets and a recurved jaw. Frost-pitted across the cranial dome.

Rarity	cranium ossarium, Rare
Biome	Frozen
Requested by	Predator Comparison Set, Apex Specimen Dossier, Bone Walker Forage Index, and Cryofluid Refinement Protocol
Analyzed at	Bio Lab

Foreclaw `bw_foreclaw`

Hooked grasping claw from the small forelimb. Salt-eroded along the inner curve; chitin sheath partly intact.

Rarity	manus rapax, Rare
Biome	Coastal
Requested by	Surf-Zone Catalog and Barnacle Fragment Manifest
Analyzed at	Bio Lab

Ribcage Segment `bw_ribcage`

Three articulated rib pairs joined to a vertebral segment. Mineral calcification fills the intercostal spaces.

Rarity	thorax costarum, Rare
Biome	Geothermal
Requested by	Caldera-Rim Catalog and Hydrothermal Fragment Manifest
Analyzed at	Bio Lab

Hindlimb Bone `bw_hindlimb`

Powerful digitigrade femur and tibia from a primary walking leg. Ember-burn scars trace the load-bearing axis.

Rarity	femur digitigradi, Rare
Biome	Volcanic
Requested by	Tephra Catalog and Volcanic Fragment Manifest
Analyzed at	Bio Lab

Tail Spike `bw_tail_spike`

Segmented tapered tail spike with lateral barbs. Bone marrow channel still carries bioluminescent residue.

Rarity	spiculum caudale, Rare
Biome	Deep
Requested by	Chemosynthetic Catalog and Hadopelagic Fragment Manifest
Analyzed at	Bio Lab

Bell Membrane `vd_bell`

Translucent dome membrane with the radial canal pattern preserved. Frost rime along the dorsal apex.

Rarity	calyptra bellaria, Rare
Biome	Frozen
Requested by	Frozen Reef Census and Cryofluid Refinement Protocol
Analyzed at	Bio Lab

Nematocyst Ring `vd_nematocyst`

Cluster of stinging cells from the bell rim. Salt-eroded but the firing capsules retain their venom load.

Rarity	cnidocytus ringens, Rare
Biome	Coastal
Requested by	Surf-Zone Catalog and Continental-Shelf Capstone
Analyzed at	Bio Lab

Neural Mesh `vd_neural_mesh`

Branching nerve net from inside the bell. Ganglia nodes still glow when warmed; mineral crust deposits on the trunks.

Rarity	rete nervosum, Rare
Biome	Geothermal
Requested by	Caldera-Rim Catalog and Geothermal-Field Capstone
Analyzed at	Bio Lab

Photophore `vd_photophore`

Light organ from the bell wall. Ember-scarred lens still produces a faint glow when exposed to air.

Rarity	photophorum luminosum, Rare
Biome	Volcanic
Requested by	Tephra Catalog and Eruption-Zone Capstone
Analyzed at	Bio Lab

Trailing Tendril `vd_tendril`

Long trailing tendril strand with photophore beads at the terminal tip. Pressure-fused along the proximal segments.

Rarity	tendrillum nematocum, Rare
Biome	Deep
Requested by	Chemosynthetic Catalog and Deep-Trench Capstone
Analyzed at	Bio Lab

Fruiting Body `mh_fruiting_body`

Mushroom cap with intact veil tissue and a frost-rimed crown. Spore print viable under cryogenic preservation.

Rarity	basidiocarpus glaciatus, Uncommon
Biome	Frozen
Requested by	Cryophyte Spore Panel, Nocturna Frozen Compendium, and Bone Walker Forage Index
Analyzed at	Bio Lab

Spore Pod `mh_spore_pod`

Suspended spore pod with the hyphal anchor threads attached. Salt-eaten release openings along the underside.

Rarity	sporangium dependens, Uncommon
Biome	Coastal
Requested by	Brine Specimen Panel, Lagoon Specimen Study, and Mangrove Tissue Consignment
Analyzed at	Bio Lab

Mycelium Root `mh_mycelium_root`

Branching hyphal root segment with septal divisions intact. Mineral crust plates the trunk junctions.

Rarity	hyphae radicales, Uncommon
Biome	Geothermal
Requested by	Sulfur Specimen Panel, Steam-Field Specimen Study, and Sinter Tissue Consignment
Analyzed at	Bio Lab

Chitin Node `mh_chitin_node`

Stiffened septal node from the mycelial network. Ember-burn craters scar the chitin reinforcement bands.

Rarity	nodus septalis, Uncommon
Biome	Volcanic
Requested by	Ashfall Specimen Panel, Cinder Specimen Study, and Igneous Tissue Consignment
Analyzed at	Bio Lab

Stigmatic Disc `mh_stigmatic_disc`

Sensory disc from the cap apex, the species' diagnostic 'eye' structure. Bioluminescent core still pulses faintly.

Rarity	stigma sapientiae, Uncommon
Biome	Deep
Requested by	Trench Specimen Panel, Brine-Pool Specimen Study, and Aphotic Tissue Consignment
Analyzed at	Bio Lab

Mandible `hs_mandible`

Hooked predator mandible with five serrated teeth along the cutting edge. Frost crystals along the chitin pits.

Rarity	mandibula chitinosa, Rare
Biome	Frozen
Requested by	Predator Comparison Set, Apex Specimen Dossier, and Glacial Wyrm Husbandry Dossier
Analyzed at	Bio Lab

Forewing Membrane `hs_wing_membrane`

Forewing membrane with intact costal vein and cross-vein cell pattern. Salt corrosion has eroded several pinholes.

Rarity	membrana alaria, Rare
Biome	Coastal
Requested by	Kelp-Forest Fragment Batch and Coastal Grand Compendium
Analyzed at	Bio Lab

Thorax Plate `hs_thorax_plate`

Broad armored thorax segment with three leg-socket pairs and two wing-base-socket pairs. Mineral gold patches along the plates.

Rarity	scutum thoracicum, Rare
Biome	Geothermal
Requested by	Mud-Pot Fragment Batch and Geothermal Grand Compendium
Analyzed at	Bio Lab

Abdomen Segment `hs_abdomen_segment`

Banded abdominal segment with the diagnostic yellow-black pattern. Ember-scarred along one flank.

Rarity	segmentum abdominis, Rare
Biome	Volcanic
Requested by	Scoria Fragment Batch and Volcanic Grand Compendium
Analyzed at	Bio Lab

Compound Eye `hs_compound_eye`

Hemispherical compound eye with the full hexagonal facet grid. Lenses still hold residual bioluminescence.

Rarity	oculus compositus, Rare
Biome	Deep
Requested by	Whale-Fall Fragment Batch and Abyssal Grand Compendium
Analyzed at	Bio Lab

Chelicera `ms_chelicera`

Articulated jaw mouthpart with a curved hypodermic fang. Venom channel runs the full length of the fang.

Rarity	chelicera venenifera, Uncommon
Biome	Frozen
Requested by	Arthropod Pair Study and Cryofluid Refinement Protocol
Analyzed at	Bio Lab

Leg Tarsus `ms_leg_tarsus`

Three distal leg segments, femur, tibia, tarsus, terminating in opposing hook claws. Sensory bristles intact.

Rarity	tarsus chelifer, Uncommon
Biome	Coastal
Requested by	Littoral Tissue Lot, Kelp-Forest Fragment Batch, and Barnacle Fragment Manifest
Analyzed at	Bio Lab

Pedipalp `ms_pedipalp`

Front-of-head sensory limb terminating in a bulbous palpal organ. Mineral calcification plates the basal segments.

Rarity	pedipalpus sensorius, Uncommon
Biome	Geothermal
Requested by	Fumarole Tissue Lot, Mud-Pot Fragment Batch, and Hydrothermal Fragment Manifest
Analyzed at	Bio Lab

Abdomen Sclerite `ms_abdomen_sclerite`

Dorsal armor plate from the opisthosoma with three chevron segments. Ember-burn craters along the median line.

Rarity	sclerites dorsalis, Uncommon
Biome	Volcanic
Requested by	Obsidian Tissue Lot, Scoria Fragment Batch, and Volcanic Fragment Manifest
Analyzed at	Bio Lab

Eye Cluster `ms_eye_cluster`

Cephalothorax dorsal plate preserving all eight simple eyes, two large principals plus six secondaries.

Rarity	ocelli octuplae, Uncommon
Biome	Deep
Requested by	Benthic Tissue Lot, Whale-Fall Fragment Batch, and Hadopelagic Fragment Manifest
Analyzed at	Bio Lab

Aperture Lip `hc_aperture_lip`

Crescent-shaped outer rim of the body chamber where soft tissue emerged. Nacre interior intact; siphon notch preserved.

Rarity	labium nacre, Rare
Biome	Frozen
Requested by	Glaciation Index
Analyzed at	Bio Lab

Shell Whorl `hc_shell_whorl`

Partial spiral fragment, one full whorl plus the inception of the next. Diagnostic wavy suture line visible.

Rarity	verticillus conchae, Rare
Biome	Coastal
Requested by	Shoal Specimen Cache and Coastal Grand Compendium
Analyzed at	Bio Lab

Septum Plate `hc_septum_plate`

Internal chamber wall, concave saucer with the diagnostic frilled suture margin and a central siphuncle perforation.

Rarity	septum chambrae, Rare
Biome	Geothermal
Requested by	Travertine Specimen Cache and Geothermal Grand Compendium
Analyzed at	Bio Lab

Hooked Beak `hc_beak`

Opposing chitin mandibles from the central mouth, parrot-style hooked jaws articulated at a single hinge.

Rarity	rhynchos sireni, Rare
Biome	Volcanic
Requested by	Pumice Specimen Cache and Volcanic Grand Compendium
Analyzed at	Bio Lab

Tentacle Crown `hc_tentacle_crown`

Cluster of six articulated tentacles converging at a central hub, each tipped with a bioluminescent photophore.

Rarity	corona brachialis, Rare
Biome	Deep
Requested by	Subduction Specimen Cache and Abyssal Grand Compendium
Analyzed at	Bio Lab

Cranial Papilla `ma_cranial_papilla`

Anterior head segment with five sensory papillae arrayed around a terminal mouth. Intra-segmental bioluminescent ring active.

Rarity	prostomium sensorium, Common
Biome	Frozen
Requested by	Frost Tissue Sample, Soil Enrichment Assay, and Frozen Tissue Consignment
Analyzed at	Bio Lab

Cuticle Molt `ma_cuticle_molt`

Discarded segment of the glass-like outer cuticle with five annular rings. Salt-eaten perforations across the dorsal surface.

Rarity	exuvium vitreum, Common
Biome	Coastal
Requested by	Tidal Tissue Sample, Coral Bio Compendium, Estuary Assay, and Kelp-Forest Fragment Batch
Analyzed at	Bio Lab

Ganglion Node `ma_ganglion_node`

Bilateral nerve ganglion, twin lobes joined by cross-commissure with four connectives intact. Core retains bioluminescence.

Rarity	ganglion paramedianum, Common
Biome	Geothermal
Requested by	Vent Tissue Sample, Thermal Bio Compendium, Mineral-Crust Assay, and Mud-Pot Fragment Batch
Analyzed at	Bio Lab

Chitinous Seta `ma_chitinous_seta`

Parapodial bristle bundle, five chitin needles radiating from a fleshy basal lobe. Barbed terminal tips intact.

Rarity	setae parapodialis, Common
Biome	Volcanic
Requested by	Basalt Tissue Sample, Caldera Bio Compendium, Lava-Tube Assay, and Scoria Fragment Batch
Analyzed at	Bio Lab

Luminous Ring `ma_luminous_ring`

Body segment cross-section preserving the diagnostic mid-segmental bioluminescent ring and lateral parapodia.

Rarity	annulus luminosus, Common
Biome	Deep
Requested by	Abyssal Tissue Sample, Bathic Bio Compendium, Cold-Seep Assay, and Whale-Fall Fragment Batch
Analyzed at	Bio Lab

Prismatic Eye `gm_compound_eye`

Hemispherical compound eye with the full hexagonal facet grid intact. Lens refraction still produces a prismatic sheen.

Rarity	oculus crystallinus, Uncommon
Biome	Frozen
Requested by	Arthropod Pair Study, Predator Comparison Set, and Glacial Wyrm Husbandry Dossier
Analyzed at	Bio Lab

Folded Wing `gm_folded_wing`

Membranous forewing folded once along the central crease, the resting configuration. Costal vein preserved.

Rarity	ala plicata, Uncommon
Biome	Coastal
Requested by	Coral Bio Compendium and Shoal Specimen Cache
Analyzed at	Bio Lab

Raptorial Claw `gm_raptorial_claw`

Folded foreleg, femur and tibia closed at the knee hinge with interlocking inner-edge spines. Terminal tarsal claw intact.

Rarity	brachium raptorium, Uncommon
Biome	Geothermal
Requested by	Thermal Bio Compendium and Travertine Specimen Cache
Analyzed at	Bio Lab

Abdominal Sheath `gm_abdominal_sheath`

Translucent tapered abdomen segment with five annular rings and paired terminal cerci. Refractive cuticle preserved.

Rarity	abdomen vitreum, Uncommon
Biome	Volcanic
Requested by	Caldera Bio Compendium and Pumice Specimen Cache
Analyzed at	Bio Lab

Antennal Whip `gm_antennal_whip`

Long filiform antenna, scape, pedicel joint, and the full beaded flagellum with sensory micro-hairs.

Rarity	flagellum sensorium, Uncommon
Biome	Deep
Requested by	Bathic Bio Compendium and Subduction Specimen Cache
Analyzed at	Bio Lab

Calyx Plate `fs_calyx_plate`

Pentagonal cup-body segment with central arm-articulation pore. Iron oxidation has stained three rust-orange patches.

Rarity	tabula calicis, Rare
Biome	Frozen
Requested by	Glaciation Index
Analyzed at	Bio Lab

Arm Segment `fs_arm_segment`

Mid-arm section with ten alternating pinnules and eight ossicle articulations. Salt-eroded pinnule barbs.

Rarity	brachium pinnatum, Rare
Biome	Coastal
Requested by	Shoal Specimen Cache
Analyzed at	Bio Lab

Stalk Columnal `fs_stalk_columnal`

Section of the supporting stalk, six stacked disc columnals threaded by the central axial canal. Mineral crusts present.

Rarity	columna stipitis, Rare
Biome	Geothermal
Requested by	Travertine Specimen Cache
Analyzed at	Bio Lab

Oral Tegmen `fs_oral_tegmen`

Disc-shaped cover of the calyx, central five-pointed mouth, five ambulacral food grooves, off-center anal pyramid.

Rarity	tegumentum orale, Rare
Biome	Volcanic
Requested by	Pumice Specimen Cache
Analyzed at	Bio Lab

Holdfast Rootlet `fs_holdfast_rootlet`

Stalk base with seven radiating root tendrils, the anchor that fixed the animal to substrate. Tip beads retain bioluminescence.

Rarity	cirri ancorae, Rare
Biome	Deep
Requested by	Subduction Specimen Cache
Analyzed at	Bio Lab

Cranial Crest `sd_cranial_crest`

Trio of obsidian-glass crest spires mounted on the dorsal skull plate. Crystalline facets intact along each leading edge.

Rarity	crista petrosa, Legendary
Biome	Frozen
Requested by	Nocturna Frozen Compendium
Analyzed at	Bio Lab

Wing Membrane `sd_wing_membrane`

Leathery wing membrane preserving the wing-finger bone and three joint articulations. Fiber striations radiate from the bone.

Rarity	patagium alarum, Legendary
Biome	Coastal
Requested by	Barnacle Fragment Manifest, Continental-Shelf Capstone, and Coastal Grand Compendium
Analyzed at	Bio Lab

Obsidian Scale `sd_obsidian_scale`

Large lozenge dorsal scale with four crystalline facets meeting at a central spine. Sharp specular sheen.

Rarity	squama obsidiana, Legendary
Biome	Geothermal
Requested by	Hydrothermal Fragment Manifest, Geothermal-Field Capstone, and Geothermal Grand Compendium
Analyzed at	Bio Lab

Tail Club `sd_tail_barb`

Terminal tail club, diamond barb with two lateral cutting spikes. Ember-burn craters scar both the barb head and shaft.

Rarity	spiculum caudae, Legendary
Biome	Volcanic
Requested by	Volcanic Fragment Manifest, Eruption-Zone Capstone, and Volcanic Grand Compendium
Analyzed at	Bio Lab

Talon `sd_talon`

Recurved hind talon with attached toe segment and articulated foot pad fragment. Talon tip retains bioluminescent residue.

Rarity	unguis raptoris, Legendary
Biome	Deep
Requested by	Hadopelagic Fragment Manifest, Deep-Trench Capstone, and Abyssal Grand Compendium
Analyzed at	Bio Lab

Stalked Eye `ce_stalked_eye`

Long peduncle stalk terminating in a hemispherical compound lens. Facet grid still produces faint refraction.

Rarity	oculus pedunculatus, Rare
Biome	Frozen
Requested by	Cephalopod Tissue Request
Analyzed at	Bio Lab

Swimmeret Lobe `ce_swimmeret_lobe`

Lateral paddle from a mid-body segment, four radial ribs from the root joint to the leading edge. Salt corrosion across the surface.

Rarity	lobus natatorius, Rare
Biome	Coastal
Requested by	Breaker Bio Survey
Analyzed at	Bio Lab

Mouth Disc `ce_mouth_disc`

Frontal mouth-plate apparatus, concentric outer ring of tooth plates surrounding an inner ring of inward-pointing fangs.

Rarity	discus oralis, Rare
Biome	Geothermal
Requested by	Solfatara Bio Survey
Analyzed at	Bio Lab

Great Appendage `ce_great_appendage`

Frontal grasping arm, five articulated segments with inward-curving hook spines on each, terminating in a recurved barb.

Rarity	brachium magnum, Rare
Biome	Volcanic
Requested by	Lahar Bio Survey
Analyzed at	Bio Lab

Cephalic Photophore `ce_cephalic_photophore`

Slender bioluminescent arm with four articulating segments terminating in a brilliant photophore lens.

Rarity	photophorum cephalicum, Rare
Biome	Deep
Requested by	Pelagic Bio Survey
Analyzed at	Bio Lab

Scute Plate `st_scute_plate`

Hexagonal carapace plate with concentric growth rings, four visible, roughly forty years of growth. Salt crystal accretions along the seams.

Rarity	scutum vertebrale, Common
Biome	Frozen
Requested by	Shell Fragment Survey, Nocturna Frozen Compendium, Soil Enrichment Assay, and Veil Mantle Diet Survey
Analyzed at	Bio Lab

Plastron Shard `st_plastron_shard`

Belly-shell fragment showing the central plastral suture and the cross-bridge to the carapace. Barnacle clusters on the underside.

Rarity	scutum plastrale, Common
Biome	Coastal
Requested by	Reef Fragment Survey, Saltmarsh Census, and Surf-Zone Catalog
Analyzed at	Bio Lab

Limb Claw `st_limb_claw`

Fore-limb toe terminating in a heavy curved keratin claw. Scale-plate hide texture preserved.

Rarity	unguis pedis, Common
Biome	Geothermal
Requested by	Geyser Fragment Survey, Hot-Spring Census, and Caldera-Rim Catalog
Analyzed at	Bio Lab

Beak `st_beak`

Keratin beak sheath, upper and lower rhamphothecae intact with their cutting edges and concentric growth ridges.

Rarity	rhamphotheca, Common
Biome	Volcanic
Requested by	Magma Fragment Survey, Pyroclast Census, and Tephra Catalog
Analyzed at	Bio Lab

Carapace Neural Plate `st_carapace_neural`

Central neural plate from the carapace dorsal column, the spine canal preserved with all six vertebral processes.

Rarity	tabula neuralis, Common
Biome	Deep
Requested by	Hadal Fragment Survey, Midnight-Zone Census, and Chemosynthetic Catalog
Analyzed at	Bio Lab

Cephalic Horn `vm_cephalic_horn`

Paired prehensile lobe from the front of the head, used to funnel small prey toward the mouth. Sensory papillae intact.

Rarity	cornu cephalicum, Uncommon
Biome	Frozen
Requested by	Veil Mantle Diet Survey
Analyzed at	Bio Lab

Wing Sheet `vm_wing_sheet`

Mid-wing section preserving five cartilage spokes and the dorsal mottle pattern. Leading-edge ridge intact.

Rarity	tegmen pectorale, Uncommon
Biome	Coastal
Requested by	Saltmarsh Census and Breaker Bio Survey
Analyzed at	Bio Lab

Gill Filament `vm_gill_filament`

Branched respiratory tissue from a gill slit, eight primary filaments with secondary lamellae intact along each.

Rarity	filamenta branchialia, Uncommon
Biome	Geothermal
Requested by	Hot-Spring Census and Solfatara Bio Survey
Analyzed at	Bio Lab

Tail Barb `vm_tail_barb`

Long serrated tail spike with five backward-pointing barb teeth. Ember-burn craters scar the shaft.

Rarity	aculeus caudae, Uncommon
Biome	Volcanic
Requested by	Pyroclast Census and Lahar Bio Survey
Analyzed at	Bio Lab

Ventral Photophore `vm_ventral_photophore`

Ventral skin patch with seven embedded photophore light organs in the diagnostic constellation pattern.

Rarity	photophora ventralis, Uncommon
Biome	Deep
Requested by	Midnight-Zone Census and Pelagic Bio Survey
Analyzed at	Bio Lab

Lab Reagents

Consumable lab stock the Biology Lab spends while analyzing fragments. Bought from the Shop.

Alkaline Buffer `alkaline_buffer`

Lab reagent used in many specimen analyses.

Sells for	1 cr
Shop price	1 cr

Cryo-Solvent `cryo_solvent`

Lab reagent for deep-frozen specimens.

Sells for	2 cr
Shop price	2 cr

Protein Marker `protein_marker`

Lab reagent for identifying rare species.

Sells for	40 cr
Shop price	40 cr

Chelating Agent `chelating_agent`

Lab reagent for processing metal-rich specimens.

Sells for	200 cr
Shop price	200 cr

Enzyme Solution `enzyme_solution`

Lab reagent for analyzing living-tissue specimens.

Sells for	1,000 cr
Shop price	1,000 cr

Fluids

Every substance that flows through pipes, one entry per fluid. Gas Pipes carry gases and Liquid Pipes carry liquids. Complete machine `connect()` relationships establish one exact substance per pipe network; different exact substances on one network conflict. See **Flow Networks, Fluids** for the complete routing model.

INDUSTRIAL FLUIDS

The three working fluids of the power and production economy.

Steam `steam`

Superheated vapor from the planet's thermal vents. Captured by a `thermal_cap`, then spent by a `steam_turbine` for grid power or a `steam_condenser` for clean water.

Type	Gas
Produced by	Thermal Cap
Consumed by	Fabricator, Steam Turbine, Steam Condenser, and Bio Caster and Titanium Ingot + Cobalt Ingot + Rare Earth Core → Turbine Rotor, Rare Earth Core + Rotor → Electric Thruster, Titanium + Glass → Cargo Pod (Small), Titanium + Glass + Panel → Cargo Pod (Medium), Titanium + Glass + Panel → Cargo Pod (Large), and Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor
Tier input	Heat Generator Mk III, 12 t/h

Water `water`

Fresh liquid water raised from underground wells by a `water_pump`, or recovered from steam by a `steam_condenser`. Pumped groundwater leaves `salt` behind as a byproduct. Feeds fluid-hungry machine tiers and industry.

Type	Liquid
Produced by	Steam Condenser and Water Pump
Consumed by	Fabricator, Bio Caster, Plant Terraformer, Sprinkler, and Reactor and Fabricate Smelter Mk II Upgrade Pack, Fabricate Fabricator Mk II Upgrade Pack, Fabricate Oil Pump Mk II Upgrade Pack, Fabricate Smelter Mk III Upgrade Pack, Fabricate Fabricator Mk III Upgrade Pack, Iron Ingot + Titanium Ingot → Machine Frame, Iron + Glass → Circuit Panel, Circuit Panel + Titanium Ingot + Glass → Control Unit, Cobalt Ingot + Iron Ingot + Glass → Battery Cell, Iron + Titanium + Glass → Tank Lining, Plastic + Forage + Water → Reinforced Biopolymer, Reinforced Biopolymer + Forage + Water → Bio Enrichment Compound, Frame + Control + Gas Pipe Segments + Liquid Pipe Segments → Drone Depot Kit, Frame + Control + Panel → Drone Depot Kit (Medium), Frame + Control + Panel → Drone Depot Kit (Large), Frame + Control + Panel → Mining Drill Kit, Frame + Control + Panel + Rotor → Industrial Mining Drill Kit, Frame + Control + Panel + Rotors → Heavy Mining Drill Kit, Rare Earth Core + Titanium Ingot + Control → Drone (Small), Rare Earth Core + Titanium Ingot + Control → Drone (Medium), Rare Earth Core + Titanium Ingot + Control → Drone (Large), Rare Earth Core + Control + Lubricant + Rubber → Heli Thruster, Titanium Ingot + Lubricant + Rubber → Oil Tank (Small), Titanium Ingot + Lubricant + Rubber + Lining → Oil Tank (Medium), Titanium Ingot + Lubricant + Rubber + Tank Lining → Oil Tank (Large), Pipes + Valves + Rare Earth Core + Titanium Ingot + Tar → Coolant Loop, Neutronium Bar + Cells + Controls + Rare Earth Core + Tar → Neutron Capacitor, Frame + Control + Panel → Seed Maker Kit, Frames + Controls + Panels + Liquid Pipe Segments → Plant Terraformer Kit, Frame + Pipes + Valve → Sprinkler Kit, Tar + Glass → Fertilizer, Tar + Glass + Panel → Fertilizer Mk II, Tar + Glass + Capacitor + Rare Earth Core → Fertilizer Mk III, Plastic + Rare Earth Core → Growth Accelerant, Capacitor + Controls + Rare Earth Core + Coolant Loop → Yield Amplifier, Controls + Panels + Rare Earth Core + Valves → Plant Terraformer Pack Mk II, Capacitor + Rare Earth Core + Controls → Grow Lamp Pack Mk III, Controls + Panels + Coolant Loop → Sprinkler Pack Mk III, Frames + Controls + Panels + Valves → Feed Maker Pack Mk II, and Capacitors + Rare Earth Core + Controls + Coolant Loop → Habitat Pack Mk II
Tier input	Oxygen Generator Mk III, 8 t/h and Pressure Generator Mk III, 5 t/h

Oil `oil`

Heavy crude drawn from oil wells by an `oil_pump`. Burned by the `oil_generator` for power and processed into polymers, `lubricant`, and `tar`.

Type	Liquid
Produced by	Oil Pump
Consumed by	Fabricator, Oil Generator, and Drone Service Station and Iron + Oil → Lubricant + Tar, Glass + Oil → Plastic + Tar, Cobalt + Oil → Rubber + Tar, and Heavy Oil Cracking: Oil → Tar

BIOME ESSENCES

Biological concentrates pressed from native life forms. Each biome yields its own essence.

Frozen Essence `frozen_essence`

Pale crystalline concentrate liquified from Frozen-biome life forms at the `essence_liquifier`.

Type	Liquid
Produced by	Essence Liquifier
Consumed by	Biomass Mixer

Coastal Essence `coastal_essence`

Brackish green concentrate liquified from Coastal-biome life forms at the `essence_liquifier`.

Type	Liquid
Produced by	Essence Liquifier
Consumed by	Biomass Mixer

Geothermal Essence `geothermal_essence`

Warm mineral concentrate liquified from Geothermal-biome life forms at the `essence_liquifier`.

Type	Liquid
Produced by	Essence Liquifier
Consumed by	Biomass Mixer

Volcanic Essence `volcanic_essence`

Deep red sulfurous concentrate liquified from Volcanic-biome life forms at the `essence_liquifier`.

Type	Liquid
Produced by	Essence Liquifier
Consumed by	Biomass Mixer

Deep Essence `deep_essence`

Violet subterranean concentrate liquified from Deep-biome life forms at the `essence_liquifier`.

Type	Liquid
Produced by	Essence Liquifier
Consumed by	Biomass Mixer

EXOTIC GASES

Gases tapped from cyclic vent deposits with an `exotic_gas_cap`. Common gases flow straight to use; rarer ones arrive raw and need refining.

Ammonia `ammonia`

Common alkaline exotic gas, tapped straight from its vent deposits. Usable as-is, no refining needed.

Type	Gas

Swamp Gas `swamp_gas`

Common marsh gas tapped from lowland vent deposits. Usable as-is, no refining needed.

Type	Gas

Raw Sulfur Gas `raw_sulfur_gas`

Unrefined sulfur feedstock from volcanic gas vents. The `refiner` purifies it into `sulfur_gas`, burning `tar` in the process.

Type	Gas
Refines into	Sulfur Gas
Consumed by	Raw Sulfur Gas + Tar → Sulfur Gas

Sulfur Gas `sulfur_gas`

Refined sulfur gas, purified from `raw_sulfur_gas` at the `refiner`.

Type	Gas
Refined from	Raw Sulfur Gas
Recipe output	Raw Sulfur Gas + Tar → Sulfur Gas

Raw Chlorine `raw_chlorine`

Unrefined chlorine feedstock from gas vents. Refines into `chlorine` at a heavy `tar` cost.

Type	Gas
Refines into	Chlorine
Consumed by	Raw Chlorine + Tar → Chlorine

Chlorine `chlorine`

Refined caustic chlorine, the rarest of the exotic gases.

Type	Gas
Refined from	Raw Chlorine
Recipe output	Raw Chlorine + Tar → Chlorine

EXOTIC LIQUIDS

Liquids tapped from spring deposits with an `exotic_spring_tap`. Common liquids flow straight to use; rarer ones arrive raw and need refining.

Brine `brine`

Common mineral-heavy liquid tapped from spring deposits. Usable as-is, no refining needed.

Type	Liquid

Raw Cryofluid `raw_cryofluid`

Unrefined supercooled feedstock from spring taps. The `refiner` purifies it into `cryofluid`.

Type	Liquid
Refines into	Cryofluid
Consumed by	Raw Cryofluid + Tar → Cryofluid

Cryofluid `cryofluid`

Refined supercooled fluid, purified from `raw_cryofluid` at the `refiner`.

Type	Liquid
Refined from	Raw Cryofluid
Recipe output	Raw Cryofluid + Tar → Cryofluid

Raw Quicksilver `raw_quicksilver`

Unrefined liquid-metal amalgam from spring taps. Refines into `quicksilver` at a heavy `tar` cost.

Type	Liquid
Refines into	Quicksilver
Consumed by	Raw Quicksilver + Tar → Quicksilver

Quicksilver `quicksilver`

Refined liquid metal, the rarest of the exotic liquids.

Type	Liquid
Refined from	Raw Quicksilver
Recipe output	Raw Quicksilver + Tar → Quicksilver

Research

The complete technology tree for this build. Research remains documented before it unlocks so you can search future systems, understand their thresholds, and plan toward them.

TERRAFORM INDEX

Camera Feed `research_camera_feed` UNLOCKED

Watch Nocturna change in real time. Bring the exterior camera online and watch the world outside through a live feed. Storms cross the basin, ice gives way, strange life takes hold, and the base you built glows against the dark.

Threshold	Terraform Index 5,000
Tech id	camera_feed_unlock

Ship Computer `research_computer` UNLOCKED

Computer dashboard access. Unlocks the Ship → Computer page: a dashboard for system status, script management, the Playground, achievements, and notifications. It also opens the `computer` component to scripts, so deploying, upgrading, undeploying, decommissioning, and renaming can run from code instead of a button. Further research adds more Computer tabs, starting with the Library.

Threshold	Terraform Index 10,000
Tech id	ship_computer

Signal Bus `research_signal_bus` UNLOCKED

Script-to-script messaging. Adds the Signal Bus tab to the Ship Computer. Scripts coordinate with each other by sending queued messages or broadcasting shared values on named channels: explicit, save-safe shared state instead of hidden globals.

Threshold	Terraform Index 35,000
Tech id	signal_bus_unlock

Automations `research_automations` UNLOCKED

Scripts that run without a machine. Opens Ship Computer > Automations: scripts that belong to no machine and keep running on their own. Use them for base-wide work such as power management, shop purchases, deploying equipment and construction planning.

Threshold	Terraform Index 50,000
Tech id	automations_unlock

Data Archive `research_data_archive` UNLOCKED

Persistent learned tables. Adds the Data Archive tab to the Ship Computer. Scripts can store lookup tables, calibration samples, discovered maps, and other learned knowledge that persists across restarts and save/load. Use the Signal Bus for live coordination and the Data Archive for long-lived data.

Threshold	Terraform Index 70,000
Tech id	data_archive_unlock

Pioneer Chassis `research_pioneer` UNLOCKED

Expedition vehicle access. Unlocks the eight-slot Pioneer chassis in the Shop. It has no built-in battery, cargo, or Nav and cannot move until separately purchased hardware is installed.

Threshold	Terraform Index 100,000
Tech id	pioneer_unlock

Supply Logistics `research_orders_system` UNLOCKED

Unlocks Orders and Supply Dock purchasing. A long-range logistics protocol for Earth demand. Supply Docks become available in the Shop and ship **25 units/h** before throughput research; build more docks to serve orders in parallel.

Threshold	Terraform Index 110,000
Tech id	orders_system_unlock

Outpost Construction `research_outpost_hub` UNLOCKED

Found new sites across the planet. Unlocks the Outpost Kit in the shop. Founding also needs Constructor Module research and a Constructor-equipped Pioneer to build the blueprint. Each outpost holds a soft **20** buildings, which Outpost Expansion later raises by **5**. Each kit costs more than the last, **15,000 cr** to **1,000,000 cr**, priced on founded expansion outposts plus unbuilt Outpost Kits held in physical storage.

Threshold	Terraform Index 120,000
Tech id	outpost_unlock

Fabrication `research_fabricator` UNLOCKED

Heavy manufacturing access. Unlocks the Fabricator in the Shop. This industrial assembly press combines refined materials into finished equipment, modules, and vehicle frames.

Threshold	Terraform Index 130,000
Tech id	fabricator_unlock

Cartography `research_cartography` UNLOCKED

Annotate the Planet Map. Adds map markers: named, colored annotations you place in Plan Mode or from any script with `get_component("markers")`. Markers are notes, not construction. They cost nothing, need no Pioneer, and appear the moment you place them, so a survey script can record what it found where it found it.

Threshold	Terraform Index 140,000
Tech id	cartography_unlock

Control Room `research_custom_panels` UNLOCKED

Create script-driven control panels. Opens the Control Room, a top-level page for script-driven dashboard cards. Build clocks, fleet boards, thermal vent cycle timelines, and recovery controls with drawing and input APIs.

Threshold	Terraform Index 150,000
Tech id	custom_panels_unlock

Basic Drone Operations `research_drones` UNLOCKED

Open aerial logistics before the biosphere era. Unlocks the drone system and its fleet dashboard, the Vehicles / Drones / Support roster under the Planet Map. Electric drones fly up to **300 m/h** and consume **5 Wh/h** at full throttle; burn rises with throttle squared. Drone hardware still requires blueprint recipes earned through Earth Orders. Fabricated facilities and chassis deploy from Inventory into outposts; field Mining Drills use Planet Map blueprints placed in Plan Mode or by script.

Threshold	Terraform Index 180,000
Tech id	drones_unlock

Biosphere `research_biosphere` UNLOCKED

Unlocks biomass supply chain. Unlocks the **Biomass tier** supply chain and makes the Portable Bio Scanner, Portable Bio Extractor, Essence Liquifier, and Biomass Mixer available in the Shop. Scan biome tiles, extract life-form samples, and run them through the chain to produce biomass.

Threshold	Terraform Index 210,000
Tech id	biosphere_unlock

Bigger Stacks `research_high_density_storage` UNLOCKED

Every inventory slot holds twice as much. Rebuilds the shelving at Nocturna Base so each slot takes a taller stack. Items that stack now go up to **20 per slot** instead of **10**. Equipment, modules, and other one-off items still take a slot each. This changes Base Inventory only, not vehicle cargo, Storage Bins, or Warehouses.

Threshold	Terraform Index 250,000
Tech id	high_density_storage_unlock

Weather Program `research_weather_program` NOT YET UNLOCKED

Track storms and treasure signals. Unlocks the **Weather Network**, installs one free completed Weather Station and its building slot at Nocturna Base, and makes additional stations available in the Shop. Observe local storms, handle live transmissions, and publish optional results to the Signal Board.

Threshold	Terraform Index 330,000
Tech id	weather_program_unlock

Industrial Machinery Mk II `research_industrial_machinery_mk2` NOT YET UNLOCKED

Manufacture upgrades for faster factories. Unlocks Fabricator recipes for individual Smelter and Fabricator Mk II upgrade packs. Installed machinery processes twice as fast with 1.5× operating power. Apply each pack through Inventory or the Computer.

Threshold	Terraform Index 375,000
Tech id	industrial_machinery_mk2_unlock

Fast Feeders `research_fast_feeders` NOT YET UNLOCKED

Twice the item-transfer throughput. Reduces the duration of every timed discrete-item transfer by **50%**. Machine endpoints remain occupied until the faster handling cycle finishes. Fluids and manual world actions are unaffected.

Threshold	Terraform Index 400,000
Tech id	fast_feeders_unlock

Advanced Oil Extraction `research_advanced_oil_extraction` NOT YET UNLOCKED

Improve pumps and prospect for deep oil. Unlocks the Oil Pump Mk II pack recipe and Seismic Sonar purchase in the Shop. Pioneers install pump packs through construction blueprints. To find three additional oil reservoirs, revisit inert formations, including ones already surveyed with ordinary sonar. Scan them with Seismic Sonar, then survey the potential deep oil contacts to reveal Oil Wells.

Threshold	Terraform Index 475,000
Tech id	advanced_oil_extraction_unlock

Bulk Orders `research_bulk_orders` NOT YET UNLOCKED

Double the weekly board. Earth starts placing bulk weekly contracts. Every Weekly Earth Order asks for **twice** the goods and pays **twice** the credits, raising the board's combined payout ceiling from **20,000 cr** to **40,000 cr**. The change applies from the next refresh, so orders already on the board keep their original size.

Threshold	Terraform Index 500,000
Tech id	bulk_orders_unlock

Cargo Expansion `research_cargo_expansion` NOT YET UNLOCKED

Buy extra inventory slots. Unlocks the **EXPAND** button on the Inventory page. Each slot costs more than the last, starting at **500 cr** for slot 37 and climbing to ~**146,000 cr** for the final slot, capped at **60** total. Total cost to fully max out: ~**666,000 cr**.

Threshold	Terraform Index 520,000
Tech id	cargo_expansion_unlock

Weather Forecasting `research_weather_forecasting` NOT YET UNLOCKED

See approaching storms three times further out. Extends new Weather Station reports and Incoming coverage from **8** to **24 world-clock hours**. Forecasts remain local to storms expected to enter powered-station coverage.

Threshold	Terraform Index 550,000
Tech id	weather_forecasting_unlock

High-Capacity Depot Handling `research_depot_handling` NOT YET UNLOCKED

Faster intake and dispatch at Drone Depots. Automatically doubles handling at every existing and newly deployed Drone Depot, stacking with Fast Feeders: a base Depot goes from **1×** to **2×**, a Medium Depot from **2×** to **4×**, and a Large Depot from **4×** to **8×**. Speeds intake and dispatch so several remote machines can share a supply route. Each receiving machine still has its own intake cooldown. Drone speed, cargo capacity and Supply Dock dispatch rates stay the same. No upgrade pack is needed.

Threshold	Terraform Index 580,000
Tech id	depot_handling_unlock

High-Pressure Fluid Transport `research_high_pressure_fluid_transport` NOT YET UNLOCKED

Triple remote gas and liquid throughput. Retrofits every completed and future Gas Pipe and Liquid Pipe network. Each matched source-side and sink-side attachment link carries up to **6,000 t/h** instead of **2,000 t/h**. Pipe length and interior branches still add no capacity; separate attachment links and independent pipe networks retain their own budgets.

Threshold	Terraform Index 600,000
Tech id	high_pressure_fluid_transport_unlock

Industrial Machinery Mk III `research_industrial_machinery_mk3` NOT YET UNLOCKED

Further improve your installed factories. Unlocks Fabricator recipes for individual Smelter and Fabricator Mk III upgrade packs. Upgrade Mk II machinery to four times the original processing speed with three times the original operating power.

Threshold	Terraform Index 625,000
Tech id	industrial_machinery_mk3_unlock

Nuclear Program `research_nuclear_program` NOT YET UNLOCKED

The Reactor. Unlocks the **Reactor** in the Shop: Fuel Rods in, cooling water through, **5,000 W** out, day and night, storm or calm. One rod lasts **72 hours** at heat **1.0**, and fuel use follows commanded heat. Keep the core in the green band; push it too hot and it overheats and shuts down safely.

Threshold	Terraform Index 650,000
Tech id	nuclear_program_unlock

TEMPERATURE

Storage Bins `research_storage_bin` UNLOCKED

Single-material stockpile bins. Unlocks the Storage Bin in the shop. Deployable single-material containers for stockpiling raw and refined materials at an outpost, one material per bin. Keep what you mine, stage what you need. (The Warehouse, researched later, is the larger multi-material depot.)

Threshold	Temperature 5
Tech id	storage_bin_unlock

Cargo Rack `research_cargo_rack_small` UNLOCKED

Modular vehicle cargo storage access. Unlocks the Small Cargo Rack and Portable Bin in the shop. A **1**-bin rack for modular vehicles, letting them haul mined ore back to base.

Threshold	Temperature 8
Tech id	cargo_rack_small_unlock

Constructor Module `research_constructor_module` UNLOCKED

Builds Planet Map blueprints. Unlocks the Constructor Module for the Pioneer in the Shop. Plan construction in Planet Map Plan Mode or with `get_component("construction_blueprint")`, load the needed kits, Gas Pipe, Liquid Pipe, or Power Line Segments, or bridges into the Pioneer's cargo, and drive out to build outposts, pipes, power lines, and machines on site.

Threshold	Temperature 10
Tech id	constructor_module_unlock

Battery Holder `research_battery_holder_small` UNLOCKED

Portable battery bay access. Unlocks the Small Battery Holder in the shop. Adds a **1**-bay portable battery mount for modular vehicles, essential for any expedition away from the Vehicle Charging Station.

Threshold	Temperature 12
Tech id	battery_holder_small_unlock

Heat Generator Mk II `research_heat_mk2_pack` UNLOCKED

Higher heat output. Unlocks the Heat Generator Mk II Upgrade Pack in the shop. Retrofits a Mk I heater to Mk II for higher output.

Threshold	Temperature 40
Tech id	heat_mk2_pack_unlock

Bioluminescent Infusion `research_luminizer` UNLOCKED

Unlocks the Bio Luminizer. Unlocks the Bio Luminizer in the shop, the coastal machine that tunes a bioluminescent fragment's glow to an order's exact target color. Coastal Bio Orders require infused fragments.

Threshold	Temperature 90
Tech id	luminizer_unlock

Medium Battery Holder `research_battery_holder_medium` UNLOCKED

Double-bay battery mount access. Unlocks the Medium Battery Holder in the shop. Two bays for portable batteries, a compact range upgrade for modular vehicles.

Threshold	Temperature 280
Tech id	battery_holder_medium_unlock

Outpost Expansion `research_outpost_expansion` UNLOCKED

Increase every outpost's building capacity by 5. Raises the soft building capacity of every current and future outpost by **5**. Nocturna Base increases from **25** to **30** buildings, and founded outposts increase from **20** to **25**. Buildings beyond the new cap still incur the normal overcrowding penalty.

Threshold	Temperature 400
Tech id	outpost_expansion_unlock

Large Battery `research_large_battery` UNLOCKED

5× a Battery's storage on one map cell. Unlocks the Large Battery in the Shop: **2,500 Wh** of base-station storage, five times the base cell. One unit replaces a cluster of Small Batteries.

Threshold	Temperature 700
Tech id	large_battery_unlock

Deep Fragment Conditioning `research_bio_conditioner` NOT YET UNLOCKED

Unlocks the Bio Conditioner. Unlocks the Bio Conditioner in the shop, the deep machine that quality-control-inspects a fragment through a 5-stage accept/reject gauntlet against a published rulebook. Deep Bio Orders require conditioned fragments. Power-only, no fluids or materials.

Threshold	Temperature 1,200
Tech id	bio_conditioner_unlock

Oil Generator `research_oil_generator` NOT YET UNLOCKED

High-density oil-burning power. Unlocks the Oil Generator in the shop. Burns oil to produce up to **700 W** from one machine (several times a Steam Turbine's peak) with full throttle control via script. Consumes **8 t/h** at full output.

Threshold	Temperature 1,500
Tech id	oil_generator_unlock

Heli-Drones `research_heli_drones` NOT YET UNLOCKED

Long-range oil-fueled aerial transport. Allows Heli Thrusters and Oil Tanks to be mounted on drones. Their Fabricator recipes are earned separately through Vestibule orders. At full throttle, heli drones fly **900 m/h** and consume **5 t/h Oil**; burn rises with throttle squared. Three oil-tank sizes set their range, and Drone Service Stations refuel them from their oil input.

Threshold	Temperature 6,000
Tech id	heli_drones_unlock

Lightning Rods `research_lightning_rod` NOT YET UNLOCKED

Catch strikes, bank the surge. Unlocks the **Lightning Rod recipe**. This storm-charged bank stores **4,000 Wh**, catches strikes within **600 m**, and feeds the grid behind batteries. Condition falls **0.05 per day**, reducing capture to zero unless a script repairs it with **1 Storm Glass**.

Threshold	Temperature 8,000
Tech id	lightning_rod_unlock

Fuel Assembler `research_fuel_assembler` NOT YET UNLOCKED

Raw Uranium + lead → Fuel Rods. Unlocks the Fuel Assembler in the Shop. This nuclear workbench presses Raw Uranium and lead casing into **Fuel Rods** and draws about **1,800 W** while running.

Threshold	Temperature 10,000
Tech id	fuel_assembler_unlock

Heat Generator Mk IV `research_heat_mk4_pack` NOT YET UNLOCKED

Nuclear-era heat tier, fuelled by rods. Unlocks the **Heat Upgrade Pack Mk IV recipe** at the Fabricator. This final tier **replaces the steam input entirely** with a **Fuel Rod** magazine fed from a connected Lead Cask and draws roughly 100x Mk I power. **A heater with no rod stops producing** instead of falling back to Mk III output.

Threshold	Temperature 42,000
Tech id	heat_mk4_pack_unlock

OXYGEN

Auto Feeders `research_auto_feeders` UNLOCKED

Material flow access. Unlocks property-preserving item flow through machine I/O ports. Inventory is a source or destination only at Nocturna Base; other endpoints must be at the machine's outpost or physically present vehicle location. Equal item ids with different properties remain distinct stacks.

Threshold	Oxygen 1
Tech id	feeder_unlock

Earth Clearance `research_earth_clearance` UNLOCKED

Advanced contract access. Unlocks three advanced contracts as atmospheric oxygen begins to rise.

Threshold	Oxygen 3
Tech id	earth_clearance

Ore Refinement `research_smelter` UNLOCKED

Refinement access. Unlocks the Smelter in the Shop. This industrial furnace reduces raw ore into refined metals and components suitable for advanced machinery.

Threshold	Oxygen 5
Tech id	smelter_unlock

Vehicle Charging Station `research_charging_station` UNLOCKED

Vehicle charging access. Unlocks the Vehicle Charging Station in the Shop. Mk I has **1 bay** and a **30 W** charging budget; later packs raise both fleet capacity and pooled solo charging speed.

Threshold	Oxygen 9
Tech id	charging_station_unlock

Battery Charger `research_battery_charger` UNLOCKED

Charge spare vehicle batteries locally. Unlocks the **Battery Charger** in the Shop. Charge loose Portable Batteries and Heavy Portable Batteries from the local grid, then exchange them with a parked Pioneer. Mk I holds **4 cells** and charges one at **30 W**.

Threshold	Oxygen 25
Tech id	tech_battery_charger

Industrial Drill `research_drill_industrial` UNLOCKED

Hardness-3 extraction access. Unlocks the Industrial Drill module in the shop. Extracts minerals up to hardness 3, which adds Titanium, Cobalt and Lead in the mid ring and Rare Earth in the outer ring. Faster dig time than the basic drill, higher power draw.

Threshold	Oxygen 30
Tech id	drill_industrial_unlock

Sport Nav `research_nav_sport` UNLOCKED

High-performance drivetrain access. Unlocks the Sport Nav module in the shop. One Sport Nav gives 2× top speed with 2.6× movement power draw compared with Basic Nav in the same rig, about 1.3× battery use per meter at full throttle. Further Sport Navs stack at a rising cost, especially on loaded long-haul routes.

Threshold	Oxygen 50
Tech id	nav_sport_unlock

Vehicle Charging Station Mk II `research_charging_station_mk2_pack` UNLOCKED

Second bay; pools to 120 W solo. Unlocks the Vehicle Charging Station Mk II Upgrade Pack in the shop. Retrofits a deployed station to Mk II, adding a second charging bay.

Threshold	Oxygen 60
Tech id	charging_station_mk2_pack_unlock

Oxygen Generator Mk II `research_oxygen_mk2_pack` UNLOCKED

Higher oxygen output. Unlocks the Oxygen Generator Mk II Upgrade Pack in the shop. Retrofits a Mk I oxygen generator to Mk II for higher output.

Threshold	Oxygen 75
Tech id	oxygen_mk2_pack_unlock

Warehouse `research_warehouse` UNLOCKED

Bulk-storage building. Unlocks the Warehouse in the shop. A **10,000-unit** multi-material depot: **5** material-locked slots of **2,000** each. Use it as a buffer for drone deliveries and large vehicle hauls.

Threshold	Oxygen 150
Tech id	warehouse_unlock

Verified Contractor `research_verified_contractor` UNLOCKED

New contract batch access. Your terraforming output has passed Earth's vetting threshold. A second batch of contracts (denser puzzles, higher rewards) routes to your terminal. Adds new tiers to the Contractor Reputation ladder.

Threshold	Oxygen 300
Tech id	verified_contractor_unlock

Volcanic Forge-Casting `research_bio_caster` UNLOCKED

Unlocks the Bio Caster. Unlocks the Bio Caster in the shop, the volcanic machine that heat-casts a fragment with fabricated materials. Volcanic Bio Orders require forged fragments.

Threshold	Oxygen 350
Tech id	bio_caster_unlock

Liquid Tank `research_liquid_tank` UNLOCKED

Buffer any one liquid. Unlocks the Liquid Tank in the Shop, a passive **100**-ton buffer that latches to the first liquid it receives (water, oil, or any biome essence) and holds only that fluid until drained.

Threshold	Oxygen 400
Tech id	liquid_tank_unlock

Waste Processing `research_garbage_disposal` NOT YET UNLOCKED

Programmable destruction for unwanted items and fluids. Unlocks the **Waste Processor Kit recipe**. The machine permanently destroys one selected stream: any item through its feeder input, any liquid through `liquid_in`, or any gas through `gas_in`. This provides an explicit overflow path for tar, surplus water, and unwanted gases without recovering power or materials. Fabricate the kit at a Fabricator; its script selects and enables the active mode.

Threshold	Oxygen 1,400
Tech id	garbage_disposal_unlock

Petroleum Survey `research_petroleum_survey` NOT YET UNLOCKED

Oil wells visible to Deep Sonar. Unlocks survey of subsurface oil deposits and the **Oil Pump recipe**. **Sonar with deep-survey capability required**: use Deep Sonar or an equivalent higher tier to discover oil wells. Fabricate an Oil Pump at a Fabricator, deploy it on a surveyed well, and pipe oil to base for power generation and refining.

Threshold	Oxygen 1,500
Tech id	petroleum_survey_unlock

Large Battery Holder `research_battery_holder_large` NOT YET UNLOCKED

Triple-bay battery mount access. Unlocks the Large Battery Holder in the shop. Three bays, enough capacity to sustain Heavy Drill power draw on long edge-ring expeditions.

Threshold	Oxygen 1,800
Tech id	battery_holder_large_unlock

Large Cargo Rack `research_cargo_rack_large` NOT YET UNLOCKED

Triple-bin cargo mount access. Unlocks the Large Cargo Rack in the shop. Three bins for maximum haul capacity on large mining runs.

Threshold	Oxygen 2,000
Tech id	cargo_rack_large_unlock

Heavy Drill `research_drill_heavy` NOT YET UNLOCKED

Hardness-4 extraction access. Unlocks the Heavy Drill module in the shop. Extracts minerals up to hardness 4, so it cuts every mineral on the planet and is the only drill that can take Neutronium, required for Mk III fabrication.

Threshold	Oxygen 2,500
Tech id	drill_heavy_unlock

Shielded Logistics `research_shielded_logistics` NOT YET UNLOCKED

Lead Casks + radioactive-cargo handling. Radioactive cargo needs a shielded lane: **Lead Casks** provide stationary storage for Raw Uranium and Fuel Rods, and **Shield Plating** lets a drone collect Raw Uranium without gaining exposure. Unlocks both items' availability; the fabrication recipes come through contractor orders.

Threshold	Oxygen 3,000
Tech id	shielded_logistics_unlock

Shielded Depot Operations `research_shielded_depot_ops` NOT YET UNLOCKED

Plated drones carry radioactive cargo between outposts. Plated drones can load radioactive cargo at any Drone Depot, straight from that outpost's Lead Cask, and unload it into the Lead Cask where they land. Depots never store Raw Uranium or Fuel Rods themselves. Until this is researched, drones can't load or unload radioactive cargo at depots at all.

Threshold	Oxygen 8,000
Tech id	shielded_depot_ops_unlock

Oxygen Generator Mk IV `research_oxygen_mk4_pack` NOT YET UNLOCKED

Nuclear-era oxygen tier, fuelled by rods. Unlocks the **Oxygen Upgrade Pack Mk IV recipe** at the Fabricator. This final tier **replaces the water input entirely** with a **Fuel Rod** magazine fed from a connected Lead Cask and draws roughly 100x Mk I power. **A generator with no rod stops producing** instead of falling back to Mk III output.

Threshold	Oxygen 40,000
Tech id	oxygen_mk4_pack_unlock

PRESSURE

Mining Operations `research_mining` UNLOCKED

Planet Map and Vehicles tab access. Unlocks the Planet Map and its Vehicles tab for managing your ground vehicles. Prospect the surface before committing to a vehicle.

Threshold	Pressure 0.1
Tech id	mining_unlock

Rover Chassis `research_rover` UNLOCKED

First expedition vehicle access. Unlocks the Rover in the Shop. Its integrated **100 Wh** battery and **10-unit** hold are fixed; Nav, Sonar, and Drill use three dedicated module slots.

Threshold	Pressure 0.11
Tech id	rover_unlock

Nav Module `research_nav_module` UNLOCKED

Drive hardware access. Unlocks Nav modules in the shop. Gives vehicles the ability to navigate and drive from their own script. Stop, completion, or error cancels script-owned movement.

Threshold	Pressure 0.12
Tech id	nav_module_unlock

Sonar Module `research_sonar_module` UNLOCKED

Survey hardware access. Unlocks Basic Sonar modules in the shop. Sonar performs point sweeps from a vehicle's current position, discovering nearby map sites and recording them in the Journal.

Threshold	Pressure 0.15
Tech id	sonar_module_unlock

Drill Module `research_deep_extraction` UNLOCKED

Drilling hardware access. Unlocks Drill modules in the shop. Extract raw ore from surveyed mining sites.

Threshold	Pressure 0.2
Tech id	deep_extraction_unlock

Pressure Generator Mk II `research_pressure_mk2_pack` UNLOCKED

Higher pressure output. Unlocks the Pressure Generator Mk II Upgrade Pack in the shop. Retrofits a Mk I pressure generator to Mk II: 25× Mk I output and 5× Mk I power draw.

Threshold	Pressure 1.2
Tech id	pressure_mk2_pack_unlock

Extended Sonar `research_sonar_wide` UNLOCKED

Mid-ring survey hardware access. Unlocks the Extended Sonar module in the shop. Extends scan range to **180 m** and detects minerals up to hardness 3, including Titanium, Cobalt and Rare Earth, out in the mid and outer rings.

Threshold	Pressure 1.8
Tech id	sonar_wide_unlock

Geological Survey `research_geological_survey` UNLOCKED

Sonar detects thermal vents. Extends the Sonar Module to detect thermal vents alongside mining sites. Surveyed vents are added to the Journal with their own readings, so you can find where steam can be tapped.

Threshold	Pressure 2
Tech id	geological_survey_unlock

Thermal Cap `research_thermal_cap` UNLOCKED

Capture steam from vents. Unlocks the Thermal Cap and its **Thermal Cap Kit** recipe. Fabricate the kit at a Fabricator, then build the Cap on a surveyed thermal vent. It captures Steam while the vent is active and releases it to connected destinations over a completed Gas Pipe route. At **100%** the chamber blows away all stored Steam, so its script must release, route, or relieve the pressure.

Threshold	Pressure 2.5
Tech id	thermal_cap_unlock

Gas Tank `research_steam_tank` UNLOCKED

Buffer one gas between machines. Unlocks the Gas Tank in the Shop, a passive **5000**-ton buffer that latches to the first gas it receives (steam, ammonia, swamp gas, or any gas) and holds only that gas until drained.

Threshold	Pressure 2.75
Tech id	steam_tank_unlock

Steam Turbine `research_steam_turbine` UNLOCKED

Steam into grid power. Unlocks the Steam Turbine in the Shop. At full throttle it consumes **90 t/h Steam** and produces **108 W**; a script controls the throttle.

Threshold	Pressure 3
Tech id	steam_turbine_unlock

Medium Cargo Rack `research_cargo_rack_medium` UNLOCKED

Double-bin cargo mount access. Unlocks the Medium Cargo Rack and Heavy Portable Bin in the shop. Two bins for larger hauls per run, with room for a second mineral.

Threshold	Pressure 4.5
Tech id	cargo_rack_medium_unlock

Hydrology Survey `research_hydrology_survey` UNLOCKED

Water wells visible to sonar. Unlocks survey of subsurface water deposits and the **Water Pump recipe**. Sonar scans now reveal water wells across the map. Wells produce continuous water at their yield tier (1× / 2× / 3×). Fabricate a Water Pump at a Fabricator, deploy it on a surveyed well, and pipe the output to base.

Threshold	Pressure 18
Tech id	hydrology_survey_unlock

Deep Sonar `research_sonar_deep` NOT YET UNLOCKED

Edge-ring survey hardware access. Unlocks the Deep Sonar module in the shop. Extends scan range to **280 m** and detects minerals up to hardness 4, which adds Neutronium at the edge ring. Also reveals extra survey detail for thermal vents and exotic deposits, such as their cycle timing. Deep Sonar can reveal oil wells after **Petroleum Survey**; oil discovery requires deep-survey capability, also provided by a higher sonar tier.

Threshold	Pressure 60
Tech id	sonar_deep_unlock

Gene Sequencing `research_dna_sequencer` NOT YET UNLOCKED

Unlocks the DNA Sequencer. Unlocks the DNA Sequencer in the shop, the geothermal machine that reads and rewrites fragment DNA. Geothermal Bio Orders require fragments engineered to carry specific genes.

Threshold	Pressure 120
Tech id	dna_sequencer_unlock

Prime Contractor `research_prime_contractor` NOT YET UNLOCKED

Top contract batch access. Unlocks three difficult contracts in the atmosphere-era program. Completing them contributes to Contractor Reputation; the final contract batch arrives with Smart Contractor.

Threshold	Pressure 130
Tech id	prime_contractor_unlock

Bulk Logistics II `research_dispatch_mk2` NOT YET UNLOCKED

Quadruple Supply Dock throughput. Quadruples each Supply Dock's per-pulse emission count from **1** to **4** units, so the effective rate jumps from **25 units/h** to **100 units/h**. Pulse cadence is unchanged; only the packet size scales.

Threshold	Pressure 150
Tech id	dispatch_mk2_unlock

Pressure Generator Mk IV `research_pressure_mk4_pack` NOT YET UNLOCKED

Nuclear-era pressure tier, fuelled by rods. Unlocks the **Pressure Upgrade Pack Mk IV recipe** at the Fabricator. This final tier **replaces the water input entirely** with a **Fuel Rod** magazine fed from a connected Lead Cask and draws roughly 100x Mk I power. **A generator with no rod stops producing** instead of falling back to Mk III output.

Threshold	Pressure 2,000
Tech id	pressure_mk4_pack_unlock

BIOMASS

Seed Maker `research_seed_maker` NOT YET UNLOCKED

Combine life-forms into seeds. Opens the **Plants** tier and unlocks the **Seed Maker Kit recipe**. The machine consumes **1 t each of three different harvested life forms** to test for a viable seed. Most combinations come out as sludge and produce nothing; the recipes that work are unique to this planet. Plant the seeds you find into your Harvester field.

Threshold	Biomass 500
Tech id	seed_maker_unlock

Plant Terraformer `research_plant_terraformer` NOT YET UNLOCKED

Turn harvested Forage into Plants progress. Unlocks the **Plant Terraformer Kit recipe** at **2,000 t Biomass**. It is the only machine that creates permanent Plants km². Feed it harvested Forage, then satisfy the cumulative phase ladder with Water, Salt, Fertilizer, and Growth Accelerant. Fabricate its kit, deploy it at an outpost, connect its supplies, and enable it from its script. Build as many as your harvest can feed.

Threshold	Biomass 2,000
Tech id	plant_terraformer_unlock

Heavy Portable Battery `research_heavy_portable_battery` NOT YET UNLOCKED

High-capacity vehicle cell access. Unlocks the Heavy Portable Battery in the shop. A **100 Wh** portable cell, double a standard battery, that drops into any vehicle holder bay to sustain long-haul fleets far from the Vehicle Charging Station.

Threshold	Biomass 3,000
Tech id	heavy_portable_battery_unlock

Battery Charger Mk II Upgrade Pack `research_battery_charger_mk2` NOT YET UNLOCKED

More battery storage and two charging bays. Unlocks the **Battery Charger Mk II Upgrade Pack** in the Shop. Upgrade an existing charger to **8 shared slots**, **2 charging bays**, and **60 W** total while preserving stored batteries and queued work.

Threshold	Biomass 5,500
Tech id	tech_battery_charger_mk2

Smart Contractor `research_smart_contractor` NOT YET UNLOCKED

Final contract batch access. Your Biomass recovery has opened Earth's final contractor channel. Unlocks Beat the System, Core Sample, and Lattice, the three hardest programming contracts, and completes the Contractor Reputation ladder.

Threshold	Biomass 20,000
Tech id	smart_contractor_unlock

Large Warehouse `research_high_bay_warehousing` NOT YET UNLOCKED

Fifteen kinds of material in one building. Unlocks the **Large Warehouse** in the shop. It holds **30,000 units** across **15** slots of **2,000**, and each slot sticks to one material. That is enough room for a wide biological and industrial supply chain to keep every material apart, without replacing Drone Depot handoffs or scripted transfers.

Threshold	Biomass 30,000
Tech id	high_bay_warehousing_unlock

Biomass Mixer Mk II `research_biomass_mixer_mk2_pack` NOT YET UNLOCKED

87.5% more biomass per ton of essence. Unlocks the Biomass Mixer Mk II Upgrade Pack in the shop. One pack retrofits one deployed Mk I Mixer for **4.5× output** on only **2.4× essence consumption**, with **5× power draw**. That is **87.5%** more biomass per ton of essence, which matters because Liquifier intake is fixed and rare life forms regrow slowly. There is no Mk III.

Threshold	Biomass 50,000
Tech id	biomass_mixer_mk2_pack_unlock

Bulk Logistics III `research_dispatch_mk3` NOT YET UNLOCKED

16× Supply Dock throughput. Bumps each Supply Dock's per-pulse emission to **16** units, an effective **400 units/h**. Pulse cadence is unchanged; only the packet size scales.

Threshold	Biomass 130,000
Tech id	dispatch_mk3_unlock

PLANTS

Sprinkler `research_sprinkler` NOT YET UNLOCKED

Water provider for the grid. Unlocks the **Sprinkler Kit recipe**. Fabricate the kit at a Fabricator, then deploy it in a Harvester field to water the four orthogonally adjacent cells (directly above, below, left, and right) while supplied. Upgrade packs raise its multiplier on the Forage output of covered crops that need water.

Threshold	Plants 100,000
Tech id	sprinkler_unlock

Dispenser `research_dispenser` NOT YET UNLOCKED

Salt provider for the grid. Unlocks the **Dispenser Kit recipe**. Fabricate the kit at a Fabricator, then deploy it in a Harvester field to salt the four orthogonally adjacent cells (directly above, below, left, and right) while supplied. Salt is a Water Pump byproduct.

Threshold	Plants 300,000
Tech id	dispenser_unlock

Grow Lamp `research_grow_lamp` NOT YET UNLOCKED

Light provider for the grid. Unlocks the **Grow Lamp Kit recipe**. Fabricate the kit at a Fabricator, then deploy it in a Harvester field to light the four orthogonally adjacent cells (directly above, below, left, and right). Upgrade packs raise its multiplier on the Forage output of covered crops that need light.

Threshold	Plants 500,000
Tech id	grow_lamp_unlock

Field Automation `research_field_automation` NOT YET UNLOCKED

Automate harvest, planting, and treatment. Unlocks the **Crop Automator Kit** in the Shop. One automator can queue harvest, plant, and treatment jobs across up to 24 other cells in a centered 5 by 5 area. It executes one at a time in FIFO order by default (first in, first out, so the oldest job runs first), while its script can reorder unfinished work.

Threshold	Plants 620,000
Tech id	field_automation_unlock

Large Liquid Tank `research_reservoir_engineering` NOT YET UNLOCKED

Holds 10× what a Liquid Tank does. Unlocks the **Large Liquid Tank** in the shop. It works just like a Liquid Tank, taking whatever liquid reaches it first and holding only that until it runs dry, but it holds **1,000 t** instead of 100 t. Use it where the liquid feeding plant, essence, or Habitat production has to ride out a long gap.

Threshold	Plants 900,000
Tech id	reservoir_engineering_unlock

Steam Condensation `research_steam_condenser` NOT YET UNLOCKED

Turn vent steam into clean water. Unlocks the **Steam Condenser** in the shop at **1,000,000 km² Plants**. It consumes up to **250 t/h Steam** and produces the same mass of clean Water, drawing **150 W** at full throttle. Use Gas Tanks to bridge dormant vent phases and decide how much steam becomes water instead of turbine power.

Threshold	Plants 1,000,000
Tech id	steam_condenser_unlock

Plant Terraformer Mk II `research_plant_terraformer_mk2` NOT YET UNLOCKED

Fertilizer and Accelerant injectors. Unlocks the **Plant Terraformer Mk II Upgrade Pack recipe** at the Fabricator. A Mk I has no Fertilizer or Growth Accelerant injector, so it converts only up to the **Fields** threshold and then stops. Mk II adds both injectors, raises throughput from **400** to **2,200 Forage/h**, raises input feeder handling from **16** to **80 items per step**, and raises enabled draw from **180 W** to **900 W**. Each machine needs its own pack.

Threshold	Plants 1,250,000
Tech id	plant_terraformer_mk2_unlock

Wildlife `research_wildlife` NOT YET UNLOCKED

Revive and house the first fauna. Opens the **Wildlife** tier and makes the **Habitat** and **Feed Maker** available in the Shop. A Habitat revives cataloged creatures and houses a breeding colony, while the Feed Maker crafts species feed from harvested **Forage** plus life forms gathered across the biomes. Keep each colony's stored feed, gas, and liquid inside its required ranges and it breeds toward its ceiling. Gated on the Plants pillar reaching its Fields phase because every feed recipe uses Forage; harvested biome life forms provide the remaining ingredients.

Threshold	Plants 2,250,000
Tech id	wildlife_unlock

Advanced Biopolymers `research_advanced_biopolymers` NOT YET UNLOCKED

Make structural composite. Unlocks the Fabricator recipe for **Reinforced Biopolymer**. The composite turns Plastic, Forage, and Water from your established production chains into a structural material used by large Earth orders and advanced biological chemistry.

Threshold	Plants 4,000,000
Tech id	advanced_biopolymers_unlock

WILDLIFE

Exotic Fluids `research_exotic_husbandry` NOT YET UNLOCKED

Source exotic gases and liquids. Unlocks exotic deposit survey, the **Exotic Gas Cap Kit** and **Exotic Spring Tap Kit** recipes, and the **Refiner** in the Shop. Common and uncommon exotic deposits can now appear on sonar; rare deposits wait for **Rare Exotics**. Capture raw exotic gas or liquid, then refine it into the fluid a Habitat colony's gas or liquid band requires. Fabricate the caps and taps at a Fabricator.

Threshold	Wildlife 1,000
Tech id	exotic_husbandry_unlock

Enrichment Chemistry `research_habitat_development` NOT YET UNLOCKED

Synthesize a late biological export. Unlocks the Fabricator recipe for **Bio Enrichment Compound** at **2,000 Wildlife**. This advanced biological material is used in late Earth exports; Habitat adaptations are purchased with Insight and do not consume Bio Enrichment Compound.

Threshold	Wildlife 2,000
Tech id	habitat_development_unlock

Feed Maker Mk II `research_feed_maker_mk2` NOT YET UNLOCKED

Faster feed production with higher power demand. Unlocks the **Feed Maker Mk II Upgrade Pack recipe in the Fabricator** at **250,000 Wildlife**. Craft a pack, then use **Upgrade in Inventory** to apply it to one existing Feed Maker at any outpost. Mk II provides **1.5× crafting speed and 1.5× Auto Feeder speed into its input** with **2× operating power**. Each machine needs its own pack and keeps its existing building slot. Ingredient quantities and feed produced per batch stay the same.

Threshold	Wildlife 250,000
Tech id	feed_maker_mk2_unlock

Rare Exotics `research_deep_exotics` NOT YET UNLOCKED

Refine the rarest exotics. Unlocks sonar discovery, extraction, and refining for the **rarest** exotic gases and liquids used by late-stage Habitat requirements.

Threshold	Wildlife 500,000
Tech id	deep_exotics_unlock

Habitat Engineering Mk II `research_habitat_pack_mk2` NOT YET UNLOCKED

Double an enclosure's carrying capacity. Unlocks the **Habitat Mk II Upgrade Pack recipe** at the Fabricator at **600,000 Wildlife**, after Rare Exotics. Each pack upgrades one Habitat from Mk I to Mk II, doubling its carrying capacity while raising its power draw to **144-168 W**, depending on population. Mk II affects capacity only; adaptations bought with Insight control breeding pace and biological costs.

Threshold	Wildlife 600,000
Tech id	habitat_pack_mk2

Bounds

Returned by: planet.get_bounds()

Import `Bounds` with `from __builtins__ import Bounds`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`Bounds(min_x: float, max_x: float, min_y: float, max_y: float) → Bounds`

Create a local `Bounds` value for your script. Creating this value does not change the world. Requires `min_x <= max_x`. Requires `min_y <= max_y`.

PARAMETERS

Name	Type	Description
`min_x`	`float`	Minimum X coordinate (meters). Must be a finite number.
`max_x`	`float`	Maximum X coordinate (meters). Must be a finite number.
`min_y`	`float`	Minimum Y coordinate (meters). Must be a finite number.
`max_y`	`float`	Maximum Y coordinate (meters). Must be a finite number.

RETURNS

`Bounds`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`ValueError`	A numeric value is outside the permitted range, or the bounds are reversed.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

.min_x: float

Minimum X coordinate (meters).

RETURNS

`float`

.max_x: float

Maximum X coordinate (meters).

RETURNS

`float`

.min_y: float

Minimum Y coordinate (meters).

RETURNS

`float`

.max_y: float

Maximum Y coordinate (meters).

RETURNS

`float`

Component

Returned by: get_component / self

Get `Component` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Programmatic identifier of this component.

RETURNS

`str`

`.type_id: str`

Which kind of component this is, as the stable type id (`"bio_lab"`, `"rover"`, `"drone_small"`). Every component of the same kind shares it, so a library function can branch on what it was handed. For a building at an outpost, `outpost.buildings(self.type_id)` lists buildings of that type there; the query excludes mobile units and sensors. Use `.id` for which individual one this is.

RETURNS

`str`

PointOfInterest

Returned by: nocturna.points_of_interest()

Get `PointOfInterest` from the APIs listed here. It has no script constructor.

PROPERTIES

`.x: int`

Whole-number X coordinate (meters from base) of this contact. Pass straight to a Rover/Pioneer `self.nav.set_target(p.x, p.y)` or a drone `self.go_to(p.x, p.y)`.

RETURNS
`int`

`.y: int`

Whole-number Y coordinate (meters from base) of this contact.

RETURNS
`int`

`.scanned: bool`

`True` once you've resolved this contact: a Rover/Pioneer sonar **survey** for a productive non-biomass site, a drone **bio-scan** at a biomass site, or a sonar **scan** for an inert contact. `False` is your work list. Filter `if not p.scanned:` for the sites left to visit, and pair it with the sweep's `blocked` contacts: one your scanner cannot identify stays `False` after a sweep that succeeded, so a plain retry loop returns to it.

RETURNS
`bool`

`.kind: str`

The contact's type: `"unknown"` **until** `.scanned` **is** `True`. Once resolved it reads `"mineral"`, `"biomass"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, or `"inert"`. A sonar `scan()` only places the contact on the map; a productive site stays `"unknown"` until you drive to it and `survey()` it, and a biomass site until a drone bio-scans it. Only inert contacts resolve from the scan alone.

RETURNS

`str`

POSSIBLE VALUES

`"unknown"`, `"mineral"`, `"biomass"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

Position

Returned by: self.nav.get_position()

Import `Position` with `from __builtins__ import Position`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`Position(x: float, y: float) → Position`

Create a local `Position` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`x`	`float`	X coordinate in meters from base for this position snapshot. Must be a finite number.
`y`	`float`	Y coordinate in meters from base for this position snapshot. Must be a finite number.

RETURNS

`Position`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

`.x: float`

X coordinate in meters from base for this position snapshot.

RETURNS

`float`

`.y: float`

Y coordinate in meters from base for this position snapshot.

RETURNS

`float`

METHODS

`.__iter__() → Iterator[float]`

Iterate over `x`, then `y`, so this position can be unpacked or passed to `list()`.

RETURNS

`Iterator[float]`

ScriptCommand

Returned by: `self.peek_command()` / `self.next_command().command` after `status == "ok"`

Get `ScriptCommand` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique command id assigned when the command entered this script's queue.

RETURNS
`str`

`.name: str`

Command name, such as `"return_base"`. Your script decides what each name means.

RETURNS
`str`

`.args: dict[str, JsonValue]`

JSON-safe argument `dict` sent with the command. Use `.get(key, default)` for optional arguments.

RETURNS
`dict[str, JsonValue]`

`.source: str`

Where the command came from: `"editor"`, `"script"`, `"signal"`, or `"system"`.

RETURNS

`str`

POSSIBLE VALUES

`"editor"`, `"script"`, `"signal"`, `"system"`

`.created_at: float`

Creation bookkeeping value: a supplied real-world timestamp in milliseconds, otherwise the enqueueing simulation tick, or **0** when neither was supplied. For gameplay timing, use `.tick`.

RETURNS

`float`

`.tick: int | None`

Game tick when the command was queued, or `None` if not available.

RETURNS

`int | None`

ScriptStatusReport

Returned by: get_status_report() / self.get_status_report()

Get `ScriptStatusReport` from the APIs listed here. It has no script constructor.

PROPERTIES

`.message: str`

The player-authored plain-text explanation.

RETURNS
`str`

`.level: str`

The player-selected `info`, `warn`, or `error` presentation level.

RETURNS
`str`

POSSIBLE VALUES
`"info"`, `"warn"`, `"error"`

`.active: bool`

`True` while the reporting run is running or waiting. `False` while paused, halted by the debugger, completed, or failed.

RETURNS
`bool`

`.run_id: str`

The exact run identity. Changes when a new execution starts.

RETURNS

`str`

Outpost

Returned by: get_component(outpost_id) / get_component_by_name(outpost_name)

Get `Outpost` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Immutable instance id (e.g. `"outpost_home"`, `"outpost_1"`). Stable for the life of the save: use this in scripts that need to outlive renames.

RETURNS

`str`

METHODS

`.name() → str`

Display name. Defaults to `"Nocturna Base"` for the home outpost or a generated `"Outpost N"` name for founded outposts; freely renameable from the Computer System tab. Mutable: prefer `id` when persistence matters.

RETURNS

`str`

`.coords() → list[int]`

World coordinates of the outpost anchor as `[x, y]`. Home outpost sits at `[0, 0]`; founded outposts carry the position chosen in Plan mode.

RETURNS

`list[int]`

`.buildings_used() → int`

Number of buildings deployed at this outpost. Sensors, mobile units, structural hubs, and POI extraction don't count.

RETURNS

`int`

`.buildings_capacity() → int`

Soft building threshold at this outpost. Each counted building above it reduces productive and service throughput. Nocturna Base has a few extra starter slots; founded outposts use the standard threshold.

RETURNS

`int`

`.is_full() → bool`

Returns `True` when this outpost has reached or exceeded its soft building threshold. The threshold itself does not block ordinary deployment.

RETURNS

`bool`

`.is_home() → bool`

Returns `True` when this is the home outpost (default name `"Nocturna Base"`).

RETURNS

`bool`

`.buildings(type_id?: str) → list[BuildingRef]`

Fresh list of `BuildingRef` snapshots: one entry per building deployed here. Pass a `type_id` (e.g. `"storage_bin"`) to filter, or omit for every building. Each ref carries `.id` / `.name` / `.type_id` / `.outpost` / `.powered` / `.position`: for type-specific live reads (`count()`, `material()`, `recipe`, etc.) call `get_component(ref.id)` to reach the full component API.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional. Machine type id to filter by (e.g. `"storage_bin"`). Omit to return every building at this outpost.

RETURNS

`list[BuildingRef]`

`.harvesting_machines(type_id?: str) → list[HarvestingMachineRef]`

Fresh list of `HarvestingMachineRef` snapshots for fixed machines deployed on this outpost's Harvesting field. Currently only the home outpost, Nocturna Base, has that field, so founded outposts return an empty list. Includes Grow Lamps, Sprinklers, Dispensers, and Crop Automators, which occupy field cells but do not use building capacity or appear in `buildings()`. It does not include the mobile Harvester, crops, loose items, Water Wells, or other POI extractors. Pass a `type_id` to filter, or omit it for all four fixed machine types. Use `get_component(ref.id)` for type-specific live reads.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional fixed Harvesting-field machine type id: `"grow_lamp"`, `"sprinkler"`, `"dispenser"`, or `"crop_automator"`. Omit it to return every fixed machine deployed on the field.

RETURNS

`list[HarvestingMachineRef]`

OutpostNetwork

Returned by: get_component("outpost_network")

Get `OutpostNetwork` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `OutpostRef`

METHODS

`.outposts() → list[OutpostRef]`

All owned outposts as read-only `OutpostRef` snapshots, including home. Use `.id` when passing an outpost to another API; call `outposts()` again when you need fresh counts/names.

RETURNS

`list[OutpostRef]`

`.home() → OutpostRef`

The home outpost as an `OutpostRef` snapshot.

RETURNS

`OutpostRef`

`.nearest(x: float, y: float) → OutpostRef`

Nearest owned outpost to the given world coordinate as an `OutpostRef` snapshot.

PARAMETERS

Name	Type	Description
`x`	`float`	World X coordinate
`y`	`float`	World Y coordinate

RETURNS

`OutpostRef`

OutpostRef

Returned by: outpost_network.outposts() / outpost_network.home() / outpost_network.nearest()

Get `OutpostRef` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `BuildingRef`
- `HarvestingMachineRef`

PROPERTIES

`.id: str`

Immutable outpost id, e.g. `"outpost_home"` or `"outpost_1"`. Use it with `get_component(id)` or APIs that need a stable target.

RETURNS

`str`

`.name: str`

Display name at the time this ref was returned.

RETURNS

`str`

`.x: int`

World X coordinate of the outpost footprint's top-left anchor, in meters from base. For an at-building action, route to that building's `BuildingRef.position` instead.

RETURNS

`int`

`.y: int`

World Y coordinate of the outpost footprint's top-left anchor, in meters from base. For an at-building action, route to that building's `BuildingRef.position` instead.

RETURNS
`int`

`.is_home: bool`

`True` for the home outpost.

RETURNS
`bool`

`.biome: str`

Biome this outpost sits in: one of `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`. Derived from the outpost's location. A building reaches its own biome via `self.outpost.biome`.

RETURNS
`str`

POSSIBLE VALUES
`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.buildings_used: int`

Buildings deployed at this outpost when this ref was returned.

RETURNS
`int`

`.buildings_capacity: int`

Soft building threshold at this outpost when this ref was returned. Buildings above it reduce productive and service throughput.

RETURNS

`int`

`.is_full: bool`

`True` when this outpost had reached or exceeded its soft building threshold at the time this ref was returned. The threshold itself does not block ordinary deployment.

RETURNS

`bool`

METHODS

`.position() → Position`

Top-left anchor of the outpost footprint as a `Position` snapshot. This identifies the outpost, not a particular building's docking point.

RETURNS

`Position`

`.coords() → list[int]`

Top-left anchor of the outpost footprint as `[x, y]`. Use `buildings()` and `BuildingRef.position` when routing to a particular service building.

RETURNS

`list[int]`

`.buildings(type_id?: str) → list[BuildingRef]`

Fresh list of `BuildingRef` snapshots: one entry per building deployed here. Pass a `type_id` (e.g. `"storage_bin"`) to filter, or omit for every building. Each ref carries `.id` / `.name` / `.type_id` / `.outpost` / `.powered` / `.position`: for type-specific live reads (`count()`, `material()`, `recipe`, etc.) call `get_component(ref.id)` to reach the full component API.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional. Machine type id to filter by (e.g. `"storage_bin"`). Omit to return every building at this outpost.

RETURNS

`list[BuildingRef]`

`.harvesting_machines(type_id?: str) → list[HarvestingMachineRef]`

Fresh list of `HarvestingMachineRef` snapshots for fixed machines deployed on this outpost's Harvesting field. Currently only the home outpost, Nocturna Base, has that field, so founded outposts return an empty list. Includes Grow Lamps, Sprinklers, Dispensers, and Crop Automators, which occupy field cells but do not use building capacity or appear in `buildings()`. It does not include the mobile Harvester, crops, loose items, Water Wells, or other POI extractors. Pass a `type_id` to filter, or omit it for all four fixed machine types. Use `get_component(ref.id)` for type-specific live reads.

PARAMETERS

Name	Type	Description
`type_id`	`str`	Optional fixed Harvesting-field machine type id: `"grow_lamp"`, `"sprinkler"`, `"dispenser"`, or `"crop_automator"`. Omit it to return every fixed machine deployed on the field.

RETURNS

`list[HarvestingMachineRef]`

BuildingRef

Returned by: outpost.buildings() / outpost_network.outposts()[i].buildings()

Get `BuildingRef` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Stable component id. Use with `get_component(id)` to reach the full machine API (counts, recipes, materials, etc.).

RETURNS

`str`

`.name: str`

Display name when this ref was returned (e.g. `"Iron Ore Bin"`). For fresh names or live machine state, call `get_component(id)`.

RETURNS

`str`

`.type_id: str`

Machine type id, e.g. `"storage_bin"`, `"smelter"`, `"solar_generator"`. Stable forever; use this for branching logic.

RETURNS

`str`

`.outpost_id: str`

Id of the outpost this building is deployed at.

RETURNS

`str`

`.outpost: OutpostRef | None`

Snapshot back-reference to the owning outpost as an `OutpostRef` (or `None` if the link can't be resolved).

RETURNS

`OutpostRef | None`

`.powered: bool`

`True` when the building's power toggle was on as this ref was returned. For live power/production reads, call `get_component(id)`.

RETURNS

`bool`

`.position: list[float]`

World coordinates as `[x, y]`. For most buildings this is the deploy anchor; doesn't move.

RETURNS

`list[float]`

HarvestingMachineRef

Returned by: outpost.harvesting_machines() / outpost_network.home().harvesting_machines()

Get `HarvestingMachineRef` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable component id. Use with `get_component(id)` to reach the full Grow Lamp, Sprinkler, Dispenser, or Crop Automator API.

RETURNS
`str`

.name: str

Display name when this ref was returned. Call `get_component(id)` for current machine state.

RETURNS
`str`

.type_id: str

Fixed Harvesting-field machine type id: `"grow_lamp"`, `"sprinkler"`, `"dispenser"`, or `"crop_automator"`.

RETURNS
`str`

POSSIBLE VALUES
`"grow_lamp"`,
`"sprinkler"`,
`"dispenser"`,
`"crop_automator"`

`.powered: bool`

`True` when the machine's power toggle was on as this ref was returned. Call `get_component(id)` for live status and supply reads.

RETURNS

`bool`

`.position: str`

Harvesting-field sector occupied by the fixed machine, such as `"B22"`.

RETURNS

`str`

Planet

Returned by: transmitter.list_planets()

Get `Planet` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Planet id: pass this to `transmitter.connect()`.

RETURNS
`str`

`.name: str`

Display name.

RETURNS
`str`

`.description: str`

Short planet description.

RETURNS
`str`

ScanResult

Returned by: scanner.scan(), harvester.collect()

Get `ScanResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Stable outcome code. Scanner results use `"ok"` / `"empty"`; Harvester results can also report `"holding"`, `"overheated"`, `"moving"` (in transit), `"busy"` (occupied by another action), or `"collecting"`.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"empty"`, `"holding"`, `"overheated"`, `"moving"`, `"busy"`, `"collecting"`

`.message: str`

Player-readable explanation of the scan or collection outcome.

RETURNS

`str`

`.id: str`

Item ID at the scanned sector (empty string if nothing).

RETURNS

`str`

`.name: str`

Item display name.

RETURNS

`str`

`.value: float`

Credit value of the item.

RETURNS

`float`

Site

ABSTRACT

Returned by: `SonarModule.scan().sites` / `SonarModule.survey().site` / journal and site-bound machine queries

Get `Site` from the APIs listed here. It has no script constructor.

CONCRETE SUBTYPES

- `ExoticDeposit`
- `GeologicalAnomaly`
- `MiningSite`
- `OilWell`
- `ThermalVent`
- `WaterWell`

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS

`str`

`.name: str`

Human-readable display name.

RETURNS

`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS

`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

ExoticDeposit

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "exotic"` (e.g. `exotic_gas_cap.deposit()`, `exotic_spring_tap.deposit()`, sonar / journal queries)

Get `ExoticDeposit` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

`.fluid() → str | None`

Fluid id this deposit emits, e.g. `"ammonia"` (common, usable direct) or `"raw_chlorine"` (rare, needs the Refiner). `None` until `surveyed`.

RETURNS

`str | None`

POSSIBLE VALUES

`"ammonia"`, `"swamp_gas"`, `"raw_sulfur_gas"`, `"raw_chlorine"`, `"brine"`, `"raw_cryofluid"`, `"raw_quicksilver"`

`.medium() → str | None`

`"gas"` (tap with an **Exotic Gas Cap**) or `"liquid"` (tap with an **Exotic Spring Tap**). `None` until `surveyed`.

RETURNS

`str | None`

POSSIBLE VALUES

`"gas"`, `"liquid"`

`.survey_level() → str | None`

Highest survey tier achieved on this deposit: `"basic"` / `"wide"` / `"deep"` / `"seismic"`, or `None` if not yet surveyed. Reads live: a deeper re-survey upgrades held `Site` objects too. Higher tiers unlock more fields below. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`str | None`

POSSIBLE VALUES

`"basic"`, `"wide"`, `"deep"`, `"seismic"`

`.current_phase() → str | None`

The deposit's phase right now: `"active"` (emitting) or `"dormant"` (idle). Reads live: poll it from a held `Site` object and it follows the cycle. Returns `None` before the deposit is surveyed. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`str | None`

POSSIBLE VALUES

`"active"`, `"dormant"`

`.cycle_active_minutes() → float | None`

Duration of the active phase in minutes. Requires **deep** survey: returns `None` otherwise.

RETURNS

`float | None`

`.cycle_dormant_minutes() → float | None`

Duration of the dormant phase in minutes (rare deposits stay dormant longest). Requires **deep** survey: returns `None` otherwise.

RETURNS

`float | None`

`.next_phase_in() → float | None`

Game-minutes until the next phase flip. Reads live: poll it in a control loop to act before dormancy hits. Requires **deep** survey: returns `None` otherwise. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`float | None`

`.base_rate() → float | None`

Peak output rate during the active phase (t/h). Requires **wide** survey: returns `None` at basic.

RETURNS

`float | None`

`.current_rate() → float | None`

Output rate right now (t/h: **0** during dormant). Reads live. Requires **wide** survey: returns `None` at basic. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`float | None`

`.has_cap() → bool`

Boolean: `True` if an Exotic Gas Cap or Exotic Spring Tap is currently deployed on this deposit. Reads live. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`bool`

`.cap_id() → str`

Machine id of the currently deployed cap/tap, or empty string when none is present. Reads live. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`str`

GeologicalAnomaly

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "inert"`

Get `GeologicalAnomaly` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

`.seismic_status: str`

Learned seismic contact: `"unscanned"` before prospecting, `"potential"` for a deep reservoir contact, or `"dry"` for a formation without oil. Only a Seismic Sonar survey confirms a potential contact as an Oil Well.

RETURNS

`str`

POSSIBLE VALUES

`"unscanned"`, `"potential"`, `"dry"`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

MiningSite

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "mineral"`

Get `MiningSite` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

`.item_id: str | None`

Item id this site yields when drilled (e.g. `"iron_ore"`, `"silicon"`). `None` until `surveyed`.

RETURNS

`str | None`

POSSIBLE VALUES

`"iron_ore"`, `"silicon"`, `"titanium"`, `"cobalt"`, `"rare_earth"`, `"neutronium"`, `"lead_ore"`

`.hardness: int | None`

Hardness rating (**1-4**): gates drill compatibility. `None` until `surveyed`.

RETURNS

`int | None`

`.purity: str | None`

Yield multiplier tier: `"standard"` (**1×**) / `"rich"` (**2×**) / `"pure"` (**3×**). `None` until `surveyed`.

RETURNS

`str | None`

POSSIBLE VALUES

`"standard"`, `"rich"`, `"pure"`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

`.has_drill() → bool`

Boolean: `True` if a Mining Drill of any tier is currently deployed on this site. A pre-survey sonar result always returns `False`; obtain a new object after surveying for live readings.

RETURNS

`bool`

`.drill_id() → str`

Current machine id of the Mining Drill deployed on this site, or empty string when no drill is present. A pre-survey sonar result always returns empty string; obtain a new object after surveying for live readings.

RETURNS

`str`

OilWell

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "oil"` (e.g. `oil_pump.well()`, sonar / journal queries)

Get `OilWell` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

`.yield_tier() → str | None`

One of `"standard"` (**1×**) / `"rich"` (**2×**) / `"pure"` (**3×**). `None` until surveyed. Checks current survey progress, except on a pre-survey sonar result, which keeps returning `None`; obtain a new object after surveying.

RETURNS

`str | None`

POSSIBLE VALUES

`"standard"`, `"rich"`, `"pure"`

`.flow_rate() → float | None`

Peak tons of oil per hour. **8 / 16 / 24** for standard / rich / pure. Oil wells pulse through active and dormant phases: a dormant well delivers nothing at any throttle (read the pump's `well_active()`). `None` until surveyed. Reads live, except on a pre-survey sonar result, which keeps returning `None`; obtain a new object after surveying.

RETURNS

`float | None`

`.has_pump() → bool`

Boolean: `True` if an Oil Pump is currently deployed on this well. A pre-survey sonar result always returns `False`; obtain a new object after surveying for live readings.

RETURNS

`bool`

`.pump_id() → str`

Current machine id of the Oil Pump deployed on this well, or empty string when no pump is present. A pre-survey sonar result always returns empty string; obtain a new object after surveying for live readings.

RETURNS

`str`

ThermalVent

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "thermal"` (e.g. `thermal_cap.vent()`, sonar / journal queries)

Get `ThermalVent` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

`.survey_level() → str | None`

Highest survey tier achieved on this vent: `"basic"` / `"wide"` / `"deep"` / `"seismic"`, or `None` if not yet surveyed. Reads live: a deeper re-survey upgrades held `Site` objects too. Higher tiers unlock more fields below. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`str | None`

POSSIBLE VALUES

`"basic"`, `"wide"`, `"deep"`, `"seismic"`

`.cycle_active_minutes() → float | None`

Duration of the active phase in minutes. Requires **deep** survey: returns `None` otherwise.

RETURNS

`float | None`

`.cycle_dormant_minutes() → float | None`

Duration of the dormant phase in minutes. Requires **deep** survey: returns `None` otherwise.

RETURNS

`float | None`

`.current_phase() → str | None`

The vent's phase right now: `"active"` or `"dormant"`. Reads live: poll it from a held `Site` object and it follows the cycle. Returns `None` before the vent is surveyed. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS
`str | None`

POSSIBLE VALUES
`"active"`, `"dormant"`

`.next_phase_in() → float | None`

Game-minutes until the next phase flip. Reads live: poll it in a control loop to act before dormancy hits. Requires **deep** survey: returns `None` otherwise. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS
`float | None`

`.base_steam_rate() → float | None`

Peak steam rate during active phase (t/h). Requires **wide** survey: returns `None` at basic.

RETURNS
`float | None`

`.current_steam_rate() → float | None`

Steam rate right now (t/h: **0** during dormant phase). Reads live. Requires **wide** survey: returns `None` at basic. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS
`float | None`

`.has_cap() → bool`

Boolean: `True` if a Thermal Cap is currently deployed on this vent. Reads live. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`bool`

`.cap_id() → str`

Machine id of the currently deployed Thermal Cap, or empty string when no cap is present. Reads live. A pre-survey sonar result stays unrevealed; obtain a new object after surveying.

RETURNS

`str`

WaterWell

Extends `Site`

Returned by: any `Site`-returning API where `kind() == "water"` (e.g. `water_pump.well()`, sonar / journal queries)

Get `WaterWell` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique site identifier.

RETURNS
`str`

`.name: str`

Human-readable display name.

RETURNS
`str`

`.x: float`

Site X coordinate in meters from base.

RETURNS
`float`

`.y: float`

Site Y coordinate in meters from base.

RETURNS

`float`

`.surveyed: bool`

Survey flag captured when this object was returned; resolved inert contacts also report `True`. The property never updates. Mining-site properties are snapshots. Mining-site, well, vent and exotic-deposit methods read live, except on pre-survey sonar results: those keep their hidden values, so obtain a new object after surveying.

RETURNS

`bool`

METHODS

`.kind() → str`

One of `"mineral"` / `"thermal"` / `"water"` / `"oil"` / `"exotic"` / `"inert"`. Use it to narrow to the concrete subtype: after `if site.kind() == "mineral":` the editor surfaces `MiningSite`-specific fields on `site`. `"inert"` means sonar resolved a physical formation that holds nothing to extract.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"thermal"`, `"water"`, `"oil"`, `"exotic"`, `"inert"`

`.position() → Position`

`Position` snapshot with `.x` / `.y` world coordinates.

RETURNS

`Position`

`.yield_tier() → str | None`

One of `"standard"` (**1×**) / `"rich"` (**2×**) / `"pure"` (**3×**). `None` until surveyed. Checks current survey progress, except on a pre-survey sonar result, which keeps returning `None`; obtain a new object after surveying. Wells reveal fully on basic survey.

RETURNS

`str | None`

POSSIBLE VALUES

`"standard"`, `"rich"`, `"pure"`

`.flow_rate() → float | None`

Tons of water per hour this well produces. **10 / 20 / 30** for standard / rich / pure. `None` until surveyed. Reads live, except on a pre-survey sonar result, which keeps returning `None`; obtain a new object after surveying.

RETURNS

`float | None`

`.has_pump() → bool`

Boolean: `True` if a Water Pump is currently deployed on this well. A pre-survey sonar result always returns `False`; obtain a new object after surveying for live readings.

RETURNS

`bool`

`.pump_id() → str`

Current machine id of the Water Pump deployed on this well, or empty string when no pump is present. A pre-survey sonar result always returns empty string; obtain a new object after surveying for live readings.

RETURNS

`str`

Battery

Returned by: `self.battery` (vehicles)

Get `Battery` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Holder`
- `PortableBattery`

METHODS

`.level() → float`

Charge level as a fraction, **0-1**.

RETURNS
`float`

`.wh() → float`

Current charge in Wh (across all batteries).

RETURNS
`float`

`.capacity() → float`

Maximum capacity in Wh.

RETURNS
`float`

`.holders() → list[Holder]`

List of every Battery Holder currently mounted on the vehicle. Empty for the Rover (sealed battery).

RETURNS

`list[Holder]`

Holder

Returned by: self.battery.holders()

Get `Holder` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Holder module id, e.g. `"battery_holder_medium"`.

RETURNS
`str`

`.size: str`

Holder size: `"small"` / `"medium"` / `"large"`.

RETURNS
`str`

POSSIBLE VALUES
`"small"`, `"medium"`, `"large"`

`.capacity: float`

Rated Wh across every battery in this holder.

RETURNS
`float`

`.wh: float`

Current Wh in this holder (pool level × rated capacity).

RETURNS

`float`

`.batteries: list[PortableBattery | None]`

List indexed by internal slot: each entry is a `PortableBattery` object, or `None` for an empty slot.

RETURNS

`list[PortableBattery | None]`

PortableBattery

Returned by: self.battery.holders()[...].batteries[...]

Get `PortableBattery` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Portable battery item id: `"portable_battery"` or `"heavy_portable_battery"`.

RETURNS
`str`

POSSIBLE VALUES
`"portable_battery"`, `"heavy_portable_battery"`

METHODS

.level() → float

Charge level as a fraction, **0-1**.

RETURNS
`float`

.wh() → float

Current charge in Wh.

RETURNS
`float`

`.capacity() → float`

Rated capacity in Wh.

RETURNS

`float`

Cargo

Returned by: `self.cargo` (vehicles)

Get `Cargo` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Rack`

METHODS

`.count() → int`

Total units currently carried by the vehicle.

RETURNS

`int`

`.capacity() → int`

Total units the vehicle can carry in its integrated hold or installed Cargo Racks.

RETURNS

`int`

`.stacks() → list[ItemStack]`

Snapshot of every property-distinct `ItemStack` in the vehicle's cargo. Use this to distinguish variants that share an item id.

RETURNS

`list[ItemStack]`

`.full() → bool`

`True` when the integrated hold or every installed cargo bin is full. A Pioneer without installed bins is also full.

RETURNS

`bool`

`.racks() → list[Rack]`

List of every Cargo Rack currently mounted. Empty for the Rover.

RETURNS

`list[Rack]`

`.compact() → TransferResult`

On a Pioneer, consolidate equal item ids into the fewest installed Portable Bins that can hold them. Property-distinct variants remain intact, and the planner preserves the fullest compatible bins to minimize physical movement. Requires **Auto Feeders**, waits for time proportional to the units repositioned, and holds the Pioneer stationary as a material endpoint during that cycle. This does not change which bin `send()` drains first.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`TransferResult`	`.status`, `.message`	`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Repositioned units: `.moved` into fewer Pioneer Portable Bins.
`"already_compact"`	success	This cargo is already compact; no Portable Bin cargo needs to move.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_full"`	rejection	The destination has no capacity for matching units.

`.discard(rack_index: int) → DiscardResult`

Permanently jettison everything in the whole-number, zero-based `rack_index`. On a Rover, which has no racks, `self.cargo.discard(0)` addresses the integrated hold. An already-empty hold or rack finishes immediately; destroying cargo takes **1 hour** and pauses the script. This is the vehicle cargo destruction API.

PARAMETERS

Name	Type	Description
`rack_index`	`int`	Whole-number zero-based index among mounted Cargo Racks (0 = the integrated hold on a Rover)

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`DiscardResult`	`.status`, `.message`	`.requested`, `.discarded`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Permanently discarded units: `.discarded`.
`"empty"`	success	The selected cargo area was already empty.
`"busy"`	transient	The vehicle is occupied by another field action.
`"invalid_rack"`	rejection	The requested cargo rack does not exist.

Rack

Returned by: self.cargo.racks()

Get `Rack` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Bin`

PROPERTIES

`.id: str`

Rack module id, e.g. `"cargo_rack_medium"`.

RETURNS
`str`

`.size: str`

Rack size: `"small"` / `"medium"` / `"large"`.

RETURNS
`str`

POSSIBLE VALUES
`"small"`, `"medium"`, `"large"`

`.bins: list[Bin | None]`

List indexed by internal slot: each entry is a `Bin` object, or `None` for an empty slot.

RETURNS
`list[Bin | None]`

Bin

Returned by: Rack.bins

Get `Bin` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Portable bin item id: `"portable_bin"` or `"heavy_portable_bin"`.

RETURNS
`str`

POSSIBLE VALUES
`"portable_bin"`, `"heavy_portable_bin"`

`.capacity: int`

Rated units: **25** basic, **50** heavy.

RETURNS
`int`

`.count: int`

Current units stored.

RETURNS
`int`

`.item_id: str | None`

Assigned item id (mineral, ingot, component: anything the bin holds), or `None` if the bin is empty/unassigned.

RETURNS

`str | None`

`.stacks: list[ItemStack]`

Property-distinct `ItemStack` snapshots in this portable bin. Equal ids with different properties remain separate entries.

RETURNS

`list[ItemStack]`

DiscardResult

Returned by: Cargo.discard(), DroneCargo.discard()

Get `DiscardResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Stable discard outcome code.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"partial"`, `"empty"`, `"busy"`, `"invalid_rack"`, `"no_op"`, `"invalid_properties"`, `"invalid_property_match"`, `"source_changed"`

.message: str

Player-readable explanation of the discard outcome.

RETURNS

`str`

.requested: int

Whole-number units requested or observed for destruction.

RETURNS

`int`

`.discarded: int`

Whole-number units permanently destroyed.

RETURNS

`int`

InputSlot

Returned by: `self.input` on stationary machines with an input buffer

Get `InputSlot` from the APIs listed here. It has no script constructor.

METHODS

`.connect(name: str) → ActionResult`

Set a compatible item source by stable id or display name. Two stationary endpoints must share an outpost. A Rover or Pioneer is reachable from any outpost, but only while it is parked inside this machine's service area. The output stockpile of a field structure on the map, such as a Mining Drill, can be pulled only by a Rover or Pioneer parked there. A drone's cargo moves through its Drone Depot. `"inventory"` is a freight source only while the endpoint is at Nocturna Base; remote stationary ports use local Storage Bins, Warehouses, or machine buffers.

PARAMETERS

Name	Type	Description
`name`	`str`	Stable id or display name of a compatible item source, or `"inventory"`

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"not_local"`	● rejection	The requested endpoint belongs to another outpost.
`"same_endpoint"`	● rejection	The requested item source and destination are the same endpoint.
`"unsupported_source"`	● rejection	The selected component does not expose a compatible item source.
`"source_is_vehicle"`	● rejection	The requested source is a drone. Drone cargo moves through its Drone Depot.

`.disconnect() → ActionResult`

Clear the current source connection. Does not move or discard buffered items; use `eject(...)` to recover them or `flush()` to destroy them.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.connected_to() → str`

Display name of the currently connected source, or empty string.

RETURNS

`str`

`.connected_id() → str`

Stable id of the currently connected source, or empty string. `connect()` accepts an id or display name, but this returns the resolved stable id. Compare it with the source's stable id; `connected_to()` returns its renameable display name.

RETURNS

`str`

`.take(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Take up to whole-number `count` units of `item_id` from the connected source. A property `dict` selects stacks containing that subset by default. Pass `"exact"` as the fourth argument for one full identity; `None, "exact"` selects only propertyless items. `"any"` ignores properties. Exact source properties are always retained. The transfer takes time proportional to units moved and requires Auto Feeders research.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to take
`count`	`int`	Whole-number max units to take
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"busy"`	● transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"no_connection"`	● rejection	The port has no configured transfer endpoint.
`"inventory_not_local"`	● rejection	This endpoint cannot access the home freight Inventory from its current outpost.

Status	Kind	Meaning
`"same_endpoint"`	rejection	The configured source and destination are the same item endpoint.
`"source_missing"`	rejection	The configured source no longer exists.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_is_vehicle"`	rejection	The mobile source is a drone, or is a ground vehicle that is not docked in this endpoint's service area.
`"source_not_local"`	rejection	The configured source belongs to another outpost.
`"not_at_source"`	rejection	The vehicle is outside the source's service area.
`"unsupported_source"`	rejection	The configured source does not expose compatible cargo.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_reserved"`	rejection	The source is reserving its cargo for an active operation.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"buffer_full"`	rejection	The receiving input buffer is full.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"wrong_biome"`	rejection	The destination rejects this biological material because its biome is incompatible.
`"target_unconfigured"`	rejection	The destination lacks the world-state configuration required to accept this material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"order_item_not_required"`	rejection	The active order does not require this item.
`"order_slots_full"`	rejection	Every Supply Dock material slot is assigned to another required item.
`"order_fulfilled"`	rejection	The active order already has all required units of this item loaded or shipped.
`"target_full"`	rejection	The destination has no capacity for matching units.

Status	Kind	Meaning
`"target_complete"`	rejection	The destination has completed its work and no longer accepts materials.
`"target_changed"`	transient	The destination changed between transfer planning and commit.
`"same_vehicle"`	rejection	A vehicle cannot transfer cargo to itself.
`"source_moving"`	transient	The source vehicle must stop before cargo can be handed off.
`"target_moving"`	transient	The destination vehicle must stop before cargo can be handed off.
`"out_of_range"`	rejection	The vehicles are outside cargo handoff range.

`.eject(destination: str, item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Recover up to whole-number `count` units of `item_id` from this buffer without changing its source connection. Use `"inventory"` at Nocturna Base, or a compatible store or machine input at the same outpost, or a ground vehicle parked in this machine's service area. Optional properties use the standard `"any"`, `"subset"`, or `"exact"` rules; item properties are preserved exactly. Destination capacity may limit it. While work is active or reserved, the call is rejected and nothing moves; a successful ejection cancels fractional work on the staged input. The transfer takes time proportional to units moved and requires Auto Feeders research.

PARAMETERS

Name	Type	Description
`destination`	`str`	Stable id or display name of a local compatible freight destination, or `"inventory"`
`item_id`	`str`	Item id to recover
`count`	`int`	Whole-number max units to recover
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"busy"`	transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.

Status	Kind	Meaning
`"inventory_not_local"`	rejection	This endpoint cannot access the home freight Inventory from its current outpost.
`"same_endpoint"`	rejection	The configured source and destination are the same item endpoint.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_reserved"`	rejection	The source is reserving its cargo for an active operation.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_missing"`	rejection	The configured target no longer exists.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_is_vehicle"`	rejection	The mobile target is a drone, or is a ground vehicle that is not docked in this endpoint's service area.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"wrong_biome"`	rejection	The destination rejects this biological material because its biome is incompatible.
`"target_unconfigured"`	rejection	The destination lacks the world-state configuration required to accept this material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"order_item_not_required"`	rejection	The active order does not require this item.
`"order_slots_full"`	rejection	Every Supply Dock material slot is assigned to another required item.
`"order_fulfilled"`	rejection	The active order already has all required units of this item loaded or shipped.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.

Status	Kind	Meaning
`"target_full"`	rejection	The destination has no capacity for matching units.
`"target_complete"`	rejection	The destination has completed its work and no longer accepts materials.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

`.flush() → TransferResult`

Permanently discard everything currently buffered in this input port. Flushed items are not returned to inventory. On processing machines, flushing also cancels any in-progress craft.

RETURNS
`TransferResult`

RESULT FIELDS
`.status`, `.message`

SUCCESS PAYLOAD
`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"no_op"`	success	No units were requested, so no state changed.

`.count() → int`

Total units currently in this port's buffer.

RETURNS
`int`

`.capacity() → int`

Maximum units this port's buffer can hold.

RETURNS
`int`

`.stacks() → list[ItemStack]`

Snapshot list of property-distinct `ItemStack` values currently buffered. Two entries may have the same id when their properties differ.

RETURNS

`list[ItemStack]`

ItemInfo

Returned by: item_catalog.lookup(item_id)

Get `ItemInfo` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable item id.

RETURNS

`str`

.name: str

Localized item display name.

RETURNS

`str`

`.category: str`

Stable category used to distinguish materials, biological items, and equipment kinds.

RETURNS

`str`

POSSIBLE VALUES

`"mineral"`, `"refined"`, `"crafted"`, `"agriculture"`, `"life_form"`, `"field_resource"`, `"biology_sample"`, `"reagent"`, `"equipment"`, `"module"`, `"portable"`, `"upgrade_pack"`, `"construction_kit"`

`.stackable: bool`

`True` when multiple units share one inventory stack; `False` when each unit occupies its own slot.

RETURNS

`bool`

`.biome: str | None`

Native biome for a life form or biology sample; `None` when biome does not apply.

RETURNS

`str | None`

POSSIBLE VALUES

`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.rarity: str | None`

Rarity for a life form or biology sample; `None` when rarity does not apply.

RETURNS

`str | None`

POSSIBLE VALUES

`"common"`, `"uncommon"`, `"rare"`, `"legendary"`

`.production_tier: int | None`

Lowest production tier that creates this item, or `None` when it is a raw, harvested, purchased, or otherwise source item. Smelter products are Tier 0.

RETURNS

`int | None`

ItemResult

Returned by: harvester.store(), inventory.drop()

Get `ItemResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"empty"`, `"inventory_full"`, or `"invalid_slot"`, narrowed further by the command that returned it.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"empty"`, `"inventory_full"`, `"invalid_slot"`

.message: str

Player-readable explanation of the item operation.

RETURNS
`str`

.item_id: str | None

Stable id of the item moved or destroyed, or `None` when no item changed.

RETURNS
`str | None`

ItemStack

Returned by: InputSlot.stacks(), VehicleInputSlot.stacks(), OutputSlot.stacks(), PickupOutputSlot.stacks(), Cargo.stacks(), Bin.stacks, storage_bin.stacks(), warehouse.stacks()

Import `ItemStack` with `from __builtins__ import ItemStack`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`ItemStack(id: str, count: int, properties: ItemProperties | None = None) → ItemStack`

Create a local `ItemStack` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`id`	`str`	Stable item id shared by every unit in this stack.
`count`	`int`	Whole-number units with this exact property identity. Must be a whole number. Must be at least 0.
`properties`	`ItemProperties` | `None`	Exact property `dict` for this item, or `None` for an ordinary commodity. Items with different properties form separate stacks and never merge.

RETURNS

`ItemStack`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`ValueError`	A numeric value is outside the permitted range, or the bounds are reversed.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

`.id: str`

Stable item id shared by every unit in this stack.

RETURNS

`str`

`.count: int`

Whole-number units with this exact property identity.

RETURNS

`int`

`.properties: ItemProperties | None`

Exact property `dict` for this item, or `None` for an ordinary commodity. Items with different properties form separate stacks and never merge.

RETURNS

`ItemProperties | None`

OutputSlot

Returned by: `self.output` (machines with output port)

Get `OutputSlot` from the APIs listed here. It has no script constructor.

METHODS

`.connect(name: str) → ActionResult`

Set a compatible item destination by stable id or display name. Two stationary endpoints must share an outpost. A Rover or Pioneer is reachable from any outpost, but only while it is parked inside this machine's service area. A drone's cargo moves through its Drone Depot. `"inventory"` is a freight destination only while the endpoint is at Nocturna Base; remote stationary ports use local Storage Bins, Warehouses, or machine inputs.

PARAMETERS

Name	Type	Description
`name`	`str`	Stable id or display name of a compatible item destination, or `"inventory"`

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"not_local"`	rejection	The requested endpoint belongs to another outpost.
`"same_endpoint"`	rejection	The requested item source and destination are the same endpoint.
`"unsupported_target"`	rejection	The selected component does not expose a compatible item destination.
`"target_is_vehicle"`	rejection	The requested destination is a drone. Drone cargo moves through its Drone Depot.

`.disconnect() → ActionResult`

Clear the current target connection. Does not move or discard buffered output.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.connected_to() → str`

Display name of the currently connected target, or empty string.

RETURNS

`str`

`.connected_id() → str`

Stable id of the currently connected target, or empty string. `connect()` accepts an id or display name, but this returns the resolved stable id. Compare it with the target's stable id; `connected_to()` returns its renameable display name.

RETURNS

`str`

`.send(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Send up to whole-number `count` units of `item_id` to the connected target. A property `dict` selects stacks containing that subset by default. Pass `"exact"` as the fourth argument for one full identity; `None, "exact"` selects only propertyless items. `"any"` ignores properties. Exact source properties are preserved. The transfer takes time proportional to units moved and requires Auto Feeders research.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to send
`count`	`int`	Whole-number max units to send
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"busy"`	● transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"no_connection"`	● rejection	The port has no configured transfer endpoint.
`"inventory_not_local"`	● rejection	This endpoint cannot access the home freight Inventory from its current outpost.

Status	Kind	Meaning
`"same_endpoint"`	rejection	The configured source and destination are the same item endpoint.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_missing"`	rejection	The configured target no longer exists.
`"target_under_construction"`	transient	The configured target is still under construction.
`"target_is_vehicle"`	rejection	The mobile target is a drone, or is a ground vehicle that is not docked in this endpoint's service area.
`"target_not_local"`	rejection	The configured target belongs to another outpost.
`"not_at_target"`	rejection	The vehicle is outside the target's service area.
`"unsupported_target"`	rejection	The configured target cannot accept compatible cargo.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"wrong_biome"`	rejection	The destination rejects this biological material because its biome is incompatible.
`"target_unconfigured"`	rejection	The destination lacks the world-state configuration required to accept this material.
`"mixed_materials"`	rejection	One transfer can contain only one item id.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"cask_accepts_hot_only"`	rejection	Lead Casks accept only radioactive cargo: Raw Uranium and Fuel Rods.
`"order_item_not_required"`	rejection	The active order does not require this item.
`"order_slots_full"`	rejection	Every Supply Dock material slot is assigned to another required item.
`"order_fulfilled"`	rejection	The active order already has all required units of this item loaded or shipped.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.

Status	Kind	Meaning
`"target_complete"`	rejection	The destination has completed its work and no longer accepts materials.
`"target_changed"`	transient	The destination changed between transfer planning and commit.
`"same_vehicle"`	rejection	A vehicle cannot transfer cargo to itself.
`"source_moving"`	transient	The source vehicle must stop before cargo can be handed off.
`"target_moving"`	transient	The destination vehicle must stop before cargo can be handed off.
`"out_of_range"`	rejection	The vehicles are outside cargo handoff range.

`.count() → int`

Total units currently in this port's buffer.

RETURNS

`int`

`.capacity() → int`

Maximum units this port's buffer can hold.

RETURNS

`int`

`.stacks() → list[ItemStack]`

Snapshot list of property-distinct `ItemStack` values currently buffered. Use `.properties` to inspect and select variants before `send()`.

RETURNS

`list[ItemStack]`

PickupOutputSlot

Returned by: `self.output` on field Water Pumps and Mining Drills

Get `PickupOutputSlot` from the APIs listed here. It has no script constructor.

METHODS

`.count() → int`

Total item units waiting for carrier pickup.

RETURNS

`int`

`.capacity() → int`

Maximum item units this pickup stockpile can hold.

RETURNS

`int`

`.stacks() → list[ItemStack]`

Snapshot list of property-distinct `ItemStack` values waiting for carrier pickup.

RETURNS

`list[ItemStack]`

Recipe

Returned by: `list_recipes()` / `find_recipe()` on Smelter, Fabricator, Feed Maker, Refiner, and Fuel Assembler

Get `Recipe` from the APIs listed here. It has no script constructor.

PROPERTIES

`.tier: int`

Derived production tier. Smelter foundations are Tier 0; every other recipe is one tier above its deepest manufactured input.

RETURNS
`int`

`.id: str`

Recipe id (e.g. `"smelt_iron_ingot"`). Stable across saves: use to compare recipes or pass back to `set_recipe(...)`.

RETURNS
`str`

`.name: str`

Pre-translated display name of the recipe.

RETURNS
`str`

`.inputs: dict[str, int]`

A `dict` `{item_id: count}` of materials consumed per run. Iterate via `.keys()`, `.values()`, `.items()`, or index: `recipe.inputs["iron_ore"]`.

RETURNS

`dict[str, int]`

`.output_item: str`

Product id produced per run: an item id for discrete recipes or a fluid id for fluid-output recipes such as the Refiner.

RETURNS

`str`

`.output_count: int`

Amount of `output_item` produced per run: whole units for an item or tons for a fluid.

RETURNS

`int`

`.duration_game_hours: float`

Cycle duration in hours at full efficiency, including the querying machine's Mk tier. Inputs, power and output space must be available; overcrowding and blocked output can extend actual completion time.

RETURNS

`float`

`.power_draw: float`

Watts the machine draws while this recipe is running.

RETURNS

`float`

`.fluid_inputs: dict[str, float]`

A `dict` `{port_name: tons_per_run}` of fluid consumed when one run completes (`"water_in"` etc.). Empty `dict` for recipes with no fluid input.

RETURNS

`dict[str, float]`

`.input_fluid: str | None`

Concrete fluid id required by a generic input port, or `None` when the port itself already identifies the fluid. Refiner recipes use this to distinguish raw feedstocks on `gas_in` / `liquid_in`.

RETURNS

`str | None`

POSSIBLE VALUES

`"raw_sulfur_gas"`, `"raw_cryofluid"`, `"raw_chlorine"`, `"raw_quicksilver"`

`.fluid_outputs: dict[str, float]`

A `dict` `{port_name: tons_per_run}` of fluid deposited when one run completes. Empty `dict` for item-output recipes; Refiner recipes identify `gas_out` or `liquid_out` here.

RETURNS

`dict[str, float]`

`.output_fluid: str | None`

Concrete fluid id produced through a generic output port, or `None` for item-output recipes. Equal to `output_item` for current fluid-only Refiner recipes.

RETURNS

`str | None`

POSSIBLE VALUES

`"sulfur_gas"`, `"cryofluid"`, `"chlorine"`, `"quicksilver"`

`.byproduct_item: str | None`

Item id of the byproduct, or `None` if this recipe has no byproduct.

RETURNS

`str | None`

`.byproduct_count: int`

Units of `byproduct_item` produced per run; **0** for recipes with no byproduct.

RETURNS

`int`

SaleResult

Returned by: shop.sell(), shop.sell_all()

Get `SaleResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"not_sellable"`, or `"no_stock"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"not_sellable"`, `"no_stock"`

.message: str

Player-readable explanation of the sale outcome.

RETURNS
`str`

.item_id: str

Stable item id requested for sale.

RETURNS
`str`

`.units: int`

Whole-number units removed from Inventory.

RETURNS

`int`

`.credits: int`

Whole-number credits earned by this sale.

RETURNS

`int`

ShopItem

Returned by: shop.get_catalogue()

Get `ShopItem` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Item ID (used for buying).

RETURNS
`str`

.name: str

Item display name.

RETURNS
`str`

.cost: int

Price in credits.

RETURNS
`int`

Slot

Returned by: inventory.get_slots()

Get `Slot` from the APIs listed here. It has no script constructor.

PROPERTIES

.slot: int

Slot index number.

RETURNS
`int`

.id: str

Item ID (empty string if slot is empty).

RETURNS
`str`

.name: str

Item display name.

RETURNS
`str`

`.value: float`

Credit value per item.

RETURNS

`float`

`.count: int`

Stack count in this slot.

RETURNS

`int`

`.properties: ItemProperties | None`

Exact property `dict` for this stack, or `None` for an ordinary item. Pass it to property-aware transfer and capacity methods.

RETURNS

`ItemProperties | None`

`.genes: list[str]`

The genes a held **geothermal** fragment carries, e.g. `["cold_tolerance", "pressure_tolerance"]`: read it straight from inventory without loading the DNA Sequencer. A spliced fragment carries exactly the set you engineered; a raw fragment carries its natural genes. Empty for non-geothermal items.

RETURNS

`list[str]`

`.glow: list[int] | None`

Per-instance bioluminescent glow `[r,g,b]` (0-255) for a held **coastal** fragment. `None` for any other item.

RETURNS

`list[int] | None`

`.spliced: bool`

`True` if this is an engineered (already gene-spliced) geothermal fragment: a further DNA Sequencer alter would destroy it. `False` for raw items.

RETURNS

`bool`

TransferResult

Returned by: `InputSlot.take()`, `InputSlot.eject()`, `InputSlot.flush()`, `VehicleInputSlot.take()`, `OutputSlot.send()`, `Cargo.compact()`, `storage_bin` transfer methods, `warehouse.compact()`

Get `TransferResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Stable result code for scripts to check. `.message` explains what happened.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"partial"`, `"no_op"`, `"already_compact"`, `"research_required"`, `"busy"`, `"invalid_properties"`, `"invalid_property_match"`, `"no_connection"`, `"inventory_not_local"`, `"source_missing"`, `"source_under_construction"`, `"source_is_vehicle"`, `"source_not_local"`, `"not_at_source"`, `"unsupported_source"`, `"source_wrong_material"`, `"source_empty"`, `"source_reserved"`, `"source_changed"`, `"buffer_full"`, `"slots_full"`, `"target_missing"`, `"target_under_construction"`, `"target_is_vehicle"`, `"target_not_local"`, `"not_at_target"`, `"unsupported_target"`, `"target_wrong_material"`, `"wrong_biome"`, `"needs_plating"`, `"cask_missing"`, `"target_unconfigured"`, `"mixed_materials"`, `"hot_cargo_requires_cask"`,

`"cask_accepts_hot_only"`,
`"order_item_not_required"`, `"order_slots_full"`,
`"order_fulfilled"`,
`"target_full"`,
`"target_complete"`,
`"target_changed"`,
`"same_endpoint"`,
`"same_storage"`,
`"same_vehicle"`,
`"source_moving"`,
`"target_moving"`,
`"out_of_range"`

`.message: str`

Player-readable, contextual explanation of this exact outcome. Suitable for logs and player-facing diagnostics.

RETURNS

`str`

`.requested: int`

Whole-number units requested by the call. For parameterless `flush()` or `compact()`, this is the buffered or repositioned amount observed by the operation.

RETURNS

`int`

`.moved: int`

Whole-number units actually transferred, repositioned, or destroyed. Always **0** for rejected outcomes.

RETURNS

`int`

VehicleInputSlot

Returned by: `self.input` on Rover and Pioneer

Get `VehicleInputSlot` from the APIs listed here. It has no script constructor.

METHODS

`.connect(name: str) → ActionResult`

Set a compatible cargo source by stable id or display name. A Rover or Pioneer must be parked inside a stationary source's service area. Field Mining Drill and Water Pump stockpiles support carrier pickup. Vehicle handoffs require both vehicles to be stopped and nearby. `"inventory"` is available only while parked at Nocturna Base; remote outposts use local Storage Bins, Warehouses, or machine buffers.

PARAMETERS

Name	Type	Description
`name`	`str`	Stable id or display name of a compatible cargo source, or `"inventory"`

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_found"`	● rejection	The requested object, target, or record does not exist.
`"same_endpoint"`	● rejection	The requested item source and destination are the same endpoint.
`"unsupported_source"`	● rejection	The selected component does not expose a compatible item source.
`"source_is_vehicle"`	● rejection	The requested source is a drone. Drone cargo moves through its Drone Depot.

`.disconnect() → ActionResult`

Clear the current cargo source connection. This does not move or discard cargo.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.connected_to() → str`

Display name of the currently connected cargo source, or empty string.

RETURNS

`str`

`.connected_id() → str`

Stable id of the currently connected cargo source, or empty string. `connect()` accepts an id or display name, so compare against this when identity must survive renaming.

RETURNS

`str`

`.take(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Load up to whole-number `count` units of `item_id` into vehicle cargo from the connected source. A property `dict` selects stacks containing that subset by default. Pass `"exact"` as the fourth argument for one full identity; `None, "exact"` selects only propertyless items. `"any"` ignores properties. Exact source properties are retained. The transfer takes time proportional to units moved and requires Auto Feeders research.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to load
`count`	`int`	Whole-number max units to load
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"busy"`	● transient	This material endpoint is already handling another material operation.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"no_connection"`	● rejection	The port has no configured transfer endpoint.
`"inventory_not_local"`	● rejection	This endpoint cannot access the home freight Inventory from its current outpost.

Status	Kind	Meaning
`"same_endpoint"`	rejection	The configured source and destination are the same item endpoint.
`"source_missing"`	rejection	The configured source no longer exists.
`"source_under_construction"`	transient	The configured source is still under construction.
`"source_is_vehicle"`	rejection	The mobile source is a drone, or is a ground vehicle that is not docked in this endpoint's service area.
`"source_not_local"`	rejection	The configured source belongs to another outpost.
`"not_at_source"`	rejection	The vehicle is outside the source's service area.
`"unsupported_source"`	rejection	The configured source does not expose compatible cargo.
`"source_wrong_material"`	rejection	The source contains a different material from the one requested.
`"source_empty"`	rejection	The source has no matching units available.
`"source_reserved"`	rejection	The source is reserving its cargo for an active operation.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"buffer_full"`	rejection	The receiving input buffer is full.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_wrong_material"`	rejection	The destination is latched to or accepts a different material.
`"wrong_biome"`	rejection	The destination rejects this biological material because its biome is incompatible.
`"target_unconfigured"`	rejection	The destination lacks the world-state configuration required to accept this material.
`"hot_cargo_requires_cask"`	rejection	A transfer endpoint does not support this radioactive cargo.
`"order_item_not_required"`	rejection	The active order does not require this item.
`"order_slots_full"`	rejection	Every Supply Dock material slot is assigned to another required item.
`"order_fulfilled"`	rejection	The active order already has all required units of this item loaded or shipped.
`"target_full"`	rejection	The destination has no capacity for matching units.

Status	Kind	Meaning
`"target_complete"`	rejection	The destination has completed its work and no longer accepts materials.
`"target_changed"`	transient	The destination changed between transfer planning and commit.
`"same_vehicle"`	rejection	A vehicle cannot transfer cargo to itself.
`"source_moving"`	transient	The source vehicle must stop before cargo can be handed off.
`"target_moving"`	transient	The destination vehicle must stop before cargo can be handed off.
`"out_of_range"`	rejection	The vehicles are outside cargo handoff range.

`.count() → int`

Total units currently carried in the vehicle's cargo.

RETURNS

`int`

`.capacity() → int`

Maximum units the vehicle's cargo can hold.

RETURNS

`int`

`.stacks() → list[ItemStack]`

Snapshot list of property-distinct `ItemStack` values currently carried. Two entries may have the same id when their properties differ.

RETURNS

`list[ItemStack]`

WarehouseSlot

Returned by: warehouse.slots()

Get `WarehouseSlot` from the APIs listed here. It has no script constructor.

PROPERTIES

.index: int

Slot index (0-based).

RETURNS
`int`

.item: str

Item id this slot holds (empty string if the slot is empty).

RETURNS
`str`

.count: int

Units currently in this slot.

RETURNS
`int`

`.capacity: int`

This slot's capacity (units).

RETURNS

`int`

`.properties: ItemProperties | None`

Exact property `dict` for the item variant in this physical slot, or `None` when the slot is empty or holds an ordinary item.

RETURNS

`ItemProperties | None`

BlockedContact

Returned by: SonarScanResult.blocked

Get `BlockedContact` from the APIs listed here. It has no script constructor.

PROPERTIES

.x: int

Whole-number X coordinate (meters from base) of the unidentified contact, matching the `"?"` marker on the map.

RETURNS
`int`

.y: int

Whole-number Y coordinate (meters from base) of the unidentified contact.

RETURNS
`int`

.reason: str

Why this sonar could not identify the contact. `"wrong_scanner"` needs a different instrument entirely, such as a drone bio-scan for a biomass contact. `"too_hard"` exceeds the mounted sonar's hardness limit, `"tier_too_low"` needs a higher sonar tier, and `"research_required"` needs a classification technology that is not unlocked.

RETURNS
`str`

POSSIBLE VALUES
`"wrong_scanner"`,
`"too_hard"`,
`"tier_too_low"`,
`"research_required"`

`.message: str`

Readable explanation of `.reason`, suitable for printing straight to the console.

RETURNS

`str`

CollectResult

Returned by: drone_small.collect(), drone_medium.collect(), drone_large.collect()

Get `CollectResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Stable collection outcome: `"ok"`, `"moving"`, `"busy"`, `"nothing_here"`, `"no_cargo_space"`, or `"scrambled"`.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"moving"`, `"busy"`, `"nothing_here"`, `"no_cargo_space"`, `"scrambled"`

.message: str

Player-readable explanation of the exact collection outcome.

RETURNS

`str`

.item_id: str | None

Stable id of the collected material, or `None` when no material was collected.

RETURNS

`str | None`

`.collected: int`

Whole-number units committed to drone cargo by this call.

RETURNS

`int`

ConstructorModule

Returned by: `self.constructor` (Pioneer)

Get `ConstructorModule` from the APIs listed here. It has no script constructor.

METHODS

`.execute(blueprint_id: str) → ActionResult`

Execute a Plan Mode construction or deconstruction blueprint. Drive the Pioneer within interaction range of `blueprint.position`, then pass a blueprint id from `get_component("construction_blueprint").pending_constructions()`, `.active_constructions()`, or `.paused_constructions()`. Yielding. One Pioneer performs one field action at a time. Stop, power loss, leaving the site, rescue, or removing the Constructor Module pauses paid work without losing progress or materials. The owning Pioneer can rejoin active work, including after save/load; another Pioneer cannot steal an owned job.

PARAMETERS

Name	Type	Description
`blueprint_id`	`str`	Pending, active, or paused blueprint id from the `construction_blueprint` queue.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"locked"`	rejection	The required feature, recipe, or operation is locked.
`"already_active"`	transient	The requested operation is already active.
`"busy"`	transient	The component is already performing another operation.
`"wrong_position"`	rejection	The worker is not at the operation's required position.
`"insufficient_materials"`	rejection	The Pioneer does not have enough of the required construction material in cargo.
`"paused_no_power"`	transient	The operation is paused because power is unavailable.
`"cargo_present"`	rejection	Existing cargo prevents the requested configuration change.
`"blocked"`	rejection	The operation is blocked by the current world state.
`"paused"`	transient	The operation is paused.

Status	Kind	Meaning
`"canceled"`	rejection	The operation was canceled.
`"not_ready"`	rejection	The operation's prerequisites are not currently satisfied.

DrillModule

Returned by: `self.drill` (vehicles)

Get `DrillModule` from the APIs listed here. It has no script constructor.

METHODS

`.mine() → ActionResult`

Drill one unit while stationary.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.
`"not_at_site"`	rejection	The vehicle is not positioned at a compatible site.
`"not_surveyed"`	rejection	The mineral site has not been surveyed.
`"too_hard"`	rejection	The target exceeds the mounted tool's hardness limit.
`"no_cargo_space"`	rejection	Cargo has no compatible space for the result.
`"no_power"`	transient	The component has no available power.
`"not_enough_power"`	rejection	The available energy is below the operation's requirement.
`"busy"`	transient	The component is already performing another operation.

`.hardness_limit() → int`

Max mineral hardness this drill can extract (**1** basic, **3** Industrial, **4** Heavy). A stale captured module reference raises `ReferenceError`.

RETURNS

`int`

RAISES

Exception	Condition
`ReferenceError`	This `DrillModule` reference is stale because its module is no longer mounted. Read `self.drill` again after mounting a drill.

`.speed_multiplier() → float`

Drill-time multiplier (**1.0** basic, **0.75** Industrial, **0.6** Heavy: lower is faster). A stale captured module reference raises `ReferenceError`.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This `DrillModule` reference is stale because its module is no longer mounted. Read `self.drill` again after mounting a drill.

DroneBattery

Returned by: `self.battery` (electric drones)

Get `DroneBattery` from the APIs listed here. It has no script constructor.

METHODS

`.level() → float`

Current charge in Wh across mounted Battery Packs. Raises `ReferenceError` when this drone does not have an electric powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have an electric powertrain.

`.capacity() → float`

Total charge capacity in Wh. Raises `ReferenceError` when this drone does not have an electric powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have an electric powertrain.

`.percent() → float`

Charge as a fraction **0-1** (`level / capacity`). Raises `ReferenceError` when this drone does not have an electric powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have an electric powertrain.

DroneCargo

Returned by: `self.cargo` (drones)

Get `DroneCargo` from the APIs listed here. It has no script constructor.

METHODS

`.count() → int`

Total units across every Cargo Pod plus the Bio Extractor chamber.

RETURNS

`int`

`.capacity() → int`

Total physical capacity summed over every container: each mounted Cargo Pod (Small **100**, Medium **250**, Large **500**) plus the Bio Extractor's **25 t** chamber. Shield Plating halves each pod's capacity.

RETURNS

`int`

`.contents() → dict[str, int]`

A `dict` mapping `item_id` → unit count for every material currently in cargo. Iterate with `.keys()` / `.items()`.

RETURNS

`dict[str, int]`

`.space_for(item_id: str) → int`

Free room for this specific material. **Each Cargo Pod holds one material**, so a pod counts only if it is empty or already holds `item_id`; the Bio Extractor's 25 t chamber counts only for life forms. Returns **0** for a material with no empty or matching pod, even while other pods still have room for their own materials.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to size remaining room for

RETURNS

`int`

`.full() → bool`

`True` when `count() >= capacity()`. A drone may still be unable to load a new item type when every pod is committed, even if `full()` is `False`; use `space_for(item_id)` for a specific item.

RETURNS

`bool`

`.load(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Move up to whole-number `count` units into this drone's cargo, retaining exact properties. Loads from the docked Drone Depot stockpile or a field Mining Drill after the drone finishes its route there. Radioactive cargo loads from a local Lead Cask and requires Shield Plating.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to load
`count`	`int`	Whole-number maximum units to load
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Moved requested units: `.moved`.
`"partial"`	partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	success	No units were requested, so no state changed.
`"invalid_properties"`	rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	rejection	The requested property matching mode is invalid.
`"research_required"`	rejection	The required material-transfer research is not unlocked.
`"target_moving"`	transient	The destination vehicle must stop before cargo can be handed off.
`"not_at_source"`	rejection	The vehicle is outside the source's service area.
`"source_empty"`	rejection	The source has no matching units available.
`"slots_full"`	rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	rejection	The destination has no capacity for matching units.

Status	Kind	Meaning
`"needs_plating"`	rejection	This radioactive cargo requires Shield Plating on the drone.
`"cask_missing"`	rejection	No compatible Lead Cask is available at this outpost.
`"source_changed"`	transient	The source changed between transfer planning and commit.
`"target_changed"`	transient	The destination changed between transfer planning and commit.

`.unload(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → TransferResult`

Move up to whole-number `count` units from this drone's cargo into its docked Drone Depot, retaining exact properties. Radioactive cargo unloads into a compatible Lead Cask.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to unload
`count`	`int`	Whole-number maximum units to unload
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`TransferResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.moved`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Moved requested units: `.moved`.
`"partial"`	● partial	Moved `.moved` of `.requested` requested units; source availability or destination capacity limited the transfer.
`"no_op"`	● success	No units were requested, so no state changed.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"research_required"`	● rejection	The required material-transfer research is not unlocked.
`"not_at_target"`	● rejection	The vehicle is outside the target's service area.
`"source_empty"`	● rejection	The source has no matching units available.
`"slots_full"`	● rejection	The destination has capacity but no slot for this material identity.
`"target_full"`	● rejection	The destination has no capacity for matching units.
`"cask_missing"`	● rejection	No compatible Lead Cask is available at this outpost.
`"source_changed"`	● transient	The source changed between transfer planning and commit.

Status	Kind	Meaning
`"target_changed"`	● transient	The destination changed between transfer planning and commit.

`.discard(item_id: str, count: int, properties: ItemProperties | None = None, property_match: str | None = None) → DiscardResult`

Permanently destroy up to whole-number `count` units from this drone's cargo. This jettison needs no station and a drained pod unlatches for a new material.

PARAMETERS

Name	Type	Description
`item_id`	`str`	Item id to destroy
`count`	`int`	Whole-number maximum units to destroy
`properties`	`ItemProperties` | `None`	Optional property `dict`, matched as a subset by default. Omitted or `None` matches any properties; use `None` with `property_match="exact"` to select propertyless items only.
`property_match`	`str` | `None`	Optional selection mode: `"any"`, `"subset"`, or `"exact"`

RETURNS

`DiscardResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

`.requested`, `.discarded`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Permanently discarded units: `.discarded`.
`"partial"`	● partial	Permanently discarded `.discarded` of `.requested` requested units.
`"empty"`	● success	The selected cargo area was already empty.
`"no_op"`	● success	No item was specified or the requested count was zero, so nothing was discarded.
`"invalid_properties"`	● rejection	The item property selector has an invalid shape or value.
`"invalid_property_match"`	● rejection	The requested property matching mode is invalid.
`"source_changed"`	● transient	Cargo changed between transfer planning and commit.

DroneOilTank

Returned by: `self.oil_tank` (heli drones)

Get `DroneOilTank` from the APIs listed here. It has no script constructor.

METHODS

`.level() → float`

Current oil in tons across mounted Oil Tanks. Raises `ReferenceError` when this drone does not have a heli powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have a heli powertrain.

`.capacity() → float`

Total oil capacity in tons. Raises `ReferenceError` when this drone does not have a heli powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have a heli powertrain.

`.percent() → float`

Oil as a fraction **0-1**. Raises `ReferenceError` when this drone does not have a heli powertrain.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This drone does not currently have a heli powertrain.

Fleet

Returned by: get_component("fleet")

Get `Fleet` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `DroneRef`
- `MobileUnitRef`
- `VehicleRef`

METHODS

`.vehicles() → list[VehicleRef]`

All owned ground vehicles as read-only `VehicleRef` snapshots. Use `.id` when passing a vehicle to station APIs; call `vehicles()` again for fresh ref fields, or `get_component(ref.id)` for the live vehicle API.

RETURNS

`list[VehicleRef]`

`.drones() → list[DroneRef]`

All owned drones as read-only `DroneRef` snapshots. Use `.id` when passing a drone to station/recovery APIs; call `drones()` again for fresh ref fields, or `get_component(ref.id)` for the live drone API.

RETURNS

`list[DroneRef]`

`.mobile_units() → list[MobileUnitRef]`

All owned vehicles and drones in one snapshot list. Use `.category` to branch between `"vehicle"` and `"drone"`; re-query for fresh positions/status.

RETURNS

`list[MobileUnitRef]`

DroneRef

Returned by: `fleet.drones()`

Get `DroneRef` from the APIs listed here. It has no script constructor.

PROPERTIES

.category: str

Always `"drone"`.

RETURNS
`str`

POSSIBLE VALUES
`"drone"`

.id: str

Stable component id. Use with `get_component(id)` or pass it to Drone Service Station and Depot APIs.

RETURNS
`str`

.name: str

Display name when this ref was returned.

RETURNS
`str`

`.kind: str`

Drone chassis id.

RETURNS

`str`

POSSIBLE VALUES

`"drone_small"`, `"drone_medium"`, `"drone_large"`

`.engine: str`

`"electric"`, `"heli"`, or empty string if no thruster is mounted.

RETURNS

`str`

POSSIBLE VALUES

`""`, `"electric"`, `"heli"`

`.status: str`

Activity/status string when this ref was returned.

RETURNS

`str`

POSSIBLE VALUES

`"idle"`, `"traveling"`, `"charging"`, `"refueling"`, `"waiting_service"`, `"waiting_oil"`, `"waiting_bay"`, `"being_rescued"`, `"holding_weather"`, `"scrambled"`, `"stalled_no_battery"`, `"stalled_no_oil"`, `"stalled_no_route"`

`.x: float`

X coordinate in meters from base when this ref was returned.

RETURNS

`float`

`.y: float`

Y coordinate in meters from base when this ref was returned.

RETURNS

`float`

`.current_station: str`

Station id when this ref was returned, or empty string.

RETURNS

`str`

`.is_docked: bool`

`True` if the drone was parked at a Drone Depot or Drone Service Station when this ref was returned.

RETURNS

`bool`

`.battery_level: float | None`

Electric drone charge fraction **0-1** when this ref was returned, or `None` for heli drones.

RETURNS

`float | None`

`.battery_wh: float | None`

Electric drone charge in Wh when this ref was returned, or `None` for heli drones.

RETURNS

`float | None`

`.battery_capacity: float | None`

Electric drone charge capacity in Wh, or `None` for heli drones.

RETURNS

`float | None`

`.oil_level: float | None`

Heli drone oil fraction **0-1** when this ref was returned, or `None` for electric drones.

RETURNS

`float | None`

`.oil_tons: float | None`

Heli drone oil in tons when this ref was returned, or `None` for electric drones.

RETURNS

`float | None`

`.oil_capacity: float | None`

Heli drone oil capacity in tons, or `None` for electric drones.

RETURNS

`float | None`

`.is_being_rescued: bool`

`True` if a Drone Service Station rescue was active for this drone when this ref was returned.

RETURNS

`bool`

`.rescue_status: str`

Rescue status when this ref was returned: `"none"`, `"outbound"`, `"charging"`, `"carrying"`, or `"returning"`.

RETURNS

`str`

POSSIBLE VALUES

`"none"`, `"outbound"`, `"charging"`, `"carrying"`, `"returning"`

METHODS

`.position() → Position`

Position snapshot from when this ref was returned.

RETURNS

`Position`

MobileUnitRef

Returned by: fleet.mobile_units()

Get `MobileUnitRef` from the APIs listed here. It has no script constructor.

PROPERTIES

.category: str

`"vehicle"` or `"drone"`.

RETURNS
`str`

POSSIBLE VALUES
`"vehicle"`, `"drone"`

.id: str

Stable component id. Use with `get_component(id)` or station APIs.

RETURNS
`str`

.name: str

Display name when this ref was returned.

RETURNS
`str`

`.kind: str`

Unit type, e.g. `"rover"`, `"pioneer"`, or a drone chassis id.

RETURNS

`str`

POSSIBLE VALUES

`"rover"`, `"pioneer"`, `"drone_small"`, `"drone_medium"`, `"drone_large"`

`.status: str`

Activity/status string when this ref was returned.

RETURNS

`str`

POSSIBLE VALUES

`"idle"`, `"moving"`, `"stranded"`, `"scanning"`, `"surveying"`, `"drilling"`, `"discarding"`, `"constructing"`, `"transferring"`, `"charging"`, `"queued"`, `"being_rescued"`, `"traveling"`, `"refueling"`, `"waiting_service"`, `"waiting_oil"`, `"waiting_bay"`, `"holding_weather"`, `"scrambled"`, `"stalled_no_battery"`, `"stalled_no_oil"`, `"stalled_no_route"`

`.x: float`

X coordinate in meters from base when this ref was returned.

RETURNS

`float`

`.y: float`

Y coordinate in meters from base when this ref was returned.

RETURNS

`float`

`.is_docked: bool`

`True` if the unit was parked at a service point/station when this ref was returned.

RETURNS

`bool`

`.is_being_rescued: bool`

`True` if a rescue/recovery mission owned this unit when this ref was returned.

RETURNS

`bool`

`.rescue_status: str`

Rescue status when this ref was returned: `"none"`, `"outbound"`, `"charging"`, `"carrying"`, or `"returning"`.

RETURNS

`str`

POSSIBLE VALUES

`"none"`, `"outbound"`, `"charging"`, `"carrying"`, `"returning"`

METHODS

`.position() → Position`

Position snapshot from when this ref was returned.

RETURNS

`Position`

VehicleRef

Returned by: `fleet.vehicles()`

Get `VehicleRef` from the APIs listed here. It has no script constructor.

PROPERTIES

.category: str

Always `"vehicle"`.

RETURNS
`str`

POSSIBLE VALUES
`"vehicle"`

.id: str

Stable component id. Use with `get_component(id)` or `charging_station.dispatch_rescue(id)`.

RETURNS
`str`

.name: str

Display name when this ref was returned.

RETURNS
`str`

`.kind: str`

`"rover"` or `"pioneer"`.

RETURNS
`str`

POSSIBLE VALUES
`"rover"`, `"pioneer"`

`.status: str`

Current vehicle activity when this ref was returned.

RETURNS
`str`

POSSIBLE VALUES
`"idle"`, `"moving"`, `"stranded"`, `"scanning"`, `"surveying"`, `"drilling"`, `"discarding"`, `"constructing"`, `"transferring"`, `"charging"`, `"queued"`, `"being_rescued"`

`.x: float`

X coordinate in meters from base when this ref was returned.

RETURNS
`float`

`.y: float`

Y coordinate in meters from base when this ref was returned.

RETURNS
`float`

`.battery_level: float`

Battery fraction **0-1** when this ref was returned.

RETURNS

`float`

`.battery_wh: float`

Battery charge in Wh when this ref was returned.

RETURNS

`float`

`.battery_capacity: float`

Maximum battery capacity in Wh.

RETURNS

`float`

`.is_docked: bool`

`True` if the vehicle was parked inside an outpost's **2 by 2-tile footprint**, including the **~2 m** service margin, when this ref was returned. A moving or merely stopped vehicle with an active route is not docked.

RETURNS

`bool`

`.docked_at: str`

Outpost id when this ref was returned, or empty string.

RETURNS

`str`

`.current_station: str`

Vehicle Charging Station id the vehicle was docked at when this ref was returned, or empty string.

RETURNS

`str`

`.is_being_rescued: bool`

`True` if a Vehicle Charging Station rescue was active for this vehicle when this ref was returned.

RETURNS

`bool`

`.rescue_status: str`

Rescue status when this ref was returned: `"none"`, `"outbound"`, `"charging"`, or `"returning"`.

RETURNS

`str`

POSSIBLE VALUES

`"none"`, `"outbound"`, `"charging"`, `"returning"`

METHODS

`.position() → Position`

Position snapshot from when this ref was returned.

RETURNS

`Position`

MountSlot

Returned by: `modules()` on a rover, pioneer, or drone

Get `MountSlot` from the APIs listed here. It has no script constructor.

PROPERTIES

`.index: int`

Slot position on the chassis, 0-indexed.

RETURNS

`int`

`.type: str`

Slot's type: determines which modules fit (`nav`, `sonar_basic`, `drill_basic`, `universal`, `thruster`, or `drone_module`).

RETURNS

`str`

POSSIBLE VALUES

`"nav"`, `"sonar_basic"`, `"drill_basic"`, `"universal"`, `"thruster"`, `"drone_module"`

`.module_id: str | None`

Module id currently mounted in this slot, or `None` if empty.

RETURNS

`str | None`

`.internal_count: int`

Number of internal slots this mounted module exposes. `0` for non-container modules.

RETURNS

`int`

`.internal_items: list[str | None]`

List of item ids currently installed in this module's internal slots. `None` entries mark empty internal slots.

RETURNS

`list[str | None]`

NavModule

Returned by: `self.nav` (vehicles)

Get `NavModule` from the APIs listed here. It has no script constructor.

METHODS

`.set_target(x: float, y: float) → ActionResult`

Set target coordinates to drive toward.

PARAMETERS

Name	Type	Description
`x`	`float`	Target X coordinate
`y`	`float`	Target Y coordinate

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_mounted"`	rejection	The required module is not mounted.
`"invalid"`	rejection	One or more supplied arguments are outside the accepted domain.
`"out_of_bounds"`	rejection	The requested position lies outside the valid world bounds.
`"busy"`	transient	The component is already performing another operation.

`.set_throttle(power: float) → ActionResult`

Set throttle (0.0-1.0; clamped).

PARAMETERS

Name	Type	Description
`power`	`float`	Throttle power (0.0-1.0)

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_mounted"`	● rejection	The required module is not mounted.
`"busy"`	● transient	The component is already performing another operation.

`.throttle() → float`

Current throttle setpoint (**0.0-1.0**). Returns the value the script last wrote via `set_throttle(...)`. Distinct from `get_speed()`: `throttle()` is intent (does not change tick-to-tick); `get_speed()` is what the vehicle actually moved last tick.

RETURNS

`float`

`.brake() → ActionResult`

Stop the vehicle (throttle set to 0).

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.
`"not_mounted"`	● rejection	The required module is not mounted.

`.get_position() → Position`

Vehicle position as a `Position` object with `.x` and `.y`.

RETURNS

`Position`

`.get_speed() → float`

Vehicle speed in m/h.

RETURNS

`float`

`.get_distance_to(x: float, y: float) → float`

Distance (meters) from the vehicle to the given point. The vehicle has arrived once this is 2 or less; it rarely reaches exactly zero. A wait loop therefore keeps going `while self.nav.get_distance_to(x, y) > 2:`. When the next action targets a building, route to its `BuildingRef.position`.

PARAMETERS

Name	Type	Description
`x`	`float`	Target X coordinate
`y`	`float`	Target Y coordinate

RETURNS

`float`

`.speed_multiplier() → float`

Top-speed multiplier: **1.0** basic, or **1 + mounted Sport Nav count** on Pioneer. Speed only; range still depends on battery, throttle, cargo load, and movement draw.

RETURNS

`float`

SonarModule

Returned by: `self.sonar` (vehicles)

Get `SonarModule` from the APIs listed here. It has no script constructor.

METHODS

`.scan()` → `SonarScanResult`

Point-sweep for nearby sites. A sweep can complete without finding any compatible contacts. Mineral contacts must be within sonar range, and a mineral harder than the sonar's hardness limit stays unidentified; thermal, water, oil, and exotic contacts also require matching research. A sweep that succeeded can still leave a contact unidentified: it is reported separately, with its coordinates and a reason, and stays unscanned until the right instrument reaches it. The sweep updates the Journal with newly classified sites. A stale captured module reference raises `ReferenceError`.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`SonarScanResult`	`.status`, `.message`	`.sites`, `.blocked`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The sonar sweep completed. `result.sites` contains every compatible contact found; an empty list means there were no compatible contacts in range.
`"too_hard"`	partial	The completed sweep detected a nearby unresolved contact above the mounted sonar's hardness limit.
`"tier_too_low"`	partial	The completed sweep detected a nearby unresolved contact that the mounted sonar tier cannot classify.
`"research_required"`	partial	The completed sweep detected a nearby unresolved contact whose classification research is not unlocked.
`"wrong_scanner"`	partial	The completed sweep detected a nearby contact that vehicle sonar cannot classify.
`"busy"`	transient	Another field action currently occupies the vehicle.
`"no_power"`	rejection	The vehicle battery does not contain enough energy for a sonar sweep.

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.survey(site: str | Site | IdRecord) → SurveyResult`

Reveal the details available for a productive site. With Seismic Sonar and Advanced Oil Extraction, an inert `GeologicalAnomaly` receives a paid deep-oil survey even if ordinary sonar already resolved it. Confirmed reservoirs become Oil Wells at the same coordinates. Known dry formations are free to re-survey. Other inert surveys are free. Re-surveying is also free unless a deeper sonar tier can reveal more. Malformed site values raise `ValueError`; a stale captured module reference raises `ReferenceError`.

PARAMETERS

Name	Type	Description
`site`	`str` | `Site` | `IdRecord`	Site id string or a `Site` from `scan()`

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`SurveyResult`	`.status`, `.message`	`.site`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Surveyed `site_id` successfully.
`"busy"`	transient	Another field action currently occupies the vehicle.
`"not_discovered"`	rejection	Site `site_id` is not in this planet's discovered-site journal.
`"research_required"`	rejection	Site `site_id` requires research that is not currently unlocked.
`"tier_too_low"`	rejection	The mounted sonar tier cannot survey site `site_id`.
`"out_of_range"`	rejection	Site `site_id` is outside the mounted sonar's current range.
`"too_hard"`	rejection	Site `site_id` exceeds the mounted sonar's mineral hardness limit.
`"no_power"`	rejection	The vehicle battery does not contain enough energy to survey site `site_id`.

RAISES

Exception	Condition
`ValueError`	`survey()` requires a non-empty site id or a `Site` object.
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.range() → float`

Sonar range in meters (**50** basic, **180** Extended, **280** Deep or Seismic). A stale captured module reference raises `ReferenceError`.

RETURNS

`float`

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.hardness_limit() → int`

Max mineral hardness this sonar can identify (**1** basic, **3** Extended, **4** Deep or Seismic). A stale captured module reference raises `ReferenceError`.

RETURNS

`int`

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

`.tier() → str`

Survey-depth tier granted by the mounted sonar: `"basic"` / `"wide"` / `"deep"` / `"seismic"`. Determines thermal/exotic survey detail; Deep or Seismic Sonar is also required to discover oil wells once **Petroleum Survey** is unlocked. A stale captured module reference raises `ReferenceError`.

RETURNS

`str`

POSSIBLE VALUES

`"basic"`, `"wide"`, `"deep"`, `"seismic"`

RAISES

Exception	Condition
`ReferenceError`	This `SonarModule` reference is stale because its module is no longer mounted. Read `self.sonar` again after mounting a sonar.

SonarScanResult

Returned by: SonarModule.scan()

Get `SonarScanResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Stable sweep outcome. An empty `.sites` list with `"ok"` is a completed sweep with no contacts requiring player action.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"too_hard"`, `"tier_too_low"`, `"research_required"`, `"wrong_scanner"`, `"busy"`, `"no_power"`

.message: str

Player-readable explanation of the sweep outcome.

RETURNS

`str`

.sites: list[Site]

Sites resolved by this completed sweep; empty when no compatible site was returned. Contacts the sweep detected but could not identify are in `.blocked` instead, never here.

RETURNS

`list[Site]`

`.blocked: list[BlockedContact]`

Contacts the sweep detected but this sonar cannot identify, nearest first, and empty when everything in range was identified. A contact appears here instead of in `.sites`, so the two lists never overlap. Each entry carries `.x`, `.y`, a `.reason` code, and a readable `.message`. This is how a script tells an unreachable contact apart from empty ground: `.status` describes the sweep itself and stays `"ok"` whenever any site was identified.

RETURNS

`list[BlockedContact]`

SurveyResult

Returned by: SonarModule.survey()

Get `SurveyResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"busy"`, `"not_discovered"`, `"research_required"`, `"tier_too_low"`, `"out_of_range"`, `"too_hard"`, or `"no_power"`.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"busy"`, `"not_discovered"`, `"research_required"`, `"tier_too_low"`, `"out_of_range"`, `"too_hard"`, `"no_power"`

.message: str

Player-readable, exact explanation of the survey outcome.

RETURNS

`str`

.site: Site | None

Surveyed concrete `Site`, or `None` when the survey was rejected.

RETURNS

`Site | None`

ComputerDeployResult

Returned by: computer.deploy()

Get `ComputerDeployResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Stable deploy outcome.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"no_kit"`, `"locked"`, `"not_deployable"`, `"deploy_limit"`, `"location_not_found"`, `"wrong_biome_for_machine"`, `"duplicate_outpost_machine"`, `"missing_drone_station"`, `"drone_station_full"`

.message: str

Localized explanation of what happened, for logs and the player.

RETURNS

`str`

`.machine_id: str`

Id of the machine that was created, empty when nothing was deployed. Pass it to `get_component()` to read the new machine.

RETURNS

`str`

Construction

Returned by: .pending_constructions() / .active_constructions() / .paused_constructions()

Get `Construction` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique blueprint id. Pass to `self.constructor.execute(id)`.

RETURNS

`str`

`.kind: str`

What's being built, upgraded or removed. One of `"pipe"` / `"power_line"` / `"gas_bridge"` / `"liquid_bridge"` / `"power_bridge"` / `"deconstruct"` / `"upgrade"` / `"outpost"` / `"thermal_cap"` / `"water_pump"` / `"oil_pump"` / `"exotic_gas_cap"` / `"exotic_spring_tap"` / `"mining_drill"` / `"mining_drill_industrial"` / `"mining_drill_heavy"`.

RETURNS

`str`

POSSIBLE VALUES

`"pipe"`, `"power_line"`, `"gas_bridge"`, `"liquid_bridge"`, `"power_bridge"`, `"upgrade"`, `"deconstruct"`, `"outpost"`, `"thermal_cap"`, `"water_pump"`, `"oil_pump"`, `"exotic_gas_cap"`, `"exotic_spring_tap"`, `"mining_drill"`, `"mining_drill_industrial"`, `"mining_drill_heavy"`

`.medium: str | None`

Physical utility layer: `"gas"`, `"liquid"`, or `"power"`; `None` for point structures. Pipe jobs keep `.kind == "pipe"`, so use this field to distinguish Gas Pipe from Liquid Pipe. Deconstruction reports the target's utility layer.

RETURNS

`str | None`

POSSIBLE VALUES

`"gas"`, `"liquid"`, `"power"`

`.position: Position`

World coordinates as a `Position` snapshot (`.x`, `.y`). Pioneer drives here to start or resume construction. Pump upgrades use the existing pump's service area.

RETURNS

`Position`

`.progress: float`

Build completion **0-1** when this `Construction` snapshot was returned. Useful especially for paused work: sort by progress to resume the most-completed first; re-query construction lists for fresh progress.

RETURNS

`float`

`.required_item: str | None`

Item id the Pioneer must carry before executing this job, or `None` for deconstruction and paused jobs that have already started.

RETURNS

`str | None`

`.required_count: int`

Units of `.required_item` needed to accept this job. Most single-piece jobs need 1; deconstruction needs 0.

RETURNS

`int`

FluidPort

Returned by: any `<fluid>_in` / `<fluid>_out` property on a flow-network machine

Get `FluidPort` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `FluidConnection`

METHODS

`.connect(target: str) → ActionResult`

Record or replace this port's one declared target, using a stable machine id or display name. The target must expose a compatible opposite-direction port. Either the provider or consumer may declare the relationship; one declaration is enough. Local machines transfer directly, while remote intent waits for any completed conflict-free pipe network of the same medium reaching both anchors.

PARAMETERS

Name	Type	Description
`target`	`str`	Stable id or display name of the provider or consumer

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"not_found"`	rejection	The requested object, target, or record does not exist.
`"incompatible"`	rejection	The selected endpoints or values are incompatible.

`.disconnect() → ActionResult`

Clear only this port's declared target. Buffered fluid remains. If the peer independently declared the same relationship, that reverse declaration remains active.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The operation completed successfully.

`.connected_to() → str`

Display name of the target declared by this port, or empty string. It does not list compatible reverse declarations owned by peer ports.

RETURNS

`str`

`.connected_id() → str`

Stable id of the target declared by this port, or empty string. `connect()` accepts an id or display name, but this returns the resolved stable id. Compare it with the target's stable id; `connected_to()` returns its renameable display name. For every effective peer, including declarations owned by the other side, use `connections()`.

RETURNS

`str`

`.connections() → list[FluidConnection]`

Read-only snapshots of every effective peer relationship on this port, including declarations authored by peer ports. Each `FluidConnection` reports the peer, exact fluid when known, declaration ownership, and structural state. Pipe ids are deliberately not exposed or selected.

RETURNS

`list[FluidConnection]`

`.level() → float`

Current tons of fluid buffered at this port. Pass-through source ports such as Pump outputs store nothing and therefore read **0**.

RETURNS

`float`

`.capacity() → float`

Max tons this port's buffer can hold.

RETURNS

`float`

`.flow_rate() → float`

Current total live flow in **t/h**. **0** may mean idle, starved, full, unreachable, conflicted, or waiting across a simulation timing boundary; it does not erase the connection or pipe identity.

RETURNS

`float`

FluidConnection

Returned by: FluidPort.connections()

Get `FluidConnection` from the APIs listed here. It has no script constructor.

PROPERTIES

.machine_id: str

Stable id of the peer machine.

RETURNS
`str`

.machine_name: str

Current display name of the peer machine.

RETURNS
`str`

`.fluid: str | None`

Exact fluid established by this relationship, or `None` while the relationship is `"neutral"` or `"incompatible"`.

RETURNS

`str | None`

POSSIBLE VALUES

`"steam"`, `"water"`, `"oil"`, `"frozen_essence"`, `"coastal_essence"`, `"geothermal_essence"`, `"volcanic_essence"`, `"deep_essence"`, `"ammonia"`, `"swamp_gas"`, `"raw_sulfur_gas"`, `"sulfur_gas"`, `"raw_chlorine"`, `"chlorine"`, `"brine"`, `"raw_cryofluid"`, `"cryofluid"`, `"raw_quicksilver"`, `"quicksilver"`

`.declared_by: str`

Who owns the durable declaration: `"self"`, `"peer"`, or `"both"`. One side is sufficient for transport.

RETURNS

`str`

POSSIBLE VALUES

`"self"`, `"peer"`, `"both"`

`.state: str`

Connection state: `"local"` for a direct same-outpost link; `"ready"` for a usable remote pipe network; `"unreachable"` when no completed pipe network connects both ends; `"conflict"` when that pipe network carries different fluid connections; `"neutral"` before a fluid is known; or `"incompatible"` when the two fluids disagree.

RETURNS

`str`

POSSIBLE VALUES

`"local"`, `"ready"`, `"unreachable"`, `"conflict"`, `"neutral"`, `"incompatible"`

Pipe

Returned by: list_pipes() / get_pipe(pipe_id)

Get `Pipe` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Unique pipe identifier.

RETURNS

`str`

METHODS

`.start() → Position | None`

Geometric start coordinate of this pipe piece as a `Position`. This is construction geometry, not flow direction. Returns `None` only if malformed state contains no segment geometry.

RETURNS

`Position | None`

`.end() → Position | None`

Geometric end coordinate of this pipe piece as a `Position`. This is construction geometry, not flow direction. Returns `None` only if malformed state contains no segment geometry.

RETURNS

`Position | None`

`.type() → str`

Pipe hardware medium: `"gas"` or `"liquid"`. Use `contents()` for the exact substance established by complete provider-consumer connections.

RETURNS

`str`

POSSIBLE VALUES

`"gas"`, `"liquid"`

`.contents() → str | None`

The one exact fluid established by complete player connections whose locations this pipe network reaches, such as `"steam"`, `"water"`, or `"oil"`. Returns `None` when there is no complete connection or multiple exact substances conflict. Activity, power, throttle, flow, and headroom do not change this identity.

RETURNS

`str | None`

POSSIBLE VALUES

`"steam"`, `"water"`, `"oil"`, `"frozen_essence"`, `"coastal_essence"`, `"geothermal_essence"`, `"volcanic_essence"`, `"deep_essence"`, `"ammonia"`, `"swamp_gas"`, `"raw_sulfur_gas"`, `"sulfur_gas"`, `"raw_chlorine"`, `"chlorine"`, `"brine"`, `"raw_cryofluid"`, `"cryofluid"`, `"raw_quicksilver"`, `"quicksilver"`

`.conflicting_contents() → list[str]`

Sorted exact substances established by complete provider-consumer connections when more than one uses this pipe network, or an empty list. A non-empty result means flow is halted.

RETURNS

`list[str]`

`.connections() → list[dict[str, str]]`

Diagnostic machine-port claims established on this pipe network. Each dictionary contains `machine_id`, `port`, `direction`, `fluid`, and a representative `pipe_id` from the pipe network; players never connect to that pipe id directly.

RETURNS

`list[dict[str, str]]`

`.incompatible_sinks() → list[str]`

Ids of directly connected consumers that cannot accept `contents()`. Those consumers receive nothing, while compatible branches continue flowing. Live read.

RETURNS

`list[str]`

`.is_complete() → bool`

Boolean: `True` once the Constructor has finished laying the pipe and flow can run. Live read.

RETURNS

`bool`

`.length() → int`

Total length of the pipe in meters, summed over every H/V segment.

RETURNS

`int`

`.laying_head() → list[int] | None`

`[x, y]` coordinates of the current laying head while incomplete, or `None` once complete. Live read.

RETURNS

`list[int] | None`

`.flow_rate() → float`

Tons per world hour (t/h) currently moving through the pipe. **0** while incomplete, stalled, source-empty, or conflicted. Live read.

RETURNS

`float`

`.state() → str`

Current pipe state: one of `"flowing"` / `"stalled"` / `"incomplete"` / `"no_source"` / `"conflict"`. Live read.

RETURNS

`str`

POSSIBLE VALUES

`"flowing"`, `"stalled"`, `"incomplete"`, `"no_source"`, `"conflict"`

PowerGrid

Returned by: power_control.grids() / power_control.grid(target_id)

Get `PowerGrid` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `PowerGridMember`

PROPERTIES

`.anchor_id: str`

Deterministic entity id that identifies this connected grid right now and can be passed to `power.grid(...)`. Re-discover grids after rewiring because merging or splitting a grid can change its anchor.

RETURNS

`str`

`.outpost_ids: list[str]`

Outpost ids connected to this grid, sorted for deterministic iteration. The list is empty for a field-only grid.

RETURNS

`list[str]`

`.machine_ids: list[str]`

Building and field power-structure ids assigned to this grid, sorted for deterministic iteration. Mobile units and ship equipment are excluded because their deployment location is not an electrical connection.

RETURNS

`list[str]`

`.members: list[PowerGridMember]`

Power details for every building and field power structure on this grid as `PowerGridMember` snapshots. Use each member's `.id` with `get_component(...)` for type-specific recipe, material, or control methods.

RETURNS

`list[PowerGridMember]`

`.generated: float`

Power currently supplied by this grid's active generators in W when this snapshot was returned.

RETURNS

`float`

`.consumed: float`

Power currently drawn by this grid's active consumers in W when this snapshot was returned.

RETURNS

`float`

`.net: float`

Current grid generation minus consumption in W. A negative value means the current load needs stored energy.

RETURNS

`float`

`.stored: float`

Energy held in conventional batteries on this grid in Wh.

RETURNS

`float`

`.capacity: float`

Total conventional battery capacity on this grid in Wh.

RETURNS

`float`

`.reserve_stored: float`

Energy currently banked by Lightning Rods on this grid in Wh. This reserve is spent only after conventional batteries and never absorbs ordinary generation surplus.

RETURNS

`float`

`.reserve_capacity: float`

Total Lightning Rod reserve capacity on this grid in Wh.

RETURNS

`float`

`.has_generator: bool`

`True` when this grid contains a generator, even when its current output is 0 W.

RETURNS

`bool`

PowerGridMember

Returned by: PowerGrid.members

Get `PowerGridMember` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Stable stationary machine instance id. Pass it to `get_component(...)`, `power.grid(...)`, or breaker APIs.

RETURNS

`str`

`.name: str`

Display name when this snapshot was returned.

RETURNS

`str`

`.type_id: str`

Stable machine type id, such as `"solar_generator"`, `"battery"`, or `"smelter"`.

RETURNS

`str`

.outpost_id: str

Owning outpost id, or an empty string for an independently placed field structure.

RETURNS

`str`

.powered: bool

`True` when the machine's power state was on as this snapshot was returned.

RETURNS

`bool`

.roles: list[str]

Power capabilities of this machine: any combination of `"generator"`, `"consumer"`, `"storage"`, and `"reserve"`. An empty list means the machine has no direct electrical role but is installed at a connected outpost.

RETURNS

`list[str]`

.generated: float

Power this machine is currently supplying in W. A generator may report 0 W because it is off, idle, unfueled, or lacks sunlight.

RETURNS

`float`

.consumed: float

Power this machine is currently drawing in W. Powered-off and idle machines report 0 W.

RETURNS

`float`

.stored: float

Energy held by this conventional battery in Wh, or 0 for another machine role.

RETURNS

`float`

.capacity: float

Conventional battery capacity in Wh, or 0 for another machine role.

RETURNS

`float`

.reserve_stored: float

Lightning reserve held by this machine in Wh, or 0 for another machine role.

RETURNS

`float`

.reserve_capacity: float

Lightning reserve capacity in Wh, or 0 for another machine role.

RETURNS

`float`

PowerSummary

Returned by: power_control.total()

Get `PowerSummary` from the APIs listed here. It has no script constructor.

PROPERTIES

.grid_count: int

Number of independent completed power grids included in this snapshot.

RETURNS
`int`

.generated: float

Power currently supplied across every grid in W when this snapshot was returned.

RETURNS
`float`

.consumed: float

Power currently drawn across every grid in W when this snapshot was returned.

RETURNS
`float`

`.net: float`

Planet-wide generation minus consumption in W across every grid.

RETURNS

`float`

`.stored: float`

Energy held in conventional batteries across every grid in Wh.

RETURNS

`float`

`.capacity: float`

Total conventional battery capacity across every grid in Wh.

RETURNS

`float`

`.reserve_stored: float`

Energy banked by Lightning Rods across every grid in Wh.

RETURNS

`float`

`.reserve_capacity: float`

Total Lightning Rod reserve capacity across every grid in Wh.

RETURNS

`float`

RunControlStatus

Returned by: run.status()

Get `RunControlStatus` from the APIs listed here. It has no script constructor.

PROPERTIES

.script_id: str

The script attached to the requested machine slot.

RETURNS
`str`

.state: str

Current execution state: `"idle"`, `"running"`, `"paused"`, `"error"`, or `"completed"`. Sleeping and waiting for actions count as `"running"`.

RETURNS
`str`

POSSIBLE VALUES
`"idle"`, `"running"`, `"paused"`, `"error"`, `"completed"`

.variant_id: str

Identity of the assigned variant, suitable for `run.apply_variant()`.

RETURNS
`str`

`.variant_name: str`

Name of the assigned variant.

RETURNS

`str`

`.modified: bool`

`True` when the machine's working code differs from its saved named variant. Editing a shared variant elsewhere can cause this difference. While `Main` is the active variant, this is always `False`, because `Main` tracks its own working code.

RETURNS

`bool`

`.source_pending: bool`

`True` when a running or paused script still has an older code version loaded than its current working code.

RETURNS

`bool`

`.editor_busy: bool`

`True` while an editor has pending changes or an unresolved conflict, or a recovered source draft differs from the current code.

RETURNS

`bool`

ScriptVariantRef

Returned by: run.variants()

Get `ScriptVariantRef` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Variant identifier passed to `run.apply_variant()`. Shared variants can be applied to compatible machines. `Main` belongs only to its original script. Renaming changes the id; reusing a deleted name in the same catalog reuses its id.

RETURNS

`str`

`.name: str`

The variant name shown in the editor.

RETURNS

`str`

`.description: str`

The description saved with the variant.

RETURNS

`str`

BroadcastInfo

Returned by: `comms.latest_info(channel)` ; `comms.wait_broadcast(channel).broadcast` after `status == "ok"`

Get `BroadcastInfo` from the APIs listed here. It has no script constructor.

PROPERTIES

`.value: JsonValue`

A copy of this broadcast's value, including `None` if that was the value published. From `latest_info()` it is the latest value; from `wait_broadcast()` it is the first publication captured by that wait.

RETURNS

`JsonValue`

`.sender: str | None`

Who published this broadcast, or `None` if the saved broadcast has no sender information.

RETURNS

`str | None`

`.age_seconds: float | None`

Simulation seconds since this broadcast, sampled when `latest_info()` is read or `wait_broadcast()` resumes. Uses the same time base as `sleep()` and `clock.elapsed_seconds()`; pausing the simulation freezes the age. `None` means the timestamp is missing or invalid. The returned age is a snapshot.

RETURNS

`float | None`

CommsMessage

Returned by: `comms.pending(channel)` list entries; `comms.receive(channel).packet`, `comms.wait(channel).packet`, or `comms.wait_any(channels).packet` after `status == "ok"`

Get `CommsMessage` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: int`

Monotonic message id assigned by the Signal Bus.

RETURNS

`int`

`.sender: str`

Script owner id that sent the message.

RETURNS

`str`

`.tick: int`

Game tick when the message was sent.

RETURNS

`int`

`.value: JsonValue`

JSON-safe message value: `None`, boolean, number, string, `list`, or `dict` with string keys.

RETURNS

`JsonValue`

DockSlot

Returned by: supply_dock.slots()

Get `DockSlot` from the APIs listed here. It has no script constructor.

PROPERTIES

`.index: int`

Zero-based slot index (**0-4**): stable across calls.

RETURNS
`int`

`.item_id: str | None`

Item id currently occupying this slot, or `None` if the slot is empty.

RETURNS
`str | None`

`.count: int`

Units currently stored in this slot.

RETURNS
`int`

Order

Returned by: `orders.list_orders()` / `orders.list_upcoming_orders()` / `orders.list_weekly_orders()` / `orders.get_order()` / `orders.completed_orders()` (Earth Orders)

Get `Order` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Immutable Earth Order id. Useful for logs and dashboards; Supply Dock scripts use it to identify the order currently assigned to their dock.

RETURNS

`str`

`.name: str`

Pre-translated display name of the Earth Order.

RETURNS

`str`

`.requires: dict[str, int]`

A `dict` `{item_id: count}` of what Earth is demanding. Use `.keys()`, `.values()`, `.items()`, or index directly: `order.requires["iron_ore"]`.

RETURNS

`dict[str, int]`

`.shipped: dict[str, int]`

A `dict` `{item_id: count}` of how many units of each item have already landed. Empty for upcoming orders. Use `.get(item_id, 0)` to read safely.

RETURNS

`dict[str, int]`

`.reward_credits: int`

Credit payout. Upcoming and active campaign orders quote the payout at your current contractor reputation; the final payout may change as reputation grows. Completed campaign orders report the recorded payout. Weekly rewards are fixed.

RETURNS

`int`

`.reward_kind: str | None`

Variable-reward kind: `"recipe"` / `"tech"`, or `None` for credits-only.

RETURNS

`str | None`

POSSIBLE VALUES

`"recipe"`, `"tech"`

`.reward_label: str | None`

Pre-translated description of the variable reward, or `None` for credits-only Earth Orders.

RETURNS

`str | None`

`.status: str`

`"upcoming"` for a future campaign order available for planning, `"active"` for a current order available for fulfillment, or `"completed"` once delivered. Weekly Earth Orders are never `"upcoming"`.

RETURNS
`str`

POSSIBLE VALUES
`"upcoming"`, `"active"`, `"completed"`

`.kind: str`

Order source: `"campaign"` or `"weekly"`.

RETURNS
`str`

POSSIBLE VALUES
`"campaign"`, `"weekly"`

`.expires_day: int | None`

Day when a Weekly Earth Order expires, or `None` for contractor campaign orders.

RETURNS
`int | None`

`.contractor_id: str | None`

Id of the issuing contractor, or `None` for Weekly Earth Orders. Stable across saves and useful for branching on specific campaign partners.

RETURNS
`str | None`

POSSIBLE VALUES
`"helios_orbital"`, `"spire_research"`, `"vestibule_logistics"`

`.contractor_name: str | None`

Pre-translated issuing contractor name, or `None` for Weekly Earth Orders.

RETURNS

`str | None`

ReceiveResult

Returned by: comms.receive(); comms.wait()

Get `ReceiveResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"` when a queued message was consumed; `"empty"` when no id was requested and the queue was empty; `"not_found"` when the requested id was absent; or `"invalid_channel"` for an invalid channel id.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"empty"`, `"not_found"`, `"invalid_channel"`

.message: str

Player-readable explanation of the receive outcome.

RETURNS

`str`

.packet: CommsMessage | None

The consumed `CommsMessage` when `status == "ok"`, otherwise `None`. When a message id was requested, this packet has that exact id.

RETURNS

`CommsMessage | None`

SendResult

Returned by: comms.send()

Get `SendResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Send outcome. `"ok"` means a message was queued; every other outcome means no message was sent.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"invalid_channel"`, `"channel_limit"`, `"queue_full"`, `"id_exhausted"`, `"invalid_value"`

`.message: str`

Player-readable explanation of the send outcome.

RETURNS

`str`

`.message_id: int | None`

The queued message's positive integer id when `status == "ok"`, or `None` when no message was sent. Matches `.id` in `pending()` and `receive().packet`. Use it with `receive()`, `cancel()`, or `update()` to target that exact request.

RETURNS

`int | None`

TransmitterInfo

Returned by: transmitter.get_info()

Get `TransmitterInfo` from the APIs listed here. It has no script constructor.

PROPERTIES

`.connected: bool`

`True` if connected to a target.

RETURNS
`bool`

`.target: str`

Connected planet id, or `"none"` when disconnected.

RETURNS
`str`

POSSIBLE VALUES
`"none"`

WaitAnyResult

Returned by: comms.wait_any()

Get `WaitAnyResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"ok"` when one queued message was consumed, or `"invalid_channel"` when any listed channel id is invalid. Valid empty queues keep waiting.

RETURNS	POSSIBLE VALUES
`str`	`"ok"`, `"invalid_channel"`

`.message: str`

Player-readable explanation of the wait outcome.

RETURNS

`str`

`.channel: str | None`

The channel the consumed message came from when `status == "ok"`; otherwise `None`.

RETURNS

`str | None`

`.packet: CommsMessage | None`

The consumed `CommsMessage` when `status == "ok"`; otherwise `None`. Its id, sender, tick, and copied value belong to the selected channel.

RETURNS

`CommsMessage | None`

WaitBroadcastResult

Returned by: comms.wait_broadcast()

Get `WaitBroadcastResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"ok"` when a new broadcast was captured, or `"invalid_channel"` for an invalid channel id. A valid channel keeps waiting until a new broadcast arrives.

RETURNS	POSSIBLE VALUES
`str`	`"ok"`, `"invalid_channel"`

`.message: str`

Player-readable explanation of the broadcast wait outcome.

RETURNS

`str`

`.broadcast: BroadcastInfo | None`

The captured `BroadcastInfo` when `status == "ok"`; otherwise `None`. It contains the publication's copied value, sender, and age when the script resumes.

RETURNS

`BroadcastInfo | None`

Panel

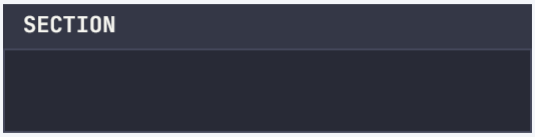
Returned by: `panel` (panel scripts only). Create one on the **Control Room** page; see `custom_panels`

Get `Panel` from the APIs listed here. It has no script constructor.

METHODS

`card(x: float, y: float, w: float, h: float, title?: str) → None`

Bordered subsection with optional title bar. Use to group related content visually, mirrors the dashboard's card aesthetic.



PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`title`	`str`	Optional card title

RETURNS

`None`

`divider(x1: float, y1: float, x2: float, y2: float) → None`

Horizontal or vertical separator line in the muted border color. Use to break a panel into visual zones.

PARAMETERS

Name	Type	Description
`x1`	`float`	Start X (pixels)
`y1`	`float`	Start Y (pixels)
`x2`	`float`	End X (pixels)
`y2`	`float`	End Y (pixels)

RETURNS

`None`

`label(x: float, y: float, text: str, style?: str, color?: str) → None`

Semantically styled text. `style` accepts `"title"` (bright, bold), `"caption"` (muted, uppercase), `"muted"` (secondary), `"value"` (numeric readout). Defaults to `"title"`. Use instead of `draw_text` when you want the dashboard's text hierarchy applied automatically.

PARAMETERS

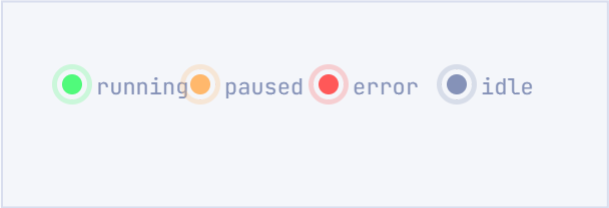
Name	Type	Description
`x`	`float`	Left edge of the text (pixels)
`y`	`float`	Vertical middle of the text (pixels)
`text`	`str`	Label text
`style`	`str`	Text style preset.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named). Defaults to the style's color.

RETURNS

`None`

`status_dot(x: float, y: float, r: float, status: str) → None`

Colored disc resolved from a status string. Recognized values: `"running"` (green), `"paused"` (warning), `"error"` (error red), `"idle"` (muted). It also takes any color a `color` parameter takes, such as `"success"` or `"#ff8800"`. Renders with a subtle outer glow ring. Useful for per-row indicators in machine lists.



PARAMETERS

Name	Type	Description
`x`	`float`	Center X (pixels)
`y`	`float`	Center Y (pixels)
`r`	`float`	Radius (pixels)
`status`	`str`	Status preset (`"running"`, `"paused"`, `"error"`, `"idle"`), or any theme token or CSS color.

RETURNS

`None`

`toggle(x: float, y: float, on: bool, label?: str, size?: float) → None`

Green/grey power-toggle pill matching the dashboard's machine on/off control. This widget only displays the `on` value you pass in; use `panel.switch(...)` for a clickable control.

PARAMETERS

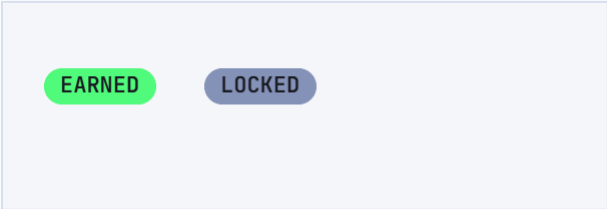
Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`on`	`bool`	Whether the toggle is on
`label`	`str`	Optional label
`size`	`float`	Pill height in pixels (default 18). The width, knob and label follow it.

RETURNS

`None`

`pill(x: float, y: float, text: str, color?: str, size?: float) → None`

Rounded badge with text, achievement-style. `color` accepts theme tokens (`"accent"`, `"success"`, `"warning"`, `"error"`, `"text-muted"`) or hex. Use for status labels or category tags.



PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`text`	`str`	Pill text
`color`	`str`	Theme token (e.g. `"accent"`), or any CSS color (hex, `rgb()`, `hsl()`, named).
`size`	`float`	Pill height in pixels (default 18). The text and padding follow it; the width still comes from the text.

RETURNS

`None`

`counter(x: float, y: float, value: object, label?: str, size?: float, color?: str) → None`

Big-number stat block, large value on top, small uppercase label below. Use for headline numbers (credits, day count, ingot inventory). `size` defaults to **24**.

PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Baseline of the number (pixels). The number sits above it and the label hangs below.
`value`	`object`	Value to display
`label`	`str`	Optional label
`size`	`float`	Optional numeric font size in pixels (default 24)
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named), for the number. Defaults to `"text-bright"`; the label stays muted.

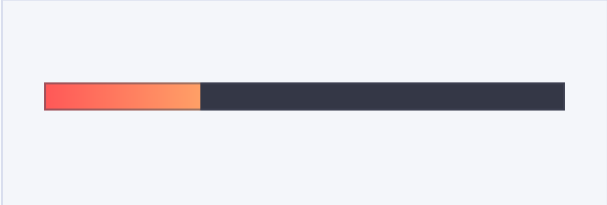
RETURNS

`None`

`progress_bar(x: float, y: float, w: float, h: float, fraction: float, color?: str) → None`

Horizontal fill bar with track + filled accent. `fraction` clamps to **0-1**. `color` defaults to `"accent"`; use `"success"`, `"warning"`, `"error"` for traffic-light cues.

`color` also takes three gradient presets, the meters on machine cards: `"good-high"` runs red, orange, green, for a value that is good when high (battery charge); `"good-low"` runs green, orange, red, for one that is good when low (heat, waste); `"neutral"` runs accent to green, for plain progress. The gradient spans the whole bar and the fill reveals it, so a low `"good-high"` bar shows only its red end. The colors follow the player's theme.



PARAMETERS

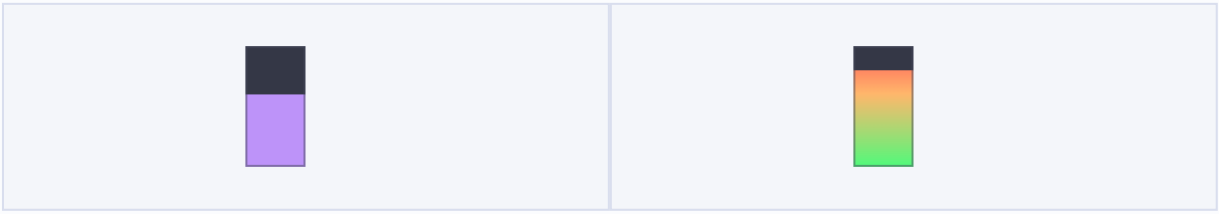
Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`fraction`	`float`	Fill (0-1)
`color`	`str`	Theme token, gradient preset (`"good-high"`, `"good-low"`, `"neutral"`), or any CSS color (hex, `rgb()`, `hsl()`, named).

RETURNS

`None`

`vertical_bar(x: float, y: float, w: float, h: float, fraction: float, color?: str) → None`

Vertical fill bar, fills from the bottom up. Same `fraction` and `color` rules as `progress_bar`, including the gradient presets `"good-high"`, `"good-low"` and `"neutral"`, which run from the bottom up. Use when the panel layout favors verticality (multi-tank stacks, atmospheric stacks).



PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`fraction`	`float`	Fill (0-1)
`color`	`str`	Theme token, gradient preset (`"good-high"`, `"good-low"`, `"neutral"`), or any CSS color (hex, `rgb()`, `hsl()`, named).

RETURNS

`None`

`bar_chart(x: float, y: float, w: float, h: float, values: list[float], max?: float, labels?: list[str], color?: str) → None`

Multi-bar comparison chart, themed alternating colors, optional labels under each bar. `max` is optional, omit to auto-scale to the largest value. Pass `color` to paint every bar one color instead.

PARAMETERS

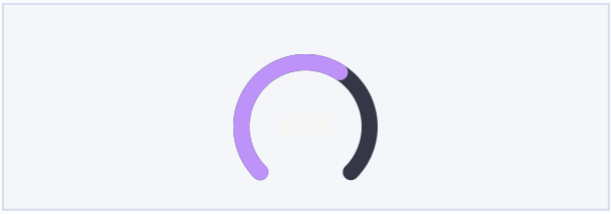
Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`values`	`list[float]`	Numeric values to plot
`max`	`float`	Optional maximum value
`labels`	`list[str]`	Optional labels
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named), for every bar. Without it the bars alternate `"accent"` and `"warning"`.

RETURNS

`None`

`gauge(x: float, y: float, radius: float, fraction: float, label?: str, color?: str) → None`

Three-quarter-circle dial with arc fill. `fraction` clamps to **0-1**, sweeping **270°** from bottom-left around to bottom-right. Optional center label sits in the middle (typically the value as text). The arc's color defaults to `"accent"`; use `"success"`, `"warning"`, `"error"` for traffic-light cues.



PARAMETERS

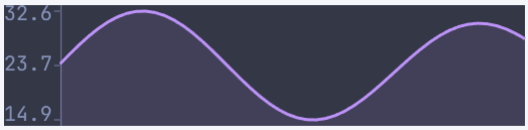
Name	Type	Description
`x`	`float`	Center X (pixels)
`y`	`float`	Center Y (pixels)
`radius`	`float`	Gauge radius in pixels
`fraction`	`float`	Fill fraction in the **0-1** range
`label`	`str`	Optional label
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named), for the filled arc.

RETURNS

`None`

`spark_line(x: float, y: float, w: float, h: float, values: list[float], color?: str, axis?: bool) → None`

Compact trend line drawn from a numeric series. Player accumulates values into a list and pushes the most recent on each tick, the widget normalizes to the series' min/max range and fills a subtle area below the line. The line and its shading default to `"accent"`; pass `color` to change both. Pass `axis` as `True` to label the top, middle and bottom of the scale at the left edge; the line moves over to make room. Empty / single-value series no-op.



PARAMETERS

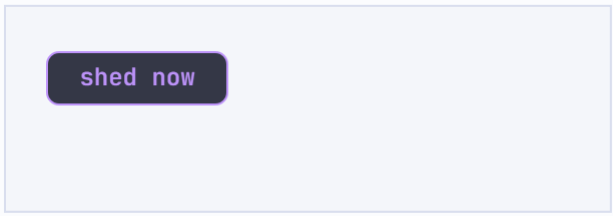
Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`values`	`list[float]`	Numeric values to plot
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named), for the line and its shading.
`axis`	`bool`	`True` labels the top, middle and bottom of the line's scale at its left edge.

RETURNS

`None`

`button(key: str, x: float, y: float, w?: float, h?: float, label?: str) → bool`

A clickable button. Pass a unique `key` so the click routes back to it. Returns `True` the single tick it's pressed (momentary), branch on it: `if panel.button("shed", 10, 12): power.set_powered(...)`. `w` / `h` default to **90×26**. The first input widget that lets a card *act* on the player's click, not just paint.



PARAMETERS

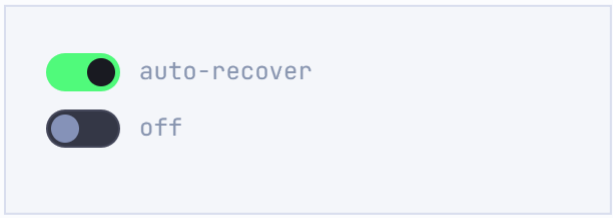
Name	Type	Description
`key`	`str`	Unique id for this control
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width in pixels (default 90)
`h`	`float`	Height in pixels (default 26)
`label`	`str`	Optional label

RETURNS

`bool`

`switch(key: str, x: float, y: float, default_on?: bool, label?: str, size?: float) → bool`

An interactive on/off switch, the player clicks it to flip. Pass a unique `key`; `default_on` sets the starting state the first time the card runs. Returns the current boolean every tick, and the flip persists in the card's state across reloads. Unlike the display `toggle` (which only shows a state you pass in), this one is clickable: `auto = panel.switch("auto_recover", 165, 128, True)`.



PARAMETERS

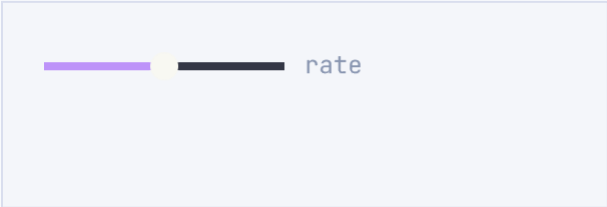
Name	Type	Description
`key`	`str`	Unique id for this control
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`default_on`	`bool`	Initial on/off state
`label`	`str`	Optional label drawn to the right. Clicking it flips the switch too.
`size`	`float`	Pill height in pixels (default 18). The clickable area follows it.

RETURNS

`bool`

`slider(key: str, x: float, y: float, w: float, default?: float, label?: str, size?: float) → float`

A horizontal slider the player clicks to set a value. Pass a unique `key`; `default` (**0-1**) sets the starting value. Returns the current value as a **0-1** number every tick, persisted in the card's state. Use it for thresholds the player tunes live, a buy-trigger level, a throttle target.



PARAMETERS

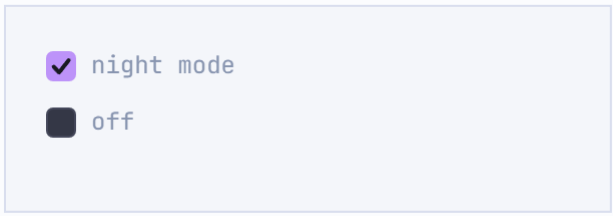
Name	Type	Description
`key`	`str`	Unique id for this control
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`default`	`float`	Initial value 0-1
`label`	`str`	Optional label
`size`	`float`	Track height in pixels (default 14). The knob, label and clickable area follow it.

RETURNS

`float`

`checkbox(key: str, x: float, y: float, default_on?: bool, label?: str, size?: float) → bool`

A check box the player clicks. Same stored state as `panel.switch(...)`, so `set_switch` drives either one; the difference is the shape. Pass a unique `key`; `default_on` sets the starting state the first time the card runs.



PARAMETERS

Name	Type	Description
`key`	`str`	Unique key this box stores its state under.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`default_on`	`bool`	Checked state the first time the card runs.
`label`	`str`	Optional label drawn to the right. Clicking it flips the box too.
`size`	`float`	Box edge length in pixels (default 14). The tick and label follow it.

RETURNS

`bool`

`radio_group(key: str, x: float, y: float, options: list[str], default?: str | int, row_height?: float) → str | None`

One choice out of several, drawn one option per row. The whole group shares a single `key`, which is why five options cost the card one stored value instead of five: building radio buttons out of five separate switches is what leaves five keys behind. Returns the selected option's text every tick, and `None` while `options` is empty. `default` accepts the option text or its row number.

PARAMETERS

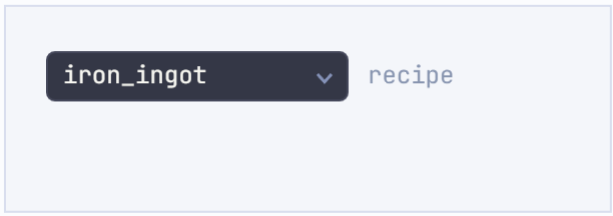
Name	Type	Description
`key`	`str`	Unique key the whole group stores its choice under.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`options`	`list[str]`	List of choices, drawn one per row.
`default`	`str` | `int`	Option text, or row number, selected the first time the card runs.
`row_height`	`float`	Row pitch in pixels (default 20). The dot and text follow it.

RETURNS

`str | None`

`combo(key: str, x: float, y: float, w: float, options: list[str], default?: str | int, label?: str, size?: float) → str | None`

A dropdown. Clicking it opens the game's own menu over the card, so the choices stay readable at any card size and never get clipped by the panel edge. Returns the selected option's text every tick, and `None` while `options` is empty. `default` accepts the option text or its row number.



PARAMETERS

Name	Type	Description
`key`	`str`	Unique key this box stores its choice under.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`options`	`list[str]`	List of choices shown when the player opens the box.
`default`	`str` | `int`	Option text, or row number, selected the first time the card runs.
`label`	`str`	Optional label drawn to the right.
`size`	`float`	Box height in pixels (default 24). The text and caret follow it.

RETURNS

`str | None`

`text_field(key: str, x: float, y: float, w: float, default?: str, placeholder?: str, label?: str, size?: float) → str`

A one-line text box. Clicking it opens a real text editor over the field, so selection, copy, paste and your keyboard's own input method all work; the card shows the committed value. Returns the current text every tick. Up to 1024 characters, saved with the card.

```
outpost_north
```

PARAMETERS

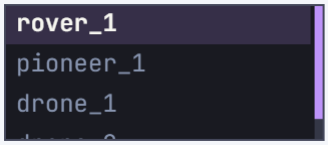
Name	Type	Description
`key`	`str`	Unique key this field stores its text under.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`default`	`str`	Text the field holds the first time the card runs.
`placeholder`	`str`	Muted text shown while the field is empty.
`label`	`str`	Optional label drawn to the right. It also titles the editor the player types into.
`size`	`float`	Field height in pixels (default 24). The text follows it.

RETURNS

`str`

`list(key: str, x: float, y: float, w: float, h: float, items: list[str], row_height?: float) → str | None`

A scrolling list of rows the player can pick from. Rows past the box height scroll with the wheel; the selected row is stored with the card and returned every tick, or `None` while `items` is empty. Use it where a card has more entries than space, which is most fleet and inventory boards.



PARAMETERS

Name	Type	Description
`key`	`str`	Unique key this list stores its selected row under.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels). Rows past it scroll.
`items`	`list[str]`	List of rows to show.
`row_height`	`float`	Row pitch in pixels (default 20). The text follows it.

RETURNS

`str | None`

`icon_button(key: str, x: float, y: float, size: float, item_id: str) → bool`

A square button whose label is an item icon. Returns `True` the single tick it is pressed, exactly like `panel.button(...)`. Use it for a toolbar strip where a word would not fit. `panel.icon_ids()` lists every id it can draw.

PARAMETERS

Name	Type	Description
`key`	`str`	Unique key so the click routes back to this button.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`size`	`float`	Button edge length in pixels (default 32).
`item_id`	`str`	Icon to draw inside the button. `panel.icon_ids()` lists every id.

RETURNS

`bool`

`set_switch(key: str, on: bool) → None`

Force a switch or check box to a state from code, instead of waiting for the player to click it. Use it to build a radio group out of switches, to reset a cockpit to a known layout, or to reflect a state the card read from the world.

```
if not power.is_online():
  panel.set_switch("auto_dispatch", False)
```

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the switch or check box to set.
`on`	`bool`	New state.

RETURNS

`None`

`set_slider(key: str, value: float) → None`

Force a slider to a value (0-1) from code.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the slider to set.
`value`	`float`	New value, clamped to 0-1.

RETURNS

`None`

`set_selected(key: str, option: str | int) → None`

Force a radio group, combo box, or list to a choice from code. Accepts the option's text or its row number.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the radio group, combo box, or list to set.
`option`	`str` | `int`	Option text, or row number.

RETURNS

`None`

`set_text(key: str, text: str) → None`

Force a text field's contents from code.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the text field to set.
`text`	`str`	New text.

RETURNS

`None`

`get_switch(key: str) → bool | None`

The stored state of a switch or check box, read without drawing it. Use it for a control on a page the card is not showing right now. `None` while the key has no stored state.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the switch or check box to read.

RETURNS

`bool | None`

`get_slider(key: str) → float | None`

The stored value (0-1) of a slider, read without drawing it. `None` while the key has no stored value.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the slider to read.

RETURNS

`float | None`

`get_selected(key: str) → int | None`

The stored row number of a radio group, combo box, or list, read without drawing it. Look it up in the same options list you draw the control with. `None` while the key has no stored choice.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the radio group, combo box, or list to read.

RETURNS

`int | None`

`get_text(key: str) → str | None`

The stored text of a text field, read without drawing it. `None` while the key has no stored text.

PARAMETERS

Name	Type	Description
`key`	`str`	Key of the text field to read.

RETURNS

`str | None`

`forget(key: str) → None`

Drop one stored widget value. The next paint of that widget starts from its declared default again, which is how a card resets one control without disturbing the others.

PARAMETERS

Name	Type	Description
`key`	`str`	Key to drop from the card's stored input state.

RETURNS

`None`

`clear_inputs() → None`

Drop every stored widget value on this card. Widget keys are never swept automatically, because a card that only paints one page at a time would lose the other page's state; this is the deliberate reset. A card is limited to 512 stored keys, and a key built from changing data (a tick count, a name that varies) is what reaches that limit.

RETURNS

`None`

`mouse() → PanelMouse`

Where the cursor is on this card, in the same coordinates you draw in. `over` is `False` while the cursor is elsewhere. The sample is taken once per tick, so a highlight drawn from it follows the cursor about a frame behind. `pressed` says the left button went down on this card, and `released` says it came back up anywhere, since your previous call. Each is handed out once, so read the mouse once per frame. Together they let a card drag things.

```
m = panel.mouse()
if m.over and m.x < panel.width() / 2:
  panel.fill_rect(0, 0, panel.width() / 2, panel.height(), "bg-surface")
```

RETURNS

`PanelMouse`

clicks() → list[PanelClick]

Every click since the last time you asked that did not land on a widget, oldest first. This is what lets a card hit-test its own drawing: a map, a chart, a seating plan. Each click is handed out once. Up to 32 are kept between reads.

```
for c in panel.clicks():
  if c.x > 250:
    print("right half", c.x, c.y)
```

RETURNS

`list[PanelClick]`

capture_keys() → None

Ask for the keyboard. After this, clicking the card gives it focus and its keystrokes go to `panel.keys()` instead of the game's own shortcuts; Escape or clicking elsewhere hands the keyboard back. Call it once above your loop. A card that never calls it can never take a key.

RETURNS

`None`

keys() → list[PanelKey]

Every key pressed since the last time you asked, oldest first, for a card that called `panel.capture_keys()` and holds focus. Each press is handed out once. Up to 32 are kept between reads.

```
panel.capture_keys()
while True:
  panel.clear()
  for k in panel.keys():
    if k.key == "ArrowUp":
      cursor = cursor - 1
```

RETURNS

`list[PanelKey]`

`draw_text(x: float, y: float, text: str, size?: float, color?: str, wrap?: float) → None`

Text rendered in monospace at the given size. Optional `wrap` (pixel width) enables greedy word-wrapping into a multi-line block, useful for log feeds and order briefings. `color` accepts theme tokens or hex.

PARAMETERS

Name	Type	Description
`x`	`float`	Left edge of the text (pixels)
`y`	`float`	Vertical middle of the first line (pixels)
`text`	`str`	Text to draw
`size`	`float`	Font size in pixels
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`wrap`	`float`	Optional wrap width in pixels

RETURNS

`None`

`icon_ids() → list[str]`

List of every id `draw_icon` can render, sorted. Covers more than the items you can hold: fluids, creatures, essences, and machine art all have icons. Use it to build a picker, validate an id before drawing, or just print the catalog once while you are writing a card.

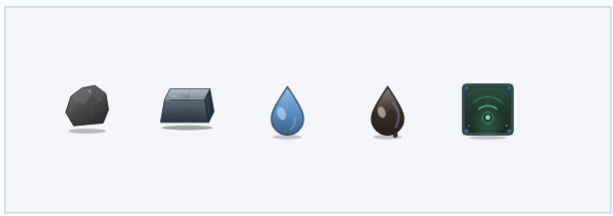
```
for icon in panel.icon_ids():
  print(icon)
```

RETURNS

`list[str]`

`draw_icon(x: float, y: float, item_id: str, size?: float) → None`

Render any icon from the game's item catalog at the requested size (default **32** pixels). Item id is the same string you'd pass to `inventory` / `storage_bin` APIs (`"iron_ore"` , `"iron_ingot"` , `"water"` , etc.), plus things you never hold such as fluids and creatures. `panel.icon_ids()` returns the full list, and every item's own page lives under **Database** in this panel. Unknown ids no-op silently.



PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`item_id`	`str`	Item id to draw the icon for
`size`	`float`	Icon size in pixels (default 32)

RETURNS

`None`

`draw_rect(x: float, y: float, w: float, h: float, color?: str, width?: float) → None`

Outlined rectangle in the given color (defaults to `"border"`). Use for custom subsection borders or visual frames.

PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`width`	`float`	Line width in pixels (default 1), drawn inside the rectangle.

RETURNS

`None`

`fill_rect(x: float, y: float, w: float, h: float, color?: str) → None`

Filled rectangle in the given color (defaults to `"accent"`). Use for backgrounds, progress fills, color blocks.

PARAMETERS

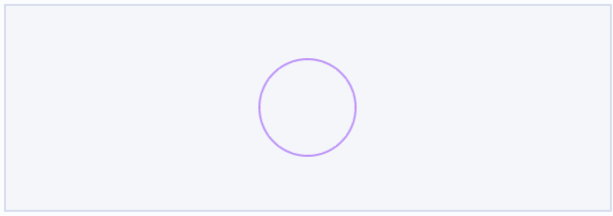
Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).

RETURNS

`None`

`draw_circle(x: float, y: float, r: float, color?: str, width?: float) → None`

Outlined circle in the given color.



PARAMETERS

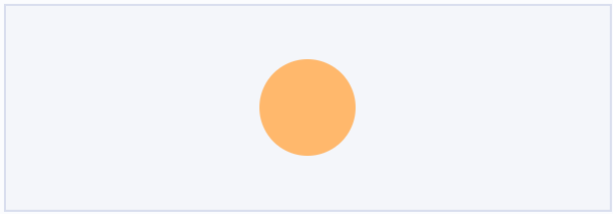
Name	Type	Description
`x`	`float`	Center X (pixels)
`y`	`float`	Center Y (pixels)
`r`	`float`	Radius
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`width`	`float`	Line width in pixels (default 1).

RETURNS

`None`

`fill_circle(x: float, y: float, r: float, color?: str) → None`

Filled disc in the given color.



PARAMETERS

Name	Type	Description
`x`	`float`	Center X (pixels)
`y`	`float`	Center Y (pixels)
`r`	`float`	Radius
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).

RETURNS

`None`

`draw_line(x1: float, y1: float, x2: float, y2: float, color?: str, width?: float) → None`

Single straight line.

PARAMETERS

Name	Type	Description
`x1`	`float`	Start X (pixels)
`y1`	`float`	Start Y (pixels)
`x2`	`float`	End X (pixels)
`y2`	`float`	End Y (pixels)
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`width`	`float`	Line width in pixels (default 1).

RETURNS

`None`

`draw_polygon(points: list[float], color?: str, width?: float) → None`

Outlined closed shape through a flat list of coordinates: `[x1, y1, x2, y2, ...]`. Needs at least two points. Up to 4096 points per call.

PARAMETERS

Name	Type	Description
`points`	`list[float]`	Flat list of coordinates: `[x1, y1, x2, y2, ...]`.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`width`	`float`	Line width in pixels (default 1).

RETURNS

`None`

`fill_polygon(points: list[float], color?: str) → None`

Filled closed shape through a flat list of coordinates. Same input as `draw_polygon`.

PARAMETERS

Name	Type	Description
`points`	`list[float]`	Flat list of coordinates: `[x1, y1, x2, y2, ...]`.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).

RETURNS

`None`

`polyline(points: list[float], color?: str, width?: float) → None`

Open line through a flat list of coordinates, with an optional width. Use it for a path, a route, or a chart trace your own code computed.

PARAMETERS

Name	Type	Description
`points`	`list[float]`	Flat list of coordinates: `[x1, y1, x2, y2, ...]`.
`color`	`str`	Theme token, or any CSS color (hex, `rgb()`, `hsl()`, named).
`width`	`float`	Line width in pixels (default 1).

RETURNS

`None`

`texture(key: str, rows: list[str], palette: dict[str, str | None]) → None`

Create a small picture for `draw_texture` to draw. `rows` is the picture, one string per row, and each character is looked up in `palette`, which maps it to a hex colour or to `None` for a transparent pixel. Every row is the same length, up to **128** by **128** pixels. Using the same `key` again replaces that texture. A card holds up to **32** textures and **131,072** pixels of them in total. Textures belong to the running script, so create them at the top, before your loop.

```
panel.texture("floor", [
  "abab",
  "baba",
], {"a": "#1b2230", "b": "#26314a"})
```

PARAMETERS

Name	Type	Description
`key`	`str`	Name for the texture. Using a name again replaces that texture.
`rows`	`list[str]`	The picture, one string per row of pixels. Every row is the same length, at most 128 by 128 pixels.
`palette`	`dict[str, str | None]`	Maps each character in `rows` to a hex colour such as `"#1b2230"`, or to `None` for a transparent pixel.

RETURNS

`None`

RAISES

Exception	Condition
`TypeError`	The `key` is not a non-empty string, `rows` is not a non-empty list of strings, `palette` is not a `dict`, a palette key is not a string, or a palette colour is neither a string nor `None`.
`ValueError`	A row is empty, the rows differ in length or exceed 128 by 128 pixels, a palette key is not exactly one character, a palette colour is not a hex colour, a row uses a character the palette does not have, or adding or replacing the texture would exceed the card's limits of 32 textures or 131,072 total pixels.

`draw_texture(key: str, x: float, y: float, w?: float, h?: float) → None`

Draw a texture made with `panel.texture()`. Without `w` and `h` it draws at its own size. With them it stretches and keeps its pixels sharp, so a small tile can fill the whole card.

```
panel.draw_texture("floor", 0, 0, panel.width(), panel.height())
```

PARAMETERS

Name	Type	Description
`key`	`str`	Name given to `panel.texture()`.
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width to draw at, in pixels. Defaults to the texture's own width.
`h`	`float`	Height to draw at, in pixels. Defaults to the texture's own height.

RETURNS

`None`

RAISES

Exception	Condition
`TypeError`	The texture key must be a non-empty string.
`KeyError`	The running script has not created a texture under this key.

`clip_rect(x: float, y: float, w: float, h: float) → None`

Confine every later drawing call to a rectangle, until `clear_clip()`. Use it to keep a scrolling or oversized drawing inside its box. A widget drawn under a clip takes clicks only where it shows, so one clipped out of sight takes none. Clips nest up to 16 deep; any left open are closed for you when the frame finishes.

```
panel.clip_rect(10, 10, 200, 80)
panel.draw_text(12, 30, long_line, 12)
panel.clear_clip()
```

PARAMETERS

Name	Type	Description
`x`	`float`	Left edge (pixels)
`y`	`float`	Top edge (pixels)
`w`	`float`	Width (pixels)
`h`	`float`	Height (pixels)

RETURNS

`None`

`clear_clip(all?: bool) → None`

Close the innermost `clip_rect`, or every open one when passed `True`.

PARAMETERS

Name	Type	Description
`all`	`bool`	`True` closes every open clip; omitted closes the innermost one.

RETURNS

`None`

`clear() → None`

Wipe the panel canvas, and with it every widget's clickable area. Call at the top of every `while True:` loop iteration so old paint doesn't ghost behind new paint. Inside a `clip_rect` it wipes only the clipped area, and only the widgets it erased stop taking clicks.

RETURNS

`None`

`last_bounds() → PanelBounds | None`

The box the last widget you drew takes up, as `.x`, `.y`, `.w` and `.h`, with `.x` and `.y` at its top-left corner whichever point that widget is placed by. Use it to put the next widget after one whose size comes from its text, such as a `pill` or `draw_text`, without guessing its width. `None` until the card draws its first widget.

```
panel.pill(10, 10, "online", "success")
b = panel.last_bounds()
panel.pill(b.x + b.w + 6, 10, "docked")
```

RETURNS

`PanelBounds | None`

`measure_text(text: str, size?: float, wrap?: float) → PanelSize`

The size `draw_text` would give this text, as `.w` and `.h`, without drawing it. Pass the same `size` and `wrap` you will draw with. Use it to right-align or center text before you paint it.

```
s = panel.measure_text("42 kWh", 14)
panel.draw_text(panel.width() - s.w - 10, 20, "42 kWh", 14)
```

PARAMETERS

Name	Type	Description
`text`	`str`	Text to measure.
`size`	`float`	Font size in pixels (default 12), as you will pass it to `draw_text`.
`wrap`	`float`	Optional wrap width in pixels, as you will pass it to `draw_text`.

RETURNS

`PanelSize`

`width() → int`

Current logical canvas width in pixels: **500** for a one-column card or **1000** for a two-column card. Use for ratio-based positioning.

RETURNS

`int`

`height() → int`

Current logical canvas height in pixels: **200** for a one-row card or **400** for a two-row card. Use for ratio-based positioning.

RETURNS

`int`

PanelBounds

Returned by: `panel.last_bounds()` (panel scripts only)

Import `PanelBounds` with `from __builtins__ import PanelBounds`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`PanelBounds(x: float, y: float, w: float, h: float) → PanelBounds`

Create a local `PanelBounds` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`x`	`float`	Left edge in panel coordinates. Must be a finite number.
`y`	`float`	Top edge in panel coordinates. Must be a finite number.
`w`	`float`	Width in pixels. Must be a finite number. Must be at least 0.
`h`	`float`	Height in pixels. Must be a finite number. Must be at least 0.

RETURNS

`PanelBounds`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`ValueError`	A numeric value is outside the permitted range, or the bounds are reversed.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

`.x: float`

Left edge in panel coordinates.

RETURNS

`float`

`.y: float`

Top edge in panel coordinates.

RETURNS

`float`

`.w: float`

Width in pixels.

RETURNS

`float`

`.h: float`

Height in pixels.

RETURNS

`float`

PanelClick

Returned by: `panel.clicks()` (panel scripts only)

Get `PanelClick` from the APIs listed here. It has no script constructor.

PROPERTIES

`.x: float`

Click X in panel coordinates.

RETURNS
`float`

`.y: float`

Click Y in panel coordinates.

RETURNS
`float`

PanelKey

Returned by: `panel.keys()` (panel scripts only)

Get `PanelKey` from the APIs listed here. It has no script constructor.

PROPERTIES

`.key: str`

Key name, such as `"a"`, `"Enter"`, or `"ArrowLeft"`.

RETURNS
`str`

`.ctrl: bool`

`True` when Ctrl was held.

RETURNS
`bool`

`.shift: bool`

`True` when Shift was held.

RETURNS
`bool`

`.alt: bool`

`True` when Alt was held.

RETURNS

`bool`

`.meta: bool`

`True` when Cmd or the Windows key was held.

RETURNS

`bool`

PanelMouse

Returned by: `panel.mouse()` (panel scripts only)

Get `PanelMouse` from the APIs listed here. It has no script constructor.

PROPERTIES

`.x: float`

Cursor X in panel coordinates.

RETURNS
`float`

`.y: float`

Cursor Y in panel coordinates.

RETURNS
`float`

`.over: bool`

`True` while the cursor is over this card.

RETURNS
`bool`

`.pressed: bool`

`True` if the left button went down on this card since the previous `mouse()` call.

RETURNS

`bool`

`.released: bool`

`True` if the left button came back up since the previous `mouse()` call, even when it was released off the card.

RETURNS

`bool`

PanelSize

Returned by: `panel.measure_text()` (panel scripts only)

Import `PanelSize` with `from __builtins__ import PanelSize`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`PanelSize(w: float, h: float) → PanelSize`

Create a local `PanelSize` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`w`	`float`	Width in pixels. Must be a finite number. Must be at least 0.
`h`	`float`	Height in pixels, every wrapped line included. Must be a finite number. Must be at least 0.

RETURNS

`PanelSize`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`ValueError`	A numeric value is outside the permitted range, or the bounds are reversed.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

`.w: float`

Width in pixels.

RETURNS

`float`

`.h: float`

Height in pixels, every wrapped line included.

RETURNS

`float`

Contract

Returned by: self.contract

Get `Contract` from the APIs listed here. It has no script constructor.

CONCRETE SUBTYPES

- `BeatTheSystemContract`
- `BuriedFiveContract`
- `ColdBootContract`
- `CoreSampleContract`
- `CorruptedArchiveContract`
- `CrosstalkContract`
- `DataTabletContract`
- `DriftingSignalContract`
- `LatticeContract`
- `RelayHackContract`
- `SealedVaultContract`
- `TerminalBreachContract`
- `TheLoomContract`
- `ThreeEchoesContract`
- `XenogeneticsContract`

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS	POSSIBLE VALUES
`str`	`"available"`, `"completed"`

BeatTheSystemContract

Extends `Contract`

Returned by: self.contract (beat_the_system)

Get `BeatTheSystemContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Arbiter`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.arbiter: Arbiter`

The alien Arbiter. It takes an immediate win, otherwise blocks your immediate win, otherwise prefers centre, then corners, then edges; tied choices are random.

RETURNS

`Arbiter`

Arbiter

Returned by: .arbiter

Get `Arbiter` from the APIs listed here. It has no script constructor.

METHODS

`.new_game() → ActionResult`

Start a fresh 3×3 game on an empty board; you move first. After a finished game this call pauses about half a second before the next board is ready.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"in_progress"`	transient	The operation is still in progress.

`.restart() → ActionResult`

Abandon any game in progress and start fresh; you move first. Abandoning a game mid-play counts as a non-win and resets your current-run streak to 0. Like `new_game()`, it pauses about half a second between games.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.

`.play(cell: int) → ActionResult`

Place your mark in a whole-number cell in the **0-8** range (row-major), then the Arbiter responds. Three marks in a row, column, or diagonal wins. A non-number cell raises `TypeError`; a non-finite, fractional, or out-of-range cell raises `ValueError` before game state is considered.

PARAMETERS

Name	Type	Description
`cell`	`int`	Whole-number board cell to mark, 0-8

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ongoing"`	● success	The game remains in progress.
`"win"`	● success	The game ended in a win.
`"loss"`	● success	The game ended in a loss.
`"draw"`	● success	The game ended in a draw.
`"occupied"`	● rejection	The requested cell, sector, or Habitat is already occupied.
`"no_game"`	● rejection	There is no active game.

RAISES

Exception	Condition
`TypeError`	`Arbiter.play()` requires a numeric cell.
`ValueError`	`Arbiter.play()` requires a finite whole-number cell in the **0-8** range.

`.board() → list[str]`

The 9 board cells as a list, index 0-8 row-major. Each cell is `""` (empty), `"you"`, or `"arbiter"`.

RETURNS

`list[str]`

`.result() → str`

Current game outcome: `"ongoing"`, `"win"`, `"loss"`, `"draw"`, or `"no_game"` (no game started yet).

RETURNS
`str`

POSSIBLE VALUES
`"ongoing"`, `"win"`, `"loss"`, `"draw"`, `"no_game"`

`.streak() → int`

Consecutive wins in the current script run. Resets to 0 on a loss, draw, abandonment, or fresh script run.

RETURNS
`int`

`.target() → int`

The consecutive-win count needed to complete the contract.

RETURNS
`int`

`.token() → str`

The passcode to transmit: a non-empty string once `streak()` reaches `target()`, otherwise an empty string.

RETURNS
`str`

BuriedFiveContract

Extends `Contract`

Returned by: self.contract (buried_five)

Get `BuriedFiveContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Analyzer`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.transmission: list[str]`

The scrambled transmission: a list of single-character tokens.

RETURNS

`list[str]`

`.analyzer: Analyzer`

The recovered analyzer device: collapses a group of five tokens into the single token they were expanded from.

RETURNS

`Analyzer`

`.layers: int`

Whole-number count of five-fold wrapping layers applied to the transmission.

RETURNS

`int`

Analyzer

Returned by: .analyzer

Get `Analyzer` from the APIs listed here. It has no script constructor.

METHODS

`.read(group: list[str]) → str`

Read a list of exactly five string tokens and return the single token they were expanded from. A non-list argument or non-string element raises `TypeError`; the wrong length or an unrecognized group raises `ValueError`.

PARAMETERS

Name	Type	Description
`group`	`list[str]`	Five-string-token group to collapse

RETURNS

`str`

RAISES

Exception	Condition
`TypeError`	`Analyzer.read()` requires a list containing only string tokens.
`ValueError`	`Analyzer.read()` requires exactly five tokens forming a recognized aligned group.

ColdBootContract

Extends `Contract`

Returned by: self.contract (cold_boot)

Get `ColdBootContract` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.program: list[int]`

The artifact's bytecode: a list of whole numbers. Copy it before running: `memory = list(program)`.

RETURNS

`list[int]`

CoreSampleContract

Extends `Contract`

Returned by: self.contract (core_sample)

Get `CoreSampleContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `CoreDevice`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.cores: list[list[int | None]]`

The 10 damaged cores, as a list of byte lists. A byte destroyed in transit reads as `None`: recover it from the construction rules.

RETURNS

`list[list[int | None]]`

`.device: CoreDevice`

The reconstruction device: submit your rebuilt cores to it. See `CoreDevice`.

RETURNS

`CoreDevice`

CoreDevice

Returned by: .device

Get `CoreDevice` from the APIs listed here. It has no script constructor.

METHODS

`.submit(index: int, bytes: list[int]) → ActionResult`

Submit a rebuilt core for whole-number slot `index` (0-9). Wrong container or element types raise `TypeError`; a fractional or out-of-range index, wrong list length, or numeric value outside the **0-255** range raises `ValueError`. A rejected submission does not lock the slot.

PARAMETERS

Name	Type	Description
`index`	`int`	Whole-number core slot, 0-9
`bytes`	`list[int]`	The rebuilt core as a list of whole-number bytes in the **0-255** range

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"locked"`	● success	The rebuilt core satisfies every construction rule and preserves all surviving bytes.
`"rejected"`	● rejection	The submitted byte content is well formed but violates one or more core construction rules.

RAISES

Exception	Condition
`TypeError`	`CoreDevice.submit()` requires a numeric index and a list containing only numeric bytes.
`ValueError`	`CoreDevice.submit()` requires a whole-number slot in the **0-9** range and a correctly sized list of whole-number bytes in the **0-255** range.

`.recovered() → int`

How many of the 10 cores are locked in the current script run. A fresh run starts at 0.

RETURNS

`int`

`.target() → int`

The number of cores you must recover to complete the contract: 10.

RETURNS

`int`

`.token() → str`

The passcode to transmit: a non-empty string once `recovered()` reaches `target()`, otherwise an empty string.

RETURNS

`str`

CorruptedArchiveContract

Extends `Contract`

Returned by: self.contract (corrupted_archive)

Get `CorruptedArchiveContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Archive`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.archive: Archive`

The scrambled data archive.

RETURNS

`Archive`

Archive

Returned by: .archive

Get `Archive` from the APIs listed here. It has no script constructor.

PROPERTIES

`.rows: int`

Number of rows in the grid.

RETURNS

`int`

`.cols: int`

Number of columns in the grid.

RETURNS

`int`

METHODS

`.flip(row: int, col: int) → str`

Reveal and return the word at a whole-number grid cell. Wrong argument types raise `TypeError`; fractional, non-finite, or out-of-bounds coordinates raise `ValueError`.

PARAMETERS

Name	Type	Description
`row`	`int`	Whole-number grid row, 0 to `rows - 1`
`col`	`int`	Whole-number grid column, 0 to `cols - 1`

RETURNS

`str`

RAISES

Exception	Condition
`TypeError`	`Archive.flip()` requires numeric row and column coordinates.
`ValueError`	`Archive.flip()` requires finite whole-number coordinates inside the archive grid.

CrosstalkContract

Extends `Contract`

Returned by: self.contract (crosstalk)

Get `CrosstalkContract` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.input_x: str`

First intercepted signal: a string of letters with 0s and 1s scattered through. Some bits are decoys.

RETURNS

`str`

`.input_y: str`

Second intercepted signal: same shape as `input_x`.

RETURNS

`str`

`.min_length: int`

Minimum palindrome length for a bit to count: a bit qualifies only if the letters mirror to this span centred on it.

RETURNS

`int`

DataTabletContract

Extends `Contract`

Returned by: self.contract (data_tablet)

Get `DataTabletContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `DataTablet`

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.tablet: DataTablet`

The data tablet scanner.

RETURNS

`DataTablet`

DataTablet

Returned by: .tablet

Get `DataTablet` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `ProbeResult`

PROPERTIES

.rows: int

Number of rows in the grid.

RETURNS

`int`

.cols: int

Number of columns in the grid.

RETURNS

`int`

METHODS

`.probe(row: int, col: int) → ProbeResult`

Probe a whole-number cell and return a `ProbeResult` with `.char` and whole-number `.distance`. Wrong argument types raise `TypeError`; fractional or out-of-bounds coordinates raise `ValueError`.

PARAMETERS

Name	Type	Description
`row`	`int`	Whole-number grid row, 0 to `rows - 1`
`col`	`int`	Whole-number grid column, 0 to `cols - 1`

RETURNS

`ProbeResult`

RAISES

Exception	Condition
`TypeError`	The contract probe received an argument or element of the wrong type.
`ValueError`	The contract probe received a value, shape, or coordinate outside its documented domain.

ProbeResult

Returned by: tablet.probe()

Get `ProbeResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.char: str`

Character at this cell.

RETURNS

`str`

`.distance: int`

Whole-number Manhattan distance (steps) to the nearest message cell. 0 means this cell IS a message cell.

RETURNS

`int`

DriftingSignalContract

Extends `Contract`

Returned by: self.contract (drifting_signal)

Get `DriftingSignalContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `SlabDevice`

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.device: SlabDevice`

The recovered slab contraption.

RETURNS

`SlabDevice`

SlabDevice

Returned by: .device

Get `SlabDevice` from the APIs listed here. It has no script constructor.

PROPERTIES

`.slabs: str`

Current state of the letter slabs. Uppercase letters, spaces preserved.

RETURNS

`str`

LatticeContract

Extends `Contract`

Returned by: self.contract (lattice)

Get `LatticeContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `LatticeGrid`

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.grid: LatticeGrid`

The alien deep-scan lattice. Probe only proven-clear cells to map the volatile nodes; a trip blocks further probes until the grid is reset. See `LatticeGrid`.

RETURNS

`LatticeGrid`

LatticeGrid

Returned by: .grid

Get `LatticeGrid` from the APIs listed here. It has no script constructor.

METHODS

`.width() → int`

The grid width in cells (32).

RETURNS

`int`

`.height() → int`

The grid height in cells (32).

RETURNS

`int`

`.start() → list[int]`

A guaranteed-clear foothold cell, returned as `[x, y]`. Probe it first to get a reading and begin the deduction.

RETURNS

`list[int]`

`.reset() → ActionResult`

Clear a tripped fault so probing can continue in the same script run. The hidden board and guaranteed-clear starting cell do not change.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The lattice fault was cleared. The hidden board and starting cell are unchanged.

`.probe(x: int, y: int) → LatticeProbeResult`

Probe a proven clear whole-number cell. The reading is the whole-number count of neighboring volatile nodes in the **0-8** range. Tripping a node faults the lattice until `reset()` is called. Wrong argument types raise `TypeError`; fractional, non-finite, or out-of-bounds coordinates raise `ValueError` without tripping an intact lattice.

PARAMETERS

Name	Type	Description
`x`	`int`	Whole-number column, 0-31
`y`	`int`	Whole-number row, 0-31

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`LatticeProbeResult`	`.status`, `.message`	`.reading`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	Neighboring volatile nodes in the clear cell: `.reading`.
`"node_tripped"`	● rejection	The probed cell contained a volatile node and faulted the lattice.
`"lattice_tripped"`	● rejection	An earlier volatile-node probe faulted the lattice.

RAISES

Exception	Condition
`TypeError`	`LatticeGrid.probe()` requires numeric x and y coordinates.
`ValueError`	`LatticeGrid.probe()` requires finite whole-number coordinates inside the 32 by 32 grid.

RelayHackContract

Extends `Contract`

Returned by: self.contract (relay_hack)

Get `RelayHackContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `RelayLock`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.lock: RelayLock`

The relay lock to crack.

RETURNS

`RelayLock`

RelayLock

Returned by: .lock

Get `RelayLock` from the APIs listed here. It has no script constructor.

PROPERTIES

`.tumblers: int`

Number of tumblers (6).

RETURNS

`int`

`.range: int`

Range per tumbler (100 = 0-99).

RETURNS

`int`

METHODS

`.intercept(code: list[int]) → list[bool]`

Test a list of exactly 6 whole-number values in the **0-99** range and return one `True` / `False` value per tumbler. Wrong argument types raise `TypeError`; wrong list length, non-finite or fractional values, and values outside the range raise `ValueError`.

PARAMETERS

Name	Type	Description
`code`	`list[int]`	Candidate list of 6 whole numbers, each 0-99

RETURNS

`list[bool]`

RAISES

Exception	Condition
`TypeError`	The contract probe received an argument or element of the wrong type.
`ValueError`	The contract probe received a value, shape, or coordinate outside its documented domain.

SealedVaultContract

Extends `Contract`

Returned by: self.contract (sealed_vault)

Get `SealedVaultContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Vault`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.vault: Vault`

The sealed vault passage system. Its position resets to (0, 0) at the start of each contract script run; the maze layout stays fixed.

RETURNS

`Vault`

Vault

Returned by: .vault

Get `Vault` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `VaultPosition`

PROPERTIES

`.position: VaultPosition`

Current cell as a `VaultPosition` snapshot with `.row` and `.col`. Store it when you need to remember an old cell; read `vault.position` again after `move()` to get the new cell.

RETURNS

`VaultPosition`

`.size: int`

Side length of the (square) maze grid. The maze is `size` × `size` cells; the start is `(0, 0)` and the exit is `(size - 1, size - 1)`.

RETURNS

`int`

METHODS

`.move(direction: str) → ActionResult`

Step one cell in `direction`: `"north"`, `"south"`, `"east"`, or `"west"`. A non-string direction raises `TypeError`; an unknown direction raises `ValueError` without moving.

PARAMETERS

Name	Type	Description
`direction`	`str`	Direction to step.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"path"`	● success	The move entered an ordinary path cell.
`"wall"`	● rejection	A wall blocked the move and the position did not change.
`"exit"`	● success	The move entered the exit cell.

RAISES

Exception	Condition
`TypeError`	`Vault.move()` requires a string direction.
`ValueError`	`Vault.move()` accepts only `"north"`, `"south"`, `"east"`, or `"west"`.

`.escape() → VaultEscapeResult`

Open the vault from its exit cell.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`VaultEscapeResult`	`.status`, `.message`	`.key`

OUTCOMES

Status	Kind	Meaning
`"ok"`	● success	The vault opened and released its key.
`"not_at_exit"`	● rejection	The vault can open only from the exit cell.

VaultPosition

Returned by: vault.position

Import `VaultPosition` with `from __builtins__ import VaultPosition`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`VaultPosition(row: int, col: int) → VaultPosition`

Create a local `VaultPosition` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`row`	`int`	Row coordinate. Must be a whole number. Must be at least 0.
`col`	`int`	Column coordinate. Must be a whole number. Must be at least 0.

RETURNS

`VaultPosition`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.
`ValueError`	A numeric value is outside the permitted range, or the bounds are reversed.
`OverflowError`	An integer cannot be represented safely as a game number.

PROPERTIES

`.row: int`

Row coordinate.

RETURNS

`int`

`.col: int`

Column coordinate.

RETURNS

`int`

METHODS

`.__iter__() → Iterator[int]`

Iterate over `row`, then `col`, so this position can be unpacked or passed to `list()`.

RETURNS

`Iterator[int]`

TerminalBreachContract

Extends `Contract`

Returned by: self.contract (terminal_breach)

Get `TerminalBreachContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `AlienTerminal`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.terminal: AlienTerminal`

The alien security terminal.

RETURNS

`AlienTerminal`

AlienTerminal

Returned by: .terminal

Get `AlienTerminal` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `GuessResult`

PROPERTIES

`.length: int`

Whole-number code length (15).

RETURNS

`int`

METHODS

`.guess(digits: list[int]) → GuessResult`

Test a list of exactly 15 whole-number digits in the **1-5** range and return `GuessResult`. Exact-position matches are removed first; `.misplaced` then counts shared remaining occurrences without over-counting duplicates. Wrong argument or element types raise `TypeError`; wrong length, fractional values, or out-of-range digits raise `ValueError`.

PARAMETERS

Name	Type	Description
`digits`	`list[int]`	Candidate list of 15 whole-number digits, each 1-5

RETURNS

`GuessResult`

RAISES

Exception	Condition
`TypeError`	The contract probe received an argument or element of the wrong type.
`ValueError`	The contract probe received a value, shape, or coordinate outside its documented domain.

GuessResult

Returned by: terminal.guess()

Get `GuessResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.correct: int`

Number of digits in the correct position.

RETURNS

`int`

`.misplaced: int`

Number of correct digits in wrong positions.

RETURNS

`int`

TheLoomContract

Extends `Contract`

Returned by: self.contract (the_loom)

Get `TheLoomContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `Loom`

PROPERTIES

.id: str

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.loom: Loom`

The recovered alien loom, your probe tool. Call `loom.weave(a, b)` to learn how it braids two strings into one.

RETURNS

`Loom`

`.record: str`

A 42-character woven record made from two equal-length 21-character threads. Reverse the loom's rule to un-weave it; one thread is the message.

RETURNS

`str`

Loom

Returned by: .loom

Get `Loom` from the APIs listed here. It has no script constructor.

METHODS

`.weave(a: str, b: str) → str`

Braid two strings into one and return it. Each character is one token. Deterministic: the same inputs always weave the same way, so probe it freely. A non-string argument raises `TypeError`; either input longer than 30 characters raises `ValueError`. The loom only weaves forward; build the reverse yourself.

PARAMETERS

Name	Type	Description
`a`	`str`	First input string
`b`	`str`	Second input string

RETURNS

`str`

RAISES

Exception	Condition
`TypeError`	`Loom.weave()` requires two strings.
`ValueError`	`Loom.weave()` accepts at most 30 characters in each input.

ThreeEchoesContract

Extends `Contract`

Returned by: self.contract (three_echoes)

Get `ThreeEchoesContract` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `ThreeEchoesBroadcast`

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.broadcast: ThreeEchoesBroadcast`

The intercepted broadcast: three frequency fragments.

RETURNS

`ThreeEchoesBroadcast`

ThreeEchoesBroadcast

Returned by: .broadcast

Get `ThreeEchoesBroadcast` from the APIs listed here. It has no script constructor.

PROPERTIES

.freq_a: str

Ordered fragment containing the 1st, 4th, 7th, and later every-third characters of the original signal.

RETURNS

`str`

.freq_b: str

Ordered fragment containing the 2nd, 5th, 8th, and later every-third characters of the original signal.

RETURNS

`str`

.freq_c: str

Ordered fragment containing the 3rd, 6th, 9th, and later every-third characters of the original signal.

RETURNS

`str`

XenogeneticsContract

Extends `Contract`

Returned by: self.contract (xenogenetics)

Get `XenogeneticsContract` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Contract ID (used for transmitting answers).

RETURNS

`str`

POSSIBLE VALUES

`"relay_hack"`, `"xenogenetics"`, `"corrupted_archive"`, `"sealed_vault"`, `"data_tablet"`, `"terminal_breach"`, `"drifting_signal"`, `"cold_boot"`, `"three_echoes"`, `"buried_five"`, `"the_loom"`, `"crosstalk"`, `"beat_the_system"`, `"core_sample"`, `"lattice"`

`.name: str`

Contract display name.

RETURNS

`str`

`.reward: int`

Credit reward for completing this contract.

RETURNS

`int`

`.status: str`

Contract status: `"available"` or `"completed"`.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"completed"`

`.earth_ref: list[str]`

50 known Earth gene sequences (list of strings).

RETURNS

`list[str]`

`.samples: list[str]`

1000 collected DNA samples (list of strings).

RETURNS

`list[str]`

ContractScript

Returned by: `self` (in contract scripts)

Get `ContractScript` from the APIs listed here. It has no script constructor.

PROPERTIES

`.name: str`

Script name.

RETURNS

`str`

`.contract: Contract`

The contract object with ID, name, reward, and contract-specific API.

RETURNS

`Contract`

LatticeProbeResult

Returned by: LatticeGrid.probe()

Get `LatticeProbeResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"node_tripped"`, or `"lattice_tripped"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"node_tripped"`, `"lattice_tripped"`

.message: str

Player-readable explanation of the probe outcome.

RETURNS
`str`

.reading: int | None

Whole-number neighboring-node count in the **0-8** range when `.status == "ok"`; otherwise `None`.

RETURNS
`int | None`

VaultEscapeResult

Returned by: Vault.escape()

Get `VaultEscapeResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"ok"` or `"not_at_exit"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"not_at_exit"`

`.message: str`

Player-readable explanation of the escape outcome.

RETURNS
`str`

`.key: str | None`

Vault key string when `.status == "ok"`; otherwise `None`.

RETURNS
`str | None`

Counter

Returned by: `Counter(iterable)` from `collections`

METHODS

`.most_common(n: int | None = None, /) → list[tuple[K, int]]`

The counts from highest to lowest as `(item, count)` pairs; with `n`, only the top `n`.

PARAMETERS

Name	Type	Description
`n`	`int` | `None`	How many of the largest counts (default all)

RETURNS

`list[tuple[K, int]]`

`.elements() → Iterator[K]`

Each item repeated as many times as its count, skipping counts below one.

RETURNS

`Iterator[K]`

`.total() → int`

The sum of all counts.

RETURNS

`int`

`.update(iterable: object = None, /, **kwargs: int) → None`

Add counts from an iterable of items, a mapping of counts, or keywords.

PARAMETERS

Name	Type	Description
`iterable`	`object`	Items to count, or a mapping of counts to add
`**kwargs`	`int`	Counts to add by name

RETURNS

`None`

`.subtract(iterable: object = None, /, **kwargs: int) → None`

Take counts away, the opposite of `update`; a count may go to zero or below.

PARAMETERS

Name	Type	Description
`iterable`	`object`	Items to uncount, or a mapping of counts to take away
`**kwargs`	`int`	Counts to take away by name

RETURNS

`None`

`.copy() → Counter[K]`

A new `Counter` with the same counts.

RETURNS

`Counter[K]`

defaultdict

Returned by: `defaultdict(factory)` from `collections`

PROPERTIES

`.default_factory: Callable | None`

The function called to make a missing key's value, or `None`.

RETURNS

`Callable | None`

METHODS

`.copy() → defaultdict[K, V]`

A new `defaultdict` with the same items and factory.

RETURNS

`defaultdict[K, V]`

deque

Returned by: `deque(iterable)` from `collections`

PROPERTIES

`.maxlen: int | None`

The size limit given when the `deque` was made, or `None`.

RETURNS

`int | None`

METHODS

`.append(x: T, /) → None`

Add an item to the right end.

PARAMETERS

Name	Type	Description
`x`	`T`	Item to add

RETURNS

`None`

`.appendleft(x: T, /) → None`

Add an item to the left end.

PARAMETERS

Name	Type	Description
`x`	`T`	Item to add

RETURNS

`None`

`.pop() → T`

Remove and return the rightmost item.

RETURNS

`T`

`.popleft() → T`

Remove and return the leftmost item.

RETURNS

`T`

`.extend(iterable: Iterable[T], /) → None`

Add every item of an iterable to the right end.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to add

RETURNS

`None`

`.extendleft(iterable: Iterable[T], /) → None`

Add every item of an iterable to the left end, one at a time, so they end up reversed.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Items to add

RETURNS

`None`

`.rotate(n: int = 1, /) → None`

Move items from the right end to the left `n` times, or the other way when `n` is negative.

PARAMETERS

Name	Type	Description
`n`	`int`	Steps to rotate

RETURNS

`None`

`.clear() → None`

Remove every item.

RETURNS

`None`

`.copy() → deque[T]`

A new `deque` with the same items and `maxlen`.

RETURNS

`deque[T]`

`.count(x: object, /) → int`

How many items equal `x`.

PARAMETERS

Name	Type	Description
`x`	`object`	Value to count

RETURNS

`int`

`.index(x: object, start: int = 0, stop: int | None = None, /) → int`

Position of the first item equal to `x`, searching from `start` up to `stop`.

PARAMETERS

Name	Type	Description
`x`	`object`	Value to find
`start`	`int`	First position to search
`stop`	`int` | `None`	Position to stop before

RETURNS

`int`

`.insert(i: int, x: T, /) → None`

Insert `x` before position `i`.

PARAMETERS

Name	Type	Description
`i`	`int`	Position to insert before
`x`	`T`	Item to insert

RETURNS

`None`

`.remove(value: object, /) → None`

Remove the first item equal to `value`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value to remove

RETURNS

`None`

`.reverse() → None`

Reverse the items in place.

RETURNS

`None`

dict

Returned by: `dict` literals `{k: v}` · `dict()` · methods returning a `dict`

PROPERTIES

`.length: int`

Number of key-value pairs. Same as `len(d)`. Property: no parens.

RETURNS

`int`

METHODS

`.keys() → list[K]`

Return a list of all keys, in insertion order.

RETURNS

`list[K]`

`.values() → list[V]`

Return a list of all values, in insertion order.

RETURNS

`list[V]`

`.items() → list[tuple[K, V]]`

Return a list of `(key, value)` tuples, in insertion order. Common pattern: `for k, v in d.items(): ...`.

RETURNS

`list[tuple[K, V]]`

`.has(key: K, /) → bool`

`True` if `key` is in the dictionary. Equivalent to Python's `key in d`. Use this for safe membership tests before subscript.

PARAMETERS

Name	Type	Description
`key`	`K`	Key to check

RETURNS

`bool`

`.get(key: K, default: V | None = None, /) → V`

Return the value for `key`, or `default` (`None` if omitted) if the key is missing. Never raises for a missing hashable key; an unhashable key still raises `TypeError`, matching Python.

PARAMETERS

Name	Type	Description
`key`	`K`	Key to look up
`default`	`V` | `None`	Value to return if missing

RETURNS

`V`

`.pop(key: K, default?: V, /) → V`

Remove `key` and return its value. Raises if the key isn't present unless a `default` is provided: in that case the missing-key path returns `default` and the dict is unchanged. Matches Python's `dict.pop(k, default)`.

PARAMETERS

Name	Type	Description
`key`	`K`	Key to remove
`default`	`V`	Value to return if the key is missing

RETURNS

`V`

`.popitem() → tuple[K, V]`

Remove and return the last inserted `(key, value)` pair. Raises if the dictionary is empty. Insertion order is deterministic, but use this only when consuming a `dict` as a stack is what you intend.

RETURNS

`tuple[K, V]`

`.setdefault(key: K, default: V | None = None, /) → V`

Return `d[key]` if it exists; otherwise set `d[key] = default` and return `default`. Useful for building grouped collections.

PARAMETERS

Name	Type	Description
`key`	`K`	Key
`default`	`V` | `None`	Value to set if missing

RETURNS

`V`

`.update(other?: dict[K, V] | Iterable[tuple[K, V]], /, **kwargs: V) → None`

Merge entries into this `dict`, overwriting matching keys. Accepts another `dict`, an iterable of `(key, value)` pairs, keyword args, OR a combination: `d.update(other, a=1, b=2)`. Mutates in place.

PARAMETERS

Name	Type	Description
`other`	`dict[K, V]` | `Iterable[tuple[K, V]]`	A `dict`, or an iterable of `(key, value)` pairs
`kwargs`	`V`	More entries, by keyword (`speed=2`)

RETURNS

`None`

`.fromkeys(iterable: Iterable[object], value: object = None, /) → dict`

A new `dict` with every item of `iterable` as a key, each set to `value`: `dict.fromkeys(["iron", "copper"], 0)` → `{"iron": 0, "copper": 0}`.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[object]`	Keys for the new `dict`
`value`	`object`	Value for every key (default `None`)

RETURNS

`dict`

`.copy() → dict[K, V]`

Return a shallow copy of the dictionary. Top-level keys/values are duplicated to a fresh `dict`; nested mutable values (`list` and `dict` values) are shared with the original.

RETURNS

`dict[K, V]`

`.clear() → None`

Remove all entries.

RETURNS

`None`

float

Returned by: decimal literals · `float(value)` · division with `/`

PROPERTIES

`.real: float`

The number itself, so any number can be read as `x.real`.

RETURNS

`float`

`.imag: float`

Always `0`: a real number has no imaginary part.

RETURNS

`float`

METHODS

`.conjugate() → float`

The number itself.

RETURNS

`float`

`.as_integer_ratio() → tuple[int, int]`

The exact fraction the number holds, in lowest terms: `(0.75).as_integer_ratio()` → `(3, 4)`.

RETURNS

`tuple[int, int]`

`.is_integer() → bool`

`True` when the number has no fractional part: `(2.0).is_integer()` → `True`.

RETURNS

`bool`

generator

Returned by: calling a function containing `yield` · generator expressions

PROPERTIES

`.__name__: str`

Name of the generator function that created this generator.

RETURNS

`str`

`.__qualname__: str`

Qualified name of the generator function that created this generator.

RETURNS

`str`

METHODS

`.send(value: object) → T`

Resume the generator and make `value` the result of its paused `yield`. Returns the next yielded value. Sending a non-`None` value before the first `yield` raises `TypeError`; completion raises `StopIteration`.

PARAMETERS

Name	Type	Description
`value`	`object`	Value delivered to the paused `yield`

RETURNS

`T`

`.throw(exception: BaseException | type) → T`

Raise an exception at the generator's paused `yield`. Returns the next value if the generator catches it and yields again; otherwise the exception propagates.

PARAMETERS

Name	Type	Description
`exception`	`BaseException` | `type`	Exception instance or exception class

RETURNS

`T`

`.close() → None`

Stop the generator by raising `GeneratorExit` at its paused `yield`. `finally` cleanup runs before this returns. A generator that yields while closing raises `RuntimeError`.

RETURNS

`None`

`.__iter__() → generator`

Return this generator. Generators are one-shot iterators.

RETURNS

`generator`

`.__next__() → T`

Resume the generator with `None` and return its next yielded value. Completion raises `StopIteration`.

RETURNS

`T`

int

Returned by: whole-number literals · `int(value)` · `len()` and other counts

PROPERTIES

`.real: int`

The number itself, so any number can be read as `x.real`.

RETURNS

`int`

`.imag: int`

Always `0`: a real number has no imaginary part.

RETURNS

`int`

`.numerator: int`

The number itself, as the top of a fraction over 1.

RETURNS

`int`

`.denominator: int`

Always `1`.

RETURNS

`int`

METHODS

`.conjugate() → int`

The number itself.

RETURNS

`int`

`.bit_length() → int`

How many binary digits the number needs, ignoring its sign: `(10).bit_length()` → `4`.

RETURNS

`int`

`.bit_count() → int`

How many 1 bits the number has, ignoring its sign: `(7).bit_count()` → `3`.

RETURNS

`int`

`.as_integer_ratio() → tuple[int, int]`

The number as a fraction: `(6).as_integer_ratio()` → `(6, 1)`.

RETURNS

`tuple[int, int]`

`.is_integer() → bool`

Always `True` for a whole number.

RETURNS

`bool`

list

Returned by: list literals `[1, 2, 3]` · `list(iterable)` · methods returning lists

PROPERTIES

`.length: int`

Number of items in the list. Same as `len(lst)`. Property: no parens.

RETURNS

`int`

METHODS

`.append(item: T, /) → None`

Add `item` to the end of the list. Returns `None`. Mutates in place. Raises if the list would exceed the interpreter's max-length cap.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to append

RETURNS

`None`

`.pop(index: int = -1, /) → T`

Remove and return one element. Default removes the last (`pop()`). Pass an integer index to remove a specific item: `lst.pop(0)` removes the first, `lst.pop(-1)` removes the last. Negative indices count from the end. Empty list or out-of-range index raises an error. Non-integer index raises.

PARAMETERS

Name	Type	Description
`index`	`int`	Position to remove (default: -1, the last item)

RETURNS

`T`

`.remove(item: T, /) → None`

Remove the first occurrence of `item` by value. Raises if not found. Use `item in lst` first if you need to check.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to remove

RETURNS

`None`

`.index(item: T, start: int = 0, end?: int, /) → int`

Return the index of the first occurrence of `item`. Optional `start` and `end` restrict the search to a slice (Python-style: negative indices count from the end, clamped to bounds). Raises if not found within the range.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to find
`start`	`int`	Start index (default 0)
`end`	`int`	End index, exclusive (default length)

RETURNS

`int`

`.count(item: T, /) → int`

Count occurrences of `item` in the list. Equality is by value (numbers, strings, booleans compared deeply).

PARAMETERS

Name	Type	Description
`item`	`T`	Value to count

RETURNS

`int`

`.sort(*, key: Callable | None = None, reverse: bool = False) → None`

Sort the list **in place** using `<` between mutually comparable keys. With `key=None` (the default), each value is its own key. Numeric and boolean keys compare across that family, strings compare lexicographically, and user objects may define the exact rich-comparison slots needed by `<`; unsupported pairs raise. `key=` is called once per item and must be **pure**: it cannot suspend the script or mutate game state. `reverse` is truth-tested, and the finite input is handled eagerly within the interpreter's collection limit.

PARAMETERS

Name	Type	Description
`key`	`Callable` | `None`	Function returning the value to compare for each item
`reverse`	`bool`	`True` to sort from largest to smallest

RETURNS

`None`

`.reverse() → None`

Reverse the list in place. Returns `None`.

RETURNS

`None`

`.copy() → list[T]`

Return a shallow copy of the list. Modifying the copy does not affect the original; nested mutable items are shared.

RETURNS

`list[T]`

`.extend(iterable: Iterable[T], /) → None`

Append every item from another finite iterable to the end of this list. Mutates in place. Raises if the result would exceed the max-length cap.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[T]`	Finite iterable of items to append

RETURNS

`None`

`.clear() → None`

Remove all items. Returns `None`. Equivalent to `lst[:] = []`.

RETURNS

`None`

OrderedDict

Returned by: `OrderedDict(...)` from `collections`

METHODS

`.move_to_end(key: K, last: bool = True) → None`

Move an existing key to the end, or to the front with `last=False`.

PARAMETERS

Name	Type	Description
`key`	`K`	Key to move
`last`	`bool`	`True` for the end, `False` for the front

RETURNS

`None`

`.popitem(last: bool = True) → tuple[K, V]`

Remove and return the last `(key, value)` pair, or the first with `last=False`.

PARAMETERS

Name	Type	Description
`last`	`bool`	`True` for the last pair, `False` for the first

RETURNS

`tuple[K, V]`

`.copy() → OrderedDict[K, V]`

A new `OrderedDict` with the same items.

RETURNS

`OrderedDict[K, V]`

set

Returned by: set literals `{1, 2}` · `set(iterable)` · set algebra operators

PROPERTIES

`.length: int`

Number of unique members. Same as `len(s)`. Property: no parens.

RETURNS

`int`

METHODS

`.add(item: T, /) → None`

Add `item` to the set. No effect if already present. The item may be any hashable value, including tuples of hashables and properly hashable user-class instances.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to add

RETURNS

`None`

`.remove(item: T, /) → None`

Remove `item`. Raises if not present. Use `.discard()` for the safe form.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to remove

RETURNS

`None`

`.discard(item: T, /) → None`

Remove `item` if present. No error if absent.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to remove

RETURNS

`None`

`.pop() → T`

Remove and return an arbitrary element. Order is deterministic (insertion order) but scripts shouldn't rely on it. Raises if the set is empty.

RETURNS

`T`

`.copy() → set[T]`

Return a shallow copy.

RETURNS

`set[T]`

`.has(item: T, /) → bool`

`True` if `item` is in the set. Equivalent to Python's `item in s`.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to check

RETURNS

`bool`

`.union(*others: Iterable[T]) → set[T]`

Return a new set with members from this set and every finite iterable in `others`. With no arguments, returns a copy.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`set[T]`

`.intersection(*others: Iterable[T]) → set[T]`

Return a new set containing members shared with every finite iterable in `others`. With no arguments, returns a copy.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`set[T]`

`.difference(*others: Iterable[T]) → set[T]`

Return a new set without members found in any finite iterable in `others`. With no arguments, returns a copy.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`set[T]`

`.symmetric_difference(other: Iterable[T], /) → set[T]`

Return a new set with members in exactly one of this set and the finite iterable `other`. Same as `a ^ b`.

PARAMETERS

Name	Type	Description
`other`	`Iterable[T]`	Finite iterable

RETURNS

`set[T]`

`.update(*others: Iterable[T]) → None`

Add every member from each finite iterable in `others`. Mutates in place; with no arguments, does nothing.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`None`

`.intersection_update(*others: Iterable[T]) → None`

Keep only members shared with every finite iterable in `others`. Mutates in place; with no arguments, does nothing.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`None`

`.difference_update(*others: Iterable[T]) → None`

Remove members found in any finite iterable in `others`. Mutates in place; with no arguments, does nothing.

PARAMETERS

Name	Type	Description
`others`	`Iterable[T]`	Other collections

RETURNS

`None`

`.symmetric_difference_update(other: Iterable[T], /) → None`

Replace this set with members in exactly one of this set and the finite iterable `other`.

PARAMETERS

Name	Type	Description
`other`	`Iterable[T]`	Finite iterable

RETURNS

`None`

`.issubset(other: Iterable[T], /) → bool`

`True` if every member of this set is also in the finite iterable `other`. Same as `a <= b` for sets.

PARAMETERS

Name	Type	Description
`other`	`Iterable[T]`	Finite iterable

RETURNS

`bool`

`.issuperset(other: Iterable[T], /) → bool`

`True` if this set contains every member of the finite iterable `other`. Same as `a >= b` for sets.

PARAMETERS

Name	Type	Description
`other`	`Iterable[T]`	Finite iterable

RETURNS

`bool`

`.isdisjoint(other: Iterable[T], /) → bool`

`True` if this set and the finite iterable `other` share no members.

PARAMETERS

Name	Type	Description
`other`	`Iterable[T]`	Finite iterable

RETURNS

`bool`

slice

Returned by: `slice(stop)` · `slice(start, stop, step)`

PROPERTIES

`.start: float | None`

Start bound, or `None` when omitted. For `slice(stop)`, `.start` is `None` and `.stop` is the argument.

RETURNS

`float | None`

`.stop: float | None`

Stop bound, or `None` when omitted.

RETURNS

`float | None`

`.step: float | None`

Step bound, or `None` when omitted. Sequence indexing rejects a step of `0`.

RETURNS

`float | None`

str

Returned by: string literals · `str(value)` · methods returning strings

PROPERTIES

`.length: int`

Number of characters in the string. Same as `len(s)`. Property: no parens.

RETURNS

`int`

METHODS

`.upper() → str`

Return a copy with all characters in uppercase.

RETURNS

`str`

`.lower() → str`

Return a copy with all characters in lowercase.

RETURNS

`str`

`.strip(chars: str | None = None, /) → str`

Return a copy with whitespace (or `chars` if given) trimmed from both ends. Passing `None` explicitly is the same as omitting `chars`. `chars` is treated as a SET of characters to strip, not a substring: `"...hello...".strip(".")` → `"hello"`.

PARAMETERS

Name	Type	Description
`chars`	`str` | `None`	Characters to strip, or `None` for whitespace

RETURNS

`str`

`.lstrip(chars: str | None = None, /) → str`

Like `strip()` but only trims from the left end. Passing `None` explicitly selects whitespace trimming.

PARAMETERS

Name	Type	Description
`chars`	`str` | `None`	Characters to strip, or `None` for whitespace

RETURNS

`str`

`.rstrip(chars: str | None = None, /) → str`

Like `strip()` but only trims from the right end. Passing `None` explicitly selects whitespace trimming.

PARAMETERS

Name	Type	Description
`chars`	`str` | `None`	Characters to strip, or `None` for whitespace

RETURNS

`str`

`.split(sep: str | None = None, maxsplit: int = -1) → list[str]`

Split into substrings. With `sep=None`, every Python whitespace character separates words, runs collapse, and leading/trailing whitespace is ignored unless a finite `maxsplit` leaves it in the unsplit remainder. With a non-empty string separator, matches are literal and empty fields are preserved. Both parameters accept keyword form.

PARAMETERS

Name	Type	Description
`sep`	`str` | `None`	Non-empty string separator or `None` for whitespace
`maxsplit`	`int`	Maximum splits (default `-1`)

RETURNS

`list[str]`

`.rsplit(sep: str | None = None, maxsplit: int = -1) → list[str]`

Like `split()`, but a `maxsplit` counts from the right, so the unsplit rest is the first item: `"a/b/c".rsplit("/", 1)` → `["a/b", "c"]`.

PARAMETERS

Name	Type	Description
`sep`	`str` | `None`	Non-empty string separator or `None` for whitespace
`maxsplit`	`int`	Maximum splits (default `-1`)

RETURNS

`list[str]`

`.splitlines(keepends: bool = False) → list[str]`

Split on Python line boundaries, including `\n`, `\r\n`, `\r`, vertical tab, form feed, Unicode NEL, and Unicode line/paragraph separators. Trailing boundaries do not add an extra empty item. Pass `keepends=True` to retain each boundary.

PARAMETERS

Name	Type	Description
`keepends`	`bool`	Keep line-boundary characters

RETURNS

`list[str]`

`.join(iterable: Iterable[str], /) → str`

Concatenate every string in a finite `iterable`, inserting this string as the separator between them. `",".join(["a", "b", "c"])` → `"a,b,c"`. All items must be strings.

PARAMETERS

Name	Type	Description
`iterable`	`Iterable[str]`	Finite iterable of strings

RETURNS

`str`

`.find(substring: str, start: int | None = 0, end: int | None = None, /) → int`

Return the lowest index where `substring` appears, or `-1` if not found. Optional `start` and `end` restrict the search to a slice. Negative indices count from the end.

PARAMETERS

Name	Type	Description
`substring`	`str`	Substring to find
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`int`

`.index(substring: str, start: int | None = 0, end: int | None = None, /) → int`

Like `find()`, but raises an error if the substring isn't present. Use `find()` if you want `-1` instead of an error.

PARAMETERS

Name	Type	Description
`substring`	`str`	Substring to find
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`int`

`.replace(old: str, new: str, count: int = -1, /) → str`

Return a copy with every occurrence of `old` replaced by `new`. Optional `count` limits the number of replacements: `s.replace("a", "b", 1)` only replaces the first.

PARAMETERS

Name	Type	Description
`old`	`str`	Substring to find
`new`	`str`	Replacement string
`count`	`int`	Max replacements (default: all)

RETURNS

`str`

`.startswith(prefix: str | tuple[str, ...], start: int | None = 0, end: int | None = None, /) → bool`

`True` if the string starts with `prefix`. `prefix` can be a single string OR a tuple containing only strings: `"abc".startswith(("ab", "xy"))` → `True`. Optional `start` / `end` restrict the check to a slice (negative indices count from the end).

PARAMETERS

Name	Type	Description
`prefix`	`str` | `tuple[str, ...]`	String or tuple of strings
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`bool`

`.endswith(suffix: str | tuple[str, ...], start: int | None = 0, end: int | None = None, /) → bool`

`True` if the string ends with `suffix`. Accepts a single string OR a tuple containing only strings. Optional `start` / `end` restrict the check to a slice: useful for checking suffixes inside a larger string without rebuilding.

PARAMETERS

Name	Type	Description
`suffix`	`str` | `tuple[str, ...]`	String or tuple of strings
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`bool`

`.count(substring: str, start: int | None = 0, end: int | None = None, /) → int`

Count non-overlapping occurrences of `substring`. Optional `start` and `end` restrict the search to a slice. Empty `substring` returns length + 1.

PARAMETERS

Name	Type	Description
`substring`	`str`	Substring to count
`start`	`int` | `None`	Start index
`end`	`int` | `None`	End index, exclusive

RETURNS

`int`

`.zfill(width: int, /) → str`

Pad with leading zeros to at least `width` characters. A leading `+` or `-` sign is preserved.

PARAMETERS

Name	Type	Description
`width`	`int`	Target width

RETURNS

`str`

`.center(width: int, fill: str = " ", /) → str`

Center the string within `width` characters, padding both sides with `fill` (default space). `fill` must be a single character.

PARAMETERS

Name	Type	Description
`width`	`int`	Target width
`fill`	`str`	Fill character (default space)

RETURNS

`str`

`.ljust(width: int, fill: str = " ", /) → str`

Left-justify within `width` characters, padding the right with `fill`.

PARAMETERS

Name	Type	Description
`width`	`int`	Target width
`fill`	`str`	Fill character (default space)

RETURNS

`str`

`.rjust(width: int, fill: str = " ", /) → str`

Right-justify within `width` characters, padding the left with `fill`.

PARAMETERS

Name	Type	Description
`width`	`int`	Target width
`fill`	`str`	Fill character (default space)

RETURNS

`str`

`.title() → str`

Return a titlecased copy: each run of cased characters starts with its Unicode titlecase mapping and continues in lowercase.

RETURNS

`str`

`.capitalize() → str`

Return a copy with the first character titlecased and the rest lowercased.

RETURNS

`str`

`.casefold() → str`

Apply Python-compatible Unicode case folding for caseless comparison. Functionally `.lower()` for ASCII and stronger for many Unicode characters.

RETURNS

`str`

`.swapcase() → str`

Swap the case of every character (lowercase ↔ uppercase).

RETURNS

`str`

`.isdigit() → bool`

`True` if the string is non-empty and contains only Unicode digit characters, including decimal and compatibility digits.

RETURNS

`bool`

`.isalpha() → bool`

`True` if the string is non-empty and contains only Unicode letters.

RETURNS

`bool`

`.isalnum() → bool`

`True` if the string is non-empty and contains only Unicode letters or numbers.

RETURNS

`bool`

`.isspace() → bool`

`True` if the string is non-empty and contains only whitespace characters.

RETURNS

`bool`

`.islower() → bool`

`True` if the string has at least one cased character and every cased character is lowercase.

RETURNS

`bool`

`.isupper() → bool`

`True` if the string has at least one cased character and every cased character is uppercase.

RETURNS

`bool`

`.istitle() → bool`

`True` when the string contains at least one cased character and each run of cased characters begins with an uppercase or Unicode titlecase character, followed only by lowercase characters. Characters without case separate the runs.

RETURNS

`bool`

`.isdecimal() → bool`

`True` when the string is not empty and every character is a decimal digit, `0` to `9` in any writing system. Stricter than `isdigit()`, which also accepts `²`.

RETURNS

`bool`

`.isnumeric() → bool`

`True` when the string is not empty and every character is numeric: digits, fractions such as `½` and numerals such as `Ⅷ`.

RETURNS

`bool`

`.isascii() → bool`

`True` when every character is ASCII, code point below 128. An empty string is `True`.

RETURNS

`bool`

`.isidentifier() → bool`

`True` when the string is spelled like a Python name, such as a variable name. Keywords such as `class` count too.

RETURNS

`bool`

`.isprintable() → bool`

`True` when every character prints visibly or is a space, and `False` for a line break, a tab or another control character. An empty string is `True`.

RETURNS

`bool`

`.rfind(substring: str, start: int | None = 0, end: int | None = None, /) → int`

Return the highest index where `substring` appears, or `-1` if not found. Mirror of `find()` from the right: useful for parsing the last separator of a string.

PARAMETERS

Name	Type	Description
`substring`	`str`	Substring to find
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`int`

`.rindex(substring: str, start: int | None = 0, end: int | None = None, /) → int`

Like `rfind()`, but raises if the substring isn't present.

PARAMETERS

Name	Type	Description
`substring`	`str`	Substring to find
`start`	`int` | `None`	Start index (default 0)
`end`	`int` | `None`	End index, exclusive (default length)

RETURNS

`int`

`.format(*args: object, **kwargs: object) → str`

Replace placeholders with arguments. `{}` takes the next positional argument, `{0}` a numbered one and `{name}` a keyword one, and a field can add a conversion and a format spec exactly like an f-string: `"{:>8.2f}|{name!r}".format(3.14159, name="ore")` → `" 3.14|'ore'"`.

PARAMETERS

Name	Type	Description
`args`	`object`	Values for the numbered `{}` fields
`kwargs`	`object`	Values for the named fields, by keyword (`name="world"`)

RETURNS

`str`

`.format_map(mapping: dict[str, object], /) → str`

Like `.format()` with every `{name}` field read from one `dict`: `"{name} is done".format_map(job)`. A numbered or `{}` field is a `ValueError`.

PARAMETERS

Name	Type	Description
`mapping`	`dict[str, object]`	The `dict` the named fields read their values from

RETURNS

`str`

`.expandtabs(tabsize: int = 8) → str`

Replace each tab with spaces up to the next multiple of `tabsize` columns, counted from the last line break: `"a\tb".expandtabs(4)` → `"a  b"`.

PARAMETERS

Name	Type	Description
`tabsize`	`int`	Columns between tab stops (default 8)

RETURNS

`str`

`.maketrans(x: str | dict[str | int, str | int | None], y: str | None = None, z: str | None = None, /) → dict[float, object]`

Build a table for `translate()`. `str.maketrans("abc", "xyz")` maps each character of the first string to the one at the same place in the second, a third string lists characters to delete, and one `dict` maps characters to their replacements.

PARAMETERS

Name	Type	Description
`x`	`str` | `dict[str | int, str | int | None]`	Characters to replace, or a `dict` of replacements
`y`	`str` | `None`	Replacement characters, as many as in `x`
`z`	`str` | `None`	Characters to delete

RETURNS

`dict[float, object]`

`.translate(table: dict[int, str | int | None], /) → str`

Replace characters through a table from `str.maketrans()`, or any `dict` from code points to a string, a code point, or `None` to delete: `"cab".translate(str.maketrans("abc", "xyz"))` → `"zxy"`.

PARAMETERS

Name	Type	Description
`table`	`dict[int, str | int | None]`	Table from `str.maketrans()`

RETURNS

`str`

`.partition(sep: str, /) → tuple[str, str, str]`

Split into three parts at the first occurrence of non-empty `sep`: `(before, sep, after)`. If `sep` isn't found, returns `(original, "", "")`.

PARAMETERS

Name	Type	Description
`sep`	`str`	Non-empty separator

RETURNS

`tuple[str, str, str]`

`.rpartition(sep: str, /) → tuple[str, str, str]`

Like `partition()` but splits at the LAST occurrence of non-empty `sep`. If not found, returns `("", "", original)`.

PARAMETERS

Name	Type	Description
`sep`	`str`	Non-empty separator

RETURNS

`tuple[str, str, str]`

`.removeprefix(prefix: str, /) → str`

Return a copy with `prefix` stripped from the start (only if present). Safer than slicing when you're not sure if the prefix is there.

PARAMETERS

Name	Type	Description
`prefix`	`str`	Prefix to remove

RETURNS

`str`

`.removesuffix(suffix: str, /) → str`

Return a copy with `suffix` stripped from the end (only if present).

PARAMETERS

Name	Type	Description
`suffix`	`str`	Suffix to remove

RETURNS

`str`

tuple

Returned by: tuple literals `(1, 2)` · `enumerate()` · methods returning tuples

PROPERTIES

`.length: int`

Number of items in the tuple. Same as `len(t)`. Property: no parens.

RETURNS

`int`

METHODS

`.index(item: T, start: int = 0, end?: int, /) → int`

Return the index of the first occurrence of `item`. Optional `start` and `end` restrict the search to a slice. Raises if not found within the range.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to find
`start`	`int`	Start index (default 0)
`end`	`int`	End index, exclusive (default length)

RETURNS

`int`

`.count(item: T, /) → int`

Count occurrences of `item` in the tuple.

PARAMETERS

Name	Type	Description
`item`	`T`	Value to count

RETURNS

`int`

AnalyzeResult

Returned by: bio_lab.analyze()

Get `AnalyzeResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"busy"`, `"input_empty"`, or `"invalid_specimen"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"busy"`,
`"input_empty"`,
`"invalid_specimen"`,
`"output_full"`

.message: str

Player-readable explanation of the analysis outcome.

RETURNS
`str`

.info: AnalyzeInfo | None

Completed `AnalyzeInfo`, or `None` when analysis was rejected.

RETURNS
`AnalyzeInfo | None`

BioExtractionResult

Returned by: PortableBioExtractor.extract()

Get `BioExtractionResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"not_mounted"`, `"scrambled"`, `"busy"`, `"not_at_location"`, `"not_scanned"`, `"cooling"`, or `"no_cargo_space"`.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"not_mounted"`, `"scrambled"`, `"busy"`, `"not_at_location"`, `"not_scanned"`, `"cooling"`, `"no_cargo_space"`

.message: str

Player-readable explanation of the biological extraction outcome.

RETURNS

`str`

.extracted: float

Tons committed to drone cargo by this extraction; **0** for every rejection.

RETURNS

`float`

BioScanResult

Returned by: PortableBioScanner.scan()

Get `BioScanResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"ok"`, `"not_mounted"`, `"scrambled"`, `"busy"`, or `"not_at_location"`.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"not_mounted"`, `"scrambled"`, `"busy"`, `"not_at_location"`

.message: str

Player-readable explanation of the biological scan outcome.

RETURNS

`str`

.scan: LifeFormScanResult | None

Completed `LifeFormScanResult`, or `None` when the scan was rejected.

RETURNS

`LifeFormScanResult | None`

Cell

Returned by: self.cell(sector) / self.cells()

Get `Cell` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Sector id, e.g. `"E13"` (row letter A-H + column 1-24). Parse it to compute the four orthogonal neighbors yourself: the grid hands you no `neighbors()` helper.

RETURNS

`str`

`.plant: str | None`

Species id of the planted flora, such as `"sunpetal"`, or `None` when the cell has no plant. This is the species id, not the `"seed_sunpetal"` item id. An empty cell may still contain a provider or uncollected item, so also check `.status`. It matches `SeedRecipe.species` and the species part of `"seed:<species>"`.

RETURNS

`str | None`

POSSIBLE VALUES

`"sunpetal"`, `"shadeleaf"`, `"dewmoss"`, `"lonethorn"`, `"packfern"`, `"twinvine"`, `"spitebud"`, `"sunspur"`, `"glowvine"`, `"crowncap"`, `"pondmoss"`, `"saltbloom"`, `"brinethorn"`, `"saltmate"`, `"grandbloom"`

`.growth: float`

Crop growth progress **0-1**. **1.0** means ready to harvest. Advances only while every requirement is met and pauses (never regresses) otherwise. Clearing the crop returns it to **0**; a partial Harvester harvest leaves the crop mature at **1.0** with its remaining forage.

RETURNS

`float`

`.status: str`

What occupies the sector or what its crop is doing: `"unknown"` until natural ground is scanned, `"empty"` (bare plantable ground), `"item"` (an uncollected surface item), `"base"` (the Harvester depot), `"provider"` (field equipment), `"growing"`, `"stalled"`, or `"mature"`. Player-created plants and providers remain visible. Only `"empty"` accepts planting or deployment.

RETURNS

`str`

POSSIBLE VALUES

`"unknown"`, `"empty"`, `"item"`, `"base"`, `"growing"`, `"stalled"`, `"mature"`, `"provider"`

`.lit: bool`

`True` while the cell has an active 24-hour Harvester light treatment or a powered Grow Lamp covers it. Satisfies the **Light** requirement; **Shade** plants need this `False`.

RETURNS

`bool`

`.watered: bool`

`True` while the cell has an active 24-hour Harvester water treatment, or while a powered, supplied Sprinkler covers it. Satisfies the **Water** requirement. A plant pauses when neither source is active.

RETURNS

`bool`

`.salted: bool`

`True` while the cell has an active 24-hour Harvester salt treatment, or while a powered, supplied Dispenser covers it. Satisfies the **Salt** requirement. A plant pauses when neither source is active.

RETURNS

`bool`

`.manual_light_remaining: float`

Hours remaining on the Harvester-applied light treatment, in the **0-24** range. This excludes Grow Lamp coverage. Each successful `light()` resets it to **24 hours**, replacing any remaining time.

RETURNS

`float`

`.manual_water_remaining: float`

Hours remaining on the Harvester-applied water treatment, in the **0-24** range. This excludes Sprinkler coverage. Each successful `water()` resets it to **24 hours**, replacing any remaining time.

RETURNS

`float`

`.manual_salt_remaining: float`

Hours remaining on the Harvester-applied salt treatment, in the **0-24** range. This excludes Dispenser coverage. Each successful `dispense_salt()` resets it to **24 hours**, replacing any remaining time.

RETURNS

`float`

`.fertilized: bool`

`True` while any Fertilizer dose remains on this cell. `False` when undosed or fully drained.

RETURNS

`bool`

`.fertilizer_remaining: float`

Hours of Fertilizer effect remaining. Each applied unit adds **8 hours**; reads **0** when none remains.

RETURNS

`float`

`.fertilizer_tier: int`

Active Fertilizer tier: **1**, **2**, or **3**. Reads **0** when no Fertilizer remains.

RETURNS

`int`

`.accelerated: bool`

`True` while any Growth Accelerant dose remains on this cell. `False` when undosed or fully drained.

RETURNS

`bool`

`.accelerant_remaining: float`

Hours of Growth Accelerant effect remaining. Each applied unit adds **8 hours**; reads **0** when none remains.

RETURNS

`float`

`.forage: int`

Whole **forage** banked by this crop. It rises with visible growth while requirements are met, includes diversity and field bonuses, and freezes at `growth == 1.0`. The Harvester moves as much as Inventory can hold and leaves any remainder on the crop. A Crop Automator moves what fits in its output, discards the remainder, and clears the cell; a completely full output blocks the job without changing the crop. Forage feeds Wildlife production and Plant Terraformer batches.

RETURNS

`int`

LifeFormScanResult

Returned by: `PortableBioScanner.scan().scan` after `status == "ok"` / journal biosite queries

Get `LifeFormScanResult` from the APIs listed here. It has no script constructor.

RELATED OBJECT TYPES

- `LifeFormSample`

PROPERTIES

`.coord: list[int]`

`[x, y]` whole-number coordinate of the scanned location.

RETURNS
`list[int]`

`.life_forms: list[LifeFormSample]`

List of `LifeFormSample` entries: 0-3 items per tile. Empty list means "scanned, nothing here."

RETURNS
`list[LifeFormSample]`

`.is_empty: bool`

`True` if no life forms found at this tile.

RETURNS
`bool`

LifeFormSample

Returned by: `PortableBioScanner.scan().scan.life_forms[i]` after `status == "ok"`

Get `LifeFormSample` from the APIs listed here. It has no script constructor.

PROPERTIES

.type: str

Life-form item id (e.g. `"ice_algae"`, `"snow_moss"`).

RETURNS

`str`

POSSIBLE VALUES

`"ice_algae"`,
`"snow_moss"`,
`"frost_lichen"`,
`"cold_spores"`,
`"ice_crust"`,
`"frost_fungus"`,
`"sea_algae"`,
`"tide_moss"`,
`"shore_lichen"`,
`"brine_plankton"`,
`"salt_crust"`,
`"coral_fungus"`,
`"vent_algae"`,
`"steam_moss"`,
`"heat_lichen"`,
`"hot_spores"`,
`"heat_crust"`,
`"vent_fungus"`,
`"sulfur_moss"`,
`"cinder_lichen"`,
`"ash_spores"`,
`"lava_algae"`,
`"magma_crust"`,
`"black_fungus"`,
`"cave_moss"`,
`"stone_lichen"`,
`"crystal_spores"`,
`"deep_algae"`,
`"stone_mat"`,
`"cave_fungus"`

`.tons: float`

Tons of this life-form available at the tile at scan time (peak value, not current state).

RETURNS
`float`

`.remaining_tons: float`

Tons currently available. Partial drone harvests reduce this; it returns to `.tons` after the coordinate's cooldown.

RETURNS
`float`

`.rarity: str`

Rarity tier: `"common"`, `"uncommon"`, or `"rare"`. Determines both essence produced per ton at the Essence Liquifier and the cooldown after extraction.

RETURNS
`str`

POSSIBLE VALUES
`"common"`, `"uncommon"`, `"rare"`

`.biome: str`

Native biome this species belongs to: `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, or `"deep"`. Fixed per species (never the terrain it grows on), and exactly the biome whose Essence Liquifier consumes it. Same value as `nocturna.life_form_biome(sample.type)`.

RETURNS
`str`

POSSIBLE VALUES
`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

PlantRequirement

Returned by: SeedRecipe.requirements

Import `PlantRequirement` with `from __builtins__ import PlantRequirement`. Arguments accept positional and keyword forms. Use `vars()` to obtain a dictionary for storage or messaging.

`PlantRequirement(kind: str, species: str | None = None) → PlantRequirement`

Create a local `PlantRequirement` value for your script. Creating this value does not change the world.

PARAMETERS

Name	Type	Description
`kind`	`str`	Cultivation condition: `"light"`, `"shade"`, `"water"`, `"salt"`, `"spacer"`, `"cluster"`, `"companion"`, or `"antagonist"`. Every entry in `SeedRecipe.requirements` is required; combined species expose multiple entries.
`species`	`str` | `None`	Related species key for relational conditions: the required neighbor for `"companion"`, or the forbidden neighbor for `"antagonist"`. `None` for all other kinds.

RETURNS

`PlantRequirement`

RAISES

Exception	Condition
`TypeError`	An argument has the wrong type, or the call has missing, excess, duplicate, or unknown arguments.

PROPERTIES

`.kind: str`

Cultivation condition: `"light"`, `"shade"`, `"water"`, `"salt"`, `"spacer"`, `"cluster"`, `"companion"`, or `"antagonist"`. Every entry in `SeedRecipe.requirements` is required; combined species expose multiple entries.

RETURNS

`str`

POSSIBLE VALUES

`"light"`, `"shade"`, `"water"`, `"salt"`, `"spacer"`, `"cluster"`, `"companion"`, `"antagonist"`

`.species: str | None`

Related species key for relational conditions: the required neighbor for `"companion"`, or the forbidden neighbor for `"antagonist"`. `None` for all other kinds.

RETURNS

`str | None`

POSSIBLE VALUES

`"sunpetal"`, `"shadeleaf"`, `"dewmoss"`, `"lonethorn"`, `"packfern"`, `"twinvine"`, `"spitebud"`, `"sunspur"`, `"glowvine"`, `"crowncap"`, `"pondmoss"`, `"saltbloom"`, `"brinethorn"`, `"saltmate"`, `"grandbloom"`

PortableBioExtractor

Returned by: `self.bio_extractor` (drones)

Get `PortableBioExtractor` from the APIs listed here. It has no script constructor.

METHODS

`.extract() → BioExtractionResult`

Harvest at the discovered permanent biosite under a hovering drone; the script waits until it finishes. The drone stays occupied until completion, including across a script stop and restart. The module has a 25 t chamber for one life-form type; Cargo Pods add capacity. Partial depletion persists and cooldown starts only when the site is empty.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`BioExtractionResult`	`.status`, `.message`	`.extracted`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	Biological material extracted into drone cargo: `.extracted` t.
`"not_mounted"`	rejection	The drone has no Portable Bio Extractor mounted.
`"scrambled"`	transient	The drone's electronics are radiation-scrambled and cannot extract.
`"busy"`	transient	Biological extraction is already in progress for this drone or at this site.
`"not_at_location"`	rejection	The drone must finish its route and hover at a biological site.
`"not_scanned"`	rejection	There is no scanned and recorded biological site under the drone.
`"cooling"`	transient	The depleted biological site is still replenishing.
`"no_cargo_space"`	rejection	Cannot extract the remaining life forms: every compatible container is full or already assigned to another type. Each extractor chamber and Cargo Pod holds one life-form type at a time.

PortableBioScanner

Returned by: `self.bio_scanner` (drones)

Get `PortableBioScanner` from the APIs listed here. It has no script constructor.

METHODS

`.scan() → BioScanResult`

Yielding scan at the hovering drone's current whole-number coordinate. A valid non-site coordinate completes with an empty biological scan. Repeat scans are free.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`BioScanResult`	`.status`, `.message`	`.scan`

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The biological scan completed at the drone's current coordinate.
`"not_mounted"`	rejection	The drone has no Portable Bio Scanner mounted.
`"scrambled"`	transient	The drone's electronics are radiation-scrambled and cannot scan.
`"busy"`	transient	The drone is occupied by another biological operation.
`"not_at_location"`	rejection	The drone must finish its route and hover at a whole-number biological survey coordinate.

SeedRecipe

Returned by: `self.recipes()` (Seed Maker)

Get `SeedRecipe` from the APIs listed here. It has no script constructor.

PROPERTIES

`.tier: int`

Production tier for this discovered Seed Maker recipe. Seed recipes are Tier 1 because their inputs are harvested source materials.

RETURNS

`int`

`.species: str`

Bare flora species id this recipe produces, such as `"sunpetal"`. It matches `Cell.plant` and identifies the biological species, not the physical seed item used by planting commands.

RETURNS

`str`

POSSIBLE VALUES

`"sunpetal"`, `"shadeleaf"`, `"dewmoss"`, `"lonethorn"`, `"packfern"`, `"twinvine"`, `"spitebud"`, `"sunspur"`, `"glowvine"`, `"crowncap"`, `"pondmoss"`, `"saltbloom"`, `"brinethorn"`, `"saltmate"`, `"grandbloom"`

`.seed_id: str`

Physical seed item id produced by this recipe, such as `"seed_sunpetal"`. Pass this value to the Harvester's `load_seed()` and `plant()`, or to a Crop Automator's `plant()` after loading the same item into its input.

RETURNS

`str`

POSSIBLE VALUES

`"seed_sunpetal"`, `"seed_shadeleaf"`, `"seed_dewmoss"`, `"seed_lonethorn"`, `"seed_packfern"`, `"seed_twinvine"`, `"seed_spitebud"`, `"seed_sunspur"`, `"seed_glowvine"`, `"seed_crowncap"`, `"seed_pondmoss"`, `"seed_saltbloom"`, `"seed_brinethorn"`, `"seed_saltmate"`, `"seed_grandbloom"`

`.blend: list[str]`

The exact list of 3 life-form ids that yields this seed (e.g. `["ice_algae", "sea_algae", "vent_moss"]`). Order doesn't matter: pass these three to `combine()` again to reproduce the seed.

RETURNS

`list[str]`

`.requirements: list[PlantRequirement]`

Lists every cultivation condition as a `PlantRequirement`. Each entry has a `.kind` and optional `.species`, identifying required companions or forbidden antagonists. Every listed requirement must be satisfied for growth to advance.

RETURNS

`list[PlantRequirement]`

`.requirement: str`

Compact summary of the requirement kinds, e.g. `"light"`, `"light+water"`, or `"salt+companion"`. Use `.requirements` for programmable layout decisions because this summary intentionally omits the companion/antagonist species identity.

RETURNS

`str`

POSSIBLE VALUES

`"light"`, `"shade"`, `"water"`, `"spacer"`, `"cluster"`, `"companion"`, `"antagonist"`, `"light+spacer"`, `"light+water"`, `"shade+cluster"`, `"water+cluster"`, `"salt"`, `"salt+spacer"`, `"salt+companion"`, `"light+water+spacer"`

`.growth_time: float`

Hours of met-requirement time to reach maturity (`growth == 1.0`). Longer-growing species are the harder, higher-value ones; growth only advances while all requirements hold.

RETURNS

`float`

`.base_yield: int`

Whole forage units this species yields before diversity, treatment, and field bonuses. Compare this with `.growth_time` and `.requirements` when choosing a crop.

RETURNS

`int`

SeedResult

Returned by: seed_maker.combine()

Get `SeedResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

`"seed_found"`, `"sludge"`, `"locked"`, `"busy"`, `"missing_life_forms"`, or `"output_full"`.

RETURNS

`str`

POSSIBLE VALUES

`"seed_found"`, `"sludge"`, `"locked"`, `"busy"`, `"missing_life_forms"`, `"output_full"`

.message: str

Player-readable explanation of the combine trial outcome.

RETURNS

`str`

`.seed_id: str | None`

Stable seed item id deposited into output, or `None` when no seed was produced.

RETURNS

`str | None`

POSSIBLE VALUES

`"seed_sunpetal"`, `"seed_shadeleaf"`, `"seed_dewmoss"`, `"seed_lonethorn"`, `"seed_packfern"`, `"seed_twinvine"`, `"seed_spitebud"`, `"seed_sunspur"`, `"seed_glowvine"`, `"seed_crowncap"`, `"seed_pondmoss"`, `"seed_saltbloom"`, `"seed_brinethorn"`, `"seed_saltmate"`, `"seed_grandbloom"`

`.species: str | None`

Stable discovered flora species id, or `None` when no seed was produced.

RETURNS

`str | None`

POSSIBLE VALUES

`"sunpetal"`, `"shadeleaf"`, `"dewmoss"`, `"lonethorn"`, `"packfern"`, `"twinvine"`, `"spitebud"`, `"sunspur"`, `"glowvine"`, `"crowncap"`, `"pondmoss"`, `"saltbloom"`, `"brinethorn"`, `"saltmate"`, `"grandbloom"`

CacheInfo

Returned by: `functools.lru_cache(fn).cache_info()` · `functools.cache(fn).cache_info()`

Get `CacheInfo` from the APIs listed here. It has no script constructor.

PROPERTIES

.hits: int

Calls answered from the cache

RETURNS
`int`

.misses: int

Calls that had to run the function

RETURNS
`int`

.maxsize: int

How many results this cache keeps before dropping the least recently used one

RETURNS
`int`

`.currsize: int`

How many results are stored right now

RETURNS

`int`

Field

Returned by: `dataclasses.fields()`

Get `Field` from the APIs listed here. It has no script constructor.

PROPERTIES

`.name: str`

Field name as declared

RETURNS
`str`

`.default: object`

The field's default value, or `MISSING` when it has none

RETURNS
`object`

`.default_factory: Callable`

The zero-argument function that builds this field's default, or `MISSING` when there is none

RETURNS
`Callable`

`.init: bool`

`True` when the field is a parameter of the generated constructor

RETURNS

`bool`

`.repr: bool`

`True` when the field appears in the generated text form

RETURNS

`bool`

`.compare: bool`

`True` when the field takes part in equality and ordering

RETURNS

`bool`

`.kw_only: bool`

`True` when the field must be passed by name

RETURNS

`bool`

Match

Returned by: `re.search()` · `re.match()` · `re.fullmatch()`

Get `Match` from the APIs listed here. It has no script constructor.

PROPERTIES

`.pattern: str`

The regular expression pattern string that produced this match.

RETURNS
`str`

`.string: str`

The string that was searched.

RETURNS
`str`

`.lastindex: int | None`

Number of the last group that matched, or `None` if no group did.

RETURNS
`int | None`

`.lastgroup: str | None`

Name of the last group that matched, or `None` if it has no name or no group matched.

RETURNS

`str | None`

`.pos: int`

Where the search started in the string.

RETURNS

`int`

`.endpos: int`

Where the search stopped in the string.

RETURNS

`int`

`.re: Pattern`

The compiled `Pattern` that produced this match.

RETURNS

`Pattern`

METHODS

`.group(group: int | str = 0, /, *groups: int | str) → str | None`

Return the matched text for a group, by number or by `(?P<name>...)` name. Group `0` is the whole match, and several groups return a `tuple`: `m.group(1, "unit")`. Optional groups that did not match return `None`; an unknown group raises. `m[1]` is the same as `m.group(1)`.

PARAMETERS

Name	Type	Description
`group`	`int` | `str`	Capture group number or name; default 0
`groups`	`int` | `str`	More groups, which make the result a `tuple`

RETURNS

`str | None`

`.groups(default: object = None) → tuple[str | None, ...]`

Return a `tuple` of captured groups, excluding group `0`. Groups that did not match use `default`, which is `None` if omitted.

PARAMETERS

Name	Type	Description
`default`	`object`	Value for unmatched optional groups

RETURNS

`tuple[str | None, ...]`

`.start(index: int | str = 0) → int`

Start character index for the group. Unmatched optional groups return `-1`.

PARAMETERS

Name	Type	Description
`index`	`int` | `str`	Capture group index; default 0

RETURNS

`int`

`.end(index: int | str = 0) → int`

End character index for the group. Unmatched optional groups return `-1`.

PARAMETERS

Name	Type	Description
`index`	`int` | `str`	Capture group index; default 0

RETURNS

`int`

`.span(index: int | str = 0) → tuple[int, ...]`

Return `(start, end)` for the group. Unmatched optional groups return `(-1, -1)`.

PARAMETERS

Name	Type	Description
`index`	`int` | `str`	Capture group index; default 0

RETURNS

`tuple[int, ...]`

`.__getitem__(group: int | str, /) → str | None`

`m[1]` gives the same text as `m.group(1)`, and `m["name"]` as `m.group("name")`; a group that took no part in the match gives `None`.

PARAMETERS

Name	Type	Description
`group`	`int` | `str`	Capture group number or name; default 0

RETURNS

`str | None`

`.groupdict(default: object = None) → dict[str, str | None]`

Return a `dict` of every named group's text, by name. Named groups that did not match use `default`, which is `None` if omitted.

PARAMETERS

Name	Type	Description
`default`	`object`	Value for unmatched optional groups

RETURNS

`dict[str, str | None]`

`.expand(template: str, /) → str`

Fill in a replacement template the way `re.sub()` does, from this match: `m.expand(r"\2-\1")`.

PARAMETERS

Name	Type	Description
`template`	`str`	Template with backreferences

RETURNS

`str`

Pattern

Returned by: `re.compile()`

Get `Pattern` from the APIs listed here. It has no script constructor.

PROPERTIES

`.pattern: str`

The pattern text this was compiled from.

RETURNS
`str`

`.flags: int`

The flags this was compiled with.

RETURNS
`int`

`.groups: int`

How many capture groups the pattern has.

RETURNS
`int`

`.groupindex: dict[str, float]`

A `dict` from each `(?P<name>...)` group name to its number.

RETURNS

`dict[str, float]`

METHODS

`.search(string: str, pos: int = 0, endpos: int | None = None) → Match | None`

Like `re.search()` with this pattern, looking only between `pos` and `endpos`: a `Match`, or `None` if nothing there matches.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`pos`	`int`	Index to start at (default 0)
`endpos`	`int` | `None`	Index to stop before (default the end)

RETURNS

`Match | None`

`.match(string: str, pos: int = 0, endpos: int | None = None) → Match | None`

Like `re.match()` with this pattern, anchored at `pos`: a `Match`, or `None` if the text at `pos` does not match.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`pos`	`int`	Index to start at (default 0)
`endpos`	`int` | `None`	Index to stop before (default the end)

RETURNS

`Match | None`

`.fullmatch(string: str, pos: int = 0, endpos: int | None = None) → Match | None`

Like `re.fullmatch()` with this pattern, over the text from `pos` to `endpos`: a `Match`, or `None` if any part is left unmatched.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`pos`	`int`	Index to start at (default 0)
`endpos`	`int` | `None`	Index to stop before (default the end)

RETURNS

`Match | None`

`.findall(string: str, pos: int = 0, endpos: int | None = None) → list`

Like `re.findall()` with this pattern, between `pos` and `endpos`.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`pos`	`int`	Index to start at (default 0)
`endpos`	`int` | `None`	Index to stop before (default the end)

RETURNS

`list`

`.finditer(string: str, pos: int = 0, endpos: int | None = None) → Iterator[Match]`

Like `re.finditer()` with this pattern, between `pos` and `endpos`.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`pos`	`int`	Index to start at (default 0)
`endpos`	`int` | `None`	Index to stop before (default the end)

RETURNS

`Iterator[Match]`

`.sub(repl: str | Callable, string: str, count: int = 0) → str`

Like `re.sub()` with this pattern.

PARAMETERS

Name	Type	Description
`repl`	`str` | `Callable`	Replacement text, or a function from `Match` to text
`string`	`str`	Text to search
`count`	`int`	Maximum replacements; 0 means all

RETURNS

`str`

`.subn(repl: str | Callable, string: str, count: int = 0) → tuple[str, int]`

Like `re.subn()` with this pattern.

PARAMETERS

Name	Type	Description
`repl`	`str` | `Callable`	Replacement text, or a function from `Match` to text
`string`	`str`	Text to search
`count`	`int`	Maximum replacements; 0 means all

RETURNS

`tuple[str, int]`

`.split(string: str, maxsplit: int = 0) → list`

Like `re.split()` with this pattern.

PARAMETERS

Name	Type	Description
`string`	`str`	Text to search
`maxsplit`	`int`	Maximum splits; 0 means all

RETURNS

`list`

CatalogedCreature

Returned by: journal.cataloged_creatures(planet_id)

Get `CatalogedCreature` from the APIs listed here. It has no script constructor.

PROPERTIES

`.creature_id: str`

Stable creature id accepted by `habitat.set_revival_target(creature_id)`.

RETURNS

`str`

POSSIBLE VALUES

`"salt_tortoise"`, `"magmatic_annelid"`, `"mycelial_husk"`, `"mantle_strider"`, `"glasswing_mantis"`, `"veil_mantle"`, `"vault_crab"`, `"tidal_cephalopod"`, `"bone_walker"`, `"vent_drifter"`, `"hive_sentinel"`, `"hollow_choir"`, `"ferric_sea_lily"`, `"crustal_echo"`, `"glacial_wyrm"`, `"spire_drake"`

`.name: str`

Player-facing creature name. Use for readable output; use `.creature_id` for automation.

RETURNS

`str`

`.rarity: str`

Rarity tier: `"common" | "uncommon" | "rare" | "legendary"`. Rarity determines the revival reagent quantities.

RETURNS

`str`

POSSIBLE VALUES

`"common"`, `"uncommon"`, `"rare"`, `"legendary"`

`.fragment_ids: list[str]`

The five analyzed fragment ids that completed this creature's catalog entry, in biome order.

RETURNS

`list[str]`

`.feed_item_id: str`

The exact feed item this creature accepts. Stage at least `.revive_feed_required` units in the Habitat before revival.

RETURNS

`str`

POSSIBLE VALUES

`"feed_salt_tortoise"`, `"feed_magmatic_annelid"`, `"feed_mycelial_husk"`, `"feed_mantle_strider"`, `"feed_glasswing_mantis"`, `"feed_veil_mantle"`, `"feed_vault_crab"`, `"feed_tidal_cephalopod"`, `"feed_bone_walker"`, `"feed_vent_drifter"`, `"feed_hive_sentinel"`, `"feed_hollow_choir"`, `"feed_ferric_sea_lily"`, `"feed_crustal_echo"`, `"feed_glacial_wyrm"`, `"feed_spire_drake"`

`.feed_recipe_id: str`

The Feed Maker recipe id that produces `.feed_item_id`. The matching recipe must still be unlocked through Bio Orders; find its current `Recipe` in `feed_maker.list_recipes()` for ingredient quantities.

RETURNS

`str`

POSSIBLE VALUES

`"craft_feed_salt_tortoise"`, `"craft_feed_magmatic_annelid"`, `"craft_feed_mycelial_husk"`, `"craft_feed_mantle_strider"`, `"craft_feed_glasswing_mantis"`, `"craft_feed_veil_mantle"`, `"craft_feed_vault_crab"`, `"craft_feed_tidal_cephalopod"`, `"craft_feed_bone_walker"`, `"craft_feed_vent_drifter"`, `"craft_feed_hive_sentinel"`, `"craft_feed_hollow_choir"`, `"craft_feed_ferric_sea_lily"`, `"craft_feed_crustal_echo"`, `"craft_feed_glacial_wyrm"`, `"craft_feed_spire_drake"`

`.revive_feed_required: int`

Minimum whole feed units that must be staged in the Habitat after selecting this creature and before `revive()` can start.

RETURNS

`int`

`.revive_reagents: dict[str, int]`

Exact revival shopping list as `{reagent_id: quantity}` for this creature's rarity. Stage every listed amount through the Habitat's `reagents` input.

RETURNS

`dict[str, int]`

CatalogedFragment

Returned by: journal.cataloged_fragments(planet_id)

Get `CatalogedFragment` from the APIs listed here. It has no script constructor.

PROPERTIES

`.fragment_id: str`

Stable fragment id (e.g. `"gw_cranial_plate"`). Matches the keys in `BioOrder.requires`, so this is what scripts use to plan order fulfillment.

RETURNS

`str`

POSSIBLE VALUES

`"gw_cranial_plate"`,
`"gw_caudal_fin"`,
`"gw_cardiac_node"`,
`"gw_jaw_fang"`,
`"gw_spinal_vertebra"`,
`"vc_dorsal_carapace"`,
`"vc_mandible_claw"`,
`"vc_antenna_cluster"`,
`"vc_walking_leg"`,
`"vc_eye_stalk"`,
`"oc_cranium"`,
`"oc_tentacle_arm"`,
`"oc_chitin_beak"`,
`"oc_ink_sac"`,
`"oc_lens_eye"`,
`"bw_skull"`,
`"bw_foreclaw"`,
`"bw_ribcage"`,
`"bw_hindlimb"`,
`"bw_tail_spike"`,
`"vd_bell"`,
`"vd_nematocyst"`,
`"vd_neural_mesh"`,
`"vd_photophore"`,
`"vd_tendril"`,
`"mh_fruiting_body"`,
`"mh_spore_pod"`,
`"mh_mycelium_root"`,
`"mh_chitin_node"`,
`"mh_stigmatic_disc"`,
`"hs_mandible"`,
`"hs_wing_membrane"`,
`"hs_thorax_plate"`,
`"hs_abdomen_segment"`,

```
"hs_compound_eye",
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",
"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw",
"st_beak",
"st_carapace_neural",
"vm_cephalic_horn",
```

`"vm_wing_sheet"`,
`"vm_gill_filament"`,
`"vm_tail_barb"`,
`"vm_ventral_photophore"`

`.name: str`

Player-facing fragment name (e.g. "Beak"). Use for readable output; use `.fragment_id` when matching `BioOrder.requires` or moving the item.

RETURNS

`str`

`.biome: str`

Biome where this fragment is found: `"frozen"` for Nocturna's starter biome. Filter the result list by biome to scope to a specific outpost's biome.

RETURNS

`str`

POSSIBLE VALUES

`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.coords: list[float]`

World coordinates `[x, y]` where the fragment can be collected. Pass to `bio_collector.collect(coords)` from the Bio Collector at the matching outpost.

RETURNS

`list[float]`

`.rarity: str`

Rarity tier: `"common" | "uncommon" | "rare" | "legendary"`. Inherited from the parent creature.

RETURNS

`str`

POSSIBLE VALUES

`"common"`, `"uncommon"`, `"rare"`, `"legendary"`

Marker

Returned by: markers.get() / markers.list()

Get `Marker` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Stable identity you passed to `markers.place(...)`. Pass it back to `markers.get(...)` or `markers.remove(...)`, and use its prefix to group families of markers.

RETURNS

`str`

`.x: float`

World X coordinate in meters. Feed it straight to `self.nav.set_target(m.x, m.y)`, a drone `self.go_to(m.x, m.y)`, or `construction_blueprint.plan_structure(kind, m.x, m.y)`.

RETURNS

`float`

`.y: float`

World Y coordinate in meters.

RETURNS

`float`

`.label: str`

Display text shown on the map and in the Markers list. Empty string means the marker was placed without one, and the map shows its id instead.

RETURNS

`str`

`.note: str`

Longer note shown when you hover the marker. Empty string means no note was written.

RETURNS

`str`

`.icon: str`

Glyph drawn inside the marker pin, such as `"x"`, `"check"`, or `"hammer"`.

RETURNS

`str`

POSSIBLE VALUES

`"pin"`, `"x"`, `"check"`, `"circle"`, `"flag"`, `"crosshair"`, `"warning"`, `"hammer"`, `"resource"`, `"power"`, `"fluid"`, `"star"`

`.color: str`

Marker color, such as `"accent"` or `"error"`.

RETURNS

`str`

POSSIBLE VALUES

`"neutral"`, `"accent"`, `"success"`, `"warning"`, `"error"`, `"violet"`

BlueprintPlanResult

Returned by: `construction_blueprint` planning commands

Get `BlueprintPlanResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Stable planning outcome. Branch on this field before reading `.blueprint_ids`.

RETURNS

`str`

POSSIBLE VALUES

`"not_found"`, `"not_upgrade_item"`, `"wrong_machine_type"`, `"under_construction"`, `"tier_not_ready"`, `"ok"`, `"locked"`, `"invalid_kind"`, `"invalid_rotation"`, `"invalid_medium"`, `"invalid_axis"`, `"invalid_layer"`, `"out_of_bounds"`, `"wrong_target"`, `"unsurveyed_target"`, `"too_hard"`, `"target_claimed"`, `"occupied"`, `"clearance"`, `"invalid_route"`, `"blocked"`, `"already_exists"`, `"already_queued"`, `"ambiguous_target"`, `"nothing_here"`

`.message: str`

Player-readable explanation of the exact planning outcome.

RETURNS

`str`

`.blueprint_ids: list[str]`

Newly created blueprint ids. Empty for every outcome except `"ok"`.

RETURNS

`list[str]`

BatteryChargerSlot

Returned by: battery_charger.slots(), battery_charger.status()

Get `BatteryChargerSlot` from the APIs listed here. It has no script constructor.

PROPERTIES

.index: int

Zero-based physical socket index.

RETURNS
`int`

.item_id: str

Battery item ID, or an empty string for an empty socket.

RETURNS
`str`

.stored_wh: float

Stored energy in watt-hours.

RETURNS
`float`

`.capacity_wh: float`

Battery capacity in watt-hours; zero when empty.

RETURNS

`float`

`.level: float`

Stored energy as a fraction of capacity; zero when empty.

RETURNS

`float`

`.target: float | None`

Requested target fraction, or `None` when no unfinished job owns the cell.

RETURNS

`float | None`

`.state: str`

Current socket state. `ready` means unqueued and removable; inspect `level` for charge.

RETURNS

`str`

POSSIBLE VALUES

`"empty"`, `"ready"`, `"handling"`, `"queued"`, `"charging"`, `"paused"`

`.rate_w: float`

Current charging power in watts.

RETURNS

`float`

ActionResult

Returned by: Gameplay commands with no extra result fields

Get `ActionResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Stable result code for scripts to check. `.message` explains what happened.

RETURNS

`str`

POSSIBLE VALUES

`"queued"`,
`"target_reached"`,
`"empty"`,
`"under_construction"`,
`"ok"`, `"no_op"`,
`"not_mounted"`,
`"invalid"`,
`"out_of_bounds"`, `"busy"`,
`"not_at_site"`,
`"not_surveyed"`,
`"too_hard"`,
`"no_cargo_space"`,
`"no_power"`,
`"not_enough_power"`,
`"not_found"`, `"locked"`,
`"already_active"`,
`"wrong_position"`,
`"insufficient_materials"`,
`"paused_no_power"`,
`"cargo_present"`,
`"blocked"`, `"paused"`,
`"canceled"`, `"not_ready"`,
`"not_local"`,
`"same_endpoint"`,
`"unsupported_source"`,
`"source_is_vehicle"`,
`"unsupported_target"`,
`"target_is_vehicle"`,
`"incompatible"`,
`"duplicate"`,
`"invalid_type"`, `"path"`,
`"wall"`, `"exit"`,
`"in_progress"`,
`"ongoing"`, `"win"`,
`"loss"`, `"draw"`,

"occupied", "no_game",
"rejected", "booting",
"already_booted",
"activating",
"already_online",
"boot_required",
"initializing",
"power_required",
"insufficient_credits",
"inventory_full",
"not_toggleable",
"not_connected",
"no_script",
"script_running",
"script_paused",
"editor_busy",
"variant_not_found",
"incompatible_variant",
"source_too_large",
"already_running",
"not_powered",
"already_paused",
"started",
"already_repaired",
"no_source",
"already_testing",
"too_far",
"already_here",
"overheated", "moving",
"dropped",
"invalid_seed",
"no_seed", "holding",
"not_empty",
"base_sector", "no_kit",
"invalid_kit",
"not_plantable",
"insufficient_water",
"tank_empty",
"already_full",
"no_salt",
"invalid_input",
"no_plant",

"already_mature",
"tier_conflict",
"no_dose", "nothing",
"script_present",
"partial", "not_mature",
"no_forage", "correct",
"incorrect",
"already_completed_correct",
"already_completed_incorrect", "accepted",
"wrong_planet",
"wrong_contract",
"unknown_contract",
"key_is_planet",
"unknown_key",
"station_not_found",
"not_at_station",
"not_at_service_point",
"item_not_in_inventory",
"unknown_module",
"slot_not_compatible",
"slot_occupied",
"invalid_slot",
"capability_already_mounted", "slot_empty",
"holder_not_empty",
"invalid_internal_slot",
"not_container",
"item_not_accepted",
"internal_slot_occupied",
"internal_slot_empty",
"container_not_empty",
"charging",
"not_docked",
"station_offline",
"already_dispatched",
"unknown_recipe",
"offline",
"recipe_locked",
"material_mismatch",
"material_present",

```
"invalid_channel",
"invalid_value",
"channel_limit",
"invalid_key",
"invalid_coords",
"invalid_icon",
"invalid_color",
"invalid_text",
"limit_reached",
"entry_limit",
"unknown_order",
"completed",
"out_of_range",
"scrambled",
"drill_not_found",
"invalid_target",
"wrong_engine_for_module
",
"cargo_capacity_exceeded
", "module_not_mounted",
"hot_cargo_requires_plati
ng", "refueling",
"no_oil",
"not_stranded",
"no_rescue",
"worker_not_present",
"construction_dependency
", "not_upgrade_item",
"not_at_outpost",
"wrong_machine_type",
"tier_too_high",
"tier_not_ready",
"not_undeployable",
"self_target",
"docked_drone",
"is_home", "name_empty",
"name_too_long",
"name_taken",
"output_full",
"complete", "no_input",
"no_active",
"cargo_occupied",
```

```
"no_fragment",
"input_occupied",
"source_empty",
"source_busy",
"wrong_outpost",
"invalid_source",
"invalid_reagent",
"invalid_qty",
"invalid_properties",
"invalid_property_match",
"insufficient_input",
"recipe_mismatch",
"input_empty",
"not_analyzed",
"invalid_specimen",
"chamber_occupied",
"not_in_input",
"invalid_fragment",
"destroyed",
"unknown_gene",
"invalid_recipe",
"wrong_materials",
"wrong_fragment",
"no_recipe",
"conditioned", "burned",
"no_run",
"species_exists",
"unknown_creature",
"not_cataloged",
"no_target",
"wrong_feed",
"insufficient_feed",
"insufficient_reagents",
"no_colony",
"not_established",
"insufficient_capacity",
"unknown_node",
"already_purchased",
"population_locked",
"insufficient_insight",
"output_busy",
"no_material"
```

`.message: str`

Player-readable explanation of the exact command outcome. Suitable for logs and player-facing diagnostics.

RETURNS

`str`

CommandResult

Returned by: Component.next_command()

Get `CommandResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"ok"` when a command was consumed, otherwise `"empty"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"empty"`

`.message: str`

Player-readable explanation of the command dequeue result.

RETURNS
`str`

`.command: ScriptCommand | None`

Consumed `ScriptCommand`, or `None` when the queue was empty.

RETURNS
`ScriptCommand | None`

CountResult

Returned by: Queue, discard, clear, and bulk-count commands

Get `CountResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"ok"` when one or more entries changed, `"no_op"` when nothing changed, or a command-specific rejection such as `"invalid_channel"` or `"invalid_key"`.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"no_op"`, `"invalid_channel"`, `"invalid_key"`

`.message: str`

Player-readable explanation of the count operation.

RETURNS
`str`

`.count: int`

Whole-number entries or units affected by the command.

RETURNS
`int`

CropJob

Returned by: Crop Automator `current_job()` and `get_queue()`

Get `CropJob` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: int`

Stable whole-number job id.

RETURNS
`int`

`.action: str`

Requested work: `"harvest"`, `"plant"`, or `"apply"`.

RETURNS
`str`

POSSIBLE VALUES
`"harvest"`, `"plant"`, `"apply"`

`.sector: str`

Target field sector.

RETURNS
`str`

`.item_id: str | None`

Requested seed or treatment item, or `None` for harvest jobs.

RETURNS

`str | None`

`.state: str`

`"working"`, `"blocked"`, or `"queued"`.

RETURNS

`str`

POSSIBLE VALUES

`"working"`, `"blocked"`, `"queued"`

`.progress: float`

Completed fraction from **0-1**. Pending jobs read **0**.

RETURNS

`float`

`.blocker: str | None`

Exact physical blocker, or `None` while working or queued.

RETURNS

`str | None`

POSSIBLE VALUES

`"not_placed"`, `"no_power"`, `"no_seed"`, `"no_material"`, `"output_full"`, `"results_full"`

CropJobResult

Returned by: Crop Automator `next_result()`

Get `CropJobResult` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

Terminal job outcome, or `"empty"` when no completed result was waiting.

RETURNS

`str`

POSSIBLE VALUES

`"ok"`, `"partial"`, `"empty"`, `"out_of_range"`, `"no_plant"`, `"not_mature"`, `"no_forage"`, `"not_empty"`, `"base_sector"`, `"already_mature"`, `"tier_conflict"`, `"invalid_seed"`, `"invalid_material"`

`.message: str`

Player-readable explanation of the terminal outcome.

RETURNS

`str`

`.job_id: int | None`

Completed job id, or `None` when empty.

RETURNS

`int | None`

`.action: str | None`

Completed action, or `None` when empty.

RETURNS

`str | None`

POSSIBLE VALUES

`"harvest"`, `"plant"`, `"apply"`

`.sector: str | None`

Completed target sector, or `None` when empty.

RETURNS

`str | None`

`.item_id: str | None`

Requested seed or treatment item, or `None` for harvest and empty results.

RETURNS

`str | None`

`.collected: int`

Whole Forage units committed to the output bin by a harvest. Always **0** otherwise.

RETURNS

`int`

`.discarded: int`

Whole Forage units a harvest could not fit and threw away. Non-zero only with `"partial"`; the cell is cleared either way.

RETURNS

`int`

JobReceipt

Returned by: Crop Automator job submission

Get `JobReceipt` from the APIs listed here. It has no script constructor.

PROPERTIES

`.status: str`

`"queued"` when the job entered the FIFO, otherwise the exact reason it was not accepted.

RETURNS

`str`

POSSIBLE VALUES

`"queued"`, `"queue_full"`, `"not_placed"`, `"out_of_range"`, `"invalid_seed"`, `"invalid_material"`

`.message: str`

Player-readable explanation of the submission outcome.

RETURNS

`str`

`.job_id: int | None`

Stable whole-number job id, or `None` when no job was created.

RETURNS

`int | None`

`.queue_position: int | None`

One-based execution position at submission time, including the active job, or `None` when rejected.

RETURNS

`int | None`

AnalyzeInfo

Returned by: `bio_lab.analyze().info` after `status == "ok"`

Get `AnalyzeInfo` from the APIs listed here. It has no script constructor.

PROPERTIES

`.fragment_id: str`

Discovered fragment id (e.g. `"gw_cranial_plate"`). Use as the inventory key for the resulting sample: Bio Orders ask for fragments by this id.

RETURNS

`str`

POSSIBLE VALUES

`"gw_cranial_plate"`, `"gw_caudal_fin"`, `"gw_cardiac_node"`, `"gw_jaw_fang"`, `"gw_spinal_vertebra"`, `"vc_dorsal_carapace"`, `"vc_mandible_claw"`, `"vc_antenna_cluster"`, `"vc_walking_leg"`, `"vc_eye_stalk"`, `"oc_cranium"`, `"oc_tentacle_arm"`, `"oc_chitin_beak"`, `"oc_ink_sac"`, `"oc_lens_eye"`, `"bw_skull"`, `"bw_foreclaw"`, `"bw_ribcage"`, `"bw_hindlimb"`, `"bw_tail_spike"`, `"vd_bell"`, `"vd_nematocyst"`, `"vd_neural_mesh"`, `"vd_photophore"`, `"vd_tendril"`, `"mh_fruiting_body"`, `"mh_spore_pod"`, `"mh_mycelium_root"`, `"mh_chitin_node"`, `"mh_stigmatic_disc"`, `"hs_mandible"`, `"hs_wing_membrane"`, `"hs_thorax_plate"`, `"hs_abdomen_segment"`,

```
"hs_compound_eye",
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",
"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw",
"st_beak",
"st_carapace_neural",
"vm_cephalic_horn",
```

`"vm_wing_sheet"`,
`"vm_gill_filament"`,
`"vm_tail_barb"`,
`"vm_ventral_photophore"`

`.name: str`

Display name of the analyzed fragment.

RETURNS
`str`

`.rarity: str`

Rarity tier: `"common" | "uncommon" | "rare" | "legendary"`.

RETURNS
`str`

POSSIBLE VALUES
`"common"`, `"uncommon"`,
`"rare"`, `"legendary"`

`.required_recipe: dict[str, int]`

Required reagents to extract a sample: `dict` `{reagent_id: qty}`. Iterate `.items()` and call `lab.load(rid, qty)` for each pair, then `lab.extract()`.

RETURNS
`dict[str, int]`

`.coords: list[float]`

World coordinates `[x, y]` the specimen was collected from.

RETURNS
`list[float]`

`.distance: float`

Straight-line distance in meters from this outpost (set at collect time).

RETURNS

`float`

BioCasterRecipe

Returned by: bio_caster.list_recipes() / bio_caster.find_recipe(fragment_id)

Get `BioCasterRecipe` from the APIs listed here. It has no script constructor.

PROPERTIES

.fragment_id: str

Volcanic fragment id this recipe forges. In the Caster's own script, pass it to `self.set_recipe(recipe.fragment_id)` when that fragment is ready to process.

RETURNS

`str`

POSSIBLE VALUES

`"gw_jaw_fang"`, `"vc_walking_leg"`, `"oc_ink_sac"`, `"bw_hindlimb"`, `"vd_photophore"`, `"mh_chitin_node"`, `"hs_abdomen_segment"`, `"ms_abdomen_sclerite"`, `"hc_beak"`, `"ma_chitinous_seta"`, `"gm_abdominal_sheath"`, `"fs_oral_tegmen"`, `"sd_tail_barb"`, `"ce_great_appendage"`, `"st_beak"`, `"vm_tail_barb"`

.tier: int

Derived production tier of this forge recipe.

RETURNS

`int`

`.materials: dict[str, int]`

Exact fabricated-material shopping list as `{item_id: count}`. A supply script can iterate `.items()` without selecting the recipe or possessing its fragment.

RETURNS

`dict[str, int]`

`.temperature_range: list[float]`

Inclusive target temperature range `[low, high]` in °C for this recipe.

RETURNS

`list[float]`

BioOrder

Returned by: bio_exchange.orders() / bio_exchange.active_order() / get_component("bio_exchange_1").orders()

Get `BioOrder` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Stable Bio Order id (e.g. `"bio_order_03"`). Pass to `bio_exchange.set_order(id)` to make this the active Bio Order.

RETURNS
`str`

`.name: str`

Display name of the Bio Order (e.g. `"Cryophyte Spore Panel"`).

RETURNS
`str`

`.biome: str`

Biome the required fragments belong to (e.g. `"frozen"`). Orders for biomes you have no outpost in can't be filled yet.

RETURNS
`str`

POSSIBLE VALUES
`"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.requires: dict[str, int]`

The fragment shopping list: `dict` `{fragment_id: count}`. Iterate `.items()` to see what to deliver and how many of each.

RETURNS

`dict[str, int]`

`.reward: int`

Credits paid when every requirement is met. Paid once: orders are one-time.

RETURNS

`int`

`.status: str`

Status relative to this Bio Exchange: `"available"` (not currently selected here), `"active"` (selected here), or `"complete"` (completed globally). Global completion takes priority over selection; an order with status `"available"` may already have delivery progress.

RETURNS

`str`

POSSIBLE VALUES

`"available"`, `"active"`, `"complete"`

`.delivered: dict[str, int]`

Per-fragment delivery progress: `dict` `{fragment_id: count}`, shared across every Bio Exchange serving this order. Compare against `.requires` to see what's left to deliver.

RETURNS

`dict[str, int]`

`.in_transit: dict[str, int]`

Samples committed to delivery as `{fragment_id: count}`, shared across every Exchange serving this order. Includes qualifying samples already staged in an Exchange input and timed deliveries underway. A sample stays counted while `deliver()` moves it into delivery, then leaves after completion or when staged for refund in `self.output`. Query the order again for current values. `.percent` counts completed deliveries only.

RETURNS

`dict[str, int]`

`.percent: float`

Overall completion: **0-100**, samples delivered divided by total required across every fragment.

RETURNS

`float`

`.target_glow: list[int] | None`

Coastal infusion orders only: the exact glow `[r,g,b]` (0-255) the required fragment must be infused to via the Bio Luminizer. `None` for non-coastal orders. `deliver()` only matches a Luminous fragment whose glow equals this.

RETURNS

`list[int] | None`

ChamberFragment

Returned by: dna_sequencer.chamber

Get `ChamberFragment` from the APIs listed here. It has no script constructor.

PROPERTIES

`.fragment_id: str`

The geothermal fragment id loaded in the chamber (e.g. `"gw_cardiac_node"`).

RETURNS

`str`

POSSIBLE VALUES

`"gw_cardiac_node"`, `"vc_antenna_cluster"`, `"oc_chitin_beak"`, `"bw_ribcage"`, `"vd_neural_mesh"`, `"mh_mycelium_root"`, `"hs_thorax_plate"`, `"ms_pedipalp"`, `"hc_septum_plate"`, `"ma_ganglion_node"`, `"gm_raptorial_claw"`, `"fs_stalk_columnal"`, `"sd_obsidian_scale"`, `"ce_mouth_disc"`, `"st_limb_claw"`, `"vm_gill_filament"`

`.name: str`

Player-facing name of the geothermal fragment loaded in the chamber.

RETURNS

`str`

`.genes: list[str]`

The genes the chambered fragment currently carries (e.g. `["cold_tolerance", "pressure_tolerance"]`). Same as calling `genes()`.

RETURNS

`list[str]`

`.spliced: bool`

`True` after this fragment has used its one safe splice. Calling `splice()` again destroys it. `False` means the fragment can still be engineered once.

RETURNS

`bool`

ChamberSample

Returned by: bio_luminizer.chamber

Get `ChamberSample` from the APIs listed here. It has no script constructor.

PROPERTIES

`.fragment_id: str`

The coastal fragment id loaded in the chamber (e.g. `"gw_caudal_fin"`).

RETURNS

`str`

POSSIBLE VALUES

`"gw_caudal_fin"`, `"vc_mandible_claw"`, `"oc_tentacle_arm"`, `"bw_foreclaw"`, `"vd_nematocyst"`, `"mh_spore_pod"`, `"hs_wing_membrane"`, `"ms_leg_tarsus"`, `"hc_shell_whorl"`, `"ma_cuticle_molt"`, `"gm_folded_wing"`, `"fs_arm_segment"`, `"sd_wing_membrane"`, `"ce_swimmeret_lobe"`, `"st_plastron_shard"`, `"vm_wing_sheet"`

`.name: str`

Player-facing name of the coastal fragment loaded in the chamber.

RETURNS

`str`

`.glow: list[int]`

The sample's current glow `[r,g,b]` (0-255): the dim start color before tuning. Read it, the order's `target_glow`, and `lamp_signature(...)` to solve the lamp settings.

RETURNS

`list[int]`

FragmentLocation

Returned by: bio_collector.scan()

Get `FragmentLocation` from the APIs listed here. It has no script constructor.

PROPERTIES

`.coords: list[float]`

World coordinates `[x, y]` of this fragment. Pass to `bio_collector.collect(coords)` to retrieve.

RETURNS
`list[float]`

`.distance: float`

Straight-line distance in meters from this outpost. Sorted ascending in the scan result: `result[0]` is the nearest.

RETURNS
`float`

`.cataloged: bool`

`True` if this fragment has already been analyzed and added to the Journal; `False` for still-unknown dots. Gates the optional `.fragment_id`, `.name`, and `.rarity` fields.

RETURNS
`bool`

`.fragment_id: str | None`

Known fragment id once `.cataloged` is `True`; `None` for still-unknown dots. Matches the keys in `BioOrder.requires`.

RETURNS

`str | None`

POSSIBLE VALUES

`"gw_cranial_plate"`,
`"gw_caudal_fin"`,
`"gw_cardiac_node"`,
`"gw_jaw_fang"`,
`"gw_spinal_vertebra"`,
`"vc_dorsal_carapace"`,
`"vc_mandible_claw"`,
`"vc_antenna_cluster"`,
`"vc_walking_leg"`,
`"vc_eye_stalk"`,
`"oc_cranium"`,
`"oc_tentacle_arm"`,
`"oc_chitin_beak"`,
`"oc_ink_sac"`,
`"oc_lens_eye"`,
`"bw_skull"`,
`"bw_foreclaw"`,
`"bw_ribcage"`,
`"bw_hindlimb"`,
`"bw_tail_spike"`,
`"vd_bell"`,
`"vd_nematocyst"`,
`"vd_neural_mesh"`,
`"vd_photophore"`,
`"vd_tendril"`,
`"mh_fruiting_body"`,
`"mh_spore_pod"`,
`"mh_mycelium_root"`,
`"mh_chitin_node"`,
`"mh_stigmatic_disc"`,
`"hs_mandible"`,
`"hs_wing_membrane"`,
`"hs_thorax_plate"`,
`"hs_abdomen_segment"`,
`"hs_compound_eye"`,
`"ms_chelicera"`,

```
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",
"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw",
"st_beak",
"st_carapace_neural",
"vm_cephalic_horn",
"vm_wing_sheet",
"vm_gill_filament",
```

`"vm_tail_barb"`,
`"vm_ventral_photophore"`

`.name: str | None`

Player-facing fragment name once `.cataloged` is `True`; `None` for still-unknown dots. Use for readable logs and UI-facing decisions while retaining `.fragment_id` as the stable automation key.

RETURNS

`str | None`

`.rarity: str | None`

Rarity tier once `.cataloged` is `True`; `None` for still-unknown dots. Scan results never expose recipe or creature identity.

RETURNS

`str | None`

POSSIBLE VALUES

`"common"`, `"uncommon"`, `"rare"`, `"legendary"`

HabitatBonusNode

Returned by: `Habitat.get_bonus_tree().nodes` and `Habitat.get_active_bonuses()`

Get `HabitatBonusNode` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable node id accepted by `unlock_bonus(...)`.

RETURNS
`str`

.source_species: str

Creature id whose tree owns this node. Global breakthroughs are still earned from one source species.

RETURNS
`str`

.slot: str

Tree position: `"adaptation"` or `"breakthrough"`.

RETURNS
`str`

POSSIBLE VALUES
`"adaptation"`,
`"breakthrough"`

`.scope: str`

Effect scope: `"species"` affects only `source_species`; `"global"` affects every current and future Habitat colony.

RETURNS
`str`

POSSIBLE VALUES
`"species"`, `"global"`

`.depth: int`

Tree depth **1-2**.

RETURNS
`int`

`.name: str`

Player-readable authored node name.

RETURNS
`str`

`.description: str`

Exact authored effect summary.

RETURNS
`str`

`.state: str`

Current purchase state.

RETURNS

`str`

POSSIBLE VALUES

`"purchased"`,
`"available"`,
`"unaffordable"`,
`"population_locked"`

`.purchased: bool`

Whether this node is permanently purchased.

RETURNS

`bool`

`.active: bool`

Whether this purchased node currently has at least one effect applying to the Habitat that returned it. Unpurchased nodes are always `False`.

RETURNS

`bool`

`.insight_cost: int`

Whole shared Insight cost.

RETURNS

`int`

`.local_population: int`

Minimum local colony population: **0** for the adaptation or **10,000** for the breakthrough.

RETURNS

`int`

`.unmet: list[str]`

All currently unmet requirements: at most `"population"` and `"insight"`.

RETURNS

`list[str]`

HabitatBonusTree

Returned by: Habitat.get_bonus_tree()

Get `HabitatBonusTree` from the APIs listed here. It has no script constructor.

PROPERTIES

.species: str

Creature id whose authored tree is shown, or an empty string before a target is selected.

RETURNS
`str`

.shared_insight: float

Exact shared Insight currently available for purchases, including retained fractions.

RETURNS
`float`

.purchased_count: int

Number of permanent nodes purchased from this species' tree.

RETURNS
`int`

`.nodes: list[HabitatBonusNode]`

The species-only adaptation and all-species breakthrough in stable order, including locked and purchased nodes.

RETURNS

`list[HabitatBonusNode]`

HabitatInsight

Returned by: Habitat.get_insight()

Get `HabitatInsight` from the APIs listed here. It has no script constructor.

PROPERTIES

.shared: float

Exact shared Insight currently spendable by any Habitat, including retained fractions.

RETURNS
`float`

.shared_exact: float

Alias of `shared`, retained for scripts that already read the explicitly named exact balance.

RETURNS
`float`

.rate_per_hour: float

This colony's projected Insight production per hour at current growth and supply.

RETURNS
`float`

`.lifetime_produced: float`

Insight this colony has produced over its lifetime, including fractions.

RETURNS

`float`

`.producing: bool`

Whether this colony is producing positive Insight at its current growth rate.

RETURNS

`bool`

Specimen

Returned by: bio_collector.cargo / bio_lab.specimen

Get `Specimen` from the APIs listed here. It has no script constructor.

PROPERTIES

`.coords: list[float]`

World coordinates `[x, y]` the specimen was collected from. Stable across save/load.

RETURNS
`list[float]`

`.distance: float`

Straight-line distance in meters from the collecting outpost (set at collect time).

RETURNS
`float`

`.stage: str`

Workflow stage: `"collected"` before analysis, `"analyzed"` after `lab.analyze()` reveals the fragment and recipe.

RETURNS
`str`

POSSIBLE VALUES
`"collected"`, `"analyzed"`

`.fragment_id: str | None`

Fragment id (e.g. `"gw_cranial_plate"`). Bio Collector cargo reveals it once the fragment is cataloged; Bio Lab input reveals it only after that specimen reaches `"analyzed"`, even if the fragment was already cataloged. Otherwise `None`.

RETURNS

`str | None`

POSSIBLE VALUES

`"gw_cranial_plate"`, `"gw_caudal_fin"`, `"gw_cardiac_node"`, `"gw_jaw_fang"`, `"gw_spinal_vertebra"`, `"vc_dorsal_carapace"`, `"vc_mandible_claw"`, `"vc_antenna_cluster"`, `"vc_walking_leg"`, `"vc_eye_stalk"`, `"oc_cranium"`, `"oc_tentacle_arm"`, `"oc_chitin_beak"`, `"oc_ink_sac"`, `"oc_lens_eye"`, `"bw_skull"`, `"bw_foreclaw"`, `"bw_ribcage"`, `"bw_hindlimb"`, `"bw_tail_spike"`, `"vd_bell"`, `"vd_nematocyst"`, `"vd_neural_mesh"`, `"vd_photophore"`, `"vd_tendril"`, `"mh_fruiting_body"`, `"mh_spore_pod"`, `"mh_mycelium_root"`, `"mh_chitin_node"`, `"mh_stigmatic_disc"`, `"hs_mandible"`, `"hs_wing_membrane"`, `"hs_thorax_plate"`, `"hs_abdomen_segment"`, `"hs_compound_eye"`,

```
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",
"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw",
"st_beak",
"st_carapace_neural",
"vm_cephalic_horn",
"vm_wing_sheet",
```

`"vm_gill_filament"`,
`"vm_tail_barb"`,
`"vm_ventral_photophore"`

`.name: str | None`

Player-facing fragment name (e.g. "Beak"). `None` while the fragment identity is still unknown.

RETURNS
`str | None`

`.rarity: str | None`

Rarity tier: `"common" | "uncommon" | "rare" | "legendary"`. `None` until analyzed.

RETURNS
`str | None`

POSSIBLE VALUES
`"common"`, `"uncommon"`,
`"rare"`, `"legendary"`

`.recipe: dict[str, int] | None`

Required reagent recipe: `dict` `{reagent_id: qty}`. `None` until analyzed.

RETURNS
`dict[str, int] | None`

`.production_tier: int | None`

Derived tier of this specimen's Bio Lab extraction recipe, or `None` while its identity and reagent recipe are unknown.

RETURNS
`int | None`

`.glow: list[int] | None`

Bioluminescent glow `[r,g,b]` (0-255) for **coastal** specimens: the dim start color the Bio Luminizer tunes to an order's target. `None` for every other biome.

RETURNS

`list[int] | None`

`.genes: list[str]`

The genes this **geothermal** specimen carries (e.g. `["heat_resistance", "acid_resistance"]`). The DNA strand itself is internal: you work at the gene level. Empty for non-geothermal specimens.

RETURNS

`list[str]`

TerraformPillar

Returned by: terraforming.pillars() / terraforming.get_pillar()

Get `TerraformPillar` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable pillar id: `"temperature"`, `"oxygen"`, `"pressure"`, `"biomass"`, `"plants"`, or `"wildlife"`.

RETURNS

`str`

POSSIBLE VALUES

`"temperature"`, `"oxygen"`, `"pressure"`, `"biomass"`, `"plants"`, `"wildlife"`

.value: float

Current pillar amount in `unit` at the time of the query. Temperature uses heat rather than degrees Celsius, so it is directly comparable with `target` and `next_target`.

RETURNS

`float`

.target: float

Final completion threshold in the same `unit` as `value`. Comes from the pillar's last milestone.

RETURNS

`float`

`.unit: str`

Unit shared by `value`, `target`, and `next_target`: `"heat"`, `"ppt"`, `"kPa"`, `"t"`, `"km²"`, or `"individuals"`.

RETURNS
`str`

POSSIBLE VALUES
`"heat"`, `"ppt"`, `"kPa"`, `"t"`, `"km²"`, `"individuals"`

`.progress: float`

Pillar completion percentage on a **0-100** scale. Uses the same five advancement bands as the Terraform Index breakdown; it is not simply `value / target * 100`.

RETURNS
`float`

`.complete: bool`

`True` when the pillar has reached its final target, using the game's completion tolerance. This is the authoritative check, including when a displayed percentage rounds to 100.

RETURNS
`bool`

`.phase: int`

Current milestone number, **1-6**. The starting milestone is 1; the final completed milestone is 6.

RETURNS
`int`

`.next_target: float | None`

The next milestone threshold in `unit`, or `None` when the pillar is complete. All fields are snapshots; query the component again for fresh values.

RETURNS

`float | None`

WasteDumpResult

Returned by: oxygen_generator.dump_waste()

Get `WasteDumpResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Always `"ok"` after the waste chamber is dumped.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`

.message: str

Player-readable explanation of the dump and its resulting efficiency penalty.

RETURNS
`str`

.penalty: float

Efficiency penalty applied by this dump, in the **0-1** range.

RETURNS
`float`

BatterySwapResult

Returned by: Pioneer.swap_batteries()

Get `BatterySwapResult` from the APIs listed here. It has no script constructor.

PROPERTIES

.status: str

Stable outcome code for script control flow.

RETURNS
`str`

POSSIBLE VALUES
`"ok"`, `"no_op"`, `"not_found"`, `"under_construction"`, `"busy"`, `"not_at_service_point"`, `"invalid_slots"`, `"no_compatible_battery"`, `"insufficient_charged_batteries"`

.message: str

Localized explanation of this outcome.

RETURNS
`str`

`.swapped_count: int`

Number of exchanged batteries; zero on rejection or no operation.

RETURNS

`int`

SignalReceiver

Returned by: weather_station.signal_receiver

Get `SignalReceiver` from the APIs listed here. It has no script constructor.

METHODS

`.transmissions() → list[SignalTransmission]`

Raw transmissions audible to this powered station right now. More than one event may be present. Ordering is stable, but copies sharing a packet number are not ordered by validity. The receiver stores no history.

RETURNS

`list[SignalTransmission]`

SignalTransmission

Returned by: SignalReceiver.transmissions()

Get `SignalTransmission` from the APIs listed here. It has no script constructor.

PROPERTIES

.event_id: str

Event id shared by every packet and noise duplicate.

RETURNS
`str`

.number: int

Declared packet number. Different copies may declare the same number.

RETURNS
`int`

.total: int

Declared packet count for the event.

RETURNS
`int`

`.channel: str`

Channel on which this copy is heard.

RETURNS

`str`

POSSIBLE VALUES

`"broadcast"`, `"frozen"`, `"coastal"`, `"geothermal"`, `"volcanic"`, `"deep"`

`.data: str`

Encoded movement record `event_id|number|total|dx|dy`. Numbered records form a movement sequence beginning at `(0, 0)`; `dx` and `dy` are that record's signed movement offsets, and the sequence endpoint is the event coordinate.

RETURNS

`str`

`.checksum: int`

Checksum supplied with the transmission: the full sum of every character's ASCII code in `.data`, with no modulo reduction.

RETURNS

`int`

`.emitted_at_gh: float`

Storm-start world-clock timestamp shared by every valid and noisy copy in this event.

RETURNS

`float`

`.expires_at_gh: float`

World-clock timestamp when this event's live listening window closes.

RETURNS

`float`

`.source_station_id: str`

Physical Weather Station hearing this copy.

RETURNS

`str`

Storm

Returned by: WeatherReport.active()

Get `Storm` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable storm id.

RETURNS
`str`

.x: float

Cell-center x at this report's observation time, m.

RETURNS
`float`

.y: float

Cell-center y at this report's observation time, m.

RETURNS
`float`

METHODS

`.kind() → str`

`"dust"` or `"thunder"`. Dust events transmit a message encoding where their Raw Uranium aftermath lies. Thunder charges eligible Lightning Rods and may transmit a message encoding where its Storm Glass aftermath lies.

RETURNS
`str`

POSSIBLE VALUES
`"dust"`, `"thunder"`

`.radius_m() → float`

Cell radius in m. A heli drone on a straight route holds once its current position is inside the cell; it does not automatically route around it. Electric drones fly through.

RETURNS
`float`

`.speed() → float`

Travel speed in m/h.

RETURNS
`float`

`.heading() → list[float]`

Unit travel direction as `[dx, dy]`.

RETURNS
`list[float]`

`.intensity() → float`

Observed strength **0-1** at report time.

RETURNS

`float`

`.expires_in() → float`

Live hours remaining before this observed cell dissipates.

RETURNS

`float`

`.eta_to(x: float, y: float) → float | None`

Hours until the cell's edge reaches the point. **0** when the point is already inside the cell; `None` when the track never gets there before dissipating.

PARAMETERS

Name	Type	Description
`x`	`float`	World x, m.
`y`	`float`	World y, m.

RETURNS

`float | None`

WeatherEventForecast

Returned by: WeatherReport.forecast()

Get `WeatherEventForecast` from the APIs listed here. It has no script constructor.

PROPERTIES

`.id: str`

Stable event id shared with the active `Storm` and later signal transmissions.

RETURNS

`str`

METHODS

`.kind() → str`

Broad event family: `"dust"` or `"thunder"`.

RETURNS

`str`

POSSIBLE VALUES

`"dust"`, `"thunder"`

`.arrival_window() → list[float]`

`[earliest, latest]` hours after this report's observation time when the cell should enter coverage.

RETURNS

`list[float]`

`.corridor() → Zone`

Coarse predicted travel corridor as a `Zone`; it is storm information, never the hidden aftermath path.

RETURNS

`Zone`

`.intensity_range() → list[float]`

Observed forecast range `[low, high]`, each **0-1**.

RETURNS

`list[float]`

WeatherReport

Returned by: weather_station.observe() / weather_station.last_report()

Get `WeatherReport` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable report id for provenance and logging.

RETURNS
`str`

.source_id: str

Physical Weather Station id that made this report.

RETURNS
`str`

.observed_at_gh: float

World-clock timestamp when the report was measured.

RETURNS
`float`

METHODS

`.age_gh() → float`

Live age of this immutable report in world-clock hours.

RETURNS

`float`

`.coverage() → Zone`

The station's local coverage as a `Zone` frozen at observation time.

RETURNS

`Zone`

`.active() → list[Storm]`

List of active `Storm` snapshots inside local coverage at observation time.

RETURNS

`list[Storm]`

`.forecast() → list[WeatherEventForecast]`

Local `WeatherEventForecast` entries expected to enter coverage within **8 world-clock hours**, or **24** after Weather Forecasting research.

RETURNS

`list[WeatherEventForecast]`

WeatherSignalBoard

Returned by: weather_station.signal_board

Get `WeatherSignalBoard` from the APIs listed here. It has no script constructor.

METHODS

`.reveal(transmission: SignalTransmission | TransmissionRecord) → ActionResult`

Publish one transmission into its declared numbered slot. A different event replaces this station's current board. An already-published slot for the same event is left unchanged. The board checks shape and bounds, not meaning or checksum validity.

PARAMETERS

Name	Type	Description
`transmission`	`SignalTransmission` | `TransmissionRecord`	A `SignalTransmission`, a saved dictionary such as `vars(signal)`, or a class instance with the same fields. Requires string `event_id`, `channel`, and `data`, plus whole-number `number` and `total` within the signal slot limits. Optional string `source_station_id` defaults to this station when omitted.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_power"`	transient	The component has no available power.
`"duplicate"`	rejection	That entry is already present and nothing was added or changed.
`"invalid_type"`	rejection	A supplied value has the wrong type or structure, or is missing required content.

`.reject(transmission: SignalTransmission | TransmissionRecord) → ActionResult`

Add one to the supplied event's refusal count without opening a slot. A different event replaces this station's current board.

PARAMETERS

Name	Type	Description
`transmission`	`SignalTransmission` | `TransmissionRecord`	A `SignalTransmission`, a saved dictionary such as `vars(signal)`, or a class instance with the same fields and validation as `reveal()`.

RETURNS

`ActionResult`

RESULT FIELDS

`.status`, `.message`

SUCCESS PAYLOAD

None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_power"`	transient	The component has no available power.
`"invalid_type"`	rejection	A supplied value has the wrong type or structure, or is missing required content.

`.resolve(event_id: str, info: dict[str, object]) → ActionResult`

Publish up to **6** labelled rows for an event. A different event replaces this station's current board. The board displays supplied values without validating them.

PARAMETERS

Name	Type	Description
`event_id`	`str`	Event id the rows describe.
`info`	`dict[str, object]`	String-keyed dictionary of labels and display values.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_power"`	transient	The component has no available power.
`"invalid_type"`	rejection	A supplied value has the wrong type or structure, or is missing required content.

`.clear() → ActionResult`

Clear the board, its rejection count, and any published conclusion.

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
`ActionResult`	`.status`, `.message`	None

OUTCOMES

Status	Kind	Meaning
`"ok"`	success	The operation completed successfully.
`"no_power"`	transient	The component has no available power.

`.status() → WeatherSignalBoardStatus`

This station's current board publication metadata.

RETURNS

`WeatherSignalBoardStatus`

WeatherSignalBoardStatus

Returned by: weather_station.signal_board.status()

Get `WeatherSignalBoardStatus` from the APIs listed here. It has no script constructor.

PROPERTIES

`.has_input: bool`

Whether this station has published a Signal Board value.

RETURNS
`bool`

`.published_at_gh: float | None`

Publication timestamp, or `None` with no input.

RETURNS
`float | None`

`.freshness: str`

`"no_input"`, `"fresh"`, or `"stale"`. A publication becomes stale after **2 world-clock hours**.

RETURNS
`str`

POSSIBLE VALUES
`"no_input"`, `"fresh"`, `"stale"`

`.event_id: str`

Event shown on this station's board, or an empty string with no input.

RETURNS

`str`

WeatherStrike

Returned by: weather_station.strikes()

Get `WeatherStrike` from the APIs listed here. It has no script constructor.

PROPERTIES

.id: str

Stable observed-strike id.

RETURNS
`str`

.event_id: str

Thunderstorm event id.

RETURNS
`str`

.observed_at_gh: float

World-clock hour when the strike landed.

RETURNS
`float`

`.energy_wh: float`

Electrical energy carried by the strike in Wh.

RETURNS

`float`

`.caught: bool`

Whether an eligible Lightning Rod banked this strike.

RETURNS

`bool`

Zone

Returned by: `WeatherReport.coverage()` and `WeatherEventForecast.corridor()`

Get `Zone` from the APIs listed here. It has no script constructor.

METHODS

`.intersect(other: Zone) → Zone`

Keep only the geometry shared by both zones.

PARAMETERS

Name	Type	Description
`other`	`Zone`	Another `Zone`.

RETURNS

`Zone`

`.union(other: Zone) → Zone`

Keep everything covered by either zone.

PARAMETERS

Name	Type	Description
`other`	`Zone`	Another `Zone`.

RETURNS

`Zone`

`.subtract(other: Zone) → Zone`

Remove the other zone's geometry from this zone.

PARAMETERS

Name	Type	Description
`other`	`Zone`	Another `Zone`.

RETURNS

`Zone`

`.diff(other: Zone) → Zone`

Keep geometry present in only one of the two zones.

PARAMETERS

Name	Type	Description
`other`	`Zone`	Another `Zone`.

RETURNS

`Zone`

`.center() → list[float] | None`

Centroid as `[x, y]` in m, or `None` for an empty zone. This describes visible storm geometry, not its hidden aftermath.

RETURNS

`list[float] | None`

`.contains(x: float, y: float) → bool`

`True` when the point is inside the zone.

PARAMETERS

Name	Type	Description
`x`	`float`	World x, m.
`y`	`float`	World y, m.

RETURNS

`bool`

`.is_empty() → bool`

`True` when the zone covers nothing.

RETURNS

`bool`